ICPC TEAM REFERENCE

Baskotian Template

Husam Zaid · Ahmed Alaa · Yousef Osama

388 templates · 12 sections · 24,247 lines · August 14, 2026

HOW TO FIND SOMETHING (in order of speed). 1. You know the routine's name → the alphabetical *Index of every routine* at the very back gives the page. 2. You know the topic → the running head on every page names the section (left) and the sub-topic (right), and the strip along the bottom edge shows which of the 12 sections you are in; thumb until it matches. 3. You know neither → section **12 Problem Solving** maps a problem's *shape* and its *constraints* onto the technique, and `docs/PLAYBOOK.md` does the same at more length.

BEFORE YOU PASTE. Line 1 of every file says what it does; a `// Needs:` line names the templates it depends on; a `// NOTE:` line warns when another file declares the same symbol. Vertices and array positions are **0-based** unless the header says otherwise. Geometry: `P<ll>` for every decision, `P<ld>` only when the answer is a real number.

Contents

1 | Miscellaneous

Template 2 · fastIO 2 · grid 3 · random 3 · coordinateCompression 3 · grayCode 4 · customHashHashTable 4 · pragmas 4 · int128 4 · SearchIdioms 4 · BitTricks 5 · StressTest 6 · Fraction 6 · Inversions 7 · ExpressionParser 7 · CycleFinding 8 · ParallelBinarySearch 8 · MeetInTheMiddle 9 · Scheduling 9 · Josephus 10 · BigInt 10 · RankingAndDates 11 · MatroidIntersection 12 · SchreierSims 14 · RangeBitset 16 · TrygubNum 17 · PermutationRoots 18 · SubarrayAggregator 19 · MexOfAllSubarrays 20 · XorEquationRange 21 · SubsetUnionOfBitsets 21 · NegativeBase 22 · TestGenerators 23

1.1 Template

```
#include <bits/stdc++.h>
using namespace std;

//#define int long long
#define ll long long
#define lll __int128                                //
  ↪ used unnamed by CHT, BigInt, PrimeCounting
#define ld long double
#define fi first
#define se second
#define pb push_back
#define eb emplace_back
#define all(x) x.begin(), x.end()
#define rall(x) x.rbegin(), x.rend()
#define sz(x) (int)(x).size()
#define ones(n) __builtin_popcountll(n)
#define msb(n) (63 - __builtin_clzll(n))
#define lsb(n) __builtin_ctzll(n)
typedef vector<int> vi;
typedef pair<int, int> pii;

const int N = 2e5 + 5, M = 1e3 + 5, LOG = 31;
const int inf = 0x3f3f3f3f;
const ll llinf = 0x3f3f3f3f3f3f3f3f;
const int MOD = 1e9 + 7;
const ld eps = 1e-9, PI = acosl(-1.0L);         // ld,
  ↪ not double: geometry needs the digits

#ifdef LOCAL                                     //
  ↪ compile with -DLOCAL
template <class T> void _p(const T& x);
void _p(const string& s) { cerr << '"' << s << '"'; }
void _p(char c) { cerr << '\'' << c << '\''; }
template <class A, class B> void _p(const pair<A, B>&
  ↪ p) {
    cerr << '('; _p(p.fi); cerr << ", "; _p(p.se);
  ↪ cerr << ')';
}
template <class T> void _p(const T& x) {
    if constexpr (is_arithmetic_v<T>) cerr << x;
    else { cerr << '{'; int f = 1;
           for (auto& e : x) { if (!f) cerr << ", ";
  ↪ f = 0; _p(e); } cerr << '}'; }
}
void _pr() { cerr << "\n"; }
template <class H, class... T> void _pr(const H& h,
  ↪ const T&... t) {
    _p(h); if (sizeof...(t)) cerr << " | "; _pr(t...);
}
#define dbg(...) (cerr << "[" << #__VA_ARGS__ << "] =
  ↪ ", _pr(__VA_ARGS__))
#else
#define dbg(...)
#endif

void testCase() {
}

void preCompute() {
}

int32_t main() {
    ios_base::sync_with_stdio(false),
  ↪ cin.tie(nullptr), cout.tie(nullptr);
    cout << fixed << setprecision(10);
    preCompute();

    int tc = 1;
    cin >> tc;                                          //
  ↪ DELETE THIS LINE for single-test problems
    for (int TC = 1; TC <= tc; TC++) {
        // cout << "Case #" << TC << ": ";
        testCase();
    }
}

/*



*/
```

1.2 fastIO

HAND-ROLLED FAST IO (fread/fwrite buffered). Use only when cin with sync off is measurably too

```
// slow: >1e6 numbers to read, or heavy output.
  ↪ Otherwise prefer the template's
// ios::sync_with_stdio.
const int MOD = 1e9 + 7;
const int BUF_SZ = 1 << 15;

inline namespace Input {
    char buf[BUF_SZ];
    int pos;
    int len;
    char next_char() {
        if (pos == len) {
            pos = 0;
            len = (int)fread(buf, 1, BUF_SZ, stdin);
            if (!len) { return EOF; }
        }
        return buf[pos++];
    }

    int read_int() {
        int x;
```

```
        char ch;
        int sgn = 1;
        while (!isdigit(ch = next_char())) {
            if (ch == '-') { sgn *= -1; }
        }
        x = ch - '0';
        while (isdigit(ch = next_char())) { x = x *
  ↪ 10 + (ch - '0'); }
        return x * sgn;
    }
} // namespace Input
inline namespace Output {
    char buf[BUF_SZ];
    int pos;

    void flush_out() {
        fwrite(buf, 1, pos, stdout);
        pos = 0;
    }

    void write_char(char c) {
        if (pos == BUF_SZ) { flush_out(); }
        buf[pos++] = c;
    }

    void write_int(int x) {
        static char num_buf[100];
        if (x < 0) {
            write_char('-');
            x *= -1;
        }
        int len = 0;
        for (; x >= 10; x /= 10) { num_buf[len++] =
  ↪ (char)('0' + (x % 10)); }
        write_char((char)('0' + x));
        while (len) { write_char(num_buf[--len]); }
        write_char('\n');
    }

    // auto-flush output when program exits
    void init_output() { assert(atexit(flush_out) ==
  ↪ 0); }
} // namespace Output
```

1.3 grid

GRID DIRECTION VECTORS. First 4 entries = 4-neighbourhood (L,R,U,D), all 8 = king moves.

```
// Knight moves: dx {1,1,-1,-1,2,2,-2,-2}, dy
  ↪ {2,-2,2,-2,1,-1,1,-1}. Guard with 0<=x<n &&
  ↪ 0<=y<m.
int dx[] = {0, 0, -1, 1, -1, -1, 1, 1};
int dy[] = {-1, 1, 0, 0, -1, 1, -1, 1};
char di[] = {'L', 'R', 'U', 'D'};

// KNIGHT DISTANCE ON AN INFINITE BOARD, O(1). BFS is
  ↪ fine up to a 1000x1000 board; past that, or
// when the coordinates are 1e9, you need this closed
  ↪ form. Reduce by symmetry to x, y >= 0, then
// the answer is driven by three lower bounds - you
  ↪ cover at most 2 in either axis per move and at
// most 3 of (x + y) per move - and parity, since
  ↪ every move changes x + y by an odd amount. Three
// positions beat the formula and must be
  ↪ special-cased; they are the classic reason a
  ↪ hand-rolled
// version WAs. Exact integer arithmetic, any
  ↪ magnitude that fits in ll.
ll knightDist(ll x, ll y) {
    x = abs(x), y = abs(y);
    if (x < y) swap(x, y);
  ↪   // now x >= y >= 0
    if (x == 1 && y == 0) return 3;
  ↪   // the three exceptions, all near the
    if (x == 2 && y == 2) return 4;
  ↪   // origin, where the board "runs out"
    ll d = max({(x + 1) / 2, (y + 1) / 2, (x + y + 2)
  ↪ / 3});
    while ((d & 1) != ((x + y) & 1)) d++;
  ↪   // parity: x + y flips every move
    return d;
}
// ON A BOUNDED BOARD the formula is wrong near the
  ↪ edges and corners - BFS, or use the formula
// and repair the O(1) corner cases by hand. On a
  ↪ HALF-PLANE (x, y >= 0) the same three exceptions
// are exactly the corrections needed, which is what
  ↪ the code above assumes.
// KING distance is max(|dx|, |dy|); ROOK is 0/1/2;
  ↪ BISHOP is 0/1/2 when the colours match and
// unreachable otherwise. Those need no code, only
  ↪ the reminder that they are not Manhattan.
```

1.4 random

RANDOMNESS. Seed from the clock so tests cannot be prepared against you.

```
mt19937_64
  ↪ rng(chrono::steady_clock::now().time_since_epoch().c
ll rnd(ll l, ll r) { return
  ↪ uniform_int_distribution<ll>(l, r)(rng); }   //
  ↪ inclusive

vector<int> randPerm(int n) {
  ↪   // random permutation of 0..n-1
    vector<int> v(n);
    iota(all(v), 0);
    shuffle(all(v), rng);
    return v;
}
// Shuffling the INPUT is a real algorithm, not just
  ↪ a utility:
//   - minimum enclosing circle is O(n) only after a
  ↪ shuffle
//   - quickselect / random pivots
//   - defeats anti-hash and anti-quicksort tests
//   - random rotation of points removes degenerate
  ↪ ties in a geometry sweep
```

1.5 coordinateCompression

Coordinate Compression - O(N log N)

```
template<typename T>
struct Compress {
    vector<T> v;

    Compress(vector<T>& a) : v(a) {
        sort(all(v));
        v.erase(unique(all(v)), v.end());
        for (auto &x : a) {
            x = lower_bound(all(v), x) - v.begin();
        }
    }

    T operator[](int i) const {
        return v[i];
```

```
    }
};
```

1.6 grayCode

GRAY CODE - orders 0..2^n-1 so consecutive values differ in exactly ONE bit. Use it to walk all

```
// subsets while changing one element at a time
   ↪ (incremental DP, XOR-basis sweeps, Tower of
   ↪ Hanoi).
// Gray Code Generator - O(2^N)
// g(n) = n ^ (n >> 1)
vector<int> grayCode(int n) {
    vector<int> res(1 << n);
    for (int i = 0; i < (1 << n); i++) {
        res[i] = i ^ (i >> 1);
    }
    return res;
}

int rev_g (int g) {
  int n = 0;
  for (; g; g >>= 1)
    n ^= g;
  return n;
}
```

1.7 customHashHashTable

UNORDERED_MAP THAT CANNOT BE HACKED. The default std::hash for integers is the identity, so an

```
// adversary can force every key into one bucket and
   ↪ turn O(1) into O(n). splitmix64 + a clock seed
// fixes that. gp_hash_table is ~3x faster than
   ↪ unordered_map; reserve() and max_load_factor
// help more.
#include <ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;

// Custom Hash to prevent anti-hash tests in
   ↪ unordered_map / gp_hash_table
struct custom_hash {
    static uint64_t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);
    }

    size_t operator()(uint64_t x) const {
        static const uint64_t FIXED_RANDOM =
   ↪ chrono::steady_clock::now().time_since_epoch().count();
        return splitmix64(x + FIXED_RANDOM);
    }
};

// Usage: gp_hash_table<ll, int, custom_hash> table;
//        unordered_map<ll, int, custom_hash>
   ↪ safe_map;
```

1.8 pragmas

COMPILER PRAGMAS. O3 + unrolling, and AVX2 for auto-vectorised loops (bitset, simple array math).

```
// Put them at the very top, before any include.
   ↪ Risk: ‘target‘ crashes judges without those CPU
// features - never use avx2 on an unknown judge, and
   ↪ never on Codeforces problems with old
// machines.
// GCC Optimizations
#pragma GCC optimize("O3,unroll-loops")
#pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt")
```

1.9 int128

__int128 Stream Operators Helper

```
typedef __int128_t int128;
typedef __uint128_t uint128;

ostream &operator<<(ostream &os, int128 n) {
    if (n == 0) return os << "0";
    if (n < 0) { os << "-"; n = -n; }
    string s;
    while (n > 0) { s += (char)('0' + (n % 10)); n /=
   ↪ 10; }
    reverse(all(s));
    return os << s;
}

istream &operator>>(istream &is, int128 &n) {
    string s; is >> s;
    n = 0; int i = 0, sign = 1;
    if (!s.empty() && s[0] == '-') { sign = -1; i =
   ↪ 1; }
    for (; i < (int)s.size(); i++) n = n * 10 + (s[i]
   ↪ - '0');
    n *= sign;
    return is;
}
```

1.10 SearchIdioms

The four searches. Getting the boundary right here saves more contest time than any template.

```
// BINARY SEARCH ON A PREDICATE. pred must be
   ↪ monotone: F F F T T T.
// firstTrue returns the first x in [lo, hi] with
   ↪ pred(x), or hi+1 if none.
template <class F> ll firstTrue(ll lo, ll hi, F pred)
   ↪ {
    hi++;
    while (lo < hi) { ll m = lo + (hi - lo) / 2;
   ↪ pred(m) ? hi = m : lo = m + 1; }
    return lo;
}
// lastTrue returns the last x in [lo, hi] with
   ↪ pred(x), or lo-1 if none.  T T T F F F
template <class F> ll lastTrue(ll lo, ll hi, F pred) {
    lo--;
    while (lo < hi) { ll m = lo + (hi - lo + 1) / 2;
   ↪ pred(m) ? lo = m : hi = m - 1; }
    return lo;
}
// Reals: fixed iteration count beats an epsilon loop
   ↪ (no infinite loops, predictable time).
// 100 iterations halve the range 2^100 times -
   ↪ always enough.
// Returns the boundary: pred is false on [lo, x) and
   ↪ true on [x, hi]. (‘lo‘ and ‘hi‘ converge.)
template <class F> ld binReal(ld lo, ld hi, F pred,
   ↪ int it = 100) {
    while (it--) { ld m = (lo + hi) / 2; pred(m) ? hi
   ↪ = m : lo = m; }
    return hi;
   ↪          // first TRUE, matching firstTrue
}
```

```
// TERNARY SEARCH on a UNIMODAL function (strictly decreasing then increasing => minimum).
// Integer version: stop when the window is tiny, then scan. Never use `l < r` alone.
template <class F> ll ternaryInt(ll lo, ll hi, F f) {          // returns argmin
    while (hi - lo > 2) {
        ll m1 = lo + (hi - lo) / 3, m2 = hi - (hi - lo) / 3;
        f(m1) <= f(m2) ? hi = m2 : lo = m1;          // <= keeps the LEFTmost min
    }
    ll best = lo;
    for (ll x = lo; x <= hi; x++) if (f(x) < f(best)) best = x;
    return best;
}
template <class F> ld ternaryReal(ld lo, ld hi, F f, int it = 200) {
    while (it--) { ld m1 = lo + (hi - lo) / 3, m2 = hi - (hi - lo) / 3;
                   f(m1) < f(m2) ? hi = m2 : lo = m1; }
    return (lo + hi) / 2;
}
// If f is only WEAKLY unimodal (has flat regions), ternary search is WRONG: one comparison
// f(m1) == f(m2) inside a plateau discards the side holding the optimum. For any DISCRETELY
// CONVEX f (non-decreasing difference sequence, plateaus allowed) use the derivative instead -
// this is strictly safer AND uses half the evaluations:
template <class F> ll convexArgmin(ll lo, ll hi, F f) {
    return firstTrue(lo, hi - 1, [&](ll x) { return f(x + 1) >= f(x); });
}

// NEWTON'S METHOD for roots of a smooth f (quadratic convergence; needs a decent start).
template <class F, class DF> ld newton(ld x, F f, DF df, int it = 60) {
    while (it--) { ld d = f(x) / df(x); x -= d;
                   if (fabsl(d) < 1e-15L * max((ld)1, fabsl(x))) break; }
    return x;
}
// Integer sqrt / cbrt with correction - NEVER trust sqrt() alone near 1e18.
ll isqrt(ll n) {
    ll r = (ll)sqrtl((ld)n);
    while (r > 0 && r * r > n) r--;
    while ((r + 1) * (r + 1) <= n) r++;
    return r;
}
ll icbrt(ll n) {
    ll r = (ll)cbrtl((ld)n);
    while (r > 0 && r * r * r > n) r--;
    while ((r + 1) * (r + 1) * (r + 1) <= n) r++;
    return r;
}

// BINARY SEARCH ON THE ANSWER - the pattern, not the code:
//   "minimise the maximum X" / "maximise the minimum X" / "can we finish within T"
//   -> guess the answer, write a GREEDY feasibility check, binary search on it.
// PARALLEL BINARY SEARCH - when you need this for MANY queries at once: bucket the queries by
// their current midpoint, sweep the shared structure once per level, O(log) sweeps total.
```

1.11 BitTricks

BIT MANIPULATION CHEAT SHEET

```
/* ==================== BIT MANIPULATION CHEAT SHEET ====================
 BUILTINS (add ll suffix for 64-bit: __builtin_popcountll, ...)
  __builtin_popcount(x)  #set bits
  __builtin_ctz(x)  #trailing zeros (x != 0)
  __builtin_clz(x)       #leading zeros
  __builtin_parity(x) popcount & 1
  __lg(x) = 63 - clzll(x) = floor(log2 x)     (GCC only; write it yourself elsewhere)
  lowest set bit: x & -x        clear it: x &= x - 1        set/flip: x |= b, x ^= b
  is power of two: x && !(x & (x - 1))
  round UP to a power of two (x >= 1): x == 1 ? 1ULL : 2ULL << (63 - __builtin_clzll(x - 1))
    - clzll(0) is UNDEFINED, so x = 1 must be special-cased, and the shift must be 64-bit

 SUBMASK ENUMERATION - iterate every subset of m:
  for (int s = m; ; s = (s - 1) & m) { ...; if (!s) break; }
  Over ALL masks this is O(3^n) total (each element is in/out/neither).

 SUPERMASK enumeration (every s with m subset of s, within n bits):
  for (int s = m; s < (1 << n); s = (s + 1) | m) { ... }

 K-SUBSETS in increasing order (Gosper's hack):
  for (int s = (1 << k) - 1; s < (1 << n); ) { ...;
      int c = s & -s, r = s + c; s = (((r ^ s) >> 2) / c) | r; }

 GRAY CODE  g(i) = i ^ (i >> 1)     inverse: x ^= x >> 1 repeatedly (see 06 - grayCode.cpp)
  consecutive gray codes differ in exactly one bit - use for "change one thing at a time".

 XOR FACTS
  x ^ x = 0, x ^ 0 = x, xor is its own inverse -> prefix xor answers range xor
  xor of 0..n = [n, 1, n+1, 0][n % 4]
  a + b = (a ^ b) + 2*(a & b)     a | b = (a ^ b) + (a & b)     a + b = (a | b) + (a & b)
  "OR of a range equals its SUM"  <=>  the values are pairwise BIT-DISJOINT
    (so at most 30 non-zero values fit in any such window -> two pointers)

 COUNTING PER BIT - the standard decomposition:
  handle each of the 30/60 bits independently.
  sum over pairs of (a_i ^ a_j) = sum over bits b of (#with bit) * (#without bit) * 2^b

 BITSET
  std::bitset<N> b; b._Find_first(), b._Find_next(i)  (GCC) - iterate set bits fast
```

```
   b <<= k, b |= other  are O(N/64) - this is what
  ↪ makes bitset DP and reachability work.
================================================================
  ↪ */

// Iterate all submasks of m (including m and 0).
template <class F> void forEachSubmask(int m, F f) {
    for (int s = m;; s = (s - 1) & m) { f(s); if (!s)
  ↪ break; }
}
// All n-bit masks with exactly k bits set, in
  ↪ increasing order (Gosper's hack).
vector<int> kSubsets(int n, int k) {
    vector<int> r;
    if (k == 0) return {0};
    for (int s = (1 << k) - 1; s < (1 << n);) {
        r.push_back(s);
        int c = s & -s, x = s + c;
        s = (((x ^ s) >> 2) / c) | x;
    }
    return r;
}
// Sum over all pairs (i<j) of (a_i XOR a_j), by
  ↪ handling each bit independently.
ll pairXorSum(const vector<ll>& a, int B = 62) {
    ll n = a.size(), r = 0;
    for (int b = 0; b < B; b++) {
        ll c = 0;
        for (ll x : a) c += (x >> b) & 1;
        r += c * (n - c) % MOD * ((1LL << b) % MOD) %
  ↪ MOD;
    }
    return r % MOD;
}
```

1.12 StressTest

STRESS TESTING

```
/* ================= STRESS TESTING =================
 The single highest-value 20 lines in this notebook.
  ↪ Use it the moment a solution WAs on a
 test you cannot see. Write gen.cpp + brute.cpp, then
  ↪ loop until they disagree.

 ---- run.sh
  ↪ ----------------------------------------------------------
 g++ -O2 -o sol sol.cpp && g++ -O2 -o brute brute.cpp
  ↪ && g++ -O2 -o gen gen.cpp
 for i in $(seq 1 100000); do
     ./gen $i > in.txt
     ./sol   < in.txt > out1.txt
     ./brute < in.txt > out2.txt
     if ! diff -qb out1.txt out2.txt > /dev/null;
  ↪ then echo "WA on test $i"; cat in.txt; break; fi
 done

  ↪ ----------------------------------------------------------
 Windows cmd: replace the loop with `for /L %%i in
  ↪ (1,1,100000) do ( ... fc out1.txt out2.txt )`

 RULES THAT MAKE IT ACTUALLY WORK
  1. Generate SMALL cases (n <= 8, values <= 5). Bugs
  ↪ hide in tiny corner cases, not big ones.
  2. Seed the generator from argv[1] so every
  ↪ iteration differs and is reproducible.
  3. If the answer is not unique, do not diff - write
  ↪ a CHECKER that validates sol's output.
  4. If there is no brute force, stress against
  ↪ yourself: compare two different solutions, or
     compare an O(n^2) version of your own idea
  ↪ against the O(n log n) one.
  5. For "find any valid X" problems, verify the
  ↪ constraints instead of comparing.
 ================================================== */

// ---- gen.cpp ----
// #include <bits/stdc++.h>
// using namespace std;
// int main(int argc, char** argv) {
//     mt19937 rng(atoi(argv[1]));
//     auto rnd = [&](int l, int r) { return
  ↪ uniform_int_distribution<int>(l, r)(rng); };
//     int n = rnd(1, 8);
//     printf("%d\n", n);
//     for (int i = 0; i < n; i++) printf("%d ",
  ↪ rnd(1, 5));
//     puts("");
// }

// ---- in-process version: no files, no scripts. Put
  ↪ brute + fast in one binary. ----
// Fastest way to start; switch to the shell loop
  ↪ only when you need the failing input on disk.
template <class Gen, class Fast, class Brute>
void stress(Gen gen, Fast fast, Brute brute, int
  ↪ iters = 100000) {
    for (int it = 1; it <= iters; it++) {
        auto in = gen(it);
        auto a = fast(in), b = brute(in);
        if (!(a == b)) {
            printf("MISMATCH on iteration %d: fast=",
  ↪ it);
            cout << a << " brute=" << b << "\n";
            return;
  ↪ // print `in` here too
        }
    }
    puts("all tests passed");
}
```

1.13 Fraction

Exact rational. Always normalised (gcd reduced, denominator > 0).

```
// Comparison uses __int128 so it is exact for
  ↪ |num|,|den| up to ~9e18.
// Use whenever comparing ratios: slopes, "maximum
  ↪ average", scheduling exchange arguments.
struct Frac {
    ll n = 0, d = 1;
    Frac(ll n = 0, ll d = 1) : n(n), d(d) { norm(); }
    void norm() {
        if (d < 0) n = -n, d = -d;
        ll g = gcd(n < 0 ? -n : n, d);
        if (g) n /= g, d /= g;
        if (!d) d = 1;
  ↪ // 1/0 -> treat as 0/1; guard your input
    }
    Frac operator+(Frac o) const { return Frac(n *
  ↪ o.d + o.n * d, d * o.d); }
    Frac operator-(Frac o) const { return Frac(n *
  ↪ o.d - o.n * d, d * o.d); }
    Frac operator*(Frac o) const { return Frac(n *
  ↪ o.n, d * o.d); }
    Frac operator/(Frac o) const { return Frac(n *
  ↪ o.d, d * o.n); }
    bool operator<(Frac o) const { return (__int128)n
```

```
    * o.d < (__int128)o.n * d; }
    bool operator==(Frac o) const { return n == o.n && d == o.d; }
    bool operator<=(Frac o) const { return !(o < *this); }
    ld val() const { return (ld)n / d; }
    friend ostream& operator<<(ostream& os, Frac f) { return os << f.n << '/' << f.d; }
};
// OVERFLOW: + and * multiply denominators, so chains of operations blow up fast. If you only
// need COMPARISONS, keep the raw pair and cross-multiply in __int128 - do not normalise.
// For unbounded exactness use Python/BigInt, or the Stern-Brocot search (ContinuedFractions).
```

1.14 Inversions

INVERSIONS: #pairs i < j with a[i] > a[j]. O(n log n) by merge sort (also sorts a).

```
ll inversions(vector<int>& a) {
    int n = a.size();
    if (n < 2) return 0;
    vector<int> buf(n);
    function<ll(int, int)> rec = [&](int l, int r) -> ll {          // [l, r)
        if (r - l < 2) return 0;
        int m = (l + r) / 2;
        ll res = rec(l, m) + rec(m, r);
        int i = l, j = m, k = l;
        while (i < m && j < r) {
            if (a[i] <= a[j]) buf[k++] = a[i++];
            else buf[k++] = a[j++], res += m - i;
                // a[i..m) all beat a[j]
        }
        while (i < m) buf[k++] = a[i++];
        while (j < r) buf[k++] = a[j++];
        copy(buf.begin() + l, buf.begin() + r, a.begin() + l);
        return res;
    };
    return rec(0, n);
}
// WHY IT KEEPS APPEARING
//  - minimum number of ADJACENT SWAPS to sort = inversion count
//  - minimum adjacent swaps to turn a into b: relabel b's positions, then count inversions
//  - "how far is this permutation from sorted" / bubble-sort distance
//  - inversions modulo 2 = permutation PARITY (= sign, used in determinants and 15-puzzle)
// BIT VERSION (needed when you must also support updates, or count online):
//   compress values, sweep left to right, ans += (#inserted so far) - query(a[i]), then add a[i].
```

1.15 ExpressionParser

EXPRESSION PARSER (shunting-yard). Handles + - * / % ^, unary minus, parentheses.

```
// Left-associative except ^. Extend by editing prec() / applyOp().
struct Parser {
    static int prec(char op) {
        switch (op) {
            case '+': case '-': return 1;
            case '*': case '/': case '%': return 2;
            case '^': return 3;
            case '~': return 4;
// unary minus (internal symbol)
            default: return -1;
        }
    }
    static bool rightAssoc(char op) { return op == '^' || op == '~'; }

    static ll apply(char op, ll a, ll b = 0) {
        switch (op) {
            case '+': return a + b;
            case '-': return a - b;
            case '*': return a * b;
            case '/': return a / b;
            case '%': return a % b;
            case '~': return -a;
            case '^': { ll r = 1; while (b > 0) { if (b & 1) r *= a; a *= a, b >>= 1; } return r; }
        }
        return 0;
    }
    static ll eval(const string& s) {
        vector<ll> val;
        vector<char> ops;
        bool wantOperand = true;
// distinguishes unary from binary minus
        auto pop = [&]() {
            char op = ops.back(); ops.pop_back();
            if (op == '~') { ll a = val.back(); val.pop_back(); val.push_back(apply(op, a)); }
            else { ll b = val.back(); val.pop_back(); ll a = val.back(); val.pop_back();
                   val.push_back(apply(op, a, b)); }
        };
        for (size_t i = 0; i < s.size(); i++) {
            char c = s[i];
            if (isspace((unsigned char)c)) continue;
            if (isdigit((unsigned char)c)) {
                ll x = 0;
                while (i < s.size() && isdigit((unsigned char)s[i])) x = x * 10 + (s[i++] - '0');
                i--;
                val.push_back(x), wantOperand = false;
            } else if (c == '(') ops.push_back(c), wantOperand = true;
            else if (c == ')') {
                while (!ops.empty() && ops.back() != '(') pop();
                if (!ops.empty()) ops.pop_back();
                wantOperand = false;
            } else {
                if (c == '-' && wantOperand) c = '~';  // unary
                while (!ops.empty() && ops.back() != '(' &&
                       (prec(ops.back()) > prec(c) ||
                        (prec(ops.back()) == prec(c) && !rightAssoc(c))))
                    pop();
                ops.push_back(c), wantOperand = (c != '~') || true;
            }
        }
        while (!ops.empty()) pop();
        return val.empty() ? 0 : val.back();
    }
```

```
};
// To build an AST instead of evaluating, push node indices instead of values.
// For boolean/comparison operators just add them to prec() with lower precedence.
```

1.16 CycleFinding

CYCLE FINDING in a FUNCTIONAL GRAPH x -> f(x). Every component is a "rho": a tail feeding

```
// into a cycle. Answers: where does x end up after k steps, cycle length, distance to cycle.

// Floyd (tortoise & hare), O(1) memory. Returns {start index of the cycle, cycle length}.
template <class F> pair<ll, ll> floydCycle(ll x0, F f) {
    ll t = f(x0), h = f(f(x0));
    while (t != h) t = f(t), h = f(f(h));
    ll mu = 0;
    // steps from x0 to the cycle entry
    t = x0;
    while (t != h) t = f(t), h = f(h), mu++;
    ll lam = 1;
    // cycle length
    h = f(t);
    while (t != h) h = f(h), lam++;
    return {mu, lam};
}
// f applied k times, using the rho structure (k can be 1e18).
template <class F> ll iterate(ll x0, ll k, F f) {
    auto [mu, lam] = floydCycle(x0, f);
    if (k < mu) { while (k--) x0 = f(x0); return x0; }
    for (ll i = 0; i < mu; i++) x0 = f(x0);
    ll rem = (k - mu) % lam;
    while (rem--) x0 = f(x0);
    return x0;
}
// WHOLE-GRAPH version when f is given as an array over 0..n-1: colour DFS, O(n).
// state: 0 unvisited, 1 on the current path, 2 done.  Fills cycId/cycPos/distToCycle.
struct FuncGraph {
    int n;
    vector<int> f, state, cyc, pos, dist, cycLen;
    // cyc[v] = id of the cycle v reaches
    FuncGraph(vector<int> f) : n(f.size()), f(f), state(n, 0), cyc(n, -1), pos(n, -1), dist(n, 0) {
        for (int i = 0; i < n; i++) if (!state[i]) {
            vector<int> path;
            int v = i;
            while (!state[v]) state[v] = 1, path.push_back(v), v = f[v];
            if (state[v] == 1) {
                // found a NEW cycle, starting at v
                int id = cycLen.size(), len = 0;
                for (int u = v;; u = f[u]) { cyc[u] = id, pos[u] = len++; if (f[u] == v) break; }
                cycLen.push_back(len);
            }
            for (int j = path.size() - 1; j >= 0; j--) {
                int u = path[j];
                state[u] = 2;
                if (cyc[u] == -1) cyc[u] = cyc[f[u]], dist[u] = dist[f[u]] + 1;
            }
        }
    }
    int kth(int v, ll k) {
        // f^k(v)
        while (k > 0 && dist[v] > 0) v = f[v], k--;
        if (!k) return v;
        int L = cycLen[cyc[v]], p = (pos[v] + k % L) % L, u = v;
        for (int i = 0; i < (p - pos[v] + L) % L; i++) u = f[u];
        return u;
    }
};
// USES: "apply the permutation k times", Pollard's rho, "where does the ball stop",
// detecting a repeat in a modular sequence, linked-list loop detection.
```

1.17 ParallelBinarySearch

PARALLEL BINARY SEARCH - binary search the answer for MANY queries at once, sweeping the

```
// shared structure once per level instead of once per query.
// PRECONDITION: test(i) must be MONOTONE in time - once it is true it stays true. If updates
// can also undo things, parallel binary search does not apply; use offline divide & conquer over
// the timeline with a rollback DSU (02 - Data Structures/14) instead.
// Total: O(log(maxT) * (cost of one full sweep)).
// Shape: "after which update does query q first become satisfiable?"
//   lo[q], hi[q] track each query's remaining range; every round, bucket the queries by their
//   midpoint, replay the updates in order, and test each query when the sweep reaches its mid.
template <class Apply, class Reset, class Test>
vector<int> parallelBinarySearch(int q, int T, Apply applyUpdate, Reset reset, Test test) {
    // applyUpdate(t): apply update t to the shared structure
    // reset(): clear the structure
    // test(i): is query i satisfied by the current structure?
    vector<int> lo(q, 0), hi(q, T);
    // answer in [lo, hi]; hi == T means "never"
    while (true) {
        vector<vector<int>> at(T + 1);
        bool active = false;
        for (int i = 0; i < q; i++)
            if (lo[i] < hi[i]) at[(lo[i] + hi[i]) / 2].push_back(i), active = true;
        if (!active) break;
        reset();
        for (int t = 0; t <= T; t++) {
            if (t) applyUpdate(t - 1);
            // now updates 0..t-1 are applied
            for (int i : at[t]) { if (test(i)) hi[i] = t; else lo[i] = t + 1; }
        }
    }
    return lo;
    // lo[i] == T  =>  never satisfied
}
// CONCRETE USES
//  - "for each query, the first moment a path/edge set connects u and v"  (structure = DSU)
```

```
//  - k-th smallest in a range, offline (structure =
  ↪ BIT over values, updates = insert a value)
//  - "smallest capacity so that a flow/matching of
  ↪ size k exists"
//  - anything where a single query's binary search
  ↪ would need its own O(T) replay.
// If the structure supports ROLLBACK, prefer
  ↪ divide-and-conquer over the timeline instead
// (see OfflineDynamicConnectivity) - it is one pass
  ↪ instead of log passes.
```

1.18 MeetInTheMiddle

MEET IN THE MIDDLE - n <= 40. Split into halves, enumerate 2^(n/2) subsets each side,

```
// then recombine by sorting + binary search / two
  ↪ pointers. O(2^(n/2) * n).

// All subset sums of a half, as a sorted vector.
vector<ll> subsetSums(const vector<ll>& a) {
    int n = a.size();
    vector<ll> s(1 << n, 0);
    for (int m = 1; m < (1 << n); m++) {
        int b = __builtin_ctz(m);
        s[m] = s[m ^ (1 << b)] + a[b];
    }
    return s;
}
// #subsets with sum <= S. Works with NEGATIVE values
  ↪ too (no pre-filter on the left half).
ll countAtMost(vector<ll> a, ll S) {
    int n = a.size(), h = n / 2;
    vector<ll> L(a.begin(), a.begin() + h),
  ↪ R(a.begin() + h, a.end());
    auto ls = subsetSums(L), rs = subsetSums(R);
    sort(all(rs));
    ll c = 0;
    for (ll x : ls) c += upper_bound(all(rs), S - x)
  ↪ - rs.begin();
    return c;
}
// Largest achievable sum <= S (the classic "closest
  ↪ subset sum from below").
ll bestAtMost(vector<ll> a, ll S) {
    int n = a.size(), h = n / 2;
    vector<ll> L(a.begin(), a.begin() + h),
  ↪ R(a.begin() + h, a.end());
    auto ls = subsetSums(L), rs = subsetSums(R);
    sort(all(rs));
    ll best = LLONG_MIN;
    for (ll x : ls) {
  ↪  // no `x <= S` filter: x may be > S
        auto it = upper_bound(all(rs), S - x);
  ↪  // and still pair with a negative right sum
        if (it != rs.begin()) best = max(best, x +
  ↪ *prev(it));
    }
    return best;
}
// OTHER SHAPES
//  - EXACT target: put one half in a hash map, look
  ↪ up (target - x) from the other.
//  - 4-SUM: split the four arrays 2+2, hash all
  ↪ pairwise sums.
//  - "choose exactly k items": bucket each half's
  ↪ subsets BY POPCOUNT, then pair bucket i with
//    bucket k-i (sort each bucket once).
//  - Optimising two objectives: sort one half by
  ↪ objective A and take a prefix maximum of
//    objective B, so each query is one binary search.
// If n <= 20 a plain 2^n loop is simpler; meet in
  ↪ the middle earns its keep from n = 30 upward.
```

1.19 Scheduling

SCHEDULING / EXCHANGE-ARGUMENT THEOREM CARD

```
/* ============== SCHEDULING / EXCHANGE-ARGUMENT
  ↪ THEOREM CARD ==============
 Each of these turns a hard-looking problem into a
  ↪ sort. Prove any new one by the EXCHANGE
 ARGUMENT: assume an optimal schedule, swap two
  ↪ ADJACENT jobs, and compare the two costs.

 SMITH'S RULE  (1 || sum w_j C_j : minimise the
  ↪ weighted sum of completion times)
   Sort by p_j / w_j ASCENDING. Adjacent swap of h,k
  ↪ improves iff w_k*p_h - w_h*p_k > 0,
   so ALWAYS compare by cross-multiplication (w_k*p_h
  ↪ vs w_h*p_k), never by float division.

 MOORE-HODGSON  (1 || sum U_j : minimise the NUMBER
  ↪ of late jobs), O(n log n)
   Sort by deadline ascending; keep a max-heap of the
  ↪ processing times taken so far.
   For each job: take it, push p_j, add p_j to the
  ↪ running time; if the running time exceeds
   this job's deadline, pop the largest p and
  ↪ subtract it. The heap size is the answer
   (the rejected jobs can go last in any order).

 EARLIEST DEADLINE FIRST  (preemptive, one machine)
   If ANY schedule meets all deadlines, EDF does.
  ↪ Feasibility check for non-preemptive
   unit-machine: sort by deadline and verify prefix
  ↪ sums of p_j never exceed d_j.

 JOHNSON'S RULE  (F2 || Cmax : two-machine flow shop,
  ↪ minimise makespan)
   Set A = jobs with p1 <  p2, sorted by p1 ASCENDING
   Set B = jobs with p1 >= p2, sorted by p2 DESCENDING
   Optimal order = A then B.

 LAWLER  (1 | prec | f_max : minimise the maximum
  ↪ penalty, with precedence), O(n^2)
   Build the schedule BACK TO FRONT. With t = total
  ↪ remaining time, among the jobs that have
   no unscheduled successor pick the one minimising
  ↪ f_j(t), and place it last.

 CLASSIC GREEDY COMPANIONS
   Interval scheduling (max #non-overlapping): sort
  ↪ by RIGHT endpoint, take greedily.
   Interval point cover (stab all intervals): sort by
  ↪ right endpoint, place a point at it.
   Minimum #machines for overlapping intervals: max
  ↪ overlap = sweep +1/-1 over endpoints.
   Deadline scheduling with profits (each job unit
  ↪ length): sort by profit DESC, place each
   job as late as possible before its deadline (DSU
  ↪ "next free slot" makes it near-linear).
   Fractional knapsack: sort by value/weight DESC.
   Minimise sum of |x - a_i|: x = the MEDIAN.
  ↪ Minimise sum of (x - a_i)^2: x = the MEAN.
============================================================
  ↪ */
```

```
// Moore-Hodgson: maximum number of jobs finished on time. jobs = {processing time, deadline}.
int maxOnTime(vector<pair<ll, ll>> jobs) {
    sort(all(jobs), [](auto& a, auto& b) { return a.second < b.second; });
    priority_queue<ll> pq;
    ll t = 0;
    for (auto [p, d] : jobs) {
        t += p, pq.push(p);
        if (t > d) t -= pq.top(), pq.pop();
    }
    return pq.size();
}
// Johnson's rule: optimal two-machine flow-shop order (returns the job indices).
vector<int> johnson(const vector<pair<ll, ll>>& jobs) {     // {p on machine 1, p on machine 2}
    vector<int> A, B;
    for (int i = 0; i < (int)jobs.size(); i++)
        (jobs[i].first < jobs[i].second ? A : B).push_back(i);
    sort(all(A), [&](int x, int y) { return jobs[x].first < jobs[y].first; });
    sort(all(B), [&](int x, int y) { return jobs[x].second > jobs[y].second; });
    A.insert(A.end(), all(B));
    return A;
}
```

1.20 Josephus

JOSEPHUS PROBLEM. n people in a circle 0..n-1, every k-th is removed; who survives?

```
// Recurrence: J(1) = 0, J(n) = (J(n-1) + k) mod n. O(n).
ll josephus(ll n, ll k) { ll r = 0; for (ll i = 2; i <= n; i++) r = (r + k) % i; return r; }
// k = 2 closed form: write n = 2^m + l with 0 <= l < 2^m, the survivor is 2*l  (0-indexed).
ll josephus2(ll n) { return 2 * (n - (1LL << (63 - __builtin_clzll(n)))); }
// LARGE n, SMALL k: one full lap kills floor(n/k) people, so recurse on n - floor(n/k) and map the
// index back. O(k log n).
ll josephusFast(ll n, ll k) {
    if (n == 1) return 0;
    if (k == 1) return n - 1;
    if (k > n) return (josephusFast(n - 1, k) + k) % n;
    ll r = josephusFast(n - n / k, k) - n % k;
    return r < 0 ? r + n : r + r / (k - 1);
}
// ORDER OF ELIMINATION / the j-th person to die: keep a BIT over alive flags and binary-search the
// ((pos + k - 1) mod alive + 1)-th alive person each step. O(n log n) and handles "k changes".
// LAST TWO SURVIVORS: run the recurrence starting from J(2) = {0, 1} instead of J(1) = 0.

// OTHER ONE-LINE CLASSICS worth having on paper:
// GRAY CODE:      g(i) = i ^ (i >> 1);   inverse: x ^= x >> 1, x >> 2, ... until 0.
// n-th PERMUTATION (factorial number system): digit i = k / (n-1-i)!, then k %= (n-1-i)!.
// 15-PUZZLE SOLVABLE iff (#inversions of the tiles, blank excluded) + (row of the blank counted
//   from the BOTTOM, 1-based) is EVEN, for a 4x4 board. For an odd width w, solvable iff the
//   inversion count is even. General w x h:
//   inversions + (blank row from bottom) parity rule above
//   when w is even, plain inversion parity when w is odd.
// TOWER OF HANOI: 2^n - 1 moves; on move t (1-based) move disk (t & -t) counted by trailing zeros,
//   and for odd n the smallest disk cycles A->C->B->A, for even n A->B->C->A.
// BULLS AND COWS / stable-marriage style pairings: Gale-Shapley is O(n^2), always terminates, and
//   is optimal for the proposing side.
// CATALAN-STYLE BALLOT COUNTING: paths from (0,0) to (n,m) staying strictly above y = x are
//   (n-m)/(n+m) * C(n+m, m)  (Bertrand's ballot theorem).
```

1.21 BigInt

BIG INTEGERS, non-negative, base 1e9. Enough for the usual "the answer does not fit in 64 bits"

```
// problems: exact factorials / Catalan numbers / determinants, huge Fibonacci, exact comparisons.
// Add/sub O(n), schoolbook multiply O(n*m) (fine to ~1e4 digits), divide by a small int O(n).
// If you need big*big beyond that, convolve the limbs with FFT and carry.
struct Big {
    static const ll B = 1000000000;
    vector<ll> d;
    // little-endian limbs, base 1e9
    Big(ll v = 0) { for (; v > 0; v /= B) d.push_back(v % B); }
    Big(const string& s) {
        for (int i = s.size(); i > 0; i -= 9)
            d.push_back(stoll(s.substr(max(0, i - 9), min(9, i))));
        trim();
    }
    void trim() { while (!d.empty() && d.back() == 0) d.pop_back(); }
    bool operator<(const Big& o) const {
        if (d.size() != o.d.size()) return d.size() < o.d.size();
        for (int i = d.size() - 1; i >= 0; i--) if (d[i] != o.d[i]) return d[i] < o.d[i];
        return false;
    }
    Big operator+(const Big& o) const {
        Big r; ll c = 0;
        for (size_t i = 0; i < max(d.size(), o.d.size()) || c; i++) {
            if (i < d.size()) c += d[i];
            if (i < o.d.size()) c += o.d[i];
            r.d.push_back(c % B), c /= B;
        }
        return r;
    }
    Big operator-(const Big& o) const {
        // requires *this >= o
        Big r = *this; ll c = 0;
        for (size_t i = 0; i < r.d.size(); i++) {
            r.d[i] -= c + (i < o.d.size() ? o.d[i] : 0);
            if ((c = r.d[i] < 0)) r.d[i] += B;
        }
        r.trim();
        return r;
```

```
    }
    Big operator*(const Big& o) const {
        if (d.empty() || o.d.empty()) return Big();
        vector<lll> t(d.size() + o.d.size(), 0);
        for (size_t i = 0; i < d.size(); i++)
            for (size_t j = 0; j < o.d.size(); j++)
  ↪ t[i + j] += (lll)d[i] * o.d[j];
        Big r; lll c = 0;
        for (size_t i = 0; i < t.size(); i++) c +=
  ↪ t[i], r.d.push_back((ll)(c % B)), c /= B;
        for (; c; c /= B) r.d.push_back((ll)(c % B));
        r.trim();
        return r;
    }
    Big operator*(ll v) const { return *this *
  ↪ Big(v); }
    Big operator/(ll v) const {
  ↪          // small divisor only
        Big r = *this; lll c = 0;
        for (int i = r.d.size() - 1; i >= 0; i--) c =
  ↪ c * B + r.d[i], r.d[i] = (ll)(c / v), c %= v;
        r.trim();
        return r;
    }
    ll operator%(ll v) const {
        lll c = 0;
        for (int i = d.size() - 1; i >= 0; i--) c =
  ↪ (c * B + d[i]) % v;
        return (ll)c;
    }
    string str() const {
        if (d.empty()) return "0";
        string s = to_string(d.back());
        for (int i = d.size() - 2; i >= 0; i--) {
            string t = to_string(d[i]);
            s += string(9 - t.size(), '0') + t;
        }
        return s;
    }
};
// SIGNED: keep a separate sign flag and dispatch +/-
  ↪ to add/sub on magnitudes; comparison first.
// BASE 1e9 packs 9 decimal digits per limb, so
  ↪ printing is trivial - that is the whole point.
// FAST DIVISION of two Bigs is long and almost never
  ↪ needed; if a problem really wants it, binary
// search the quotient limb by limb, or move to
  ↪ Python-style thinking and avoid it.
// SQRT: binary search with the multiply above, or
  ↪ Newton on doubles then fix up by a few steps.
```

1.22 RankingAndDates

RANKING / UNRANKING COMBINATORIAL OBJECTS, and CALENDAR ARITHMETIC. Small, and each one costs

```
// 20 minutes to re-derive under pressure.

// --- FACTORIAL NUMBER SYSTEM: rank and unrank a
  ↪ PERMUTATION. O(n^2) as written; use an
// order-statistic tree or a BIT for O(n log n) when
  ↪ n is large.
// order(p) = how many permutations of the same
  ↪ multiset are lexicographically smaller.
ll permRank(vector<int> p) {
  ↪          // p is a permutation of 0..n-1
    int n = p.size();
    ll r = 0, f = 1;
    for (int i = n - 1; i >= 0; i--) {
        int c = 0;
        for (int j = i + 1; j < n; j++) if (p[j] <
  ↪ p[i]) c++;   // the Lehmer code digit
        r += c * f, f *= (n - i);
    }
    return r;
}
vector<int> permUnrank(int n, ll k) {
  ↪          // k-th permutation, 0-indexed
    vector<int> avail(n), p(n);
    iota(all(avail), 0);
    vector<ll> f(n + 1, 1);
    for (int i = 1; i <= n; i++) f[i] = f[i - 1] * i;
    for (int i = 0; i < n; i++) {
        ll d = k / f[n - 1 - i];
        k %= f[n - 1 - i];
        p[i] = avail[d], avail.erase(avail.begin() +
  ↪ d);
    }
    return p;
}
// NEXT PERMUTATION in place is
  ↪ std::next_permutation, O(n) - use it unless you
  ↪ need the rank.
// COMBINATIONS: rank of a k-subset {c_1<...<c_k} of
  ↪ [n] is sum C(c_i, i); unrank greedily by
// finding the largest c with C(c, i) <= remaining.
  ↪ Same shape, C instead of factorials.
// GRAY CODE is g = i ^ (i >> 1) (06 - grayCode.cpp).

// --- BALANCED BRACKET SEQUENCES: rank, unrank, and
  ↪ the next one in lexicographic order, with
// '(' < ')'. The counting table is the BALLOT
  ↪ NUMBERS: f[i][j] = number of ways to finish when
  ↪ i
// characters remain and j brackets are still open.
  ↪ f[n][0] is the Catalan number C_{n/2}.
// USES: "the k-th valid bracket sequence", "how many
  ↪ valid sequences are smaller than this one",
// iterating all sequences of length 2m without
  ↪ generating and filtering. Same skeleton works for
// any object counted by a DP: the rank is a walk
  ↪ that adds the subtree sizes you skip over.
// O(n^2) to build the table, O(n) per rank / unrank
  ↪ / next. Overflows for n > 60 - use __int128
// or a modulus if you only need the count.
vector<vector<ll>> ballot(int n) {
  ↪          // n = total length, must be even
    vector<vector<ll>> f(n + 1, vector<ll>(n + 2, 0));
    f[0][0] = 1;
    for (int i = 1; i <= n; i++)
        for (int j = 0; j <= i; j++)
            f[i][j] = f[i - 1][j + 1] + (j ? f[i -
  ↪ 1][j - 1] : 0);   // place '(' then ')'
    return f;
}
ll bracketRank(const string& s, const
  ↪ vector<vector<ll>>& f) {  // 0-indexed rank, s
  ↪ must be valid
    int n = s.size(), j = 0;
    ll r = 0;
    for (int i = 0; i < n; i++) {
        if (s[i] == ')') r += f[n - i - 1][j + 1];
  ↪           // skip every sequence with '(' here
        j += s[i] == '(' ? 1 : -1;
    }
    return r;
}
```

```
string bracketUnrank(int n, ll k, const
  ↪ vector<vector<ll>>& f) {
    string s;
    int j = 0;
    for (int i = 0; i < n; i++) {
        ll c = f[n - i - 1][j + 1];
  ↪           // completions starting with '('
        if (k < c) s += '(', j++;
        else k -= c, s += ')', j--;
    }
    return s;
}
bool nextBracket(string& s) {
  ↪          // in place; false if s is the last
    int n = s.size(), d = 0;
    for (int i = n - 1; i >= 0; i--) {
        d += s[i] == '(' ? -1 : 1;
        if (s[i] == '(' && d > 0) {
  ↪           // flip it to ')' and reset the rest
            d--;
            int open = (n - i - 1 - d) / 2;
            s = s.substr(0, i) + ')' + string(open,
  ↪ '(') + string(n - i - 1 - open, ')');
            return true;
        }
    }
    return false;
}

// --- JULIAN DAY NUMBER: date <-> integer, closed
  ↪ form, no loops, correct for the Gregorian
// calendar including every leap rule. Date
  ↪ differences, "what weekday", and "add 1000 days"
  ↪ all
// become integer arithmetic once you are here.
ll dateToInt(int y, int m, int d) {
  ↪          // y-m-d Gregorian -> day number
    return 1461 * (y + 4800 + (m - 14) / 12) / 4
         + 367 * (m - 2 - (m - 14) / 12 * 12) / 12
         - 3 * ((y + 4900 + (m - 14) / 12) / 100) / 4
  ↪ + d - 32075;
}
array<int, 3> intToDate(ll jd) {
  ↪          // inverse: {y, m, d}
    ll x, n, i, j, dd;
    x = jd + 68569, n = 4 * x / 146097, x -= (146097
  ↪ * n + 3) / 4;
    i = 4000 * (x + 1) / 1461001, x -= 1461 * i / 4 -
  ↪ 31;
    j = 80 * x / 2447, dd = x - 2447 * j / 80, x = j
  ↪ / 11;
    return {(int)(100 * (n - 49) + i + x), (int)(j +
  ↪ 2 - 12 * x), (int)dd};
}
int weekday(ll jd) { return (int)((jd + 1) % 7); }
  ↪           // 0 = Sunday
// LEAP YEAR: y % 4 == 0 && (y % 100 != 0 || y % 400
  ↪ == 0).
// ZELLER'S CONGRUENCE gives the weekday directly,
  ↪ but the day number is strictly more useful.

// --- STERN-BROCOT SEARCH: the simplest fraction
  ↪ satisfying a MONOTONE predicate. ---
// Descend the mediant tree, but take many steps in
  ↪ one direction at once (double the step size)
// so a 1e18 denominator costs O(log^2) instead of
  ↪ O(answer).
// USES: "smallest denominator q with property P",
  ↪ recovering p/q from a decimal prefix, the
// simplest fraction strictly inside an interval, and
  ↪ best rational approximation with a bound.
// ok() must be MONOTONE in the value p/q: false on
  ↪ small fractions, true from some point on.
// Returns the SMALLEST such fraction with
  ↪ denominator <= N. Each iteration takes as many
  ↪ steps
// as possible in one direction (found by doubling,
  ↪ then binary search), so it is O(log^2 N)
// rather than O(answer) - the difference between
  ↪ instant and hopeless at N = 1e18.
template <class F>
pair<ll, ll> fracSearch(ll N, F ok) {
    ll lp = 0, lq = 1, rp = 1, rq = 0;
  ↪           // 0/1 (not ok) .. 1/0 (ok)
    for (;;) {
        ll moved = 0;
        auto walk = [&](ll& ap, ll& aq, ll bp, ll bq,
  ↪ bool want) {   // steps from a toward b
            ll k = 0, s = 1;
            auto test = [&](ll t) {
                return aq + t * bq <= N && ok(ap + t
  ↪ * bp, aq + t * bq) == want;
            };
            while (test(k + s)) k += s, s *= 2;
            for (s /= 2; s; s /= 2) if (test(k + s))
  ↪ k += s;
            ap += k * bp, aq += k * bq, moved += k;
        };
        walk(lp, lq, rp, rq, false);
  ↪           // push the FALSE end up
        if (lq + rq > N) return {rp, rq};
        walk(rp, rq, lp, lq, true);
  ↪           // pull the TRUE end down
        if (lq + rq > N || !moved) return {rp, rq};
  ↪           // !moved: nothing left to refine
    }
}
// The Stern-Brocot tree IS the continued fraction
  ↪ expansion: going left/right k times is one
// partial quotient. See 04 - Number Theory/05 -
  ↪ Rational for the CF machinery and
  ↪ semiconvergents.
```

1.23 MatroidIntersection

MATROID INTERSECTION - the largest set that is independent in TWO matroids at once, and the

```
// max-weight version. Ground set is {0..n-1}; each
  ↪ matroid arrives as an ORACLE object exposing
//   build(S)      prepare for the current
  ↪ independent set S (a sorted vector of element
  ↪ ids)
//   canAdd(e)     is S + e independent?          (e
  ↪ not in S)
//   canSwap(e, x) is S - x + e independent?      (e
  ↪ not in S, x in S)
// build is called once per augmenting phase, so it
  ↪ may rebuild everything from scratch - that is
// exactly why the concrete oracles below can get
  ↪ away with |S| + 1 rebuilt structures per phase.
// matroidIsect returns a maximum-CARDINALITY common
  ↪ independent set, sorted, as element ids.
// matroidIsectW returns a maximum-WEIGHT one over
  ↪ all cardinalities; force maximum cardinality
// first by adding C > sum|w| to every weight.
// WHEN: one selection problem carrying TWO
```

```
  ↪ independent structural constraints. Colourful
  ↪ spanning
// tree / spanning tree with at most c_i edges of
  ↪ colour i, and rainbow spanning tree (c_i = 1):
// graphic + partition. Arborescence: graphic on the
  ↪ underlying undirected graph + partition by
// head vertex with cap 1, so "one incoming edge per
  ↪ vertex, no cycle". Spanning tree with bounded
// degrees: partition the edges by an endpoint -
  ↪ legal ONLY if no edge joins two constrained
// vertices, otherwise an edge lands in two classes
  ↪ and the reduction is silently wrong. Bipartite
// matching is two partition matroids (but use
  ↪ Hopcroft-Karp). THREE matroids is NP-hard.
// MIN-MAX: max |I| = min over U of (rank1(U) +
  ↪ rank2(complement of U)), handy for proving a
  ↪ bound
// or for a "why is the answer stuck at k" argument.
// COMPLEXITY in oracle calls: one phase pops each
  ↪ element once and spends O(|S|) or O(n) calls on
// it, so O(r n) calls plus 2 builds per phase, and
  ↪ there are r + 1 phases: O(r^2 n) calls total.
// O(r^1.5 n) is the best known bound for this scheme
  ↪ and needs Cunningham's phase batching
// (augment along many shortest paths per BFS,
  ↪ Hopcroft-Karp style) - rarely worth typing.
  ↪ Budget
// r * n * (cost of one oracle call) per phase and
  ↪ multiply by the number of phases. Weighted:
// Bellman-Ford over n nodes and O(r n) arcs, so O(r
  ↪ n^2) per phase on top of the same oracle work.
// Memory is the oracles': O(r V) ints for graphic,
  ↪ O(63 r) for xor, O(colours) for partition.
// DEGENERATE: n = 0 or every element a LOOP
  ↪ (self-loop edge, zero vector, colour with cap 0)
// returns {}; loops fail canAdd on an empty S and
  ↪ are never picked. Duplicate / parallel elements
// are fine. Ties are broken arbitrarily - no
  ↪ lexicographic guarantee. Weighted with all
  ↪ weights
// negative returns {}. Both routines start from S =
  ↪ {} and cost nothing extra if the answer is 0.
// EXACT: pure combinatorics, no floating point;
  ↪ weights are ll and one path cost sums at most n
  ↪ of
// them, so |w| <= 1e17 is safe. Two traps that fail
  ↪ SILENTLY: the unweighted search must use BFS
// (a shortest augmenting path, fewest arcs) and the
  ↪ weighted one must break cost ties by fewest
// arcs - swap in a DFS or drop the tiebreak and you
  ↪ get a set that is not independent.
template <class M1, class M2>
vector<int> matroidIsect(int n, M1& m1, M2& m2) {
    vector<char> in(n, 0);
    vector<int> S;
    for (;;) {
        m1.build(S), m2.build(S);
        vector<int> par(n, -2), q;
  ↪ // -2 unvisited, -1 source; q is the BFS order
        int t = -1;
        for (int e = 0; e < n; e++) if (!in[e] &&
  ↪ m1.canAdd(e)) par[e] = -1, q.pb(e);
        for (int h = 0; h < sz(q) && t < 0; h++) {
            int u = q[h];
            if (in[u]) {
  ↪ // leaving S: u out, some v in, keep M1 happy
                for (int v = 0; v < n; v++)
                    if (par[v] == -2 && !in[v] &&
  ↪ m1.canSwap(v, u)) par[v] = u, q.pb(v);
            } else if (m2.canAdd(u)) t = u;
  ↪ // reached the sink side: augment
            else for (int v : S) if (par[v] == -2 &&
  ↪ m2.canSwap(u, v)) par[v] = u, q.pb(v);
        }
        if (t < 0) return S;
  ↪ // no augmenting path: S is maximum
        for (int v = t; v != -1; v = par[v]) in[v] ^=
  ↪ 1;
        S.clear();
        for (int e = 0; e < n; e++) if (in[e])
  ↪ S.pb(e);
    }
}
template <class M1, class M2>
vector<int> matroidIsectW(int n, vector<ll> w, M1&
  ↪ m1, M2& m2) {
    vector<char> in(n, 0);
    vector<int> S, best;
    ll cur = 0, bw = 0;
    for (;;) {
        m1.build(S), m2.build(S);
        vector<pair<ll, int>> d(n, {llinf, inf});
  ↪ // (path cost, arcs) - lexicographic
        vector<int> par(n, -1);
        vector<pii> ed;
        vector<char> snk(n, 0);
        for (int e = 0; e < n; e++) if (!in[e]) {
            if (m1.canAdd(e)) d[e] = {-w[e], 1};
  ↪ // sources; a node costs -w in, +w out
            if (m2.canAdd(e)) snk[e] = 1;
            for (int x : S) {
                if (m1.canSwap(e, x)) ed.pb({x, e});
                if (m2.canSwap(e, x)) ed.pb({e, x});
            }
        }
        for (int it = 0; it < n; it++) {
  ↪ // Bellman-Ford: weights are negative on entry
            bool ch = false;
            for (auto& [u, v] : ed) if (d[u].fi <
  ↪ llinf) {
                pair<ll, int> nd = {d[u].fi + (in[v]
  ↪ ? w[v] : -w[v]), d[u].se + 1};
                if (nd < d[v]) d[v] = nd, par[v] = u,
  ↪ ch = true;
            }
            if (!ch) break;
        }
        int t = -1;
        for (int e = 0; e < n; e++) if (snk[e] &&
  ↪ d[e].fi < llinf && (t < 0 || d[e] < d[t])) t = e;
        if (t < 0) break;
        cur -= d[t].fi;
  ↪ // the path cost is minus the weight gained
        for (int v = t; v != -1; v = par[v]) in[v] ^=
  ↪ 1;
        S.clear();
        for (int e = 0; e < n; e++) if (in[e])
  ↪ S.pb(e);
        if (cur > bw) bw = cur, best = S;
  ↪ // S is now optimal among sets of size |S|
    }
    return best;
}
// --- ORACLE 1: COLOURFUL / PARTITION matroid.
  ↪ Independent iff colour c is used <= cap[c] times.
struct ColourM {
```

```
    vector<int> col, cap, cnt;
  ↪ // col[e] in [0, |cap|)
    ColourM(vector<int> col, vector<int> cap) :
  ↪ col(col), cap(cap), cnt(cap.size()) {}
    void build(const vector<int>& S) {
        fill(all(cnt), 0);
        for (int e : S) cnt[col[e]]++;
    }
    bool canAdd(int e) { return cnt[col[e]] <
  ↪ cap[col[e]]; }
    bool canSwap(int e, int x) { return canAdd(e) ||
  ↪ col[e] == col[x]; }
};
// --- ORACLE 2: GRAPHIC matroid. Ground set = edges
  ↪ over vertices 0..V-1, independent iff acyclic.
// One DSU for S and one for each S - x, all rebuilt
  ↪ per phase: O(r^2) per build, O(r V) memory.
struct GraphicM {
    int V; vector<pii> e; vector<int> pos; vector<vi>
  ↪ d;   // d[0] = S, d[i + 1] = S without S[i]
    GraphicM(int V, vector<pii> e) : V(V), e(e) {}
    int f(vi& p, int x) { return p[x] == x ? x : p[x]
  ↪ = f(p, p[x]); }
    bool ok(vi& p, int i) { return f(p, e[i].fi) !=
  ↪ f(p, e[i].se); }
    void build(const vector<int>& S) {
        int m = sz(S);
        pos.assign(sz(e), 0), d.assign(m + 1, vi(V));
        for (auto& p : d) iota(all(p), 0);
        for (int i = 0; i < m; i++) pos[S[i]] = i;
        for (int i = 0; i < m; i++) for (int j = 0; j
  ↪ <= m; j++) if (j != i + 1) {
            vi& p = d[j];
            p[f(p, e[S[i]].fi)] = f(p, e[S[i]].se);
  ↪ // union; a no-op when already joined
        }
    }
    bool canAdd(int x) { return ok(d[0], x); }
    bool canSwap(int x, int y) { return ok(d[pos[y] +
  ↪ 1], x); }
};
// --- ORACLE 3: LINEAR matroid over GF(2). Ground
  ↪ set = masks, independent iff xor-independent.
// b[j][i] is the basis vector whose top bit is i.
  ↪ Masks must be in [0, 2^62); 0 is a loop.
struct XorM {
    vector<ll> v; vector<int> pos; vector<vector<ll>>
  ↪ b;   // b[0] = S, b[i + 1] = S without S[i]
    XorM(vector<ll> v) : v(v) {}
    bool ok(vector<ll>& b, ll x, bool ins = false) {
        for (int i = 62; i >= 0; i--) if (x >> i & 1)
  ↪ {
            if (!b[i]) { if (ins) b[i] = x; return
  ↪ true; }
            x ^= b[i];
        }
        return false;
    }
    void build(const vector<int>& S) {
        int m = sz(S);
        pos.assign(sz(v), 0), b.assign(m + 1,
  ↪ vector<ll>(63, 0));
        for (int i = 0; i < m; i++) pos[S[i]] = i;
        for (int i = 0; i < m; i++)
            for (int j = 0; j <= m; j++) if (j != i +
  ↪ 1) ok(b[j], v[S[i]], true);
    }
    bool canAdd(int x) { return ok(b[0], v[x]); }
    bool canSwap(int x, int y) { return ok(b[pos[y] +
  ↪ 1], v[x]); }
};
// MORE ORACLES, all trivial: UNIFORM U(k, n) -
  ↪ canAdd = sz(S) < k, canSwap = true. FREE matroid
// (everything independent) - both true; intersecting
  ↪ with it degenerates to "greedy on one
// matroid". TRANSVERSAL matroid (elements matchable
  ↪ into a bipartite graph) - one Kuhn augmenting
// path per query, legal but slow. COGRAPHIC
  ↪ (complement stays connected).
// ONE matroid only, max weight: sort by weight
  ↪ descending and add while canAdd - that is
  ↪ Kruskal,
// and matroid intersection is the price you pay for
  ↪ the second constraint.
// MATROID UNION and "partition the ground set into k
  ↪ independent sets" both reduce to intersection.
```

1.24 SchreierSims

SCHREIER-SIMS - a strong generating set (a stabiliser chain) for the permutation group generated

```
// by the given permutations of {0..n-1}. Gives |G|
  ↪ exactly, membership testing in O(n^2), and an
// explicit word in the generators for any member.
  ↪ Permutations are 0-indexed images: p[i] is where
// i goes, and mul(a, b) means "apply a, THEN b", so
  ↪ (a*b)[i] = b[a[i]]. Level i of the chain holds
// coset representatives of the pointwise stabiliser
  ↪ of 0..i-1, indexed by the image of i, so
// |G| = product of the transversal sizes and every
  ↪ element factors uniquely as one rep per level.
// WHEN: "you may apply these shuffles to the array
  ↪ any number of times, in any order - how many
// distinct arrangements are reachable, and is
  ↪ arrangement X reachable". Model a shuffle as the
// permutation it applies, then the reachable
  ↪ arrangements are exactly the group elements:
  ↪ order()
// counts them and has() decides reachability. word()
  ↪ prints the moves that get you there. Also:
// the group generated by a puzzle's legal moves, the
  ↪ size of a symmetry group before a Burnside
// count, and "do these generators generate all of
  ↪ S_n" (compare order() with n!).
// CONVENTION TRAP: an arrangement is the permutation
  ↪ taking the START to it, so ask has(q) with
// q[i] = the index the item now at position i
  ↪ started at (build it once, consistently, then
  ↪ stay
// consistent). If the array has DUPLICATE values,
  ↪ the number of distinct arrangements is |G| over
// the size of the stabiliser of that multiset, NOT
  ↪ |G| - this file does not give you that.
// COMPLEXITY: level i holds at most n - i reps, so
  ↪ O(n^2) stored permutations and O(n^3) ints of
// memory (under 1 MB even at n = 60). The
  ↪ constructor sifts every rep at level i against
  ↪ every rep
// at a level >= i: O(n^4) sifts costing O(n^2) each,
  ↪ so O(n^6) when the group is all of S_n.
// MEASURED on three random generators of S_n: n = 40
  ↪ takes 0.5 s, n = 50 takes 2 s, n = 60 takes
// 6 s, and that is the wall - past it you need a
  ↪ real Schreier-Sims (Schreier generators kept per
// level, plus Jerrum's filter) or the randomised
  ↪ variant. has() and word() are O(n^2) per query.
```

```
// DEGENERATE: no generators or only identities gives order() == 1 and has(id) == true; n = 1 the
// same. Duplicate generators are free. word(identity) is the empty word.
// EXACT: integers only. OVERFLOW: |G| <= n! exceeds ll at n = 21 (20! = 2.4e18 fits), so switch
// order() to __int128 (good to 33!) or accumulate modulo the answer's modulus for larger n.
struct PermGroup {
    int n; bool keepW;
    vector<vi> lk;
    // lk[i][j] = index of the level i rep i->j
    vector<vector<vi>> T, Ti, W;
    // reps, their inverses, and their words
    static vi mul(const vi& a, const vi& b) {
        vi c(sz(a));
        for (int i = 0; i < sz(a); i++) c[i] = b[a[i]];
        return c;
    }
    static vi inv(const vi& a) {
        vi c(sz(a));
        for (int i = 0; i < sz(a); i++) c[a[i]] = i;
        return c;
    }
    // Strip p through the chain. Returns the level where p is NEW (and inserts it there if add),
    // or -1 when p reduces to the identity, i.e. p is in the group. w accumulates the word of the
    // running residue, so it ends as the word of p^-1 when p sifts through.
    int sift(vi p, vi& w, bool add) {
        for (int i = 0; i < n; i++) {
            int k = lk[i][p[i]];
            if (k < 0) {
                if (!add) return i;
                lk[i][p[i]] = sz(T[i]), T[i].pb(p), Ti[i].pb(inv(p));
                if (keepW) W[i].pb(w);
                return i;
            }
            if (keepW) for (int j = sz(W[i][k]) - 1; j >= 0; j--) w.pb(~W[i][k][j]);  // w *= rep^-1
            bool one = true;
            // p *= rep^-1 in place; now p fixes 0..i, so
            for (int j = i; j < n; j++)
                // only j >= i can move and only they matter
                one &= (p[j] = Ti[i][k][p[j]]) == j;
            if (one) return -1;
            // already the identity: skip the rest
        }
        return -1;
    }
    PermGroup(int n, const vector<vi>& gen, bool keepW = false) : n(n), keepW(keepW) {
        lk.assign(n, vi(n, -1)), T.resize(n), Ti.resize(n), W.resize(n);
        vi id(n);
        iota(all(id), 0);
        for (int i = 0; i < n; i++) lk[i][i] = 0, T[i].pb(id), Ti[i].pb(id), W[i].pb(vi());
        vector<pii> el;
        // every non-identity rep, as (level, index)
        auto push = [&](vi p, vi w) {
            int i = sift(p, w, true);
            if (i >= 0) el.pb({i, sz(T[i]) - 1});
        };
        for (int g = 0; g < sz(gen); g++)
            push(gen[g], vi{g});
        for (int i = 0; i < sz(el); i++) for (int j = 0; j <= i; j++) {   // el grows as we go
            auto prod = [&](pii a, pii b) {
                vi w;
                if (keepW) w = W[a.fi][a.se], w.insert(w.end(), all(W[b.fi][b.se]));
                push(mul(T[a.fi][a.se], T[b.fi][b.se]), w);
            };
            if (el[i].fi <= el[j].fi) prod(el[i], el[j]);       // a level i rep times a
            if (el[j].fi <= el[i].fi && i != j) prod(el[j], el[i]);   // level >= i element is
                // a Schreier generator
        }
    }
    ll order() {
        ll r = 1;
        for (int i = 0; i < n; i++) r *= sz(T[i]);
        return r;
    }
    bool has(vi p) { vi w; return sift(p, w, false) < 0; }
    vi word(vi p) {
        // PRECONDITION: has(p), and keepW was set.
        vi w;
        // token g >= 0 is gen[g], ~g is its inverse
        sift(p, w, false);
        reverse(all(w));
        for (int& x : w) x = ~x;
        // w held p^-1; invert it
        return w;
    }
};
// Rebuild p from word(p) with res = mul(res, x >= 0 ? gen[x] : inv(gen[~x])) over the tokens in
// order. Word LENGTH IS NOT BOUNDED - it is a product of stored words and can blow up; use it only
// where the checker accepts any move sequence. If you need short words, use only for small groups.
// ORBIT of one point (which positions a single item can reach) is a plain BFS on the generators
// in O(n t) - do not build the whole chain for that. The level-0 transversal IS the orbit of 0.
// The pointwise stabiliser of 0..i-1 is generated by the reps at levels >= i, which is how you
// answer "how many arrangements fix these positions" (= product of |T[j]| for j >= i).
// Related: Burnside / Polya for counting colourings up to the group, and the parity argument that
// solves the 15-puzzle style problems in one line when the group is the full alternating group.
```

1.25 RangeBitset

RANGE BITSET - a bitset with SET / RESET / FLIP of a whole range and POPCOUNT of a range, all

```
// in O((r - l) / 64 + 1). std::bitset cannot do range updates at all (its set()/reset()/flip()
// are either one bit or the whole thing), so anything that wants "turn on bits [l, r]" needs
// this. Bits are 0..n-1, ranges are INCLUSIVE on both ends, and l > r is a silent no-op.
// WHEN: a sweep that paints intervals into one bitset per colour ("recolour [l, r] to c" over
// 100 colours: AND out the range from the old bitset, OR it into the new one); interval DP
// reachability; "is any position in [l, r] still free" answered by findNext; knapsack-style DP
// where the transition is x |= x << k.
// COMPLEXITY: n/64 words. O((r-l)/64 + 1) per range op, O(1) test/set of one bit, O(n/64) for
// the word-parallel operators and for a shift. The point is the 64x constant, not the O.
// EXACT: pure unsigned integer, nothing can overflow.
// THE WORD-BOUNDARY TRAP - the only thing that ever goes wrong in this file:
//  - a range touches THREE pieces: the tail of word l>>6 (bits l&63..63), the whole words
//    strictly between, and the head of word r>>6 (bits 0..r&63).
//  - if l>>6 == r>>6 you must do ONE masked write and return. Falling through to the three-piece
//    path writes bits l&63..63 AND bits 0..r&63 of that same word, i.e. the entire word - the
//    classic off-by-one that makes this silently wrong only when l and r are 64-aligned near
//    each other. Test n = 64, 65, 128 and l, r in the same word.
//  - never shift by 64: ~0ULL << 64 is UB (x86 shifts by 0 and hands you all-ones). mask(a, b)
//    below shifts by a and by 63-b with 0 <= a <= b <= 63, so both counts are in [0, 63].
//  - keep the bits >= n zero (trim()), or count() and findNext() over the whole bitset lie.
typedef unsigned long long ull;
struct RangeBitset {
    int n; vector<ull> w;
        // w[i] holds bits 64i .. 64i + 63
    RangeBitset(int n = 0) : n(n), w((n + 63) / 64, 0) {}
    static ull mask(int a, int b) {
        // bits a..b of ONE word, 0<=a<=b<=63
        return (~0ULL << a) & (~0ULL >> (63 - b));
    }
    template <class F> void each(int l, int r, F f) {
        // f(word, mask) over the range
        if (l > r) return;
        int a = l >> 6, b = r >> 6;
        if (a == b) { f(a, mask(l & 63, r & 63)); return; }  // SAME word: one write, then STOP
        f(a, mask(l & 63, 63)), f(b, mask(0, r & 63));       // the two partial words
        for (int i = a + 1; i < b; i++) f(i, ~0ULL);
        // the full words between them
    }
    void set(int l, int r)   { each(l, r, [&](int i, ull m) { w[i] |=  m; }); }
    void reset(int l, int r) { each(l, r, [&](int i, ull m) { w[i] &= ~m; }); }
    void flip(int l, int r)  { each(l, r, [&](int i, ull m) { w[i] ^=  m; }); }
    int count(int l, int r) {
        int c = 0;
        each(l, r, [&](int i, ull m) { c += ones(w[i] & m); });
        return c;
    }
    int count() { return count(0, n - 1); }
        // n == 0 gives l > r, so 0
    bool test(int i) const { return w[i >> 6] >> (i & 63) & 1; }
    void set(int i)   { w[i >> 6] |=  1ULL << (i & 63); }
    void reset(int i) { w[i >> 6] &= ~(1ULL << (i & 63)); }
    void flip(int i)  { w[i >> 6] ^=  1ULL << (i & 63); }
    int findNext(int i) {
        // first set bit >= i, or n if none
        if (i < 0) i = 0;
        if (i >= n) return n;
        int b = i >> 6;
        ull c = w[b] & (~0ULL << (i & 63));
        // drop the bits before i
        while (!c) { if (++b == sz(w)) return n; c = w[b]; }
        return b * 64 + lsb(c);
    }
    void trim() { if (n & 63) w.back() &= mask(0, (n - 1) & 63); }   // bits >= n must stay 0
    void operator|=(const RangeBitset& o) { for (int i = 0; i < sz(w); i++) w[i] |= o.w[i]; }
    void operator&=(const RangeBitset& o) { for (int i = 0; i < sz(w); i++) w[i] &= o.w[i]; }
    void operator^=(const RangeBitset& o) { for (int i = 0; i < sz(w); i++) w[i] ^= o.w[i]; }
    void orShl(int k) {
        // x |= x << k, the subset-sum step
        int q = k >> 6, s = k & 63;
        for (int i = sz(w) - 1; i >= q; i--) {
            // DESCENDING: every source word is
            ull v = w[i - q] << s;
            // below i, so still unmodified
            if (s && i - q) v |= w[i - q - 1] >> (64 - s);   // s == 0 would be a shift by 64
            w[i] |= v;
        }
        trim();
        // bits that slid past n
    }
};
// VARIANTS / RELATED
// - Whole-bitset AND/OR/XOR plus _Find_first/_Find_next only: use std::bitset, it is faster and
//   already written. Come here the moment a RANGE update appears.
// - Range ASSIGN with O(log) amortized per op and no n/64 term: the interval set ("Chtholly",
//   a std::set of maximal constant runs). It does assign, not flip, and needs random data.
// - Range assign + range count with true polylog: a lazy segment tree. This struct wins up to
//   n around 1e7 purely on constant factor, and is a third of the code.
// - Static "count ones in [l, r]": a prefix popcount array, no structure needed.
// - findPrev: mask with mask(0, i & 63) and scan
```

```
  ↪ words downward, taking msb().
```

1.26 TrygubNum

TRYGUB NUMBER - a big number that eats 1e5+ updates of the form "add x * b^e" in O(1)

```
// amortized carries, and answers "what is digit k"
  ↪ at any time.
// WHAT: maintains V = sum over e of d[e] * base^e
  ↪ exactly, where base = b^K packs K base-b digits
// into one machine digit. add(x, e) does V += x *
  ↪ b^e with x AND e of either sign. kthDigit(k)
// returns floor(V / b^k) mod b, i.e. the k-th digit
  ↪ of V in base b counting from the units digit
// at k = 0. sign() returns +1 / 0 / -1.
// THE TRICK: digits are NOT normalised. d[e] may be
  ↪ anywhere in (-base, base), the carry is
// pushed lazily one position at a time and stops as
  ↪ soon as it dies, and only NONZERO digits are
// stored, in a map - so a negative e (a fractional
  ↪ position) costs nothing and the number needs
// no length. Amortized it is O(1) carry steps per
  ↪ add (binary-counter argument), each step one
// map operation, so O(log D) with D = number of
  ↪ stored digits. Memory O(D).
// WHEN: 1e5 updates "add 5 * 10^k" to a huge number
  ↪ with digit queries interleaved. A plain
// vector is O(len) per update; a segment tree over
  ↪ digits drowns in carry logic. Also the clean
// way to compare two such numbers: subtract, look at
  ↪ sign().
// NEGATIVE e: split e = q*K + j with FLOOR division,
  ↪ not C++ truncation. For e = -1, K = 30 the
// language gives e/K == 0 and e%K == -1, which would
  ↪ multiply x by b^(-1). Take
// j = ((e % K) + K) % K first, then q = (e - j) / K,
  ↪ which is exact in both signs.
// NEGATIVE x: nothing special in add - integer
  ↪ division truncates toward zero, so after
// d[q] -= t * base the residue KEEPS the sign of
  ↪ d[q] and |d[q]| < base. That is exactly how
// negative digits appear, and it is why kthDigit
  ↪ needs the borrow below.
// THE BORROW RULE in kthDigit (the whole
  ↪ difficulty): the digits below q may sum to a
  ↪ negative
// number. Their total is bounded by base^q in
  ↪ absolute value (sum of (base-1)*base^e for e < q
// is base^q - 1), so the top stored digit below q
  ↪ decides the SIGN of the whole low part, and a
// negative low part eats exactly 1 from position q:
  ↪ `if (prev(it)->se < 0) a--`. This is why
// zeros MUST be erased from the map - a stored 0
  ↪ sitting just under q would answer "not
// negative" and lose the borrow.
// OVERFLOW BUDGET: a digit lives in (-base, base)
  ↪ and add pushes x * b^(K-1) into it, so you
// need base + |x| * b^(K-1) < 9.2e18. b = 2, K = 30
  ↪ (base ~ 1.07e9) and b = 10, K = 9
// (base = 1e9) both leave room for |x| <= 1e9.
  ↪ Bigger K = shorter carry chains, so take the
// largest one the budget allows.
struct TrygubNum {
    ll b, K, base; vector<ll> pw;
  ↪  // pw[i] = b^i, base = b^K = one machine
    map<ll, ll> d;
  ↪  // digit. e -> digit, ZEROS ARE ERASED
    TrygubNum(ll b, ll K) : b(b), K(K), pw(K + 1, 1) {
        for (ll i = 1; i <= K; i++) pw[i] = pw[i - 1]
  ↪ * b;
        base = pw[K];
    }
    void add(ll x, ll e) {
  ↪  // V += x * b^e, both signs allowed
        ll j = ((e % K) + K) % K, q = (e - j) / K;
  ↪  // FLOOR split e = q*K + j, 0 <= j < K
        d[q] += x * pw[j];
        ll t;
        do {
            t = d[q] / base;
  ↪  // truncates toward 0: residue keeps sign
            d[q] -= t * base, d[q + 1] += t;
            if (!d[q]) d.erase(q);
  ↪  // erasing zeros is load-bearing, see above
            q++;
        } while (t);
  ↪  // carry died -> stop, O(1) amortized
        if (!d[q]) d.erase(q);
  ↪  // the digit above may have been left at 0
    }
    int kthDigit(ll k) {
  ↪  // floor(V / b^k) mod b, k >= 0
        ll q = k / K, a = 0;
        auto it = d.lower_bound(q);
        if (it != d.end() && it->fi == q) a = it->se;
        if (it != d.begin() && prev(it)->se < 0) a--;
  ↪  // low part is negative: it borrows a 1
        a = ((a % base) + base) % base;
        return (int)(a / pw[k % K] % b);
  ↪  // the (k % K)-th base-b digit of a
    }
    int sign() {
  ↪  // top stored digit dominates all the rest
        return d.empty() ? 0 : (prev(d.end())->se > 0
  ↪ ? 1 : -1);
    }
};
// DEGENERATE: empty number -> sign() 0 and
  ↪ kthDigit() 0 everywhere. add(0, e) stores
  ↪ nothing.
// Adding and then subtracting the same term returns
  ↪ the map to empty. k above the top digit
// gives 0. kthDigit is meant for V >= 0; for V < 0
  ↪ it still returns floor(V / b^k) mod b, which
// is the b-complement expansion (...bbb1 for -1),
  ↪ not the digits of |V| - negate first.
// VARIANTS / RELATED
// - Reading the whole number: from prev(d.end())->fi
  ↪ * K + K - 1 downward through kthDigit.
// - Only nonnegative e and no queries until the end:
  ↪ just use a vector and normalise once.
// - Full arithmetic (multiply, divide, print): 21 -
  ↪ BigInt.cpp. This struct only does += x*b^e.
// - Same lazy-carry idea works for "add x at
  ↪ position e" in any positional structure; the map
  ↪ is
//   the only reason it stays sparse.
```

1.27 PermutationRoots

CYCLE DECOMPOSITION AND WHAT IT ANSWERS: the k-th root of a permutation, the minimum number of

```
// transpositions (with the swaps), and the order.
// Convention: p is 0-indexed, p[i] is the image of
  ↪ i, (p . q)[i] = p[q[i]], p^k is p applied k
// times. All three routines are O(n) after the O(n)
  ↪ decomposition (the order pays a small
// factorisation on top).
vector<vector<int>> cycles(const vector<int>& p) {
  ↪       // each cycle as i, p[i], p[p[i]], ...
    int n = sz(p); vector<vector<int>> c;
  ↪ vector<char> vis(n, 0);
    for (int i = 0; i < n; i++) if (!vis[i]) {
        c.pb({});
        for (int j = i; !vis[j]; j = p[j]) vis[j] =
  ↪ 1, c.back().pb(j);
    }
    return c;
  ↪       // fixed points are cycles of length 1
}
vector<int> fromCycles(int n, const
  ↪ vector<vector<int>>& c) {
    vector<int> p(n);
    for (auto& v : c) for (int i = 0; i < sz(v); i++)
  ↪ p[v[i]] = v[(i + 1) % sz(v)];
    return p;
}
// --- K-TH ROOT: q with q^k = p, or {} if none
  ↪ exists. k >= 1 (k = 0 is not a question).
// DERIVATION. Raising to the k-th power splits a
  ↪ q-cycle of length m into gcd(m, k) cycles of
// length m / gcd(m, k) - so the q-cycle behind a
  ↪ group of p-cycles of length L must have length
// m = g*L where g is the group size, and it must
  ↪ satisfy g = gcd(gL, k). Divide that out: g = k
// is not required, the condition is exactly
//        g | k   AND   gcd(L, k / g) = 1.
// Let t = product over primes r dividing L of
  ↪ r^(exponent of r in k) - computed below by
// stripping gcd(L, .) off k until it is 1. Then k/t
  ↪ is coprime to L, every legal g is a multiple
// of t, and every g = t*z with z | (k/t) is legal.
  ↪ So the p-cycles of length L can be packed iff
// t divides their COUNT, and greedily taking groups
  ↪ of exactly t always works. Groups of size t
// also maximise the number of cycles of q, so this
  ↪ returns the root with the FEWEST
// transpositions. Existence therefore = "for every
  ↪ L, t(L, k) divides the number of cycles of
// length L". A cycle length that does not appear is
  ↪ vacuously fine.
// CONSTRUCTION: interleave the g cycles v_0..v_{g-1}
  ↪ into one array c of length m = g*L by
// c[(x + y*k) mod m] = v_x[y]. That is a bijection
  ↪ precisely because gcd(m, k) = g, and then
// stepping one place in c is q, so q^k moves v_x[y]
  ↪ to v_x[y+1] = p(v_x[y]).
vector<int> permRoot(const vector<int>& p, ll k) {
    int n = sz(p);
    vector<vector<vector<int>>> by(n + 1);
  ↪       // p-cycles bucketed by their length
    for (auto& v : cycles(p)) by[sz(v)].pb(v);
    vector<vector<int>> out;
    for (int L = 1; L <= n; L++) {
        if (by[L].empty()) continue;
        ll t = 1, rem = k, g;
        while ((g = gcd((ll)L, rem)) != 1) t *= g,
  ↪ rem /= g;         // t = minimal legal group
        if (sz(by[L]) % t) return {};
  ↪       // count not divisible: NO ROOT
        for (ll i = 0; i < sz(by[L]); i += t) {
            ll m = t * L, kk = k % m; vector<int>
  ↪ c(m);     // reduce k first: y*k can overflow
            for (ll x = 0; x < t; x++) for (ll y = 0;
  ↪ y < L; y++)
                c[(x + y * kk) % m] = by[L][i +
  ↪ x][y];      // the interleave
            out.pb(c);
        }
    }
    return fromCycles(n, out);
  ↪       // n == 0 returns {}, which is right
}
// --- MINIMUM TRANSPOSITIONS to sort p is n -
  ↪ (number of cycles), and no fewer: one swap
  ↪ changes
// the cycle count by exactly +-1 (inside a cycle it
  ↪ splits, across two it merges), and sorted
// means n cycles. To turn a into b it is n -
  ↪ cycles(b^-1 . a). Both are the SAME loop: drag
  ↪ the
// wanted value into place, which kills one cycle per
  ↪ swap.
vector<pii> swapsToMatch(vector<int> a, const
  ↪ vector<int>& b) {     // b = 0,1,..,n-1 means
  ↪ "sort"
    int n = sz(a); vector<int> pos(n); vector<pii> s;
    for (int i = 0; i < n; i++) pos[a[i]] = i;
  ↪       // pos[v] = where value v sits now
    for (int i = 0; i < n; i++) if (a[i] != b[i]) {
        int j = pos[b[i]];
  ↪       // j > i always, so one pass suffices
        s.pb({i, j}), pos[a[i]] = j, pos[a[j]] = i,
  ↪ swap(a[i], a[j]);
    }
    return s;
  ↪       // sz(s) == n - cycles(b^-1 . a)
}
// --- ORDER of p = lcm of the cycle lengths. It
  ↪ OVERFLOWS EARLY: the max over permutations of n
// (Landau's g(n) ~ e^sqrt(n ln n)) passes 1e18
  ↪ around n = 130, so never store it in an ll. Keep
// it factored, or reduce it mod M. Returns prime ->
  ↪ exponent; order mod M is then the product of
// q^e mod M over the map, and the exponents sum to
  ↪ at most log2(order).
map<ll, int> permOrder(const vector<int>& p) {
    map<ll, int> f;
    for (auto& v : cycles(p)) {
        int m = sz(v);
        for (ll q = 2; q * q <= m; q++) if (m % q ==
  ↪ 0) {
            int e = 0; while (m % q == 0) m /= q, e++;
            f[q] = max(f[q], e);
        }
        if (m > 1) f[m] = max(f[m], 1);
  ↪       // the leftover prime factor
    }
    return f;
  ↪       // identity / n == 0 gives {} = order 1
}
// VARIANTS / RELATED
// - p^k in O(n): inside each cycle of length L step
  ↪ by k, giving gcd(L, k) cycles of length
```

```
//   L / gcd(L, k). That is the root loop read backwards, and the cheap way to check a root.
// - p^k for huge k: reduce k modulo the cycle length of each element, not modulo the order.
// - SQUARE ROOT (k = 2): t = 2 for even L and 1 for odd L, so a root exists iff the cycles of
//   every even length come in pairs. Odd-length cycles always have a root alone.
// - Need a root with a prescribed PARITY of transposition count: once, merge 2t cycles of some
//   length instead of t (legal iff k/t is even and that length has >= 2t cycles). It changes the
//   cycle count by one, hence the parity.
// - MINIMUM ADJACENT swaps is a completely different quantity - the inversion count. See
//   14 - Inversions.cpp. Do not confuse it with n - cycles.
// - The permutation is even iff n - cycles is even; that is the sign, and it is what tells you
//   whether the 15-puzzle position is solvable.
```

1.28 SubarrayAggregator

LOGARITHMIC SUBARRAY AGGREGATOR - every value of op(a[l..r]) over every subarray, in O(n log A).

```
// Fix r. As l walks from r down to 0 the aggregate op(a[l..r]) only ever moves one way, and every
// strict move is a big jump, so only O(log A) DISTINCT values occur and equal ones form contiguous
// runs of l. subarrayAgg calls cb(r, runs) for every r (0-indexed) with runs = {v, lo, hi} meaning
// op(a[l..r]) == v for exactly the l in [lo, hi]; the runs partition [0, r] and are listed from
// l = r downwards, so runs[0].hi == r and runs.back().lo == 0.
// WHEN: "count / find the subarrays whose gcd (or / and / min) is X", "how many subarrays have
// gcd > 1", "longest subarray with or <= k". Replaces a sparse table plus a binary search per l,
// and it is shorter and exact.
// PRECONDITION for correctness: op associative, and op(x, y) monotone in the same direction for
// every x (so equal values are adjacent and the merge below is legal).
// PRECONDITION for the O(log A) run count: every STRICT change must shrink a potential of size A -
// gcd at least halves, and / or turn at least one bit off / on, lcm capped at LIM at least doubles.
// gcd, and, or, capped lcm all qualify. min and max are monotone so the code is CORRECT for them,
// but their run count is O(n), not O(log A) (a[i] = i gives n runs at r = n-1) - use a monotonic
// stack there. sum, xor and product do not qualify at all: nothing forces values to repeat.
// COMPLEXITY: O(n log A) time, O(log A) live memory plus whatever cb stores.
// DEGENERATE: empty a calls cb zero times. n == 1 gives one run {a[0], 0, 0}. All-equal gives one
// run per r. Zeros are fine for gcd (gcd(0, x) == x), so an all-zero array reports v == 0.
// EXACTNESS: exact on integers, the aggregate never leaves the value type. Counts of subarrays
// need ll (n(n+1)/2 overflows int at n = 65536), the run endpoints stay int.
template <class T> struct Run { T v; int lo, hi; };
// op(a[l..r]) == v for every l in [lo, hi]
template <class T, class Op, class F>
void subarrayAgg(const vector<T>& a, Op op, F cb) {
    vector<Run<T>> cur, nxt;
    for (int r = 0; r < sz(a); r++) {      // runs for r-1, then the ones for r
        nxt.clear();
        nxt.pb({a[r], r, r});
        for (auto& e : cur) {              // l == r: the element on its own
            T v = op(e.v, a[r]);           // associativity: op(a[l..r-1], a[r])
            if (v == nxt.back().v) nxt.back().lo = e.lo;   // unchanged: swallow the whole old run
            else nxt.pb({v, e.lo, e.hi});
        }                                  // changed: at most O(log A) times
        cur.swap(nxt);
        cb(r, cur);
    }
}
// --- CONSUMER 1: cnt[x] = number of subarrays whose gcd is exactly x. O(n log A (log A + log n)).
// std::gcd (C++17) and not __gcd: __gcd is a libstdc++ internal and rejects signed types on clang.
map<ll, ll> gcdHistogram(const vector<ll>& a) {
    map<ll, ll> cnt;
    subarrayAgg(a, [](ll x, ll y) { return gcd(x, y); }, [&](int, auto& v) {
        for (auto& e : v) cnt[e.v] += e.hi - e.lo + 1;
    });
    return cnt;
}
// --- CONSUMER 2: number of subarrays with or(a[l..r]) >= k, and the set of distinct subarray ORs
// (which has size O(n log A), not O(n^2) - that bound is the reason the question gets asked).
ll orAtLeast(const vector<ll>& a, ll k, set<ll>* distinct = nullptr) {
    ll c = 0;
    subarrayAgg(a, [](ll x, ll y) { return x | y; }, [&](int, auto& v) {
        for (auto& e : v) {                // v is sorted by value, so you could
            if (e.v >= k) c += e.hi - e.lo + 1;
            // binary search - but it has O(log A)
            if (distinct) distinct->insert(e.v);
            // entries, a scan is already free
        }
    });
    return c;
}
// VARIANTS
//  - LONGEST / SHORTEST subarray with a target property: the runs are sorted by l, so per r the
//    best l is an endpoint of the first / last run that satisfies it. O(n log A) overall.
//  - "gcd of a[l..r] == 1" for all pairs: gcdHistogram(a)[1].
//  - LCM needs a cap or it overflows instantly: g = gcd(x, y), then x / g > LIM / y ? LIM :
//    x / g * y. Capping keeps monotonicity, so the run structure survives.
//  - Two-pointer twin: for or / and / gcd the aggregate is monotone in l, so "count subarrays with
```

```
//    op == k" also falls to two pointers plus a
   ↪ sliding structure, but that needs undo; this does
//    not and generalises to any associative monotone
   ↪ op in the same 10 lines.
//  - The same list carried from the LEFT (fix l,
   ↪ grow r) works identically; pick the end your
//    query is indexed by.
```

1.29 MexOfAllSubarrays

MEX OF EVERY SUBARRAY - which values occur as mex(a[l..r]) over all O(n^2) subarrays, O(n log n).

```
// Input a[0..n-1] with a[i] >= 0. Output ok[0..n],
   ↪ ok[m] == 1 iff some NON-EMPTY subarray has mex
// exactly m. Nothing above n is reachable: mex m
   ↪ needs m distinct values, so m <= n.
// WHY IT IS ONLY O(n log n): m is reachable iff some
   ↪ MAXIMAL m-free block contains all of 0..m-1.
// If a block avoids m and holds 0..m-1 its mex is
   ↪ exactly m; if the maximal block misses some
// value < m then so does every sub-block of it, so
   ↪ testing the maximal blocks alone is complete.
// The blocks for m are the gaps between consecutive
   ↪ occurrences of m, so over all m there are only
// sum(cnt[m] + 1) <= 2n + 1 blocks to test. Each
   ↪ test is one range-mex query, answered offline
// sorted by right end: keep last[v] = the last
   ↪ position of v seen so far in a MIN segment tree
   ↪ over
// v, and mex(a[L..R]) is the leftmost v with last[v]
   ↪ < L - one descend, go left iff the left
// child's min is < L. That descend is the reusable
   ↪ half of this file.
// WHEN: Codeforces 1436E ("mex of the set of all
   ↪ subarray mexes"), "is m achievable", and any
// range-mex problem - the tree below answers those
   ↪ directly.
// COMPLEXITY: O(n log n) time, O(n) memory (4n tree
   ↪ nodes plus <= 2n + 1 buffered queries).
// DEGENERATE: n == 0 returns {0} (no subarray,
   ↪ nothing reachable). All-zero returns only ok[1].
// All-distinct 0..n-1 returns every ok[0..n].
   ↪ Duplicates are fine. Values > n are ignored -
   ↪ they
// can neither BE a mex (too big) nor block one - but
   ↪ they still occupy a position and split
// nothing, which is correct. Negative values are
   ↪ undefined input, mex assumes non-negative.
// EXACTNESS: pure integer index arithmetic, no
   ↪ overflow anywhere (positions fit int).
struct MexTree {                                    //
   ↪ min, over a range of VALUES, of "last seen at"
    int V; vector<int> t;                           //
   ↪ values 0..V; -1 = never seen
    MexTree(int V) : V(V), t(4 * (V + 1), -1) {}
    void see(int v, int p, int x, int l, int r) {
        if (l == r) { t[x] = p; return; }
        int m = (l + r) / 2;
        v <= m ? see(v, p, 2 * x, l, m) : see(v, p, 2
   ↪ * x + 1, m + 1, r);
        t[x] = min(t[2 * x], t[2 * x + 1]);
    }
    void see(int v, int p) { if (v <= V) see(v, p, 1,
   ↪ 0, V); }    // v > V cannot affect a mex <= V
    int mex(int L, int x, int l, int r) {
   ↪            // leftmost v with last[v] < L
        if (l == r) return l;
   ↪            // a leaf always exists: [L..R] has
        int m = (l + r) / 2;
   ↪            // <= V+1 elements so some value
        return t[2 * x] < L ? mex(L, 2 * x, l, m)
   ↪            // in 0..V must be missing
                            : mex(L, 2 * x + 1, m +
   ↪ 1, r);
    }
    int mex(int L) { return mex(L, 1, 0, V); }     //
   ↪ == mex(a[L..R]), R = last position fed to see
};
vector<char> allSubarrayMex(const vector<int>& a) {
    int n = sz(a);
    vector<char> ok(n + 1, 0);
    if (!n) return ok;
    vector<vi> pos(n + 1);
   ↪            // pos[m] = positions holding m
    for (int i = 0; i < n; i++) if (a[i] <= n)
   ↪ pos[a[i]].pb(i);
    vector<vector<pii>> qs(n);
   ↪            // at R: (L, m), "is mex[L..R] m?"
    for (int m = 0; m <= n; m++) {
   ↪            // the maximal m-free blocks
        int lo = 0;
        for (int p : pos[m]) { if (lo < p) qs[p -
   ↪ 1].pb({lo, m}); lo = p + 1; }
        if (lo < n) qs[n - 1].pb({lo, m});
    }
    MexTree T(n);
    for (int R = 0; R < n; R++) {
        T.see(a[R], R);
        for (auto& [L, m] : qs[R]) if (T.mex(L) == m)
   ↪ ok[m] = 1;  // block avoids m, so mex <= m
    }
    return ok;
}
// CF1436E wants the mex OF THAT SET: int r = 0;
   ↪ while (r <= n && ok[r]) r++;
// VARIANTS
//  - RANGE MEX, offline: drop the block machinery,
   ↪ bucket the real queries by R and run the same
//    sweep - T.mex(L) after feeding a[0..R] is
   ↪ mex(a[L..R]). O((n + q) log n).
//  - RANGE MEX, online: make the same "last
   ↪ occurrence" tree PERSISTENT, one version per R,
   ↪ and
//    descend in version R. O(log n) per query, O(n
   ↪ log n) memory. Merge-sort tree also works.
//  - "smallest missing value >= x in a range":
   ↪ identical descend restricted to values [x, V].
//  - MEX OF A MULTISET under insert/erase: segment
   ↪ tree over counts, descend to the leftmost zero.
//  - MEX OF THE WHOLE ARRAY is O(n) with a bucket
   ↪ array; never build a tree for that.
//  - mex(a[l..r]) is non-increasing in l for fixed
   ↪ r, and non-decreasing in r for fixed l - that
//    monotonicity is what lets a two-pointer answer
   ↪ "shortest subarray with mex >= k".
```

1.30 XorEquationRange

COUNT THE SOLUTIONS OF x_0 xor x_1 xor ... xor x_{n-1} == X with 0 <= x_i <= a[i], modulo M.

```
// Returns the count mod M. a[i] >= 0, X >= 0, M >= 1.
// WHEN: "how many tuples bounded componentwise xor
  ↪ to X" - Nim-position counting, POJ 3986, the
// stone-game family. The bounds are ARBITRARY; if
  ↪ every a[i] were 2^k - 1 the answer would just be
// 2^(k(n-1)) [X < 2^k], and this file is what you
  ↪ type when they are not.
// HOW: process bit k from the top. Split on whether
  ↪ every x_i copies a[i]'s bit k ("all tight") or
// at least one x_i drops a bit that a[i] has
  ↪ ("relaxed"). Relaxed is countable in one pass:
  ↪ the
// dropper is then free over ALL k lower bits, so
  ↪ designate the LOWEST-indexed dropper as the
// absorber - its lower bits are forced to whatever
  ↪ makes the xor come out right, factor 1 - and
// every other variable contributes its own
  ↪ independent count: a[i] - 2^k + 1 if it keeps
  ↪ bit k,
// 2^k if it drops it too, a[i] + 1 if a[i] has no
  ↪ bit k at all. A 4-state DP over
// (parity of chosen k-bits, has anyone dropped yet)
  ↪ does that in O(n) per bit. The all-tight case
// forces the k-th bit of the xor to the parity of
  ↪ #{i : a[i] has bit k}; if that misses X's bit k
// nothing survives, otherwise strip bit k everywhere
  ↪ and recurse to bit k-1. Bottoming out leaves
// exactly the single tuple x_i == a[i], hence the
  ↪ final +1.
// COMPLEXITY: O(n log(max a)) time, O(1) extra
  ↪ memory. No recursion, no globals.
// DEGENERATE: n == 0 returns [X == 0]. All a[i] == 0
  ↪ returns [X == 0]. X with a bit above every
// a[i] returns 0. X == 0 always returns >= 1 (the
  ↪ all-zero tuple). M == 1 returns 0.
// EXACTNESS: exact modulo M for any M up to 9.2e18 -
  ↪ every product goes through __int128. Bounds
// a[i] up to 9.2e18 are fine (a[i] + 1 must not
  ↪ overflow, so keep a[i] < 2^63 - 1).
ll xorEqCount(vector<ll> a, ll X, ll M) {
    auto mul = [&](ll p, ll q) { return
  ↪ (ll)((__int128)p * q % M); };
    int K = 0;
    for (ll v : a) while (v >> K) K++;
  ↪         // every a[i] < 2^K
    if (X >> K) return 0;
  ↪         // xor of values < 2^K stays < 2^K
    ll res = 0;
    for (int k = K - 1; k >= 0; k--) {
        ll two = (1LL << k) % M, dp[2][2] = {{1 % M,
  ↪ 0}, {0, 0}};   // dp[parity of bit k][dropped?]
        for (ll v : a) {
            ll nd[2][2] = {}, cnt = ((v & ~(1LL <<
  ↪ k)) + 1) % M;   // v - 2^k + 1 if bit k is set,
            for (int p = 0; p < 2; p++) for (int f =
  ↪ 0; f < 2; f++) {   // v + 1 if it is not: the
                ll c = dp[p][f];
  ↪                     // SAME expression
                if (!c) continue;
                if (v >> k & 1) {
                    nd[p ^ 1][f] = (nd[p ^ 1][f] +
  ↪ mul(c, cnt)) % M;    // keep it: x_i in [2^k, v]
                    nd[p][1] = (nd[p][1] + (f ?
  ↪ mul(c, two) : c)) % M;  // drop it: x_i in [0,
  ↪ 2^k),
                } else nd[p][f] = (nd[p][f] + mul(c,
  ↪ cnt)) % M;         // and the FIRST dropper is
            }
  ↪                   // the absorber: factor 1
            memcpy(dp, nd, sizeof dp);
        }
        res = (res + dp[X >> k & 1][1]) % M;
  ↪         // [1]: at least one variable dropped
        int par = 0;
        for (ll& v : a) if (v >> k & 1) par ^= 1, v
  ↪ ^= 1LL << k;        // strip, stay all-tight
        if (par != (int)(X >> k & 1)) return res;
  ↪                   // all-tight branch dies
    }
    return (res + 1) % M;
  ↪         // the surviving tuple is x_i == a[i]
}
// VARIANTS
//  - LOWER BOUNDS TOO (lo_i <= x_i <= hi_i): you
  ↪ cannot shift, xor is not linear under +. Split
//    each range into O(log) aligned blocks "fixed
  ↪ prefix + j free bits"; a block with j >= 1 free
//    bits plays the role of a dropper (one absorber,
  ↪ the rest contribute 2^j), so the same count
//    works over the O(n log A) blocks - at the cost
  ↪ of a product over per-variable block choices.
//  - AND / OR instead of xor: per bit the constraint
  ↪ decouples completely, so it is a plain
//    per-bit product, no absorber trick needed.
//  - Sum over all X of the answer is prod(a[i] + 1)
  ↪ - a free sanity check on the implementation.
//  - CONSTRUCTING one solution: run the same DP
  ↪ keeping the counts, then walk it back greedily.
//  - If all a[i] are equal to 2^k - 1 the answer is
  ↪ 2^(k(n-1)) for X < 2^k, 0 otherwise.
```

1.31 SubsetUnionOfBitsets

SUBSET UNION SIZES - given m <= ~20 sets over the ground set {0..n-1}, get |union of the chosen

```
// sets| for EVERY one of the 2^m subsets, in O(n +
  ↪ 2^m * m). Output u[S] for every mask S over the
// m sets, u[S] = number of ground elements covered
  ↪ by at least one set of S; u[0] == 0.
// THE TRICK IS THE TRANSPOSE, everything else is
  ↪ bookkeeping. You are handed "set j contains these
// elements"; flip it to own[e] = the mask of sets
  ↪ that contain element e. Then element e is MISSED
// by S iff S is disjoint from own[e] iff S is a
  ↪ SUBSET of ~own[e]. So drop one token at ~own[e]
  ↪ for
// each e and run a SUPERSET-sum (SOS) transform:
  ↪ cnt[S] = #{e : ~own[e] contains S} = number of
// elements S misses, and the union size is n -
  ↪ cnt[S]. Counting the complements is what turns a
// union (no closed form under subsets) into a plain
  ↪ subset-lattice sum.
// WHEN: "cheapest set of towers covering >= k
  ↪ houses", "for each mask, how much does it cover",
// any 2^m enumeration whose inner cost is a union -
  ↪ it removes the O(n / 64) bitset OR per mask.
// COMPLEXITY: O(n + 2^m * m) time, 2^m ints = 4 MB
  ↪ at m = 20 (plus 8 MB if you also build costs).
// DEGENERATE: m == 0 gives the single answer u[0] ==
  ↪ 0. n == 0 gives all zeros. Duplicate or empty
// sets are fine. An element in no set has own[e] ==
  ↪ 0, is missed by every S, and is never counted.
```

```
// EXACTNESS: pure integer counting, no overflow (u
   ↪ fits int as long as n does). Weighted version:
// replace the ++ by += w[e] and use ll - then the
   ↪ budget is sum|w| <= 9e18.
vi transposeSets(int n, const vector<vi>& sets) {
   ↪      // sets[j] = the elements of set j
    vi own(n, 0);
   ↪      // own[e] = mask of sets holding e
    for (int j = 0; j < sz(sets); j++) for (int e :
   ↪ sets[j]) own[e] |= 1 << j;
    return own;
}
vi subsetUnionSize(int n, int m, const vi& own) {
    int full = (1 << m) - 1;
    vi u(1 << m, 0);
    for (int e = 0; e < n; e++) u[own[e] ^ full]++;
   ↪      // bucket e at "the sets that MISS e"
    for (int b = 0; b < m; b++)
   ↪      // SOS over SUPERSETS: pull from S | bit
        for (int S = 0; S <= full; S++)
   ↪      // into S. Entries with bit b set are not
          if (!(S >> b & 1)) u[S] += u[S | 1 << b];
   ↪      // touched in this pass, so order is free
    for (int S = 0; S <= full; S++) u[S] = n - u[S];
   ↪      // missed -> covered
    return u;
}
// --- THE DUAL CONSUMER: cheapest subset covering AT
   ↪ LEAST k elements, for every k. O(2^m * m).
vector<ll> minCostToCover(int n, int m, const vi&
   ↪ own, const vector<ll>& cost) {
    vi u = subsetUnionSize(n, m, own);
    int full = (1 << m) - 1;
    vector<ll> c(1 << m, 0), best(n + 1, llinf);
    for (int S = 1; S <= full; S++) c[S] = c[S ^ (1
   ↪ << lsb(S))] + cost[lsb(S)];   // cost by lowest
    for (int S = 0; S <= full; S++) best[u[S]] =
   ↪ min(best[u[S]], c[S]);           // set bit
    for (int k = n - 1; k >= 0; k--) best[k] =
   ↪ min(best[k], best[k + 1]);   // exactly -> at
   ↪ least
    return best;
   ↪      // best[k] == llinf: k is unreachable
}
// VARIANTS
//  - INTERSECTION SIZES for every S: bucket at
   ↪ own[e] instead of ~own[e] and run the SAME
   ↪ superset
//    SOS - #{e : own[e] contains S} is exactly
   ↪ |intersection of S|. One character of difference.
//  - "covered by EXACTLY the sets in S": the raw
   ↪ histogram of own[e], no transform at all.
//  - MAX / MIN over the covered set instead of a
   ↪ count: SOS with max / min in place of +.
//  - SUBSET sum (Zeta over subsets) is the mirror:
   ↪ if (S >> b & 1) f[S] += f[S ^ 1 << b]. Mobius
//    (the inverse) is the same loop with -=, which
   ↪ recovers "exactly" counts from "at least".
//  - m > 20: meet in the middle over two halves of
   ↪ the sets, or drop to a greedy / randomised
//    cover - 2^m unions is the whole budget and it
   ↪ is 1e6 masks at m = 20.
//  - The ground set may be huge: only n and 2^m
   ↪ matter, never n * m or n / 64 words per mask.
```

1.32 NegativeBase

NEGATIVE-BASE ARITHMETIC. toNegBase(n, b) writes ANY integer n, either sign, in base -b with

```
// digits in [0, b): n == sum d[i] * (-b)^i, digits
   ↪ LEAST SIGNIFICANT FIRST, no leading zero, and
// zero is the single digit {0}. Every integer has
   ↪ exactly one such representation and there is no
// sign anywhere - that is the entire point of the
   ↪ base. fromNegBase inverts it.
// WHEN: "print n in base -2", "add two negabinary
   ↪ strings", puzzle problems where a value must be
// encoded without a sign, and the folklore fact that
   ↪ negabinary addition needs a NEGATIVE carry.
// THE DIVISION RULE: d = n mod b forced into [0, b),
   ↪ then n = (n - d) / (-b), an EXACT division
// since b divides n - d. Do not write n /= -b and
   ↪ patch d afterwards: C++ truncates toward zero,
// so after d += b the quotient also needs n += 1,
   ↪ and that is the step everyone forgets.
// WHY IT TERMINATES: |n - d| <= |n| + b - 1, so the
   ↪ new |n| is at most |n|/b + (b-1)/b, which is
// strictly smaller than |n| whenever |n| >= 2; from
   ↪ |n| == 1 it reaches 0 in at most two steps.
// The digit count is O(log_b |n|) + 1, and the sign
   ↪ of n alternates as you descend.
// ADDITION digit by digit: s = x[i] + y[i] + carry
   ↪ lands in [-b, 2b - 1]; force the digit into
// [0, b) by moving b, and since the place value of
   ↪ the next digit is -b, moving +b DOWN here means
// carrying -1 UP. A leftover carry of +1 appends the
   ↪ digit 1; a leftover of -1 appends TWO digits,
// b-1 then 1, because -1 == (b-1) + 1 * (-b). The
   ↪ same loop does subtraction (pass sgn = -1): the
// range of s only widens to [-b, b], still inside
   ↪ what the one carry line handles.
// COMPLEXITY: O(log_b |n|) per conversion, O(len)
   ↪ per add. DEGENERATE: n == 0 gives {0}; adding to
// or from {0} works; results are stripped of leading
   ↪ zeros so equality is plain vector ==.
// EXACTNESS: exact integers. Budget: |n| <= 9.2e18
   ↪ minus b for toNegBase (n - d must not overflow,
// so LLONG_MIN is out), and fromNegBase must be told
   ↪ digits whose value fits in ll.
vi toNegBase(ll n, int b) {                          //
   ↪ b >= 2; the base used is -b
    vi d;
    while (n) {
        int r = (int)(n % b);
        if (r < 0) r += b;                           //
   ↪ C++ % keeps the sign of n; digits must be >= 0
        d.pb(r);
        n = (n - r) / -b;                            //
   ↪ exact division, never a truncation
    }
    return d.empty() ? vi{0} : d;
}
ll fromNegBase(const vi& d, int b) {
    ll v = 0;
    for (int i = sz(d) - 1; i >= 0; i--) v = v * -b +
   ↪ d[i];       // Horner, top digit down
    return v;
}
vi negAdd(vi x, const vi& y, int b, int sgn = 1) {
   ↪           // x + sgn * y, sgn is 1 or -1
    x.resize(max(sz(x), sz(y)), 0);
    int c = 0;
    for (int i = 0; i < sz(x); i++) {
```

```
        int s = x[i] + (i < sz(y) ? sgn * y[i] : 0) +
  ↪ c;
        c = s < 0 ? 1 : (s >= b ? -1 : 0);          //
  ↪ next place is worth -b, so shifting the digit
        x[i] = s + b * c;                           //
  ↪ by -b here is a carry of +1, and vice versa
    }
    if (c == 1) x.pb(1);
    else if (c == -1) x.pb(b - 1), x.pb(1);         //
  ↪ -1 == (b-1) * 1 + 1 * (-b)
    while (sz(x) > 1 && !x.back()) x.pop_back();    //
  ↪ normalise, so == is equality of values
    return x;
}
// VARIANTS
//  - NEGATE: negAdd({0}, x, b, -1). Do not hunt for
  ↪ a bit pattern, subtraction already does it.
//  - SIGN: a normalised nonzero number is NEGATIVE
  ↪ iff its digit count is even, i.e. the leading
//    digit sits on an odd power of -b. Compare x and
  ↪ y by the sign of negAdd(x, y, b, -1).
//  - MULTIPLY BY -b: prepend a zero digit (shift).
  ↪ Divide by -b: drop d[0].
//  - MULTIPLY: schoolbook into a wide accumulator,
  ↪ then normalise from the bottom with the same
//    rule - while (d[i] < 0) d[i] += b, d[i+1] += 1;
  ↪ while (d[i] >= b) d[i] -= b, d[i+1] -= 1.
//  - NEGABINARY (b == 2) IN ONE LINE, no loop: the
  ↪ digits of n as a bit pattern are
//    (n + 0xAAAAAAAAAAAAAAAAull) ^
  ↪ 0xAAAAAAAAAAAAAAAAull, bit i being d[i] - the
  ↪ mask is the whole
//    carry cascade done in parallel. All 64 digits
  ↪ must fit, which covers every negative ll but
//    only n <= (4^32 - 1) / 3 = 6148914691236517205
  ↪ upward; past that it silently truncates.
//  - BASE -1+i represents every Gaussian integer
  ↪ with digits {0, 1}: take d = (re + im) & 1, then
//    (re, im) -> ((im - re + d) / 2, -(re + im - d)
  ↪ / 2). Same loop, same termination argument.
```

1.33 TestGenerators

RANDOM TEST GENERATORS for stress testing. 12 - StressTest.cpp is the harness; this is the

```
// library it feeds on. Writing a generator from
  ↪ scratch under pressure is where stress testing
// usually dies, so keep these on the page.
// SEED FROM argv SO A FAILING CASE IS REPRODUCIBLE:
//   int main(int argc, char** argv) { if (argc > 1)
  ↪ rng.seed(stoull(argv[1])); ... }
// then the driver loop passes the iteration number
  ↪ and you can replay any failure exactly.
ll rnd(ll l, ll r) { return
  ↪ uniform_int_distribution<ll>(l, r)(rng); }   //
  ↪ inclusive
bool coin(int pct = 50) { return rnd(1, 100) <= pct; }
vector<ll> randArray(int n, ll lo, ll hi) {
    vector<ll> a(n);
    for (ll& x : a) x = rnd(lo, hi);
    return a;
}
vector<int> randPermutation(int n) {
  ↪          // 0-indexed
    vector<int> p(n);
    iota(all(p), 0);
    shuffle(all(p), rng);
    return p;
}
string randString(int n, char lo = 'a', char hi =
  ↪ 'z') {
    string s;
    while (n--) s += (char)rnd(lo, hi);
    return s;
}
vector<string> randGrid(int n, int m, const string&
  ↪ alpha = ".#") {
    vector<string> g(n, string(m, '.'));
    for (auto& row : g) for (char& c : row) c =
  ↪ alpha[rnd(0, alpha.size() - 1)];
    return g;
}
// --- TREES. All 1-indexed, n-1 edges, always
  ↪ connected. ---
vector<pair<int, int>> randTree(int n) {
  ↪          // uniform-ish, shuffled labels
    vector<int> lab = randPermutation(n);
    vector<pair<int, int>> e;
    for (int i = 1; i < n; i++)
  ↪ e.push_back({lab[rnd(0, i - 1)] + 1, lab[i] +
  ↪ 1});
    return e;
}
vector<pair<int, int>> pathTree(int n) {
  ↪          // the DEEP case: recursion, HLD
    vector<pair<int, int>> e;
    for (int i = 2; i <= n; i++) e.push_back({i - 1,
  ↪ i});
    return e;
}
vector<pair<int, int>> starTree(int n) {
  ↪          // the WIDE case: heavy-child logic
    vector<pair<int, int>> e;
    int c = rnd(1, n);
    for (int i = 1; i <= n; i++) if (i != c)
  ↪ e.push_back({c, i});
    return e;
}
vector<pair<int, int>> caterpillar(int n) {
  ↪          // a spine with legs: breaks naive
    vector<pair<int, int>> e;
  ↪          // centroid and diameter code
    int spine = max(1LL, (ll)rnd(1, n));
    for (int i = 2; i <= spine; i++) e.push_back({i -
  ↪ 1, i});
    for (int i = spine + 1; i <= n; i++)
  ↪ e.push_back({(int)rnd(1, spine), i});
    return e;
}
// --- GRAPHS. m edges, no self loops, no duplicates
  ↪ unless you ask for them. ---
vector<pair<int, int>> randGraph(int n, int m) {
    vector<pair<int, int>> all;
    for (int i = 1; i <= n; i++) for (int j = i + 1;
  ↪ j <= n; j++) all.push_back({i, j});
    shuffle(all(all), rng);
    all.resize(min((size_t)m, all.size()));
    return all;
}
vector<pair<int, int>> connectedGraph(int n, int m) {
  ↪          // a random tree plus extra edges
    auto e = randTree(n);
    set<pair<int, int>> used;
    for (auto [u, v] : e) used.insert({min(u, v),
  ↪ max(u, v)});
    m = min<ll>(m, (ll)n * (n - 1) / 2);
```

```
    while ((int)e.size() < m) {
        int u = rnd(1, n), v = rnd(1, n);
        if (u == v) continue;
        if (u > v) swap(u, v);
        if (used.insert({u, v}).second)
  ↪ e.push_back({u, v});
    }
    shuffle(all(e), rng);
    return e;
}
vector<pair<int, int>> randDAG(int n, int m) {
  ↪           // acyclic by a random topo order
    vector<int> p = randPermutation(n);
    vector<pair<int, int>> all;
    for (int i = 0; i < n; i++) for (int j = i + 1; j
  ↪ < n; j++) all.push_back({p[i] + 1, p[j] + 1});
    shuffle(all(all), rng);
    all.resize(min((size_t)m, all.size()));
    return all;
}
vector<pair<int, int>> randQueries(int q, int n) {
  ↪           // random inclusive [l, r]
    vector<pair<int, int>> v;
    while (q--) {
        int l = rnd(1, n), r = rnd(1, n);
        if (l > r) swap(l, r);
        v.push_back({l, r});
    }
    return v;
}
/* THE CASES A UNIFORM GENERATOR NEVER PRODUCES - and
  ↪ which is exactly where the bug is
   n = 1, n = 2, and an EMPTY structure
  ↪       (off-by-one, .back() on empty)
   ALL VALUES EQUAL, all zero, all negative
  ↪       (tie-breaks, strict vs non-strict)
   values at the EXTREMES of the stated range
  ↪       (overflow)
   a SORTED or REVERSE-SORTED array
  ↪       (quicksort, two pointers, hulls)
   a PATH tree and a STAR tree
  ↪       (recursion depth, heavy-child logic)
   a DISCONNECTED graph, an isolated vertex, a self
  ↪ loop, a DOUBLED edge   (bridges, DSU, flows)
   COLLINEAR / DUPLICATE points, a degenerate polygon
  ↪       (all of geometry)
   Generate with a SMALL value range (say [1,3]) most
  ↪ of the time: collisions and ties are what
   break code, and a uniform draw from [1, 1e9] never
  ↪ creates one.
   SHRINK a failure before reading it: re-run the
  ↪ generator with progressively smaller bounds
   until the counterexample fits on one line, then
  ↪ stare at that. */
```

2 | Data Structures (DS)

2.1 Segment Tree

2.1.1 SegmentTree

SEGMENT TREE, point update + range query, O(log n) each. Swap the merge in SEG to change the

```
// operation (sum/min/max/gcd/matrix); any
  ↪ associative monoid works. Size is rounded up to
  ↪ a power
// of two so the tree is perfect and indices are
  ↪ simple.
struct SEG {
    ll sum = 0;
    SEG() {
    }
    SEG(ll x){
        sum = x;
    }
};
struct segTree {
    vector<SEG> seg;
    int sz = 1,n;
    segTree(int nn){
        n = nn;
        while (sz < nn)
            sz *= 2;
        seg.assign(2 * sz , SEG());
    }

    SEG merge(SEG seg1, SEG seg2) {
        SEG ret;
        ret.sum = seg1.sum + seg2.sum;
        return ret;
    }

    void build(vector<int> &v,int x, int lx, int rx) {
        if (lx == rx) {
            seg[x] = SEG(v[lx]);
            return;
        }
        int mid = (lx + rx) / 2;
        int lf = 2 * x + 1, rt = 2 * x + 2;

        build(v,lf, lx, mid);
        build(v,rt, mid + 1, rx);
        seg[x] = merge(seg[lf], seg[rt]);
    }
    void build(vector<int> &v){
        build(v,0, 0, n-1);
    }

    void update(int i, ll val, int x, int lx, int rx)
  ↪ {
```

```
        if (lx == rx) {
            seg[x] = SEG(val);
            return;
        }
        int mid = (lx + rx) / 2,lf = 2*x+1,rt= 2*x+2;
        if(i <= mid)
            update(i, val, lf, lx, mid);
        else
            update(i, val, rt, mid + 1, rx);
        seg[x] = merge(seg[lf] , seg[rt]);
    }

    void update(int i, ll val) {
        update(i, val, 0, 0, n-1);
    }

    SEG query(int l, int r, int x, int lx, int rx) {
        if (l <= lx && r >= rx)
            return seg[x];
        if (l > rx || r < lx)
            return SEG();

        int mid = (lx + rx) / 2,lf = 2 * x + 1,rt =
    ↪ 2*x+2;

        return merge(query(l, r, lf, lx, mid) ,
    ↪ query(l, r, rt, mid + 1, rx));
    }

    SEG query(int l, int r) {
        return query(l, r, 0, 0, n-1);
    }
};
```

2.1.2 LazySegmentTree

Lazy Segment Tree (range add, range sum) - build O(n), update/query O(log n)

```
// 1-based node indices, 0-based positions, ALL
  ↪ recursions on [0, n-1].
// To change the op: edit merge(), apply() and the
  ↪ identity in qry().
struct LazySeg {
    int n;
    vector<ll> t, lz;

    LazySeg(int n = 0) : n(n), t(4 * n, 0), lz(4 * n,
    ↪ 0) {}

    ll merge(ll a, ll b) { return a + b; }
    void apply(int v, int l, int r, ll x) { t[v] += x
    ↪ * (r - l + 1); lz[v] += x; }

    void push(int v, int l, int r) {
        if (!lz[v]) return;
        int m = (l + r) / 2;
        apply(2 * v, l, m, lz[v]);
        apply(2 * v + 1, m + 1, r, lz[v]);
        lz[v] = 0;
    }

    void build(const vector<ll>& a, int v, int l, int
    ↪ r) {
        if (l == r) { t[v] = a[l]; return; }
        int m = (l + r) / 2;
        build(a, 2 * v, l, m), build(a, 2 * v + 1, m
    ↪ + 1, r);
        t[v] = merge(t[2 * v], t[2 * v + 1]);
    }
    void build(const vector<ll>& a) { if (n) build(a,
    ↪ 1, 0, n - 1); }

    void upd(int ql, int qr, ll x, int v, int l, int
    ↪ r) {
        if (qr < l || r < ql) return;
        if (ql <= l && r <= qr) return apply(v, l, r,
    ↪ x);
        push(v, l, r);
        int m = (l + r) / 2;
        upd(ql, qr, x, 2 * v, l, m), upd(ql, qr, x, 2
    ↪ * v + 1, m + 1, r);
        t[v] = merge(t[2 * v], t[2 * v + 1]);
    }
    void upd(int l, int r, ll x) { upd(l, r, x, 1, 0,
    ↪ n - 1); }

    ll qry(int ql, int qr, int v, int l, int r) {
        if (qr < l || r < ql) return 0;
    ↪   // identity
        if (ql <= l && r <= qr) return t[v];
        push(v, l, r);
        int m = (l + r) / 2;
        return merge(qry(ql, qr, 2 * v, l, m),
    ↪ qry(ql, qr, 2 * v + 1, m + 1, r));
    }
    ll qry(int l, int r) { return qry(l, r, 1, 0, n -
    ↪ 1); }
};
```

2.1.3 PersistentSegmentTree

PERSISTENT SEGMENT TREE - every update returns a NEW root and old versions stay queryable.

```
// O(log n) time and O(log n) new nodes per update.
  ↪ The standard use is offline k-th smallest in a
// range (build a version per prefix, then query
  ↪ version[r] minus version[l-1]).
struct Node {
    Node *l, *r;
    ll val = 0;

    Node(ll val) : l(NULL), r(NULL), val(val) {
    }

    Node() : l(NULL), r(NULL) {
    }

    Node(Node* l, Node* r) : l(l), r(r) {
        if (l != NULL) val += l->val;
        if (r != NULL) val += r->val;
    }

    void addChild() {
        l = new Node();
        r = new Node();
    }
};

struct PST {
    int n;

    PST(int n) : n(n + 1) {
    }

    Node merge(Node x, Node y) {
        Node ret;
        ret.val = x.val + y.val;
        return ret;
```

```
    }

    Node* update(Node* v, int i, ll val, int lx, int
  ↪ rx) {
        if (lx == rx) return new Node(v->val + val);
        int mid = (lx + rx) / 2;
        if (!v->l) v->addChild();
        if (i <= mid) return new Node(update(v->l, i,
  ↪ val, lx, mid), v->r);
        return new Node(v->l, update(v->r, i, val,
  ↪ mid + 1, rx));
    }

    Node* update(Node* v, int i, ll val) { return
  ↪ update(v, i, val, 0, n - 1); }

    Node query(Node* v, int l, int r, int lx, int rx)
  ↪ {
        if (l > rx || r < lx) return {};
        if (l <= lx && r >= rx) return *v;
        if (!v->l) v->addChild();
        int mid = (lx + rx) / 2;
        return merge(query(v->l, l, r, lx, mid),
  ↪ query(v->r, l, r, mid + 1, rx));
    }

    Node query(Node* v, int l, int r) { return
  ↪ query(v, l, r, 0, n - 1); }

    int getKth(Node* a, Node* b, int k, int lx, int
  ↪ rx) {
        if (lx == rx) return lx;
        if (!a->l) a->addChild();
        if (!b->l) b->addChild();
        int rem = b->l->val - a->l->val;
        int mid = (lx + rx) / 2;
        if (rem >= k) return getKth(a->l, b->l, k,
  ↪ lx, mid);
        return getKth(a->r, b->r, k - rem, mid + 1,
  ↪ rx);
    }

    int getKth(Node* a, Node* b, int k) { return
  ↪ getKth(a, b, k, 0, n - 1); }
};
```

2.1.4 SegmentTreeBeats

Rename one if you ever paste two of them together.

```
// NOTE: several segment-tree files here declare
  ↪ their own `struct Node`.
//       Rename one if you ever paste two of them
  ↪ together.
struct Node {
    ll mn;
    int mnCnt;
    ll secondMn;

    ll mx;
    int mxCnt;
    ll secondMx;

    ll sum;

    ll lazyAdd;
    ll lazyEqual;
    bool isLazyEqual;

    Node() {
        mx = -inf;
        secondMx = -inf;
        mxCnt = 0;

        mn = inf;
        secondMn = inf;
        mnCnt = 0;

        sum = 0;

        lazyAdd = 0;
        lazyEqual = 0;
        isLazyEqual = 0;
    };

    Node(int x) {
        mx = x;
        secondMx = -inf;
        mxCnt = 1;

        mn = x;
        secondMn = inf;
        mnCnt = 1;

        sum = x;

        lazyAdd = 0;
        lazyEqual = 0;
        isLazyEqual = 0;
    }
};

struct SegTree {
    int tree_size;
    vector<Node> seg_data;

    SegTree(int n) {
        tree_size = 1;
        while (tree_size < n) tree_size *= 2;
        seg_data.resize(2 * tree_size, Node());
    }

    Node merge(Node& lf, Node& ri) {
        Node ans = Node();

        ans.sum = lf.sum + ri.sum;

        ans.mx = max(lf.mx, ri.mx);
        ans.secondMx = max(lf.secondMx, ri.secondMx);
        ans.mxCnt = 0;
        if (lf.mx == ans.mx) ans.mxCnt += lf.mxCnt;
        else ans.secondMx = max(ans.secondMx, lf.mx);
        if (ri.mx == ans.mx) ans.mxCnt += ri.mxCnt;
        else ans.secondMx = max(ans.secondMx, ri.mx);

        ans.mn = min(lf.mn, ri.mn);
        ans.secondMn = min(lf.secondMn, ri.secondMn);
        ans.mnCnt = 0;
        if (lf.mn == ans.mn) ans.mnCnt += lf.mnCnt;
        else ans.secondMn = min(ans.secondMn, lf.mn);
        if (ri.mn == ans.mn) ans.mnCnt += ri.mnCnt;
        else ans.secondMn = min(ans.secondMn, ri.mn);

        return ans;
    }

    void init(vector<int>& nums, int ni, int lx, int
  ↪ rx) {
```

```
    if (rx - lx == 1) {
        if (lx < nums.size())
            seg_data[ni] = Node(nums[lx]);
        return;
    }

    int mid = lx + (rx - lx) / 2;
    init(nums, 2 * ni + 1, lx, mid);
    init(nums, 2 * ni + 2, mid, rx);

    seg_data[ni] = merge(seg_data[2 * ni + 1],
↪ seg_data[2 * ni + 2]);
}

void init(vector<int>& nums) {
    init(nums, 0, 0, tree_size);
}

void doPushEq(int ni, int lx, int rx, int x) {
    seg_data[ni].mn = seg_data[ni].mx =
↪ seg_data[ni].lazyEqual = x;
    seg_data[ni].isLazyEqual = 1;
    seg_data[ni].mnCnt = seg_data[ni].mxCnt = rx
↪ - lx;
    seg_data[ni].secondMx = -inf;
    seg_data[ni].secondMn = inf;

    seg_data[ni].sum = (ll)x * (rx - lx);
    seg_data[ni].lazyAdd = 0;
}

void doPushMinEq(int ni, int lx, int rx, int x) {
    if (seg_data[ni].mn >= x) doPushEq(ni, lx,
↪ rx, x);
    else if (seg_data[ni].mx > x) {
        if (seg_data[ni].secondMn ==
↪ seg_data[ni].mx)
            seg_data[ni].secondMn = x;
        seg_data[ni].sum -= (ll)(seg_data[ni].mx
↪ - x) * seg_data[ni].mxCnt;
        seg_data[ni].mx = x;
    }
}

void doPushMaxEq(int ni, int lx, int rx, int x) {
    if (seg_data[ni].mx <= x) doPushEq(ni, lx,
↪ rx, x);
    else if (seg_data[ni].mn < x) {
        if (seg_data[ni].secondMx ==
↪ seg_data[ni].mn)
            seg_data[ni].secondMx = x;

        seg_data[ni].sum += (ll)(x -
↪ seg_data[ni].mn) * seg_data[ni].mnCnt;
        seg_data[ni].mn = x;
    }
}

void doPushSum(int ni, int lx, int rx, int x) {
    if (seg_data[ni].mn == seg_data[ni].mx)
        doPushEq(ni, lx, rx, seg_data[ni].mn + x);
    else {
        seg_data[ni].mx += x;
        if (seg_data[ni].secondMx != -inf)
            seg_data[ni].secondMx += x;

        seg_data[ni].mn += x;
        if (seg_data[ni].secondMn != inf)
            seg_data[ni].secondMn += x;

        seg_data[ni].sum += (ll)(rx - lx) * x;
        seg_data[ni].lazyAdd += x;
    }
}

void propagete(int ni, int lx, int rx) {
    if (rx - lx == 1) return;

    int mid = lx + (rx - lx) / 2;
    if (seg_data[ni].isLazyEqual) {
        doPushEq(2 * ni + 1, lx, mid,
↪ seg_data[ni].lazyEqual);
        doPushEq(2 * ni + 2, mid, rx,
↪ seg_data[ni].lazyEqual);
        seg_data[ni].lazyEqual =
↪ seg_data[ni].isLazyEqual = 0;
    }
    else {
        doPushSum(2 * ni + 1, lx, mid,
↪ seg_data[ni].lazyAdd);
        doPushSum(2 * ni + 2, mid, rx,
↪ seg_data[ni].lazyAdd);
        seg_data[ni].lazyAdd = 0;

        doPushMinEq(2 * ni + 1, lx, mid,
↪ seg_data[ni].mx);
        doPushMinEq(2 * ni + 2, mid, rx,
↪ seg_data[ni].mx);

        doPushMaxEq(2 * ni + 1, lx, mid,
↪ seg_data[ni].mn);
        doPushMaxEq(2 * ni + 2, mid, rx,
↪ seg_data[ni].mn);
    }
}

void minimize(int l, int r, int x, int ni, int
↪ lx, int rx) {
    if (rx <= l || lx >= r || seg_data[ni].mx <=
↪ x)
        return;
    if (lx >= l && rx <= r &&
↪ seg_data[ni].secondMx < x) {
        doPushMinEq(ni, lx, rx, x);
        return;
    }

    propagete(ni, lx, rx);

    int mid = lx + (rx - lx) / 2;
    minimize(l, r, x, 2 * ni + 1, lx, mid);
    minimize(l, r, x, 2 * ni + 2, mid, rx);

    seg_data[ni] = merge(seg_data[2 * ni + 1],
↪ seg_data[2 * ni + 2]);
}

void minimize(int l, int r, int x) {
    minimize(l, r, x, 0, 0, tree_size);
}

void maximize(int l, int r, int x, int ni, int
↪ lx, int rx) {
    if (rx <= l || lx >= r || seg_data[ni].mn >=
↪ x)
        return;
```

```
        if (lx >= l && rx <= r &&
↪ seg_data[ni].secondMn > x) {
            doPushMaxEq(ni, lx, rx, x);
            return;
        }

        propagete(ni, lx, rx);

        int mid = lx + (rx - lx) / 2;
        maximize(l, r, x, 2 * ni + 1, lx, mid);
        maximize(l, r, x, 2 * ni + 2, mid, rx);

        seg_data[ni] = merge(seg_data[2 * ni + 1],
↪ seg_data[2 * ni + 2]);
    }

    void maximize(int l, int r, int x) {
        maximize(l, r, x, 0, 0, tree_size);
    }

    void add(int l, int r, int x, int ni, int lx, int
↪ rx) {
        if (rx <= l || lx >= r)
            return;
        if (lx >= l && rx <= r) {
            doPushSum(ni, lx, rx, x);
            return;
        }

        propagete(ni, lx, rx);

        int mid = lx + (rx - lx) / 2;
        add(l, r, x, 2 * ni + 1, lx, mid);
        add(l, r, x, 2 * ni + 2, mid, rx);

        seg_data[ni] = merge(seg_data[2 * ni + 1],
↪ seg_data[2 * ni + 2]);
    }

    void add(int l, int r, int x) {
        add(l, r, x, 0, 0, tree_size);
    }

    void set(int l, int r, int x, int ni, int lx, int
↪ rx) {
        if (rx <= l || lx >= r)
            return;
        if (lx >= l && rx <= r) {
            doPushEq(ni, lx, rx, x);
            return;
        }

        propagete(ni, lx, rx);

        int mid = lx + (rx - lx) / 2;
        set(l, r, x, 2 * ni + 1, lx, mid);
        set(l, r, x, 2 * ni + 2, mid, rx);

        seg_data[ni] = merge(seg_data[2 * ni + 1],
↪ seg_data[2 * ni + 2]);
    }

    void set(int l, int r, int x) {
        set(l, r, x, 0, 0, tree_size);
    }

    Node get_range(int l, int r, int ni, int lx, int
↪ rx) {
        propagete(ni, lx, rx);

        if (lx >= l && rx <= r)
            return seg_data[ni];
        if (rx <= l || lx >= r)
            return Node();

        int mid = (lx + rx) / 2;

        Node lf = get_range(l, r, 2 * ni + 1, lx,
↪ mid);
        Node ri = get_range(l, r, 2 * ni + 2, mid,
↪ rx);
        return merge(lf, ri);
    }

    ll get_sum(int l, int r) {
        return get_range(l, r, 0, 0, tree_size).sum;
    }

    int get_mx(int l, int r) {
        return get_range(l, r, 0, 0, tree_size).mx;
    }

    int get_mn(int l, int r) {
        return get_range(l, r, 0, 0, tree_size).mn;
    }
};
```

2.1.5 DynamicSegTree

Dynamic (Implicit) Segment Tree - Space: O(Q log N) per query

```
// Allocates nodes on demand for ranges up to 1e9 or
↪ 1e18
template<typename T = ll>
struct DynamicSegTree {
    struct Node {
        T val = 0;
        int lc = -1, rc = -1;
    };

    vector<Node> tree;
    ll L, R;

    DynamicSegTree(ll L = 0, ll R = 1e9) : L(L), R(R)
↪ {
        add_node();
    }

    int add_node() {
        tree.pb(Node());
        return tree.size() - 1;
    }

    void update(int node, ll l, ll r, ll pos, T val) {
        tree[node].val += val;
        if (l == r) return;
        ll mid = l + (r - l) / 2;
        if (pos <= mid) {
            if (tree[node].lc == -1) {
                int nxt = add_node();
                tree[node].lc = nxt;
            }
            update(tree[node].lc, l, mid, pos, val);
        } else {
            if (tree[node].rc == -1) {
                int nxt = add_node();
                tree[node].rc = nxt;
            }
```

```
            update(tree[node].rc, mid + 1, r, pos,
    ↪ val);
        }
    }

    void update(ll pos, T val) {
        update(0, L, R, pos, val);
    }

    T query(int node, ll l, ll r, ll ql, ll qr) {
        if (node == -1 || ql > r || qr < l) return 0;
        if (ql <= l && r <= qr) return tree[node].val;
        ll mid = l + (r - l) / 2;
        return query(tree[node].lc, l, mid, ql, qr) +
    ↪ query(tree[node].rc, mid + 1, r, ql, qr);
    }

    T query(ll ql, ll qr) {
        return query(0, L, R, ql, qr);
    }
};
```

2.1.6 MergeSortTree

Merge Sort Tree - Space: O(N log N), Build: O(N log N), Query: O(log^2 N)

```
// Stores sorted elements in each node to answer 2D
    ↪ range count queries
template<typename T = int>
struct MergeSortTree {
    int n;
    vector<vector<T>> tree;

    MergeSortTree(const vector<T>& a) : n(a.size()),
    ↪ tree(4 * n) {
        build(1, 0, n - 1, a);
    }

    void build(int node, int l, int r, const
    ↪ vector<T>& a) {
        if (l == r) {
            tree[node] = {a[l]};
            return;
        }
        int mid = (l + r) / 2;
        build(2 * node, l, mid, a);
        build(2 * node + 1, mid + 1, r, a);
        merge(all(tree[2 * node]), all(tree[2 * node
    ↪ + 1]), back_inserter(tree[node]));
    }

    // Returns number of elements <= k in index range
    ↪ [ql, qr]
    int query_count_le(int node, int l, int r, int
    ↪ ql, int qr, T k) {
        if (ql > r || qr < l) return 0;
        if (ql <= l && r <= qr) {
            return upper_bound(all(tree[node]), k) -
    ↪ tree[node].begin();
        }
        int mid = (l + r) / 2;
        return query_count_le(2 * node, l, mid, ql,
    ↪ qr, k) + query_count_le(2 * node + 1, mid + 1,
    ↪ r, ql, qr, k);
    }

    int query_count_le(int ql, int qr, T k) {
        return query_count_le(1, 0, n - 1, ql, qr, k);
    }
};
```

2.1.7 IterativeSegTree

Iterative (Non-Recursive) Segment Tree - Space: O(N), Update: O(log N), Query: O(log N)

```
// Fast, cache-friendly implementation for point
    ↪ update and range query
template<typename T = ll>
struct IterativeSegTree {
    int n;
    vector<T> tree;
    T neutral;
    function<T(T, T)> combine;

    IterativeSegTree(int n = 0, T neutral = 0,
    ↪ function<T(T, T)> combine = [](T a, T b) {
    ↪ return a + b; })
        : n(n), tree(2 * n, neutral),
    ↪ neutral(neutral), combine(combine) {}

    IterativeSegTree(const vector<T>& a, T neutral =
    ↪ 0, function<T(T, T)> combine = [](T a, T b) {
    ↪ return a + b; })
        : n(a.size()), tree(2 * a.size(), neutral),
    ↪ neutral(neutral), combine(combine) {
        for (int i = 0; i < n; i++) tree[n + i] =
    ↪ a[i];
        for (int i = n - 1; i > 0; i--) tree[i] =
    ↪ combine(tree[i << 1], tree[i << 1 | 1]);
    }

    void update(int p, T val) {
        // combine(LEFT, RIGHT) - do NOT write
    ↪ combine(tree[p], tree[p ^ 1]): when p is the
    ↪ right
        // child that reverses the arguments, which
    ↪ is wrong for any non-commutative combine
        // (matrix product, affine composition,
    ↪ "merge two structs").
        for (tree[p += n] = val; p > 1; p >>= 1)
            tree[p >> 1] = combine(tree[p & ~1],
    ↪ tree[p | 1]);
    }

    // Range query on [l, r] inclusive
    T query(int l, int r) {
        T resl = neutral, resr = neutral;
        for (l += n, r += n + 1; l < r; l >>= 1, r
    ↪ >>= 1) {
            if (l & 1) resl = combine(resl,
    ↪ tree[l++]);
            if (r & 1) resr = combine(tree[--r],
    ↪ resr);
        }
        return combine(resl, resr);
    }
};
```

2.1.8 SegTree2D

2D Segment Tree - Space: O(N * M), Update: O(log N * log M), Query: O(log N * log M)

```
// Supports point updates and 2D subgrid range sum queries
template<typename T = ll>
struct SegTree2D {
    int n, m;
    vector<vector<T>> tree;

    SegTree2D(int n = 0, int m = 0) : n(n), m(m), tree(4 * n, vector<T>(4 * m, 0)) {}

    void update_y(int vx, int lx, int rx, int vy, int ly, int ry, int x, int y, T val) {
        if (ly == ry) {
            if (lx == rx) tree[vx][vy] += val;
            else tree[vx][vy] = tree[2 * vx][vy] + tree[2 * vx + 1][vy];
            return;
        }
        int my = (ly + ry) / 2;
        if (y <= my) update_y(vx, lx, rx, 2 * vy, ly, my, x, y, val);
        else update_y(vx, lx, rx, 2 * vy + 1, my + 1, ry, x, y, val);
        tree[vx][vy] = tree[vx][2 * vy] + tree[vx][2 * vy + 1];
    }

    void update_x(int vx, int lx, int rx, int x, int y, T val) {
        if (lx != rx) {
            int mx = (lx + rx) / 2;
            if (x <= mx) update_x(2 * vx, lx, mx, x, y, val);
            else update_x(2 * vx + 1, mx + 1, rx, x, y, val);
        }
        update_y(vx, lx, rx, 1, 0, m - 1, x, y, val);
    }

    void add(int x, int y, T val) {
        update_x(1, 0, n - 1, x, y, val);
    }

    T query_y(int vx, int vy, int ly, int ry, int qy1, int qy2) {
        if (qy1 > ry || qy2 < ly) return 0;
        if (qy1 <= ly && ry <= qy2) return tree[vx][vy];
        int my = (ly + ry) / 2;
        return query_y(vx, 2 * vy, ly, my, qy1, qy2) + query_y(vx, 2 * vy + 1, my + 1, ry, qy1, qy2);
    }

    T query_x(int vx, int lx, int rx, int qx1, int qx2, int qy1, int qy2) {
        if (qx1 > rx || qx2 < lx) return 0;
        if (qx1 <= lx && rx <= qx2) return query_y(vx, 1, 0, m - 1, qy1, qy2);
        int mx = (lx + rx) / 2;
        return query_x(2 * vx, lx, mx, qx1, qx2, qy1, qy2) + query_x(2 * vx + 1, mx + 1, rx, qx1, qx2, qy1, qy2);
    }

    T query(int x1, int y1, int x2, int y2) {
        return query_x(1, 0, n - 1, x1, x2, y1, y2);
    }
};
```

2.1.9 MergeSortTreeFractional

MERGE SORT TREE WITH FRACTIONAL CASCADING - static "how many values <= x in [l,r]" in O(log n)

```
// instead of O(log^2 n): each node stores its sorted values plus pointers into the children, so the
// binary search happens ONCE at the root and is then followed down for free.
const int MAXN = 30005;
int a[MAXN];

struct Node {
    vector<int> vals;
    vector<int> to_left;
    vector<int> to_right;
} tree[4 * MAXN];

void build(int node, int start, int end) {
    if (start == end) {
        tree[node].vals.push_back(a[start]);
        tree[node].to_left.assign(2, 0);
        tree[node].to_right.assign(2, 0);
        return;
    }

    int mid = (start + end) / 2;
    int left_child = 2 * node;
    int right_child = 2 * node + 1;

    build(left_child, start, mid);
    build(right_child, mid + 1, end);

    int left_sz = tree[left_child].vals.size();
    int right_sz = tree[right_child].vals.size();
    int total_sz = left_sz + right_sz;

    tree[node].vals.resize(total_sz);
    tree[node].to_left.resize(total_sz + 1);
    tree[node].to_right.resize(total_sz + 1);

    int p1 = 0, p2 = 0;

    for (int i = 0; i < total_sz; i++) {
        tree[node].to_left[i] = p1;
        tree[node].to_right[i] = p2;

        if (p1 < left_sz && (p2 == right_sz || tree[left_child].vals[p1] <= tree[right_child].vals[p2])) {
            tree[node].vals[i] = tree[left_child].vals[p1++];
        } else {
            tree[node].vals[i] = tree[right_child].vals[p2++];
        }
    }

    tree[node].to_left[total_sz] = p1;
    tree[node].to_right[total_sz] = p2;
}

int query(int node, int start, int end, int l, int r, int idx) {
    if (l > end || r < start) {
        return 0;
```

```
    }

    if (l <= start && end <= r) {
        return idx;
    }

    int mid = (start + end) / 2;
    int next_idx_left = tree[node].to_left[idx];
    int next_idx_right = tree[node].to_right[idx];

    return query(2 * node, start, mid, l, r,
    ↪ next_idx_left) +
           query(2 * node + 1, mid + 1, end, l, r,
    ↪ next_idx_right);
}
```

2.1.10 RangeMajorityCandidates

```
const int MAXN = 200005;                          //
    ↪ array bound (Template.cpp calls this N)
int a[MAXN];
vector<int> pos[MAXN];
int K_LIMIT = 2;

struct Node {
    vector<pair<int, int>> cands;
};

Node tree_seg[4 * MAXN];

Node merge_nodes(const Node& left, const Node& right)
    ↪ {
    vector<pair<int, int>> combined;
    combined.reserve(left.cands.size() +
    ↪ right.cands.size());

    int i = 0, j = 0;
    while (i < left.cands.size() && j <
    ↪ right.cands.size()) {
        if (left.cands[i].first ==
    ↪ right.cands[j].first) {
            combined.push_back({left.cands[i].first,
    ↪ left.cands[i].second + right.cands[j].second});
            i++; j++;
        } else if (left.cands[i].first <
    ↪ right.cands[j].first) {
            combined.push_back(left.cands[i++]);
        } else {
            combined.push_back(right.cands[j++]);
        }
    }
    while (i < left.cands.size())
    ↪ combined.push_back(left.cands[i++]);
    while (j < right.cands.size())
    ↪ combined.push_back(right.cands[j++]);

    if (combined.size() <= K_LIMIT - 1) {
        return {combined};
    }

    vector<int> counts;
    counts.reserve(combined.size());
    for (const auto& p : combined) {
        counts.push_back(p.second);
    }

    nth_element(counts.begin(), counts.begin() +
    ↪ K_LIMIT - 1, counts.end(), greater<int>());
    int penalty = counts[K_LIMIT - 1];

    vector<pair<int, int>> survived;
    survived.reserve(K_LIMIT - 1);
    for (const auto& p : combined) {
        if (p.second > penalty) {
            survived.push_back({p.first, p.second -
    ↪ penalty});
        }
    }

    return {survived};
}

void build(int node, int start, int end) {
    if (start == end) {
        tree_seg[node].cands.push_back({a[start], 1});
        return;
    }
    int mid = start + (end - start) / 2;
    build(2 * node, start, mid);
    build(2 * node + 1, mid + 1, end);
    tree_seg[node] = merge_nodes(tree_seg[2 * node],
    ↪ tree_seg[2 * node + 1]);
}

Node query_candidates(int node, int start, int end,
    ↪ int l, int r) {
    if (r < start || end < l) return {{}};
    if (l <= start && end <= r) return tree_seg[node];

    int mid = start + (end - start) / 2;
    Node left_res = query_candidates(2 * node, start,
    ↪ mid, l, r);
    Node right_res = query_candidates(2 * node + 1,
    ↪ mid + 1, end, l, r);

    if (left_res.cands.empty()) return right_res;
    if (right_res.cands.empty()) return left_res;

    return merge_nodes(left_res, right_res);
}

// O(log N) fast static frequency count
int get_frequency(int val, int l, int r) {
    auto& v = pos[val];
    auto right_it = upper_bound(v.begin(), v.end(),
    ↪ r);
    auto left_it = lower_bound(v.begin(), v.end(), l);
    return distance(left_it, right_it);
}

void solve() {
    int n, q;
    cin >> n >> q;

    for (int i = 1; i <= n; i++) {
        cin >> a[i];
        pos[a[i]].push_back(i);
    }

    build(1, 1, n);

    while (q--) {
        int l, r;
        cin >> l >> r;

        int len = r - l + 1;
        Node res = query_candidates(1, 1, n, l, r);
```

```
        int max_freq = 0;
        for (const auto& p : res.cands) {
            int cand = p.first;
            int freq = get_frequency(cand, l, r);
            max_freq = max(max_freq, freq);
        }

        if (max_freq > (len + 1) / 2) {
            cout << 2 * max_freq - len << "\n";
        } else {
            cout << 1 << "\n";
        }
    }
}
```

2.1.11 SegTreeDescent

Segment Tree DESCENT - answer "first / last index satisfying P" in O(log n), not O(log^2 n).

```
// Here: max-segtree. first_ge(l, x) = smallest i >= l with a[i] >= x, or n if none.
// Swap the stored aggregate + the two ‘good()‘ tests to get: first prefix-sum >= S,
// first zero, k-th set bit, "how far right can I extend". This is the workhorse behind
// greedy "extend as far as possible" and interval-cover problems.
struct SegMax {
    int n;
    vector<ll> t;

    SegMax(int n = 0) : n(n), t(4 * n, LLONG_MIN) {}

    void build(const vector<ll>& a, int v, int l, int r) {
        if (l == r) { t[v] = a[l]; return; }
        int m = (l + r) / 2;
        build(a, 2 * v, l, m), build(a, 2 * v + 1, m + 1, r);
        t[v] = max(t[2 * v], t[2 * v + 1]);
    }
    void build(const vector<ll>& a) { if (n) build(a, 1, 0, n - 1); }

    void set(int p, ll x, int v, int l, int r) {
        if (l == r) { t[v] = x; return; }
        int m = (l + r) / 2;
        p <= m ? set(p, x, 2 * v, l, m) : set(p, x, 2 * v + 1, m + 1, r);
        t[v] = max(t[2 * v], t[2 * v + 1]);
    }
    void set(int p, ll x) { set(p, x, 1, 0, n - 1); }

    ll qry(int ql, int qr, int v, int l, int r) {
        if (qr < l || r < ql) return LLONG_MIN;
        if (ql <= l && r <= qr) return t[v];
        int m = (l + r) / 2;
        return max(qry(ql, qr, 2 * v, l, m), qry(ql, qr, 2 * v + 1, m + 1, r));
    }
    ll qry(int l, int r) { return qry(l, r, 1, 0, n - 1); }

    // smallest i in [ql, n) with a[i] >= x (returns n if none)
    int first_ge(int ql, ll x, int v, int l, int r) {
        if (r < ql || t[v] < x) return n;
        if (l == r) return l;
        int m = (l + r) / 2, res = first_ge(ql, x, 2 * v, l, m);
        return res < n ? res : first_ge(ql, x, 2 * v + 1, m + 1, r);
    }
    int first_ge(int ql, ll x) { return first_ge(ql, x, 1, 0, n - 1); }

    // largest i in [0, qr] with a[i] >= x (returns -1 if none)
    int last_ge(int qr, ll x, int v, int l, int r) {
        if (l > qr || t[v] < x) return -1;
        if (l == r) return l;
        int m = (l + r) / 2, res = last_ge(qr, x, 2 * v + 1, m + 1, r);
        return res >= 0 ? res : last_ge(qr, x, 2 * v, l, m);
    }
    int last_ge(int qr, ll x) { return last_ge(qr, x, 1, 0, n - 1); }
};
```

2.1.12 PersistentLazySegTree

PERSISTENT LAZY SEGMENT TREE - range add and range sum on ANY past version, O(log n) time and

```
// O(log n) new nodes per update.
// WHEN: you need RANGE updates and old versions. 03 - PersistentSegmentTree only does point
// updates, which is all the "k-th smallest in a range" trick needs - prefer it when that suffices,
// it is half the code and half the memory. If you only ever query the LATEST version, use
// 02 - LazySegmentTree.
// KEY INVARIANT: a tag is NEVER pushed down - that would mutate a node an old version still points
// at. Instead t[u].sum is always the CORRECT sum of u’s whole range (tag included) and t[u].add is
// the amount still owed to the two children. So a query carries the accumulated ancestor tags down
// with it, and an update fixes sum on the way back up as sum(l) + sum(r) + add * len.
struct PLazySeg {
    struct Node { int l = 0, r = 0; ll sum = 0, add = 0; };
    vector<Node> t;
    int n;
    PLazySeg(int n) : t(1), n(n) {}          // node 0 = the shared all-zero node
    int build(const vector<ll>& a, int lo, int hi) {
        int u = t.size();
        t.push_back({});
        if (lo == hi) { t[u].sum = a[lo]; return u; }
        int m = (lo + hi) / 2, L = build(a, lo, m), R = build(a, m + 1, hi);
        t[u].l = L, t[u].r = R, t[u].sum = t[L].sum + t[R].sum;
        return u;
    }
    int build(const vector<ll>& a) { return build(a, 0, n - 1); }
    int update(int u, int ql, int qr, ll v, int lo, int hi) { // add v to [ql, qr], returns new root
        int c = t.size();
        t.push_back(t[u]);
        if (ql <= lo && hi <= qr) { t[c].sum += v * (hi - lo + 1), t[c].add += v; return c; }
        int m = (lo + hi) / 2;
```

```
        if (ql <= m) t[c].l = update(t[c].l, ql, qr,
    ↪ v, lo, m);
        if (qr > m) t[c].r = update(t[c].r, ql, qr,
    ↪ v, m + 1, hi);
        t[c].sum = t[t[c].l].sum + t[t[c].r].sum +
    ↪ t[c].add * (hi - lo + 1);
        return c;
      }
      int update(int u, int l, int r, ll v) { return
    ↪ update(u, l, r, v, 0, n - 1); }
      ll query(int u, int ql, int qr, ll acc, int lo,
    ↪ int hi) { // acc = sum of the ancestors' tags
        if (ql <= lo && hi <= qr) return t[u].sum +
    ↪ acc * (hi - lo + 1);
        int m = (lo + hi) / 2;
        acc += t[u].add;
        ll res = 0;
        if (ql <= m) res += query(t[u].l, ql, qr,
    ↪ acc, lo, m);
        if (qr > m) res += query(t[u].r, ql, qr, acc,
    ↪ m + 1, hi);
        return res;
      }
      ll query(int u, int l, int r) { return query(u,
    ↪ l, r, 0, 0, n - 1); }
};
// MEMORY is the real constraint: O((n + q log n))
    ↪ nodes, ~32 bytes each. reserve() up front.
// RANGE ASSIGN instead of range add: the same
    ↪ no-push-down trick works for any tag that
    ↪ composes,
// but the accumulated 'acc' must then be "the last
    ↪ assign seen on the way down", so pass the tag
// itself rather than a sum, and let a deeper tag
    ↪ override a shallower one.
// MIN/MAX aggregate: replace v * len by v and the
    ↪ pull by min(sum(l), sum(r)) + add.
// A COMMON ALTERNATIVE that avoids all of this: make
    ↪ the update offline, sort by version, and use
// an ordinary lazy segment tree with rollback (14 -
    ↪ OfflineDynamicConnectivity's timeline trick).
```

2.1.13 ArithmeticProgressionSegTree

ARITHMETIC-PROGRESSION SEGMENT TREE - "add a, a+d, a+2d, ... to a[l..r]", range sum, O(log n).

```
// WHEN: updates whose value DEPENDS ON THE POSITION
    ↪ inside the range - "add i-l+1 to a[l..r]",
// "add w to every node on a path, weighted by
    ↪ depth", any linear ramp. A plain lazy segment
    ↪ tree
// cannot do it because its tag must be one number.
// KEY INVARIANT: a progression restricted to a
    ↪ sub-range is still a progression, so the tag
    ↪ stays
// two numbers (a, d) and stays CLOSED under
    ↪ composition: two tags add componentwise. The tag
    ↪ is
// ANCHORED at the node's left endpoint, so pushing
    ↪ to the right child skips (m+1-b) terms.
struct APSeg {
    struct Tag { ll a = 0, d = 0; };
    ↪           // a, a+d, a+2d, ... from node's left
    int n;
    vector<ll> s;
    vector<Tag> lz;
    APSeg(int n) : n(n), s(4 * n, 0), lz(4 * n) {}
    static ll tot(Tag t, ll len) { return t.a * len +
    ↪ t.d * (len * (len - 1) / 2); }
    static Tag skip(Tag t, ll k) { return {t.a + k *
    ↪ t.d, t.d}; }          // drop the first k terms
    void apply(int x, Tag t, int len) { s[x] +=
    ↪ tot(t, len), lz[x].a += t.a, lz[x].d += t.d; }
    void push(int x, int b, int e) {
        if (!lz[x].a && !lz[x].d) return;
        int m = (b + e) / 2;
        apply(2 * x, lz[x], m - b + 1), apply(2 * x +
    ↪ 1, skip(lz[x], m + 1 - b), e - m);
        lz[x] = {};
    }
    void build(const vector<ll>& a, int x, int b, int
    ↪ e) {
        lz[x] = {};
        if (b == e) { s[x] = a[b]; return; }
        int m = (b + e) / 2;
        build(a, 2 * x, b, m), build(a, 2 * x + 1, m
    ↪ + 1, e);
        s[x] = s[2 * x] + s[2 * x + 1];
    }
    void build(const vector<ll>& a) { build(a, 1, 0,
    ↪ n - 1); }
    void upd(int x, int b, int e, int l, int r, Tag
    ↪ t) {       // t is anchored at index l
        if (r < b || e < l) return;
        if (l <= b && e <= r) { apply(x, skip(t, b -
    ↪ l), e - b + 1); return; }
        push(x, b, e);
        int m = (b + e) / 2;
        upd(2 * x, b, m, l, r, t), upd(2 * x + 1, m +
    ↪ 1, e, l, r, t);
        s[x] = s[2 * x] + s[2 * x + 1];
    }
    void upd(int l, int r, ll a, ll d) { upd(1, 0, n
    ↪ - 1, l, r, {a, d}); } // a[l]+=a, a[l+1]+=a+d..
    ll qry(int x, int b, int e, int l, int r) {
        if (r < b || e < l) return 0;
        if (l <= b && e <= r) return s[x];
        push(x, b, e);
        int m = (b + e) / 2;
        return qry(2 * x, b, m, l, r) + qry(2 * x +
    ↪ 1, m + 1, e, l, r);
    }
    ll qry(int l, int r) { return qry(1, 0, n - 1, l,
    ↪ r); }
};
// OVERFLOW: tot() is O(len^2 * d) - with n = 2e5 and
    ↪ d = 1e9 that is 2e19, past ll. Work mod p
// (replace /2 by * inv2) or bound d.
// DECREASING ramps: pass a negative d; "add a, a-d,
    ↪ a-2d" to [l,r] read right-to-left is the same
// tag anchored at r, so mirror the array or negate d
    ↪ and anchor at l with a' = a + (r-l)*d.
// LIGHTER ALTERNATIVES, use these when they fit:
//   Range AP add + POINT query only: one BIT over
    ↪ the second difference b[i] = a[i]-2a[i-1]+a[i-2];
//     an AP add touches 4 cells, and a[i] is a
    ↪ double prefix sum. 5 lines (02 - BIT).
//   OFFLINE with all updates before all queries:
    ↪ apply the second-difference trick to a plain
//     array and prefix-sum twice at the end. O(n)
    ↪ total.
// GENERAL RULE: any tag family closed under
    ↪ composition and restriction works. Polynomials of
// degree k need k+1 coefficients; affine "a[i] =
    ↪ p*a[i] + q" needs (p, q) composed as function
// composition (that one is NOT commutative - compose
    ↪ in the right order).
```

2.1.14 XORSegTree

XOR SEGMENT TREE - sum of a[p ^ x] over l <= p <= r, for an x given per query, plus point

```
// updates. O(log n) both, and NO extra memory over
   ↪ an ordinary segment tree.
// WHEN: "the array is permuted by xor-ing every
   ↪ index with x" - subset/bit problems where a query
// asks about a[l..r] of a xor-shifted array, e.g. CF
   ↪ 1654F. Also the fast way to combine two
// arrays under an xor convolution restricted to a
   ↪ range.
// KEY INVARIANT: an ALIGNED block [b, b + 2^k) has
   ↪ its low k bits free, so xor-ing every element
// of it with x maps the block ONTO the aligned block
   ↪ starting at b ^ (x >> k << k) - a block goes
// to a block, never to a straddling range. The
   ↪ canonical decomposition of [l, r] is O(log n)
// aligned blocks, so the answer is O(log n) node
   ↪ lookups; the only work is finding each partner
// node, which at level k is just xor-ing the node's
   ↪ position-within-level by x >> k.
struct XorSeg {                                          //
   ↪ n MUST be a power of two; queries use x < n
    int n;
    vector<ll> t;
    XorSeg(const vector<ll>& a) : n(a.size()), t(2 *
   ↪ n) {
        copy(all(a), t.begin() + n);
        for (int i = n - 1; i; i--) t[i] = t[2 * i] +
   ↪ t[2 * i + 1];
    }
    void add(int i, ll v) { for (t[i += n] += v; i
   ↪ >>= 1;) t[i] = t[2 * i] + t[2 * i + 1]; }
    ll query(int l, int r, int x) {                      //
   ↪ sum of a[p ^ x] over l <= p <= r
        ll res = 0;
        for (int lo = l + n, hi = r + n + 1, b = n;
   ↪ lo < hi; lo >>= 1, hi >>= 1, b >>= 1, x >>= 1) {
            if (lo & 1) res += t[((lo - b) ^ x) + b],
   ↪ lo++;    // b = number of nodes on this level
            if (hi & 1) res += t[((--hi - b) ^ x) +
   ↪ b];        //   and also the level's index base
        }
        return res;
    }
};
// THE AGGREGATE must be COMMUTATIVE (sum, xor, min,
   ↪ gcd): xor-ing reorders the elements inside
// each block, so anything order-sensitive (max
   ↪ prefix sum, matrix product) breaks.
// PAD n up to a power of two with identity elements
   ↪ - the alignment argument needs it.
// THE SAME TRICK, no data structure: if you only
   ↪ need it once, enumerate the O(log n) blocks of
// [l, r] yourself and answer each with a prefix sum
   ↪ of the ORIGINAL array. That is the version to
// write when there are no updates.
// RANGE UPDATES do not survive this: a lazy tag
   ↪ would have to travel to the xor-partner subtree.
// If you need them, keep the tag OUTSIDE (a separate
   ↪ BIT of "add v to a[l..r]" offsets) and add
// the contribution of the covered blocks
   ↪ analytically.
```

2.1.15 KineticSegTree

KINETIC SEGMENT TREE (kinetic tournament) - each index i holds a LINE y = a[i]*T + b[i]; the

```
// tree answers "max over i in [l, r] of a[i]*T +
   ↪ b[i]" at a global time T that only MOVES FORWARD,
// and supports adding a constant to b over a range.
// WHEN: DP where the candidate values are linear in
   ↪ a parameter that sweeps monotonically -
// "max over j of dp[j] + (i-j)*c", scheduling with a
   ↪ moving deadline, CHT where the queries are
// sorted but the LINES also change over time (a Li
   ↪ Chao tree cannot delete or shift lines).
// If T is arbitrary (not monotone) use Li Chao /
   ↪ convex hull trick instead - this degenerates.
// COMPLEXITY: O(log n) per query and per range-add;
   ↪ advancing T is O(log^2 n) AMORTISED, and the
// total over a whole run is O((n + q) log^2 n). A
   ↪ single advance can cost O(n) in the worst case.
// KEY INVARIANT (the "certificate"): every node
   ↪ stores the winning line of its subtree AND melt =
// the earliest future time at which the winner of
   ↪ some node below it changes. Advancing to T
// recurses ONLY into nodes with melt <= T;
   ↪ everything else is provably still correct. Each
// recursion strictly changes a winner, and a winner
   ↪ can only change O(log n) times amortised per
// element because two lines cross at most once.
struct KineticSeg {
    int n;
    ll T = 0;
    vector<ll> a, b, lz, melt;
   ↪          // a, b = the node's WINNING line
    KineticSeg(const vector<ll>& A, const vector<ll>&
   ↪ B)
        : n(A.size()), a(4 * n), b(4 * n), lz(4 * n,
   ↪ 0), melt(4 * n, llinf) {
        build(A, B, 1, 0, n - 1);
    }
    static ll fdiv(ll p, ll q) { return p / q - ((p %
   ↪ q != 0) && ((p < 0) != (q < 0))); }
    ll cross(int w, int o) {
   ↪          // first time o strictly beats w
        if (a[o] <= a[w]) return llinf;
   ↪          // parallel or flatter: never
        return fdiv(b[w] - b[o], a[o] - a[w]) + 1;
    }
    void pull(int x) {
        int L = 2 * x, R = L + 1;
        int w = a[L] * T + b[L] >= a[R] * T + b[R] ?
   ↪ L : R, o = L + R - w;
        a[x] = a[w], b[x] = b[w];
        melt[x] = min({melt[L], melt[R], cross(w,
   ↪ o)});
    }
    void apply(int x, ll v) { b[x] += v, lz[x] += v;
   ↪ }         // uniform shift keeps every melt
    void push(int x) { if (lz[x]) apply(2 * x,
   ↪ lz[x]), apply(2 * x + 1, lz[x]), lz[x] = 0; }
    void build(const vector<ll>& A, const vector<ll>&
   ↪ B, int x, int lo, int hi) {
        lz[x] = 0, melt[x] = llinf;
        if (lo == hi) { a[x] = A[lo], b[x] = B[lo];
   ↪ return; }
        int m = (lo + hi) / 2;
        build(A, B, 2 * x, lo, m), build(A, B, 2 * x
   ↪ + 1, m + 1, hi);
        pull(x);
    }
```

```
    void heaten(int x, int lo, int hi) {
        if (melt[x] > T || lo == hi) return;
  ↪           // certificate still valid
        int m = (lo + hi) / 2;
        push(x);
        heaten(2 * x, lo, m), heaten(2 * x + 1, m +
  ↪ 1, hi);
        pull(x);
    }
    void advance(ll t) { T = t; heaten(1, 0, n - 1);
  ↪ }         // t must be >= the current T
    void add(int x, int lo, int hi, int l, int r, ll
  ↪ v) {      // b[i] += v for i in [l, r]
        if (r < lo || hi < l) return;
        if (l <= lo && hi <= r) { apply(x, v);
  ↪ return; }
        int m = (lo + hi) / 2;
        push(x);
        add(2 * x, lo, m, l, r, v), add(2 * x + 1, m
  ↪ + 1, hi, l, r, v);
        pull(x);
    }
    void add(int l, int r, ll v) { add(1, 0, n - 1,
  ↪ l, r, v); }
    void set(int x, int lo, int hi, int i, ll na, ll
  ↪ nb) {     // replace the line at index i
        if (lo == hi) { a[x] = na, b[x] = nb; return;
  ↪ }
        int m = (lo + hi) / 2;
        push(x);
        i <= m ? set(2 * x, lo, m, i, na, nb) : set(2
  ↪ * x + 1, m + 1, hi, i, na, nb);
        pull(x);
    }
    void set(int i, ll na, ll nb) { set(1, 0, n - 1,
  ↪ i, na, nb); }
    ll qry(int x, int lo, int hi, int l, int r) {
  ↪           // max at the CURRENT T
        if (r < lo || hi < l) return -llinf;
        if (l <= lo && hi <= r) return a[x] * T +
  ↪ b[x];
        int m = (lo + hi) / 2;
        push(x);
        return max(qry(2 * x, lo, m, l, r), qry(2 * x
  ↪ + 1, m + 1, hi, l, r));
    }
    ll qry(int l, int r) { return qry(1, 0, n - 1, l,
  ↪ r); }
};
// MIN instead of max: negate both a and b, or flip
  ↪ the comparison in pull() and cross().
// ADDING TO THE SLOPES over a range destroys every
  ↪ certificate below and is NOT amortised - do it
// only at a single index (set()) or rebuild.
// SEGMENT TREE BEATS (04) is the same idea with a
  ↪ different certificate: recurse only where the
// tag cannot be applied wholesale, and bound the
  ↪ recursions by a potential. If you can phrase your
// update as "the answer only changes when two stored
  ↪ candidates swap order", this is the shape.
// A CHEAPER SPECIAL CASE: if the lines are only ever
  ↪ ADDED (never removed) and queries are sorted,
// a monotone CHT is O(1) amortised per op - reach
  ↪ for that first.
```

2.1.16 HistoricSegTree

HISTORIC-MAXIMUM SEGMENT TREE - range add, range max of the CURRENT values, and range max of

```
// the HISTORIC value hmax[i] = max over every past
  ↪ moment of a[i]. O(log n) per operation.
// WHEN: "what is the largest this cell ever
  ↪ reached", "peak stock price / peak queue length /
// high-water mark over a range", and as the engine
  ↪ behind offline problems that sweep an index
// while range-adding. If you only need the current
  ↪ max, use 02 - LazySegmentTree.
// KEY INVARIANT: the tag is a PAIR (add, mx) = "the
  ↪ total of this batch of adds" and "the largest
// PARTIAL SUM (prefix) any moment inside the batch
  ↪ reached". That pair is closed under
// composition: applying t and then u gives {t.add +
  ↪ u.add, max(t.mx, t.add + u.mx)}. So a whole
// run of pending updates can sit on a node and still
  ↪ be replayed exactly for the historic max.
// Careful: mx is a max over prefixes INCLUDING the
  ↪ empty one, so it is always >= 0.
struct HistoricSeg {
    struct Tag { ll add = 0, mx = 0; };
    int n;
    vector<ll> cur, his;
  ↪          // current max, historic max
    vector<Tag> lz;
    HistoricSeg(int n) : n(n), cur(4 * n, 0), his(4 *
  ↪ n, 0), lz(4 * n) {}
    void apply(int x, Tag t) {
        his[x] = max(his[x], cur[x] + t.mx);
  ↪          // peak inside this batch
        cur[x] += t.add;
        lz[x] = {lz[x].add + t.add, max(lz[x].mx,
  ↪ lz[x].add + t.mx)};
    }
    void push(int x) {
        if (!lz[x].add && !lz[x].mx) return;
        apply(2 * x, lz[x]), apply(2 * x + 1, lz[x]);
        lz[x] = {};
    }
    void pull(int x) {
        cur[x] = max(cur[2 * x], cur[2 * x + 1]);
        his[x] = max(his[2 * x], his[2 * x + 1]);
    }
    void build(const vector<ll>& a, int x, int lo,
  ↪ int hi) {
        lz[x] = {};
        if (lo == hi) { cur[x] = his[x] = a[lo];
  ↪ return; }
        int m = (lo + hi) / 2;
        build(a, 2 * x, lo, m), build(a, 2 * x + 1, m
  ↪ + 1, hi);
        pull(x);
    }
    void build(const vector<ll>& a) { build(a, 1, 0,
  ↪ n - 1); }
    void upd(int x, int lo, int hi, int l, int r, ll
  ↪ v) {
        if (r < lo || hi < l) return;
        if (l <= lo && hi <= r) { apply(x, {v, max(v,
  ↪ 0LL)}); return; }
        int m = (lo + hi) / 2;
        push(x);
        upd(2 * x, lo, m, l, r, v), upd(2 * x + 1, m
  ↪ + 1, hi, l, r, v);
        pull(x);
    }
```

```
    void upd(int l, int r, ll v) { upd(1, 0, n - 1,
  ↪ l, r, v); }
    // hist = false -> max of current values; hist =
  ↪ true -> max of the all-time values
    ll qry(int x, int lo, int hi, int l, int r, bool
  ↪ hist) {
        if (r < lo || hi < l) return -llinf;
        if (l <= lo && hi <= r) return hist ? his[x]
  ↪ : cur[x];
        int m = (lo + hi) / 2;
        push(x);
        return max(qry(2 * x, lo, m, l, r, hist),
  ↪ qry(2 * x + 1, m + 1, hi, l, r, hist));
    }
    ll qry(int l, int r, bool hist) { return qry(1,
  ↪ 0, n - 1, l, r, hist); }
};
// HISTORIC MIN: mirror everything (track the
  ↪ smallest prefix of the batch).
// HISTORIC SUM (sum over time of every past value,
  ↪ i.e. integrate the array) needs a bigger tag:
// keep sum, historic sum, and a count of elapsed
  ↪ time steps, and let the tag carry (add, time)
// with hsum += sum * dt. Same idea, one more field -
  ↪ only write it if the problem forces it.
// RANGE ASSIGN + historic max is the same tag with
  ↪ "assign" replacing "add"; compose so the later
// assign wins and mx tracks the largest value ever
  ↪ assigned.
// THE OFFLINE ALTERNATIVE, almost always shorter: if
  ↪ the queries are known in advance, sweep time
// and answer with an ordinary max segment tree plus
  ↪ a BIT for "was this cell touched after t".
```

2.1.17 SegTree2DSparse

SPARSE 2D SEGMENT TREE (segment tree of TREAPS) - point assign a[i][j] = v and rectangle sum on

```
// a grid whose columns are huge or unknown in
  ↪ advance. O(log n * log q) per operation and only
// O(q log n) memory - one node per touched cell per
  ↪ level.
// WHEN: 08 - SegTree2D is a dense n x m tree and
  ↪ needs O(nm) memory; a merge-sort tree (06) is
// static; an offline problem should use 02 - BIT/03
  ↪ - Offline2DFenwickTree, which is smaller and
// faster because it can compress the coordinates
  ↪ first. Reach for THIS one only when the updates
// are ONLINE and the grid is too big to allocate.
// KEY INVARIANT: the outer tree is indexed by ROW
  ↪ and every outer node owns a treap keyed by
// COLUMN holding, for each column j that some row in
  ↪ its range touches, the aggregate of that
// column over the node's rows. So an outer node is
  ↪ only ever asked about columns that exist, and
// after a point update the parents repair a SINGLE
  ↪ column by recombining their two children's
// value at that one column.
struct Seg2D {
    struct T { int l = 0, r = 0, key = 0; unsigned pr
  ↪ = 0; ll val = 0, agg = 0; };
    vector<T> tr;
  ↪          // shared pool for every treap
    vector<int> rt;
  ↪          // treap root of each outer node
    int n;
    Seg2D(int n) : tr(1), rt(4 * n, 0), n(n) {}
    unsigned rnd() {
  ↪          // xorshift; any prng will do
        static unsigned s = 2463534242u;
        return s ^= s << 13, s ^= s >> 17, s ^= s <<
  ↪ 5;
    }
    void pull(int x) { tr[x].agg = tr[tr[x].l].agg +
  ↪ tr[x].val + tr[tr[x].r].agg; }
    void split(int x, int k, int& a, int& b) {
  ↪          // a keeps keys < k
        if (!x) { a = b = 0; return; }
        if (tr[x].key < k) a = x, split(tr[x].r, k,
  ↪ tr[a].r, b), pull(a);
        else b = x, split(tr[x].l, k, a, tr[b].l),
  ↪ pull(b);
    }
    int merge(int a, int b) {
        if (!a || !b) return a | b;
        if (tr[a].pr > tr[b].pr) { tr[a].r =
  ↪ merge(tr[a].r, b); pull(a); return a; }
        tr[b].l = merge(a, tr[b].l);
        pull(b);
        return b;
    }
    void set(int& root, int k, ll v) {
  ↪          // a[k] = v inside one treap
        int a, b, c;
        split(root, k, a, b), split(b, k + 1, b, c);
        if (b) tr[b].val = v, pull(b);
        else tr.push_back({0, 0, k, rnd(), v, v}), b
  ↪ = tr.size() - 1;
        root = merge(merge(a, b), c);
    }
    ll get(int& root, int lo, int hi) {
  ↪          // sum of keys in [lo, hi]
        int a, b, c;
        split(root, lo, a, b), split(b, hi + 1, b, c);
        ll res = tr[b].agg;
        root = merge(merge(a, b), c);
        return res;
    }
    void upd(int x, int lo, int hi, int i, int j, ll
  ↪ v) {      // a[i][j] = v
        if (lo == hi) { set(rt[x], j, v); return; }
        int m = (lo + hi) / 2;
        i <= m ? upd(2 * x, lo, m, i, j, v) : upd(2 *
  ↪ x + 1, m + 1, hi, i, j, v);
        set(rt[x], j, get(rt[2 * x], j, j) + get(rt[2
  ↪ * x + 1], j, j));
    }
    void upd(int i, int j, ll v) { upd(1, 0, n - 1,
  ↪ i, j, v); }
    ll qry(int x, int lo, int hi, int r1, int r2, int
  ↪ c1, int c2) {
        if (r2 < lo || hi < r1) return 0;
        if (r1 <= lo && hi <= r2) return get(rt[x],
  ↪ c1, c2);
        int m = (lo + hi) / 2;
        return qry(2 * x, lo, m, r1, r2, c1, c2) +
  ↪ qry(2 * x + 1, m + 1, hi, r1, r2, c1, c2);
    }
    ll qry(int r1, int r2, int c1, int c2) { return
  ↪ qry(1, 0, n - 1, r1, r2, c1, c2); }
};
// ANY IDEMPOTENT OR GROUP AGGREGATE works (sum, xor,
  ↪ gcd, min): upd() rebuilds a parent's single
// column from its two children, so the operation
  ↪ only has to be associative and commutative.
// ROWS TOO BIG TOO: make the outer tree dynamic as
  ↪ well (05 - DynamicSegTree) - allocate the outer
```

```
// node on first touch. Memory stays O(q log n).
// BIT OF TREAPS is the same shape with a shorter
   ↪ outer layer, but it cannot do "descend to find
// the k-th" and needs a group (no min/max).
// IF THE PROBLEM IS OFFLINE, do not write this. Sort
   ↪ the queries by row, sweep, and keep a single
// 1D BIT over columns: O((n + q) log n), five lines,
   ↪ and no memory pressure.
```

2.1.18 MexSegTree

MEX UNDER UPDATES - the smallest non-negative integer NOT present in the multiset, maintained

```
// under insert and erase in O(log V), read in O(log
   ↪ V) by ONE descent (not a scan).
// WHEN: "after each operation print the mex",
   ↪ Grundy/nim-value maintenance (10 - Game Theory),
// sliding-window mex. If the multiset only ever
   ↪ GROWS, a pointer that walks forward is O(1)
// amortised and this file is overkill. If you need
   ↪ the mex of a RANGE, see the recipe below.
// KEY INVARIANT: a segment tree over the VALUE axis
   ↪ [0, V] storing the MINIMUM count in each value
// range. The mex is the leftmost value with count 0,
   ↪ so the descent is "if the left child's
// minimum is 0 go left, else go right" - which is
   ↪ why the tree stores a MIN and not a sum.
// V only needs to be n + 1: with n elements the mex
   ↪ can never exceed n.
struct MexSet {
    int V;
    vector<int> cnt, mn;
    MexSet(int V) : V(V), cnt(V + 1, 0), mn(4 * (V +
   ↪ 1), 0) {}
    void upd(int x, int lo, int hi, int p) {
        if (lo == hi) { mn[x] = cnt[lo]; return; }
        int m = (lo + hi) / 2;
        p <= m ? upd(2 * x, lo, m, p) : upd(2 * x +
   ↪ 1, m + 1, hi, p);
        mn[x] = min(mn[2 * x], mn[2 * x + 1]);
    }
    void add(int v) { if (v <= V) cnt[v]++, upd(1, 0,
   ↪ V, v); }
    void del(int v) { if (v <= V) cnt[v]--, upd(1, 0,
   ↪ V, v); }
    int mex(int x, int lo, int hi) {
        if (lo == hi) return lo;
        int m = (lo + hi) / 2;
        return mn[2 * x] == 0 ? mex(2 * x, lo, m) :
   ↪ mex(2 * x + 1, m + 1, hi);
    }
    int mex() { return mex(1, 0, V); }
};
// MEX OF A RANGE [l, r], offline: sort the queries
   ↪ by r and keep a segment tree over VALUES whose
// leaf v holds the LAST POSITION where v occurs in
   ↪ the prefix ending at r (-1 if never). The mex
// of [l, r] is the leftmost v whose last position is
   ↪ < l - the same descent, comparing against l
// instead of against 0. Online: make that tree
   ↪ persistent (03 - PersistentSegmentTree), version
   ↪ r.
// MEX OF A RANGE with array updates: no clean
   ↪ polylog solution - use Mo's plus the O(1)-add/del
// counter in 06 - Square Root Decomposition/07,
   ↪ scanning for the first empty count bucket.
// MEX OF ALL SUBARRAYS / sum of mex over all
   ↪ subarrays: divide and conquer on the position of
   ↪ the
// value 0, or the "for each mex value, count the
   ↪ subarrays achieving it" sweep - not this file.
// FIRST MISSING POSITIVE (mex over 1..) is the same
   ↪ with the axis shifted by one.
```

2.2 Binary Indexed Tree (BIT)

2.2.1 FenwickTree

Fenwick Tree (Binary Indexed Tree) - 0-based indexing

```
// Space: O(N), Time: O(log N) per update/query
template<typename T = ll>
struct FenwickTree {
    int n;
    vector<T> tree;

    FenwickTree(int n = 0) : n(n), tree(n + 1, 0) {}

    void add(int i, T delta) {
        for (i++; i <= n; i += i & -i) tree[i] +=
   ↪ delta;
    }

    void set(int i, T val) {
        add(i, val - query(i, i));
    }

    T query(int i) {
        T sum = 0;
        for (i++; i > 0; i -= i & -i) sum += tree[i];
        return sum;
    }

    T query(int l, int r) {
        if (l > r) return 0;
        return query(r) - query(l - 1);
    }

    // Returns first 0-based index where prefix sum
   ↪ >= val
    int lower_bound(T val) {
        int pos = 0;
        for (int sz = 1 << (n ? __lg(n) : 0); sz > 0;
   ↪ sz >>= 1) {
            if (pos + sz <= n && tree[pos + sz] <
   ↪ val) {
                val -= tree[pos + sz];
                pos += sz;
            }
        }
        return pos;
    }
};
```

2.2.2 FenwickTree2D

2D Fenwick Tree - 0-based indexing

```
// Space: O(N * M), Time: O(log N * log M) per
   ↪ update/query
template<typename T = ll>
struct FenwickTree2D {
    int n, m;
    vector<vector<T>> tree;

    FenwickTree2D(int n = 0, int m = 0) : n(n), m(m),
   ↪ tree(n + 1, vector<T>(m + 1, 0)) {}

    void add(int x, int y, T val) {
        for (int i = x + 1; i <= n; i += i & -i) {
            for (int j = y + 1; j <= m; j += j & -j) {
```

```
                tree[i][j] += val;
            }
        }
    }

    void set(int x, int y, T val) {
        add(x, y, val - query(x, y, x, y));
    }

    T query(int x, int y) {
        T sum = 0;
        for (int i = min(x + 1, n); i > 0; i -= i &
↪ -i) {
            for (int j = min(y + 1, m); j > 0; j -= j
↪ & -j) {
                sum += tree[i][j];
            }
        }
        return sum;
    }

    // 2D Subgrid Range Sum from (x1, y1) to (x2, y2)
↪ inclusive
    T query(int x1, int y1, int x2, int y2) {
        if (x1 > x2 || y1 > y2) return 0;
        return query(x2, y2) - query(x1 - 1, y2) -
↪ query(x2, y1 - 1) + query(x1 - 1, y1 - 1);
    }
};
```

2.2.3 Offline2DFenwickTree

Offline 2D Fenwick Tree (Compressed 2D BIT)

```
// Space: O(N + Q log N), Time: O(log^2 N) per query
template<typename T = ll>
struct BIT2D {
    int n;
    vector<vector<int>> vals;
    vector<vector<T>> bit;

    int ind(const vector<int>& v, int x) {
        return upper_bound(all(v), x) - v.begin() - 1;
    }

    // mx: max row index, todo: all (r, c) update
↪ coordinates that will be called
    BIT2D(int mx, vector<array<int, 2>> todo) :
↪ n(mx), vals(mx + 1), bit(mx + 1) {
        sort(all(todo), [](const auto& a, const auto&
↪ b) { return a[1] < b[1]; });

        for (int i = 0; i <= n; i++)
↪ vals[i].pb(INT_MIN);
        for (auto [r, c] : todo) {
            for (int x = r; x <= n; x |= x + 1) {
                if (vals[x].back() != c)
↪ vals[x].pb(c);
            }
        }
        for (int i = 0; i <= n; i++)
↪ bit[i].resize(vals[i].size(), 0);
    }

    void add(int r, int c, T val) {
        for (; r <= n; r |= r + 1) {
            for (int i = ind(vals[r], c); i <
↪ (int)bit[r].size(); i |= i + 1) {
                bit[r][i] += val;
            }
        }
    }

    T query(int r, int c) {
        T res = 0;
        for (; r >= 0; r = (r & (r + 1)) - 1) {
            for (int i = ind(vals[r], c); i >= 0; i =
↪ (i & (i + 1)) - 1) {
                res += bit[r][i];
            }
        }
        return res;
    }

    T query(int r1, int c1, int r2, int c2) {
        return query(r2, c2) - query(r2, c1 - 1) -
↪ query(r1 - 1, c2) + query(r1 - 1, c1 - 1);
    }

    void reset() {
        for (auto& v : bit) fill(all(v), 0);
    }
};
```

2.2.4 BITRangeUpdateRangeQuery

BIT with RANGE UPDATE + RANGE QUERY (two BITs). 0-based, O(log n) per op.

```
// pre(i) = sum_{j<=i} a[j] = S1(i)*(i+1) - S2(i),
↪ where a range-add of v on [l,r] does
//   S1: +v at l, -v at r+1        S2: +v*l at l,
↪ -v*(r+1) at r+1
template <typename T = ll>
struct BIT2 {
    int n;
    vector<T> b1, b2;

    BIT2(int n = 0) : n(n), b1(n + 1, 0), b2(n + 1,
↪ 0) {}
    void add(vector<T>& b, int i, T v) { for (i++; i
↪ <= n; i += i & -i) b[i] += v; }
    T sum(const vector<T>& b, int i) const { T s = 0;
↪ for (i++; i > 0; i -= i & -i) s += b[i]; return
↪ s; }

    void upd(int l, int r, T v) {               //
↪ add v to [l, r]
        add(b1, l, v), add(b1, r + 1, -v);
        add(b2, l, v * l), add(b2, r + 1, -v * (r +
↪ 1));
    }
    T pre(int i) const { return i < 0 ? 0 : sum(b1,
↪ i) * (i + 1) - sum(b2, i); }
    T qry(int l, int r) const { return pre(r) - pre(l
↪ - 1); }
};
```

2.2.5 BITOfSegTrees

BIT OF SEGMENT TREES ("tree in tree") - k-th SMALLEST in a[l..r] WITH point assignments, in

```
// O(log^2 n) per operation and O(n log^2 n) nodes.
// WHEN: the array changes. Without updates the
  ↪ answer is a persistent segment tree over prefixes
// (01 - Segment Tree/03) at O(log n) per query and a
  ↪ third of the memory, or a wavelet tree (13);
// offline, Mo's or parallel binary search is usually
  ↪ simpler. Reach for this only for ONLINE
// "k-th smallest / count values <= x in a range"
  ↪ mixed with assignments.
// KEY INVARIANT: the BIT is over POSITIONS and every
  ↪ BIT node owns a dynamic segment tree over the
// VALUE axis counting the values of the positions
  ↪ that node covers. A prefix [0, r] is a sum of
// O(log n) BIT nodes, so [l, r] is O(log n)
  ↪ value-trees to add and O(log n) to subtract; the
  ↪ k-th
// smallest is then ONE simultaneous top-down descent
  ↪ through all of them at once, adding up the
// left-child counts to decide which way to go.
struct BITSeg {
    int n, V;
  ↪          // values live in [0, V)
    vector<int> rt, lc, rc, cnt, a;
  ↪          // rt[i] = value tree of BIT node i
    BITSeg(int n, int V) : n(n), V(V), rt(n + 1, 0),
  ↪ lc(1, 0), rc(1, 0), cnt(1, 0), a(n, -1) {}
    int upd(int x, int lo, int hi, int p, int d) {
  ↪            // += d at value p, returns node id
        if (!x) x = cnt.size(), lc.push_back(0),
  ↪ rc.push_back(0), cnt.push_back(0);
        cnt[x] += d;
        if (lo < hi) {
            int m = (lo + hi) / 2, c;
            if (p <= m) c = upd(lc[x], lo, m, p, d),
  ↪ lc[x] = c;
            else c = upd(rc[x], m + 1, hi, p, d),
  ↪ rc[x] = c;
        }
        return x;
    }
    void add(int i, int v, int d) {
        for (i++; i <= n; i += i & -i) rt[i] =
  ↪ upd(rt[i], 0, V - 1, v, d);
    }
    void set(int i, int v) {
  ↪          // a[i] = v (first call = insert)
        if (a[i] >= 0) add(i, a[i], -1);
        add(i, a[i] = v, 1);
    }
    int kth(int l, int r, int k) {
  ↪          // k-th smallest in [l, r], k >= 1
        vector<int> A, B;
        for (int i = r + 1; i > 0; i -= i & -i)
  ↪ A.push_back(rt[i]);
        for (int i = l; i > 0; i -= i & -i)
  ↪ B.push_back(rt[i]);
        int lo = 0, hi = V - 1;
        while (lo < hi) {
            int m = (lo + hi) / 2, c = 0;
            for (int x : A) c += cnt[lc[x]];
            for (int x : B) c -= cnt[lc[x]];
            bool left = k <= c;
            if (!left) k -= c, lo = m + 1;
            else hi = m;
            for (int& x : A) x = left ? lc[x] : rc[x];
            for (int& x : B) x = left ? lc[x] : rc[x];
        }
        return lo;
    }
    int less(int x, int lo, int hi, int ql, int qr) {
  ↪          // count of values in [ql, qr]
        if (!x || qr < lo || hi < ql) return 0;
        if (ql <= lo && hi <= qr) return cnt[x];
        int m = (lo + hi) / 2;
        return less(lc[x], lo, m, ql, qr) +
  ↪ less(rc[x], m + 1, hi, ql, qr);
    }
    int count(int l, int r, int vl, int vr) {
  ↪          // how many a[i] in [vl, vr]
        int res = 0;
        for (int i = r + 1; i > 0; i -= i & -i) res
  ↪ += less(rt[i], 0, V - 1, vl, vr);
        for (int i = l; i > 0; i -= i & -i) res -=
  ↪ less(rt[i], 0, V - 1, vl, vr);
        return res;
    }
};
// COORDINATE COMPRESS first (01 - Miscellaneous/05)
  ↪ - and if updates introduce NEW values, compress
// the update values too, offline, before starting.
// MEMORY is the trap: n log^2 n nodes at 12 bytes is
  ↪ ~100 MB at n = 2e5. If that is too much,
// replace the inner dynamic tree by a MERGE SORT
  ↪ TREE rebuilt in blocks, or go offline.
// THE SAME SHAPE solves: "sum of the k smallest in a
  ↪ range", "rank of x in a range", "predecessor
// / successor of x in a range" - all are the same
  ↪ descent with a different accumulator.
```

2.2.6 OfflineDistinct

COUNT DISTINCT VALUES IN A RANGE, OFFLINE - the single most reused sweep in the whole notebook.

```
// Needs: 02 - Binary Indexed Tree (BIT)/01 -
  ↪ FenwickTree.cpp
// O((n + q) log n), O(n) memory, ~20 lines. (SPOJ
  ↪ DQUERY and a hundred clones.)
// WHEN: any "how many different / how many values
  ↪ appear exactly once / sum of distinct values"
// question with all queries known up front. Mo's (06
  ↪ - Square Root Decomposition/01) also does it
// but is O(n sqrt n); a persistent segment tree does
  ↪ it ONLINE at O(log n) per query with the very
// same invariant, so start here and upgrade only if
  ↪ the queries are forced online.
// KEY INVARIANT: sweep r left to right and keep the
  ↪ BIT holding a 1 at the LAST OCCURRENCE SO FAR
// of every distinct value and 0 everywhere else.
  ↪ Then the number of ones in [l, r] is exactly the
// number of distinct values in [l, r]: each value is
  ↪ counted at its rightmost occurrence, which
// lies in [l, r] if and only if the value occurs
  ↪ there at all. Moving r to a new position means
// clearing the previous occurrence of a[r] and
  ↪ setting the new one - two BIT updates.
struct OfflineDistinct {
    int n;
    vector<int> a;
  ↪          // values already compressed to [0,V)
    vector<array<int, 3>> qs;
  ↪          // {r, l, id}
    OfflineDistinct(const vector<int>& a) :
  ↪ n(a.size()), a(a) {}
    void ask(int l, int r, int id) { qs.push_back({r,
```

```
    ↪ l, id}); }
    vector<int> solve(int V) {
        sort(all(qs));
        FenwickTree<int> bit(n);
        vector<int> last(V, -1), res(qs.size());
        int at = 0;
        for (int r = 0; r < n; r++) {
            if (last[a[r]] >= 0) bit.add(last[a[r]],
    ↪ -1);
            bit.add(last[a[r]] = r, 1);
            while (at < (int)qs.size() && qs[at][0]
    ↪ == r)
                res[qs[at][2]] = bit.query(qs[at][1],
    ↪ r), at++;
        }
        return res;
    }
};
// VALUES APPEARING EXACTLY ONCE in [l, r]: keep TWO
    ↪ BITs, one with a 1 at the last occurrence and
// one with a 1 at the second-to-last; the answer is
    ↪ ones(last) - ones(secondLast) over [l, r].
// SUM of the distinct values: put a[r] instead of 1
    ↪ at the last occurrence. Any per-value weight
// works the same way, including "count values whose
    ↪ multiplicity is >= k" (mark the k-th last).
// ONLINE version: build a persistent segment tree
    ↪ where version r is "the array of last
// occurrences after processing r", then the answer
    ↪ is a range count on version r. Same invariant,
// O(n log n) memory (01 - Segment Tree/03).
// UPDATES to the array as well: this sweep dies; use
    ↪ Mo's with updates (06/05) or a BIT of segment
// trees (05) keyed by "previous occurrence" - both
    ↪ are much heavier, so check first whether the
// problem really needs them.
```

2.2.7 BIT2DRangeUpdate

2D BIT WITH RANGE UPDATE AND RANGE QUERY - add v to a RECTANGLE, ask the sum of a RECTANGLE, in

```
// O(log n log m) with four BITs and no lazy
    ↪ propagation anywhere.
// WHEN: grid problems with rectangle add / rectangle
    ↪ sum. 02 - FenwickTree2D only does point
// update; 04 - BITRangeUpdateRangeQuery is the 1D
    ↪ version of exactly this trick; a 2D segment tree
// with lazy propagation does NOT exist in any usable
    ↪ form (you cannot push a tag through two
// dimensions), which is why this identity is the
    ↪ standard answer.
// KEY INVARIANT: work on the 2D difference array d,
    ↪ where a[i][j] = sum of d over the rectangle
// [0..i] x [0..j]. A rectangle add touches exactly
    ↪ FOUR cells of d. Expanding the double prefix
// sum gives
//   pre(x, y) = (x+1)(y+1) * S(d) - (y+1) * S(d*i) -
    ↪ (x+1) * S(d*j) + S(d*i*j)
// over i <= x, j <= y - four independent BIT-able
    ↪ quantities, hence four BITs.
template <typename T = ll>
struct BIT2DRange {
    int n, m;
    vector<vector<T>> b[4];
    BIT2DRange(int n, int m) : n(n), m(m) {
        for (int k = 0; k < 4; k++) b[k].assign(n +
    ↪ 2, vector<T>(m + 2, 0));
    }
    void add(int x, int y, T v) {
    ↪          // d[x][y] += v (internal)
        if (x > n || y > m) return;
        T c[4] = {v, v * x, v * y, v * x * y};
        for (int i = x + 1; i <= n + 1; i += i & -i)
            for (int j = y + 1; j <= m + 1; j += j &
    ↪ -j)
                for (int k = 0; k < 4; k++)
    ↪ b[k][i][j] += c[k];
    }
    void upd(int x1, int y1, int x2, int y2, T v) {
    ↪          // add v to the rectangle
        add(x1, y1, v), add(x1, y2 + 1, -v), add(x2 +
    ↪ 1, y1, -v), add(x2 + 1, y2 + 1, v);
    }
    T pre(int x, int y) {
    ↪          // sum over [0..x] x [0..y]
        if (x < 0 || y < 0) return 0;
        T s[4] = {0, 0, 0, 0};
        for (int i = x + 1; i > 0; i -= i & -i)
            for (int j = y + 1; j > 0; j -= j & -j)
                for (int k = 0; k < 4; k++) s[k] +=
    ↪ b[k][i][j];
        return (T)(x + 1) * (y + 1) * s[0] - (T)(y +
    ↪ 1) * s[1] - (T)(x + 1) * s[2] + s[3];
    }
    T qry(int x1, int y1, int x2, int y2) {
        return pre(x2, y2) - pre(x1 - 1, y2) -
    ↪ pre(x2, y1 - 1) + pre(x1 - 1, y1 - 1);
    }
};
// MEMORY is 4 * n * m of your value type - 12.8 GB
    ↪ at 2000x2000 with ll. If that hurts, use int
// where safe, or compress one axis and go offline
    ↪ (03 - Offline2DFenwickTree).
// THE SAME EXPANSION generalises: range-add of a
    ↪ polynomial of degree k in each axis needs
// (k+1)^2 BITs. In 1D the arithmetic-progression
    ↪ case is 01 - Segment Tree/13.
// XOR instead of sum: the (x+1)(y+1) coefficients
    ↪ become parities, so keep four BITs and multiply
// by the parity of (x+1)*(y+1) etc. Usually simpler
    ↪ to just use a 2D segment tree of counts.
// SPARSE GRIDS: replace each inner vector by a hash
    ↪ map, or compress coordinates first - the outer
// structure is unchanged.
```

2.3 Sparse Table

2.3.1 SparseTable

SPARSE TABLE - O(1) range query after O(N log N) build, NO updates.

```
// The operation must be IDEMPOTENT (min, max, gcd,
    ↪ and, or) because the two covering blocks
// OVERLAP. For sum/product/xor - anything not
    ↪ idempotent - use 12 - DisjointSparseTable.cpp
// (still O(1) query) or a Fenwick tree. Guard l <=
    ↪ r: __lg(0) is undefined.
template<typename T, typename F = function<T(T, T)>>
struct SparseTable {
    int n;
    vector<vector<T>> mat;
    F func;

    SparseTable() = default;
    SparseTable(const vector<T>& a, F f = [](T x, T
    ↪ y) { return max(x, y); }) : n(a.size()), func(f)
    ↪ {
        int k = n ? __lg(n) + 1 : 0;
```

```
        mat.assign(k, a);
        for (int j = 1; j < k; j++) {
            for (int i = 0; i + (1 << j) <= n; i++) {
                mat[j][i] = func(mat[j - 1][i], mat[j
    ↪ - 1][i + (1 << (j - 1))]);
            }
        }
    }

    T query(int l, int r) const {
        int j = __lg(r - l + 1);
        return func(mat[j][l], mat[j][r - (1 << j) +
    ↪ 1]);
    }
};
```

2.3.2 SparseTable2D

2D Sparse Table - O(N * M * log N * log M) build, O(1) query

```
template<typename T = ll>
struct SparseTable2D {
    int n, m;
    // jmp[kr][kc][i][j] = max in 2^kr x 2^kc subgrid
    ↪ starting at (i, j)
    vector<vector<vector<vector<T>>>> jmp;

    SparseTable2D(const vector<vector<T>>& grid) {
        n = grid.size();
        if (!n) return;
        m = grid[0].size();
        if (!m) return;

        int log_n = __lg(n) + 1, log_m = __lg(m) + 1;
        jmp.assign(log_n,
    ↪ vector<vector<vector<T>>>(log_m,
    ↪ vector<vector<T>>(n, vector<T>(m))));

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                jmp[0][0][i][j] = grid[i][j];
            }
        }

        for (int kc = 1; kc < log_m; kc++) {
            for (int i = 0; i < n; i++) {
                for (int j = 0; j + (1 << kc) <= m;
    ↪ j++) {
                    jmp[0][kc][i][j] = max(jmp[0][kc
    ↪ - 1][i][j], jmp[0][kc - 1][i][j + (1 << (kc -
    ↪ 1))]);
                }
            }
        }

        for (int kr = 1; kr < log_n; kr++) {
            for (int kc = 0; kc < log_m; kc++) {
                for (int i = 0; i + (1 << kr) <= n;
    ↪ i++) {
                    for (int j = 0; j + (1 << kc) <=
    ↪ m; j++) {
                        jmp[kr][kc][i][j] =
    ↪ max(jmp[kr - 1][kc][i][j], jmp[kr - 1][kc][i +
    ↪ (1 << (kr - 1))][j]);
                    }
                }
            }
        }
    }

    T query(int r1, int c1, int r2, int c2) const {
        int kr = __lg(r2 - r1 + 1), kc = __lg(c2 - c1
    ↪ + 1);
        return max({jmp[kr][kc][r1][c1],
                    jmp[kr][kc][r1][c2 - (1 << kc) +
    ↪ 1],
                    jmp[kr][kc][r2 - (1 << kr) +
    ↪ 1][c1],
                    jmp[kr][kc][r2 - (1 << kr) +
    ↪ 1][c2 - (1 << kc) + 1]});
    }
};
```

2.3.3 SquareSparseTable

SQUARE SPARSE TABLE - O(1) query for any SQUARE sub-grid, O(N*M*log(min(N,M))) build.

```
// The 2D sparse table (02 - SparseTable2D.cpp)
    ↪ stores one table per (row-power, col-power) pair,
// so it costs log(N)*log(M) memory. If every query
    ↪ is a SQUARE - "best k x k block", "does some
// k x k window contain only ones", template matching
    ↪ on a grid - you only ever need the diagonal
// of that table, which is a factor of log cheaper in
    ↪ both time and memory.
// Query a k x k square: cover it with four
    ↪ overlapping 2^t x 2^t squares, t = floor(log2 k).
// The operation must be IDEMPOTENT
    ↪ (min/max/gcd/and/or) because those four squares
    ↪ overlap.
template<typename T = ll>
struct SquareSparseTable {
    int n, m;
    // jmp[k][i][j] = max of 2^k x 2^k square
    ↪ starting at (i, j)
    vector<vector<vector<T>>> jmp;

    SquareSparseTable(const vector<vector<T>>& grid) {
        n = grid.size();
        if (!n) return;
        m = grid[0].size();
        if (!m) return;

        int log_max = __lg(min(n, m)) + 1;
        jmp.assign(log_max, vector<vector<T>>(n,
    ↪ vector<T>(m)));

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                jmp[0][i][j] = grid[i][j];
            }
        }

        for (int k = 1; k < log_max; k++) {
            int pw = 1 << (k - 1);
            for (int i = 0; i + (pw * 2) <= n; i++) {
                for (int j = 0; j + (pw * 2) <= m;
    ↪ j++) {
                    jmp[k][i][j] = max({jmp[k -
    ↪ 1][i][j],
                                        jmp[k -
    ↪ 1][i][j + pw],
                                        jmp[k - 1][i
    ↪ + pw][j],
                                        jmp[k - 1][i
    ↪ + pw][j + pw]});
                }
            }
        }
```

```
    }

    // Queries max in square subgrid from (r1, c1) to
    ↪ (r2, c2)
    T query(int r1, int c1, int r2, int c2) const {
        int k = __lg(r2 - r1 + 1), pw = 1 << k;
        return max({jmp[k][r1][c1],
                    jmp[k][r1][c2 - pw + 1],
                    jmp[k][r2 - pw + 1][c1],
                    jmp[k][r2 - pw + 1][c2 - pw +
    ↪ 1]});
    }
};
```

2.3.4 SqrtTree

SQRT TREE - O(1) range query for ANY associative operation, WITH point updates in O(sqrt n).

```
// Build O(n log log n), memory O(n log log n).
// WHEN: the operation is associative but NOT
    ↪ idempotent and NOT invertible (matrix product,
    ↪ "max
// suffix sum", min-plus, string hash concat), so a
    ↪ sparse table (01) is wrong and a BIT is
// impossible - and you need O(1) queries or updates
    ↪ on top. If the op is idempotent use
// 01 - SparseTable; if there are no updates use 12 -
    ↪ DisjointSparseTable (O(n log n) build, O(1)
// query, 20 lines); if O(log n) queries are fine
    ↪ just use a segment tree.
// KEY INVARIANT: cut the array into sqrt(n) blocks
    ↪ of sqrt(n). Store prefix and suffix aggregates
// inside every block and a table `between[i][j]` for
    ↪ whole blocks i..j. A query then costs one
// suffix + one between + one prefix. Blocks are
    ↪ handled RECURSIVELY by the same structure, which
// is why the depth is log log n: n -> sqrt(n) ->
    ↪ n^(1/4) -> ... The top-level `between` table is
// itself a Sqrt Tree over the sqrt(n) block
    ↪ aggregates, so an update rebuilds only one block
    ↪ per
// layer plus one entry of the index structure.
struct SqrtTree {
    typedef ll T;
    static T op(T a, T b) { return a + b; }
    ↪          // ANY associative op
    int n, lg, isz;
    vector<T> v;
    vector<int> clz, layers, onLayer;
    vector<vector<T>> pref, suf, betw;
    void buildBlock(int k, int l, int r) {
        pref[k][l] = v[l];
        for (int i = l + 1; i < r; i++) pref[k][i] =
    ↪ op(pref[k][i - 1], v[i]);
        suf[k][r - 1] = v[r - 1];
        for (int i = r - 2; i >= l; i--) suf[k][i] =
    ↪ op(v[i], suf[k][i + 1]);
    }
    void buildBetween(int k, int lb, int rb, int off)
    ↪ {
        int sl = (layers[k] + 1) >> 1, cl = layers[k]
    ↪ >> 1, bs = 1 << sl;
        int bc = (rb - lb + bs - 1) >> sl;
        for (int i = 0; i < bc; i++) {
            T cur = T();
            for (int j = i; j < bc; j++) {
                T add = suf[k][lb + (j << sl)];
                cur = i == j ? add : op(cur, add);
                betw[k - 1][off + lb + (i << cl) + j]
    ↪ = cur;
            }
        }
    }
    void build(int k, int lb, int rb, int off) {
        if (k >= (int)layers.size()) return;
        int bs = 1 << ((layers[k] + 1) >> 1);
        for (int l = lb; l < rb; l += bs) {
            int r = min(l + bs, rb);
            buildBlock(k, l, r), build(k + 1, l, r,
    ↪ off);
        }
        if (k) buildBetween(k, lb, rb, off);
        else {
    ↪           // layer 0: rebuild the index array
            int sl = (lg + 1) >> 1;
            for (int i = 0; i < isz; i++) v[n + i] =
    ↪ suf[0][i << sl];
            build(1, n, n + isz, (1 << lg) - n);
        }
    }
    void update(int k, int lb, int rb, int off, int
    ↪ x) {
        if (k >= (int)layers.size()) return;
        int sl = (layers[k] + 1) >> 1, bs = 1 << sl,
    ↪ bi = (x - lb) >> sl;
        int l = lb + (bi << sl), r = min(l + bs, rb);
        buildBlock(k, l, r);
        if (k) buildBetween(k, lb, rb, off);
        else {
            v[n + bi] = suf[0][bi << ((lg + 1) >> 1)];
            update(1, n, n + isz, (1 << lg) - n, n +
    ↪ bi);
        }
        update(k + 1, l, r, off, x);
    }
    void update(int x, T val) { v[x] = val, update(0,
    ↪ 0, n, 0, x); }
    T query(int l, int r, int off, int base) {
    ↪           // inclusive [l, r]
        if (l == r) return v[l];
        if (l + 1 == r) return op(v[l], v[r]);
        int k = onLayer[clz[(l - base) ^ (r - base)]];
        int sl = (layers[k] + 1) >> 1, cl = layers[k]
    ↪ >> 1;
        int lb = (((l - base) >> layers[k]) <<
    ↪ layers[k]) + base;
        int lblk = ((l - lb) >> sl) + 1, rblk = ((r -
    ↪ lb) >> sl) - 1;
        T res = suf[k][l];
        if (lblk <= rblk)
            res = op(res, k ? betw[k - 1][off + lb +
    ↪ (lblk << cl) + rblk]
                            : query(n + lblk, n +
    ↪ rblk, (1 << lg) - n, n));
        return op(res, pref[k][r]);
    }
    T query(int l, int r) { return query(l, r, 0, 0);
    ↪ }
    SqrtTree(const vector<T>& a) : n(a.size()),
    ↪ lg(0), v(a) {
        while ((1 << lg) < n) lg++;
        clz.assign(1 << lg, 0), onLayer.assign(lg +
    ↪ 1, 0);
        for (int i = 1; i < (1 << lg); i++) clz[i] =
    ↪ clz[i >> 1] + 1;
        for (int t = lg; t > 1; t = (t + 1) >> 1)
    ↪ onLayer[t] = layers.size(), layers.push_back(t);
```

```
        for (int i = lg - 1; i >= 0; i--) onLayer[i]
    ↪ = max(onLayer[i], onLayer[i + 1]);
        int sl = (lg + 1) >> 1, bs = 1 << sl;
        isz = (n + bs - 1) >> sl;
        v.resize(n + isz);
        pref.assign(layers.size(), vector<T>(n +
    ↪ isz));
        suf.assign(layers.size(), vector<T>(n + isz));
        betw.assign(max(0, (int)layers.size() - 1),
    ↪ vector<T>((1 << lg) + bs));
        build(0, 0, n, 0);
    }
};
// NON-COMMUTATIVE ops are fine and are the whole
    ↪ point - op(a, b) is always applied left to right.
// T MUST have a sensible default constructor; it is
    ↪ never used as an identity, only as a filler.
// THE O(1)-QUERY / O(sqrt n)-UPDATE trade is
    ↪ unusual. Its real niche is "answer 1e6 queries
    ↪ of a
// non-invertible op with occasional updates". If
    ↪ updates outnumber queries, a segment tree wins.
// THE TWO-LEVEL VERSION (one layer of blocks, no
    ↪ recursion) is 20 lines and gives O(1) query with
// O(n) build only if you can afford a sqrt(n) x
    ↪ sqrt(n) between-table; recursion is what makes
    ↪ the
// update cheap. Write the two-level one first if the
    ↪ array is static and small.
```

2.4 Trie

2.4.1 StringTrie

PREFIX TRIE over an alphabet. O(L) per insert/query. Reach for it for prefix counting, autocomplete,

```
// "is any inserted word a prefix of this one", and
    ↪ as the skeleton Aho-Corasick is built on.
// Prefix Trie for Strings
// Time: O(L) per insert/query, Space: O(N * L *
    ↪ Alphabet)
struct Trie {
    vector<vector<int>> nxt;
    vector<int> cnt, end_cnt;
    int alpha;

    Trie(int alpha = 26) : alpha(alpha) {
        add_node();
    }

    int add_node() {
        nxt.emplace_back(alpha, -1);
        cnt.emplace_back(0);
        end_cnt.emplace_back(0);
        return nxt.size() - 1;
    }

    void insert(const string& s, int val = 1) {
        int u = 0;
        cnt[u] += val;
        for (char c : s) {
            int ch = c - 'a';
            if (nxt[u][ch] == -1) nxt[u][ch] =
    ↪ add_node();
            u = nxt[u][ch];
            cnt[u] += val;
        }
        end_cnt[u] += val;
    }
    int count_prefix(const string& s) {
        int u = 0;
        for (char c : s) {
            int ch = c - 'a';
            if (nxt[u][ch] == -1) return 0;
            u = nxt[u][ch];
        }
        return cnt[u];
    }
};
```

2.4.2 BinaryTrie

Binary trie over [0, 2^(B+1)). insert / ERASE / query all O(B).

```
// erase() is what lets you keep the trie equal to
    ↪ the current root->node path during a DFS
// ("min XOR with any ancestor"): add on entry,
    ↪ add(-1) on exit.
struct BTrie {
    static const int B = 30;
    vector<array<int, 2>> ch = {{-1, -1}};
    vector<int> cnt = {0};

    int node() { ch.push_back({-1, -1});
    ↪ cnt.push_back(0); return ch.size() - 1; }
    void add(ll x, int d = 1) {
    ↪   // d = +1 insert, -1 erase
        int u = 0; cnt[0] += d;
        for (int i = B; i >= 0; i--) {
            int b = x >> i & 1;
            if (ch[u][b] < 0) ch[u][b] = node();
            u = ch[u][b], cnt[u] += d;
        }
    }
    int cn(int u, int b) const { return u < 0 ||
    ↪ ch[u][b] < 0 ? 0 : cnt[ch[u][b]]; }
    int size() const { return cnt[0]; }

    ll bestXor(ll x, bool mx) const {
    ↪   // mx=1 -> max x^y, mx=0 -> min x^y
        if (!cnt[0]) return -1;
        int u = 0; ll r = 0;
        for (int i = B; i >= 0; i--) {
            int b = (x >> i & 1) ^ mx;
    ↪   // bit we'd like y to have
            if (!cn(u, b)) b ^= 1;
            r |= (ll)((x >> i & 1) ^ b) << i;
            u = ch[u][b];
        }
        return r;
    }
    ll maxXor(ll x) const { return bestXor(x, 1); }
    ll minXor(ll x) const { return bestXor(x, 0); }

    int cntLess(ll x, ll k) const {
    ↪   // #{y in trie : (x ^ y) < k}
        int u = 0, r = 0;
        for (int i = B; i >= 0 && u >= 0; i--) {
            int xb = x >> i & 1;
            if (k >> i & 1) r += cn(u, xb), u =
    ↪ ch[u][xb ^ 1];   // xor-bit 0 branch is all < k
            else u = ch[u][xb];
        }
        return r;
    }
};
```

2.4.3 PersistentTrie

Persistent Binary Trie - Space: O(N * BITS), Time: O(BITS) per insert/query

```
// Supports versioning and range max XOR queries between versions [l_ver, r_ver]
struct PersistentTrie {
    struct Node {
        int cnt = 0;
        array<int, 2> nxt = {-1, -1};
    };

    vector<Node> tree;
    vector<int> roots;
    int max_bit;

    PersistentTrie(int max_bit = 30) : max_bit(max_bit) {
        roots.pb(add_node());
    }

    int add_node(int old_node = -1) {
        if (old_node != -1) {
            tree.pb(tree[old_node]);
        } else {
            tree.pb(Node());
        }
        return tree.size() - 1;
    }

    // Inserts x starting from prev_root and returns the new version root
    int insert(int prev_root, ll x) {
        int new_root = add_node(prev_root);
        int u = new_root, prev_u = prev_root;
        tree[u].cnt++;

        for (int i = max_bit; i >= 0; i--) {
            int b = (x >> i) & 1;
            int prev_child = (prev_u != -1) ? tree[prev_u].nxt[b] : -1;
            tree[u].nxt[b] = add_node(prev_child);
            u = tree[u].nxt[b];
            if (prev_u != -1) prev_u = tree[prev_u].nxt[b];
            tree[u].cnt++;
        }
        roots.pb(new_root);
        return new_root;
    }

    // Returns max (x ^ val) over elements inserted in version range [l_ver, r_ver]
    ll max_xor(int l_ver, int r_ver, ll x) const {
        int u_r = roots[r_ver], u_l = (l_ver > 0) ? roots[l_ver - 1] : -1;
        ll res = 0;

        for (int i = max_bit; i >= 0; i--) {
            int b = (x >> i) & 1;
            int want = b ^ 1;

            int count_r = (u_r != -1 && tree[u_r].nxt[want] != -1) ? tree[tree[u_r].nxt[want]].cnt : 0;
            int count_l = (u_l != -1 && tree[u_l].nxt[want] != -1) ? tree[tree[u_l].nxt[want]].cnt : 0;
            if (count_r - count_l > 0) {
                res |= (1LL << i);
                u_r = tree[u_r].nxt[want];
                u_l = (u_l != -1 && tree[u_l].nxt[want] != -1) ? tree[u_l].nxt[want] : -1;
            } else {
                u_r = (u_r != -1 && tree[u_r].nxt[b] != -1) ? tree[u_r].nxt[b] : -1;
                u_l = (u_l != -1 && tree[u_l].nxt[b] != -1) ? tree[u_l].nxt[b] : -1;
            }
        }
        return res;
    }
};
```

2.5 Disjoint Set Union (DSU)

2.5.1 DSU

Disjoint Set Union (DSU) - Path Compression & Union by Size

```
// Time: O(alpha(N)) per operation
struct DSU {
    vector<int> par, sz;

    DSU(int n = 0) : par(n), sz(n, 1) {
        iota(all(par), 0);
    }

    int find(int x) {
        return par[x] == x ? x : par[x] = find(par[x]);
    }

    bool same(int x, int y) {
        return find(x) == find(y);
    }

    bool join(int x, int y) {
        x = find(x);
        y = find(y);
        if (x == y) return false;
        if (sz[x] < sz[y]) swap(x, y);
        sz[x] += sz[y];
        par[y] = x;
        return true;
    }

    int size(int x) {
        return sz[find(x)];
    }
};
```

2.5.2 DSURollback

DSU with Rollback - No path compression to allow backtracking

```
// Time: O(log N) per operation, O(1) rollback
struct DSURollback {
    vector<int> par, siz;
    vector<pair<int, int>> history; // {child_node, old_parent_size}

    DSURollback(int n = 0) : par(n), siz(n, 1) {
        iota(all(par), 0);
    }

    int find(int x) {
        return par[x] == x ? x : find(par[x]);
    }
```

```
    bool same(int x, int y) {
        return find(x) == find(y);
    }

    bool join(int x, int y) {
        x = find(x);
        y = find(y);
        if (x == y) {
            history.eb(-1, -1);
            return false;
        }
        if (siz[x] < siz[y]) swap(x, y);
        history.eb(y, siz[x]);
        siz[x] += siz[y];
        par[y] = x;
        return true;
    }

    int size(int x) {
        return siz[find(x)];
    }

    void rollback() {
        if (history.empty()) return;
        auto [y, old_sz_x] = history.back();
        history.pop_back();
        if (y != -1) {
            siz[par[y]] = old_sz_x;
            par[y] = y;
        }
    }

    int get_state() const {
        return history.size();
    }

    void rollback_to(int state) {
        while ((int)history.size() > state)
  ↪ rollback();
    }
};
```

2.5.3 WeightedDSU

DSU with POTENTIALS: maintains pot[v] = value(v) - value(root), so you can answer

```
// "what is value(u) - value(v)" and detect
  ↪ contradictions.
// Group is (Z, +) here; for parity/bipartiteness use
  ↪ XOR: change ALL FOUR of pot[x] += p -> ^=,
// pu - pv == w -> (pu ^ pv) == w, pot[rv] = pu - pv
  ↪ - w -> pu ^ pv ^ w, and DELETE the w = -w line
// (negation is not the inverse in GF(2)), or Z_k for
  ↪ mod-k relations.
struct WDSU {
    vector<int> par;
    vector<ll> pot;                                 //
  ↪ potential relative to the parent
    vector<int> sz;

    WDSU(int n = 0) : par(n), pot(n, 0), sz(n, 1) {
  ↪ iota(all(par), 0); }
    pair<int, ll> find(int x) {                     //
  ↪ {root, value(x) - value(root)}
        if (par[x] == x) return {x, 0};
        auto [r, p] = find(par[x]);
        par[x] = r, pot[x] += p;                    //
  ↪ path compression accumulates
        return {r, pot[x]};
    }
    // assert value(u) - value(v) == w. false =
  ↪ CONTRADICTION with what we already know.
    bool join(int u, int v, ll w) {
        auto [ru, pu] = find(u);
        auto [rv, pv] = find(v);
        if (ru == rv) return pu - pv == w;
        if (sz[ru] < sz[rv]) swap(ru, rv), swap(pu,
  ↪ pv), w = -w;
        par[rv] = ru, pot[rv] = pu - pv - w, sz[ru]
  ↪ += sz[rv];
        return true;
    }
    // value(u) - value(v), or nullopt if they are
  ↪ not related yet
    bool diff(int u, int v, ll& out) {
        auto [ru, pu] = find(u);
        auto [rv, pv] = find(v);
        if (ru != rv) return false;
        out = pu - pv;
        return true;
    }
};
// BIPARTITENESS under edge additions: use XOR
  ↪ potentials with w = 1 per edge; join() returning
// false means an odd cycle appeared. Same structure
  ↪ answers "are u and v the same colour".
```

2.5.4 RangeParallelDSU

RANGE-PARALLEL DSU ("DSU on a sparse table") - merge two whole RANGES elementwise in O(log n).

```
// The operation is: "a[u..u+len-1] must equal
  ↪ a[v..v+len-1] elementwise", i.e. union u+i with
  ↪ v+i
// for every i < len. Doing that naively is O(len)
  ↪ per constraint; this does it in O(log n).
// IDEA: keep LOGN separate DSUs. par[k] answers
  ↪ "these two blocks of length 2^k are identical".
// A range constraint of length len is covered by TWO
  ↪ blocks of length 2^k (k = floor(log2 len)),
// exactly like a sparse-table query - so one union
  ↪ at level k, twice. Afterwards, PUSH DOWN:
// if two blocks of length 2^k are equal then their
  ↪ two halves of length 2^(k-1) are equal too.
// After the push-down, level 0 holds the true
  ↪ elementwise DSU.
// USES: "count the strings/arrays satisfying these
  ↪ equal-substring constraints" (the answer is
// alphabet^(#components at level 0)), grid/string
  ↪ pattern unification, and any problem that says
// "these two intervals must match".
struct RangeDSU {
    int n, LOG;
    vector<vector<int>> par;
    RangeDSU(int n) : n(n), LOG(__lg(max(n, 1)) + 1),
  ↪ par(LOG, vector<int>(n)) {
        for (int k = 0; k < LOG; k++)
  ↪ iota(all(par[k]), 0);
    }
    int find(int k, int u) { return par[k][u] == u ?
  ↪ u : par[k][u] = find(k, par[k][u]); }
    void join(int k, int u, int v) {
        u = find(k, u), v = find(k, v);
        if (u != v) par[k][u] = v;
    }
    // a[u .. u+len-1] equals a[v .. v+len-1],
  ↪ elementwise. O(log n) amortised.
```

```
    void equalRanges(int u, int v, int len) {
        if (len <= 0) return;
        int k = __lg(len);
        join(k, u, v);
        join(k, u + len - (1 << k), v + len - (1 <<
    ↪ k));
    }
    // Call ONCE after all the constraints. Then
    ↪ find(0, i) is the real elementwise component.
    void pushDown() {
        for (int k = LOG - 1; k > 0; k--) {
            int half = 1 << (k - 1);
            for (int i = 0; i + (1 << k) <= n; i++) {
                int j = find(k, i);
                if (j == i) continue;
                join(k - 1, i, j);
    ↪               // both halves inherit it
                join(k - 1, i + half, j + half);
            }
        }
    }
    int components() {
    ↪               // at level 0, after pushDown
        int c = 0;
        for (int i = 0; i < n; i++) if (find(0, i) ==
    ↪ i) c++;
        return c;
    }
};
// The same "cover a range with two power-of-two
  ↪ blocks, then push down" trick works for any
// idempotent relation, not just equality - it is the
  ↪ DSU analogue of a sparse table.
// If instead you need to union a range of
  ↪ CONSECUTIVE elements to each other ("a[l..r] all
  ↪ equal"),
// that is much simpler: a "next unvisited pointer"
  ↪ DSU where find(i) jumps to the next index not
// yet merged, giving O(n alpha) total over all
  ↪ range-unions.
```

2.5.5 PartiallyPersistentDSU

PARTIALLY PERSISTENT DSU - a DSU you may QUERY at any past moment, while only the present can be

```
// modified. O(log n) per query, O(log n) per union,
  ↪ and only O(n) memory - far cheaper than the
// fully persistent version in 20 -
  ↪ PersistentAndUndo, which costs O(log^2 n) and
  ↪ O(n log n).
// WHEN: "what is the earliest time at which u and v
  ↪ are connected", "how big was u's component
// after the first t edges", offline MST /
  ↪ bottleneck-path questions. Partial persistence
  ↪ is enough
// whenever the timeline is a LINE and not a tree of
  ↪ branching versions - which is nearly always.
// If you also want to UNDO, use 02 - DSURollback; if
  ↪ you want to delete arbitrary edges, use
// 14 - OfflineDynamicConnectivity.
// KEY INVARIANT: NO path compression and union by
  ↪ size, so once a vertex becomes a child it keeps
// that parent FOREVER. Every vertex therefore has a
  ↪ monotone timeline: a list of component sizes
// while it was a root, then (at most) one final "I
  ↪ became a child of p at time t" entry. Walking
// up while the stamp is <= t reconstructs the forest
  ↪ at time t exactly, and the height stays
// O(log n) because of the union by size.
struct PartialDSU {
    vector<vector<pair<int, int>>> par;
  ↪          // {-size or parent, timestamp}
    int T = 0;
  ↪          // number of unions performed
    PartialDSU(int n) : par(n, {{-1, 0}}) {}
    int root(int u, int t) {
  ↪          // representative of u at time t
        auto b = par[u].back();
        return b.fi >= 0 && b.se <= t ? root(b.fi, t)
  ↪ : u;
    }
    bool same(int u, int v, int t) { return root(u,
  ↪ t) == root(v, t); }
    int size(int u, int t) {
  ↪          // |component of u| at time t
        u = root(u, t);
        int lo = 0, hi = par[u].size() - 1, at = 0;
        while (lo <= hi) {
  ↪          // last size entry stamped <= t
            int m = (lo + hi) / 2;
            par[u][m].se <= t ? at = m, lo = m + 1 :
  ↪ hi = m - 1;
        }
        return -par[u][at].fi;
    }
    bool join(int u, int v) {
  ↪          // happens at time ++T
        ++T;
        if ((u = root(u, T)) == (v = root(v, T)))
  ↪ return false;
        if (par[u].back().fi > par[v].back().fi)
  ↪ swap(u, v);   // u is the bigger (more negative)
        par[u].push_back({par[u].back().fi +
  ↪ par[v].back().fi, T});
        par[v].push_back({u, T});
        return true;
    }
    int firstConnected(int u, int v) {
  ↪          // earliest time, or -1 if never
        int lo = 1, hi = T, ans = -1;
        while (lo <= hi) {
            int m = (lo + hi) / 2;
            same(u, v, m) ? ans = m, hi = m - 1 : lo
  ↪ = m + 1;
        }
        return ans;
    }
};
// TIME IS THE UNION COUNTER, not the query index: a
  ↪ query "after the first k edges" must first map
// k to the number of SUCCESSFUL unions among those k
  ↪ edges (keep an array while you insert).
// BOTTLENECK PATH / minimax edge: sort the edges by
  ↪ weight, union them in that order, then
// firstConnected(u, v) gives the index of the edge
  ↪ that answers "minimise the maximum edge on a
// path". That is exactly the Kruskal reconstruction
  ↪ tree (03 - Graph/08 - MST) in 25 lines less
// code, at O(log^2 n) per query instead of O(1) -
  ↪ use the KRT if you have many queries.
// OFFLINE ALTERNATIVE: if every query is known up
  ↪ front, PARALLEL BINARY SEARCH (01 - Misc/17)
// answers all of them with one plain DSU and no
  ↪ persistence at all.
```

2.6 Square Root Decomposition

2.6.1 MoAlgorithm

MO'S ALGORITHM - answers m OFFLINE range queries in O((n + m) sqrt n) when you can move an endpoint

```
// by one in O(1) (add/remove an element and maintain the answer). Sort queries by block of l, then by
// r (alternating direction per block). Use it for distinct counts, frequency-of-frequency, XOR pairs.
class MoArray {
private:
    struct Query {
        int l, r, idx;
        int bnd;

        bool operator<(const Query& other) const {
            if (bnd != other.bnd)
                return bnd < other.bnd;
            return (bnd & 1) ? (r < other.r) : (r > other.r);
        }
    };

    int n;
    int block_size;
    vector<int> a;

    // --- PROBLEM SPECIFIC STATE ---
    long long ans;
    vector<int> freq;

    inline void add(int idx) {
        if (freq[a[idx]] == 0) ans++;
        freq[a[idx]]++;
    }

    inline void remove(int idx) {
        freq[a[idx]]--;
        if (freq[a[idx]] == 0) ans--;
    }

    inline long long get_answer() const {
        return ans;
    }
    // ------------------------------

public:
    MoArray(const vector<int>& arr) : n(arr.size()), a(arr) {
        int max_val = 0;
        for (int x : a) {
            max_val = max(max_val, x);
        }
        freq.assign(max_val + 1, 0);
        ans = 0;
    }

    vector<long long> solve(const vector<pair<int, int>>& raw_queries) {
        int q = raw_queries.size();
        if (q == 0) return {};

        block_size = max(1, (int)(n / sqrt(q)));
        vector<Query> queries(q);

        for (int i = 0; i < q; ++i) {
            queries[i].l = raw_queries[i].first;
            queries[i].r = raw_queries[i].second;
            queries[i].idx = i;
            queries[i].bnd = queries[i].l / block_size;
        }

        sort(queries.begin(), queries.end());

        vector<long long> answers(q);
        int cur_l = 0, cur_r = -1;

        for (const Query& qry : queries) {
            while (cur_l > qry.l) add(--cur_l);
            while (cur_r < qry.r) add(++cur_r);
            while (cur_l < qry.l) remove(cur_l++);
            while (cur_r > qry.r) remove(cur_r--);
            answers[qry.idx] = get_answer();
        }

        return answers;
    }
};
```

2.6.2 MoOnSubtrees

MO'S ON SUBTREES - flatten the tree with an Euler tour so every subtree is a contiguous range,

```
// then run plain Mo's on it. Same O((n + q) sqrt n). Use for "query over the subtree of v" when no
// online structure fits (distinct colours in a subtree, k-th most frequent, etc.).
class MoSubtree {
private:
    struct Query {
        int l, r, idx;
        int bnd;

        bool operator<(const Query& other) const {
            if (bnd != other.bnd) return bnd < other.bnd;
            return (bnd & 1) ? (r < other.r) : (r > other.r);
        }
    };

    int n;
    int block_size;
    vector<vector<int>> adj;
    vector<int> in, out, flat, clr;

    // --- PROBLEM SPECIFIC STATE ---
    int mx_freq;
    vector<int> freq;
    vector<long long> sum_freq;

    inline void add(int node_idx) {
        int c = clr[node_idx];
        sum_freq[freq[c]] -= c;
        freq[c]++;
        sum_freq[freq[c]] += c;
        if (freq[c] > mx_freq) mx_freq = freq[c];
    }

    inline void remove(int node_idx) {
        int c = clr[node_idx];
        sum_freq[freq[c]] -= c;
        freq[c]--;
        sum_freq[freq[c]] += c;
        while (mx_freq > 0 && sum_freq[mx_freq] == 0) mx_freq--;
    }
```

```
    inline long long get_answer() const {
        return sum_freq[mx_freq];
    }
    // ------------------------------

    void dfs(int u, int p) {
        in[u] = flat.size();
        flat.push_back(u);
        for (int v : adj[u]) {
            if (v == p) continue;
            dfs(v, u);
        }
        out[u] = flat.size() - 1;
    }

public:
    MoSubtree(int n, int max_val, const vector<int>&
    ↪ colors)
        : n(n), adj(n + 1), in(n + 1), out(n + 1),
    ↪ clr(colors) {
        mx_freq = 0;
        freq.assign(max_val + 1, 0);
        sum_freq.assign(n + 1, 0);
    }

    void add_edge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    vector<long long> solve(int root, const
    ↪ vector<int>& query_nodes) {
        flat.reserve(n);
        dfs(root, 0);

        int q = query_nodes.size();
        if (q == 0) return {};

        block_size = max(1, (int)(n / sqrt(q)));
        vector<Query> queries(q);

        for (int i = 0; i < q; ++i) {
            int u = query_nodes[i];
            queries[i].l = in[u];
            queries[i].r = out[u];
            queries[i].idx = i;
            queries[i].bnd = queries[i].l /
    ↪ block_size;
        }

        sort(queries.begin(), queries.end());

        vector<long long> answers(q);
        int cur_l = 0, cur_r = -1;

        for (const Query& qry : queries) {
            while (cur_l > qry.l) add(flat[--cur_l]);
            while (cur_r < qry.r) add(flat[++cur_r]);
            while (cur_l < qry.l)
    ↪ remove(flat[cur_l++]);
            while (cur_r > qry.r)
    ↪ remove(flat[cur_r--]);
            answers[qry.idx] = get_answer();
        }

        return answers;
    }
};
```

2.6.3 MoOnPaths

MO'S ON TREE PATHS - the Euler tour trick where a vertex appearing TWICE in the range cancels out,

```
// plus the LCA added back by hand. Answers path
  ↪ queries offline in O((n + q) sqrt n) with no HLD.
class MoTreePath {
private:
    int n, LOG, block_size;
    vector<vector<int>> adj, anc;
    vector<int> depth, in, out, flat;
    vector<long long> up, a;
    vector<bool> exist;

    // --- PROBLEM SPECIFIC STATE ---
    long long ans;
    vector<deque<int>> dep; // Tracks nodes by depth

    void add(int idx) {
        if (!dep[depth[idx]].empty()) {
            ans += 1ll * a[dep[depth[idx]].back()] *
    ↪ a[idx];
        }
        dep[depth[idx]].push_back(idx);
    }

    void remove(int idx) {
        if (dep[depth[idx]].size() == 2) {
            ans -= a[*dep[depth[idx]].begin()] *
    ↪ a[*next(dep[depth[idx]].begin())];
        }
        if (dep[depth[idx]].front() == idx) {
            dep[depth[idx]].pop_front();
        } else {
            dep[depth[idx]].pop_back();
        }
    }

    void update(int idx) {
        int node = flat[idx];
        if (exist[node]) {
            remove(node);
        } else {
            add(node);
        }
        exist[node] = !exist[node];
    }

    inline long long get_answer() const {
        return ans;
    }
    // ------------------------------

    void dfs(int u, int p = -1) {
        if (p != -1) {
            anc[u][0] = p;
            up[u] = a[u] * a[u] + up[p];
        } else {
            depth[u] = 0;
            up[u] = a[u] * a[u];
        }
        in[u] = flat.size();
        flat.push_back(u);

        for (int v : adj[u]) {
            if (v == p) continue;
            depth[v] = depth[u] + 1;
```

```
            dfs(v, u);
        }

        out[u] = flat.size();
        flat.push_back(u); // Push again for Eulerian
    ↪ tour
    }

    void build_lca() {
        for (int k = 1; k < LOG; k++) {
            for (int i = 1; i <= n; i++) {
                anc[i][k] = anc[anc[i][k - 1]][k - 1];
            }
        }
    }

    int lift(int x, int k) {
        for (int bit = 0; bit < LOG; bit++) {
            if ((1 << bit) & k) x = anc[x][bit];
        }
        return x;
    }

    int lca(int u, int v) {
        if (depth[u] < depth[v]) swap(u, v);
        u = lift(u, depth[u] - depth[v]);
        if (u == v) return u;
        for (int bit = LOG - 1; bit >= 0; bit--) {
            if (anc[u][bit] != anc[v][bit]) {
                u = anc[u][bit];
                v = anc[v][bit];
            }
        }
        return anc[u][0];
    }

public:
    struct Query {
        int l, r, idx, x = -1;
        int bnd;

        bool operator<(const Query& other) const {
            if (bnd != other.bnd) return bnd <
    ↪ other.bnd;
            return (bnd & 1) ? (r < other.r) : (r >
    ↪ other.r);
        }
    };

    MoTreePath(int n_nodes, const vector<long long>&
    ↪ node_values)
        : n(n_nodes), a(node_values),
    ↪ LOG(log2(n_nodes) + 2) {

        adj.assign(n + 1, vector<int>());
        anc.assign(n + 1, vector<int>(LOG, 0));
        depth.assign(n + 1, 0);
        in.assign(n + 1, 0);
        out.assign(n + 1, 0);
        up.assign(n + 1, 0);
        exist.assign(n + 1, false);
        dep.assign(n + 1, deque<int>());
        ans = 0;
    }

    void add_edge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    // PRIMARY ENTRY POINT: give it the paths as
    ↪ vertex pairs and it builds the queries itself.
    // (in[], out[] and lca() are only valid AFTER
    ↪ the dfs, so you cannot build them yourself.)
    vector<long long> solve(int root, const
    ↪ vector<pair<int, int>>& paths) {
        flat.clear();
        dfs(root);
        build_lca();
        vector<Query> queries;
        for (int i = 0; i < (int)paths.size(); i++) {
            int u = paths[i].first, v =
    ↪ paths[i].second;
            if (in[u] > in[v]) swap(u, v);
            int w = lca(u, v);
            // u inside v's subtree -> plain range;
    ↪ otherwise use out[u]..in[v] and add back the LCA
            if (w == u) queries.push_back({in[u],
    ↪ in[v], i, -1, 0});
            else queries.push_back({out[u], in[v], i,
    ↪ w, 0});
        }
        return solveRanges(queries);
    }
    vector<long long> solveRanges(vector<Query>&
    ↪ queries) {
        int q = queries.size();
        if (q == 0) return {};

        block_size = max(1, (int)(flat.size() /
    ↪ sqrt(q)));

        for (auto& qry : queries) {
            qry.bnd = qry.l / block_size;
        }

        sort(queries.begin(), queries.end());

        vector<long long> answers(q);
        int cur_l = 0, cur_r = -1;

        for (const Query& qry : queries) {
            while (cur_l > qry.l) update(--cur_l);
    ↪    // GROW first, then shrink - the other
            while (cur_r < qry.r) update(++cur_r);
    ↪    // order can pass through an invalid window
            while (cur_l < qry.l) update(cur_l++);
            while (cur_r > qry.r) update(cur_r--);

            answers[qry.idx] = get_answer() + (qry.x
    ↪ != -1 ? up[qry.x] : 0);
        }
        return answers;
    }
};
```

2.6.4 BlockDecomposition

SQRT / BLOCK DECOMPOSITION. Split [0,n) into blocks of B ~ sqrt(n) and keep one aggregate per

```
// block. Point update O(1..B), range query O(sqrt
  ↪ n). A BIT/segtree beats it for sum, so reach for
// this when the aggregate is NOT a group / has no
  ↪ lazy tag: block-sorted arrays, block frequency
// tables, "count values > x in [l,r]", range-assign
  ↪ + weird query, or a per-block rebuild that is
// cheap. General shape below with sum; swap ‘agg‘
  ↪ and the two combine lines for anything else.
struct Blocks {
    int n, B;
    vector<ll> a, agg;
  ↪          // agg[b] = aggregate of block b
    Blocks(const vector<ll>& v) : n(v.size()),
  ↪ B(max(1, (int)sqrt(n) + 1)), a(v),
                                  agg((n + B - 1) /
  ↪ B, 0) {
        for (int i = 0; i < n; i++) agg[i / B] +=
  ↪ a[i];
    }
    void set(int i, ll v) { agg[i / B] += v - a[i],
  ↪ a[i] = v; } // O(1); for non-group aggregates

  ↪          // rebuild the block instead, O(B)
    ll query(int l, int r) {
  ↪          // inclusive, O(n/B + B)
        ll s = 0;
        int bl = l / B, br = r / B;
        if (bl == br) { for (int i = l; i <= r; i++)
  ↪ s += a[i]; return s; }
        for (int i = l; i < (bl + 1) * B; i++) s +=
  ↪ a[i];
        for (int b = bl + 1; b < br; b++) s += agg[b];
        for (int i = br * B; i <= r; i++) s += a[i];
        return s;
    }
};
// RANGE UPDATE, RANGE QUERY without lazy
  ↪ propagation: keep add[b] per block. A partial
  ↪ block is
// updated element-wise and its aggregate rebuilt; a
  ↪ full block only bumps add[b]. Query adds
// add[b] * (elements taken from b).
// BLOCK-SORTED ARRAY ("sqrt tree"): store each block
  ↪ also as a sorted copy. Then
//   count of values <= x in [l,r] = partial blocks
  ↪ scanned + binary search inside full blocks,
//   O(sqrt(n) log n) per query, O(B log B) per point
  ↪ update (re-sort one block). This is the
//   offline-free alternative to a wavelet tree /
  ↪ merge-sort tree when updates are present.
// REBUILD TRICK: when an operation is expensive but
  ↪ rare (e.g. inserting into the middle), buffer
// changes and rebuild the whole decomposition every
  ↪ sqrt(q) operations -> O((n+q) sqrt q) total.
// CHOOSING B: query cost n/B + B*c is minimised at B
  ↪ = sqrt(n*c) where c is the per-element cost;
// if updates are much more frequent than queries,
  ↪ take bigger blocks, and vice versa.
```

2.6.5 MoWithUpdates

MO'S WITH UPDATES (3D Mo). Queries {l, r, t} where t = #updates applied before this query.

```
// Block size n^(2/3) gives O(n^(5/3)) total. Use
  ↪ when the array changes between queries and no
// online structure fits.
struct MoUpd {
    struct Q { int l, r, t, idx; };
    struct U { int pos, oldV, newV; };

    int n, B;
    vector<int> a, rec;
  ↪ // a = state at time 0; rec = while recording
    vector<Q> qs;
    vector<U> us;

    // ---- problem state: #distinct in the window
  ↪ ----
    vector<int> cnt;
    int cur = 0;
    void add(int v) { if (cnt[v]++ == 0) cur++; }
    void rem(int v) { if (--cnt[v] == 0) cur--; }
    int answer() const { return cur; }
    //
  ↪ ------------------------------------------------

    MoUpd(vector<int> arr, int maxVal) :
  ↪ n(arr.size()), a(arr), rec(arr), cnt(maxVal + 1,
  ↪ 0) {
        B = max(1, (int)pow((double)max(n, 1), 2.0 /
  ↪ 3));
    }
    void addQuery(int l, int r) { qs.push_back({l, r,
  ↪ (int)us.size(), (int)qs.size()}); }
    void addUpdate(int pos, int newV) {
  ↪ us.push_back({pos, rec[pos], newV}); rec[pos] =
  ↪ newV; }

    void applyU(int i, int l, int r) {
  ↪ // step over update i, forwards or backwards
        auto& u = us[i];
        if (l <= u.pos && u.pos <= r) rem(a[u.pos]),
  ↪ add(u.newV);
        a[u.pos] = u.newV;
        swap(u.oldV, u.newV);
  ↪  // makes the operation its own inverse
    }
    vector<int> solve() {
        sort(all(qs), [&](const Q& x, const Q& y) {
            int bx = x.l / B, by = y.l / B;
            if (bx != by) return bx < by;
            int cx = x.r / B, cy = y.r / B;
            if (cx != cy) return (bx & 1) ? cx > cy :
  ↪ cx < cy;
            return (cx & 1) ? x.t > y.t : x.t < y.t;
        });
        vector<int> res(qs.size());
        int L = 0, R = -1, T = 0;
        for (auto& q : qs) {
            while (T < q.t) applyU(T++, L, R);
            while (T > q.t) applyU(--T, L, R);
            while (R < q.r) add(a[++R]);
            while (L > q.l) add(a[--L]);
            while (R > q.r) rem(a[R--]);
            while (L < q.l) rem(a[L++]);
            res[q.idx] = answer();
        }
        return res;
```

```
    }
};
// NOTE: 'a' stays at time 0; 'rec' tracks values while you record updates, so applyU() can
// step forwards AND backwards through time using the same swap-based inverse.
```

2.6.6 MoWithRollback

MO'S WITH ROLLBACK - for structures where ADD is easy but REMOVE is hard or impossible

```
// (running maximum, DSU, "best pair so far", knapsack-ish state).
// Per block of left endpoints: sort by r, keep a persistent right part that only grows, and
// rebuild the short left part from scratch for every query, rolling back afterwards.
// O((n + q) sqrt n) adds, ZERO removes.
struct MoRollback {
    struct Q { int l, r, idx; };
    int n, B;
    vector<ll> a;
    vector<Q> qs;

    // ---- problem state: max (value * count) seen; only add() and a snapshot/rollback ----
    vector<int> cnt;
    ll best = 0;
    vector<pair<int, ll>> hist;
    // {value index, previous best}
    void add(int v, bool rec) {
        if (rec) hist.push_back({v, best});
        cnt[v]++;
        best = max(best, (ll)cnt[v] * v);
    }
    void rollback(size_t to) {
        while (hist.size() > to) {
            auto [v, b] = hist.back();
            hist.pop_back();
            cnt[v]--, best = b;
        }
    }
    void clearAll() { fill(all(cnt), 0), best = 0, hist.clear(); }
    ll answer() const { return best; }
    //
    ----------------------------------------------------------------------------------------

    MoRollback(vector<ll> arr, int maxVal) : n(arr.size()), a(arr), cnt(maxVal + 1, 0) {
        B = max(1, (int)sqrtl((ld)max(1, n)));
    }
    void addQuery(int l, int r) { qs.push_back({l, r, (int)qs.size()}); }

    vector<ll> solve() {
        sort(all(qs), [&](const Q& x, const Q& y) {
            int bx = x.l / B, by = y.l / B;
            return bx != by ? bx < by : x.r < y.r;
        });
        vector<ll> res(qs.size());
        size_t i = 0;
        while (i < qs.size()) {
            int b = qs[i].l / B, rEnd = min(n - 1, (b + 1) * B - 1);
            size_t j = i;
            while (j < qs.size() && qs[j].l / B == b) j++;
            // pass 1: queries contained in this block - answer each from a CLEAN state
            for (size_t k2 = i; k2 < j; k2++) if (qs[k2].r <= rEnd) {
                clearAll();
                for (int k = qs[k2].l; k <= qs[k2].r; k++) add(a[k], false);
                res[qs[k2].idx] = answer();
            }
            // pass 2: long queries - right part grows monotonically, left part is rolled back
            clearAll();
            int R = rEnd;
            for (size_t k2 = i; k2 < j; k2++) if (qs[k2].r > rEnd) {
                while (R < qs[k2].r) add(a[++R], false);              // grow-only, not recorded
                size_t snap = hist.size();
                for (int k = qs[k2].l; k <= rEnd; k++) add(a[k], true);   // short left part
                res[qs[k2].idx] = answer();
                rollback(snap);
            }
            i = j;
        }
        return res;
    }
};
```

2.6.7 RangeMode

RANGE MODE - the most frequent value in a range, and its frequency. Both structures below have

```
// O(1) ADD and O(1) DELETE, so dropped into Mo's (01 - MoAlgorithm) they solve the problem in
// O(n sqrt n) total.
// WHEN: mode / "maximum multiplicity" queries. There is no known linear-space polylog-per-query
// solution, so sqrt IS the intended answer here, not a fallback. The mode is NOT decomposable -
// the mode of a union is not a function of the two modes - which is why a segment tree cannot do
// it and why the counter has to be maintained incrementally.
// KEY INVARIANT (both): adding one element raises some count by exactly one, so the running
// maximum can only go UP by one; deleting lowers one count by one, so the maximum can only go DOWN
// by one, and only when the deleted value was the SOLE holder of it. A count-of-counts array turns
// "is it the sole holder" into an O(1) test.

// --- max FREQUENCY only. O(1) everything, O(V) memory. Use this unless you need the value. ---
struct FreqCounter {
    vector<int> cnt, f;
    //          // f[c] = #values with count == c
    int best = 0;
    FreqCounter(int V, int n) : cnt(V, 0), f(n + 2, 0) {}
    void add(int v) { f[cnt[v]]--, f[++cnt[v]]++, best = max(best, cnt[v]); }
    void del(int v) {
        f[cnt[v]]--;
        if (cnt[v] == best && !f[cnt[v]]) best--;
        f[--cnt[v]]++;
    }
    int query() { return best; }
    //          // the frequency of the mode
};
```

```
// --- the VALUE too: bucket the value axis and keep a max per bucket. O(sqrt V) query. ---
struct ModeCounter {
    int V, B, W;
    vector<int> cnt, bmx, bc;          // bc[b * W + c] flattened
    ModeCounter(int V, int n) : V(V), B(max(1, (int)sqrt((double)V) + 1)), W(n + 2),
                                cnt(V, 0), bmx(V / B + 2, 0), bc((V / B + 2) * W, 0) {}
    void add(int v) {
        int b = v / B;
        bc[b * W + cnt[v]]--, bc[b * W + ++cnt[v]]++;
        bmx[b] = max(bmx[b], cnt[v]);
    }
    void del(int v) {
        int b = v / B;
        if (bmx[b] == cnt[v] && bc[b * W + cnt[v]] == 1) bmx[b]--;
        bc[b * W + cnt[v]]--, bc[b * W + --cnt[v]]++;
    }
    pii query() {                      // {value, count}, O(sqrt V)
        int best = 0;
        for (int x : bmx) best = max(best, x);
        for (int b = 0; b < (int)bmx.size(); b++)
            if (bmx[b] == best)
                for (int v = b * B; v < min(V, (b + 1) * B); v++)
                    if (cnt[v] == best) return {v, best};
        return {-1, 0};
    }
};
// MEMORY WARNING: bc is O(sqrt(V) * n) ints - about 160 MB at V = n = 2e5. Shrink it by passing
// the largest range length as n, by widening B (query cost O(V/B + B), memory O(V/B * n)), or by
// dropping to FreqCounter when the value itself is not needed.
// COORDINATE COMPRESS the values first so V is the number of DISTINCT values (01 - Misc/05).
// BECAUSE DELETE IS O(1) you do not need Mo's with rollback (06) - plain Mo's is enough.
// THE ONLINE VERSION, O(sqrt n) per query and no Mo's: cut the array into sqrt(n) blocks,
// precompute the mode of every block RANGE [i..j] and the prefix count of every value per block.
// A query starts from the precomputed mode of its maximal inner block range and folds in the
// O(sqrt n) leftover elements one at a time; each can only raise the answer, and the count of any
// value in the current range is (prefix count) + (leftovers seen so far), i.e. O(1). The same
// scaffolding answers any statistic that is monotone under adding one element.
// MEX WITH UPDATES is the same shape: keep cnt[] plus f[], and scan f[1], f[2], ... for the first
// zero. O(sqrt n) per query inside Mo's, O(1) amortised if elements are only ever added.
```

2.6.8 MoOnTwoIntervals

MO'S ON TWO INTERVALS ("4D Mo") - each query is a PAIR of ranges [l1, r1] and [l2, r2] and the

```
// answer depends on the multiset union of both. Four pointers instead of two.
// COMPLEXITY: with block size B the cost is O(q * B) for the three bucketed pointers plus
// O(n^4 / B^3) for the monotone one; B ~ n^(3/4) balances them at O(n^(7/4)) when q ~ n.
// WHEN: "compare two subtrees / two subarrays" - e.g. the most common value in the union of two
// subtrees (CF 1767F). Both ranges must be flattened to intervals first: for subtrees use the
// Euler tour (09 - Trees/06), which is what makes a tree query a pair of intervals.
// Do NOT reach for this before checking whether the two ranges can be merged into one, or whether
// the answer is decomposable (then a segment tree merge, 16 - SegTreeMerging, is far faster).
// KEY INVARIANT: the same amortisation as ordinary Mo's, one level deeper. Sort by
// (l1/B, r1/B, l2/B, r2): the first three keys change O(B) per query each because they are
// bucketed, and the last one is monotone inside every fixed triple of buckets, so it sweeps the
// array once per triple. The state must support ADD and DELETE in O(1) - it is shared by both
// intervals, so an element can be inserted twice and that must be meaningful.
struct Mo2 {
    struct Q { int l1, r1, l2, r2, id; };
    int n, B;
    vector<Q> qs;
    Mo2(int n, int q) : n(n), B(max(1, (int)pow((double)n, 0.75))) { qs.reserve(q); }
    void addQuery(int l1, int r1, int l2, int r2, int id) { qs.push_back({l1, r1, l2, r2, id}); }
    // add(i) / del(i) update the shared state with position i; get() reads the answer.
    template <class A, class D, class G>
    void run(A add, D del, G get) {
        int Bl = B;
        sort(all(qs), [&](const Q& a, const Q& b) {
            array<int, 7> x{a.l1 / Bl, a.r1 / Bl, a.l2 / Bl, a.r2, a.l1, a.r1, a.l2};
            array<int, 7> y{b.l1 / Bl, b.r1 / Bl, b.l2 / Bl, b.r2, b.l1, b.r1, b.l2};
            return x < y;
        });
        int a = 0, b = -1, c = 0, d = -1;
                                       // [a, b] and [c, d], both empty
        for (auto& q : qs) {
            while (b < q.r1) add(++b);
            while (a > q.l1) add(--a);
            while (b > q.r1) del(b--);
            while (a < q.l1) del(a++);
            while (d < q.r2) add(++d);
            while (c > q.l2) add(--c);
            while (d > q.r2) del(d--);
            while (c < q.l2) del(c++);
            get(q.id);
        }
    }
};
// THE ORDER OF THE FOUR LOOPS MATTERS: always GROW before you SHRINK, exactly as in 1D Mo's,
// or the two ranges can transiently cross into an invalid state (negative counts).
```

```
// OVERLAPPING RANGES: if the two intervals may
   ↪ intersect, an element gets added twice. Decide up
// front whether the state should count it twice
   ↪ (multiset union) or once (set union) - for set
// union keep a per-position "times present" counter
   ↪ and only touch the state on 0 <-> 1.
// TREE PAIRS: l1 = tin[u], r1 = tout[u], l2 =
   ↪ tin[v], r2 = tout[v]. If u is an ancestor of v
   ↪ the
// intervals nest, so the "counted twice" question
   ↪ above is not hypothetical.
// 3+ INTERVALS: the same sort with more keys works
   ↪ but the exponent gets ugly fast - at that point
// look for a completely different decomposition.
```

2.6.9 BlockList

BLOCK LIST (chunked / unrolled list, a "sqrt rope") - an array that supports insert and erase in

```
// the middle, RANGE ASSIGN, RANGE REVERSE, and range
   ↪ statistics, all in O(sqrt n).
// WHEN: an implicit treap (07 - BBST/02) does the
   ↪ same in O(log n) and is usually the right answer.
// Reach for this instead when the per-block
   ↪ statistic is something a treap node cannot merge
   ↪ in
// O(1) - "how many elements >= x in this range"
   ↪ (each block keeps a SORTED copy), "the k-th
// smallest", "count of distinct" - or when you want
   ↪ a structure you can debug by printing.
// COMPLEXITY: O(sqrt n) per operation amortised,
   ↪ with a full rebuild every ~sqrt n operations to
// stop the blocks from fragmenting. Rebuild is O(n).
// KEY INVARIANT: the array is a LIST OF BLOCKS, each
   ↪ holding at most ~2B elements plus two lazy
// tags (assigned-value, reversed). Every range
   ↪ operation starts by SPLITTING at both endpoints,
   ↪ so
// afterwards the range is an exact run of whole
   ↪ blocks and each one can be tagged in O(1). A
// reverse then only has to flip each block's tag and
   ↪ reverse the ORDER OF THE BLOCK LIST itself.
struct BlockList {
    struct Block {
        vector<int> v, s;
  ↪           // s = sorted copy of v
        int len = 0, asg = -1;
  ↪           // asg >= 0: whole block is asg
        bool rev = false;
        void resort() { s = v, sort(all(s)); }
        void solid() {
  ↪           // drop both tags
            if (asg >= 0) v.assign(len, asg), asg =
  ↪ -1, resort();
            if (rev) std::reverse(all(v)), rev =
  ↪ false;
        }
        int atLeast(int x) {
  ↪           // how many elements >= x
            if (asg >= 0) return asg >= x ? len : 0;
            return s.end() - lower_bound(all(s), x);
        }
    };
    vector<Block> b;
    int n, B, ops = 0;
    BlockList(const vector<int>& a) : n(a.size()),
  ↪ B(max(1, (int)sqrt((double)n) + 1)) { chunk(a); }
    void chunk(const vector<int>& a) {
        b.clear();
        for (int i = 0; i < (int)a.size(); i += B) {
            Block x;
            x.v.assign(a.begin() + i, a.begin() +
  ↪ min((int)a.size(), i + B));
            x.len = x.v.size(), x.resort();
            b.push_back(x);
        }
    }
    void rebuild() {
  ↪           // O(n), every ~B operations
        vector<int> a;
        a.reserve(n);
        for (auto& x : b) {
            x.solid();
            for (int y : x.v) a.push_back(y);
        }
        chunk(a);
    }
    int split(int i) {
  ↪           // index of the block starting at i
        if (i >= n) return b.size();
        int cur = 0;
        for (int k = 0; k < (int)b.size(); k++) {
            if (i < cur + b[k].len) {
                if (i == cur) return k;
                int off = i - cur;
                b[k].solid();
                Block nb;
                nb.v.assign(b[k].v.begin() + off,
  ↪ b[k].v.end());
                nb.len = nb.v.size(), nb.resort();
                b[k].v.resize(off), b[k].len = off,
  ↪ b[k].resort();
                b.insert(b.begin() + k + 1, nb);
                return k + 1;
            }
            cur += b[k].len;
        }
        return b.size();
    }
    void tick() { if (++ops % B == 0) rebuild(); }
  ↪           // call after every modification
    void assign(int l, int r, int x) {
        int p = split(l), q = split(r + 1);
        for (int k = p; k < q; k++) b[k].asg = x,
  ↪ b[k].rev = false;
        tick();
    }
    void reverse(int l, int r) {
        int p = split(l), q = split(r + 1);
        for (int k = p; k < q; k++) b[k].rev ^= 1;
        std::reverse(b.begin() + p, b.begin() + q);
        tick();
    }
    void insert(int i, int x) {
        int p = split(i);
        Block nb;
        nb.v = {x}, nb.len = 1, nb.resort();
        b.insert(b.begin() + p, nb), n++;
        tick();
    }
    void erase(int l, int r) {
        int p = split(l), q = split(r + 1);
        b.erase(b.begin() + p, b.begin() + q), n -= r
  ↪ - l + 1;
        tick();
    }
    int count(int l, int r, int x) {
```

```
    ↪          // how many a[i] >= x in [l, r]
        int p = split(l), q = split(r + 1), res = 0;
        for (int k = p; k < q; k++) res +=
    ↪ b[k].atLeast(x);
        return res;
    }
};
// THE TWO TAGS INTERACT: assigning kills a pending
    ↪ reverse (all values are equal), reversing an
// assigned block is a no-op. solid() is the only
    ↪ place that has to get the order right.
// OTHER PER-BLOCK STATISTICS drop straight in: a
    ↪ frequency map (count of a value), a prefix sum
// (range sum with range assign), a max, a gcd.
    ↪ Anything you can rebuild in O(B) works.
// WITHOUT insert / erase / reverse, do not write
    ↪ this: 04 - BlockDecomposition is half the code.
// WITHOUT the sorted-copy statistic, an implicit
    ↪ treap is faster and shorter.
```

2.7 Balanced Binary Search Tree (BBST)

2.7.1 Treap

TREAP (keyed) - a BST by key and a heap by random priority, so it is balanced in expectation.

```
// NOTE: 01 - Miscellaneous/04 - random.cpp declares
    ↪ the same `rng`. Keep one copy.
// O(log n) insert/erase/find, plus split and merge,
    ↪ which is what makes it better than std::set:
// you can cut the tree at a key, operate on a whole
    ↪ range, and glue it back.
mt19937_64
    ↪ rng(chrono::steady_clock::now().time_since_epoch().count());

template <typename T>
struct Treap {
    struct Node {
        T key, val, sum;
        T lazy = 0; // 1. Added lazy tag
        int sz = 1;
        uint64_t p = rng();
        Node *l = nullptr, *r = nullptr;

        Node(T k, T v = T()) : key(k), val(v), sum(v)
    ↪ {}
    }* root = nullptr;

    ~Treap() { destroy(root); }

    void destroy(Node* t) {
        if (!t) return;
        destroy(t->l);
        destroy(t->r);
        delete t;
    }

    int sz(Node* t) { return t ? t->sz : 0; }
    T sub_val(Node* t) { return t ? t->sum : 0; }

    // 2. The Push Function: Propagates pending
    ↪ updates to children
    void push(Node* t) {
        if (!t || t->lazy == 0) return;

        if (t->l) {
            t->l->key += t->lazy; // Shift the key
    ↪ (critical for the DP solution)
            t->l->val += t->lazy; // Shift the value
            t->l->sum += t->lazy * sz(t->l); //
    ↪ Update subtree sum
            t->l->lazy += t->lazy; // Pass the tag
    ↪ down
        }
        if (t->r) {
            t->r->key += t->lazy;
            t->r->val += t->lazy;
            t->r->sum += t->lazy * sz(t->r);
            t->r->lazy += t->lazy;
        }
        t->lazy = 0; // Clear the tag on the current
    ↪ node
    }

    Node* update(Node* t) {
        if (t) {
            t->sz = 1 + sz(t->l) + sz(t->r);
            t->sum = t->val + sub_val(t->l) +
    ↪ sub_val(t->r);
        }
        return t;
    }

    Node* merge(Node* l, Node* r) {
        push(l); // 3. Push before making routing
    ↪ decisions
        push(r);
        if (!l || !r) return l ? l : r;
        if (l->p > r->p) {
            l->r = merge(l->r, r);
            return update(l);
        }
        r->l = merge(l, r->l);
        return update(r);
    }

    pair<Node*, Node*> split(Node* t, T k) {
        if (!t) return {nullptr, nullptr};
        push(t); // 3. Push before evaluating keys
    ↪ for splits
        if (t->key <= k) {
            auto [l, r] = split(t->r, k);
            t->r = l;
            return {update(t), r};
        }
        auto [l, r] = split(t->l, k);
        t->l = r;
        return {l, update(t)};
    }

    // --- Core Operations ---

    // Range Update Method
    void add_range(T l_val, T r_val, T add_val) {
        if (l_val > r_val) return;

        auto [left_mid, right] = split(root, r_val);
        auto [left, mid] = split(left_mid, l_val - 1);

        // Apply the lazy tag to the entire isolated
    ↪ segment
        if (mid) {
            mid->key += add_val;
            mid->val += add_val;
            mid->sum += add_val * sz(mid);
            mid->lazy += add_val;
        }
```

```
        root = merge(merge(left, mid), right);
    }

    bool search(T x) {
        Node* curr = root;
        while (curr) {
            push(curr); // Push during traversal
            if (curr->key == x) return true;
            curr = (x < curr->key) ? curr->l :
↪ curr->r;
        }
        return false;
    }

    void insert(T x, T val = T()) {
        auto [l, r] = split(root, x);
        auto [l2, r2] = split(l, x - 1);
        if (!r2) {
            r2 = new Node(x, val);
        } else {
            push(r2); // Push before modifying
            r2->val += val;
            update(r2);
        }
        root = merge(merge(l2, r2), r);
    }

    void erase(T x) {
        auto [l, r] = split(root, x);
        auto [l2, r2] = split(l, x - 1);
        destroy(r2);
        root = merge(l2, r);
    }

    // --- Utility Operations ---

    Node* find_by_order(Node* t, int k) {
        if (!t) return nullptr;
        push(t); // Push before accessing children
↪ sizes
        int left_sz = sz(t->l);
        if (k < left_sz) return find_by_order(t->l,
↪ k);
        if (k == left_sz) return t;
        return find_by_order(t->r, k - left_sz - 1);
    }
    Node* find_by_order(int k) { return
↪ find_by_order(root, k); }

    int order_of_key(Node* t, T k) {
        if (!t) return 0;
        push(t); // Push before comparing keys
        if (t->key >= k) return order_of_key(t->l, k);
        return sz(t->l) + 1 + order_of_key(t->r, k);
    }
    int order_of_key(T k) { return order_of_key(root,
↪ k); }

    void print(Node* t) {
        if (!t) return;
        push(t); // Push before printing to get true
↪ values
        print(t->l);
        cout << t->key << " ";
        print(t->r);
    }
    void print() {
        print(root);
        cout << '\n';
    }
};
```

2.7.2 ImplicitTreap

IMPLICIT TREAP - a treap keyed by POSITION (subtree size) instead of value, i.e. an array that

```
// NOTE: 01 - Miscellaneous/04 - random.cpp declares
  ↪ the same `rng`. Keep one copy.
// supports insert/erase in the middle, reverse a
  ↪ range, and move a block, all in O(log n).
// This is the structure for "cut [l,r] out and paste
  ↪ it elsewhere" problems.
mt19937_64
  ↪ rng(chrono::steady_clock::now().time_since_epoch().c

template <typename T>
struct ImplicitTreap {
    struct Node {
        T val, sum;
        T lazy = 0;       // Lazy tag for range
  ↪ additions
        bool rev = false; // Lazy tag for range
  ↪ reversals
        int sz = 1;
        uint64_t p = rng();
        Node *l = nullptr, *r = nullptr;

        Node(T v = T()) : val(v), sum(v) {}
    }* root = nullptr;

    // Default Constructor
    ImplicitTreap() = default;

    // Vector Constructor - Builds treap in O(N) time
    ImplicitTreap(const vector<T>& a) {
        build(a);
    }

    ~ImplicitTreap() { destroy(root); }

    void destroy(Node* t) {
        if (!t) return;
        destroy(t->l);
        destroy(t->r);
        delete t;
    }

    int sz(Node* t) { return t ? t->sz : 0; }
    T sub_val(Node* t) { return t ? t->sum : 0; }

    // Propagates pending lazy updates to children
    void push(Node* t) {
        if (!t) return;

        // 1. Process pending reversal
        if (t->rev) {
            swap(t->l, t->r);
            if (t->l) t->l->rev ^= true;
            if (t->r) t->r->rev ^= true;
            t->rev = false;
        }

        // 2. Process pending value addition
        if (t->lazy != 0) {
            if (t->l) {
                t->l->val += t->lazy;
```

```
            t->l->sum += t->lazy * sz(t->l);
            t->l->lazy += t->lazy;
        }
        if (t->r) {
            t->r->val += t->lazy;
            t->r->sum += t->lazy * sz(t->r);
            t->r->lazy += t->lazy;
        }
        t->lazy = 0; // Clear the tag
    }
}

Node* update(Node* t) {
    if (t) {
        t->sz = 1 + sz(t->l) + sz(t->r);
        t->sum = t->val + sub_val(t->l) +
↪ sub_val(t->r);
    }
    return t;
}

// --- O(N) Linear Time Build ---
void build(const vector<T>& a) {
    destroy(root);
    root = nullptr;
    if (a.empty()) return;

    vector<Node*> st;
    for (const T& val : a) {
        Node* curr = new Node(val);
        Node* last = nullptr;

        // Maintain max-heap property on random
↪ priority 'p'
        while (!st.empty() && st.back()->p <
↪ curr->p) {
            last = st.back();
            st.pop_back();
        }

        curr->l = last;
        if (!st.empty()) {
            st.back()->r = curr;
        }
        st.push_back(curr);
    }

    // Post-order pass to compute subtree sizes
↪ and sums in O(N)
    auto recompute = [&](auto& self, Node* t) ->
↪ void {
        if (!t) return;
        self(self, t->l);
        self(self, t->r);
        update(t);
    };

    root = st.front(); // Bottom of stack is the
↪ root
    recompute(recompute, root);
}

// --- Dynamic Array Operations ---

Node* merge(Node* l, Node* r) {
    push(l);
    push(r);
    if (!l || !r) return l ? l : r;
    if (l->p > r->p) {
        l->r = merge(l->r, r);
        return update(l);
    }
    r->l = merge(l, r->l);
    return update(r);
}

pair<Node*, Node*> split(Node* t, int k) {
    if (!t) return {nullptr, nullptr};
    push(t);

    int left_sz = sz(t->l);
    if (left_sz < k) {
        auto [l, r] = split(t->r, k - left_sz -
↪ 1);
        t->r = l;
        return {update(t), r};
    } else {
        auto [l, r] = split(t->l, k);
        t->l = r;
        return {l, update(t)};
    }
}

int size() { return sz(root); }

void insert(int idx, T val) {
    auto [l, r] = split(root, idx);
    Node* node = new Node(val);
    root = merge(merge(l, node), r);
}

void erase(int idx) {
    if (idx < 0 || idx >= sz(root)) return;
    auto [left_mid, right] = split(root, idx + 1);
    auto [left, mid] = split(left_mid, idx);
    destroy(mid);
    root = merge(left, right);
}

void reverse_range(int l, int r) {
    if (l >= r || l < 0 || r >= sz(root)) return;

    auto [left_mid, right] = split(root, r + 1);
    auto [left, mid] = split(left_mid, l);

    if (mid) mid->rev ^= true;

    root = merge(merge(left, mid), right);
}

void add_range(int l, int r, T add_val) {
    if (l > r || l < 0 || r >= sz(root)) return;

    auto [left_mid, right] = split(root, r + 1);
    auto [left, mid] = split(left_mid, l);

    if (mid) {
        mid->val += add_val;
        mid->sum += add_val * sz(mid);
        mid->lazy += add_val;
    }

    root = merge(merge(left, mid), right);
}

T query_range(int l, int r) {
```

```
        if (l > r || l < 0 || r >= sz(root)) return
    ↪ T();

        auto [left_mid, right] = split(root, r + 1);
        auto [left, mid] = split(left_mid, l);

        T ans = sub_val(mid);

        root = merge(merge(left, mid), right);
        return ans;
    }

    T get(int idx) {
        return query_range(idx, idx);
    }

    void print(Node* t) {
        if (!t) return;
        push(t);
        print(t->l);
        cout << t->val << " ";
        print(t->r);
    }

    void print() {
        print(root);
        cout << '\n';
    }
};
```

2.7.3 PersistentTreap

PERSISTENT IMPLICIT TREAP - a persistent ARRAY that supports insert / erase / concatenate / cut

```
// a block in the middle, with every version staying
    ↪ queryable. O(log n) time and O(log n) new
// nodes per operation.
// WHEN: "version v of the text, then insert this
    ↪ string at position k", copy-paste editors, and
// the killer application: COPY A RANGE OF AN OLD
    ↪ VERSION AND PASTE IT ANYWHERE - two splits of an
// old root share all their nodes with it, so the
    ↪ copy is free. A persistent segment tree
// (01 - Segment Tree/03) cannot do this because its
    ↪ index set is fixed; 02 - ImplicitTreap can,
// but destroys the old version.
// KEY INVARIANT: COPY ON WRITE. split and merge only
    ↪ ever touch one root-to-leaf path, so cloning
// each node before modifying it costs O(log n) extra
    ↪ nodes and leaves every earlier root pointing
// at a consistent tree. Priorities are copied too,
    ↪ so the shape (and hence the depth bound) is
// unchanged by cloning.
struct PTreap {
    struct Node { int l = 0, r = 0, sz = 0; unsigned
    ↪ pr = 0; ll val = 0, sum = 0; };
    vector<Node> t;
    PTreap() : t(1) {}
    ↪          // node 0 = the empty tree
    unsigned rnd() {
        static unsigned s = 88172645u;
        return s ^= s << 13, s ^= s >> 17, s ^= s <<
    ↪ 5;
    }
    int cp(int x) { t.push_back(t[x]); return
    ↪ t.size() - 1; }
    void pull(int x) {
        t[x].sz = t[t[x].l].sz + 1 + t[t[x].r].sz;
        t[x].sum = t[t[x].l].sum + t[x].val +
    ↪ t[t[x].r].sum;
    }
    pair<int, int> split(int x, int k) {
    ↪          // first k elements go left
        if (!x) return {0, 0};
        int c = cp(x);
        if (t[t[c].l].sz < k) {
            auto [a, b] = split(t[c].r, k -
    ↪ t[t[c].l].sz - 1);
            t[c].r = a, pull(c);
            return {c, b};
        }
        auto [a, b] = split(t[c].l, k);
        t[c].l = b, pull(c);
        return {a, c};
    }
    int merge(int a, int b) {
        if (!a || !b) return a | b;
        if (t[a].pr > t[b].pr) {
            int c = cp(a), r = merge(t[c].r, b);
            t[c].r = r, pull(c);
            return c;
        }
        int c = cp(b), l = merge(a, t[c].l);
        t[c].l = l, pull(c);
        return c;
    }
    int mk(ll v) { t.push_back({0, 0, 1, rnd(), v,
    ↪ v}); return t.size() - 1; }
    int build(const vector<ll>& a) {
    ↪          // O(n log n), fine for a one-off
        int r = 0;
        for (ll v : a) r = merge(r, mk(v));
        return r;
    }
    int insert(int root, int pos, ll v) {
    ↪          // returns the new root
        auto [a, b] = split(root, pos);
        return merge(merge(a, mk(v)), b);
    }
    int erase(int root, int l, int r) {
    ↪          // drop positions [l, r]
        auto [a, b] = split(root, l);
        auto [m, c] = split(b, r - l + 1);
        return merge(a, c);
    }
    int cut(int root, int l, int r) {
    ↪          // the block [l, r] as its own tree
        auto [a, b] = split(root, l);
        return split(b, r - l + 1).first;
    }
    int paste(int root, int pos, int piece) {
    ↪          // splice another tree in
        auto [a, b] = split(root, pos);
        return merge(merge(a, piece), b);
    }
    ll sum(int x, int l, int r) {
    ↪          // read-only: allocates nothing
        if (!x || l > r || r < 0 || l >= t[x].sz)
    ↪ return 0;
        if (l <= 0 && r >= t[x].sz - 1) return
    ↪ t[x].sum;
        int L = t[t[x].l].sz;
        return sum(t[x].l, l, min(r, L - 1)) + (l <=
    ↪ L && L <= r ? t[x].val : 0) +
                sum(t[x].r, max(0, l - L - 1), r - L -
    ↪ 1);
    }
```

```
    ll get(int root, int i) { return sum(root, i, i);
  ↪ }
};
// LAZY TAGS (range add, range reverse) DO work, but
  ↪ push() must CLONE both children before
// pushing into them - otherwise you mutate a node an
  ↪ old version still owns. That is one extra
// line in push and doubles the node count; write it
  ↪ only if the problem needs it.
// MEMORY is the binding constraint: O(q log n) nodes
  ↪ at ~32 bytes. reserve() and, if a version is
// provably dead, do not bother freeing - contests
  ↪ end first.
// IF YOU ONLY NEED THE LATEST VERSION use 02 -
  ↪ ImplicitTreap (no allocation per split).
// IF THE OPERATIONS ARE POSITION-STABLE (no insert /
  ↪ erase, only updates) a persistent segment
// tree is 3x faster and half the memory.
```

2.8 Policy Based Data Structures

2.8.1 OrderedSet

Policy-Based Data Structure: ordered_set

```
// Time: O(log N) per operation
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

using namespace __gnu_pbds;
template<typename T>
using ordered_set = tree<T, null_type, less<T>,
  ↪ rb_tree_tag, tree_order_statistics_node_update>;

// Usage:
// st.find_by_order(k): Iterator to k-th element
  ↪ (0-based)
// st.order_of_key(x): Number of elements strictly
  ↪ smaller than x
```

2.9 Monotonic Data Structures

2.9.1 MonotonicStack

Monotonic Stack - Time: O(N), Space: O(N)

```
// Computes Next/Previous Smaller and Greater element
  ↪ indices for all positions
template<typename T = ll>
struct MonotonicStack {
    int n;
    vector<T> a;
    vector<int> prev_less, next_less;
    vector<int> prev_greater, next_greater;

    MonotonicStack(const vector<T>& arr) :
  ↪ n(arr.size()), a(arr),
        prev_less(n, -1), next_less(n, n),
        prev_greater(n, -1), next_greater(n, n) {

        vector<int> st;
        // Next Less & Previous Less
        for (int i = 0; i < n; i++) {
            while (!st.empty() && a[st.back()] >=
  ↪ a[i]) {
                next_less[st.back()] = i;
                st.pop_back();
            }
            if (!st.empty()) prev_less[i] = st.back();
            st.pb(i);
        }
        st.clear();
        // Next Greater & Previous Greater
        for (int i = 0; i < n; i++) {
            while (!st.empty() && a[st.back()] <=
  ↪ a[i]) {
                next_greater[st.back()] = i;
                st.pop_back();
            }
            if (!st.empty()) prev_greater[i] =
  ↪ st.back();
            st.pb(i);
        }
    }
};
```

2.9.2 TwoStackQueue

Monotonic Queue via Two Stacks (Sliding Window Min/Max)

```
// Space: O(N), Time: O(1) amortized per push/pop/get
struct TwoStackQ {
    struct Node {
        ll mx = -llinf, mn = llinf, val;

        Node() : val(0) {}
        Node(ll x) : mx(x), mn(x), val(x) {}
    };

    stack<Node> a, b;

    int size() {
        return a.size() + b.size();
    }

    void mrg(Node &a, Node &b) {
        a.mn = min(a.mn, b.mn);
        a.mx = max(a.mx, b.mx);
    }

    // Pushes value into the queue
    void push(ll val) {
        auto nd = Node(val);
        if (!a.empty()) mrg(nd, a.top());
        a.push(nd);
    }

    // Transfers elements from stack a to stack b
  ↪ when b is empty
    void move() {
        while (!a.empty()) {
            auto nd = Node(a.top().val);
            if (!b.empty()) mrg(nd, b.top());
            b.push(nd);
            a.pop();
        }
    }

    // Returns min/max of all current elements in the
  ↪ queue in O(1)
    Node get() {
        Node res;
        if (!b.empty()) mrg(res, b.top());
        if (!a.empty()) mrg(res, a.top());
        return res;
    }

    // Removes the oldest element from the queue
    void pop() {
        if (b.empty()) move();
        if (!b.empty()) b.pop();
```

```
    }
};
```

2.9.3 MonotonicQueue2D

2D Monotonic Queue (Sliding Window 2D Min/Max) - Time: O(N * M), Space: O(N * M)

```
// Computes min in every K x L subgrid of an N x M matrix
template<typename T = ll>
struct MonotonicQueue2D {
    int n, m;

    // Returns a matrix res where res[i][j] is the min in subgrid [i..i+K-1][j..j+L-1]
    vector<vector<T>> sliding_window_2d(const vector<vector<T>>& grid, int K, int L) {
        n = grid.size();
        m = grid[0].size();

        // 1. Sliding window min horizontally along each row
        vector<vector<T>> row_min(n, vector<T>(m - L + 1));
        for (int i = 0; i < n; i++) {
            deque<int> dq;
            for (int j = 0; j < m; j++) {
                while (!dq.empty() && grid[i][dq.back()] >= grid[i][j]) dq.pop_back();
                dq.pb(j);
                if (dq.front() <= j - L) dq.pop_front();
                if (j >= L - 1) row_min[i][j - L + 1] = grid[i][dq.front()];
            }
        }

        // 2. Sliding window min vertically down each column of row_min
        vector<vector<T>> res(n - K + 1, vector<T>(m - L + 1));
        for (int j = 0; j < m - L + 1; j++) {
            deque<int> dq;
            for (int i = 0; i < n; i++) {
                while (!dq.empty() && row_min[dq.back()][j] >= row_min[i][j]) dq.pop_back();
                dq.pb(i);
                if (dq.front() <= i - K) dq.pop_front();
                if (i >= K - 1) res[i - K + 1][j] = row_min[dq.front()][j];
            }
        }

        return res;
    }
};
```

2.9.4 LazyDeleteAndTopK

TWO SMALL STRUCTURES THAT KEEP SHOWING UP, and are always retyped badly under pressure.

```
// --- LAZY-DELETION HEAP: a priority_queue that supports erase(x), O(log n) amortised. ---
// Keep a second heap of "things already deleted"; before reading the top, pop matching pairs.
// This is why you almost never need decrease-key: push the new value and lazily drop the old one.
template <class T, class Cmp = less<T>>
struct LazyHeap {
    priority_queue<T, vector<T>, Cmp> in, del;
    void push(T x) { in.push(x); }
    void erase(T x) { del.push(x); }          // x must actually be present
    void clean() { while (!del.empty() && in.top() == del.top()) in.pop(), del.pop(); }
    bool empty() { clean(); return in.empty(); }
    int size() { return (int)in.size() - (int)del.size(); }
    T top() { clean(); return in.top(); }
    void pop() { clean(); in.pop(); }
};
// A multiset does the same thing with erase(find(x)) and is shorter - reach for THIS when you
// need heap performance (no red-black overhead) or when the values are not comparable-unique.
// TRAP: erase(x) must be matched by an earlier push(x), or `size()` and `clean()` drift.

// --- TOP-K SUM: maintain the sum of the k largest elements under insert and erase. ---
// Two multisets with a fixed frontier: `hi` holds the k largest (and its sum), `lo` the rest.
// After every operation, move elements across the frontier until |hi| == min(k, total).
// USES: "sum of the k best so far" while sweeping, sliding-window k-th order statistics, and
// scheduling/greedy problems where you keep the k cheapest and pay for the rest.
struct TopK {
    int k;
    ll sum = 0;                               // sum of the k largest
    multiset<ll> hi, lo;                      // hi = the chosen k, lo = the rest
    TopK(int k) : k(k) {}
    void fix() {
        while ((int)hi.size() < k && !lo.empty()) {
            auto it = prev(lo.end());
            sum += *it, hi.insert(*it), lo.erase(it);
        }
        while ((int)hi.size() > k) {
            auto it = hi.begin();
            sum -= *it, lo.insert(*it), hi.erase(it);
        }
        while (!hi.empty() && !lo.empty() && *hi.begin() < *prev(lo.end())) {
            auto a = hi.begin(), b = prev(lo.end());
            sum += *b - *a;
            hi.insert(*b), lo.insert(*a), hi.erase(a), lo.erase(b);
        }
    }
    void insert(ll x) { lo.insert(x), fix(); }
    void erase(ll x) {
        auto it = hi.find(x);
```

```
        if (it != hi.end()) sum -= x, hi.erase(it);
        else lo.erase(lo.find(x));
        fix();
    }
    int size() const { return hi.size() + lo.size(); }
};
// SMALLEST k instead: negate on the way in and out,
   ↪ or mirror the two sets.
// If k CHANGES too, just set k and call fix() - the
   ↪ frontier moves in O(|dk| log n).
// If you only ever ADD (no erase), a single size-k
   ↪ min-heap is shorter: push, and pop while the
// size exceeds k, keeping a running sum.
```

2.10 DynamicMedian

RUNNING MEDIAN / k-th SMALLEST under insertions, via two multisets kept balanced around the split.

```
// O(log n) per insert, O(1) to read the median and
   ↪ either side's sum. The pattern generalises to
// "maintain the sum of the k smallest" - which is
   ↪ what cost-to-move-to-median problems need.
multiset<ll> L, R;
ll sum_L = 0, sum_R = 0;
int k;

// Rebalances the multisets so L always has the lower
   ↪ half/median of CURRENT elements
void balance() {
    int total_size = sz(L) + sz(R);
    int k_L = (total_size + 1) / 2; // Fixed: Target
   ↪ size based on actual current elements

    // If L is too big, move its largest element to R
    while (sz(L) > k_L) {
        auto it = prev(L.end());
        ll val = *it;

        sum_L -= val;
        sum_R += val;

        R.insert(val);
        L.erase(it);
    }

    // If L is too small, move the smallest element
   ↪ of R to L
    while (sz(L) < k_L) {
        auto it = R.begin();
        ll val = *it;

        sum_R -= val;
        sum_L += val;

        L.insert(val);
        R.erase(it);
    }
}

void add_val(ll val) {
    if (L.empty() || val <= *L.rbegin()) {
        L.insert(val);
        sum_L += val;
    } else {
        R.insert(val);
        sum_R += val;
    }
    balance();
}

void remove_val(ll val) {
    auto it = L.find(val);
    if (it != L.end()) {
        sum_L -= val;
        L.erase(it);
    } else {
        it = R.find(val);
        if (it == R.end()) return;
   ↪ // not present in either half: do nothing
        sum_R -= val;
        R.erase(it);
    }
    balance();
}

ll get_cost() {                                    //
   ↪ UB if empty - guard the caller
    ll median = *L.rbegin();
    return sum_R - sum_L + median * (sz(L) - sz(R));
}
```

2.11 IntervalSetODT

INTERVAL SET / "Chtholly tree" (ODT). Keeps the array as maximal runs of equal value.

```
// assign(l, r, v) is amortised near-O(log n) WHEN
   ↪ the data has many range-assigns (random or
// adversarial-but-assign-heavy); each assign
   ↪ destroys the intervals it covers, so the total
// number of intervals created is O(q).
// THIS is the standard way to support range
   ↪ bitwise-AND / OR / MOD / assign together with
// range sum, because those operations rapidly make
   ↪ long stretches constant.
struct ODT {
    map<int, pair<int, ll>> mp;
   ↪ // l -> {r, value}, covering [l, r]

    ODT(int n, ll v = 0) { mp[0] = {n - 1, v}; }

    // split so that an interval starts exactly at p;
   ↪ returns the iterator to it
    auto split(int p) {
        auto it = prev(mp.upper_bound(p));
        if (it->first == p) return it;
        auto [r, v] = it->second;
        it->second.first = p - 1;
        return mp.emplace(p, make_pair(r, v)).first;
    }
    void assign(int l, int r, ll v) {
        auto itr = split(r + 1), itl = split(l);
        mp.erase(itl, itr);
        mp[l] = {r, v};
    }
    // Generic scan over [l, r]: f(l, r, value) is
   ↪ called once per run. O(#runs touched).
    template <class F> void forEach(int l, int r, F
   ↪ f) {
        auto itr = split(r + 1), itl = split(l);
        for (auto it = itl; it != itr; ++it)
   ↪ f(it->first, it->second.first,
   ↪ it->second.second);
    }
    ll sum(int l, int r) {
        ll s = 0;
        forEach(l, r, [&](int a, int b, ll v) { s +=
```

```
    ↪ v * (b - a + 1); });
        return s;
    }
    // RANGE ADD: touch each run once. Runs are not
    ↪ merged, so adds alone do NOT keep the
    // amortised bound - this is only cheap when
    ↪ assigns keep collapsing the structure.
    void add(int l, int r, ll x) {
        auto itr = split(r + 1), itl = split(l);
        for (auto it = itl; it != itr; ++it)
    ↪ it->second.second += x;
    }
    // RANGE AND / OR / MOD follow the same loop,
    ↪ applying the op to each run's value.
    // They are fast because every value can only
    ↪ decrease O(log V) times before it stops
    // changing, at which point neighbouring runs
    ↪ become equal and can be merged.
    template <class Op> void apply(int l, int r, Op
    ↪ op) {
        auto itr = split(r + 1), itl = split(l);
        for (auto it = itl; it != itr; ++it)
    ↪ it->second.second = op(it->second.second);
        merge(l, r);
    }
    void merge(int l, int r) {
    ↪ // glue neighbouring runs of equal value
        auto it = split(l);
        while (it != mp.end() && it->first <= r) {
            auto nx = next(it);
            if (nx != mp.end() && nx->second.second
    ↪ == it->second.second &&
                it->second.first + 1 == nx->first) {
                it->second.first = nx->second.first;
                mp.erase(nx);
            } else it = nx;
        }
    }
};
// Careful: worst case WITHOUT assigns is O(n) per
  ↪ operation. If the problem has only adds and
// queries, use a segment tree instead.
```

2.12 DisjointSparseTable

DISJOINT SPARSE TABLE - O(1) query for ANY associative operation (not just idempotent ones).

```
// Build O(n log n) time and memory. Use when a plain
  ↪ sparse table would double-count:
// sum, product, matrix product, hashing, "merge two
  ↪ structs", min-count pairs, ...
// (For min/max/gcd/or/and a normal sparse table is
  ↪ smaller - use that instead.)
template <typename T, class F = function<T(T, T)>>
struct DisjointST {
    int n, LOG;
    vector<vector<T>> t;
    F op;

    DisjointST(const vector<T>& a, F op) :
    ↪ n(a.size()), op(op) {
        LOG = 1;
        while ((1 << LOG) < n) LOG++;
        t.assign(LOG + 1, a);
        for (int lvl = 1; lvl <= LOG; lvl++) {
            int half = 1 << lvl;
            for (int mid = half; mid < n + half; mid
    ↪ += 2 * half) {
                if (mid - 1 < n) {
    ↪         // build leftwards from the split
                    t[lvl][mid - 1] = a[mid - 1];
                    for (int i = mid - 2; i >= mid -
    ↪ half && i >= 0; i--)
                        t[lvl][i] = op(a[i], t[lvl][i
    ↪ + 1]);
                }
                if (mid < n) {
    ↪         // and rightwards
                    t[lvl][mid] = a[mid];
                    for (int i = mid + 1; i < min(n,
    ↪ mid + half); i++)
                        t[lvl][i] = op(t[lvl][i - 1],
    ↪ a[i]);
                }
            }
        }
    }
    T query(int l, int r) const {
    ↪         // inclusive [l, r]
        if (l == r) return t[0][l];
        int lvl = 63 - __builtin_clzll((unsigned long
    ↪ long)(l ^ r));   // highest differing bit
        return op(t[lvl][l], t[lvl][r]);
    }
};
// Every query splits at the unique power-of-two
  ↪ boundary between l and r, and both halves are
// already prefix/suffix-accumulated at that level -
  ↪ hence exactly one op() per query.
```

2.13 WaveletTree

WAVELET TREE over values in [lo, hi]. Build O(n log V).

```
// kth(l, r, k)      : k-th SMALLEST in a[l..r]
  ↪                   O(log V)
// countLeq(l,r,x)   : how many a[i] <= x in a[l..r]
  ↪                   O(log V)
// countEq(l,r,x)    : how many a[i] == x in a[l..r]
  ↪                   O(log V)
// Online (no offline sorting, no persistence) - the
  ↪ practical alternative to a merge-sort tree.
struct Wavelet {
    int lo, hi;
    Wavelet *l = nullptr, *r = nullptr;
    vector<int> b;
  ↪ // b[i] = #elements going left among first i

    // build on a[from, to) - the array IS reordered
  ↪ in place (stable partition by midpoint)
    Wavelet(vector<int>::iterator from,
  ↪ vector<int>::iterator to, int x, int y) : lo(x),
  ↪ hi(y) {
        if (from >= to) return;
        if (lo == hi) { b.assign(to - from + 1, 0);
  ↪ return; }
        int mid = lo + (hi - lo) / 2;
        auto f = [mid](int v) { return v <= mid; };
        b.reserve(to - from + 1);
        b.push_back(0);
        for (auto it = from; it != to; ++it)
  ↪ b.push_back(b.back() + f(*it));
        auto pivot = stable_partition(from, to, f);
        l = new Wavelet(from, pivot, lo, mid);
        r = new Wavelet(pivot, to, mid + 1, hi);
    }
    int kth(int L, int R, int k) {
```

```
    // 1-indexed k, inclusive [L, R] (1-based)
        if (L > R) return 0;
        if (lo == hi) return lo;
        int inLeft = b[R] - b[L - 1], lb = b[L - 1],
    rb = b[R];
        if (k <= inLeft) return l->kth(lb + 1, rb, k);
        return r->kth(L - lb, R - rb, k - inLeft);
      }
      int countLeq(int L, int R, int k) {
    // #{i in [L,R] : a[i] <= k}
        if (L > R || k < lo) return 0;
        if (hi <= k) return R - L + 1;
        int lb = b[L - 1], rb = b[R];
        return l->countLeq(lb + 1, rb, k) +
    r->countLeq(L - lb, R - rb, k);
      }
      int countEq(int L, int R, int k) {
        if (L > R || k < lo || k > hi) return 0;
        if (lo == hi) return R - L + 1;
        int lb = b[L - 1], rb = b[R], mid = lo + (hi
    - lo) / 2;
        if (k <= mid) return l->countEq(lb + 1, rb,
    k);
        return r->countEq(L - lb, R - rb, k);
      }
      ~Wavelet() { delete l; delete r; }
};
// Usage: compress values to [0, V), copy the array,
    then Wavelet w(a.begin(), a.end(), 0, V-1);
// queries are 1-BASED on the original positions.
```

2.14 OfflineDynamicConnectivity

OFFLINE DYNAMIC CONNECTIVITY - edges appear and disappear over time; answer queries at each

```
// Needs: DSURollback (02 - DSU/02 - DSURollback.cpp).
// moment. Segment tree over the TIMELINE + DSU with
    rollback. O(q log q * log n) - DSURollback has
    no path compression, so find is log n, not alpha.
// The same skeleton answers ANY "structure supports
    add + rollback, but not delete" problem:
// put each element on the time interval where it is
    alive, DFS the segment tree, add on entry,
// roll back on exit.  ("Deleting from a data
    structure in O(T log n)")
struct DynConn {
    int n, T;
    vector<vector<pair<int, int>>> seg;
    // edges alive on each timeline node
    DSURollback d;
    vector<int> comps;
    // comps[t] = #components at time t

    DynConn(int n, int T) : n(n), T(T), seg(4 *
    max(1, T)), d(n), comps(T, 0) {}

    void addSeg(int v, int l, int r, int ql, int qr,
    pair<int, int> e) {
        if (qr < l || r < ql) return;
        if (ql <= l && r <= qr) {
    seg[v].push_back(e); return; }
        int m = (l + r) / 2;
        addSeg(2 * v, l, m, ql, qr, e), addSeg(2 * v
    + 1, m + 1, r, ql, qr, e);
    }
    // edge (u,v) exists during times [tl, tr]
    (inclusive)
    void addEdge(int u, int v, int tl, int tr) { if
    (tl <= tr) addSeg(1, 0, T - 1, tl, tr, {u, v}); }

    void dfs(int v, int l, int r, int cur) {
        int snap = d.get_state();
        for (auto [a, b] : seg[v]) cur -= d.join(a,
    b);
        if (l == r) comps[l] = cur;
        else {
            int m = (l + r) / 2;
            dfs(2 * v, l, m, cur), dfs(2 * v + 1, m +
    1, r, cur);
        }
        d.rollback_to(snap);
    }
    void solve() { if (T) dfs(1, 0, T - 1, n); }
};
// USAGE: collect every edge's [birth, death]
    interval first (an edge that is never removed
    dies
// at T-1), then one solve() fills comps[]. To answer
    "are u and v connected at time t", instead
// record the queries per leaf and test d.same(u, v)
    inside the l == r branch.
```

2.15 CartesianTree

CARTESIAN TREE in O(n): heap-ordered by VALUE, BST-ordered by INDEX.

```
// The root of any range is its minimum, so "range
    minimum" structure becomes a TREE -
// which turns range problems into subtree problems
    (and enables divide & conquer on the min).
// Built with a monotonic stack; ties go left so the
    tree is unique.
struct CartTree {
    int n, root = -1;
    vector<int> l, r, par;

    CartTree(const vector<ll>& a) : n(a.size()), l(n,
    -1), r(n, -1), par(n, -1) {
        vector<int> st;
        for (int i = 0; i < n; i++) {
            int last = -1;
            while (!st.empty() && a[st.back()] >
    a[i]) last = st.back(), st.pop_back();
            l[i] = last;
            if (last != -1) par[last] = i;
            if (!st.empty()) r[st.back()] = i, par[i]
    = st.back();
            st.push_back(i);
        }
        root = st[0];
    }
};
// USES
//  - "for every element, the maximal range where it
    is the minimum" = its subtree range,
//    which is the contribution technique (sum over
    subarrays of the min) in tree form
//  - largest rectangle in a histogram = best over
    nodes of value * subtreeSize
//  - divide & conquer "solve(l, r) splits at the
    minimum" without recomputing RMQ
//  - counting subarrays whose minimum is a given
    element
//  - a Treap built on (index, random priority) is
    exactly a Cartesian tree - same code shape
```

2.16 SegTreeMerging

SEGMENT TREE MERGING - one dynamic segment tree per vertex over the VALUE domain, merged

```
// bottom-up along a tree. Total O(n log n) nodes touched over ALL merges (amortised), because
// each merge destroys as many nodes as it visits.
// Solves, per subtree and in one DFS: most frequent value, k-th value, count of a value,
// "sum of values in a range" - i.e. everything a merge-sort tree gives, but online per subtree.
struct SegMerge {
    struct Node { int l = 0, r = 0; ll sum = 0; int best = 0; ll bestCnt = 0; };
    vector<Node> t;
    int V;
    // value domain is [0, V)

    SegMerge(int V, int reserve = 0) : V(V) { t.reserve(reserve), t.push_back(Node()); }  // 0 = null
    int newNode() { t.push_back(Node()); return t.size() - 1; }

    void pull(int v) {
        int L = t[v].l, R = t[v].r;
        t[v].sum = t[L].sum + t[R].sum;
        if (t[L].bestCnt >= t[R].bestCnt) t[v].best = t[L].best, t[v].bestCnt = t[L].bestCnt;
        else t[v].best = t[R].best, t[v].bestCnt = t[R].bestCnt;
    }
    // Returns the (possibly new) node index. NEVER take an `int&` into t here: newNode() can
    // reallocate the vector and invalidate the reference mid-recursion.
    int update(int v, int lo, int hi, int p, ll d) {
        if (!v) v = newNode();
        if (hi - lo == 1) { t[v].sum += d, t[v].best = lo, t[v].bestCnt = t[v].sum; return v; }
        int m = (lo + hi) / 2;
        if (p < m) { int c = update(t[v].l, lo, m, p, d); t[v].l = c; }
        else { int c = update(t[v].r, m, hi, p, d); t[v].r = c; }
        pull(v);
        return v;
    }
    int merge(int a, int b, int lo, int hi) {
    // destroys b; returns the merged root
        if (!a || !b) return a | b;
        if (hi - lo == 1) { t[a].sum += t[b].sum, t[a].bestCnt = t[a].sum; return a; }
        int m = (lo + hi) / 2;
        int L = merge(t[a].l, t[b].l, lo, m), R = merge(t[a].r, t[b].r, m, hi);
        t[a].l = L, t[a].r = R;
        pull(a);
        return a;
    }
    ll query(int v, int lo, int hi, int ql, int qr) {
    // sum of counts in [ql, qr)
        if (!v || qr <= lo || hi <= ql) return 0;
        if (ql <= lo && hi <= qr) return t[v].sum;
        int m = (lo + hi) / 2;
        return query(t[v].l, lo, m, ql, qr) + query(t[v].r, m, hi, ql, qr);
    }
    int kth(int v, int lo, int hi, ll k) {
    // k-th smallest (1-indexed), -1 if k too big
        if (!v || k > t[v].sum) return -1;
        if (hi - lo == 1) return lo;
        int m = (lo + hi) / 2;
        return k <= t[t[v].l].sum ? kth(t[v].l, lo, m, k) : kth(t[v].r, m, hi, k - t[t[v].l].sum);
    }
    // wrappers
    void update(int& root, int p, ll d) { root = update(root, 0, V, p, d); }
    int merge(int a, int b) { return merge(a, b, 0, V); }
    ll query(int root, int ql, int qr) { return query(root, 0, V, ql, qr); }
    int kth(int root, ll k) { return kth(root, 0, V, k); }
};
// TREE USAGE: root[v] starts with just v's own value; after recursing, merge every child's root
// into v's. Answer v's query right there. Never reuse a root after merging it away.
```

2.17 BitsetTricks

BITSET SPEEDUPS - the /64 family. All of these are "obviously too slow" DPs that fit anyway.

```
// TRANSITIVE CLOSURE (reachability) in O(n^3 / 64). n up to ~5000.
template <int N>
void transitiveClosure(vector<bitset<N>>& g, int n) {
    for (int k = 0; k < n; k++)
        for (int i = 0; i < n; i++) if (g[i][k]) g[i] |= g[k];
}
// LCS in O(n*m/64) - bit-parallel (Allison-Dix). `a` is the string whose length bounds the
// bitset width; row is a bitmask over positions of `a`.
//   x    = row | match[b_i]
//   row' = x & ((x - (row << 1 | 1)) ^ x)
// The "| 1" is the carry-in that makes the shift borrow correctly.
typedef unsigned long long u64;
int bitLCS(const string& a, const string& b) {
    int n = a.size(), W = (n + 63) / 64;
    if (!n || b.empty()) return 0;
    vector<vector<u64>> match(256, vector<u64>(W, 0));
    for (int i = 0; i < n; i++) match[(unsigned char)a[i]][i >> 6] |= 1ULL << (i & 63);
    vector<u64> row(W, 0), x(W), sh(W);
    for (char c : b) {
        const auto& M = match[(unsigned char)c];
        for (int i = 0; i < W; i++) x[i] = row[i] | M[i];
        u64 carry = 1;
        // sh = (row << 1) | 1
        for (int i = 0; i < W; i++) { u64 nc = row[i] >> 63; sh[i] = (row[i] << 1) | carry; carry = nc; }
        u64 borrow = 0;
        // row = x & ((x - sh) ^ x)
        for (int i = 0; i < W; i++) {
            u64 t = sh[i] + borrow;
            borrow = (t < sh[i]) || (x[i] < t);
            // t < sh[i] catches the carry overflow
            u64 d = x[i] - t;
```

```
            row[i] = x[i] & (d ^ x[i]);
        }
    }
    int r = 0;
    for (u64 v : row) r += __builtin_popcountll(v);
    return r;
}

// SUBSET SUM / KNAPSACK FEASIBILITY: dp |= dp << w
  ↪ (see KnapsackFamily.cpp)
// BIPARTITE MATCHING on dense graphs: keep each left
  ↪ vertex's neighbourhood as a bitset and
//   pick the next free right vertex with (adj[u] &
  ↪ free)._Find_first().
// COUNTING PAIRS with a property: bitset per value
  ↪ class, then popcount of ANDs.
// GCC EXTRAS: b._Find_first(), b._Find_next(i)
  ↪ iterate set bits in O(N/64) total.
// RULE OF THUMB: bitset turns 1e9 "simple"
  ↪ operations into ~1.5e7 word operations - budget
// about n*m/64 <= 1e8 for a comfortable pass.
```

2.18 VeniceAndSWAG

VENICE TECHNIQUE - a multiset with an O(1) "add x to EVERY element".

```
// Keep a global offset; store values shifted by it.
  ↪ 15 lines, and it turns a whole family of
// "everyone loses X health, remove the dead"
  ↪ problems into a one-liner.
struct Venice {
    multiset<ll> s;
    ll water = 0;
  ↪  // global offset

    void add(ll v) { s.insert(v + water); }
  ↪  // insert the true value v
    void erase(ll v) { auto it = s.find(v + water);
  ↪ if (it != s.end()) s.erase(it); }
    void addAll(ll v) { water -= v; }
  ↪  // O(1): add v to every element
    ll getMin() const { return *s.begin() - water; }
    ll getMax() const { return *s.rbegin() - water; }
    int size() const { return s.size(); }
    // Pop everything whose TRUE value is <= t (e.g.
  ↪ everyone who died).
    template <class F> void popLE(ll t, F onPop) {
        while (!s.empty() && *s.begin() - water <= t)
  ↪ { onPop(*s.begin() - water); s.erase(s.begin());
  ↪ }
    }
};
// SAME IDEA elsewhere: a binary trie with a global
  ↪ XOR mask ("xor every element by x"),
// a BIT with a global additive offset, a lazy "add
  ↪ to all" on a heap.

// SWAG (Sliding Window Aggregation) for ANY monoid -
  ↪ non-commutative and non-invertible ops
// included: matrix product, affine composition, gcd,
  ↪ "merge two structs".
// Amortised O(1) push/pop/fold. This generalises
  ↪ 09/02 - TwoStackQueue.cpp, which is hardwired
// to min/max. Prefix-product-and-divide does NOT
  ↪ work without an inverse - this does.
template <class T, class Op>
struct SWAG {
    vector<pair<T, T>> front_, back_;
  ↪  // {value, aggregate from here to the bottom}
    T id;
    Op op;
    SWAG(T id, Op op) : id(id), op(op) {}

    T foldFront() const { return front_.empty() ? id
  ↪ : front_.back().second; }
    T foldBack() const { return back_.empty() ? id :
  ↪ back_.back().second; }
    T fold() const { return op(foldFront(),
  ↪ foldBack()); }        // front first: order
  ↪ matters
    void push(T v) { back_.push_back({v,
  ↪ op(foldBack(), v)}); }
    void pop() {
        if (front_.empty())
            while (!back_.empty()) {
                T v = back_.back().first;
  ↪ back_.pop_back();
                front_.push_back({v, op(v,
  ↪ foldFront())});        // reversed accumulation
            }
        if (!front_.empty()) front_.pop_back();
    }
    int size() const { return front_.size() +
  ↪ back_.size(); }
};
// USAGE: SWAG<ll, function<ll(ll,ll)>> q(0, [](ll a,
  ↪ ll b){ return a + b; });
// For non-commutative ops keep the argument order
  ↪ exactly as written above, or the window
// product comes out reversed.
```

2.19 MergeableHeap

MERGEABLE (MELDABLE) HEAP - a priority queue that also supports MERGING two heaps in O(log n),

```
// which std::priority_queue cannot do. This is a
  ↪ skew heap: merge by walking the two right spines
// and swapping children, which keeps the amortised
  ↪ depth logarithmic with no stored rank.
// Supports a lazy "add v to every key in this heap"
  ↪ tag, because that is what the two algorithms
// that need a mergeable heap actually want.
// USES: Chu-Liu/Edmonds directed MST in O(E log V)
  ↪ (each contracted cycle merges its in-edge
// heaps and subtracts the chosen edge's weight from
  ↪ the whole heap); Dijkstra-style k-shortest
// paths (Eppstein); "merge the two children's
  ↪ candidate sets" DP on trees; leftist-heap
  ↪ solutions
// to scheduling problems (repeatedly merge and pop
  ↪ the largest, e.g. minimise total lateness).
struct MHeap {
    struct Node { ll key, lz = 0; Node *l = 0, *r =
  ↪ 0; };
    static void down(Node* t) {
  ↪          // apply the lazy tag downwards
        if (!t || !t->lz) return;
        t->key += t->lz;
        if (t->l) t->l->lz += t->lz;
        if (t->r) t->r->lz += t->lz;
        t->lz = 0;
    }
    static Node* merge(Node* a, Node* b) {
  ↪          // O(log n) amortised, MIN-heap
        if (!a || !b) return a ? a : b;
        down(a), down(b);
```

```
        if (a->key > b->key) swap(a, b);
        a->r = merge(a->r, b);
        swap(a->l, a->r);
    ↪          // the "skew" step
        return a;
    }
    static Node* push(Node* t, ll key) { return
    ↪ merge(t, new Node{key}); }
    static ll top(Node* t) { return down(t), t->key; }
    static Node* pop(Node* t) { down(t); return
    ↪ merge(t->l, t->r); }
    static void addAll(Node* t, ll v) { if (t) t->lz
    ↪ += v; }   // O(1) add to every key
};
// TO GET A MAX-HEAP: negate the keys, or flip the
    ↪ comparison in merge.
// LEFTIST HEAP is the same interface with a stored
    ↪ ‘dist‘ (null-path length) and the rule "keep the
// larger dist on the left"; it gives WORST-CASE
    ↪ O(log n) instead of amortised. Skew is shorter
    ↪ and
// fine unless you need the worst-case bound inside
    ↪ another amortised argument. 24 - PersistentHeap
// is that leftist version, made persistent (meld
    ↪ without destroying either input).
// PAIRING HEAP has O(1) merge and decrease-key in
    ↪ practice and is what you want if you need
// decrease-key (Dijkstra on very dense graphs); in
    ↪ contests a binary heap with lazy deletion wins.
// LAZY DELETION is usually enough: push (key, id),
    ↪ and pop until the top is still valid. That
// removes the need for decrease-key entirely and is
    ↪ 5 lines.
// SMALL-TO-LARGE with std::priority_queue also
    ↪ "merges" heaps: repeatedly pop from the smaller
    ↪ and
// push into the larger, O(n log^2 n) total. Shorter
    ↪ than this file - use it unless the log^2 hurts.
```

2.20 PersistentAndUndo

PERSISTENCE AND UNDO - three ways to travel in a structure's history, and when each applies.

```
//   ROLLBACK  (05 - DSU/02): undo the last op.
    ↪ Cheapest. Needs a stack discipline (LIFO).
//   OFFLINE DELETION (14 -
    ↪ OfflineDynamicConnectivity): segment tree over
    ↪ time + rollback.
//     Handles arbitrary insert/delete, but you must
    ↪ know every operation in advance.
//   PERSISTENCE (this file): branch the history. The
    ↪ only one that works ONLINE with deletions,
//     and the only one that lets you query a past
    ↪ version after moving on.

// --- PERSISTENT ARRAY: a segment tree over indices.
    ↪ get/set in O(log n), O(log n) new nodes. ---
struct PArray {
    struct Node { int l = 0, r = 0; ll v = 0; };
    vector<Node> t;
    int n;
    PArray(int n) : t(1), n(n) {}
    ↪          // node 0 is the shared all-zero
    int build(const vector<ll>& a, int lo, int hi) {
        int id = t.size();
        t.push_back({});
        if (lo == hi) { t[id].v = a[lo]; return id; }
        int m = (lo + hi) / 2, L = build(a, lo, m), R
    ↪ = build(a, m + 1, hi);
        t[id].l = L, t[id].r = R;
        return id;
    }
    int build(const vector<ll>& a) { return build(a,
    ↪ 0, n - 1); }
    ll get(int u, int i, int lo, int hi) {
        if (lo == hi) return t[u].v;
        int m = (lo + hi) / 2;
        return i <= m ? get(t[u].l, i, lo, m) :
    ↪ get(t[u].r, i, m + 1, hi);
    }
    ll get(int u, int i) { return get(u, i, 0, n -
    ↪ 1); }
    int set(int u, int i, ll v, int lo, int hi) {
        int id = t.size();
        t.push_back(t[u]);
        if (lo == hi) { t[id].v = v; return id; }
        int m = (lo + hi) / 2;
        if (i <= m) t[id].l = set(t[u].l, i, v, lo,
    ↪ m);
        else t[id].r = set(t[u].r, i, v, m + 1, hi);
        return id;
    }
    int set(int u, int i, ll v) { return set(u, i, v,
    ↪ 0, n - 1); }
};
// --- PERSISTENT DSU on top of it. O(log^2 n) per
    ↪ operation. ---
// The catch that makes it work: NO PATH COMPRESSION
    ↪ (that would mutate an old version), so you
// MUST union by size/rank to keep the height O(log
    ↪ n).
struct PDSU {
    PArray par, siz;
    PDSU(int n) : par(n), sz(n) {}
    pair<int, int> init(int n) {
    ↪          // returns {parRoot, szRoot}
        vector<ll> p(n), s(n, 1);
        iota(all(p), 0);
        return {par.build(p), siz.build(s)};
    }
    int find(int pr, int x) { ll p = par.get(pr, x);
    ↪ return p == x ? x : find(pr, p); }
    pair<int, int> join(int pr, int sr, int a, int b)
    ↪ {
        a = find(pr, a), b = find(pr, b);
        if (a == b) return {pr, sr};
        if (siz.get(sr, a) < siz.get(sr, b)) swap(a,
    ↪ b);
        return {par.set(pr, b, a), siz.set(sr, a,
    ↪ siz.get(sr, a) + siz.get(sr, b))};
    }
};
/* THE OTHER TWO TIME-TRAVEL TRICKS, as recipes
    ↪ (code: 21 - QueueUndoTrick, 25 - StaticToDynamic)
 QUEUE UNDO. You have a structure that supports only
    ↪ "undo the last op" (rollback DSU), but the
    deletions arrive in FIFO order (a sliding window).
    ↪ Keep the undo stack partitioned so the
    OLDEST element is near the top; when it is not,
    ↪ pop a prefix and re-push it in the other order,
    always rebuilding the SMALLER half. O(log n)
    ↪ amortised extra. This is the online alternative
    ↪ to
    the offline segment-tree-on-time trick, and it is
    ↪ what "sliding window connectivity" needs.
 STATIC TO DYNAMIC (the logarithmic method). Any
```

```
   ↪ STATIC structure with O(B(n)) build and O(Q(n))
   query becomes insert-only: keep O(log n) instances
   ↪ of sizes 2^0, 2^1, 2^2, ...; inserting is
   binary-counter carrying, so you rebuild an
   ↪ instance of size 2^k only every 2^k insertions.
   Insert becomes O(B(n)/n * log n) amortised and
   ↪ query O(Q(n) log n) (ask all instances).
   Use it to make a convex hull / a wavelet tree / an
   ↪ Aho-Corasick automaton accept insertions.
 PARTIALLY vs FULLY PERSISTENT: partially = you may
   ↪ query any past version but only modify the
   latest - for a DSU that is 05 - DSU/05 -
   ↪ PartiallyPersistentDSU, O(n) memory instead of
   ↪ this
   file's O(n log n); fully = you may modify any
   ↪ version, branching the history into a tree.
   The code above is fully persistent.
 WHEN NOT TO BOTHER: if the problem is offline and
   ↪ only needs "delete", the segment tree over the
   timeline with a rollback DSU is shorter, faster,
   ↪ and has a smaller constant than any of this. */
```

2.21 QueueUndoTrick

QUEUE UNDO TRICK - turns a structure that can only "undo the LAST operation" into one that can

```
// Needs: 05 - Disjoint Set Union (DSU)/02 -
   ↪ DSURollback.cpp
// also "undo the OLDEST operation", i.e. a QUEUE of
   ↪ operations instead of a stack.
// O(log n) amortised extra factor on top of the
   ↪ underlying structure.
// WHEN: a sliding window over operations - "for each
   ↪ l, the largest r such that edges l..r keep
// the graph bipartite / acyclic / with <= k
   ↪ components" (two pointers over edges), or any
   ↪ online
// problem where insertions and deletions are FIFO.
   ↪ Deletions in arbitrary order need
// 14 - OfflineDynamicConnectivity (offline) or a
   ↪ link-cut tree (forests only).
// KEY INVARIANT: the undo stack is kept partitioned
   ↪ into a "to be deleted soon" prefix and the
// rest. To pop the front, undo a suffix of the
   ↪ stack, then re-push it with the OLDEST elements
// nearest the top; the rebuild always processes the
   ↪ SMALLER of the two groups, which is what makes
// the potential argument work - each operation is
   ↪ re-pushed O(log n) times over its lifetime.
// The underlying structure must support: apply(op)
   ↪ and undo-last, both cheap; nothing else.
struct QueueUndo {
    struct Op { int u, v; bool old; };
   ↪          // old = "already re-pushed once"
    DSURollback d;
    vector<Op> st;
   ↪          // mirrors the undo stack exactly
    int bot = 0;
   ↪          // first index that is not yet `old`
    QueueUndo(int n) : d(n) {}
    void push(Op o) { d.join(o.u, o.v),
   ↪ st.push_back(o); }
    void push(int u, int v) { push({u, v, false}); }
    void pop() {
   ↪          // remove the OLDEST operation
        vector<Op> a[2];
        while (bot < (int)st.size() && st[bot].old)
   ↪ bot++;
        if (bot == (int)st.size()) {
   ↪          // no old ops left: flip everything
            while (!st.empty())
   ↪ a[0].push_back(st.back()), st.pop_back(),
   ↪ d.rollback();
            for (auto& o : a[0]) o.old = true,
   ↪ push(o);
            a[0].clear(), bot = 0;
        }
        do {
   ↪          // pull back until the front is up
            a[st.back().old].push_back(st.back());
            st.pop_back(), d.rollback();
        } while (a[1].size() < a[0].size() &&
   ↪ (int)st.size() > bot);
        for (int i : {0, 1}) {
            reverse(all(a[i]));
            for (auto& o : a[i]) push(o);
        }
        st.pop_back(), d.rollback();
   ↪          // the front is now on top
    }
};
// THE STRUCTURE UNDERNEATH can be anything with
   ↪ rollback: a DSU (connectivity, bipartiteness by
// storing the parity to the parent, number of
   ↪ components), a XOR basis, a max-stack, a knapsack
// with rollback. Replace push()/pop() bodies and
   ↪ keep the stack discipline identical.
// SLIDING WINDOW RECIPE: for l = 1..m, push edges
   ↪ while the invariant holds, record the answer,
// then pop() the edge at l. Total O(m log m * alpha).
// CHEAPER OPTIONS FIRST: if the problem is offline,
   ↪ the segment tree over the timeline
// (14 - OfflineDynamicConnectivity) is shorter and
   ↪ has a better constant. If the deletions are
// LIFO, plain rollback is enough. This file is only
   ↪ for FIFO + online.
```

2.22 PermutationTree

PERMUTATION TREE (PQ-tree / substitution decomposition) - represents ALL "common intervals" of a

```
// permutation: the ranges [l, r] whose VALUES also
   ↪ form a contiguous range. There can be O(n^2) of
// them; this tree stores every one of them in O(n)
   ↪ space, in O(n log n) time.
// EVERY node has a contiguous span of POSITIONS and
   ↪ a contiguous range of VALUES, and is one of:
//   LEAF        - a single position.
//   JOIN node   - its children's value ranges are
   ↪ consecutive and monotone (increasing = type 1,
//                 decreasing = type 2). ANY
   ↪ consecutive run of >= 2 of its children is
   ↪ itself a
//                 common interval. This is where the
   ↪ O(n^2) of them hide.
//   CUT (prime) - no proper consecutive run of its
   ↪ children is a common interval; only the whole
//                 node is. A cut node always has >=
   ↪ 4 children.
// WHEN: "count subarrays whose values are
   ↪ consecutive integers", "the number of ways to
   ↪ split a
// permutation into monotone blocks",
   ↪ simple-permutation questions, and the smallest
   ↪ common
// interval containing a given [l, r]. If you only
```

```
  ↪ need to COUNT common intervals, the divide and
// conquer with min/max stacks is shorter - reach for
  ↪ the tree when you need the STRUCTURE.
// KEY INVARIANT: sweep r and keep, for every l, the
  ↪ quantity (max - min) - (r - l) over a[l..r],
// which is >= 0 and equals 0 exactly when [l, r] is
  ↪ a common interval. Monotone stacks maintain it
// under range add in O(1) amortised each, and a min
  ↪ segment tree finds the zeros. Whenever the new
// element extends an existing node's value range by
  ↪ one the node absorbs it (join), and whenever a
// zero appears further left a cut node is created
  ↪ over everything between.
struct PermTree {
    int n, id, root = 0;
    vector<int> p, ty, par, mn, lz;
  ↪          // ty: 0 cut, 1 up-join, 2 down-join
    vector<pii> val, span;
  ↪          // value range, position range
    vector<vector<int>> ch;
    PermTree(const vector<int>& perm)
  ↪          // perm[1..n], a permutation of 1..n
        : n(perm.size() - 1), id(0), p(perm), ty(2 *
  ↪ perm.size(), 0), par(2 * perm.size(), -1),
          mn(8 * perm.size(), 0), lz(8 * perm.size(),
  ↪ 0), val(2 * perm.size()),
          span(2 * perm.size()), ch(2 * perm.size())
  ↪ { build(); }
    void apply(int x, int v) { mn[x] += v, lz[x] +=
  ↪ v; }
    void push(int x) { if (lz[x]) apply(2 * x,
  ↪ lz[x]), apply(2 * x + 1, lz[x]), lz[x] = 0; }
    void upd(int x, int b, int e, int l, int r, int
  ↪ v) {
        if (r < b || e < l) return;
        if (l <= b && e <= r) { apply(x, v); return; }
        int m = (b + e) / 2;
        push(x);
        upd(2 * x, b, m, l, r, v), upd(2 * x + 1, m +
  ↪ 1, e, l, r, v);
        mn[x] = min(mn[2 * x], mn[2 * x + 1]);
    }
    int qry(int x, int b, int e, int l, int r) {
        if (r < b || e < l || l > r) return inf;
        if (l <= b && e <= r) return mn[x];
        int m = (b + e) / 2;
        push(x);
        return min(qry(2 * x, b, m, l, r), qry(2 * x
  ↪ + 1, m + 1, e, l, r));
    }
    void link(int u, int v) { par[v] = u,
  ↪ ch[u].push_back(v); }
    bool adj(int i, int j) { return val[i].se ==
  ↪ val[j].fi - 1; }     // j starts right after i
    int len(int i) { return val[i].se - val[i].fi +
  ↪ 1; }
    static pii un(pii a, pii b) { return {min(a.fi,
  ↪ b.fi), max(a.se, b.se)}; }
    void build() {
        id = n + 1;
        vector<int> mx{0}, mi{0}, st;
  ↪          // monotone stacks + node stack
        for (int i = 1; i <= n; i++) {
            while (mx.back() && p[mx.back()] < p[i]) {
                int r = mx.back();
                mx.pop_back();
                upd(1, 1, n, mx.back() + 1, r, p[i] -
  ↪ p[r]);
            }
            mx.push_back(i);
            while (mi.back() && p[mi.back()] > p[i]) {
                int r = mi.back();
                mi.pop_back();
                upd(1, 1, n, mi.back() + 1, r, p[r] -
  ↪ p[i]);
            }
            mi.push_back(i);
            val[i] = {p[i], p[i]}, span[i] = {i, i};
            int cur = i;
            while (true) {
                if (!st.empty() && (adj(st.back(),
  ↪ cur) || adj(cur, st.back()))) {
                    int b = st.back(), want = adj(b,
  ↪ cur) ? 1 : 2;
                    if (ty[b] == want) {
  ↪          // absorb into the existing join
                        link(b, cur);
                        val[b] = un(val[b],
  ↪ val[cur]), span[b] = un(span[b], span[cur]);
                        cur = b;
                    } else {
  ↪          // start a new join node
                        ty[id] = want, link(id, b),
  ↪ link(id, cur);
                        val[id] = un(val[b],
  ↪ val[cur]), span[id] = un(span[b], span[cur]);
                        cur = id++;
                    }
                    st.pop_back();
                } else if (qry(1, 1, n, 1, i -
  ↪ len(cur)) == 0) {     // a cut node closes here
                    int L = len(cur);
                    pii v = val[cur], s = span[cur];
                    link(id, cur);
                    do {
                        int b = st.back();
                        L += len(b), v = un(v,
  ↪ val[b]), s = un(s, span[b]);
                        link(id, b), st.pop_back();
                    } while (v.se - v.fi + 1 != L);
                    reverse(all(ch[id]));
                    val[id] = v, span[id] = s, cur =
  ↪ id++;
                } else break;
            }
            st.push_back(cur);
            upd(1, 1, n, 1, i, -1);
        }
        root = st.back();
    }
};
// COUNTING COMMON INTERVALS: sum over join nodes
  ↪ with k children of k*(k-1)/2, plus n (the
  ↪ leaves),
// plus the number of cut nodes. That is the classic
  ↪ "count subarrays forming a value range".
// SMALLEST COMMON INTERVAL CONTAINING [l, r]:
  ↪ binary-lift from leaf l to the highest ancestor
  ↪ whose
// span still ends before r, take its parent u. If u
  ↪ is a cut node the answer is span[u]; if u is a
// join node the answer is the run of u's children
  ↪ from the one containing l to the one containing
// r (binary search over the children, whose spans
  ↪ are sorted).
// NODE COUNT is < 2n, so size every array 2n. Leaves
```

```
  ↪ are 1..n, internal nodes n+1..id-1.
// SIMPLE PERMUTATIONS are exactly those whose tree
  ↪ is a single cut node over n leaves.
// THIS IS MODULAR DECOMPOSITION specialised to
  ↪ permutation graphs - the same "join / cut" shape
// appears for cographs and for general graphs, where
  ↪ it is much harder to compute.
```

2.23 DynamicTreeDiameter

DYNAMIC TREE DIAMETER - the diameter of a weighted tree under ONLINE edge-weight changes,

```
// O(log n) per update, O(1) to read the answer. The
  ↪ tree SHAPE is fixed; only the weights move.
// WHEN: "after each update print the diameter" (CF
  ↪ 1192B and every problem shaped like it). If the
// weights are static, two DFS or the tree DP in 09 -
  ↪ Trees/05 is O(n). If edges are ADDED or
// REMOVED you need a link-cut tree (09 - Trees/11)
  ↪ or offline divide and conquer instead.
// KEY INVARIANT: put the tree on its Euler tour and
  ↪ let f[p] = depth of the vertex visited at
// position p. For any x <= y <= z in the tour, f[x]
  ↪ - 2f[y] + f[z] <= dist between the vertices at
// x and z (with equality when y is their LCA, which
  ↪ is always visited between them), so the
// diameter is exactly max over x <= y <= z of f[x] -
  ↪ 2f[y] + f[z]. Changing an edge's weight by
// delta adds delta to f over the CONTIGUOUS tour
  ↪ range of the child's subtree - a range add. The
// segment tree node keeps the five best partial
  ↪ expressions so they merge in O(1):
//   a = max f[x],  b = max -2f[y],  c = max
  ↪ f[x]-2f[y],  d = max -2f[y]+f[z],  e = the
  ↪ answer.
struct DynDiameter {
    struct Node { ll a = 0, b = 0, c = 0, d = 0, e =
  ↪ 0; };
    static Node mrg(const Node& L, const Node& R) {
        Node x;
        x.a = max(L.a, R.a), x.b = max(L.b, R.b);
        x.c = max({L.c, R.c, L.a + R.b});
        x.d = max({L.d, R.d, L.b + R.a});
        x.e = max({L.e, R.e, L.c + R.a, L.a + R.d});
        return x;
    }
    int n, T = 0;
    vector<Node> t;
    vector<ll> lz, w;
    vector<int> st, en, low;
  ↪          // low[e] = deeper endpoint of edge e
    vector<vector<pii>> g;
  ↪          // {to, edge id}
    DynDiameter(int n) : n(n), t(16 * n), lz(16 * n,
  ↪ 0), st(n), en(n), g(n) {}
    void addEdge(int u, int v, ll c) {
        int e = w.size();
        w.push_back(c), low.push_back(-1);
        g[u].push_back({v, e}), g[v].push_back({u,
  ↪ e});
    }
    void dfs(int u, int p) {
        st[u] = ++T;
        for (auto [v, e] : g[u])
            if (v != p) low[e] = v, dfs(v, u), ++T;
        en[u] = T;
    }
    void apply(int x, ll v) {
        t[x].a += v, t[x].b -= 2 * v, t[x].c -= v,
  ↪ t[x].d -= v, lz[x] += v;
    }
    void push(int x) { if (lz[x]) apply(2 * x,
  ↪ lz[x]), apply(2 * x + 1, lz[x]), lz[x] = 0; }
    void upd(int x, int lo, int hi, int l, int r, ll
  ↪ v) {
        if (r < lo || hi < l) return;
        if (l <= lo && hi <= r) { apply(x, v);
  ↪ return; }
        int m = (lo + hi) / 2;
        push(x);
        upd(2 * x, lo, m, l, r, v), upd(2 * x + 1, m
  ↪ + 1, hi, l, r, v);
        t[x] = mrg(t[2 * x], t[2 * x + 1]);
    }
    void build(int root = 0) {
  ↪          // call after every addEdge
        dfs(root, -1);
        for (int e = 0; e < (int)w.size(); e++) {
            ll c = w[e];
            w[e] = 0;
            setW(e, c);
  ↪          // all f start at 0, so just add
        }
    }
    void setW(int e, ll c) {
  ↪          // set edge e's weight to c
        upd(1, 1, T, st[low[e]], en[low[e]], c -
  ↪ w[e]);
        w[e] = c;
    }
    ll diameter() { return t[1].e; }
};
// NEGATIVE WEIGHTS break it: the argument needs f[x]
  ↪ - 2f[LCA] + f[z] to be maximised at the true
// deepest pair, which assumes non-negative edges.
  ↪ Shift all weights up if you must.
// THE SAME TOUR TRICK gives dist(u, v) queries under
  ↪ updates: dist = f[st[u]] + f[st[v]] -
// 2 * min f over the tour between them, so keep a
  ↪ min segment tree alongside.
// DIAMETER ENDPOINTS: store argmaxes next to each of
  ↪ the five maxima; the merge picks them up
// unchanged. Only do this if the problem asks - it
  ↪ triples the node.
// SIMPLER SPECIAL CASE: if updates only ever
  ↪ INCREASE weights, the diameter is monotone and a
// small-to-large "merge the two deepest per subtree"
  ↪ DP is enough.
```

2.24 PersistentHeap

PERSISTENT MELDABLE HEAP - push / pop / meld that RETURN a new heap and leave the old ones

```
// intact. O(log n) time and O(log n) new nodes per
  ↪ operation.
// WHEN: an algorithm needs many heaps that share
  ↪ most of their contents. The canonical one is
// EPPSTEIN's k shortest walks: every vertex's
  ↪ candidate heap is its parent's heap plus its own
// out-edges, so you meld into a COPY and keep the
  ↪ parent's version alive. Also "k best paths in a
// DAG", "k-th best subset", and persistent priority
  ↪ queues in offline simulations.
// If nothing needs the old version, use 19 -
  ↪ MergeableHeap (destructive, no allocation per
  ↪ op).
```

```
// KEY INVARIANT: leftist heap - every node stores d
   ↪ = its null-path length and keeps the LONGER
// path on the left, so the right spine is O(log n)
   ↪ and meld only ever walks right spines. Making
// it persistent then costs nothing extra: COPY each
   ↪ node you touch on that spine instead of
// mutating it, which is exactly O(log n) copies.
struct PHeap {
    struct Node { int l = 0, r = 0, d = 0; ll key =
   ↪ 0; };
    vector<Node> t;
    PHeap() : t(1) {}
   ↪          // node 0 is the empty heap, d = 0
    int meld(int a, int b) {
   ↪          // returns a NEW root, min-heap
        if (!a || !b) return a | b;
        if (t[a].key > t[b].key) swap(a, b);
        int c = t.size();
        t.push_back(t[a]);
   ↪          // copy, never mutate a
        int r = meld(t[c].r, b);
        t[c].r = r;
        if (t[t[c].l].d < t[r].d) swap(t[c].l,
   ↪ t[c].r);
        t[c].d = t[t[c].r].d + 1;
        return c;
    }
    int push(int a, ll key) {
        t.push_back({0, 0, 1, key});
        return meld(a, (int)t.size() - 1);
    }
    ll top(int a) { return t[a].key; }
   ↪          // a != 0
    int pop(int a) { return meld(t[a].l, t[a].r); }
    bool empty(int a) { return !a; }
};
// MEMORY: O((n + q) log n) nodes. If a heap is only
   ↪ ever pushed into (never melded), a persistent
// BINARY heap over a persistent array (20 -
   ↪ PersistentAndUndo) is smaller but has no meld.
// A LAZY ADD-TO-ALL tag does NOT combine with
   ↪ persistence for free: a tag is a mutation, so
   ↪ store
// the offset OUTSIDE the heap (carry "this version's
   ↪ keys are all shifted by s" in the caller) -
// that is what Eppstein and the directed-MST
   ↪ contraction actually do.
// PERSISTENT QUEUE, if that is what you came for, is
   ↪ much cheaper: keep a persistent ARRAY plus
// two indices (head, tail) per version. push = write
   ↪ at tail and store {head, tail+1}; pop = read
// at head and store {head+1, tail}. O(log n) per op,
   ↪ no heap needed.
// PERSISTENT STACK is a linked list: every version
   ↪ is just a node pointer. O(1) per op.
```

2.25 StaticToDynamic

STATIC TO DYNAMIC (the LOGARITHMIC METHOD) - makes ANY static structure accept insertions.

```
// If the static one builds in O(B(n)) and answers in
   ↪ O(Q(n)), you get insert in O(B(n)/n * log n)
// AMORTISED and query in O(Q(n) * log n) - you ask
   ↪ every bucket and combine.
// WHEN: you have a build-once structure (convex
   ↪ hull, sorted array, wavelet tree, suffix
// automaton, Aho-Corasick, KD-tree, sparse table)
   ↪ and the problem inserts items online. Do NOT
// use it when a natively dynamic structure exists
   ↪ and is fast enough - a BIT beats "buckets of
// sorted arrays" for prefix counting, and a balanced
   ↪ BST beats "buckets of sorted arrays" for
// order statistics.
// KEY INVARIANT: keep at most one instance of each
   ↪ size 2^0, 2^1, 2^2, ... - a BINARY COUNTER.
// Inserting sets the lowest empty bit and merges
   ↪ every full bucket below it, so a bucket of size
// 2^k is rebuilt only once per 2^k insertions;
   ↪ summed over k that is O(log n) rebuilds of
// amortised cost B(n)/n each. Because the buckets
   ↪ partition the items, ANY decomposable query
// (one whose answer over a union is a cheap
   ↪ combination of the answers over the parts:
   ↪ count, sum,
// min, max, "does there exist") can be answered by
   ↪ asking all O(log n) of them.
template <class Static, class Item>
struct StaticToDynamic {
    vector<vector<Item>> buf;
   ↪          // buf[k] is empty or has 2^k items
    vector<Static> ds;
    void insert(Item x) {
        vector<Item> cur{x};
        int k = 0;
        while (k < (int)buf.size() &&
   ↪ !buf[k].empty()) {
            for (auto& y : buf[k]) cur.push_back(y);
            buf[k].clear(), ds[k] = Static();
            k++;
        }
        while ((int)buf.size() <= k)
   ↪ buf.emplace_back(), ds.emplace_back();
        buf[k] = cur, ds[k] = Static(cur);
   ↪          // one rebuild, size 2^k
    }
    template <class F> void each(F f) {
   ↪          // f(instance) over every bucket
        for (int k = 0; k < (int)ds.size(); k++) if
   ↪ (!buf[k].empty()) f(ds[k]);
    }
};
// EXAMPLE static structure - a sorted array
   ↪ answering "how many items <= x".
struct SortedBucket {
    vector<ll> v;
    SortedBucket() {}
    SortedBucket(vector<ll> a) : v(a) { sort(all(v));
   ↪ }
    int count(ll x) { return upper_bound(all(v), x) -
   ↪ v.begin(); }
};
// USAGE: StaticToDynamic<SortedBucket, ll> d;
   ↪ d.insert(5);
//        int c = 0; d.each([&](SortedBucket& b) { c
```

```
    ↪ += b.count(x); });
// DELETIONS, if the aggregate is INVERTIBLE (counts,
    ↪ sums): keep a SECOND instance holding the
// deleted items and answer query(all) -
    ↪ query(deleted). This is "deletion by counting"
    ↪ and it is
// how you get a fully dynamic convex-hull-trick or a
    ↪ deletable "count <= x".
// DELETIONS otherwise (min, max, "exists"): you
    ↪ cannot subtract - use offline dynamic
    ↪ connectivity
// style divide and conquer over the timeline (14)
    ↪ instead.
// WORST CASE: a single insertion can cost O(B(n))
    ↪ (rebuilding the whole thing). If the problem has
// a per-query time limit rather than a total one,
    ↪ spread the rebuild over the next 2^k insertions
// (the "global rebuilding" / de-amortisation trick)
    ↪ - rarely needed in contests.
```

2.26 TreeBinarization

TREE BINARIZATION - rewrite a tree so that EVERY vertex has degree <= 3, by inserting dummy

```
// vertices joined with ZERO-weight edges. O(n) time,
    ↪ at most n - 2 new vertices, and every
// distance between original vertices is unchanged.
// WHEN: it is a PRECONDITION, not an algorithm.
    ↪ Reach for it whenever a per-vertex cost is
// proportional to the DEGREE and you need it to be
    ↪ O(1):
//   - PERSISTENT centroid decomposition (27):
    ↪ copying a centroid node must be O(1), so a
    ↪ centroid
//     may have at most 3 children in the centroid
    ↪ tree.
//   - "merge the children's DP tables"
    ↪ small-to-large / knapsack-on-tree DPs: after
    ↪ binarization
//     each merge has exactly two operands, which is
    ↪ what makes the classic O(n^2) tree knapsack
//     analysis (and the O(n log n) small-to-large
    ↪ bound) actually hold.
//   - dynamic-tree structures whose node stores a
    ↪ fixed number of child pointers (top trees).
// KEY INVARIANT: replace the star u -> c1, c2, ...,
    ↪ ck by a CATERPILLAR: u keeps c1 and a chain of
// dummies d1 - d2 - ... carries the rest, each dummy
    ↪ holding one real child. Zero-weight edges mean
// distances are preserved exactly; original vertex
    ↪ ids are preserved, so any answer indexed by
// vertex still makes sense. Depth grows by at most
    ↪ O(deg), so a DFS on the result is still O(n).
struct Binarize {
    int n, cnt;
    ↪           // 1..n original, n+1..cnt dummies
    vector<vector<pii>> g;
    ↪           // result: {neighbour, weight}
    Binarize(const vector<vector<pii>>& G, int root =
    ↪ 1)
        : n(G.size() - 1), cnt(G.size() - 1), g(2 *
    ↪ G.size()) { rec(G, root, 0); }
    void link(int u, int v, int w) {
    ↪ g[u].push_back({v, w}), g[v].push_back({u, w}); }
    void rec(const vector<vector<pii>>& G, int u, int
    ↪ p) {
        int last = u, k = 0, deg = G[u].size() - (p
    ↪ != 0);
        for (auto [v, w] : G[u]) {
            if (v == p) continue;
            if (++k == 1 || k == deg) link(last, v,
    ↪ w);          // first and last hang off `last`
            else {
                int d = ++cnt;
    ↪           // dummy carrying one middle child
                link(last, d, 0), link(d, v, w), last
    ↪ = d;
            }
        }
        for (auto [v, w] : G[u]) if (v != p) rec(G,
    ↪ v, u);
    }
};
// SIZE EVERY ARRAY 2n - the vertex count can nearly
    ↪ double.
// WEIGHTS: dummy edges are 0, so shortest paths,
    ↪ diameters and centroid distances are identical.
// If your problem counts EDGES rather than weights,
    ↪ give the real edges weight 1 and the dummies 0.
// VERTEX VALUES: dummies must be neutral for
    ↪ whatever you aggregate (0 for sums, +inf for
    ↪ minima).
// BINARY (degree <= 3) IS THE RIGHT TARGET, not
    ↪ "binary rooted tree": a rooted binarization that
// keeps exactly two children per node needs the same
    ↪ construction plus a root of degree <= 2.
```

2.27 PersistentCentroid

PERSISTENT CENTROID DECOMPOSITION - "sum of dist(u, x) over all marked vertices x", where the

```
// Needs: 26 - TreeBinarization.cpp
// set of marked vertices is versioned: version i =
    ↪ the first i marks. Insert O(log n), query
// O(log n), memory O(n log n).
// WHEN: queries of the form "consider only the marks
    ↪ with index in [l, r], answer for vertex u" -
// subtract two versions (CF 757G). Also "the k-th
    ↪ mark", online, when marks arrive in a fixed
// order. If the mark set only ever grows and you
    ↪ never look back, plain centroid decomposition
// (09 - Trees/04) with the same accumulators is
    ↪ enough and far cheaper.
// COMPLEXITY REQUIRES BINARIZATION FIRST: a centroid
    ↪ node must have O(1) children so that copying
// it is O(1). Feed the tree through 26 -
    ↪ TreeBinarization.
// KEY INVARIANT: the centroid tree has depth O(log
    ↪ n) and every vertex u lies in exactly one
// component per level, so dist(u, x) = d[level][u] +
    ↪ d[level][x] where the level is that of the
// LOWEST common centroid of u and x. Each centroid
    ↪ node stores cnt (marks in its component), sum
// (total distance from the centroid to them), and
    ↪ par (their total distance to the centroid's
// PARENT centroid). Walking from the root of the
    ↪ centroid tree down to u then adds, at each step,
// the contribution of exactly the marks that "branch
    ↪ off" there - the standard inclusion-exclusion
// that stops double counting. Marking one vertex
    ↪ touches only the O(log n) centroid nodes on that
// root-to-u path, so persistence costs O(log n)
    ↪ copies.
struct PCentroid {
    struct Node { array<int, 3> ct{{0, 0, 0}}; int id
    ↪ = 0, lvl = 0, cnt = 0; ll sum = 0, par = 0; };
```

```
    vector<Node> t;
    vector<vector<pii>> g;
                // BINARISED tree, 1-indexed
    vector<vector<ll>> d;
                // d[level][v]
    vector<int> st, en, sub, done;
    int n, DT = 0, tot = 0, root = 0;
    PCentroid(const vector<vector<pii>>& g, int nv,
int LG = 20)
        : t(nv + 1), g(g), d(LG, vector<ll>(nv + 1,
0)), st(nv + 1), en(nv + 1), sub(nv + 1),
          done(nv + 1, 0), n(nv) { root =
decompose(1, 0, 0); }
    void calcSz(int u, int p) {
        tot++, sub[u] = 1;
        for (auto [v, w] : g[u])
            if (v != p && !done[v]) calcSz(v, u),
sub[u] += sub[v];
    }
    int findCen(int u, int p) {
        for (auto [v, w] : g[u])
            if (v != p && !done[v] && sub[v] > tot /
2) return findCen(v, u);
        return u;
    }
    void dist(int u, int p, ll cur, int l) {
        d[l][u] = cur;
        for (auto [v, w] : g[u])
            if (v != p && !done[v]) dist(v, u, cur +
w, l);
    }
    int decompose(int u, int p, int l) {
        tot = 0, calcSz(u, p);
        int c = findCen(u, p);
        done[c] = 1, st[c] = ++DT, t[c].id = c,
t[c].lvl = l;
        dist(c, 0, 0, l);
        int k = 0;
        for (auto [v, w] : g[c])
            if (!done[v]) t[c].ct[k++] = decompose(v,
c, l + 1);
        en[c] = DT;
        return c;
    }
    bool insub(int x, int u) { return st[t[x].id] <=
st[u] && en[u] <= en[t[x].id]; }
    int mark(int cur, int u) {
                // returns the NEW version root
        t.push_back(t[cur]);
        int c = t.size() - 1;
        ll dv = d[t[c].lvl][u];
        t[c].cnt++, t[c].sum += dv;
        for (int k = 0; k < 3; k++) {
            int v = t[c].ct[k];
            if (v && insub(v, u)) {
                int nv = mark(v, u);
                t[nv].par += dv, t[c].ct[k] = nv;
            }
        }
        return c;
    }
    ll query(int cur, int u) {
                // sum of dist(u, x) over marks
        ll ans = 0;
        while (t[cur].id != u) {
            int v = 0;
            for (int k = 0; k < 3; k++)
                if (t[cur].ct[k] &&
insub(t[cur].ct[k], u)) v = t[cur].ct[k];
                ans += d[t[cur].lvl][u] * (t[cur].cnt -
t[v].cnt) + t[cur].sum - t[v].par;
                cur = v;
            }
            return ans + t[cur].sum;
        }
};
// USAGE: Binarize b(G); PCentroid pc(b.g, b.cnt);
then ver[0] = pc.root and
// ver[i] = pc.mark(ver[i-1], x_i). The answer for
marks l..r is
// pc.query(ver[r], u) - pc.query(ver[l-1], u).
// RESERVE t: n + (marks) * log n nodes; each is ~40
bytes.
// OTHER ACCUMULATORS: replace cnt/sum by whatever is
additive over the marks - count within a
// distance bound (keep a BIT per centroid instead,
which kills persistence), sum of weights,
// number of pairs. Anything that needs a
per-centroid ARRAY does not persist cheaply.
// DUMMY VERTICES from binarization are never marked
and never queried, but they DO appear as
// centroids - which is fine, since query() walks the
centroid tree by vertex id and only stops at
// the id it was asked for.
```

3 | Graph Theory

3.1 Graph Traversal

3.1.1 TopoSort

Topological Sort (Kahn's Algorithm) - O(V + E)

```
struct TopoSort {
    int n;
    vector<vector<int>> adj;
    vector<int> deg, order;

    TopoSort(int n = 0) : n(n), adj(n + 1), deg(n +
1, 0) {}

    void add_edge(int u, int v) {
        adj[u].pb(v);
        deg[v]++;
    }

    bool solve() {
        queue<int> q;
        for (int i = 1; i <= n; i++) {
            if (!deg[i]) q.push(i);
        }
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            order.pb(u);
            for (int v : adj[u]) {
                if (!--deg[v]) q.push(v);
            }
        }
        return (int)order.size() == n; // false if
graph has cycles
    }
};
```

3.1.2 BFS_DFS

BFS / DFS / 0-1 BFS / multi-source. 0-based. The bread and butter - keep them short.

```
// Unweighted shortest path + parent for reconstruction.
pair<vector<int>, vector<int>> bfs(int n, const vector<vector<int>>& adj, vector<int> src) {
    vector<int> d(n, -1), par(n, -1);
    queue<int> q;
    for (int s : src) d[s] = 0, q.push(s);        // MULTI-SOURCE: pass every source at once
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : adj[u]) if (d[v] < 0) d[v] = d[u] + 1, par[v] = u, q.push(v);
    }
    return {d, par};
}
vector<int> pathTo(int t, const vector<int>& par) {
    vector<int> p;
    for (int v = t; v >= 0; v = par[v])
        p.push_back(v);
    reverse(all(p));
    return p;
}

// 0-1 BFS: edge weights are only 0 or 1 -> deque instead of a heap, O(V + E).
// Use for "turning costs 1", "breaking one wall is free", "flip at most k edges".
vector<ll> bfs01(int n, const vector<vector<pair<int, int>>>& adj, int s) {
    vector<ll> d(n, llinf);
    deque<int> q;
    d[s] = 0, q.push_back(s);
    while (!q.empty()) {
        int u = q.front(); q.pop_front();
        for (auto [v, w] : adj[u]) if (d[u] + w < d[v]) {
            d[v] = d[u] + w;
            w ? q.push_back(v) : q.push_front(v);
        }
    }
    return d;
}

// Iterative DFS with entry/exit times (avoids stack overflow at n = 2e5).
void dfsIter(int n, const vector<vector<int>>& adj, int root, vector<int>& tin, vector<int>& tout) {
    tin.assign(n, -1), tout.assign(n, -1);
    vector<int> it(n, 0), st = {root};
    int timer = 0;
    tin[root] = timer++;
    while (!st.empty()) {
        int u = st.back();
        if (it[u] < (int)adj[u].size()) {
            int v = adj[u][it[u]++];
            if (tin[v] < 0) tin[v] = timer++, st.push_back(v);
        } else tout[u] = timer++, st.pop_back();
    }
}

// GRID: 4- and 8-neighbour offsets (see 03 - grid.cpp). Encode a cell as r*m+c and reuse the
// graph routines above; add extra state dimensions (mask, parity, keys) by widening the index.
// CYCLE DETECTION, directed: colour DFS (0 white, 1 grey, 2 black); a grey->grey edge closes a
// cycle - walk the recursion stack back to extract it. Undirected: any edge to an already-visited
// non-parent vertex closes one.
```

3.2 Shortest Paths

3.2.1 Dijkstra

Dijkstra's Shortest Path Algorithm - O((V + E) log V)

```
struct Dijkstra {
    int n;
    vector<vector<pair<int, ll>>> adj;
    vector<ll> dist;
    vector<int> par;

    Dijkstra(int n = 0) : n(n), adj(n + 1), dist(n + 1, llinf), par(n + 1, -1) {}

    void add_edge(int u, int v, ll w, bool directed = false) {
        adj[u].eb(v, w);
        if (!directed) adj[v].eb(u, w);
    }

    void solve(int src) {
        fill(all(dist), llinf);
        fill(all(par), -1);
        priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<>> pq;

        dist[src] = 0;
        pq.emplace(0, src);

        while (!pq.empty()) {
            auto [d, u] = pq.top();
            pq.pop();
            if (d > dist[u]) continue;

            for (auto& [v, w] : adj[u]) {
                if (dist[u] + w < dist[v]) {
                    dist[v] = dist[u] + w;
                    par[v] = u;
                    pq.emplace(dist[v], v);
                }
            }
        }
    }
};
```

3.2.2 FloydWarshall

Floyd-Warshall All-Pairs Shortest Path - O(V^3)

```
struct FloydWarshall {
    int n;
    vector<vector<ll>> dist;

    FloydWarshall(int n = 0) : n(n), dist(n + 1, vector<ll>(n + 1, llinf)) {
        for (int i = 0; i <= n; i++) dist[i][i] = 0;
    }

    void add_edge(int u, int v, ll w, bool directed = false) {
        dist[u][v] = min(dist[u][v], w);
        if (!directed) dist[v][u] = min(dist[v][u], w);
    }
```

```
    void solve() {
        for (int k = 1; k <= n; k++) {
            for (int i = 1; i <= n; i++) {
                for (int j = 1; j <= n; j++) {
                    if (dist[i][k] < llinf &&
    ↪ dist[k][j] < llinf) {
                        dist[i][j] = min(dist[i][j],
    ↪ dist[i][k] + dist[k][j]);
                    }
                }
            }
        }
    }
};
```

3.2.3 BellmanFord

If you paste both, keep only ONE copy of it.

```
// NOTE: 08 - Spanning Trees/01 - MST.cpp declares an
  ↪ identical 'struct Edge'.
//       If you paste both, keep only ONE copy of it.
// Bellman-Ford Shortest Path with Negative Cycle
  ↪ Detection - O(V * E)
struct Edge {
    int u, v;
    ll w;
};

struct BellmanFord {
    int n;
    vector<Edge> edges;
    vector<ll> dist;
    vector<int> par;

    BellmanFord(int n = 0) : n(n), dist(n + 1,
  ↪ llinf), par(n + 1, -1) {}

    void add_edge(int u, int v, ll w) {
        edges.pb({u, v, w});
    }

    // Returns false if a negative cycle is reachable
  ↪ from src
    bool solve(int src) {
        fill(all(dist), llinf);
        fill(all(par), -1);
        dist[src] = 0;

        for (int i = 0; i < n - 1; i++) {
            bool relaxed = false;
            for (const auto& e : edges) {
                if (dist[e.u] < llinf && dist[e.u] +
  ↪ e.w < dist[e.v]) {
                    dist[e.v] = dist[e.u] + e.w;
                    par[e.v] = e.u;
                    relaxed = true;
                }
            }
            if (!relaxed) break;
        }

        for (const auto& e : edges) {
            if (dist[e.u] < llinf && dist[e.u] + e.w
  ↪ < dist[e.v]) {
                return false; // Negative cycle
  ↪ detected
            }
        }
        return true;
    }
};
```

3.2.4 DominatorTree

```
/*
============================================================
   CATALOGUE: Lengauer-Tarjan Dominator Tree (O((N +
  ↪ M) log N))
============================================================
   DESCRIPTION:
   Finds the "bottlenecks" in any directed graph. A
  ↪ node 'u' dominates 'v' if
   EVERY path from the root to 'v' is forced to go
  ↪ through 'u'.

   USAGE:
   1. Build graph: Create a 2D vector 'adj' (1-based)
  ↪ containing the directed edges.
   2. Initialize:  DominatorTree dt(n, root, adj);
                   (Automatically builds the tree on
  ↪ instantiation)

   EXTRACTING ANSWERS:
   - dt.dom[u]:       The Immediate Dominator of 'u'
  ↪ (the closest bottleneck to 'u').
   - dt.id[u]:        If dt.id[u] == 0 after
  ↪ building, 'u' is UNREACHABLE from the root.
============================================================
*/
struct DominatorTree {
    int T = 0, n;
    vector<vector<int>> rg, bucket, adj;
    vector<int> dsu, par, sdom, idom, dom, label, id,
  ↪ rev;

    DominatorTree(int _n, int r, vector<vector<int>>&
  ↪ adj):
        n(_n + 1), rg(n), bucket(n), adj(adj), dsu(n),
        par(n), sdom(n), idom(n), dom(n), label(n),
  ↪ id(n), rev(n) { dfs(r); build(r); }

    int find(int u, int x = 0) {
        if (u == dsu[u]) return x ? -1 : u;
        int v = find(dsu[u], x + 1);
        if (v < 0) return u;
        if (sdom[label[dsu[u]]] < sdom[label[u]])
  ↪ label[u] = label[dsu[u]];
        dsu[u] = v;
        return x ? v : label[u];
    }

    void dfs(int u) {
        id[u] = ++T, rev[T] = u;
        label[T] = sdom[T] = dsu[T] = T;
        for (int& w : adj[u]) {
            if (!id[w]) {
                dfs(w);
                par[id[w]] = id[u];
            }
            rg[id[w]].push_back(id[u]);
        }
    }

    void build(int r) {
        // FIXED: Loop MUST go down to i >= 1 to
  ↪ process the root's bucket!
        for (int i = T; i >= 1; i--) {
```

```
            for (int& u : rg[i]) sdom[i] =
    ↪ min(sdom[i], sdom[find(u)]);
            if (i > 1) bucket[sdom[i]].push_back(i);

            for (int& w : bucket[i]) {
                int v = find(w);
                idom[w] = sdom[v] == sdom[w] ?
    ↪ sdom[w] : v;
            }

            if (i > 1) dsu[i] = par[i];
        }

        for (int i = 2; i <= T; i++) {
            if (idom[i] != sdom[i]) idom[i] =
    ↪ idom[idom[i]];
        }

        for (int u = 1; u <= T; ++u) {
            dom[rev[u]] = rev[idom[u]];
        }
    }
};

void testCase() {
    int n, m, s;
    cin >> n >> m >> s;
    s++; // Convert to 1-based indexing

    vector<vector<int>> adj(n + 1);
    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        adj[u + 1].push_back(v + 1);
    }

    // Automatically builds upon instantiation
    DominatorTree dt(n, s, adj);

    for (int i = 1; i <= n; i++) {
        if (i == s) {
            // pS is S
            cout << s - 1 << (i == n ? "" : " ");
        } else if (!dt.id[i]) {
            // If we can not reach i from S, print -1
            cout << -1 << (i == n ? "" : " ");
        } else {
            // Print the 0-based parent (which is now
    ↪ guaranteed to be calculated)
            cout << dt.dom[i] - 1 << (i == n ? "" : "
    ↪ ");
        }
    }
    cout << "\n";
}
```

3.2.5 ShortestPathEdgeModify

SHORTEST PATH WITH ONE EDGE CHANGED, answered OFFLINE in O(1) per query after O((n+m) log n).

```
// Query: "if edge id had weight x instead, what is
    ↪ dist(s,t)?" - each query is independent.
// IDEA. Let ds/dt be the distance arrays from s and
    ↪ to t, P the shortest path, base = ds[t].
//   * Edge NOT on P: the new path either ignores it
    ↪ (base) or crosses it once:
//         min(base, ds[u] + x + dt[v], ds[v] + x +
    ↪ dt[u]).
//   * Edge ON P at position i: either still use it
    ↪ (base - w + x), or avoid it entirely, and
//     "the best s->t path avoiding path-edge i" is
    ↪ precomputed for every i at once:
//     for a non-path edge (u,v,w), the path ds[u] ->
    ↪ u -> v -> dt[v] skips exactly the path edges
//     in the window [L[u], R[v]), where L[x] = last
    ↪ path vertex on some shortest s->x path and
//     R[x] = first path vertex on some shortest x->t
    ↪ path. Sweep i from 0 to k-1 with a heap.
struct SPEdgeModify {
    struct Edge { int u, v; ll w; };
    int n, k;
    ↪          // k = #edges on the shortest path
    ll base;
    vector<Edge> E;
    vector<ll> ds, dt, best;
    ↪          // best[i] = s->t avoiding path edge i
    vector<int> pos;
    ↪          // pos[e] = index on P, or -1
    SPEdgeModify(int n, vector<Edge> E, int s, int t)
    ↪ : n(n), E(E) {
        int m = E.size();
        vector<vector<pair<int, int>>> adj(n);
        for (int i = 0; i < m; i++)
            adj[E[i].u].push_back({E[i].v, i}),
    ↪ adj[E[i].v].push_back({E[i].u, i});
        vector<int> par(n, -1), pare(n, -1), ordS,
    ↪ ordT;
        auto dij = [&](int src, vector<ll>& d,
    ↪ vector<int>& ord, bool tree) {
            d.assign(n, llinf);
            priority_queue<pair<ll, int>,
    ↪ vector<pair<ll, int>>, greater<>> pq;
            d[src] = 0, pq.push({0, src});
            while (!pq.empty()) {
                auto [dd, u] = pq.top(); pq.pop();
                if (dd > d[u]) continue;
                ord.push_back(u);
                for (auto [v, id] : adj[u]) if (d[u]
    ↪ + E[id].w < d[v]) {
                    d[v] = d[u] + E[id].w;
                    if (tree) par[v] = u, pare[v] =
    ↪ id;
                    pq.push({d[v], v});
                }
            }
        };
        dij(s, ds, ordS, true), dij(t, dt, ordT,
    ↪ false);
        base = ds[t];
        vector<int> pv, pe;
    ↪          // the path, s ... t
        for (int c = t; c != s; c = par[c])
    ↪ pv.push_back(c), pe.push_back(pare[c]);
        pv.push_back(s), reverse(all(pv)),
    ↪ reverse(all(pe));
        k = pe.size();
        pos.assign(m, -1);
        vector<char> onP(n, 0);
        for (int x : pv) onP[x] = 1;
        for (int i = 0; i < k; i++) pos[pe[i]] = i;
        vector<int> L(n, 0), R(n, k);
        for (int i = 0; i <= k; i++) L[pv[i]] =
    ↪ R[pv[i]] = i;
        for (int u : ordS) for (auto [v, id] :
    ↪ adj[u])          // relaxed in shortest-path
    ↪ order
            if (!onP[v] && ds[v] == ds[u] + E[id].w)
```

```
    L[v] = max(L[v], L[u]);
        for (int u : ordT) for (auto [v, id] : adj[u])
            if (!onP[v] && dt[v] == dt[u] + E[id].w)
    R[v] = min(R[v], R[u]);
        vector<vector<pair<ll, int>>> ev(max(k, 1));
              // ev[start] = {cost, end}
        for (int i = 0; i < m; i++) {
            if (pos[i] >= 0) continue;
            auto [u, v, w] = E[i];
            if (L[u] < R[v])
    ev[L[u]].push_back({ds[u] + w + dt[v], R[v]});
            if (L[v] < R[u])
    ev[L[v]].push_back({ds[v] + w + dt[u], R[u]});
        }
        best.assign(max(k, 1), llinf);
        priority_queue<pair<ll, int>, vector<pair<ll,
    int>>, greater<>> sw;
        for (int i = 0; i < k; i++) {
            for (auto& e : ev[i]) sw.push(e);
            while (!sw.empty() && sw.top().second <=
    i) sw.pop();
            if (!sw.empty()) best[i] = sw.top().first;
        }
    }
    ll query(int id, ll x) const {
              // edge id gets weight x
        auto [u, v, w] = E[id];
        if (pos[id] < 0) return min({base, ds[u] + x
    + dt[v], ds[v] + x + dt[u]});
        return min(base - w + x, best[pos[id]]);
    }
};
// RELATED, same precomputation:
//   * "Does edge e lie on SOME shortest path?"  <=>
    ds[u] + w + dt[v] == base (either direction).
//   * "Does e lie on EVERY shortest path?"      <=>
    removing it increases the distance; the sweep
//     above answers this for path edges (best[i] >
    base means path edge i is essential).
//   * REPLACEMENT PATHS (delete one edge, ask the
    new distance) = query(id, +infinity).
//   * SECOND SHORTEST WALK (may reuse edges): min
    over edges of ds[u] + w + dt[v] strictly > base.
//   * If instead the whole graph changes per query,
    you need a fresh Dijkstra - this trick only
//     works because s, t and every other weight stay
    fixed.
```

3.2.6 CyclesAndConstraints

MINIMUM MEAN CYCLE, and SYSTEMS OF DIFFERENCE CONSTRAINTS - the two shortest-path tools that

```
// are not about shortest paths.

// --- KARP'S MINIMUM MEAN CYCLE, O(V*E). Returns the
    smallest possible (cycle weight)/(length).
// dp[k][v] = min weight of a walk of EXACTLY k edges
    ending at v (from a virtual source that
// reaches everything). Karp: the answer is  min over
    v of  max over k<n of (dp[n][v]-dp[k][v])/(n-k).
// Reach for it whenever the objective is an AVERAGE
    around a cycle: "minimum cost per unit time",
// "is there a cycle with negative average",
    profit-per-day loops.
ld minMeanCycle(int n, const vector<array<ll, 3>>& e)
    {        // e = {u, v, w}, directed
    vector<vector<ll>> dp(n + 1, vector<ll>(n,
    llinf));
    fill(all(dp[0]), 0);
              // virtual source reaches every v
    for (int k = 1; k <= n; k++)
        for (auto [u, v, w] : e)
            if (dp[k - 1][u] < llinf) dp[k][v] =
    min(dp[k][v], dp[k - 1][u] + w);
    ld best = 1e18;
    for (int v = 0; v < n; v++) {
        if (dp[n][v] == llinf) continue;
        ld cur = -1e18;
        for (int k = 0; k < n; k++)
            if (dp[k][v] < llinf) cur = max(cur,
    (ld)(dp[n][v] - dp[k][v]) / (n - k));
        best = min(best, cur);
    }
    return best;
              // 1e18 if the graph is acyclic
}
// MINIMUM COST-TO-TIME RATIO CYCLE (minimise
    sum(cost)/sum(time), time > 0): binary search x
    and
// test for a negative cycle with weights cost -
    x*time. Same skeleton, one Bellman-Ford per step.
// MAXIMUM mean cycle: negate the weights.

// --- SYSTEM OF DIFFERENCE CONSTRAINTS. Every
    constraint x_j - x_i <= w becomes an edge i -> j
    of
// weight w; add a super-source with a 0-edge to
    every vertex and run Bellman-Ford.
// FEASIBLE iff there is no negative cycle. The
    distances are then a solution, and they are the
// POINTWISE MAXIMUM one (every other solution is <=
    it componentwise, after fixing x_source = 0).
// TRANSLATIONS you will need: x_j - x_i >= w  is
    x_i - x_j <= -w.  x_j - x_i = w is both.
// x_i <= c is x_i - x_0 <= c against the source.
    Strict "<" on integers is "<= w-1".
// USES: scheduling with precedence and deadlines,
    "arrange these values so all the gaps hold",
// consistency of a set of relative measurements, and
    the dual of a min-cost-flow.
bool differenceConstraints(int n, const
    vector<array<ll, 3>>& c, vector<ll>& x) {
    vector<array<ll, 3>> e = c;
              // c = {i, j, w} meaning x_j-x_i<=w
    for (int v = 0; v < n; v++) e.push_back({(ll)n,
    (ll)v, 0});    // super-source n
    x.assign(n + 1, 0);
    for (int it = 0; it <= n; it++) {
        bool any = false;
        for (auto [u, v, w] : e)
            if (x[u] + w < x[v]) x[v] = x[u] + w, any
    = true;
        if (!any) { x.resize(n); return true; }
    }
    return false;
              // negative cycle: infeasible
}
// --- BELLMAN-FORD EXTRAS worth remembering
// NEGATIVE CYCLE RECOVERY: after n rounds, take a
    vertex that still relaxed, walk n parent
// pointers back (this lands you ON the cycle - the
    relaxed vertex may only be reachable from it),
// then follow parents until a vertex repeats.
// SPFA is Bellman-Ford with a queue; it is O(VE)
    worst case and killable, but the SLF heuristic
// (push to the front if the new distance beats the
    front) makes it fast in practice.
```

```
// ONLY SHORTEST PATHS WITH AT MOST K EDGES: run exactly k rounds, updating from a SNAPSHOT of the
// previous round (otherwise one round can propagate along many edges).
// JOHNSON'S APSP for sparse graphs with negative edges: Bellman-Ford from a super-source gives
// potentials h[], reweight w'(u,v) = w + h[u] - h[v] >= 0, then run n Dijkstras. O(VE + V E log V).
```

3.2.7 SPFA

SPFA (Bellman-Ford with a queue) - shortest paths with NEGATIVE edges, plus negative-cycle

```
// detection. O(V*E) worst case and constructible-against, but usually far faster than plain
// Bellman-Ford because only vertices whose distance actually improved get re-examined.
// Returns false iff a negative cycle is reachable from s: a vertex relaxed n times must be on
// (or reachable from) one, since a shortest walk uses at most n-1 edges.
// WHEN: negative weights and you also want the cycle check. With NON-negative weights always use
// Dijkstra - SPFA is asymptotically worse and hackable. For a guaranteed bound use Bellman-Ford.
bool spfa(int n, int s, const vector<vector<pair<int, ll>>>& g, vector<ll>& d) {
    d.assign(n, llinf);
    vector<int> cnt(n, 0);
    vector<char> inq(n, 0);
    deque<int> q;
    d[s] = 0, q.push_back(s), inq[s] = 1;
    while (!q.empty()) {
        int v = q.front(); q.pop_front();
        inq[v] = 0;
        for (auto [to, w] : g[v]) if (d[v] + w < d[to]) {
            d[to] = d[v] + w;
            if (!inq[to]) {
                if (++cnt[to] >= n) return false;
                // negative cycle
                inq[to] = 1;
                // SLF: a smaller tentative distance goes to the FRONT. Cheap, and it is what
                // makes SPFA fast in practice;
                // without it the queue degenerates.
                if (!q.empty() && d[to] < d[q.front()]) q.push_front(to);
                else q.push_back(to);
            }
        }
    }
    return true;
}
// RECOVERING THE CYCLE: keep par[] and, on detection, walk n parent pointers back (that lands you
// ON the cycle - the detected vertex may only be reachable from it), then follow parents until a
// vertex repeats. See 06 - CyclesAndConstraints.cpp, which uses exactly this for difference
// constraints and for the minimum mean cycle.
// KILLER TESTS EXIST: grid-like graphs make SPFA quadratic. If the problem looks adversarial,
// use Bellman-Ford (predictable O(VE)) or reweight with Johnson potentials and run Dijkstra.
```

3.2.8 SegmentTreeGraph

SEGMENT-TREE GRAPH - add "u -> every vertex in [l, r]" (or the reverse) with O(log n) edges

```
// instead of O(n). Then run Dijkstra/BFS as usual. Total O((n + q log n) log n).
// Two trees: OUT (parent -> children, cost 0) to broadcast INTO a range,
//            IN  (children -> parent, cost 0) to collect FROM a range.
// Leaves of both trees are the original vertices.
struct SegGraph {
    int n, cnt;
    vector<int> outId, inId;
    // node ids of the two trees
    vector<vector<pair<int, ll>>> g;

    SegGraph(int n) : n(n) {
        cnt = n;
        outId.assign(4 * n, -1), inId.assign(4 * n, -1);
        g.assign(n + 8 * n, {});
        buildOut(1, 0, n - 1), buildIn(1, 0, n - 1);
    }
    int node() { return cnt++; }
    void buildOut(int v, int l, int r) {
        if (l == r) { outId[v] = l; return; }
        outId[v] = node();
        int m = (l + r) / 2;
        buildOut(2 * v, l, m), buildOut(2 * v + 1, m + 1, r);
        g[outId[v]].push_back({outId[2 * v], 0});
        g[outId[v]].push_back({outId[2 * v + 1], 0});
    }
    void buildIn(int v, int l, int r) {
        if (l == r) { inId[v] = l; return; }
        inId[v] = node();
        int m = (l + r) / 2;
        buildIn(2 * v, l, m), buildIn(2 * v + 1, m + 1, r);
        g[inId[2 * v]].push_back({inId[v], 0});
        g[inId[2 * v + 1]].push_back({inId[v], 0});
    }
    void addRange(int v, int l, int r, int ql, int qr, int u, ll w, bool toRange) {
        if (qr < l || r < ql) return;
        if (ql <= l && r <= qr) {
            if (toRange) g[u].push_back({outId[v], w});          // u -> everything in [ql,qr]
            else g[inId[v]].push_back({u, w});                  // everything in [ql,qr] -> u
            return;
        }
        int m = (l + r) / 2;
        addRange(2 * v, l, m, ql, qr, u, w, toRange);
        addRange(2 * v + 1, m + 1, r, ql, qr, u, w, toRange);
    }
    void edgeToRange(int u, int l, int r, ll w) { addRange(1, 0, n - 1, l, r, u, w, true); }
    void edgeFromRange(int l, int r, int u, ll w) { addRange(1, 0, n - 1, l, r, u, w, false); }
    void edge(int u, int v, ll w) { g[u].push_back({v, w}); }

    vector<ll> dijkstra(int s) {
        vector<ll> d(cnt, llinf);
        priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<>> pq;
```

```
        d[s] = 0, pq.push({0, s});
        while (!pq.empty()) {
            auto [dd, u] = pq.top(); pq.pop();
            if (dd > d[u]) continue;
            for (auto [v, w] : g[u]) if (dd + w <
  ↪ d[v]) d[v] = dd + w, pq.push({d[v], v});
        }
        d.resize(n);
  ↪ // only the original vertices matter
        return d;
    }
};
// The same construction gives 2-SAT range clauses
  ↪ and "range -> range" edges (route both
// through one fresh auxiliary node).
```

3.2.9 KShortestWalks

EPPSTEIN'S ALGORITHM - the k SHORTEST s-t WALKS (repeated vertices and edges allowed), in

```
// O(m log m + k log k). Directed, 0-indexed,
  ↪ non-negative weights. Returns the k lengths in
// non-decreasing order, padding with -1 when fewer
  ↪ than k walks exist.
// Reach for it for "the k cheapest routes", "the
  ↪ k-th smallest path length", "is there a path of
// length exactly L" (enumerate until you pass L),
  ↪ and for problems that sum over the k best plans.
// THE IDEA. Fix the shortest path TREE towards t,
  ↪ with d[v] = dist(v, t). Any walk is described by
// the SIDETRACK edges it takes - the edges it uses
  ↪ that are not tree edges - and its length is
// d[s] + (sum of the sidetracks' reduced costs w +
  ↪ d[to] - d[from]), each of which is >= 0. So the
// k shortest walks = the k cheapest ANTICHAINS in a
  ↪ heap-ordered structure, and a best-first search
// over that structure emits them one at a time.
// H[v] is a PERSISTENT leftist heap holding the
  ↪ sidetracks available at v and at all its
  ↪ ancestors
// in the tree; persistence is what makes "inherit
  ↪ the parent's heap" O(log n) instead of O(n).
struct KShortestWalks {
    struct Node { Node *l = 0, *r = 0; int sz = 1,
  ↪ to; ll val; };
    Node* meld(Node* a, Node* b) {
        if (!a) return b;
        if (!b) return a;
        if (a->val > b->val) swap(a, b);
        Node* c = new Node(*a);
  ↪           // copy-on-write: keeps old versions
        c->r = meld(c->r, b);
        if (!c->l || c->l->sz < c->r->sz) swap(c->l,
  ↪ c->r);
        c->sz = (c->r ? c->r->sz : 0) + 1;
        return c;
    }
    Node* push(Node* h, ll val, int to) {
        Node* c = new Node();
        c->val = val, c->to = to;
        return meld(h, c);
    }
    // e = {u, v, w} directed arcs. Returns the k
  ↪ shortest s -> t walk lengths.
    vector<ll> solve(int n, const vector<array<ll,
  ↪ 3>>& e, int s, int t, int k) {
        vector<vector<array<ll, 3>>> fwd(n), rev(n);
  ↪           // {other endpoint, weight, edge id}
        for (int i = 0; i < (int)e.size(); i++) {
            fwd[e[i][0]].push_back({e[i][1], e[i][2],
  ↪ (ll)i});
            rev[e[i][1]].push_back({e[i][0], e[i][2],
  ↪ (ll)i});
        }
        vector<ll> d(n, llinf);
        vector<int> par(n, -1), tid(n, -1);
  ↪           // tid[v] = v's tree edge id
        priority_queue<pair<ll, int>, vector<pair<ll,
  ↪ int>>, greater<>> pq;
        d[t] = 0, pq.push({0, t});
        while (!pq.empty()) {
  ↪           // Dijkstra on the REVERSED graph
            auto [dd, u] = pq.top(); pq.pop();
            if (dd > d[u]) continue;
            for (auto [v, w, id] : rev[u]) if (dd + w
  ↪ < d[v])
                d[v] = dd + w, par[v] = u, tid[v] =
  ↪ id, pq.push({d[v], (int)v});
        }
        vector<ll> ans;
        if (d[s] == llinf) return vector<ll>(k, -1);
        vector<vector<int>> kids(n);
        for (int v = 0; v < n; v++) if (par[v] >= 0)
  ↪ kids[par[v]].push_back(v);
        vector<Node*> H(n, nullptr);
        vector<int> bfs{t};
  ↪           // build H down the tree from t
        for (int qi = 0; qi < (int)bfs.size(); qi++) {
            int u = bfs[qi];
            if (par[u] >= 0) H[u] = meld(H[u],
  ↪ H[par[u]]);
            for (auto [v, w, id] : fwd[u])
                if ((int)id != tid[u] && d[v] <
  ↪ llinf) H[u] = push(H[u], w + d[v] - d[u],
  ↪ (int)v);
            for (int v : kids[u]) bfs.push_back(v);
        }
        auto cmp = [](const pair<ll, Node*>& a, const
  ↪ pair<ll, Node*>& b) {
            return a.first > b.first;
        };
        priority_queue<pair<ll, Node*>,
  ↪ vector<pair<ll, Node*>>, decltype(cmp)> Q(cmp);
        Node* root = new Node();
        root->val = d[s], root->to = s;
        Q.push({d[s], root});
        while (!Q.empty() && (int)ans.size() < k) {
            auto [w, cur] = Q.top(); Q.pop();
            ans.push_back(w);
            if (cur->l) Q.push({w + cur->l->val -
  ↪ cur->val, cur->l});   // swap the last sidetrack
            if (cur->r) Q.push({w + cur->r->val -
  ↪ cur->val, cur->r});
            if (H[cur->to]) Q.push({w +
  ↪ H[cur->to]->val, H[cur->to]});     // append a
  ↪ new sidetrack
        }
        while ((int)ans.size() < k) ans.push_back(-1);
        return ans;
    }
};
// WALKS, NOT PATHS. The second-shortest walk may
  ↪ repeat a vertex. If the problem insists on SIMPLE
// paths you need YEN'S ALGORITHM: keep the best
  ↪ path, then for each prefix of it (the "spur"),
  ↪ ban
// the edges that would recreate an already-found
```

```
  ↪ path, run one Dijkstra from the spur vertex, and
// keep the best candidate. O(k * n * (m + n log n))
  ↪ - much slower, but it is the only correct
// answer for "k shortest SIMPLE paths". On a DAG
  ↪ walks and paths coincide, so use this file.
// K-TH SHORTEST PATH WITH A COUNT LIMIT PER VERTEX:
  ↪ a plain Dijkstra that pops each vertex up to k
// times is O(k m log) and far simpler - reach for
  ↪ that first if k is small (<= a few hundred).
// SECOND SHORTEST PATH ONLY: just take ans[1] here,
  ↪ or do the two-array trick (best and
// second-best distance per vertex) in one Dijkstra.
```

3.3 Graph Connectivity

3.3.1 LowLink

LowLink: BRIDGES + ARTICULATION POINTS + 2-EDGE-CONNECTED COMPONENTS - O(V + E)

```
// Skips the parent by EDGE ID, so MULTI-EDGES are
  ↪ handled correctly
// (a doubled edge is never a bridge). 0-based
  ↪ vertices.
// comp[v] = id of v's 2-edge-connected component;
  ↪ contracting those gives the BRIDGE TREE,
// on which "number of bridges between u and v" is
  ↪ just a tree path length.
struct LowLink {
    int n, timer = 0, nc = 0;
    vector<vector<pair<int, int>>> adj;
  ↪   // {to, edge id}
    vector<int> tin, low, comp;
    vector<bool> isCut;
    vector<pair<int, int>> bridges;
    vector<int> st;

    LowLink(int n = 0) : n(n), adj(n), tin(n, -1),
  ↪ low(n), comp(n, -1), isCut(n, false) {}
    void add_edge(int u, int v, int id) {
  ↪ adj[u].push_back({v, id}), adj[v].push_back({u,
  ↪ id}); }

    void dfs(int u, int pe) {
        tin[u] = low[u] = timer++;
        st.push_back(u);
        int kids = 0;
        for (auto [v, id] : adj[u]) {
            if (id == pe) continue;
  ↪     // skip the edge we came in on, not the vertex
            if (tin[v] != -1) low[u] = min(low[u],
  ↪ tin[v]);
            else {
                dfs(v, id), kids++;
                low[u] = min(low[u], low[v]);
                if (low[v] > tin[u])
  ↪ bridges.push_back({u, v});
                if (low[v] >= tin[u] && pe != -1)
  ↪ isCut[u] = true;
            }
        }
        if (pe == -1 && kids > 1) isCut[u] = true;
        if (low[u] == tin[u]) {
  ↪     // u is the top of a 2-edge-connected comp
            while (true) {
                int v = st.back(); st.pop_back();
                comp[v] = nc;
                if (v == u) break;
            }
            nc++;
        }
    }
    void solve() { for (int i = 0; i < n; i++) if
  ↪ (tin[i] == -1) dfs(i, -1); }

    // Bridge tree: nc nodes, one edge per bridge.
    vector<vector<int>> bridgeTree() {
        vector<vector<int>> t(nc);
        for (auto [u, v] : bridges)
  ↪ t[comp[u]].push_back(comp[v]),
  ↪ t[comp[v]].push_back(comp[u]);
        return t;
    }
};
```

3.3.2 SCC

Strongly Connected Components (Tarjan's Algorithm) - O(V + E)

```
// IMPORTANT: component ids come out in REVERSE
  ↪ topological order - for every edge u->v with
// comp[u] != comp[v] we have comp[u] > comp[v].
  ↪ Iterate ids DOWNWARD to walk the condensation
// forwards. build_dag() may emit duplicate arcs
  ↪ (fine for reachability, wrong for counting).
struct SCC {
    int n, timer = 0, num_comps = 0;
    vector<vector<int>> adj;
    vector<int> tin, low, comp;
    vector<bool> in_st;
    stack<int> st;
    vector<vector<int>> comps;

    SCC(int n = 0) : n(n), adj(n + 1), tin(n + 1, 0),
  ↪ low(n + 1, 0), comp(n + 1, -1), in_st(n + 1,
  ↪ false) {}

    void add_edge(int u, int v) {
        adj[u].pb(v);
    }

    void dfs(int u) {
        tin[u] = low[u] = ++timer;
        st.push(u);
        in_st[u] = true;

        for (int v : adj[u]) {
            if (!tin[v]) {
                dfs(v);
                low[u] = min(low[u], low[v]);
            } else if (in_st[v]) {
                low[u] = min(low[u], tin[v]);
            }
        }

        if (low[u] == tin[u]) {
            comps.pb({});
            while (true) {
                int v = st.top();
                st.pop();
                in_st[v] = false;
                comp[v] = num_comps;
                comps.back().pb(v);
                if (u == v) break;
            }
            num_comps++;
        }
    }

    void solve() {
```

```
        for (int i = 1; i <= n; i++) {
            if (!tin[i]) dfs(i);
        }
    }

    vector<vector<int>> build_dag() {
        vector<vector<int>> dag(num_comps);
        for (int u = 1; u <= n; u++) {
            for (int v : adj[u]) {
                if (comp[u] != comp[v])
  ↪ dag[comp[u]].pb(comp[v]);
            }
        }
        return dag;
    }
};
```

3.3.3 BlockCutTree

BLOCK-CUT TREE (vertex biconnected components). O(V + E).

```
// Builds a forest whose nodes are: the original CUT
  ↪ VERTICES, plus one node per BICONNECTED
// COMPONENT ("block"). Every cut vertex is joined to
  ↪ each block containing it.
// Answers "is there a path u -> v avoiding vertex
  ↪ w", "which blocks does a path cross",
// "articulation structure of the graph" - all become
  ↪ tree queries.
struct BlockCut {
    int n, timer = 0;
    vector<vector<int>> adj, comps, tree;
  ↪ // comps[i] = vertices of block i
    vector<int> tin, low, stk, id;
  ↪ // id[v] = tree node of original vertex v
    vector<bool> isCut;

    BlockCut(int n = 0) : n(n), adj(n), tin(n, -1),
  ↪ low(n), id(n, -1), isCut(n, false) {}
    void add_edge(int u, int v) {
  ↪ adj[u].push_back(v), adj[v].push_back(u); }

    void dfs(int u, int p) {
        tin[u] = low[u] = timer++;
        stk.push_back(u);
        int kids = 0;
        for (int v : adj[u]) {
            if (v == p) { p = -1; continue; }
  ↪ // skip ONE parent edge (multi-edge safe)
            if (tin[v] != -1) low[u] = min(low[u],
  ↪ tin[v]);
            else {
                dfs(v, u), kids++;
                low[u] = min(low[u], low[v]);
                if (low[v] >= tin[u]) {
  ↪ // u closes a block
                    isCut[u] = (tin[u] > 0 || kids >
  ↪ 1);
                    comps.push_back({u});
                    while (true) {
                        int w = stk.back();
  ↪ stk.pop_back();
                        comps.back().push_back(w);
                        if (w == v) break;
                    }
                }
            }
        }
    }
    void build() {
        for (int v = 0; v < n; v++) if
  ↪ (adj[v].empty()) comps.push_back({v});   //
  ↪ isolated vertex
        for (int i = 0; i < n; i++) if (tin[i] == -1)
  ↪ { timer = 0; dfs(i, -1); stk.clear(); }
        tree.assign(comps.size(), {});
        for (int v = 0; v < n; v++) if (isCut[v])
  ↪ id[v] = tree.size(), tree.push_back({});
        for (int i = 0; i < (int)comps.size(); i++)
            for (int v : comps[i]) {
                if (!isCut[v]) id[v] = i;
                else tree[i].push_back(id[v]),
  ↪ tree[id[v]].push_back(i);
            }
    }
};
// A vertex that is NOT a cut vertex lies in exactly
  ↪ one block -> id[v] is that block.
// An edge (u,v) always lies in exactly one block.
  ↪ Leaves of the tree are blocks; a graph is
// biconnected iff the tree has a single node.
```

3.3.4 TriangleAndCycleCounting

COUNTING SMALL SUBGRAPHS in O(m sqrt m) - the "degree ordering" trick. Orient every edge from

```
// the lower (degree, index) endpoint to the higher
  ↪ one. In that DAG every out-degree is O(sqrt m),
// because a vertex with out-degree d has d
  ↪ neighbours of degree >= d, so d^2 <= 2m.
// Undirected simple graph, 0-indexed, edges given as
  ↪ pairs.

// --- TRIANGLES: enumerate each once. O(m sqrt m).
  ↪ ---
ll countTriangles(int n, const vector<pair<int,
  ↪ int>>& es) {
    vector<int> deg(n, 0);
    for (auto [u, v] : es) deg[u]++, deg[v]++;
    auto better = [&](int a, int b) { return deg[a]
  ↪ != deg[b] ? deg[a] < deg[b] : a < b; };
    vector<vector<int>> out(n);
    for (auto [u, v] : es) better(u, v) ?
  ↪ out[u].push_back(v) : out[v].push_back(u);
    vector<char> mark(n, 0);
    ll cnt = 0;
    for (int u = 0; u < n; u++) {
        for (int v : out[u]) mark[v] = 1;
        for (int v : out[u]) for (int w : out[v]) cnt
  ↪ += mark[w];
        for (int v : out[u]) mark[v] = 0;
    }
    return cnt;
  ↪          // each triangle counted exactly once
}
// LISTING them: inside the inner loop, (u,v,w) with
  ↪ mark[w] IS a triangle.
// PER-VERTEX / PER-EDGE triangle counts: add 1 to
  ↪ each of u,v,w (or to each of the three edges)
// at that moment - this is what "local clustering
  ↪ coefficient" and many CF problems want.

// --- 4-CYCLES: O(m sqrt m) with the same
  ↪ orientation. ---
// For each u, walk two steps (u -> v -> w) in the
  ↪ oriented graph and count how many times each w
// is reached; a pair of distinct paths u->w gives a
  ↪ 4-cycle. Sum C(cnt[w], 2).
ll countFourCycles(int n, const vector<pair<int,
```

```
    ↪ int>>& es) {
    vector<int> deg(n, 0);
    for (auto [u, v] : es) deg[u]++, deg[v]++;
    auto better = [&](int a, int b) { return deg[a]
  ↪ != deg[b] ? deg[a] < deg[b] : a < b; };
    vector<vector<int>> adj(n), out(n);
    for (auto [u, v] : es) {
        adj[u].push_back(v), adj[v].push_back(u);
        better(u, v) ? out[u].push_back(v) :
  ↪ out[v].push_back(u);
    }
    vector<ll> cnt(n, 0);
    ll r = 0;
    for (int u = 0; u < n; u++) {
        for (int v : adj[u]) for (int w : out[v]) if
  ↪ (better(u, w)) r += cnt[w]++;
        for (int v : adj[u]) for (int w : out[v]) if
  ↪ (better(u, w)) cnt[w] = 0;
    }
    return r;
}
// WHY IT IS O(m sqrt m): the inner loop runs sum
  ↪ over v of deg(v) * outdeg(v) <= O(m sqrt m).
//
// OTHER COUNTING FACTS WORTH HAVING:
//   * #triangles = trace(A^3) / 6 for the adjacency
  ↪ matrix A; with bitsets that is O(n^3 / 64) and
//     is often the shorter answer when n <= 2000
  ↪ (also gives per-pair common-neighbour counts).
//   * #paths of length 2 = sum over v of C(deg v,
  ↪ 2). Subtract 3 * #triangles for "paths that are
//     not part of a triangle".
//   * A graph is TRIANGLE-FREE  =>  m <= n^2/4
  ↪ (Mantel / Turan for K3). More generally Turan:
//     a K_{r+1}-free graph has at most (1 - 1/r) n^2
  ↪ / 2 edges.
//   * #closed walks of length k = trace(A^k) - use
  ↪ matrix power for k up to 1e18 on small n.
//   * COUNTING CYCLES OF LENGTH <= 5 is polynomial
  ↪ by similar tricks; length >= 6 is as hard as
//     matrix multiplication in general. For counting
  ↪ ALL simple cycles use inclusion-exclusion
//     over subsets (2^n) or DP over bitmask
  ↪ start-vertex.
//   * GIRTH (shortest cycle) of an unweighted graph:
  ↪ BFS from every vertex, O(nm). For the
//     shortest cycle THROUGH a given edge, delete it
  ↪ and run BFS.
```

3.3.5 ThreeEdgeConnectivity

THREE-EDGE-CONNECTED COMPONENTS in ONE DFS, O(n + m). Undirected, 0-indexed, may be

```
// disconnected; self loops and PARALLEL EDGES are
  ↪ handled (parallel edges matter here - two
// vertices joined by three parallel edges are
  ↪ 3-edge-connected).
// u and v land in the same component iff no set of
  ↪ at most TWO edges separates them, i.e. iff
// there are 3 edge-disjoint u-v paths (Menger). This
  ↪ is the next step after LowLink (01), which
// gives the 2-edge-connected components (= no single
  ↪ edge separates them).
// Reach for it on "the network survives any two
  ↪ cable failures", "contract every pair of parallel
// bridges", and as the preprocessing for
  ↪ 3-edge-connectivity augmentation problems.
// Tsin's algorithm: a DFS that maintains, for each
  ↪ vertex, a PATH of still-undecided descendants;
// whenever a back edge proves two of them
  ↪ 3-edge-connected the path is absorbed into one
  ↪ class.
// The classes live in a circular linked list, so
  ↪ "merge" is a single swap.
vector<vector<int>> threeEdgeCC(const
  ↪ vector<vector<int>>& g) {
    int n = g.size(), cnt = 0;
    vector<int> in(n), out(n), low(n, n), deg(n, 0),
  ↪ path(n, n), nxt(n), it(n, 0), stk;
    vector<char> vis(n, 0);
    iota(all(nxt), 0);
  ↪          // circular list = the components
    auto absorb = [&](int v, int w) { swap(nxt[v],
  ↪ nxt[w]), deg[v] += deg[w]; };
    for (int s = 0; s < n; s++) {
  ↪          // iterative DFS over the forest
        if (vis[s]) continue;
        stk = {s}, vis[s] = 1, in[s] = cnt++;
        vector<int> par(1, n), pe(1, -1);
  ↪          // pe = "parent edge already used"
        while (!stk.empty()) {
            int v = stk.back(), p = par.back();
            if (it[v] < (int)g[v].size()) {
                int w = g[v][it[v]++];
                if (w == v) continue;
                if (w == p && !pe.back()) { pe.back()
  ↪ = 1; continue; }   // skip ONE parent edge
                if (vis[w]) {
                    if (in[w] < in[v]) deg[v]++,
  ↪ low[v] = min(low[v], in[w]);
                    else {
  ↪          // back edge to a descendant
                        deg[v]--;
                        int u = path[v];
                        while (u != n && in[u] <=
  ↪ in[w] && in[w] < out[u])
                            absorb(v, u), u = path[u];
                        path[v] = u;
                    }
                    continue;
                }
                vis[w] = 1, in[w] = cnt++,
  ↪ stk.push_back(w), par.push_back(v),
  ↪ pe.push_back(0);
            } else {
  ↪          // leaving v: fold it into its parent
                out[v] = cnt, stk.pop_back(),
  ↪ par.pop_back(), pe.pop_back();
                if (p == n) continue;
                int w = v;
                if (path[w] == n && deg[w] <= 1) {
  ↪ deg[p] += deg[w], low[p] = min(low[p], low[w]); }
                else {
                    if (deg[w] == 0) w = path[w];
                    if (low[w] < low[p]) low[p] =
  ↪ low[w], swap(w, path[p]);
                    while (w != n) absorb(p, w), w =
  ↪ path[w];
                }
            }
        }
    }
    vector<vector<int>> comp;
  ↪          // walk each circular list once
    vector<char> seen(n, 0);
    for (int i = 0; i < n; i++) if (!seen[i]) {
        comp.push_back({});
```

```
        for (int j = i; !seen[j]; j = nxt[j]) seen[j]
  ↪ = 1, comp.back().push_back(j);
    }
    return comp;
}
// CONTRACTING each component gives a graph in which
  ↪ every 3-edge cut is visible; the 2-edge-cuts
// are exactly the bridges plus the "cactus" pairs
  ↪ from LowLink.
// 3-VERTEX-CONNECTIVITY (SPQR / triconnected
  ↪ components) is a genuinely harder, ~400-line
// algorithm. If you only need to test
  ↪ 3-vertex-connectivity for small n, delete every
  ↪ pair of
// vertices and check connectivity in O(n^2 (n + m)).
```

3.3.6 OnlineBridges

ONLINE (INCREMENTAL) BRIDGES - maintain the number of bridges while edges are ADDED, in

```
// O(alpha) amortised per edge. Undirected,
  ↪ 0-indexed, edges may repeat (a doubled edge
  ↪ kills the
// bridge, which is exactly right).
// LowLink (01) recomputes bridges from scratch in
  ↪ O(n + m); use this when the graph grows and you
// must answer after every insertion: "after each new
  ↪ road, how many roads are still critical",
// "the first moment u and v are 2-edge-connected".
// STRUCTURE. Contract every 2-edge-connected
  ↪ component to a node; what is left is the BRIDGE
// FOREST, kept as a rooted forest with parent
  ↪ pointers. Adding (u, v):
//   * same 2ecc                -> nothing changes;
//   * different trees          -> a new bridge;
  ↪ re-root the smaller tree and hang it under v;
//   * same tree, different 2ecc -> the tree path
  ↪ u..v becomes a cycle: contract it, losing
//                                (path length)
  ↪ bridges.
// Two DSUs: dsu2ecc = the contracted components,
  ↪ dsuCC = which tree of the forest you are in.
// DELETIONS are NOT supported - that needs offline
  ↪ divide and conquer or a link-cut tree.
struct OnlineBridges {
    vector<int> par, e2, cc, ccSize, stamp;
  ↪          // par is in the BRIDGE FOREST
    int bridges = 0, iter = 0;
    OnlineBridges(int n) : par(n, -1), e2(n), cc(n),
  ↪ ccSize(n, 1), stamp(n, 0) {
        iota(all(e2), 0), iota(all(cc), 0);
    }
    int find2(int u) { return u < 0 || e2[u] == u ? u
  ↪ : e2[u] = find2(e2[u]); }
    int findCC(int u) { u = find2(u); return cc[u] ==
  ↪ u ? u : cc[u] = findCC(cc[u]); }
    void makeRoot(int u) {
  ↪          // reverse the parent chain above u
        u = find2(u);
        int root = u, child = -1;
        while (u != -1) { int p = find2(par[u]);
  ↪ par[u] = child, cc[u] = root, child = u, u = p; }
        ccSize[root] = ccSize[child];
    }
    void mergePath(int u, int v) {
  ↪          // contract the u..v tree path
        ++iter;
        vector<int> pu, pv;
        int lca = -1;
        while (lca < 0) {
            if (u != -1) {
                u = find2(u), pu.push_back(u);
                if (stamp[u] == iter) { lca = u;
  ↪ break; }
                stamp[u] = iter, u = par[u];
            }
            if (v != -1) {
                v = find2(v), pv.push_back(v);
                if (stamp[v] == iter) { lca = v;
  ↪ break; }
                stamp[v] = iter, v = par[v];
            }
        }
        for (auto& p : {pu, pv})
            for (int x : p) { if (x == lca) break;
  ↪ e2[x] = lca, bridges--; }
    }
    void addEdge(int u, int v) {
        u = find2(u), v = find2(v);
        if (u == v) return;
  ↪           // already 2-edge-connected
        int a = findCC(u), b = findCC(v);
        if (a != b) {
            bridges++;
            if (ccSize[a] > ccSize[b]) swap(u, v),
  ↪ swap(a, b);  // re-root the SMALLER tree
            makeRoot(u);
            par[u] = cc[u] = v, ccSize[b] +=
  ↪ ccSize[u];
        } else mergePath(u, v);
    }
    bool sameComponent(int u, int v) { return
  ↪ find2(u) == find2(v); }   // 2-edge-connected?
};
// WHY IT IS AMORTISED O(alpha): makeRoot walks the
  ↪ smaller tree, which is the union-by-size
// argument (O(log n) total per vertex), and
  ↪ mergePath destroys one bridge per step.
// ONLINE ARTICULATION POINTS / biconnected
  ↪ components can be maintained the same way, but
  ↪ the
// bookkeeping is much heavier - use offline divide
  ↪ and conquer over time instead.
// OFFLINE DELETIONS: process the queries in reverse
  ↪ so deletions become insertions, or run
// divide-and-conquer on the time axis with a
  ↪ rollback DSU (see 02 - DS/05 - DSU).
```

3.3.7 STNumbering

ST-NUMBERING (bipolar orientation) - order the vertices v_1 = s, ..., v_n = t so that EVERY

```
// other vertex has at least one neighbour BEFORE it
  ↪ and one AFTER it. O(n + m), 0-indexed,
// undirected. Returns the order, or an EMPTY vector
  ↪ when none exists.
// EXISTS iff G + edge(s, t) is 2-VERTEX-CONNECTED
  ↪ (that is exactly what the first DFS checks).
// Why you want it: orienting every edge from the
  ↪ smaller number to the larger gives a DAG with a
// UNIQUE source s and a UNIQUE sink t, and every
  ↪ vertex lies on some s-t path. That is the tool
// for "orient the edges so the graph stays connected
  ↪ / has one source and one sink", for planarity
// testing and st-planar embeddings, and for "peel
  ↪ the graph one vertex at a time keeping both
// halves connected" (any prefix AND its complement
  ↪ induce connected subgraphs).
```

```
// Even-Tarjan: DFS from t, then insert the vertices into a list in preorder, each one just before
// or just after its parent according to a sign flipped at every insertion.
vector<int> stNumbering(int n, vector<vector<int>> adj, int s, int t) {
  if (n == 1) return {0};
  bool has = false;
  for (int v : adj[s]) has |= (v == t);          // add the s-t edge if missing
  if (!has) adj[s].push_back(t), adj[t].push_back(s);
  vector<int> dis(n, 0), low(n, 0), par(n, -1), sg(n + 2, 0), pre;
  int T = 0;
  bool bicon = true;
  function<void(int, int)> dfs1 = [&](int u, int p) {         // articulation-point check
    low[u] = dis[u] = ++T;
    int kids = 0;
    for (int v : adj[u]) {
      if (v == p) { p = -1; continue; }
      // skip ONE parent edge
      if (!dis[v]) {
        kids++, dfs1(v, u), low[u] = min(low[u], low[v]);
        if (par[u] != -2 && low[v] >= dis[u]) bicon = false;    // par[u]==-2 marks the root
      } else low[u] = min(low[u], dis[v]);
    }
    if (par[u] == -2 && kids > 1) bicon = false;
  };
  par[t] = -2, dfs1(t, -1);
  for (int i = 0; i < n; i++) if (!dis[i]) bicon = false;
  if (!bicon) return {};
  fill(all(dis), 0), fill(all(low), 0), fill(all(par), -1), T = 0;
  function<void(int, int)> dfs2 = [&](int u, int p) {          // tree rooted at t; s is fixed
    low[u] = dis[u] = ++T;
    for (int v : adj[u]) {
      if (v == p) { p = -1; continue; }
      if (!dis[v]) pre.push_back(v), par[v] = u, dfs2(v, u), low[u] = min(low[u], low[v]);
      else low[u] = min(low[u], dis[v]);
    }
  };
  dis[s] = low[s] = ++T, sg[dis[s]] = -1;           // s takes number 1, never entered
  dfs2(t, -1);
  list<int> L{s, t};
  vector<list<int>::iterator> it(n + 2);
  it[dis[s]] = L.begin(), it[dis[t]] = next(L.begin());
  for (int v : pre) {                               // insert before/after the parent
    if (sg[low[v]] == -1) it[dis[v]] = L.insert(it[dis[par[v]]], v);
    else it[dis[v]] = L.insert(next(it[dis[par[v]]]), v);
    sg[dis[par[v]]] = -sg[low[v]];
  }
  return vector<int>(all(L));                       // order[0] = s, order[n-1] = t
}
// THE DAG. With pos[order[i]] = i, orient every edge from the smaller pos to the larger. Then
// every vertex is on an s-t path, so "assign directions keeping everything reachable from s and
// able to reach t" is solved. Reversing all edges swaps the roles of s and t.
// EAR DECOMPOSITION is the other standard certificate of 2-connectivity and is often what a
// problem really wants: an open ear decomposition exists iff the graph is 2-vertex-connected, and
// the ears are found from the same DFS (each back edge starts one ear).
// If the graph is only 2-EDGE-connected, use the bridge tree instead; st-numbering needs vertex
// connectivity.
```

3.3.8 CactusTree

CACTUS -> TREE. A CACTUS is a connected graph in which every EDGE lies on at most one simple

```
// cycle (equivalently m <= 3(n-1)/2 and the cycles are edge-disjoint). O(n + m), 0-indexed, no
// self loops and no parallel edges.
// Build a tree on n + (number of cycles) nodes: node n + i represents the i-th cycle and is joined
// to every vertex of that cycle; every BRIDGE stays a tree edge. Then path/subtree queries on the
// cactus become ordinary tree queries (LCA, HLD, rerooting), which is the whole point.
// Reach for it whenever the input says "each edge belongs to at most one cycle" or "n vertices and
// n edges" (a single cycle with trees hanging off - a "functional graph" shape) and the question
// is about paths, distances, or counting: "number of simple paths between u and v" (2 per cycle
// the path enters and leaves through two different vertices), "shortest path in a cactus", "how
// many vertices can a path from u to v visit".
// This is really the BLOCK-CUT TREE (03) specialised: on a cactus every block is a single edge or
// a single cycle, so the block nodes ARE the cycle nodes and the construction is much shorter.
struct CactusTree {
  int n, cycles = 0;
  vector<vector<int>> tree;
  // size n + cycles after build()
  vector<char> inCycle;
  // per EDGE id
  vector<int> par, parE, state;
  vector<vector<pair<int, int>>> g;
  // g[u] = {neighbour, edge id}
  CactusTree(int n) : n(n), par(n, -1), parE(n, -1), state(n, 0), g(n) {}
  void addEdge(int u, int v) {
    int id = inCycle.size();
    inCycle.push_back(0), g[u].push_back({v, id}), g[v].push_back({u, id});
  }
  void dfs(int u, int p) {
    state[u] = 1, par[u] = p;
    for (auto [v, id] : g[u]) {
      if (v == p) continue;
      if (!state[v]) parE[v] = id, dfs(v, u);
      else if (state[v] == 1) {
        // back edge: close a cycle
        int c = n + cycles++;
        tree.push_back({});
        for (int x = u, e = parE[u]; x != v; x = par[x], e = parE[x])
```

```
                    tree[x].push_back(c),
    ↪ tree[c].push_back(x), inCycle[e] = 1;
                tree[v].push_back(c),
    ↪ tree[c].push_back(v), inCycle[id] = 1;
            }
        }
        state[u] = 2;
    }
    void build() {
        tree.assign(n, {});
        for (int i = 0; i < n; i++) if (!state[i])
    ↪ dfs(i, -1);
        for (int u = 0; u < n; u++) for (auto [v, id]
    ↪ : g[u])   // the bridges stay as tree edges
            if (!inCycle[id]) tree[u].push_back(v);
    }
};
// The tree is BIPARTITE between real vertices and
    ↪ cycle nodes only in the cycle parts; a bridge
// joins two real vertices directly, so do not assume
    ↪ alternation.
// NUMBER OF SIMPLE u-v PATHS = 2^(number of CYCLE
    ↪ NODES on the u-v tree path), because inside a
// cycle you may go either way round. That is the
    ↪ standard use of this decomposition.
// SHORTEST PATH IN A CACTUS: on the tree path, a
    ↪ bridge contributes its weight and a cycle node
// contributes min(clockwise, anticlockwise) between
    ↪ its two attachment points - store each
// vertex's position and prefix sum along its cycle
    ↪ to get that in O(1).
// VERTEX CACTUS (every VERTEX in at most one cycle)
    ↪ is a strictly smaller class and the same code
// builds its tree.
// IF THE GRAPH IS NOT A CACTUS this silently
    ↪ produces nonsense - verify m and the block
    ↪ sizes, or
// just use the block-cut tree, which is correct on
    ↪ any graph.
```

3.4 Satisfiability

3.4.1 TwoSat

2-SAT via Kosaraju - O(n + m). Self-contained, 0-based variables.

```
// Literal encoding: 2*i = (x_i false), 2*i+1 = (x_i
    ↪ true).
// Satisfiable iff x and !x are never in the same
    ↪ SCC; then take the literal whose SCC is LATER
// in topological order.
struct TwoSat {
    int n;
    vector<vector<int>> adj, radj;
    vector<int> ord, comp;
    vector<bool> used, val;

    TwoSat(int n = 0) : n(n), adj(2 * n), radj(2 * n)
    ↪ {}
    int lit(int i, bool t) { return 2 * i + t; }
    int add_var() { n++, adj.resize(2 * n),
    ↪ radj.resize(2 * n); return n - 1; }

    void imply(int a, int b) {
    ↪    // literal a  =>  literal b  (and
    ↪ contrapositive)
        adj[a].push_back(b), radj[b].push_back(a);
        adj[b ^ 1].push_back(a ^ 1), radj[a ^
    ↪ 1].push_back(b ^ 1);
    }
    void either(int a, int b) { imply(a ^ 1, b); }
    ↪    // (a OR b)
    void force(int a) { imply(a ^ 1, a); }
    ↪    // a must be true
    void nand_(int a, int b) { either(a ^ 1, b ^ 1);
    ↪ }  // not both
    void eq(int a, int b) { imply(a, b), imply(b, a);
    ↪ } // a <=> b
    void xor_(int a, int b) { eq(a, b ^ 1); }

    // AT MOST ONE of `lits` is true, with O(k) edges
    ↪ instead of O(k^2) (prefix/"lightbulb" trick):
    // aux_i means "someone at index <= i was chosen".
    void at_most_one(const vector<int>& lits) {
        int prev = -1;
        for (int c : lits) {
            int cur = lit(add_var(), true);
            imply(c, cur);
            if (prev != -1) imply(prev, cur),
    ↪ imply(c, prev ^ 1);
            prev = cur;
        }
    }

    void dfs1(int u) { used[u] = true; for (int v :
    ↪ adj[u]) if (!used[v]) dfs1(v); ord.push_back(u);
    ↪ }
    void dfs2(int u, int c) { comp[u] = c; for (int v
    ↪ : radj[u]) if (comp[v] < 0) dfs2(v, c); }

    bool solve() {
        used.assign(2 * n, false), comp.assign(2 * n,
    ↪ -1), val.assign(n, false), ord.clear();
        for (int i = 0; i < 2 * n; i++) if (!used[i])
    ↪ dfs1(i);
        int c = 0;
        for (int i = 2 * n - 1; i >= 0; i--) if
    ↪ (comp[ord[i]] < 0) dfs2(ord[i], c++);
        for (int i = 0; i < n; i++) {
            if (comp[2 * i] == comp[2 * i + 1])
    ↪ return false;
            val[i] = comp[2 * i + 1] > comp[2 * i];
        }
        return true;
    ↪    // read val[0..base_n-1]; aux vars are padding
    }
};
// RANGE CLAUSE ("if x then ALL of [l,r]") in O(log
    ↪ n) edges: build a segment tree over the
// variables, give every internal node an aux
    ↪ variable, add parent => both children, then
// imply(x, node) for the O(log n) covering nodes.
    ↪ Mirror the tree (child => parent) for
// "if any of [l,r] then x".
```

3.4.2 DPLL

GENERAL SAT (3-SAT and beyond) by DPLL - Davis-Putnam-Logemann-Loveland. 0-indexed variables.

```
// Exponential in the worst case, but with unit
    ↪ propagation and pure-literal elimination it
    ↪ decides
// most contest-sized instances (n in the hundreds,
    ↪ even thousands) instantly.
// 2-SAT is polynomial (01 - TwoSat) - always check
    ↪ whether the clauses really need three literals.
// Reach for this when a problem reduces to "assign
    ↪ booleans so that all these constraints hold" and
// some clause has three or more literals: exact
```

```
  ↪ cover with few options, graph colouring encoded
  ↪ as
// clauses, puzzle solvers, "is this configuration
  ↪ reachable" searches with heavy pruning.
// LITERAL ENCODING: the positive literal of variable
  ↪ v is v, the negative one is ~v (= -v-1).
// A clause is a vector of literals meaning their OR.
  ↪ Internally literal u lives at index u + n.
// THE THREE MOVES.
//   UNIT PROPAGATION: a clause with all but one
  ↪ literal already false forces that last literal.
//   PURE LITERAL: a variable that only ever appears
  ↪ with one sign can be set that way for free.
//   BRANCH: pick the unassigned variable with the
  ↪ MOST remaining occurrences, try it, backtrack.
struct SAT {
    int n;
    vector<int> occ, pos, neg, unitS, pureS;
    vector<vector<int>> g, cls, level;
  ↪          // g[lit] = clauses containing lit
    vector<char> val;
  ↪          // val[u + n] = "literal u is true"
    SAT(int n) : n(n), occ(2 * n, 0), g(2 * n) {}
    void addClause(const vector<int>& c) {
        for (int u : c) g[u +
  ↪ n].push_back(cls.size()), occ[u + n]++;
        cls.push_back(c);
    }
    void set(int u) {
  ↪          // assign literal u = true
        val[u + n] = 1, level.back().push_back(u);
        for (int i : g[u + n]) if (pos[i]++ == 0) for
  ↪ (int w : cls[i]) --occ[w + n];
        for (int i : g[~u + n]) { ++neg[i]; if
  ↪ (pos[i] == 0) unitS.push_back(i); }
    }
    void undo() {
        int u = level.back().back();
        level.back().pop_back(), val[u + n] = 0;
        for (int i : g[u + n]) if (--pos[i] == 0) for
  ↪ (int w : cls[i]) ++occ[w + n];
        for (int i : g[~u + n]) --neg[i];
    }
    bool reduce() {
  ↪          // propagate until nothing is forced
        while (!unitS.empty() || !pureS.empty()) {
            if (!pureS.empty()) {
                int u = pureS.back();
  ↪ pureS.pop_back();
                if (occ[u + n] == 1 && occ[~u + n] ==
  ↪ 0) set(u);
            } else {
                int i = unitS.back();
  ↪ unitS.pop_back();
                if (pos[i] > 0) continue;
  ↪          // clause already satisfied
                if (neg[i] == (int)cls[i].size())
  ↪ return false; // clause already violated
                if (neg[i] + 1 == (int)cls[i].size())
  ↪ {
                    int w = 0;
                    bool found = false;
                    for (int u : cls[i]) if (!val[u +
  ↪ n] && !val[~u + n]) w = u, found = true;
                    if (!found) return false;
                    set(w);
                }
            }
        }
        return true;
    }
    bool solve() {
  ↪          // val[v + n] = the model afterwards
        val.assign(2 * n, 0);
        pos.assign(cls.size(), 0),
  ↪ neg.assign(cls.size(), 0);
        level.assign(1, {}), unitS.clear(),
  ↪ pureS.clear();
        for (;;) {
            if (reduce()) {
                int s = -1, bc = 0;
  ↪          // branch on the most constrained
                for (int u = 0; u < n; u++) {
  ↪          // UNASSIGNED variable
                    if (val[u + n] || val[~u + n])
  ↪ continue;
                    int c = occ[u + n] + occ[~u + n];
                    if (c > bc) bc = c, s = u;
                }
                if (s < 0) return true;
  ↪          // every clause is satisfied
                level.push_back({}), set(s);
            } else {
                if (level.back().empty()) return
  ↪ false;
                int s = level.back()[0];
                while (!level.back().empty()) undo();
                level.pop_back();
                if (level.empty()) return false;
                set(~s);
  ↪          // the other branch
            }
        }
    }
    bool value(int v) const { return val[v + n]; }
};
// EXAMPLE. (x0 OR NOT x1) AND (NOT x1) AND (NOT x1
  ↪ OR x3 OR x5) is
//   addClause({0, ~1});  addClause({~1});
  ↪ addClause({~1, 3, 5});
// WATCHED LITERALS + CLAUSE LEARNING (CDCL) is what
  ↪ real solvers add on top; it is far more code
// and rarely needed in a contest - if DPLL is too
  ↪ slow, the intended solution is not SAT.
// EXACTLY-ONE OF k VARIABLES: one clause (v1 OR ...
  ↪ OR vk) plus C(k,2) clauses (NOT vi OR NOT vj);
// for large k use the commander/sequential encoding
  ↪ to keep the clause count linear.
// AT MOST ONE with 2-SAT structure? Only for k = 2.
  ↪ Beyond that you are in this file.
// MAX-SAT and #SAT are harder still; for #SAT on n
  ↪ <= 20 variables, use inclusion-exclusion or a
// subset DP over the clause set instead.
```

3.5 Eulerian Path

3.5.1 EulerianPath

Hierholzer's Algorithm for Eulerian Path / Circuit - O(V + E)

```
// Works for both Directed and Undirected graphs
struct EulerianPath {
    int n, m;
    vector<vector<pair<int, int>>> adj; // {neighbor, edge_id}
    vector<bool> used_edge;
    vector<int> in_deg, out_deg, path;

    EulerianPath(int n = 0, int m = 0) : n(n), m(m), adj(n + 1), used_edge(m, false), in_deg(n + 1, 0), out_deg(n + 1, 0) {}

    void add_edge(int u, int v, int edge_id, bool directed = true) {
        adj[u].pb({v, edge_id});
        out_deg[u]++;
        in_deg[v]++;
        if (!directed) {
            adj[v].pb({u, edge_id});
            out_deg[v]++;
            in_deg[u]++;
        }
    }

    // Finds Eulerian path starting at start_node. Returns false if graph is not Eulerian
    bool solve(int start_node) {
        vector<int> ptr(n + 1, 0);
        stack<int> st;
        st.push(start_node);

        while (!st.empty()) {
            int u = st.top();
            if (ptr[u] < adj[u].size()) {
                auto [v, id] = adj[u][ptr[u]++];
                if (!used_edge[id]) {
                    used_edge[id] = true;
                    st.push(v);
                }
            } else {
                path.pb(u);
                st.pop();
            }
        }
        reverse(all(path));

        // Verify that every edge was traversed
        for (bool used : used_edge) {
            if (!used) return false;
        }
        return true;
    }
};
```

3.6 Flow and Min Cut

3.6.1 Dinic

DINIC MAX FLOW. O(V^2 E) in general, O(E sqrt(V)) on unit-capacity graphs, O(V^2 E) worst case but

```
// fast in practice. After maxflow, vertices reachable from s in the residual graph form the min cut.
// Scaling (start with big capacities) helps on dense graphs with large capacities.
// DINIC MAX FLOW. O(V^2 E) general, O(E sqrt(E)) with unit capacities, O(E sqrt(V)) for bipartite matching. This is the SCALING variant (big capacities first), so it stays fast on wide ranges.
// addEdge(a, b, cap, rcap): pass rcap = cap for an UNDIRECTED edge. Edge::flow() recovers the flow on each arc. After calc(s,t), leftOfMinCut(v) is true exactly for the SOURCE side of a minimum cut - the cut edges are those from a left vertex to a right vertex. 0-indexed.
struct Dinic {
    struct Edge {
        int to, rev;
        ll c, oc;
        ll flow() { return max(oc - c, 0LL); } // if you need flows
    };

    vector<int> lvl, ptr, q;
    vector<vector<Edge> > adj;

    Dinic(int n) : lvl(n), ptr(n), q(n), adj(n) {
    }

    void addEdge(int a, int b, ll c, ll rcap = 0) {
        adj[a].push_back({b, (int) adj[b].size(), c, c});
        adj[b].push_back({a, (int) adj[a].size() - 1, rcap, rcap});
    }

    ll dfs(int v, int t, ll f) {
        if (v == t || !f) return f;
        for (int &i = ptr[v]; i < (int) adj[v].size(); i++) {
            Edge &e = adj[v][i];
            if (lvl[e.to] == lvl[v] + 1)
                if (ll p = dfs(e.to, t, min(f, e.c))) {
                    e.c -= p, adj[e.to][e.rev].c += p;
                    return p;
                }
        }
        return 0;
    }

    ll calc(int s, int t) {
        ll flow = 0;
        q[0] = s;
        for (int L = 0; L < 31; L++)
            do {
                // 'int L=30' maybe faster for random data
                lvl = ptr = vector<int>((int) q.size());
                int qi = 0, qe = lvl[s] = 1;
                while (qi < qe && !lvl[t]) {
                    int v = q[qi++];
                    for (Edge e: adj[v])
                        if (!lvl[e.to] && e.c >> (30 - L))
                            q[qe++] = e.to, lvl[e.to] = lvl[v] + 1;
                }
                while (ll p = dfs(s, t, LLONG_MAX)) flow += p;
            } while (lvl[t]);
        return flow;
    }
```

```
    bool leftOfMinCut(int a) { return lvl[a] != 0; }
};
```

3.6.2 MCMF

MIN-COST MAX-FLOW with JOHNSON POTENTIALS (Dijkstra instead of SPFA on every augmentation).

```
// O(F * E log V) where F = number of augmentations. Much faster than the SPFA version and the
// one to use by default. Negative edge costs are allowed at construction PROVIDED there is no negative-cost CYCLE:
// run one Bellman-Ford
// to initialise the potentials (setPotentialsBF), after that every reduced cost is >= 0.
struct MCMF {
    struct E { int to, rev; ll cap, cost, flow = 0; };
    int n;
    vector<vector<E>> g;
    vector<ll> pot, dist;
    vector<int> pv, pe;

    MCMF(int n) : n(n), g(n), pot(n, 0), dist(n), pv(n), pe(n) {}
    void addEdge(int u, int v, ll cap, ll cost) {
        g[u].push_back({v, (int)g[v].size(), cap, cost});
        g[v].push_back({u, (int)g[u].size() - 1, 0, -cost});
    }
    void setPotentialsBF(int s) { // only needed if some cost < 0
        fill(all(pot), llinf);
        pot[s] = 0;
        for (int it = 0; it < n; it++) {
            bool ch = false;
            for (int u = 0; u < n; u++) if (pot[u] < llinf)
                for (auto& e : g[u]) if (e.cap - e.flow > 0 && pot[u] + e.cost < pot[e.to])
                    pot[e.to] = pot[u] + e.cost, ch = true;
            if (!ch) break;
        }
        for (auto& x : pot) if (x == llinf) x = 0;
    }
    bool dijkstra(int s, int t) {
        fill(all(dist), llinf);
        priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<>> pq;
        dist[s] = 0, pq.push({0, s});
        while (!pq.empty()) {
            auto [d, u] = pq.top(); pq.pop();
            if (d > dist[u]) continue;
            for (int i = 0; i < (int)g[u].size(); i++) {
                auto& e = g[u][i];
                if (e.cap - e.flow <= 0) continue;
                ll nd = d + e.cost + pot[u] - pot[e.to];      // reduced cost, always >= 0
                if (nd < dist[e.to]) dist[e.to] = nd, pv[e.to] = u, pe[e.to] = i, pq.push({nd, e.to});
            }
        }
        return dist[t] < llinf;
    }
    // Returns {flow, cost}. Pass maxf to cap the flow (e.g. for min-cost k-flow).
    pair<ll, ll> run(int s, int t, ll maxf = llinf) {
        ll flow = 0, cost = 0;
        while (flow < maxf && dijkstra(s, t)) {
            for (int i = 0; i < n; i++) if (dist[i] < llinf) pot[i] += dist[i];
            ll aug = maxf - flow;
            for (int v = t; v != s; v = pv[v]) aug = min(aug, g[pv[v]][pe[v]].cap - g[pv[v]][pe[v]].flow);
            for (int v = t; v != s; v = pv[v]) {
                auto& e = g[pv[v]][pe[v]];
                e.flow += aug, g[v][e.rev].flow -= aug;
                cost += aug * e.cost;
            }
            flow += aug;
        }
        return {flow, cost};
    }
    // MIN-COST flow of exactly K units: run(s, t, K) and check flow == K.
    // MIN-COST (not max) flow: stop as soon as dist[t] + pot[t] - pot[s] > 0 (no profitable path).
};
// MODELLING: assignment on a SPARSE graph; for a dense/complete one use 07 - Matching/03 - Hungarian * transportation * "k disjoint paths of minimum total cost" *
// min-cost bipartite matching * scheduling with penalties * MCMF is also how you solve
// "choose k items with a matroid-ish constraint minimising cost".
```

3.6.3 FlowWithLowerBounds

FLOWS WITH LOWER BOUNDS (circulation with demands). Every edge needs d(e) <= f(e) <= c(e).

```
// Build an auxiliary network, run one max flow; feasible iff every edge out of s' saturates.
//   excess(v) = (sum of d over edges INTO v) - (sum of d over edges OUT of v)
//   excess(v) > 0 -> edge s'->v of capacity excess(v);  < 0 -> edge v->t' of capacity -excess
//   every original edge (u,v) keeps capacity c - d
// For an s-t flow, add t->s with capacity INF first, which turns it into a circulation.
// Needs Dinic (01 - Dinic.cpp) with addEdge(u, v, cap) and calc(s, t).
struct LowerBoundFlow {
    int n, S, T; // S, T are the auxiliary super source/sink
    Dinic din;
    vector<ll> excess, low;
    vector<array<int, 2>> eid; // per original edge: {u, index in din.adj[u]}

    LowerBoundFlow(int n) : n(n), S(n), T(n + 1), din(n + 2), excess(n + 2, 0) {}
    void addEdge(int u, int v, ll lo, ll hi) {
        low.push_back(lo);
        eid.push_back({u, (int)din.adj[u].size()});
        din.addEdge(u, v, hi - lo);
        excess[v] += lo, excess[u] -= lo;
    }
    bool feasible() { // circulation only (no s, t)
        ll need = 0;
        for (int v = 0; v < n; v++) {
```

```
            if (excess[v] > 0) din.addEdge(S, v,
    ↪ excess[v]), need += excess[v];
            else if (excess[v] < 0) din.addEdge(v, T,
    ↪ -excess[v]);
        }
        return din.calc(S, T) == need;
    }
    // Min / max s-t flow subject to the bounds - see
    ↪ the RECIPES block below for the ONLY
    // correct order of operations. You must DELETE
    ↪ the t->s edge (zero both residual directions)
    // before the second max flow, otherwise Dinic's
    ↪ first augmenting path runs s -> t backwards
    // along that arc and CANCELS the feasible flow
    ↪ instead of extending it.
    // feasible() appends the S/T edges every time it
    ↪ is called - call it exactly once.
    ll flowOn(int i) { return low[i] +
    ↪ din.adj[eid[i][0]][eid[i][1]].flow(); }
};
/* RECIPES
   "each edge must be used at least once"          ->
   ↪ d(e) = 1
   "each vertex must be visited between L and R"   ->
   ↪ split v into v_in -> v_out with [L, R]
   "exact degree subgraph"                         ->
   ↪ d = c = required degree on the split edge
   MAX s-t FLOW WITH BOUNDS: add t->s with capacity
   ↪ INF, run feasible(); then delete that edge
     and run one more max flow from s to t on the
   ↪ residual graph. The answer is the flow that
     was on t->s plus the extra augmentation.
   MIN s-t FLOW WITH BOUNDS: same, but augment from t
   ↪ to s to cancel as much as possible. */
```

3.6.4 StoerWagner

STOER-WAGNER GLOBAL MIN CUT - minimum cut over ALL vertex pairs, undirected weighted, O(V^3).

```
// No source/sink. Returns {cut weight, one side of
  ↪ the cut}.
// Key lemma: in a maximum-adjacency order, if s is
  ↪ the second-to-last and t the last vertex
// added, then the minimum s-t cut is exactly {t}.
  ↪ Merge s and t and repeat n-1 times.
pair<ll, vector<int>> globalMinCut(vector<vector<ll>>
  ↪ w) {
    int n = w.size();
    if (n < 2) return {0, {}};
    vector<char> gone(n, 0);
    vector<vector<int>> grp(n);
    for (int i = 0; i < n; i++) grp[i] = {i};
    ll bestW = llinf;
    vector<int> best;
    for (int phase = 0; phase + 1 < n; phase++) {
        vector<ll> ww(n, 0);
        vector<char> added(n, 0);
        int prev = -1, last = -1;
        for (int it = 0; it < n - phase; it++) {
  ↪  // maximum-adjacency order
            int sel = -1;
            for (int j = 0; j < n; j++)
                if (!gone[j] && !added[j] && (sel < 0
  ↪ || ww[j] > ww[sel])) sel = j;
            added[sel] = 1, prev = last, last = sel;
            for (int j = 0; j < n; j++) if (!gone[j]
  ↪ && !added[j]) ww[j] += w[sel][j];
        }
        ll cut = 0;
  ↪  // cut-of-the-phase separates {last}
        for (int j = 0; j < n; j++) if (!gone[j] && j
  ↪ != last) cut += w[last][j];
        if (cut < bestW) bestW = cut, best =
  ↪ grp[last];
        gone[last] = 1;
  ↪  // merge last into prev
        grp[prev].insert(grp[prev].end(),
  ↪ all(grp[last]));
        for (int j = 0; j < n; j++) if (!gone[j] && j
  ↪ != prev)
            w[prev][j] += w[last][j], w[j][prev] =
  ↪ w[prev][j];
    }
    return {bestW, best};
}
// USES: "split the network into two non-empty parts
  ↪ as cheaply as possible", edge connectivity
// of a weighted undirected graph, "minimum number of
  ↪ edges to delete to disconnect the graph"
// (unit weights). Doing this with n-1 max-flows is
  ↪ both slower and longer to write.
// KARGER (randomized contraction) finds a min cut
  ↪ with probability >= 1/C(n,2) per run; the
// useful corollary is that a graph has AT MOST
  ↪ C(n,2) distinct minimum cuts.
```

3.6.5 MinCutModelling

MIN-CUT MODELLING - the reason flow appears at ICPC. The code is never the hard part; choosing

```
// Needs: Dinic (01 - Dinic.cpp).
// what the vertices and the infinite capacities mean
  ↪ is. Rule of thumb: if the objective contains
// an "infinity", you are CUTTING, not flowing.

// --- MAXIMUM WEIGHT CLOSURE / PROJECT SELECTION ---
// Choose a set S closed under the dependency arcs
  ↪ (if v in S and v -> u is a dependency, u in S)
// maximising the total weight. Build: s -> v with
  ↪ capacity w(v) for every w(v) > 0;
// v -> t with capacity -w(v) for every w(v) < 0; u
  ↪ -> v with INF for every dependency u needs v.
// Answer = (sum of positive weights) - maxflow.  S =
  ↪ the source side of the min cut.
// USES: "each project earns p but needs these
  ↪ machines that cost c"; "pick cells maximising
// profit but a chosen cell forces its neighbours";
  ↪ densest-subgraph style selections.
ll maxClosure(int n, const vector<ll>& w, const
  ↪ vector<pair<int, int>>& need, vector<int>*
  ↪ chosen = nullptr) {
    Dinic din(n + 2);
    int s = n, t = n + 1;
    ll pos = 0;
    for (int v = 0; v < n; v++)
        if (w[v] > 0) pos += w[v], din.addEdge(s, v,
  ↪ w[v]);
        else if (w[v] < 0) din.addEdge(v, t, -w[v]);
    for (auto [u, v] : need) din.addEdge(u, v,
  ↪ llinf);         // choosing u forces choosing v
    ll cut = din.calc(s, t);
    if (chosen) for (int v = 0; v < n; v++) if
  ↪ (din.leftOfMinCut(v)) chosen->push_back(v);
    return pos - cut;
}
/* THE OTHER STANDARD REDUCTIONS
 TWO LABELS WITH PAIR PENALTIES ("cats or dogs",
  ↪ image segmentation)
```

```
  Every vertex gets label A or B. Cost a_v for A,
 ↪ b_v for B, and c_uv extra if u and v differ.
  Build s -> v with b_v, v -> t with a_v, and u <->
 ↪ v with c_uv in both directions.
  The min cut is the minimum total cost; the source
 ↪ side takes label A.
  PRECONDITION: the pair penalty must be SUBMODULAR
 ↪ (cost(A,A) + cost(B,B) <= cost(A,B) +
  cost(B,A)). A penalty that rewards disagreement
 ↪ cannot be modelled by a cut - it is NP-hard.

VERTEX CAPACITIES / VERTEX-DISJOINT PATHS
  Split v into v_in -> v_out with the vertex
 ↪ capacity. Vertex-disjoint s-t paths use capacity
 ↪ 1.

BIPARTITE FAMILY (see 07 - Matching and 10 -
 ↪ Theorems)
  max matching        = max flow with unit capacities
  min vertex cover    = max matching (Konig)
 ↪  -> recover via the alternating-BFS set
  max independent set = n - max matching
  min path cover of a DAG (vertex-disjoint) = n -
 ↪ max matching of the split graph

"MUST BE ON THE SOURCE SIDE" -> add s -> v with INF.
 ↪   "MUST BE ON THE SINK SIDE" -> v -> t INF.
That is how you force decisions inside an otherwise
 ↪ free min cut.

EDGE / VERTEX DISJOINT PATHS = Menger. Recover the
 ↪ paths by decomposing the integral flow.

LOWER BOUNDS ON EDGES -> 03 -
 ↪ FlowWithLowerBounds.cpp.

MAXIMUM DENSITY SUBGRAPH (maximise |E(S)| / |S|):
 ↪ binary search lambda, then min cut with
  s -> v capacity m, v -> t capacity m + 2*lambda -
 ↪ deg(v), u <-> v capacity 1. Feasible iff
  the min cut is < n*m. O(log(n^2) * maxflow) since
 ↪ the density is a ratio of integers <= n^2.

MINIMUM CUT WITHOUT s AND t (global) -> 04 -
 ↪ StoerWagner.cpp, O(V^3), or n-1 max flows.

ALL-PAIRS MIN CUT -> the Gomory-Hu tree: n-1 max
 ↪ flows build a weighted tree in which the min
  u-v cut equals the MINIMUM EDGE on the u-v tree
 ↪ path. Use it when many pairs are asked.

COMPLEXITY TABLE - read this at minute 3, not minute
 ↪ 90
  Dinic                  O(V^2 E) general | O(E sqrt
 ↪ E) unit caps | O(E sqrt V) bipartite
  Edmonds-Karp           O(V E^2)
 ↪ -- do not
  Push-relabel / HLPP    O(V^2 sqrt E)
 ↪ -- dense graphs, n ~ 1000, m ~ n^2/2
  MCMF (SSP + potentials) O(F * E log V)
 ↪ -- F = total flow, so bound F first
  Hungarian              O(n^2 m)
 ↪ -- dense assignment, n <= ~1000
  Hopcroft-Karp          O(E sqrt V)
 ↪ -- bipartite matching, n >= few thousand
  Kuhn                   O(V E)
 ↪ -- shorter, fine to ~2000 vertices
  Blossom (general)      O(V^3)
   Stoer-Wagner          O(V^3)
   Gomory-Hu             O(V * maxflow)
 If the flow value itself can be huge (capacities up
  ↪ to 1e9), an O(F ...) bound is not a bound -
 scale the capacities or use a strongly polynomial
  ↪ algorithm. */
```

3.6.6 GomoryHu

GOMORY-HU TREE - ALL-PAIRS MINIMUM CUT from only n-1 max-flow runs.

```
// Needs: Dinic (01 - Dinic.cpp).
// The result is a weighted tree on the same vertex
  ↪ set with the property that, for EVERY pair
// (u, v), the minimum u-v cut in the original graph
  ↪ equals the MINIMUM EDGE WEIGHT on the u-v
// path in the tree. (There are at most n-1 distinct
  ↪ min-cut values in any graph - that is the
// whole content of the theorem.)
// Reach for it when a problem asks about min cuts
  ↪ between many pairs: "sum/max/k-th of all
// pairwise min cuts", "order the vertices to
  ↪ maximise the sum of consecutive cuts", "how many
// pairs have min cut >= k". Undirected graphs only.
// Complexity: (n-1) x Dinic. The tree is returned as
  ↪ parent + weight arrays (par[0] = -1).
struct GomoryHu {
    int n;
    vector<array<ll, 3>> edges;
  ↪          // {u, v, capacity}, undirected
    vector<int> par;
    vector<ll> w;
    GomoryHu(int n) : n(n), par(n, 0), w(n, 0) {}
    void addEdge(int u, int v, ll c) {
  ↪ edges.push_back({(ll)u, (ll)v, c}); }
    void build() {
        par.assign(n, 0), w.assign(n, 0), par[0] = -1;
        for (int i = 1; i < n; i++) {
            Dinic din(n);
            for (auto [u, v, c] : edges)
  ↪ din.addEdge(u, v, c, c);   // undirected: both
  ↪ directions
            w[i] = din.calc(i, par[i]);
            for (int j = i + 1; j < n; j++)
  ↪           // re-parent the same-side vertices
                if (par[j] == par[i] &&
  ↪ din.leftOfMinCut(j)) par[j] = i;
        }
    }
    ll minCut(int u, int v) const {
  ↪          // min edge on the tree path
        ll best = llinf;
        vector<int> du(n, 0);
        for (int x = u; x != -1; x = par[x]) du[x] =
  ↪ 1;
        vector<int> pv;
        int m = v;
        while (!du[m]) pv.push_back(m), m = par[m];
  ↪           // m = the meeting vertex
        for (int x = u; x != m; x = par[x]) best =
  ↪ min(best, w[x]);
        for (int x : pv) best = min(best, w[x]);
        return best;
    }
};
// EQUIVALENT-FLOW TREE (Gusfield's variant) is the
  ↪ same n-1 flows without the re-parenting step;
// it gets the min-cut VALUES right but not the
  ↪ actual cuts. The version above is the real
```

```
// Gomory-Hu, so the tree edges are genuine cuts.
// GLOBAL MIN CUT is the minimum edge of the whole tree - but Stoer-Wagner (04) is O(V^3) and
// needs no flow at all, so prefer it when that is all you want.
// DIRECTED graphs have no Gomory-Hu tree; you genuinely need O(n^2) flows there.
```

3.6.7 PushRelabel

HIGHEST-LABEL PUSH-RELABEL max flow, O(V^2 sqrt(E)), 0-indexed. Same interface as Dinic (01).

```
// WHEN TO PREFER IT OVER DINIC: dense graphs (V ~ 1000-5000, E ~ V^2/2) and graphs with large
// capacities, where Dinic's O(V^2 E) bound bites - usually 3-10x faster there. On unit-capacity
// or bipartite-matching graphs Dinic is O(E sqrt V) and wins, so keep both.
// It does NOT maintain a flow at intermediate times - only at the end - so it cannot be used for
// the incremental "add capacity and continue" tricks; Dinic can.
// IDEA: give every vertex a HEIGHT and let excess flow downhill (push); a vertex with excess and
// nowhere to go is lifted (relabel). Source starts at height V, so nothing can return to it.
// The GAP HEURISTIC (if no vertex has height h any more, lift everything above h out of range) is
// what makes it fast in practice - it is the `co[]` bookkeeping below, do not drop it.
// addEdge(a, b, cap, rcap): rcap = cap gives an UNDIRECTED edge. After calc, leftOfMinCut(v) marks
// the SOURCE side of a minimum cut, and e.f is the flow on that arc.
struct PushRelabel {
    struct Edge { int to, rev; ll f, c; };
    int n;
    vector<vector<Edge>> g;
    vector<ll> ec;
    vector<int> H, cur, co;
    vector<vector<int>> hs;
    PushRelabel(int n) : n(n), g(n), ec(n), H(n), cur(n), co(2 * n + 1), hs(2 * n + 1) {}
    void addEdge(int a, int b, ll cap, ll rcap = 0) {
        if (a == b) return;
        g[a].push_back({b, (int)g[b].size(), 0, cap});
        g[b].push_back({a, (int)g[a].size() - 1, 0, rcap});
    }
    void addFlow(int u, Edge& e, ll f) {
        Edge& back = g[e.to][e.rev];
        if (!ec[e.to] && f)
            hs[H[e.to]].push_back(e.to);
        e.f += f, e.c -= f, ec[e.to] += f;
        back.f -= f, back.c += f, ec[u] -= f;
    }
    ll calc(int s, int t) {
        fill(all(H), 0), fill(all(ec), 0), fill(all(co), 0), fill(all(cur), 0);
        for (auto& v : hs) v.clear();
        H[s] = n, ec[t] = 1, co[0] = n - 1;
        for (auto& e : g[s]) addFlow(s, e, e.c);
        for (int hi = 0;;) {
            while (hs[hi].empty()) if (!hi--) return -ec[s];
            int u = hs[hi].back(); hs[hi].pop_back();
            while (ec[u] > 0) {
                // discharge u
                if (cur[u] == (int)g[u].size()) {
                    // relabel + gap heuristic
                    H[u] = 2 * n;
                    for (int i = 0; i < (int)g[u].size(); i++)
                        if (g[u][i].c && H[u] > H[g[u][i].to] + 1)
                            H[u] = H[g[u][i].to] + 1, cur[u] = i;
                    if (++co[H[u]], !--co[hi] && hi < n)
                        for (int i = 0; i < n; i++)
                            if (hi < H[i] && H[i] < n) --co[H[i]], H[i] = n + 1;
                    hi = H[u];
                } else if (g[u][cur[u]].c && H[u] == H[g[u][cur[u]].to] + 1)
                    addFlow(u, g[u][cur[u]], min(ec[u], g[u][cur[u]].c));
                else ++cur[u];
            }
        }
    }
    bool leftOfMinCut(int a) { return H[a] >= n; }
};
// COMPLEXITY NOTES. FIFO push-relabel is O(V^3); the highest-label rule above is O(V^2 sqrt E).
// The gap heuristic and the global relabel (BFS from t every so often) are the two practical
// speedups; the version here has the gap only, which is already enough for contests.
// TIE-BREAKING WITH DINIC: if you also need the actual min cut EDGES, both give them the same way
// (an edge from a leftOfMinCut vertex to a non-leftOfMinCut one). If you need the flow DECOMPOSED
// into paths, repeatedly DFS from s along positive-flow arcs and subtract the bottleneck.
```

3.6.8 CostScalingMCMF

MIN COST MAX FLOW BY COST SCALING (push-relabel), O(V^2 E log(V*C)) but very fast in practice -

```
// 3e4 edges are comfortable. 0-indexed, INTEGER costs and capacities.
// WHY THIS EXISTS ALONGSIDE 02 - MCMF. The successive-shortest-path version needs the residual
// graph to have NO NEGATIVE CYCLE, so it forbids negative-cost edges (Johnson potentials only fix
// negative EDGES on a DAG-like start, not negative CYCLES). This one accepts ARBITRARY costs,
// including negative cycles: it first pushes a max flow ignoring cost, then repeatedly "refines"
// the eps-optimality of the flow, halving eps each round. It also does not depend on the flow
// VALUE, so a huge max flow is fine, whereas SSP is O(F * E log V).
// Reach for it when: costs can be negative, or the flow value is large, or the graph is dense.
// The source and sink are fixed at construction. Costs are internally multiplied by V, so keep
// |cost| * V * V inside the cost type - use long long unless you have checked.
struct CostScalingMCMF {
    struct Edge { ll c, f; int to, rev; };
    ll eps = 0;
    int n, S, T;
    vector<vector<Edge>> g;
```

```
  vector<vector<int>> hs;
  vector<int> cur, isq, co;
  vector<ll> ex, h;
  CostScalingMCMF(int n, int s, int t) : n(n),
↪ S(s), T(t), g(n) {}
  void addEdge(int a, int b, ll cost, ll cap) {
↪          // cost per unit, cap >= 0
      if (a == b) return;
      cost *= n, eps = max(eps, abs(cost));
      g[a].push_back({cost, cap, b,
↪ (int)g[b].size()});
      g[b].push_back({-cost, 0, a, (int)g[a].size()
↪ - 1});
  }
  void addFlow(int u, Edge& e, ll f) {
      Edge& bk = g[e.to][e.rev];
      if (!ex[e.to] && f)
↪ hs[h[e.to]].push_back(e.to);
      e.f -= f, ex[e.to] += f, bk.f += f, ex[u] -=
↪ f;
  }
  ll maxFlow() {
↪          // plain highest-label push-relabel
      ex.assign(n, 0), h.assign(n, 0), hs.assign(2
↪ * n + 1, {}), co.assign(2 * n + 1, 0);
      cur.assign(n, 0), h[S] = n, ex[T] = 1, co[0]
↪ = n - 1;
      for (auto& e : g[S]) addFlow(S, e, e.f);
      if (hs[0].size()) for (int hi = 0; hi >= 0;) {
          int u = hs[hi].back(); hs[hi].pop_back();
          while (ex[u] > 0) {
              if (cur[u] == (int)g[u].size()) {
                  h[u] = 2 * n;
                  for (int i = 0; i <
↪ (int)g[u].size(); i++)
                      if (g[u][i].f && h[u] >
↪ h[g[u][i].to] + 1)
                          h[u] = h[g[u][i].to] + 1,
↪ cur[u] = i;
                  if (++co[h[u]], !--co[hi] && hi <
↪ n)
                      for (int i = 0; i < n; i++)
                          if (hi < h[i] && h[i] <
↪ n) --co[h[i]], h[i] = n + 1;
                  hi = h[u];
              } else if (g[u][cur[u]].f && h[u] ==
↪ h[g[u][cur[u]].to] + 1)
                  addFlow(u, g[u][cur[u]],
↪ min(ex[u], g[u][cur[u]].f));
              else ++cur[u];
          }
          while (hi >= 0 && hs[hi].empty()) --hi;
      }
      return -ex[S];
  }
  void push(int u, Edge& e, ll amt) {
      amt = min(amt, e.f);
      e.f -= amt, ex[e.to] += amt, g[e.to][e.rev].f
↪ += amt, ex[u] -= amt;
  }
  void relabel(int v) {
      ll nh = -llinf;
      for (int i = 0; i < (int)g[v].size(); i++)
          if (g[v][i].f && nh < h[g[v][i].to] -
↪ g[v][i].c)
              nh = h[g[v][i].to] - g[v][i].c,
↪ cur[v] = i;
      h[v] = nh - eps;
  }
  pair<ll, ll> solve() {
↪          // {max flow, its minimum cost}
      ll cost = 0;
      for (int i = 0; i < n; i++) for (Edge& e :
↪ g[i]) cost += e.c * e.f;
      ll flow = maxFlow();
      h.assign(n, 0), ex.assign(n, 0),
↪ isq.assign(n, 0), cur.assign(n, 0);
      queue<int> q;
      for (; eps; eps >>= 2) {
↪          // refine: halve eps twice per round
          fill(all(cur), 0);
          for (int i = 0; i < n; i++) for (auto& e
↪ : g[i])
              if (h[i] + e.c - h[e.to] < 0 && e.f)
↪ push(i, e, e.f);
          for (int i = 0; i < n; i++) if (ex[i] >
↪ 0) q.push(i), isq[i] = 1;
          while (!q.empty()) {
↪          // restore feasibility
              int u = q.front(); q.pop(), isq[u] =
↪ 0;
              while (ex[u] > 0) {
                  if (cur[u] == (int)g[u].size())
↪ relabel(u);
                  for (int& i = cur[u]; i <
↪ (int)g[u].size(); i++) {
                      Edge& e = g[u][i];
                      if (h[u] + e.c - h[e.to] < 0)
↪ {
                          push(u, e, ex[u]);
                          if (ex[e.to] > 0 &&
↪ !isq[e.to]) q.push(e.to), isq[e.to] = 1;
                          if (!ex[u]) break;
                      }
                  }
              }
          }
          if (eps > 1 && (eps >> 2) == 0) eps = 4;
      }
      for (int i = 0; i < n; i++) for (Edge& e :
↪ g[i]) cost -= e.c * e.f;
      return {flow, cost / 2 / n};
  }
  ll flowOn(const Edge& e) const { return
↪ g[e.to][e.rev].f; }
};
// MIN COST FLOW OF A GIVEN VALUE k (not necessarily
↪ maximum): cap the source arc at k, or add an
// arc T -> S of capacity INF and cost -BIG and take
↪ a min cost CIRCULATION.
// NEGATIVE COST CYCLES in the input are handled: the
↪ answer is the true minimum, which may be
// negative. With 02 - MCMF you would have to cancel
↪ the cycles by hand first (saturate every
// negative edge, then fix the resulting excesses
↪ with a circulation) - that is what this replaces.
// ASSIGNMENT PROBLEMS with n <= 500 are still better
↪ served by Hungarian (07 - Matching/03).
```

3.6.9 MinCostFlowLowerBounds

MIN COST FLOW WITH LOWER BOUNDS (min cost circulation with demands). 0-indexed, costs >= 0.

```
// Needs: MCMF (02 - MCMF.cpp).
// This is 03 - FlowWithLowerBounds with a price tag on every unit: each edge needs
// lo <= f(e) <= hi and costs c per unit, and among all FEASIBLE flows you want the cheapest.
// Reach for it for "every task must be done at least once, minimise total cost", "each shift needs
// between L and R workers", "cover every row of the matrix at least k times as cheaply as
// possible", and generally any min-cut/flow model where a constraint reads "at least".
// CONSTRUCTION. Send the mandatory lo units for free by hand, then repair the resulting imbalance:
//   base cost += lo * c ;  the arc keeps capacity hi - lo at cost c
//   excess(v) = (sum of lo entering v) - (sum of lo leaving v)
//   excess(v) > 0 -> SS -> v with capacity excess(v), cost 0;  < 0 -> v -> TT, cost -excess(v)
// One min cost max flow from SS to TT. FEASIBLE iff it saturates every SS arc; the answer is
// base + that cost. For an s-t flow, add t -> s with capacity INF and cost 0 FIRST.
struct MinCostLowerBounds {
    int n, S, T;
    MCMF f;
    ll base = 0, need = 0;
    vector<ll> low, exc;
    vector<array<int, 2>> eid;
    // {u, index in f.g[u]} per edge
    MinCostLowerBounds(int n) : n(n), S(n), T(n + 1), f(n + 2), exc(n + 2, 0) {}
    void addEdge(int u, int v, ll lo, ll hi, ll cost) {
        low.push_back(lo), eid.push_back({u, (int)f.g[u].size()});
        f.addEdge(u, v, hi - lo, cost);
        base += lo * cost, exc[v] += lo, exc[u] -= lo;
    }
    pair<bool, ll> solve() {
        // {feasible, minimum total cost}
        for (int v = 0; v < n; v++) {
            if (exc[v] > 0) f.addEdge(S, v, exc[v], 0), need += exc[v];
            else if (exc[v] < 0) f.addEdge(v, T, -exc[v], 0);
        }
        auto [fl, c] = f.run(S, T);
        return {fl == need, base + c};
    }
    ll flowOn(int i) { return low[i] + f.g[eid[i][0]][eid[i][1]].flow; }
};
// MIN COST *MAX* s-t FLOW WITH BOUNDS: add t -> s with capacity INF and cost 0, run solve(); the
// flow that ended up on that arc is the feasible s-t flow value. Then delete the arc and run one
// more MCMF from s to t on the residual to push the flow up to maximum.
// MIN COST *MIN* s-t FLOW: same, but afterwards cancel flow by running MCMF from t to s.
// NEGATIVE COSTS: this construction is still correct, but MCMF's Dijkstra is not - either call
// f.setPotentialsBF(S) first (no negative cycle allowed) or switch to 08 - CostScalingMCMF.
// b-FLOW (each vertex has a supply/demand b(v) instead of a single source): put b(v) directly into
// exc[v] before solving - the SS/TT construction is exactly the same machinery.
// SANITY CHECK: if lo > hi on any edge the instance is infeasible; the code will not notice.
```

3.6.10 MaxDensitySubgraph

MAXIMUM DENSITY SUBGRAPH - choose a NON-EMPTY vertex set S maximising |E(S)| / |S|, the number of induced edges per vertex. Undirected, 0-indexed, O(log(n^2) * maxflow) via parametric min cut.

```
// Needs: Dinic (01 - Dinic.cpp).
// (Note this is NOT the maximum CLIQUE relaxation - it is polynomial, unlike everything nearby.)
// Reach for it for "find the most tightly knit group", "the subgraph with the best edges-to-people
// ratio", spam/fraud ring detection, and as the LP-relaxation certificate for densest-subgraph
// style scoring problems.
// FRACTIONAL PROGRAMMING. Ask "is there S with |E(S)| - g|S| > 0?" for a guess g. Build
//   s -> v (capacity m), v -> t (capacity m + 2g - deg(v)), u <-> v (capacity 1 each way)
// then min cut = n*m - 2 * max over S of (|E(S)| - g|S|), so the answer is yes iff the min cut is
// strictly below n*m. Binary search g. Two distinct densities differ by at least 1/(n(n-1)), so
// O(log(n^2)) iterations of the binary search are enough for an EXACT answer with rationals.
// Below every capacity is scaled by K = n^2 so they stay INTEGRAL and the guess is g = g2 / K.
pair<ld, vector<int>> maxDensitySubgraph(int n, const vector<pair<int, int>>& e) {
    int m = e.size();
    if (n == 1 || m == 0) return {0, {0}};
    ll K = (ll)n * n;
    vector<int> deg(n, 0), best{0};
    for (auto [u, v] : e) deg[u]++, deg[v]++;
    ll lo = 0, hi = (ll)m * K;
    // density * K, density <= m
    while (lo < hi) {
        ll g2 = (lo + hi + 1) / 2;
        // test the density g = g2 / K
        Dinic din(n + 2);
        int s = n, t = n + 1;
        for (auto [u, v] : e) din.addEdge(u, v, K, K);          // undirected, capacity 1 scaled
        for (int v = 0; v < n; v++) {
            din.addEdge(s, v, (ll)m * K);
            din.addEdge(v, t, (ll)m * K + 2 * g2 - (ll)deg[v] * K);
        }
        if (din.calc(s, t) < (ll)n * m * K) {
            // some S beats density g
            lo = g2, best.clear();
            for (int v = 0; v < n; v++) if (din.leftOfMinCut(v)) best.push_back(v);
        } else hi = g2 - 1;
    }
    return {(ld)lo / K, best};
    // {density, the vertex set}
}
// The source side of the min cut IS the densest set
```

```
  ↪ - no separate extraction pass is needed.
// MAXIMUM DENSITY with VERTEX WEIGHTS or edge
  ↪ weights: replace deg(v) by the weighted degree
  ↪ and
// the capacity-1 edges by the edge weights; the same
  ↪ cut works.
// AT LEAST k VERTICES / AT MOST k VERTICES turns the
  ↪ problem NP-hard - do not try to patch this.
// MINIMUM density is trivial (a single vertex,
  ↪ density 0), so the question is always maximum.
// RELATED: "maximum (sum of vertex profits) - (sum
  ↪ of edge penalties)" is plain project selection
// (05 - MinCutModelling); the RATIO is what forces
  ↪ the binary search.
// GOLDBERG'S ORIGINAL formulation uses capacities m
  ↪ + 2g - deg(v) exactly as above; if you prefer
// floating point, binary search g as a double for
  ↪ ~40 iterations and skip the scaling.
```

3.6.11 UniqueMinCut

IS THE MINIMUM s-t CUT UNIQUE? One extra pair of DFS over the residual graph, O(V + E) after the

```
// Needs: Dinic (01 - Dinic.cpp).
// max flow. 0-indexed, directed or undirected.
// After a max flow, let A = the vertices REACHABLE
  ↪ FROM s in the residual graph (the smallest
// minimum cut) and B = the vertices that can REACH t
  ↪ in the residual graph (the largest minimum
// cut's source side is V \ B). Every minimum cut
  ↪ lies "between" these two, so:
//   the minimum cut is UNIQUE  <=>  A and B together
  ↪ cover every vertex.
// A vertex in neither set can be put on either side
  ↪ without changing the cut value, and moving it
// gives a genuinely different cut of the same weight.
// Reach for it for "is the answer unique / is the
  ↪ bottleneck forced", "which edges are in EVERY
// minimum cut", and for problems that ask you to
  ↪ output AMBIGUOUS when a tie exists.
bool uniqueMinCut(Dinic& din, int s, int t) {
  ↪          // call AFTER din.calc(s, t)
    int n = din.adj.size();
    vector<char> a(n, 0), b(n, 0);
    vector<int> st{s};
    a[s] = 1;
    while (!st.empty()) {
  ↪          // forward residual reachability
        int u = st.back(); st.pop_back();
        for (auto& e : din.adj[u]) if (e.c > 0 &&
  ↪ !a[e.to]) a[e.to] = 1, st.push_back(e.to);
    }
    st = {t}, b[t] = 1;
    while (!st.empty()) {
  ↪          // backward: residual arcs INTO u
        int u = st.back(); st.pop_back();
        for (auto& e : din.adj[u]) if
  ↪ (din.adj[e.to][e.rev].c > 0 && !b[e.to])
            b[e.to] = 1, st.push_back(e.to);
    }
    for (int v = 0; v < n; v++) if (!a[v] && !b[v])
  ↪ return false;
    return true;
}
// THE TWO EXTREME CUTS come out of the same two
  ↪ searches: A is the source side of the minimum cut
// with the FEWEST source-side vertices, and V \ B is
  ↪ the one with the MOST.
// AN EDGE IS IN SOME MINIMUM CUT iff it is saturated
  ↪ and its endpoints are in different strongly
// connected components of the residual graph
  ↪ (contract the residual SCCs and the min cuts are
// exactly the closed sets of the resulting DAG).
// AN EDGE IS IN EVERY MINIMUM CUT iff it is
  ↪ saturated and removing its capacity lowers the
  ↪ max
// flow, i.e. iff u is in A and v is in B - checkable
  ↪ directly from these two arrays.
// COUNTING MINIMUM CUTS = counting antichains/closed
  ↪ sets in that residual-SCC DAG, which is
// #P-hard in general but easy when the DAG is a path
  ↪ or has few vertices.
```

3.7 Matching

3.7.1 HopcroftKarp

HOPCROFT-KARP - maximum bipartite matching in O(E sqrt(V)), the fast alternative to Kuhn's O(VE).

```
// Use it when n is above a few thousand; below that
  ↪ Kuhn is shorter and fast enough.
// HOPCROFT-KARP - maximum bipartite matching in O(E
  ↪ sqrt(V)), the fast alternative to Kuhn's
// O(VE). Use above a few thousand vertices; below
  ↪ that Kuhn (02) is shorter and fast enough.
// INDEXING TRAP: leftMatch has size m and is indexed
  ↪ by a RIGHT vertex; rightMatch has size n and
// is indexed by a LEFT vertex. Reading the names the
  ↪ obvious way is the default way to get this
// wrong. For the minimum vertex cover / maximum
  ↪ independent set (Konig), see 02 - KuhnAndKonig.
struct HopcroftKarp {
    vector<int> leftMatch, rightMatch, dist, cur;
    vector<vector<int> > a;
    int n, m;

    HopcroftKarp() {}

    HopcroftKarp(int n, int m) {
        this->n = n;
        this->m = m;
        a = vector<vector<int> >(n);
        leftMatch = vector<int>(m, -1);
        rightMatch = vector<int>(n, -1);
        dist = vector<int>(n, -1);
        cur = vector<int>(n, -1);
    }

    void addEdge(int x, int y) {
        a[x].push_back(y);
    }

    int bfs() {
        int found = 0;
        queue<int> q;
        for (int i = 0; i < n; i++)
            if (rightMatch[i] < 0) dist[i] = 0,
  ↪ q.push(i);
            else dist[i] = -1;

        while (!q.empty()) {
            int x = q.front();
            q.pop();
            for (int i = 0; i < int(a[x].size());
  ↪ i++) {
                int y = a[x][i];
                if (leftMatch[y] < 0) found = 1;
                else if (dist[leftMatch[y]] < 0)
```

```
                    dist[leftMatch[y]] = dist[x] + 1,
    ↪ q.push(leftMatch[y]);
            }
        }

        return found;
    }

    int dfs(int x) {
        for (; cur[x] < int(a[x].size()); cur[x]++) {
            int y = a[x][cur[x]];
            if (leftMatch[y] < 0 ||
    ↪ (dist[leftMatch[y]] == dist[x] + 1 &&
    ↪ dfs(leftMatch[y]))) {
                leftMatch[y] = x;
                rightMatch[x] = y;
                return 1;
            }
        }
        return 0;
    }

    int maxMatching() {
        int match = 0;
        while (bfs()) {
            for (int i = 0; i < n; i++) cur[i] = 0;
            for (int i = 0; i < n; i++)
                if (rightMatch[i] < 0) match +=
    ↪ dfs(i);
        }
        return match;
    }
};
```

3.7.2 KuhnAndKonig

Kuhn's bipartite matching, O(V*E) but tiny and easy to modify. Left size n, right size m.

```
// In practice fast; randomise the adjacency order +
    ↪ greedy pre-match if you need more speed.
struct Kuhn {
    int n, m;
    vector<vector<int>> adj;                        //
    ↪ left -> right
    vector<int> mt;                                 //
    ↪ mt[right] = matched left, or -1
    vector<bool> used;

    Kuhn(int n = 0, int m = 0) : n(n), m(m), adj(n),
    ↪ mt(m, -1) {}
    void add_edge(int u, int v) {
    ↪ adj[u].push_back(v); }

    bool try_(int u) {
        for (int v : adj[u]) if (!used[v]) {
            used[v] = true;
            if (mt[v] < 0 || try_(mt[v])) { mt[v] =
    ↪ u; return true; }
        }
        return false;
    }
    int maxMatching() {
        int r = 0;
        vector<int> ord(n);
        iota(all(ord), 0);
        shuffle(all(ord), rng);
        for (int u : ord) { used.assign(m, false); r
    ↪ += try_(u); }
        return r;
    }
    // KONIG: minimum VERTEX COVER = maximum
    ↪ matching. Recover it:
    //   alternate from every UNMATCHED left vertex;
    ↪ let Z = reached set.
    //   cover = (left \ Z) union (right and Z);  max
    ↪ independent set = complement of the cover.
    pair<vector<int>, vector<int>> minVertexCover() {
        vector<int> matchL(n, -1);
        for (int v = 0; v < m; v++) if (mt[v] >= 0)
    ↪ matchL[mt[v]] = v;
        vector<bool> zl(n, false), zr(m, false);
        vector<int> st;
        for (int u = 0; u < n; u++) if (matchL[u] <
    ↪ 0) zl[u] = true, st.push_back(u);
        while (!st.empty()) {
            int u = st.back(); st.pop_back();
            for (int v : adj[u]) if (!zr[v]) {
                zr[v] = true;
                if (mt[v] >= 0 && !zl[mt[v]])
    ↪ zl[mt[v]] = true, st.push_back(mt[v]);
            }
        }
        vector<int> L, R;
        for (int u = 0; u < n; u++) if (!zl[u])
    ↪ L.push_back(u);
        for (int v = 0; v < m; v++) if (zr[v])
    ↪ R.push_back(v);
        return {L, R};                              //
    ↪ |L| + |R| == maxMatching()
    }
};
/* THEOREMS you actually quote (bipartite graphs):
   Konig:    max matching = min vertex cover
   max independent set = V - min vertex cover
   MIN PATH COVER of a DAG (VERTEX-DISJOINT paths
    ↪ covering all vertices)
             = n - max matching of the split graph
    ↪ (u_out -> v_in for every edge)
             CAREFUL: this is the vertex-disjoint
    ↪ cover. To get Dilworth (FINITE poset)'s width
    ↪ (minimum
             number of CHAINS, i.e. paths allowed to
    ↪ share vertices) run it on the
             TRANSITIVE CLOSURE of the DAG first.
   Hall:     a perfect matching on the left exists
    ↪ iff |N(S)| >= |S| for every subset S of the LEFT
    ↪ part X
   Dilworth: min chain cover of a poset = longest
    ↪ antichain (build the DAG, TRANSITIVELY CLOSE
             it, then use the path cover above)
   Mirsky:   the dual - longest chain = minimum
    ↪ number of antichains partitioning the poset
   Gallai:   alpha + tau = n always;  nu + rho = n
    ↪ only if there are NO ISOLATED VERTICES
             (alpha = max independent set, tau = min
    ↪ vertex cover, nu = max matching,
              rho = min edge cover)
   Hall defect: nu(G) = |X| - max over W subset of X
    ↪ of ( |W| - |N(W)| )
   Tutte:    G has a perfect matching iff odd(G - U)
    ↪ <= |U| for every U
   Tutte-Berge: nu(G) = (1/2) * min over U of ( |V| +
    ↪ |U| - odd(G - U) )
   Menger:   min edge cut(s,t) = max edge-disjoint
    ↪ s-t paths;
             min vertex cut(s,t) = max
    ↪ internally-vertex-disjoint s-t paths (s,t
```

```
   ↪ non-adjacent) */
```

3.7.3 Hungarian

HUNGARIAN ALGORITHM - minimum-cost perfect matching on a DENSE bipartite graph, O(n^2 m).

```
// a is 1-indexed: a[1..n][1..m] with n <= m. Returns
   ↪ the cost; ans[j] = row matched to column j.
// For MAXIMUM cost, negate the matrix.
pair<ll, vector<int>> hungarian(const
   ↪ vector<vector<ll>>& a) {
    int n = a.size() - 1, m = a[0].size() - 1;
    vector<ll> u(n + 1, 0), v(m + 1, 0);
    vector<int> p(m + 1, 0), way(m + 1, 0);
    for (int i = 1; i <= n; i++) {
        p[0] = i;
        int j0 = 0;
        vector<ll> minv(m + 1, llinf);
        vector<char> used(m + 1, false);
        do {
            used[j0] = true;
            int i0 = p[j0], j1 = 0;
            ll delta = llinf;
            for (int j = 1; j <= m; j++) if
   ↪ (!used[j]) {
                ll cur = a[i0][j] - u[i0] - v[j];
                if (cur < minv[j]) minv[j] = cur,
   ↪ way[j] = j0;
                if (minv[j] < delta) delta = minv[j],
   ↪ j1 = j;
            }
            for (int j = 0; j <= m; j++)
                if (used[j]) u[p[j]] += delta, v[j]
   ↪ -= delta;
                else minv[j] -= delta;
            j0 = j1;
        } while (p[j0] != 0);
        do { int j1 = way[j0]; p[j0] = p[j1], j0 =
   ↪ j1; } while (j0);
    }
    vector<int> ans(m + 1);
    for (int j = 1; j <= m; j++) ans[j] = p[j];
    return {-v[0], ans};
   ↪ // -v[0] is the optimal total cost
}
// Use it for: assignment problems, "match every task
   ↪ to a worker minimising total time",
// minimum-cost perfect matching in a complete
   ↪ bipartite graph.
// FORBIDDEN PAIRS: use a large finite cost (1e15),
   ↪ NOT llinf - the reduced cost a[i][j]-u[i]-v[j]
// would overflow.  ans[j] == 0 means column j is
   ↪ unmatched (possible only when n < m).
// WHICH ONE: on a DENSE / complete graph this beats
   ↪ MCMF - n^2*m here versus n augmentations of
// Dijkstra over n^2 arcs (n^3 log n) there. Use MCMF
   ↪ only when the graph is genuinely SPARSE.
```

3.7.4 Blossom

MAXIMUM MATCHING IN A GENERAL (NON-BIPARTITE) GRAPH - Edmonds' blossom algorithm, O(V^3).

```
// Everything else in this folder assumes bipartite.
   ↪ Reach for this the moment the two sides are
// not fixed: "pair up people who are compatible",
   ↪ "tile a board where the colouring argument
// fails", "maximum set of disjoint edges" on an
   ↪ arbitrary graph.
// The trick it exists for: an ODD cycle can be
   ↪ traversed in two directions, so an alternating
// search can enter and leave it on the same side.
   ↪ Edmonds CONTRACTS such a cycle (a "blossom")
// into a single vertex, searches the contracted
   ↪ graph, and expands afterwards.
// Certificate: the answer equals the Tutte-Berge
   ↪ bound (see 10 - Theorems/01 - GraphTheorems).
struct Blossom {
    int n;
    vector<vector<char>> g;
   ↪          // adjacency MATRIX
    vector<int> match, p, base;
    vector<char> used, inBlossom;
    Blossom(int n) : n(n), g(n, vector<char>(n, 0)),
   ↪ match(n, -1), p(n), base(n),
                     used(n), inBlossom(n) {}
    void addEdge(int u, int v) { g[u][v] = g[v][u] =
   ↪ 1; }
    int lca(int a, int b) {
        vector<char> seen(n, 0);
        for (;;) { a = base[a], seen[a] = 1; if
   ↪ (match[a] < 0) break; a = p[match[a]]; }
        for (;;) { b = base[b]; if (seen[b]) return
   ↪ b; b = p[match[b]]; }
    }
    void mark(int v, int b, int child) {
   ↪          // walk v up to the blossom base
        while (base[v] != b) {
            inBlossom[base[v]] =
   ↪ inBlossom[base[match[v]]] = 1;
            p[v] = child, child = match[v], v =
   ↪ p[match[v]];
        }
    }
    bool augment(int root) {
   ↪          // one BFS for an augmenting path
        fill(all(used), 0), fill(all(p), -1);
        iota(all(base), 0);
        used[root] = 1;
        queue<int> q({root});
        while (!q.empty()) {
            int v = q.front(); q.pop();
            for (int to = 0; to < n; to++) {
                if (!g[v][to] || base[v] == base[to]
   ↪ || match[v] == to) continue;
                if (to == root || (match[to] >= 0 &&
   ↪ p[match[to]] >= 0)) {   // odd cycle -> contract
                    int b = lca(v, to);
                    fill(all(inBlossom), 0);
                    mark(v, b, to), mark(to, b, v);
                    for (int i = 0; i < n; i++) if
   ↪ (inBlossom[base[i]]) {
                        base[i] = b;
                        if (!used[i]) used[i] = 1,
   ↪ q.push(i);
                    }
                } else if (p[to] < 0) {
                    p[to] = v;
```

```
                    if (match[to] < 0) {
  ↪           // free vertex: flip the path
                        for (int u = to, pv, nv; u >=
  ↪ 0; u = nv)
                            pv = p[u], nv =
  ↪ match[pv], match[u] = pv, match[pv] = u;
                        return true;
                    }
                    used[match[to]] = 1,
  ↪ q.push(match[to]);
                }
            }
        }
        return false;
    }
    int solve() {
        int r = 0;
        for (int v = 0; v < n; v++)
  ↪           // greedy start: fewer BFS rounds
            if (match[v] < 0)
                for (int u = 0; u < n; u++)
                    if (g[v][u] && match[u] < 0) {
  ↪ match[v] = u, match[u] = v, r++; break; }
        for (int v = 0; v < n; v++) if (match[v] < 0)
  ↪ r += augment(v);
        return r;
  ↪           // match[v] = partner, or -1
    }
};
// PERFECT MATCHING EXISTS iff solve() * 2 == n.
  ↪ Tutte's theorem gives the certificate when it
// does not: some U with odd(G - U) > |U|.
// MINIMUM EDGE COVER (no isolated vertices) = n -
  ↪ maximum matching (Gallai).
// MAXIMUM INDEPENDENT SET is still NP-hard here -
  ↪ Konig's alpha = n - nu is BIPARTITE ONLY.
// WEIGHTED general matching needs the dual-variable
  ↪ version (~150 lines); if the graph is small
// (n <= 20) a bitmask DP over matched vertices is
  ↪ far shorter and usually enough.
// CHINESE POSTMAN (undirected): make every degree
  ↪ even by duplicating a minimum-weight perfect
// matching on the ODD-degree vertices, using
  ↪ all-pairs shortest paths as the edge weights,
  ↪ then
// take an Eulerian circuit. The MIXED version (some
  ↪ edges directed) is NP-hard.
// THE MATCHING GAME (a token moves along edges to
  ↪ unvisited vertices; stuck loses): the first
// player wins iff the start vertex is covered by
  ↪ EVERY maximum matching, i.e. iff deleting it
// decreases the matching size. With this file that
  ↪ works on a general graph, not just bipartite.
```

3.7.5 BipartiteEdgeColouring

BIPARTITE EDGE COLOURING in exactly D colours (D = maximum degree), O(m * D). 0-indexed.

```
// KONIG'S EDGE COLOURING THEOREM: a bipartite
  ↪ (multi)graph is always D-edge-colourable, and D
  ↪ is
// obviously necessary. Equivalently: the edges split
  ↪ into D perfect matchings of the D-regular
// completion, which is the "round robin timetable"
  ↪ primitive.
// Reach for it for "schedule all the games/lectures
  ↪ in the fewest rounds so nobody plays twice in
// one round", "decompose a bipartite multigraph into
  ↪ matchings", "Latin square completion",
// and for the routing step of a Clos/Benes network.
// The insertion argument: give edge (u, v) the
  ↪ smallest colour cu free at u and look at the
// smallest colour cv free at v. If they differ, the
  ↪ cu/cv alternating path starting at v cannot
// reach u (it would have to ENTER u by cu, which is
  ↪ free there), so flipping the two colours
// along it frees cu at v as well - then colour the
  ↪ edge cu.
vector<int> bipartiteEdgeColouring(int nl, int nr,
  ↪ const vector<pair<int, int>>& e) {
    int m = e.size(), D = 0;
    vector<int> dl(nl, 0), dr(nr, 0);
    for (auto [u, v] : e) D = max({D, ++dl[u],
  ↪ ++dr[v]});
    vector<vector<int>> L(nl, vector<int>(D, -1)),
  ↪ R(nr, vector<int>(D, -1));   // colour -> edge id
    vector<int> col(m, -1);
    for (int i = 0; i < m; i++) {
        auto [u, v] = e[i];
        int cu = 0, cv = 0;
        while (L[u][cu] >= 0) cu++;
        while (R[v][cv] >= 0) cv++;
        if (cu != cv) {
  ↪           // walk the cu/cv alternating path
            vector<int> path;
            for (int x = v, side = 1, c = cu;;) {
                int id = side ? R[x][c] : L[x][c];
                if (id < 0) break;
                path.push_back(id);
                x = side ? e[id].first :
  ↪ e[id].second, side ^= 1, c = (c == cu ? cv : cu);
            }
            for (int id : path)
  ↪ L[e[id].first][col[id]] =
  ↪ R[e[id].second][col[id]] = -1;
            for (int id : path) {
                col[id] = (col[id] == cu ? cv : cu);
                L[e[id].first][col[id]] =
  ↪ R[e[id].second][col[id]] = id;
            }
        }
        col[i] = cu, L[u][cu] = R[v][cu] = i;
    }
    return col;
  ↪           // col[i] in [0, D)
}
// The number of colours actually used IS the maximum
  ↪ degree - no search needed.
// EQUAL-SIZE ROUNDS: to split the edges into k
  ↪ groups whose sizes differ by at most one and
  ↪ where
// each vertex has at most ceil(deg/k) edges per
  ↪ group, split every vertex into ceil(deg/k) copies
// of degree <= k and edge-colour that graph.
// GENERAL (non-bipartite) graphs need D or D+1
  ↪ colours - Vizing (06), and deciding which is
// NP-hard. On a bipartite graph the answer is always
  ↪ D, which is why this file is separate.
// A REGULAR bipartite graph decomposes into perfect
  ↪ matchings; taking the colour classes one at a
// time is much faster than running a matching
  ↪ algorithm D times.
```

3.7.6 VizingEdgeColouring

VIZING EDGE COLOURING - colour the edges of a SIMPLE undirected graph with at most D+1 colours

```
// (D = maximum degree), constructively, in O(n * m). 0-indexed in, colours returned 0-indexed.
// VIZING'S THEOREM says D or D+1 always suffices; deciding WHICH is NP-hard, so D+1 is the answer
// you ship. (Bipartite graphs always need only D - use 05 instead, it is much shorter.)
// Reach for it for "split the matches/tasks into as few rounds as possible so no player/machine is
// used twice per round" on a general graph, and for "orient/partition the edges into D+1 matchings"
// arguments. A D+1 colouring also proves the D+1 bound constructively, which some problems ask for.
// MULTIGRAPHS are NOT covered (Shannon's bound is 3D/2 there); remove parallel edges first.
// Method: insert the edges one at a time. For a new edge (u, v) build a MAXIMAL FAN around u -
// a sequence of neighbours where each one's free colour is used by the next - and either rotate the
// fan (when some colour is free at both ends) or flip an alternating two-colour path and rotate.
// C[u][c] = the neighbour joined to u by colour c (0 = none); X[u] = smallest free colour at u.
// Vertices are shifted to 1..n inside, because 0 doubles as the "no such vertex" sentinel.
vector<int> vizing(int n, const vector<pair<int, int>>& e) {
  int D = 0;
  vector<int> deg(n, 0);
  for (auto [u, v] : e) D = max({D, ++deg[u], ++deg[v]});
  int K = D + 2;
  vector<vector<int>> C(n + 1, vector<int>(K + 1, 0)), col(n + 1, vector<int>(n + 1, 0));
  vector<int> X(n + 1, 1);
  auto update = [&](int u) { for (X[u] = 1; C[u][X[u]]; X[u]++); };
  auto color = [&](int u, int v, int c) {
    int p = col[u][v];
    col[u][v] = col[v][u] = c, C[u][c] = v, C[v][c] = u, C[u][p] = C[v][p] = 0;
    if (p) X[u] = X[v] = p; else update(u), update(v);
    return p;
  };
  auto flip = [&](int u, int c1, int c2) {
    int p = C[u][c1];
    swap(C[u][c1], C[u][c2]);
    if (p) col[u][p] = col[p][u] = c2;
    if (!C[u][c1]) X[u] = c1;
    if (!C[u][c2]) X[u] = c2;
    return p;
  };
  for (int t = 0; t < (int)e.size(); t++) {
    int u = e[t].first + 1, v0 = e[t].second + 1, v = v0, c0 = X[u], c = c0, d = 0, a;
    vector<pair<int, int>> L;
    vector<char> vis(K + 1, 0);
    while (!col[u][v0]) {
      L.push_back({v, d = X[v]});
      if (!C[v][c]) for (a = (int)L.size() - 1; a >= 0; a--) c = color(u, L[a].first, c);
      else if (!C[u][d])
        for (a = (int)L.size() - 1; a >= 0; a--) color(u, L[a].first, L[a].second);
      else if (vis[d]) break;
      else vis[d] = 1, v = C[u][d];
    }
    if (!col[u][v0]) {
      for (; v; v = flip(v, c, d), swap(c, d));
      if (C[u][c0]) {
        for (a = (int)L.size() - 2; a >= 0 && L[a].second != c; a--);
        for (; a >= 0; a--) color(u, L[a].first, L[a].second);
      } else t--;
    }
  }
  vector<int> res(e.size());
  for (int i = 0; i < (int)e.size(); i++) res[i] = col[e[i].first + 1][e[i].second + 1] - 1;
  return res;
}
// D IS ENOUGH (class 1) for: bipartite graphs, every graph with a vertex of degree D that is not
// "overfull", and every planar graph with D >= 7. D+1 is forced (class 2) for odd cycles, the
// Petersen graph, and any OVERFULL graph (m > D * floor(n/2)) - a cheap sufficient test.
// EDGE COLOURING = VERTEX COLOURING OF THE LINE GRAPH, so chi'(G) = chi(L(G)); that is usually the
// wrong direction to compute, but it is the right way to see a problem stated on the line graph.
// MEMORY is O(n^2) for the col matrix; at n ~ 2000 that is 16 MB of int - use short if it is tight.
```

3.7.7 WeightedBlossom

MAXIMUM WEIGHT MATCHING IN A GENERAL GRAPH - blossom with DUAL VARIABLES, O(n^3), O(n^2) memory.

```
// 1-INDEXED (vertices 1..n; index 0 is the "none" sentinel and 0 weight means "no edge").
// This is the weighted counterpart of 04 - Blossom. It returns the maximum TOTAL WEIGHT over all
// matchings (NOT necessarily a maximum-cardinality one) together with how many edges it used.
// Reach for it when the pairs are not bipartite and the pairing has a value: "pair up the people
// to maximise total compatibility", Chinese postman (08), "minimum cost to make every degree even",
// and any problem where Hungarian would work if only the graph had two sides.
// RECIPES
//   MINIMUM weight PERFECT matching: add a big constant B to every edge (B >= sum of |w|) and
//   negate, i.e. addEdge(u, v, B - w); a perfect matching then dominates any smaller one, and the
//   answer is B * n/2 - result. Verify solve().second * 2 == n before trusting it.
//   MAXIMUM CARDINALITY first, weight second: use w * (n + 1) + 1 per edge.
// The duals lab[] are Edmonds' y-values; a blossom b keeps lab[b] = 2 * (its dual). Every step
// either tightens an edge (slack 0), contracts a
```

```
  ↪ blossom, or expands one with zero dual.
struct WeightedBlossom {
    struct E { int u, v; ll w; };
    int n, nx, tk = 0;
    vector<vector<E>> g;
    vector<vector<int>> ff, flower;
  ↪          // ff = which sub-blossom x came from
    vector<int> match, slack, st, pa, S, vis;
    vector<ll> lab;
    deque<int> q;
    WeightedBlossom(int n) : n(n), nx(n), g(2 * n +
  ↪ 1, vector<E>(2 * n + 1)),
        ff(2 * n + 1, vector<int>(2 * n + 1, 0)),
  ↪ flower(2 * n + 1), match(2 * n + 1, 0),
        slack(2 * n + 1, 0), st(2 * n + 1, 0), pa(2 *
  ↪ n + 1, 0), S(2 * n + 1, 0),
        vis(2 * n + 1, 0), lab(2 * n + 1, 0) {
        for (int u = 1; u <= 2 * n; u++) for (int v =
  ↪ 1; v <= 2 * n; v++) g[u][v] = {u, v, 0};
    }
    void addEdge(int u, int v, ll w) { g[u][v].w =
  ↪ g[v][u].w = max(g[u][v].w, w); }
    ll d(const E& e) { return lab[e.u] + lab[e.v] -
  ↪ g[e.u][e.v].w * 2; }   // slack
    void upSlack(int u, int x) { if (!slack[x] ||
  ↪ d(g[u][x]) < d(g[slack[x]][x])) slack[x] = u; }
    void setSlack(int x) {
        slack[x] = 0;
        for (int u = 1; u <= n; u++)
            if (g[u][x].w > 0 && st[u] != x &&
  ↪ S[st[u]] == 0) upSlack(u, x);
    }
    void qPush(int x) { if (x <= n) q.push_back(x);
  ↪ else for (int y : flower[x]) qPush(y); }
    void setSt(int x, int b) { st[x] = b; if (x > n)
  ↪ for (int y : flower[x]) setSt(y, b); }
    int getPr(int b, int xr) {
        int pr = find(all(flower[b]), xr) -
  ↪ flower[b].begin();
        if (pr & 1) {
            reverse(flower[b].begin() + 1,
  ↪ flower[b].end());
            return (int)flower[b].size() - pr;
        }
        return pr;
    }
    void setMatch(int u, int v) {
        match[u] = g[u][v].v;
        if (u <= n) return;
        E e = g[u][v];
        int xr = ff[u][e.u], pr = getPr(u, xr);
        for (int i = 0; i < pr; i++)
  ↪ setMatch(flower[u][i], flower[u][i ^ 1]);
        setMatch(xr, v);
        rotate(flower[u].begin(), flower[u].begin() +
  ↪ pr, flower[u].end());
    }
    void augment(int u, int v) {
        int nv = st[match[u]];
        setMatch(u, v);
        if (!nv) return;
        setMatch(nv, st[pa[nv]]), augment(st[pa[nv]],
  ↪ nv);
    }
    int getLca(int u, int v) {
        for (++tk; u || v; swap(u, v)) {
            if (!u) continue;
            if (vis[u] == tk) return u;
            vis[u] = tk, u = st[match[u]];
            if (u) u = st[pa[u]];
        }
        return 0;
    }
    void addBlossom(int u, int lca, int v) {
  ↪          // contract the odd cycle u..lca..v
        int b = n + 1;
        while (b <= nx && st[b]) ++b;
        if (b > nx) ++nx;
        lab[b] = 0, S[b] = 0, match[b] = match[lca];
        flower[b].assign(1, lca);
        for (int x = u, y; x != lca; x = st[pa[y]])
            flower[b].push_back(x),
  ↪ flower[b].push_back(y = st[match[x]]), qPush(y);
        reverse(flower[b].begin() + 1,
  ↪ flower[b].end());
        for (int x = v, y; x != lca; x = st[pa[y]])
            flower[b].push_back(x),
  ↪ flower[b].push_back(y = st[match[x]]), qPush(y);
        setSt(b, b);
        for (int x = 1; x <= nx; x++) g[b][x].w =
  ↪ g[x][b].w = 0;
        for (int x = 1; x <= n; x++) ff[b][x] = 0;
        for (int xs : flower[b]) {
            for (int x = 1; x <= nx; x++)
                if (g[b][x].w == 0 || d(g[xs][x]) <
  ↪ d(g[b][x]))
                    g[b][x] = g[xs][x], g[x][b] =
  ↪ g[x][xs];
            for (int x = 1; x <= n; x++) if
  ↪ (ff[xs][x]) ff[b][x] = xs;
        }
        setSlack(b);
    }
    void expandBlossom(int b) {
  ↪          // only if S[b] == 1, lab[b] == 0
        for (int x : flower[b]) setSt(x, x);
        int xr = ff[b][g[b][pa[b]].u], pr = getPr(b,
  ↪ xr);
        for (int i = 0; i < pr; i += 2) {
            int xs = flower[b][i], xns = flower[b][i
  ↪ + 1];
            pa[xs] = g[xns][xs].u, S[xs] = 1, S[xns]
  ↪ = 0;
            slack[xs] = 0, setSlack(xns), qPush(xns);
        }
        S[xr] = 1, pa[xr] = pa[b];
        for (int i = pr + 1; i <
  ↪ (int)flower[b].size(); i++) S[flower[b][i]] = -1,
            setSlack(flower[b][i]);
        st[b] = 0;
    }
    bool onFound(const E& e) {
        int u = st[e.u], v = st[e.v];
        if (S[v] == -1) {
            pa[v] = e.u, S[v] = 1;
            int nu = st[match[v]];
            slack[v] = slack[nu] = 0, S[nu] = 0,
  ↪ qPush(nu);
        } else if (S[v] == 0) {
            int lca = getLca(u, v);
            if (!lca) return augment(u, v),
  ↪ augment(v, u), true;
            addBlossom(u, lca, v);
        }
        return false;
    }
```

```
    bool matching() {
        // one augmentation, or false
        fill(S.begin(), S.begin() + nx + 1, -1);
        fill(slack.begin(), slack.begin() + nx + 1, 0);
        q.clear();
        for (int x = 1; x <= nx; x++) if (st[x] == x && !match[x]) pa[x] = 0, S[x] = 0, qPush(x);
        if (q.empty()) return false;
        for (;;) {
            while (!q.empty()) {
                int u = q.front(); q.pop_front();
                if (S[st[u]] == 1) continue;
                for (int v = 1; v <= n; v++) if (g[u][v].w > 0 && st[u] != st[v]) {
                    if (d(g[u][v]) == 0) { if (onFound(g[u][v])) return true; }
                    else upSlack(u, st[v]);
                }
            }
            ll dt = llinf;
            // dual update step
            for (int b = n + 1; b <= nx; b++) if (st[b] == b && S[b] == 1) dt = min(dt, lab[b] / 2);
            for (int x = 1; x <= nx; x++) if (st[x] == x && slack[x]) {
                if (S[x] == -1) dt = min(dt, d(g[slack[x]][x]));
                else if (S[x] == 0) dt = min(dt, d(g[slack[x]][x]) / 2);
            }
            for (int u = 1; u <= n; u++) {
                if (S[st[u]] == 0) { if (lab[u] <= dt) return false; lab[u] -= dt; }
                else if (S[st[u]] == 1) lab[u] += dt;
            }
            for (int b = n + 1; b <= nx; b++) if (st[b] == b) {
                if (S[st[b]] == 0) lab[b] += dt * 2;
                else if (S[st[b]] == 1) lab[b] -= dt * 2;
            }
            q.clear();
            for (int x = 1; x <= nx; x++)
                if (st[x] == x && slack[x] && st[slack[x]] != x && d(g[slack[x]][x]) == 0)
                    if (onFound(g[slack[x]][x])) return true;
            for (int b = n + 1; b <= nx; b++)
                if (st[b] == b && S[b] == 1 && lab[b] == 0) expandBlossom(b);
        }
    }
    pair<ll, int> solve() {
        // {total weight, #matched edges}
        fill(match.begin(), match.begin() + n + 1, 0);
        nx = n;
        ll ans = 0, wmax = 0;
        int cnt = 0;
        for (int u = 0; u <= n; u++) st[u] = u, flower[u].clear();
        for (int u = 1; u <= n; u++) for (int v = 1; v <= n; v++)
            ff[u][v] = (u == v ? u : 0), wmax = max(wmax, g[u][v].w);
        for (int u = 1; u <= n; u++) lab[u] = wmax;
        while (matching()) ++cnt;
        for (int u = 1; u <= n; u++) if (match[u] && match[u] < u) ans += g[u][match[u]].w;
        return {ans, cnt};
        // match[u] = partner of u, or 0
    }
};
// WEIGHTS MUST BE NON-NEGATIVE here (an edge of weight 0 is indistinguishable from a non-edge);
// shift them if needed. Everything is integral - with real weights use long double and an eps.
// FASTER ALTERNATIVES: if n <= 20, a bitmask DP over matched vertices is 5 lines and beats this.
// If the graph is bipartite, use Hungarian (03) - O(n^3) too but with a far smaller constant.
// TUTTE MATRIX: for the UNWEIGHTED decision "is there a perfect matching", a random skew-symmetric
// matrix over a large field has full rank with high probability iff one exists - O(n^3) and 10
// lines, and rank/2 is the maximum matching size.
```

3.7.8 ChinesePostman

CHINESE POSTMAN (route inspection) - the cheapest CLOSED WALK that uses EVERY edge at least

```
// Needs: WeightedBlossom (07 - WeightedBlossom.cpp).
// once, on an undirected connected weighted graph. 0-indexed in, O(n^3) via general matching.
// THEORY. A closed walk using every edge exists with no repeats iff every degree is even (Euler).
// Otherwise you must DUPLICATE some edges, and the duplicated set must fix exactly the odd-degree
// vertices - i.e. it is a union of paths pairing them up. So: take all-pairs shortest paths, build
// the complete graph on the ODD-degree vertices (there is an even number of them, by handshake),
// and find a MINIMUM WEIGHT PERFECT MATCHING there. Answer = total edge weight + matching weight.
// Reach for it for "walk every street and come back, minimum distance", "traverse every corridor",
// and "make every degree even at minimum cost" (that last one IS the matching part alone).
ll chinesePostman(int n, const vector<array<ll, 3>>& e) {      // e = {u, v, w}, undirected
    vector<vector<ll>> d(n, vector<ll>(n, llinf));
    vector<int> deg(n, 0);
    ll total = 0;
    for (int i = 0; i < n; i++) d[i][i] = 0;
    for (auto [u, v, w] : e) {
        d[u][v] = d[v][u] = min(d[u][v], w), total += w, deg[u]++, deg[v]++;
    }
    for (int k = 0; k < n; k++) for (int i = 0; i < n; i++) for (int j = 0; j < n; j++)
        if (d[i][k] < llinf && d[k][j] < llinf) d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
    vector<int> odd;
    for (int i = 0; i < n; i++) if (deg[i] & 1) odd.push_back(i);
    int k = odd.size();
    if (!k) return total;          // already Eulerian
    ll mx = 0;
    // shift so that MIN becomes MAX and
    for (int i = 0; i < k; i++) for (int j = 0; j < k; j++)    // a PERFECT matching always wins
        if (d[odd[i]][odd[j]] < llinf) mx = max(mx, d[odd[i]][odd[j]]);
    ll B = mx * k + 1;
```

```
    WeightedBlossom M(k);
    for (int i = 0; i < k; i++) for (int j = i + 1; j
  ↪ < k; j++)
        if (d[odd[i]][odd[j]] < llinf) M.addEdge(i +
  ↪ 1, j + 1, B - d[odd[i]][odd[j]]);
    auto [val, cnt] = M.solve();
    if (cnt * 2 != k) return -1;
  ↪          // odd vertices cannot be paired
    return total + (ll)cnt * B - val;
}
// RECOVERING THE ROUTE: duplicate every edge on the
  ↪ shortest path between each matched pair, then
// run an Eulerian circuit (05 - Eulerian Path) on
  ↪ the resulting even-degree multigraph.
// DIRECTED CHINESE POSTMAN is a MIN COST FLOW, not a
  ↪ matching: give each vertex a supply of
// (outdeg - indeg), send flow from the surplus
  ↪ vertices to the deficit ones along the original
// arcs with cost = weight and infinite capacity, and
  ↪ add that cost to the total weight.
// MIXED (some edges directed, some not) is NP-hard.
// RURAL POSTMAN (only a SUBSET of edges must be
  ↪ covered) is NP-hard as well - it contains TSP.
// OPEN version ("start anywhere, end anywhere"):
  ↪ leave two odd vertices unmatched, i.e. run a
// maximum-weight (not perfect) matching, which is
  ↪ what solve() gives you without the count check.
```

3.8 Spanning Trees

3.8.1 MST

MINIMUM SPANNING TREE. Kruskal O(m log m) - use on sparse graphs / when you also want the

```
// Needs: DSU (02 - Data Structures/05 - DSU/01).
// KRUSKAL RECONSTRUCTION TREE. Prim O(n^2) - use on
  ↪ dense or IMPLICIT complete graphs.
struct Edge { int u, v; ll w; bool operator<(const
  ↪ Edge& o) const { return w < o.w; } };

ll kruskal(int n, vector<Edge> e, vector<Edge>* used
  ↪ = nullptr) {
    sort(all(e));
    DSU d(n);
    ll tot = 0;
    for (auto& x : e) if (d.join(x.u, x.v)) { tot +=
  ↪ x.w; if (used) used->push_back(x); }
    return tot;                                  //
  ↪ check d.size(0) == n for connectivity
}
// Dense / implicit graphs: cost(i,j) given by a
  ↪ function. O(n^2), no edge list needed.
template <class F>
ll prim(int n, F cost) {
    vector<ll> best(n, llinf);
    vector<bool> in(n, false);
    best[0] = 0;
    ll tot = 0;
    for (int it = 0; it < n; it++) {
        int u = -1;
        for (int i = 0; i < n; i++) if (!in[i] && (u
  ↪ < 0 || best[i] < best[u])) u = i;
        in[u] = true, tot += best[u];
        for (int v = 0; v < n; v++) if (!in[v])
  ↪ best[v] = min(best[v], cost(u, v));
    }
    return tot;
}
// KRUSKAL RECONSTRUCTION TREE: binary tree with 2n-1
  ↪ nodes; internal node = merging edge weight.
// Then min possible MAX edge on a path u->v =
  ↪ val[lca(u,v)], and "all vertices reachable using
// edges of weight <= w" is exactly a subtree. Turns
  ↪ "binary search the threshold + DSU" into O(1).
struct KRT {
    int n, cnt;
    vector<ll> val;
    vector<array<int, 2>> ch;
    KRT(int n, vector<Edge> e) : n(n), cnt(n), val(2
  ↪ * n, 0), ch(2 * n, {-1, -1}) {
        sort(all(e));
        DSU d(2 * n);
        vector<int> top(2 * n);
        iota(all(top), 0);
        for (auto& x : e) {
            int a = top[d.find(x.u)], b =
  ↪ top[d.find(x.v)];
            if (a == b) continue;
            val[cnt] = x.w, ch[cnt] = {a, b};
            d.join(x.u, x.v);
            top[d.find(x.u)] = cnt++;
        }
    }                                            //
  ↪ root = cnt-1 (if connected); then build LCA on it
};
// SECOND-BEST MST: build the MST, then for each
  ↪ non-tree edge (u,v,w) the answer candidate is
// mst - maxEdgeOnPath(u,v) + w. Get maxEdgeOnPath
  ↪ with binary lifting or the KRT.
```

3.8.2 DirectedMSTandExtras

DIRECTED MST (Chu-Liu / Edmonds) - minimum spanning ARBORESCENCE rooted at r, O(V*E).

```
// Needs: struct Edge {int u,v; ll w;} and DSU (both
  ↪ from 01 - MST.cpp).
// Every non-root vertex takes its cheapest incoming
  ↪ edge; if that creates a cycle, contract it,
// subtract the chosen weights, and repeat. Returns
  ↪ -1 if some vertex is unreachable from r.
ll directedMST(int n, int r, vector<Edge> e) {
  ↪  // Edge = {u, v, w} meaning u -> v
    ll total = 0;
    vector<int> id(n), par(n);
    while (true) {
        vector<ll> best(n, llinf);
        vector<int> from(n, -1);
        for (auto& x : e) if (x.u != x.v && x.w <
  ↪ best[x.v]) best[x.v] = x.w, from[x.v] = x.u;
        best[r] = 0;
        for (int i = 0; i < n; i++) if (i != r &&
  ↪ from[i] < 0) return -1;   // unreachable
        int cnt = 0;
        fill(all(id), -1), fill(all(par), -1);
        for (int i = 0; i < n; i++) {
            if (i != r) total += best[i];
            int v = i;
            while (v != r && par[v] == -1 && id[v] ==
  ↪ -1) par[v] = i, v = from[v];
            if (v != r && id[v] == -1 && par[v] == i)
  ↪ {  // found a new cycle
                for (int u = from[v]; u != v; u =
  ↪ from[u]) id[u] = cnt;
                id[v] = cnt++;
            }
        }
        if (!cnt) break;
  ↪   // no cycle -> done
```

```
        for (int i = 0; i < n; i++) if (id[i] == -1)
    ↪ id[i] = cnt++;
        for (auto& x : e) {
            int u = x.u, v = x.v;
            x.u = id[u], x.v = id[v];
            if (x.u != x.v) x.w -= best[v];
    ↪    // reduced cost inside the contraction
        }
        n = cnt, r = id[r];
    }
    return total;
}

// SECOND-BEST MST, O(E log V + E * log V). The
    ↪ second-best tree differs from the MST by exactly
// ONE edge swap. For each non-tree edge (u,v,w) the
    ↪ candidate is mst - maxEdgeOnPath(u,v) + w.
// You must track BOTH the maximum and the strict
    ↪ second maximum on the path, because if the
// maximum equals w the swap is not a real change -
    ↪ use the second maximum instead.
// With binary lifting: up[k][v], mx1[k][v],
    ↪ mx2[k][v] merged pairwise while lifting.
// (The Kruskal Reconstruction Tree in 01 - MST.cpp
    ↪ gives maxEdgeOnPath directly as val[lca].)

// BORUVKA - MST in O(E log V) by phases; each phase
    ↪ every component picks its cheapest outgoing
// edge and they are all merged at once. Use it when
    ↪ the edge set is IMPLICIT:
//   XOR-MST      : cheapest cross edge per component
    ↪ via a binary trie
//   Manhattan MST: cheapest cross edge per octant
    ↪ (see Transforms.cpp)
ll boruvka(int n, const vector<Edge>& e) {
    DSU d(n);
    ll total = 0;
    int comps = n;
    while (comps > 1) {
        vector<int> best(n, -1);
        for (int i = 0; i < (int)e.size(); i++) {
            int a = d.find(e[i].u), b =
    ↪ d.find(e[i].v);
            if (a == b) continue;
            if (best[a] < 0 || e[i].w < e[best[a]].w)
    ↪ best[a] = i;
            if (best[b] < 0 || e[i].w < e[best[b]].w)
    ↪ best[b] = i;
        }
        bool any = false;
        for (int i = 0; i < n; i++) if (best[i] >= 0)
    ↪ {
            auto& x = e[best[i]];
            if (d.join(x.u, x.v)) total += x.w,
    ↪ comps--, any = true;
        }
        if (!any) break;
    ↪    // disconnected
    }
    return comps == 1 ? total : -1;
}
```

3.8.3 SecondBestMST

SECOND-BEST SPANNING TREE - cheapest spanning tree whose weight is STRICTLY greater than the MST.

```
// Needs: Edge, kruskal (01 - MST.cpp).
// It differs from the MST in exactly one edge: add a
    ↪ non-tree edge (u,v,w), drop the heaviest edge
// on the tree path u->v whose weight is < w. By the
    ↪ cycle property every edge on that path has
// weight <= w, so "heaviest edge < w" is either the
    ↪ path maximum (if it is < w) or the second
// LARGEST DISTINCT value on the path. Binary lifting
    ↪ carries both. O(m log n).
ll secondBestMST(int n, const vector<Edge>& e) {
    vector<Edge> used;
    ll mst = kruskal(n, e, &used);
    if ((int)used.size() != n - 1) return -1;
    ↪          // disconnected
    vector<vector<pair<int, ll>>> g(n);
    for (auto& x : used) g[x.u].push_back({x.v,
    ↪ x.w}), g[x.v].push_back({x.u, x.w});
    typedef array<ll, 2> T;
    ↪          // {largest, 2nd largest DISTINCT}
    const T NONE = {LLONG_MIN, LLONG_MIN};
    auto join = [&](T a, T b) {
        array<ll, 4> v = {a[0], a[1], b[0], b[1]};
        sort(v.rbegin(), v.rend());
        T r = {v[0], LLONG_MIN};
        for (ll x : v) if (x < v[0]) { r[1] = x;
    ↪ break; }
        return r;
    };
    int L = 1;
    while ((1 << L) < n) L++;
    vector<vector<int>> up(L + 1, vector<int>(n, 0));
    vector<vector<T>> mx(L + 1, vector<T>(n, NONE));
    vector<int> dep(n, 0), st{0};
    vector<char> vis(n, 0);
    vis[0] = 1;
    while (!st.empty()) {
    ↪          // iterative DFS from 0
        int u = st.back(); st.pop_back();
        for (auto [v, w] : g[u]) if (!vis[v])
            vis[v] = 1, dep[v] = dep[u] + 1, up[0][v]
    ↪ = u, mx[0][v] = {w, LLONG_MIN}, st.push_back(v);
    }
    for (int k = 1; k <= L; k++)
        for (int v = 0; v < n; v++) {
            int m = up[k - 1][v];
            up[k][v] = up[k - 1][m], mx[k][v] =
    ↪ join(mx[k - 1][v], mx[k - 1][m]);
        }
    auto path = [&](int u, int v) {
    ↪          // top-2 distinct weights on u->v
        T r = NONE;
        if (dep[u] < dep[v]) swap(u, v);
        for (int k = L; k >= 0; k--) if
    ↪ (dep[up[k][u]] >= dep[v]) r = join(r, mx[k][u]),
    ↪ u = up[k][u];
        if (u == v) return r;
        for (int k = L; k >= 0; k--) if (up[k][u] !=
    ↪ up[k][v])
            r = join(join(r, mx[k][u]), mx[k][v]), u
    ↪ = up[k][u], v = up[k][v];
        return join(join(r, mx[0][u]), mx[0][v]);
    };
    ll best = LLONG_MAX;
    for (auto& x : e) {
    ↪          // tree edges give p = {w, MIN} and
```

```
        T p = path(x.u, x.v);
        // fall through to the MIN branch
        ll rem = p[0] < x.w ? p[0] : p[1];
        if (rem != LLONG_MIN) best = min(best, mst - rem + x.w);
    }
    return best == LLONG_MAX ? -1 : best;
    // -1: no second tree exists
}
// K-TH BEST SPANNING TREE: repeat the swap argument with a priority queue over (tree, forced-in / forced-out edge sets) - Gabow's algorithm, O(k m log n). Rare; the second-best case is the one that actually shows up.
// MINIMUM BOTTLENECK SPANNING TREE = any MST (minimising the maximum edge is implied by Kruskal).
// MST OF A GRAPH PLUS ONE NEW EDGE: add it, find the cycle, drop the cycle maximum. O(n).
// MINIMUM SPANNING TREE WITH DEGREE CONSTRAINT at one vertex r: build the MST of G - r, then add the cheapest edge from r into each component, then improve by swaps while degree < k.
```

3.8.4 MinDiameterSpanningTree

MINIMUM DIAMETER SPANNING TREE - the spanning tree whose longest path is as SHORT as possible.

```
// O(V^3) (Floyd-Warshall plus one sweep per edge). Undirected, non-negative weights, 0-indexed.
// This is NOT the MST: the MST minimises total weight and can have a terrible diameter.
// KEY FACT. An MDST is always a SHORTEST PATH TREE from the ABSOLUTE 1-CENTRE - a point that may sit in the MIDDLE OF AN EDGE, not only at a vertex. So: for every edge (u, v) of weight w, slide
// a point c at distance x along it and minimise its eccentricity ecc(x) = max over i of min(x + d[u][i], w - x + d[v][i]). Each i contributes a "tent" function of x, the upper envelope
// is piecewise linear, and it is enough to test the breakpoints - which is the sweep below, after sorting the vertices by d[u][.] descending. The answer is the minimum eccentricity, doubled.
// Reach for it for "build a network minimising the worst-case delay between any two nodes", "connect all cities so the farthest pair is as close as possible", and broadcast-tree problems.
// ALL WEIGHTS ARE DOUBLED INTERNALLY so the centre's position stays an integer; the returned diameter is already in the ORIGINAL units (it is twice the eccentricity, which cancels out).
struct MDST {
    ll diameter = llinf;
    int edge = -1;
    // the edge holding the centre
    ll offset = 0;
    // 2 * distance from e[edge][0]
    vector<ll> distFromCentre;
    // 2 * dist(centre, i), after solve
    ll solve(int n, const vector<array<ll, 3>>& e) {
        // e = {u, v, w}
        if (n == 1) return diameter = 0;
        vector<vector<ll>> d(n, vector<ll>(n, llinf));
        for (int i = 0; i < n; i++) d[i][i] = 0;
        for (auto [u, v, w] : e) d[u][v] = min(d[u][v], 2 * w), d[v][u] = min(d[v][u], 2 * w);
        for (int k = 0; k < n; k++) for (int i = 0; i < n; i++) for (int j = 0; j < n; j++)
            if (d[i][k] < llinf && d[k][j] < llinf) d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
        vector<int> id(n);
        iota(all(id), 0);
        for (int u = 0; u < n; u++) {
            sort(all(id), [&](int a, int b) { return d[u][a] > d[u][b]; });
            for (int k = 0; k < (int)e.size(); k++) {
                int a = e[k][0], b = e[k][1];
                if (a != u && b != u) continue;
                int v = a ^ b ^ u;
                ll W = 2 * e[k][2], opt = 0;
                int last = id[0];
                ll val = d[u][last];
                // centre at u itself
                for (int t = 1; t < n; t++) {
                // walk the envelope breakpoints
                    int i = id[t];
                    if (d[v][i] <= d[v][last]) continue;
                    ll cx = (W - d[u][i] + d[v][last]) / 2, cy = (W + d[u][i] + d[v][last]) / 2;
                    if (val > cy) val = cy, opt = cx;
                    last = i;
                }
                if (val > d[v][last]) val = d[v][last], opt = W;   // centre at v itself
                if (val < diameter) diameter = val, edge = k, offset = opt;
            }
        }
        distFromCentre.assign(n, llinf);
        // Dijkstra from the virtual centre
        int a = e[edge][0], b = e[edge][1];
        priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<>> pq;
        distFromCentre[a] = offset, distFromCentre[b] = 2 * e[edge][2] - offset;
        pq.push({distFromCentre[a], a}), pq.push({distFromCentre[b], b});
        vector<vector<pair<int, ll>>> g(n);
        for (auto [x, y, w] : e) g[x].push_back({(int)y, 2 * w}), g[y].push_back({(int)x, 2 * w});
        while (!pq.empty()) {
            auto [dd, x] = pq.top(); pq.pop();
            if (dd > distFromCentre[x]) continue;
            for (auto [y, w] : g[x]) if (dd + w < distFromCentre[y])
                distFromCentre[y] = dd + w, pq.push({distFromCentre[y], y});
        }
        return diameter;
    }
};
// RECOVERING THE TREE: take the shortest path tree of distFromCentre (each vertex keeps one incoming edge that is tight), plus the two halves of the centre edge. If offset is 0 or 2*w the centre is a vertex and the tree is just that vertex's shortest path tree.
// UNWEIGHTED graphs: the answer is the shortest path tree from the middle of the graph's own diameter path, and BFS from every vertex (O(VE))
```

```
   ↪ is enough - do not bother with this file.
// MINIMUM RADIUS spanning tree is the same
   ↪ computation without the final doubling; the
   ↪ absolute
// 1-centre is the object both problems share.
// MINIMUM DIAMETER with a bound on the total weight,
   ↪ or "diameter <= k with fewest edges", is
// NP-hard - the polynomial case is exactly the
   ↪ unconstrained one above.
```

3.9 Exact Exponential

3.9.1 CliqueColouringMIS

EXACT ALGORITHMS FOR NP-HARD GRAPH PROBLEMS ON SMALL n. The constraint IS the hint:

```
//   n <= 20-24  -> plain subset DP, O(2^n * n)
//   n <= 40-50  -> meet in the middle, or
   ↪ Bron-Kerbosch (fast in practice on sparse graphs)
//   n <= 100+   -> only if the graph is
   ↪ bipartite/perfect/a tree - then it is polynomial
   ↪ (Konig)
// adj[] is an ADJACENCY BITMASK: bit j of adj[i] is
   ↪ set iff i-j is an edge. No self loops.

// --- MAXIMUM CLIQUE, Bron-Kerbosch with PIVOTING.
   ↪ Worst case O(3^(n/3)), very fast in practice. ---
// R = clique built so far, P = candidates, X =
   ↪ already-explored vertices (avoids duplicates).
// The pivot u in P|X is chosen to maximise |P &
   ↪ adj[u]|; only P \ adj[u] needs branching, because
// any maximal clique either omits u or contains a
   ↪ non-neighbour of u.
int bestClique;
unsigned bestSet;
void bk(unsigned R, unsigned P, unsigned X, const
   ↪ vector<unsigned>& adj) {
    if (!P && !X) {
        if (__builtin_popcount(R) > bestClique)
   ↪ bestClique = __builtin_popcount(R), bestSet = R;
        return;
    }
    int u = __builtin_ctz(P | X), best = -1;
    for (unsigned t = P | X; t; t &= t - 1) {
        int v = __builtin_ctz(t), c =
   ↪ __builtin_popcount(P & adj[v]);
        if (c > best) best = c, u = v;
    }
    for (unsigned cand = P & ~adj[u]; cand; cand &=
   ↪ cand - 1) {
        int v = __builtin_ctz(cand);
        bk(R | 1u << v, P & adj[v], X & adj[v], adj);
        P &= ~(1u << v), X |= 1u << v;
    }
}
int maxClique(const vector<unsigned>& adj) {
   ↪          // n <= 32
    bestClique = 0, bestSet = 0;
    bk(0, (1u << adj.size()) - 1, 0, adj);
    return bestClique;
   ↪          // the clique itself is in bestSet
}
// MAXIMUM INDEPENDENT SET = maximum clique of the
   ↪ COMPLEMENT graph. Just flip adj and mask off i.
// MINIMUM VERTEX COVER = n - MIS. On BIPARTITE
   ↪ graphs both are polynomial: Konig says
//   min vertex cover = maximum matching, and MIS = n
   ↪ - matching (see 07 - Matching).
```

```
// --- CHROMATIC NUMBER, O(2^n * n) by
   ↪ inclusion-exclusion
   ↪ (Bjorklund-Husfeldt-Koivisto). ---
// Let ind[S] = 1 if S is independent. The number of
   ↪ ways to cover V with k ORDERED independent
// sets is  sum_S (-1)^(n-|S|) * I(S)^k,  where I(S)
   ↪ = #independent subsets of S. The chromatic
// number is the smallest k making that non-zero.
   ↪ Compute I over subsets with one SOS pass.
int chromatic(const vector<unsigned>& adj) {
   ↪          // n <= 20 with this modulus
    int n = adj.size(), N = 1 << n;
    if (!n) return 0;
    const ll M = (1LL << 61) - 1;
   ↪          // any large prime works
    vector<ll> I(N), pw(N, 1);
    I[0] = 1;
    for (int S = 1; S < N; S++) {
   ↪          // I[S] = #independent subsets of S
        int v = __builtin_ctz(S), rest = S ^ 1 << v;
   ↪          // (v in it, or not)
        I[S] = (I[rest] + I[rest & ~adj[v]]) % M;
    }
    for (int k = 1; k <= n; k++) {
        ll tot = 0;
        for (int S = 0; S < N; S++) {
            pw[S] = (__int128)pw[S] * I[S] % M;
   ↪          // I(S)^k
            ll t = (n - __builtin_popcount(S)) & 1 ?
   ↪ M - pw[S] : pw[S];
            tot = (tot + t) % M;
        }
        if (tot % M) return k;
    }
    return n;
}
// The count is done MODULO a prime, so a zero could
   ↪ in principle be a false negative; with a
// 61-bit modulus that never happens on contest
   ↪ inputs. Check bipartiteness first - k <= 2 is
   ↪ O(n+m).

// --- MINIMUM STEINER TREE (connect a set of k
   ↪ terminals in a weighted graph), O(3^k n + 2^k
   ↪ n^2).
// dp[S][v] = cheapest tree containing terminal set S
   ↪ and touching vertex v.
//   merge:  dp[S][v] = min over submasks T of
   ↪ dp[T][v] + dp[S\T][v]
//   grow:   dp[S][v] = min over edges (u,v) of
   ↪ dp[S][u] + w         (run Dijkstra per S)
ll steinerTree(int n, const vector<vector<pair<int,
   ↪ ll>>>& g, const vector<int>& term) {
    int k = term.size();
    if (!k) return 0;
    vector<vector<ll>> dp(1 << k, vector<ll>(n,
   ↪ llinf));
    for (int i = 0; i < k; i++) dp[1 << i][term[i]] =
   ↪ 0;
    for (int S = 1; S < 1 << k; S++) {
        for (int T = (S - 1) & S; T; T = (T - 1) & S)
            for (int v = 0; v < n; v++)
                if (dp[T][v] < llinf && dp[S ^ T][v]
   ↪ < llinf)
                    dp[S][v] = min(dp[S][v], dp[T][v]
   ↪ + dp[S ^ T][v]);
        priority_queue<pair<ll, int>, vector<pair<ll,
   ↪ int>>, greater<>> pq;
```

```
        for (int v = 0; v < n; v++) if (dp[S][v] <
    ↪ llinf) pq.push({dp[S][v], v});
        while (!pq.empty()) {
    ↪           // Dijkstra to relax the tree outward
            auto [d, u] = pq.top(); pq.pop();
            if (d > dp[S][u]) continue;
            for (auto [v, w] : g[u]) if (dp[S][u] + w
    ↪ < dp[S][v])
                dp[S][v] = dp[S][u] + w,
    ↪ pq.push({dp[S][v], v});
        }
    }
    return *min_element(all(dp[(1 << k) - 1]));
}
// STEINER TREE IN A GRID / with k up to ~10 is the
    ↪ usual contest size. If k == 2 it is a shortest
// path; if k == n it is the MST; the DP interpolates
    ↪ between them.
// RELATED EXACT-EXPONENTIAL TOOLS:
//   * HAMILTONIAN PATH / TSP: dp[mask][v], O(2^n
    ↪ n^2). See 08 - DP/02 - BitmaskDP.
//   * SET COVER: dp over covered elements, O(2^n *
    ↪ #sets), or IE like the chromatic number.
//   * GRAPH COLOURING with k fixed: try to 3-colour
    ↪ by 2-SAT? No - 3-colouring is NP-hard; use
//     the IE formula above, or branch on the
    ↪ highest-degree vertex with memoisation.
//   * MAXIMUM INDEPENDENT SET for n <= 40-50: meet
    ↪ in the middle - split the vertices in half,
//     enumerate independent subsets of the left
    ↪ half, and for each store the best independent
//     subset of the right half compatible with its
    ↪ neighbourhood (SOS over the right mask).
//   * COUNTING INDEPENDENT SETS / colourings on a
    ↪ graph of small TREEWIDTH: DP over a tree
//     decomposition, O(2^tw * n) - usually a grid
    ↪ problem in disguise (see broken-profile DP).
```

3.9.2 MaxCliqueBitset

MAXIMUM CLIQUE FOR n UP TO ~150 (about 1 s at n = 155, much faster on sparse graphs). 0-indexed.

```
// 01 - CliqueColouringMIS is capped at 32 vertices
    ↪ by its unsigned mask; this is the same problem
// with a bitset adjacency matrix and a far stronger
    ↪ pruning rule, so it is the one to paste when
// n is in the 40..200 range. MAXIMUM INDEPENDENT SET
    ↪ = run it on the COMPLEMENT graph.
// The bound: greedily colour the candidate set
    ↪ (vertices of the same colour are pairwise
// non-adjacent, so at most one of them can join the
    ↪ clique). If |current| + #colours <= |best|,
// prune. The extra trick is the ADAPTIVE re-sorting
    ↪ - it recomputes the degree order in a
// sub-branch only when that branch has been visited
    ↪ often enough to pay for it (the 'limit').
// PRECONDITION: symmetric matrix, NO self edges. Set
    ↪ MV to the smallest power-of-two-ish bound
// above n; a bitset<MV> is scanned in MV/64 words,
    ↪ so an over-large MV costs real time.
const int MV = 160;
    ↪          // <-- adjust to your n
struct MaxClique {
    typedef vector<bitset<MV>> Graph;
    struct Vertex { int i, d = 0; };
    double limit = 0.025, pk = 0;
    Graph e;
    vector<Vertex> V;
    vector<vector<int>> C;
    vector<int> qmax, q, S, old;
    MaxClique(Graph g) : e(g), C(g.size() + 1),
    ↪ S(C.size()), old(S) {
        for (int i = 0; i < (int)e.size(); i++)
    ↪ V.push_back({i});
    }
    void init(vector<Vertex>& r) {
    ↪          // order by degree inside r
        for (auto& v : r) v.d = 0;
        for (auto& v : r) for (auto& j : r) v.d +=
    ↪ e[v.i][j.i];
        sort(all(r), [](Vertex a, Vertex b) { return
    ↪ a.d > b.d; });
        int mxD = r[0].d;
        for (int i = 0; i < (int)r.size(); i++)
    ↪ r[i].d = min(i, mxD) + 1;
    }
    void expand(vector<Vertex>& R, int lev = 1) {
        S[lev] += S[lev - 1] - old[lev], old[lev] =
    ↪ S[lev - 1];
        while (R.size()) {
            if ((int)q.size() + R.back().d <=
    ↪ (int)qmax.size()) return;     // colour bound
            q.push_back(R.back().i);
            vector<Vertex> T;
            for (auto v : R) if (e[R.back().i][v.i])
    ↪ T.push_back({v.i});
            if (T.size()) {
                if (S[lev]++ / ++pk < limit) init(T);
                int j = 0, mxk = 1, mnk =
    ↪ max((int)qmax.size() - (int)q.size() + 1, 1);
                C[1].clear(), C[2].clear();
                for (auto v : T) {
    ↪           // greedy colouring
                    int k = 1;
                    auto f = [&](int i) { return
    ↪ e[v.i][i]; };
                    while (any_of(all(C[k]), f)) k++;
                    if (k > mxk) mxk = k, C[mxk +
    ↪ 1].clear();
                    if (k < mnk) T[j++].i = v.i;
                    C[k].push_back(v.i);
                }
                if (j > 0) T[j - 1].d = 0;
                for (int k = mnk; k <= mxk; k++) for
    ↪ (int i : C[k]) T[j].i = i, T[j++].d = k;
                expand(T, lev + 1);
            } else if (q.size() > qmax.size()) qmax =
    ↪ q;
            q.pop_back(), R.pop_back();
        }
    }
    vector<int> solve() { init(V), expand(V); return
    ↪ qmax; }   // the clique itself
};
// MAXIMUM INDEPENDENT SET: build h with h[i][j] = (i
    ↪ != j && !g[i][j]) and run this on h.
// MINIMUM VERTEX COVER = n - MIS.  MINIMUM CLIQUE
    ↪ COVER of G = chromatic number of the complement.
// WEIGHTED maximum clique needs a different bound
    ↪ (sum of the per-colour maxima); the unweighted
// colour count above is not valid there.
// MAXIMUM CLIQUE ON A PERFECT GRAPH (bipartite,
    ↪ chordal, interval, comparability) is POLYNOMIAL -
// check the structure before reaching for
    ↪ exponential search.
// COUNTING all maximal cliques instead:
    ↪ Bron-Kerbosch with pivoting (01), O(3^(n/3)) of
```

```
    ↪ them.
```

3.9.3 ChromaticPolynomial

CHROMATIC POLYNOMIAL P(x) = the number of proper colourings with at most x colours, for n <= 20.

```
// Needs: Mint (06 - Math/01 - Mint.cpp).
// O(2^n * n^2) to get all of P(0..n), then O(n^2)
   ↪ per evaluation elsewhere. 0-indexed; adj[] is an
// adjacency BITMASK (bit j of adj[i] set iff i-j is
   ↪ an edge), no self loops.
// 01 - CliqueColouringMIS finds the chromatic NUMBER
   ↪ with the same inclusion-exclusion; this gets
// the whole polynomial, which is what you need for
   ↪ "count the proper k-colourings" (k possibly
// huge), for "count acyclic orientations", and for
   ↪ deletion-contraction arguments.
// THE FORMULA. With I(S) = the number of independent
   ↪ subsets of S (including the empty one),
//     P(k) = sum over S of (-1)^(n-|S|) * I(S)^k.
// P has degree n, so evaluating it at 0..n pins it
   ↪ down; Lagrange interpolation gives any other
// point. All arithmetic is mod MOD, so a zero result
   ↪ is "almost surely" a genuine zero.
struct ChromaticPolynomial {
    int n;
    vector<Mint> p;
    ↪          // p[k] = P(k) for k = 0..n
    ChromaticPolynomial(const vector<int>& adj) :
    ↪ n(adj.size()), p(n + 1, 0) {
        int F = 1 << n;
        vector<Mint> I(F), pw(F, 1);
        I[0] = 1;
        for (int S = 1; S < F; S++) {
        ↪          // v is in the subset, or it is not
            int v = __builtin_ctz(S), rest = S ^ 1 <<
        ↪ v;
            I[S] = I[rest] + I[rest & ~adj[v]];
        }
        for (int k = 1; k <= n; k++) {
            Mint tot = 0;
            for (int S = 0; S < F; S++) {
                pw[S] *= I[S];
            ↪          // pw[S] = I(S)^k
                tot += ((n - __builtin_popcount(S)) &
            ↪ 1) ? Mint(0) - pw[S] : pw[S];
            }
            p[k] = tot;
        }
    }
    Mint at(ll x) const {
    ↪          // P(x) for any x, by interpolation
        if (x >= 0 && x <= n) return p[x];
        Mint ans = 0;
        for (int i = 0; i <= n; i++) {
            Mint num = p[i];
            for (int j = 0; j <= n; j++) if (j != i)
        ↪ num *= (Mint(x) - j) / (Mint(i) - j);
            ans += num;
        }
        return ans;
    }
    int chromaticNumber() const {
        for (int k = 1; k <= n; k++) if (!(p[k] ==
    ↪ Mint(0))) return k;
        return n;
    }
};
// KNOWN VALUES, useful as checks: a tree on n
   ↪ vertices has P(x) = x (x-1)^(n-1); the cycle C_n
   ↪ has
// (x-1)^n + (-1)^n (x-1); K_n has x(x-1)...(x-n+1);
   ↪ a forest of k components on n vertices in total
// has P(x) = x^k (x-1)^(n-k) only when every
   ↪ component is a star... just use the tree formula
   ↪ per
// component and multiply, since P is MULTIPLICATIVE
   ↪ over connected components.
// ACYCLIC ORIENTATIONS of G = |P(-1)| = (-1)^n P(-1)
   ↪    (Stanley's theorem). That also counts the
// DAGs you can make by directing the edges of G,
   ↪ which is how "count the DAGs whose underlying
// graph is G" problems are solved.
// DELETION-CONTRACTION: P_G(x) = P_{G-e}(x) -
   ↪ P_{G/e}(x). Exponential in general, but it is the
// right tool when the graph is nearly a tree or you
   ↪ can memoise on small components.
// LABELLED DAGs ON n VERTICES (a different question
   ↪ - no fixed graph): with a_n the count,
// a_n = sum_{k=1..n} (-1)^(k+1) C(n,k) 2^(k(n-k))
   ↪ a_{n-k}, by inclusion-exclusion on the sources.
```

3.9.4 Hafnian

HAFNIAN of a symmetric matrix = the NUMBER OF PERFECT MATCHINGS of a general weighted graph

```
// Needs: Mint (06 - Math/01 - Mint.cpp).
// (the sum over perfect matchings of the product of
   ↪ the chosen entries). O(2^(n/2) * n^3), which
// is about 2 s for a 38x38 matrix. 0-indexed, n must
   ↪ be EVEN, the diagonal is ignored.
// The hafnian is to the permanent what the general
   ↪ graph is to the bipartite one:
//   permanent(A) = #perfect matchings of a BIPARTITE
   ↪ graph  -> Ryser, O(2^n n)
//   hafnian(A)   = #perfect matchings of a GENERAL
   ↪ graph    -> this file
//   determinant  = the SIGNED version, polynomial,
   ↪ and counts nothing on its own
// Reach for it for "count the perfect matchings /
   ↪ domino tilings / pairings" when the graph is not
// bipartite and n is around 30-40. If the graph IS
   ↪ bipartite use the permanent (2x more vertices
// here would be hopeless); if you only need
   ↪ EXISTENCE, use Blossom or a Tutte matrix.
// METHOD (Bjorklund): keep, for every pair (i, j) of
   ↪ the first m vertices, a POLYNOMIAL in a
// formal variable that counts partial matchings by
   ↪ size. Removing the last two vertices splits
// into "they are matched to each other" and "they
   ↪ are not", and the recursion has depth n/2 with
// two branches that share all the work below them.
struct Hafnian {
    int h;
    void add(vector<Mint>& ans, const vector<Mint>&
    ↪ a, const vector<Mint>& b) {
        for (int i = 0; i < h; i++)
            for (int j = 0; j < h - 1 - i; j++) ans[i
        ↪ + j + 1] += a[i] * b[j];
    }
    vector<Mint> rec(const
    ↪ vector<vector<vector<Mint>>>& v) {
        vector<Mint> ans(h, 0);
        if (v.empty()) { ans[0] = 1; return ans; }
        int m = (int)v.size() - 2;
        auto V = v;
        V.resize(m);
        vector<Mint> zero = rec(V);
```

```
        // last two vertices dropped
    for (int i = 0; i < m; i++) for (int j = 0; j < i; j++) {
        add(V[i][j], v[m][i], v[m + 1][j]);
        // ... or routed through them
        add(V[i][j], v[m + 1][i], v[m][j]);
    }
    vector<Mint> one = rec(V);
    for (int i = 0; i < h; i++) ans[i] += one[i] - zero[i];
    add(ans, one, v[m + 1][m]);
    return ans;
  }
  Mint solve(const vector<vector<Mint>>& a) {
        // a symmetric, a[i][i] unused
    int n = a.size();
    h = n / 2 + 1;
    vector<vector<vector<Mint>>> v(n);
    for (int i = 0; i < n; i++) {
        v[i].resize(i);
        for (int j = 0; j < i; j++) v[i][j].assign(h, 0), v[i][j][0] = a[i][j];
    }
    return rec(v).back();
  }
};
// 0/1 MATRIX -> plain count of perfect matchings. General weights -> weighted count, so this also
// answers "sum over perfect matchings of the product of edge weights" (partition functions).
// PERMANENT by Ryser, for comparison, is 6 lines and O(2^n n):
//   perm(A) = (-1)^n * sum over subsets S of columns of (-1)^|S| * prod_i (sum_{j in S} A[i][j]).
// COUNTING MATCHINGS OF ALL SIZES (not just perfect): pad the matrix with a zero-weight vertex per
// vertex - or read off rec()'s whole coefficient vector, which is exactly that generating function.
// PLANAR GRAPHS have a POLYNOMIAL count via the FKT algorithm (Pfaffian orientation + determinant),
// O(n^3) - that is the one to use for domino tilings of a grid.
```

3.9.5 MeetInTheMiddle

MEET IN THE MIDDLE ON GRAPHS - the n <= 40..50 band, where 2^n is hopeless but 2^(n/2) is not.

```
// 0-indexed, adj[] is an adjacency BITMASK (ll, so n <= 63), no self loops unless stated.
// THE PATTERN. Split the vertices into halves L (first k) and R (the rest). Enumerate every valid
// configuration of L; each one imposes a REQUIREMENT on R that is a single bitmask; precompute for
// every R-mask the best/total over all valid R-configurations COMPATIBLE with it, using a SOS
// (subset-sum / superset-sum) transform. Then one pass over the 2^k left masks answers everything.

// --- COUNT ALL CLIQUES (including the empty one), O(2^(n/2) * n). ---
// A set is a clique iff every chosen vertex is adjacent to all the others, i.e. the intersection of
// (adj[v] | bit v) over v in the set contains the set.
ll countCliques(int n, const vector<ll>& adj) {
    int k = n / 2, r = n - k;
    ll full = n == 64 ? -1LL : (1LL << n) - 1;
    vector<ll> g(n);
    for (int i = 0; i < n; i++) g[i] = adj[i] | 1LL << i;
    vector<ll> dp(1 << k, 0);
    dp[0] = 1;
    for (int m = 1; m < (1 << k); m++) {
        // is the left mask m a clique?
        ll nw = full;
        for (int j = 0; j < k; j++) if (m >> j & 1) nw &= g[j];
        if ((nw & m) == m) dp[m] = 1;
    }
    for (int i = 0; i < k; i++) for (int m = 0; m < (1 << k); m++)    // SOS: dp[m] = #subsets
        if (m >> i & 1) dp[m] += dp[m ^ 1 << i];
    ll ans = dp[(1 << k) - 1];
        // right half empty
    for (int m = 1; m < (1 << r); m++) {
        ll nw = full, p = (ll)m << k;
        for (int j = 0; j < r; j++) if (m >> j & 1) nw &= g[k + j];
        if ((nw & p) == p) ans += dp[nw & ((1LL << k) - 1)];    // left must sit inside nw
    }
    return ans;
}

// --- MINIMUM COST VERTEX COVER OF A GENERAL GRAPH, O(2^(n/2) * n). ---
// Every edge needs at least one endpoint chosen; minimise the total cost of the chosen vertices.
// A left mask fixes the left choices; the UNCHOSEN left vertices force their right neighbours in.
// SELF LOOP at v (bit v of adj[v]) = "v must be taken", which is how "forbidden" vertices are said.
// (On a BIPARTITE graph this is polynomial - Konig / min cut. This is the general case.)
ll minCostVertexCover(int n, const vector<ll>& adj, const vector<ll>& cost) {
    int k = n / 2, r = n - k;
    vector<ll> cl(1 << k, 0), need(1 << k, 0), dp(1 << r, 0);
    for (int m = 0; m < (1 << k); m++) {
        for (int i = 0; i < k; i++) {
            if (m >> i & 1) { cl[m] += cost[i]; continue; }
            if (adj[i] >> i & 1) { cl[m] = llinf; break; }    // self loop: i had to be taken
            for (int j = 0; j < n; j++) if (adj[i] >> j & 1) {
                if (j < k) { if (!(m >> j & 1)) cl[m] = llinf; }   // uncovered left edge
                else need[m] |= 1 << (j - k);
            }
        }
    }
    for (int m = 0; m < (1 << r); m++) {
        for (int i = 0; i < r; i++) {
            if (m >> i & 1) { dp[m] += cost[k + i]; continue; }
            for (int j = k; j < n; j++)
                if ((adj[k + i] >> j & 1) && !(m >> (j - k) & 1)) dp[m] = llinf;
        }
    }
    for (int i = 0; i < r; i++) for (int m = (1 << r)
```

```
    - 1; m >= 0; m--)   // SOS over SUPERSETS
        if (!(m >> i & 1)) dp[m] = min(dp[m], dp[m ^ 1 << i]);
    ll ans = llinf;
    for (int m = 0; m < (1 << k); m++)
        if (cl[m] < llinf && dp[need[m]] < llinf) ans = min(ans, cl[m] + dp[need[m]]);
    return ans;
}
// MAXIMUM INDEPENDENT SET = (total cost) - (minimum cost vertex cover) with unit costs, so the
// second routine solves MIS for n <= 50 too. MAXIMUM CLIQUE for that range: 02 - MaxCliqueBitset.
// OTHER PROBLEMS THAT SPLIT THE SAME WAY: subset sum / knapsack on n <= 40, "count the subsets
// with property X" when X only couples the halves through one mask, and 3-colouring by enumerating
// the colour classes of one half.
// THE SOS DIRECTION MATTERS: use the subset transform when the right half must fit INSIDE a mask
// (counting), and the superset transform when the right half must CONTAIN a mask (covering).
```

3.10 Theorems

3.10.1 GraphTheorems

GRAPH THEOREM SHEET

```
/* ==================== GRAPH THEOREM SHEET ====================
 Every hypothesis here matters. A theorem quoted without its precondition is a wrong answer.

 MATCHING AND COVERING
   KONIG (BIPARTITE only):  max matching = min vertex cover.
     Constructive: Z = vertices reachable by alternating paths from the UNMATCHED left vertices;
     the cover is (X \ Z) union (Y and Z).
   GALLAI: for EVERY graph on n vertices,  alpha + tau = n   (independent set + vertex cover).
           for every graph WITH NO ISOLATED VERTEX, nu + rho = n   (matching + edge cover).
           Bipartite + Konig gives  alpha = n - nu.
   HALL (bipartite): a matching saturating every vertex of the LEFT part X exists iff
     |N(S)| >= |S| for every S contained in X.   (Not every S of V - the side matters.)
   ORE'S DEFECT FORM:  nu(G) = |X| - max over S contained in X of ( |S| - |N(S)| ).
   HALL FOR REGULAR BIPARTITE: every k-regular bipartite graph (k >= 1) has a perfect matching,
     so its edges split into exactly k perfect matchings. Equivalently KONIG'S EDGE COLOURING
     THEOREM: chi'(G) = Delta(G) for every bipartite MULTIGRAPH. (Timetabling, "split into
     Delta rounds", Latin-square completion.)
   BERGE: a matching is maximum iff there is no augmenting path. (Why Kuhn / HK / Blossom stop.)
   TUTTE (general graphs): G has a PERFECT matching iff odd(G - U) <= |U| for every U contained
     in V, where odd(H) = the number of components of H with an ODD number of vertices.
     U = empty forces n even.
   TUTTE-BERGE:  nu(G) = (1/2) * min over U of ( |V| + |U| - odd(G - U) ).
   DILWORTH (FINITE poset): min number of chains covering P = size of the maximum antichain.
   MIRSKY (dual): min number of antichains partitioning P = length of the longest chain.
   SPERNER: the largest antichain in the subset lattice of [n] has size C(n, floor(n/2)).
   PATH COVERS - the distinction that most notebooks get wrong:
     min VERTEX-DISJOINT path cover of a DAG = n - (max matching of the split graph, no closure).
     min chain cover for Dilworth = n - (max matching of the TRANSITIVE CLOSURE's split graph).

 CONNECTIVITY AND FLOW
   MENGER: for NON-ADJACENT s, t: min s-t VERTEX cut = max internally-vertex-disjoint s-t paths.
     For any s != t: min s-t EDGE cut = max edge-disjoint s-t paths.
   WHITNEY: kappa(G) <= lambda(G) <= delta(G) (vertex conn <= edge conn <= min degree).
     A graph on >= 3 vertices is 2-connected iff every two vertices lie on a common cycle.
   MAX-FLOW MIN-CUT, with INTEGRALITY: integer capacities => some maximum flow is integral (and
     Dinic produces one). Corollary used constantly: a 0/1-capacity max flow decomposes into that
     many edge-disjoint s-t paths.
   ROBBINS: an undirected graph has a STRONG orientation iff it is connected and BRIDGELESS.
     Constructive in O(V+E): DFS, orient tree edges away from the root and back edges toward it.
   ESWARAN-TARJAN (augmentation):
     directed - min arcs to make a digraph strongly connected = max(#sources, #sinks) of the
       condensation (0 if the condensation is a single node).
     undirected - for a CONNECTED graph, min edges to make it 2-edge-connected = ceil(L/2) where
       L is the number of leaves of the bridge tree (0 if the bridge tree is one node).

 DEGREE SEQUENCES
   ERDOS-GALLAI: d1 >= ... >= dn >= 0 is the degree sequence of a SIMPLE graph iff sum d_i is
     even and for every k in [1, n]:
       sum_{i<=k} d_i  <=  k(k-1) + sum_{i>k} min(d_i, k).
     Only k up to the largest index with d_k >= k needs checking (the Durfee square), so it is
     O(n) after sorting with prefix sums.
   HAVEL-HAKIMI: graphical iff d1 <= n-1 and, after deleting d1 and subtracting 1 from the next
     d1 entries (re-sorted), the rest is graphical. Gives a CONSTRUCTION, which Erdos-Gallai
     does not. O(n^2) naive.
   LANDAU (tournaments): s1 <= ... <= sn is a score sequence iff sum_{i<=k} s_i >= C(k,2) for
     every k, WITH EQUALITY at k = n.

 TOURNAMENTS
   REDEI: every finite tournament has a Hamiltonian PATH (insertion sort, O(n log n)).
   CAMION: every STRONGLY CONNECTED tournament has a Hamiltonian CYCLE.
   MOON: every strongly connected tournament is vertex-pancyclic - for every vertex v and every
     k in [3, n] there is a k-cycle through v.

 COUNTING
```

```
  MATRIX-TREE (Kirchhoff), undirected: #spanning
  ↪ trees (or the sum over trees of the product of
    weights) = ANY cofactor of L = D - A, loops
  ↪ excluded.
  MATRIX-TREE, DIRECTED (Tutte), with A_ij = #arcs i
  ↪ -> j:
    L_out = D_out - A, delete row AND column r  ->
  ↪ arborescences CONVERGING TO r (in-trees).
    L_in  = D_in  - A, delete row AND column r  ->
  ↪ arborescences DIVERGING FROM r (out-trees).
    These two are the pair everyone swaps. Verify on
  ↪ the 2-vertex graph 1 -> 2.
  BEST THEOREM: for a CONNECTED digraph with
  ↪ indeg(v) = outdeg(v) everywhere, the number of
    Eulerian circuits up to starting point is  ec(G)
  ↪ = t_w(G) * prod_v (outdeg(v) - 1)!,
    where t_w is the number of arborescences
  ↪ CONVERGING TO any fixed w (the L_out form above);
    t_w is the same for every w in such a graph.
  ↪ Multiply by outdeg(v) to fix a start vertex.
  CAYLEY: n^(n-2) labelled trees; with prescribed
  ↪ degrees, (n-2)! / prod (d_i - 1)!;
    labelled forests with k trees whose roots are k
  ↪ specified vertices: k * n^(n-k-1).

EULERIAN CONDITIONS  (the check EulerianPath.cpp
  ↪ needs before it starts)
  UNDIRECTED: a circuit exists iff every degree is
  ↪ even and all EDGES lie in one component.
    A trail exists iff exactly 0 or 2 vertices have
  ↪ odd degree (start at an odd one), same
    connectivity condition.
  DIRECTED: a circuit exists iff indeg = outdeg at
  ↪ every vertex and all vertices of nonzero
    degree lie in one component of the underlying
  ↪ undirected graph. A trail exists iff exactly
    one vertex has out - in = 1 (the start), exactly
  ↪ one has in - out = 1 (the end), the rest
    are balanced, same connectivity condition.

COLOURING
  BROOKS: for connected G, chi(G) <= Delta(G) UNLESS
  ↪ G is complete or an odd cycle, where
    chi = Delta + 1. Greedy on any order gives
  ↪ Delta+1; greedy on a degeneracy order gives d+1.
  VIZING (SIMPLE graphs): Delta <= chi' <= Delta +
  ↪ 1. Multigraphs: chi' <= Delta + mu, and
    Shannon's chi' <= floor(3 Delta / 2). Deciding
  ↪ class 1 vs class 2 is NP-complete - do not
    look for a polynomial algorithm mid-contest.
  ↪ Bipartite is the exception (Konig above).
  GALLAI-HASSE-ROY-VITAVER: chi(G) = 1 + min over
  ↪ orientations of (longest directed path length).
    Equivalently every orientation contains a
  ↪ directed path on chi(G) vertices.

EXTREMAL
  TURAN: a K_{r+1}-free graph has e <= (1 - 1/r) n^2
  ↪ / 2, with equality only for the balanced
    complete r-partite Turan graph. r = 2 is MANTEL:
  ↪ triangle-free => e <= floor(n^2/4).
  Every graph has a bipartite subgraph with >= m/2
  ↪ edges (greedy max-cut).
  A graph is bipartite iff it has no odd cycle.
  RAMSEY, the known small values: R(3,3)=6,
  ↪ R(3,4)=9, R(3,5)=14, R(3,6)=18, R(3,7)=23,
    R(3,8)=28, R(3,9)=36, R(4,4)=18, R(4,5)=25.
  ↪ R(4,6) in [36,40] and R(5,5) in [43,46] are
    OPEN. Bound: R(s,t) <= C(s+t-2, s-1).
  NASH-WILLIAMS ARBORICITY: a(G) = max over
  ↪ subgraphs S with >= 2 vertices of
    ceil( m_S / (n_S - 1) ). Consequences:
  ↪ degeneracy d satisfies a <= d <= 2a - 1; m <=
  ↪ a(n-1);
    triangle enumeration is O(m * a).
  TUTTE / NASH-WILLIAMS TREE PACKING: G has t
  ↪ edge-disjoint spanning trees iff every partition
    of V into k non-empty parts has at least t(k-1)
  ↪ crossing edges.
  PETERSEN: every BRIDGELESS CUBIC graph has a
  ↪ perfect matching. Every 2k-regular graph splits
    into k edge-disjoint 2-factors, so it has an
  ↪ Eulerian circuit and an orientation with
    indeg = outdeg = k.

PLANARITY
  EULER: connected plane graph V - E + F = 2; with c
  ↪ components, V - E + F = 1 + c.
  Simple planar with V >= 3: E <= 3V - 6.
  ↪ Triangle-free (girth >= 4): E <= 2V - 4.
  Every planar graph has a vertex of degree <= 5,
  ↪ i.e. it is 5-degenerate.
  KURATOWSKI / WAGNER: planar iff no K5 and no
  ↪ K_{3,3} subdivision (minor).
  FOUR COLOUR: every planar graph is 4-colourable -
  ↪ but there is no usable algorithm, so never
    plan around it. FIVE COLOUR is constructive in
  ↪ O(n) from the degree-<=5 vertex.
  K5, K_{3,3} and the Petersen graph are non-planar.
  ↪ In a planar graph min cut = shortest path
    in the dual.

HAMILTONICITY (sufficient conditions only - the
  ↪ general problem is NP-complete)
  DIRAC: n >= 3 and min degree >= n/2  =>
  ↪ Hamiltonian.
  ORE: deg(u) + deg(v) >= n for every non-adjacent
  ↪ pair  =>  Hamiltonian.
  Both are constructive by rotation-extension.

TREE FACTS
  JORDAN: every tree has 1 or 2 CENTRES; if 2 they
  ↪ are adjacent; they are the middle of any
    diameter path.
  CENTROID: every tree has 1 or 2 centroids; c is a
  ↪ centroid iff every component of T - c has
    <= n/2 vertices iff c minimises the sum of
  ↪ distances. If 2 they are adjacent and n is even.
    That <= n/2 bound is exactly what makes centroid
  ↪ decomposition O(log n) deep.
  DIAMETER: the greedy two-BFS is valid on trees and
  ↪ on positively weighted trees, NOT on
    general graphs. Every longest path passes
  ↪ through a centre. Joining two trees by an edge
    gives diameter max(d1, d2, r1 + r2 + 1),
  ↪ minimised by joining centre to centre.
  A tree is bipartite, has >= 2 leaves, and sum of
  ↪ degrees = 2(n-1).

WHAT IS NP-HARD, AND WHAT ONLY LOOKS IT
  NP-hard: max clique, chromatic number (>= 3
  ↪ colours), Hamiltonian path/cycle, TSP, MIXED
    Chinese postman, vertex cover / independent set
  ↪ on general graphs, max cut (general),
    Steiner tree with unbounded terminals, longest
  ↪ path, dominating set, feedback vertex/arc
```

```
    set, edge-colouring class decision, bandwidth.
   Polynomial despite appearances: vertex cover and
  ↪ independent set on BIPARTITE graphs (Konig)
    and on trees / interval / chordal graphs;
  ↪ 2-colouring; 2-SAT; max cut on PLANAR graphs;
    min mean cycle; min-cost flow; directed MST;
  ↪ maximum matching in a GENERAL graph (Blossom);
    min path cover of a DAG; Steiner tree with k <=
  ↪ ~15 terminals; TSP with n <= ~20.
============================================================
  ↪ */
```

3.11 Special Structures

3.11.1 StableChordalComplement

THREE GRAPH TOOLS THAT ARE NOT ABOUT PATHS OR FLOW.

```
// --- GALE-SHAPLEY STABLE MARRIAGE, O(n^2). ---
// n proposers, n receivers, each with a full
  ↪ preference ranking. A matching is STABLE if no
  ↪ pair
// mutually prefer each other to their assigned
  ↪ partners. One always exists, and this finds the
// PROPOSER-OPTIMAL one: every proposer gets the best
  ↪ partner they could have in ANY stable
// matching, and simultaneously every receiver gets
  ↪ their worst. (Swap the roles to flip that.)
// pref[i] = the proposers' preference lists;
  ↪ rank[j][i] = receiver j's ranking of proposer i.
vector<int> stableMarriage(const vector<vector<int>>&
  ↪ pref, const vector<vector<int>>& rank) {
    int n = pref.size();
    vector<int> match(n, -1), nxt(n, 0);
  ↪          // match[receiver] = proposer
    vector<int> free_(n);
    iota(all(free_), 0);
    while (!free_.empty()) {
        int m = free_.back(); free_.pop_back();
        int w = pref[m][nxt[m]++];
  ↪           // propose down the list
        if (match[w] < 0) match[w] = m;
        else if (rank[w][m] < rank[w][match[w]])
  ↪ free_.push_back(match[w]), match[w] = m;
        else free_.push_back(m);
    }
    return match;
}
// HOSPITALS/RESIDENTS (one side has capacity c) is
  ↪ the same loop with a heap of accepted
// proposers per hospital. STABLE ROOMMATES (one
  ↪ pool, no sides) may have NO stable matching -
// Irving's algorithm decides it, and it is a
  ↪ different, longer algorithm.

// --- TRAVERSAL OF THE COMPLEMENT GRAPH in O(n + m),
  ↪ never materialising the O(n^2) edges. ---
// Keep the not-yet-visited vertices in a linked
  ↪ structure; from u, walk that set and step to
// every w that is NOT a neighbour of u. Each vertex
  ↪ leaves the set once, and each REAL edge is
// inspected once, so the total is O(n + m).
// Reach for it whenever the input describes the
  ↪ NON-edges: "connected components of the graph of
// pairs that do NOT conflict", complement colouring,
  ↪ complement BFS distances.
vector<int> complementComponents(int n, const
  ↪ vector<vector<int>>& adj) {
    vector<char> isAdj(n, 0);
    set<int> unvis;
    for (int i = 0; i < n; i++) unvis.insert(i);
    vector<int> comp(n, -1);
    int c = 0;
    while (!unvis.empty()) {
        int s = *unvis.begin();
        unvis.erase(unvis.begin());
        vector<int> st{s};
        comp[s] = c;
        while (!st.empty()) {
            int u = st.back(); st.pop_back();
            for (int v : adj[u]) isAdj[v] = 1;
            vector<int> take;
            for (int w : unvis) if (!isAdj[w])
  ↪ take.push_back(w);
            for (int w : take) unvis.erase(w),
  ↪ comp[w] = c, st.push_back(w);
            for (int v : adj[u]) isAdj[v] = 0;
        }
        c++;
    }
    return comp;
  ↪          // comp[v] = component id
}
// The complement of a DISCONNECTED graph is
  ↪ connected, so a complement graph almost always
  ↪ has
// very few components - that is why this is fast in
  ↪ practice as well as in theory.

// --- CHORDAL GRAPHS: maximum cardinality search
  ↪ gives a PERFECT ELIMINATION ORDERING. ---
// A graph is CHORDAL if every cycle of length >= 4
  ↪ has a chord. Equivalently it has a PEO: an
// order in which every vertex's LATER neighbours
  ↪ form a clique.
// Why you care: on a chordal graph, MAXIMUM CLIQUE,
  ↪ CHROMATIC NUMBER, MAXIMUM INDEPENDENT SET
// and MINIMUM CLIQUE COVER are all POLYNOMIAL - the
  ↪ four problems that are NP-hard in general.
// Interval graphs and trees are chordal, so "these
  ↪ intervals conflict" problems land here.
vector<int> mcsOrder(int n, const
  ↪ vector<vector<int>>& adj) {   // reverse of a
  ↪ PEO if chordal
    vector<int> w(n, 0), ord;
    vector<char> used(n, 0);
    for (int it = 0; it < n; it++) {
        int b = -1;
        for (int v = 0; v < n; v++) if (!used[v] &&
  ↪ (b < 0 || w[v] > w[b])) b = v;
        used[b] = 1, ord.push_back(b);
        for (int u : adj[b]) if (!used[u]) w[u]++;
    }
    reverse(all(ord));
    return ord;
  ↪          // this IS a PEO iff chordal
}
bool isChordal(int n, const vector<vector<int>>& adj)
  ↪ {
    auto ord = mcsOrder(n, adj);
    vector<int> pos(n);
    for (int i = 0; i < n; i++) pos[ord[i]] = i;
    vector<char> mark(n, 0);
    for (int i = 0; i < n; i++) {
  ↪          // v's later neighbours must be
        int v = ord[i], par = -1;
```

```
        // a clique; it is enough to check
    for (int u : adj[v]) if (pos[u] > i && (par < 0 || pos[u] < pos[par])) par = u;
    if (par < 0) continue;
    for (int u : adj[par]) mark[u] = 1;
    for (int u : adj[v]) if (pos[u] > i && u != par && !mark[u]) {
      for (int x : adj[par]) mark[x] = 0;
      return false;
    }
    for (int u : adj[par]) mark[u] = 0;
  }
  return true;
}
// GIVEN A PEO: greedy colouring in PEO order uses exactly omega colours (so chi = omega); the
// maximum clique is the largest "v plus its later neighbours"; the maximum independent set is the
// greedy "take v, delete its later neighbours" over the PEO.
```

3.11.2 MinimumWeightCycle

MINIMUM WEIGHT CYCLE THROUGH EVERY VERTEX (and hence the global minimum weight cycle), by one

```
// Dijkstra per source, O(V^3) with the dense version below or O(V (E + V log V)) with a heap.
// UNDIRECTED, non-negative weights, 0-indexed, g is an adjacency MATRIX with llinf for "no edge".
// Returns best[u] = the weight of the lightest simple cycle CONTAINING u (llinf if there is none);
// the global answer is the minimum over u, and the GIRTH is the unweighted case (all weights 1).
// WHY IT WORKS. A lightest cycle through u splits into two shortest paths from u that leave along
// DIFFERENT first edges and are joined by one more edge (cur, i). So run Dijkstra from u, tag every
// vertex with the first edge (the "branch") its shortest path used, and combine any two settled
// vertices in different branches. The ‘par[cur] != cur‘ guard rejects the degenerate 2-cycle that
// walks one edge out and back.
vector<ll> minWeightCycles(int n, const vector<vector<ll>>& g) {
  vector<ll> best(n, llinf), d(n);
  vector<int> par(n), vis(n);
  for (int u = 0; u < n; u++) {
    fill(all(vis), 0), fill(all(d), llinf);
    iota(all(par), 0);
    d[u] = 0;
    for (;;) {
      int cur = -1;
      // dense Dijkstra: O(n^2) per source
      for (int i = 0; i < n; i++) if (!vis[i] && d[i] < llinf && (cur < 0 || d[i] < d[cur]))
        cur = i;
      if (cur < 0) break;
      vis[cur] = 1;
      if (par[cur] != cur && g[cur][u] < llinf)
        // close the cycle straight to u
        best[u] = min(best[u], d[cur] + g[cur][u]);
      for (int i = 0; i < n; i++) {
        if (i == u || g[cur][i] == llinf) continue;
        if (vis[i]) {
          if (par[i] != par[cur])
            // two different branches meet
            best[u] = min(best[u], d[cur] + d[i] + g[cur][i]);
        } else if (d[cur] + g[cur][i] < d[i])
          d[i] = d[cur] + g[cur][i], par[i] = (cur == u ? i : par[cur]);
      }
    }
  }
  return best;
}
// GLOBAL MINIMUM WEIGHT CYCLE: min over u of best[u]. If that is all you want and the graph is
// DIRECTED, Floyd-Warshall is shorter: run it, then take min over v of d[v][v] (initialise the
// diagonal to llinf, not 0).
// GIRTH of an unweighted graph: replace Dijkstra by BFS - O(V * E), and for a cycle through a
// FIXED vertex the two-branch rule is the same.
// MINIMUM CYCLE THROUGH A GIVEN EDGE (u, v): delete the edge and take dist(u, v) + w.
// MINIMUM MEAN cycle (weight divided by length) is a different problem - Karp, see 02 - Shortest
// Paths/06 - CyclesAndConstraints.
// NEGATIVE WEIGHTS break this completely (Dijkstra); a minimum weight cycle with negative edges is
// NP-hard in general, since a Hamiltonian cycle can be encoded in it.
// LONGEST cycle is NP-hard. Shortest cycle THROUGH k GIVEN VERTICES is FPT via colour coding.
```

3.11.3 IncrementalDAGReachability

INCREMENTAL DAG REACHABILITY (Italiano) - answer "is there a path u -> v?" while ARCS ARE BEING

```
// ADDED, in O(1) per query and O(n) amortised per insertion. Directed, 0-indexed, O(n^2) memory.
// addEdge(s, t) refuses (and returns false) any arc that would create a CYCLE, so the structure
// doubles as an online "would this arc break acyclicity?" oracle - which is the usual reason to
// reach for it: "process the constraints in order and report the first one that is contradictory",
// online topological ordering, incremental dependency graphs.
// STRUCTURE. For every vertex s it keeps a REACHABILITY TREE of everything s can reach, stored as
// par[s][v] = v’s parent in that tree. Adding s -> t only ever ADDS vertices to those trees, and a
// vertex is added to a tree at most once, so the total work over all insertions is O(n^2) per
// source - i.e. O(n) amortised per update. Nothing is ever removed, hence "incremental only".
struct IncrementalDAG {
  int n;
  vector<vector<int>> par;
    // par[root][v], -1 = not reachable
  vector<vector<vector<int>>> ch;
    // ch[root][v] = children in the tree
  IncrementalDAG(int n) : n(n), par(n, vector<int>(n, -1)), ch(n, vector<vector<int>>(n)) {}
  bool reachable(int s, int t) const { return s == t || par[s][t] >= 0; }
  void meld(int root, int sub, int u, int v) {
    // graft v (and its subtree) under u
    par[root][v] = u, ch[root][u].push_back(v);
    for (int c : ch[sub][v]) if (!reachable(root,
```

```
    ↪ c)) meld(root, sub, v, c);
    }
    bool addEdge(int s, int t) {
    ↪          // false = it would close a cycle
        if (reachable(t, s)) return false;
        if (reachable(s, t)) return true;
    ↪          // nothing changes
        for (int p = 0; p < n; p++)
            if (reachable(p, s) && !reachable(p, t))
    ↪ meld(p, t, s, t);
        return true;
    }
};
// FULLY DYNAMIC reachability (with deletions) has no
    ↪ simple solution; if the updates are known in
// advance, run OFFLINE divide and conquer over the
    ↪ time axis instead.
// DENSE ALTERNATIVE: bitset transitive closure, n/64
    ↪ words per vertex, rebuilt from scratch in
// O(n * m / 64) - simpler and often faster for n <=
    ↪ 2000 when the number of queries is small.
// TRANSITIVE CLOSURE of a static DAG: process in
    ↪ reverse topological order and OR the successors'
// bitsets - O(n m / 64).
// COUNTING PAIRS (u, v) with u reaching v is the
    ↪ same bitset sweep; there is no better general
// algorithm (it is equivalent to boolean matrix
    ↪ multiplication).
```

4 | Number Theory

4.1 Euclidean Algorithm

4.1.1 ExtendedEuclid

EXTENDED EUCLID - returns g = gcd(a,b) and x, y with a*x + b*y = g. This is the one routine that

```
// gives you modular inverses for a NON-prime
    ↪ modulus, solves linear Diophantine equations, and
// underpins CRT. |x| <= b/(2g) and |y| <= a/(2g), so
    ↪ no overflow for 64-bit inputs.
long long extGCD(long long a, long long b, long long
    ↪ &x, long long &y) {
    if (b == 0) {
        x = 1; y = 0;
        return a;
    }
    long long x1, y1;
    long long d = extGCD(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return d;
}

long long modInverse(long long a, long long m) {
    long long x, y;
    long long g = extGCD(a, m, x, y);
    if (g != 1) return -1; // Inverse doesn't exist
    return (x % m + m) % m;
}
```

4.1.2 LinearDiophantine

Linear Diophantine Equations (2 Variables & N Variables)

```
// 1. Solves a * x + b * y = c (Particular solution &
    ↪ GCD check)
// 2. Solves a_1*x_1 + a_2*x_2 + ... + a_n*x_n = c
    ↪ for N variables
struct LinearDiophantine {
    static ll ext_gcd(ll a, ll b, ll &x, ll &y) {
        if (b == 0) {
            x = 1; y = 0;
            return a;
        }
        ll x1, y1;
        ll d = ext_gcd(b, a % b, x1, y1);
        x = y1;
        y = x1 - y1 * (a / b);
        return d;
    }

    // Solves a * x + b * y = c. Returns false if no
    ↪ solution exists
    static bool solve_2var(ll a, ll b, ll c, ll &x,
    ↪ ll &y, ll &g) {
        if (a == 0 && b == 0) {
            if (c == 0) { x = y = g = 0; return true;
    ↪ }
            return false;
        }
        g = ext_gcd(abs(a), abs(b), x, y);
        if (c % g != 0) return false;
        x *= c / g;
        y *= c / g;
        if (a < 0) x = -x;
        if (b < 0) y = -y;
        return true;
    }

    // Solves a_1*x_1 + a_2*x_2 + ... + a_n*x_n = c
    ↪ for N variables.
    // Returns true if a solution exists and
    ↪ populates x vector
    static bool solve_nvar(const vector<ll>& a, ll c,
    ↪ vector<ll>& x) {
        int n = a.size();
        if (n == 0) return c == 0;
        x.assign(n, 0);

        vector<ll> prefix_g(n);
        prefix_g[0] = abs(a[0]);
        for (int i = 1; i < n; i++) prefix_g[i] =
    ↪ std::gcd(prefix_g[i - 1], a[i]);

        if (prefix_g.back() == 0) return c == 0;
        if (c % prefix_g.back() != 0) return false;

        ll cur_c = c;
        for (int i = n - 1; i > 0; i--) {
            ll x_prev, y_curr, g;
            solve_2var(prefix_g[i - 1], a[i], cur_c,
    ↪ x_prev, y_curr, g);
            x[i] = y_curr;
            cur_c = prefix_g[i - 1] * x_prev;
        }
        x[0] = cur_c / a[0];
        return true;
    }
};
```

4.1.3 FloorSum

EUCLIDEAN-LIKE ALGORITHM (AtCoder Library 'floor_sum').

```
//   floorSum(n, m, a, b) = sum_{i=0}^{n-1}
  ↪ floor((a*i + b) / m)     in O(log m).
// This is THE tool for "sum over a lattice line":
  ↪ counting lattice points under a line, sums of
// floor/mod expressions, and anything of the shape
  ↪ sum floor((a i + b) / m) that a loop cannot do
// because n is up to 1e9 or more. Works like the
  ↪ Euclidean algorithm: reduce a mod m, then SWAP
// the roles of the axes (count by rows instead of by
  ↪ columns) and recurse.
// Requires n >= 0, m >= 1, a >= 0, b >= 0. Overflow:
  ↪ a*n + b must fit in ll (use __int128 if not).
ll floorSum(ll n, ll m, ll a, ll b) {
    ll ans = 0;
    if (a >= m) ans += (n - 1) * n / 2 * (a / m), a
  ↪ %= m;
    if (b >= m) ans += n * (b / m), b %= m;
    ll ymax = (a * n + b) / m;
    if (ymax == 0) return ans;
    ll xmax = ymax * m - b;
    ans += ymax * (n - (xmax + a - 1) / a);
    return ans + floorSum(ymax, a, m, (a - xmax % a)
  ↪ % a);
}
// NEGATIVE a or b: shift them into range first.
//   a < 0: let a2 = a mod m in [0,m);
  ↪ floorSum(n,m,a,b) = floorSum(n,m,a2,b) -
  ↪ (a2-a)/m * (n-1)n/2
//   b < 0: let b2 = b mod m in [0,m); subtract n *
  ↪ (b2-b)/m.  (Same trick, easier: use __int128.)
//
// WHAT IT SOLVES, directly:
//   * #lattice points (x,y) with 1 <= x <= n and 1
  ↪ <= y <= (a x + b)/m       -> floorSum itself
//   * sum_{i=0}^{n-1} ((a i + b) mod m) = (a *
  ↪ n(n-1)/2 + b*n) - m * floorSum(n, m, a, b)
//   * sum_{i=1}^{n} floor(n / i) in O(sqrt n)
  ↪ instead - use DIVISOR BLOCKS (below), not this.
//   * "how many multiples of q lie in [l, r]" =
  ↪ floor(r/q) - floor((l-1)/q); the basic case.
//
// THE OTHER TWO SUMS of the same family (needed
  ↪ together often enough to be worth knowing they
// exist): S1 = sum i * floor((a i + b)/m) and S2 =
  ↪ sum floor((a i + b)/m)^2 obey the same
// recursion with a 3-tuple state - the "universal
  ↪ Euclidean algorithm" generalises
// this to any monoid: you supply the two operators U
  ↪ ("move right") and R ("move up") and it
// composes the word for the line in O(log) monoid
  ↪ multiplications.
//
// DIVISOR BLOCKS - the other floor trick, O(sqrt n):
//   floor(n / i) is constant on maximal blocks; for
  ↪ a block starting at l the block ends at
//   r = n / (n / l). Use it for sum_{i=1}^{n}
  ↪ floor(n/i), sum of tau/sigma, Du sieve, Mobius
  ↪ sums.
//     for (ll l = 1, r; l <= n; l = r + 1) { r = n /
  ↪ (n / l); /* value n/l on [l, r] */ }
//   Two-argument version (blocks constant for BOTH
  ↪ n/i and m/i): r = min(n/(n/l), m/(m/l)).
```

4.1.4 FloorSumExtended

EUCLIDEAN-LIKE ALGORITHM, THE FULL TRIPLE. With q_i = floor((a*i + b) / c) it returns

```
//     f = sum_{i=0..n} q_i        g = sum_{i=0..n} i
  ↪ * q_i        h = sum_{i=0..n} q_i^2
// all three at once in O(log c). NOTE i runs to n
  ↪ INCLUSIVE, unlike floorSum in 03, which stops
// at n-1: floorSum(n, m, a, b) = fgh(n - 1, a, b,
  ↪ m).f.
// Reach for it when a lattice-line sum has an i or a
  ↪ square in it: sum of i*floor, sum of
// (a i + b mod c)^2, the second moment of a modular
  ↪ AP, "sum over lattice points under a line of
// (x + y)" and friends. The recursion is the same as
  ↪ floorSum: strip a/c and b/c, then swap axes.
// Requires n >= -1, a >= 0, b >= 0, c >= 1. Exact
  ↪ (no modulus) in __int128, which is good for
// n and m = (a n + b)/c up to ~1e11 (h is of order
  ↪ n*m^2). Take everything mod p if they are
// bigger - the recursion only ever adds and
  ↪ multiplies, so it is modulus-friendly.
struct FGH { __int128 f, g, h; };
FGH fgh(ll n, ll a, ll b, ll c) {
    if (n < 0) return {0, 0, 0};
    __int128 M = n, s1 = M * (M + 1) / 2, s2 = M * (M
  ↪ + 1) * (2 * M + 1) / 6;
    if (a >= c || b >= c) {
  ↪     // q_i = A*i + B + (reduced q_i)
        __int128 A = a / c, B = b / c;
        FGH t = fgh(n, a % c, b % c, c);
        return {t.f + A * s1 + B * (M + 1), t.g + A *
  ↪ s2 + B * s1,
                t.h + 2 * B * t.f + 2 * A * t.g + A *
  ↪ A * s2 + B * B * (M + 1) + 2 * A * B * s1};
    }
    __int128 m = ((__int128)a * n + b) / c;
  ↪     // the largest value any q_i takes
    if (m == 0) return {0, 0, 0};
    FGH t = fgh((ll)m - 1, c, c - b - 1, a);
  ↪     // count by rows instead of by columns
    __int128 f = M * m - t.f;
    return {f, (m * M * (M + 1) - t.f - t.h) / 2, M *
  ↪ m * (m + 1) - 2 * t.g - 2 * t.f - f};
}
// DERIVED SUMS (r_i = (a*i + b) mod c = a*i + b -
  ↪ c*q_i):
//   sum r_i   = a*s1 + b*(n+1) - c*f
//   sum r_i^2 = sum (a i + b)^2 - 2c * (a*g + b*f) +
  ↪ c^2 * h
//   sum i*r_i = a*s2 + b*s1 - c*g
// The same recursion with a bigger state gives sum
  ↪ i^k1 * q_i^k2 for any k1 + k2 <= K in
// O(K^3 log c) - rarely needed. The clean way to
  ↪ state the whole family is the UNIVERSAL
// EUCLIDEAN ALGORITHM: walk the line as a word in {U
  ↪ = "step up", R = "step right"} and
// multiply the letters in ANY monoid; supply U and R
  ↪ and you get whatever statistic you like.
```

4.1.5 LatticeUnderLine

COUNT THE NONNEGATIVE LATTICE POINTS UNDER A LINE: #{(x, y) : x, y >= 0, a*x + b*y <= c}

```
// in O(log min(a, b)) - a Euclidean-style descent that peels off a full rectangle of solutions
// each round, never touching a single point individually.
// This is the "how many ways to pay at most c with coins a and b" primitive, and the engine
// behind 06 - ModCountInRange.cpp. Requires a >= 1, b >= 1; returns 0 for c < 0 (a = 0 or b = 0
// would be infinite - handle that case yourself).
// OVERFLOW: the answer is about c^2 / (2ab) and the intermediate (f+1)*f/2*k is of the same
// order, so it must fit in ll; c up to ~4e18 is fine as long as the count itself does.
ll latticeUnderLine(ll a, ll b, ll c) {
    if (c < 0) return 0;
    if (a > b) swap(a, b);
    // descend on the smaller coefficient
    ll ans = 0;
    while (c >= 0) {
        // b = k*a + l, then swap roles
        ll k = b / a, l = b % a, f = c / b, e = c % b / a, g = c % b % a;
        ans += (f + 1) * (e + 1) + (f + 1) * f / 2 * k; // the axis-aligned part
        c = f * l - a + g;
        // l = 0 makes c < 0: the loop ends
        b = a, a = l;
    }
    return ans;
}
// VARIANTS, all from this one call:
//   a*x + b*y <  c        -> latticeUnderLine(a, b, c - 1)
//   a*x + b*y <= c, x >= 1, y >= 1  -> latticeUnderLine(a, b, c - a - b)
//   a*x + b*y == c exactly -> latticeUnderLine(a,b,c) - latticeUnderLine(a,b,c-1), or solve the
//     Diophantine equation and count the admissible k directly (02 - LinearDiophantine.cpp).
//   x in [0, X] as well   -> subtract the shifted count latticeUnderLine(a, b, c - a*(X+1)).
// Related: sum_{x} floor((c - a x)/b) is the same quantity as a floor sum, so 03 - FloorSum.cpp
// also answers it in O(log) - use whichever shape the problem hands you.
```

4.1.6 ModCountInRange

HOW MANY x IN [xl, xr] HAVE (a*x mod m) INSIDE [cl, cr]? O(log m), any a, m >= 1.

```
// Needs: latticeUnderLine (05 - LatticeUnderLine.cpp).
// The classic use: "how many of the first n multiples of a land in a window mod m" - collision
// counting, clock/lap problems, counting i with (a*i + b) mod m <= c (shift by b: replace the
// window [cl, cr] by [cl - b, cr - b] reduced mod m, splitting it if it wraps past 0).
// modCountLE counts 0 <= x <= n with (a*x mod m) <= c; everything else is inclusion-exclusion
// on that. OVERFLOW: latticeUnderLine is called with c ~ 2*m*n, so m*n must stay below ~4e18
// (use __int128 inside latticeUnderLine if it does not).
ll modCountLE(ll a, ll m, ll c, ll n) {
    // count 0 <= x <= n with a*x mod m <= c
    if (n < 0) return 0;
    ++c, ++n;
    // work with half-open [0, c) and [0, n)
    a %= m; if (a < 0) a += m;
    ll ec = c / m; c %= m; if (c < 0) ec--, c += m;
    // c outside [0, m) means whole periods
    ll ans = ec * n, en = n / m; n %= m;
    if (n < 0) en--, n += m;
    auto blk = [&](ll len) {
        // x in [0, len): len < m, uses a + m > 0
        return latticeUnderLine(m, a + m, (a + m) * (len - 1))
             - latticeUnderLine(m, a + m, (a + m) * (len - 1) - c);
    };
    if (en) ans += en * blk(m);
    // en full periods of length m
    if (n) ans += blk(n);
    return ans;
}
ll modCountRange(ll a, ll m, ll cl, ll cr, ll xl, ll xr) {
    return modCountLE(a, m, cr, xr) - modCountLE(a, m, cr, xl - 1)
         - modCountLE(a, m, cl - 1, xr) + modCountLE(a, m, cl - 1, xl - 1);
}
// SANITY CHECK: with g = gcd(a, m) the values a*x mod m are exactly the multiples of g, and over
// one full period x in [0, m) each of them is hit exactly g times. So modCountRange over a full
// period must equal g * (number of multiples of g inside [cl, cr]).
// COMPANION ROUTINES: the smallest such x is 07 - SmallestModInRange.cpp; if you need the whole
// set of x, note that it is a union of O(sqrt m) arithmetic progressions
// (03 - Modular Arithmetic/09 - ModularAPBlocks.cpp).
```

4.1.7 SmallestModInRange

SMALLEST x >= 0 WITH l <= (a*x mod p) <= r, or -1 if no such x exists. O(log p).

```
// The "when does the ticking counter first land in the window" problem: jumps of size a on a
// circle of circumference p, first time inside the arc [l, r]. Also solves "smallest x with
// a*x mod p <= r" (l = 0) and, by symmetry, the first time OUTSIDE a window.
// PRECONDITIONS: 0 <= a < p, 0 <= l <= r < p, p >= 1. p need NOT be prime.
// Idea: if ceil(l/a)*a <= r we are done in one step. Otherwise write p = k*a + b; asking for
// l <= a*x - p*y <= r turns into the same question one Euclidean step down, with the roles of
// the modulus and the multiplier swapped - so the recursion depth is O(log p), like gcd.
// OVERFLOW: the internal products are up to p*p, hence the __int128.
ll smallestModIn(ll p, ll a, ll l, ll r) {
    if (a == 0) return l == 0 ? 0 : -1;
    auto cdiv = [](__int128 x, __int128 y) { return (ll)((x + y - 1) / y); };
    ll c = cdiv(l, a);
    // first x with a*x >= l, ignoring wraps
    if ((__int128)a * c <= r) return c;
    ll b = p % a;
    // p = (p/a)*a + b
    ll y = smallestModIn(a, b, a - r % a, a - l % a);
    // -r <= b*y (mod a) <= -l
    return y == -1 ? -1 : cdiv((__int128)l + (__int128)b * y, a) + p / a * y;
}
// COMPANIONS
//   count of such x in a range          -> 06 - ModCountInRange.cpp
//   the whole solution set              -> it is a union of O(sqrt p) APs, see
//                                          03 - Modular Arithmetic/09 - ModularAPBlocks.cpp
//   smallest x with a*x mod p == target -> solve the linear congruence instead
//                                          (03 - Modular Arithmetic/04 - CongruencesAndTowers.cpp)
// TYPICAL USE: interactive/recovery problems where you know x mod M for a huge M and want the
// smallest representative that also satisfies 1 <= q <= N; and "minimum of (k*a mod p) over
// k <= n", which the three-distance theorem says is achieved at a continued-fraction denominator.
```

4.1.8 CoinReachability

WHICH TOTALS ARE PAYABLE WITH COINS a[0..n-1]? (all x_i >= 0, unbounded supply)

```
//   sum a_i x_i = k solvable?   how many solvable k in [l, r]?   largest unsolvable k?
// THE TRICK ("congruence shortest path" / Dijkstra on residues): fix A = min a_i and build a
// graph on the residues mod A with an edge u -> (u + a_i) mod A of weight a_i. Then
//   d[u] = the SMALLEST payable value congruent to u (mod A)
// and k is payable iff d[k mod A] <= k. So every residue class contributes the arithmetic
// progression d[u], d[u] + A, d[u] + 2A, ... and everything below d[u] in that class is not.
// Build O(A * n * log A) time, O(A) memory. Take A = min a_i (that is what bounds the table).
// PRECONDITIONS: all a_i >= 1. Values may be up to 1e18; A up to a few million is the real cap.
struct CoinReach {
    ll A;
    vector<ll> d;
    // d[u] = min payable value = u (mod A)
    CoinReach(vector<ll> a) {
        A = *min_element(all(a));
        d.assign(A, llinf);
        d[0] = 0;
        priority_queue<pair<ll, ll>, vector<pair<ll, ll>>, greater<>> pq;
        pq.push({0, 0});
        while (!pq.empty()) {
            auto [val, u] = pq.top(); pq.pop();
            if (val != d[u]) continue;
            for (ll c : a) {
                ll nv = val + c, v = nv % A;
                if (nv < d[v]) d[v] = nv, pq.push({nv, v});
            }
        }
    }
    bool payable(ll k) { return k >= 0 && d[k % A] <= k; }
    ll count(ll l, ll r) {
        // #payable k in [l, r], l >= 0
        ll res = 0;
        for (ll u = 0; u < A; u++) {
            if (d[u] <= r) res += (r - d[u]) / A + 1;
            if (d[u] <= l - 1) res -= (l - 1 - d[u]) / A + 1;
        }
        return res;
    }
    ll frobenius() {
        // largest UNpayable value, or -1
        ll mx = 0;
        // requires gcd(a_i) = 1, else infinite
        for (ll u = 1; u < A; u++) mx = max(mx, d[u]);
        return mx == 0 ? -1 : mx - A;
    }
};
// FROBENIUS / CHICKEN McNUGGET closed forms exist only for two coins: with gcd(a,b) = 1 the
// largest unpayable value is a*b - a - b and exactly (a-1)(b-1)/2 values are unpayable.
// For three or more coins there is no formula - this table IS the algorithm.
// SANITY: if gcd of all a_i is g > 1, only multiples of g are ever payable; divide everything
```

```
// by g first, otherwise frobenius() is meaningless (infinitely many unpayable values).
// SAME TABLE ANSWERS: smallest payable value >= k (that is d[u] + ceil((k - d[u])/A)*A minimised
// over u), and "can I pay exactly k with at most/at least c coins" if you make d 2-dimensional.
```

4.1.9 ThreeVarDiophantine

NUMBER OF NONNEGATIVE SOLUTIONS OF a*x + b*y + c*z = n, in O(log max(a,b,c)).

```
// (Popoviciu's formula for two coins, extended to three by Binner - the denumerant of a
// three-coin system.) Reach for it when a, b, c and n are all up to 1e18 and the DP over n or
// the O(n / max) loop over one variable are both hopeless. For two coins use the much simpler
// count of lattice points under a line, 05 - LatticeUnderLine.cpp.
// PRECONDITIONS: a, b, c >= 1, n >= 0. Handles arbitrary gcds (the wrapper strips them).
// All arithmetic is __int128 because the numerator N1 is of order n^2 and the products a*b*c and
// c*b*b1p reach max(a,b,c)^3; keep a, b, c below ~1e12 and n below ~1e18.
__int128 gcd128(__int128 a, __int128 b) {
    while (b) { __int128 t = a % b; a = b, b = t; }
    return a;
}
__int128 xgcd128(__int128 a, __int128 b, __int128 &x, __int128 &y) {
    if (!b) { x = 1, y = 0; return a; }
    __int128 x1, y1, d = xgcd128(b, a % b, x1, y1);
    x = y1, y = x1 - y1 * (a / b);
    return d;
}
__int128 inv128(__int128 a, __int128 m) {   // m >= 1, gcd(a, m) = 1
    __int128 x, y; xgcd128(((a % m) + m) % m, m, x, y);
    return ((x % m) + m) % m;
}
__int128 fsum128(__int128 n, __int128 m, __int128 a, __int128 b) {   // = floorSum, in __int128
    __int128 ans = 0;
    if (a >= m) ans += (n - 1) * n / 2 * (a / m), a %= m;
    if (b >= m) ans += n * (b / m), b %= m;
    __int128 ymax = (a * n + b) / m;
    if (ymax == 0) return ans;
    __int128 xmax = ymax * m - b;
    ans += ymax * (n - (xmax + a - 1) / a);
    return ans + fsum128(ymax, a, m, (a - xmax % a) % a);
}
__int128 denum3coprime(__int128 a, __int128 b, __int128 c, __int128 n) {  // a,b,c PAIRWISE coprime
    auto fix = [](__int128 v, __int128 m) { return v == 0 ? m : v; };
    __int128 b1 = fix((-n % a * inv128(b, a) % a + a) % a, a);
    __int128 c1 = fix(b % a * inv128(c, a) % a, a);
    __int128 c2 = fix((-n % b * inv128(c, b) % b + b) % b, b);
    __int128 a2 = fix(c % b * inv128(a, b) % b, b);
    __int128 a3 = fix((-n % c * inv128(a, c) % c + c) % c, c);
    __int128 b3 = fix(a % c * inv128(b, c) % c, c);
    __int128 N1 = n * (n + a + b + c);
    N1 += c * b * b1 * (a + 1 - c1 * (b1 - 1));
    N1 += a * c * c2 * (b + 1 - a2 * (c2 - 1));
    N1 += b * a * a3 * (c + 1 - b3 * (a3 - 1));
    return N1 / (2 * a * b * c) + fsum128(b1, a, c1, 0) + fsum128(c2, b, a2, 0)
                                + fsum128(a3, c, b3, 0) - 2;
}
__int128 denum3(__int128 a, __int128 b, __int128 c, __int128 n) {
    __int128 g = gcd128(gcd128(a, b), c);
    if (n % g) return 0;
    a /= g, b /= g, c /= g, n /= g;
    __int128 g1 = gcd128(b, c), g2 = gcd128(c, a), g3 = gcd128(a, b);
    __int128 n1 = n * inv128(a, g1) % g1, n2 = n * inv128(b, g2) % g2, n3 = n * inv128(c, g3) % g3;
    return denum3coprime(a / g2 / g3, b / g3 / g1, c / g1 / g2,
                         (n - a * n1 - b * n2 - c * n3) / g1 / g2 / g3);
}
// TWO COINS, for reference: #solutions of a*x + b*y = n with gcd(a,b) = 1 and x, y >= 0 is
//   n/(a*b) - {n * inv(a) mod b / b} - {n * inv(b) mod a / a} + 1   (Popoviciu),
// i.e. floor(n/(ab)) or that plus one; the largest n with NO solution is a*b - a - b.
// FOUR OR MORE coins: no such formula. Use 08 - CoinReachability.cpp (needs min a_i small), or
// a generating-function / polynomial approach when n is small.
```

4.1.10 IntegerLattice2D

A SYSTEM OF TWO LINEAR DIOPHANTINE EQUATIONS IN ANY NUMBER OF UNKNOWNS:

```
// Needs: extGCD (01 - ExtendedEuclid.cpp).
//   sum x_i * a_i = p   and   sum x_i * b_i = q,   x_i ANY integers - is it solvable?
// Equivalently: is (p, q) in the subgroup of Z^2 generated by the vectors (a_i, b_i)? Keep that
// lattice in Hermite normal form as two vectors (g, s) and (0, t): every lattice point is
// m*(g, s) + k*(0, t), with g >= 0, t >= 0 and 0 <= s < t when t > 0. Adding a generator is one
// extended-gcd step, so this is ONLINE - generators may arrive interleaved with queries, which
// is what makes it more than "solve a 2 x N system once".
// add() is O(log), contains() is O(1). No preconditions; zero vectors and rank 0/1 are handled.
// OVERFLOW: add() forms s*(c/g) and d*(g/g'), so keep coordinates below ~2e9 or switch to __int128.
struct Lattice2D {
    ll g = 0, s = 0, t = 0;   // t = 0 means "no free second direction"
    void add(ll c, ll d) {   // insert the generator (c, d)
        if (c < 0) c = -c, d = -d;
        // (c, d) and its negative are equivalent
        if (c == 0) {
        // pure vertical generator
            t = gcd(t, d < 0 ? -d : d);
            if (t) s = ((s % t) + t) % t;
            return;
```

```
        }
        if (g == 0) { g = c, s = t ? ((d % t) + t) %
    ↪ t : d; return; }
        ll x, y, ng = extGCD(g, c, x, y);
    ↪   // ng = gcd(g, c) = g*x + c*y
        ll cross = s * (c / ng) - d * (g / ng);
    ↪   // the leftover pure-vertical direction
        t = gcd(t, cross < 0 ? -cross : cross);
        ll ns = s * x + d * y;
        g = ng, s = t ? ((ns % t) + t) % t : ns;
    }
    bool contains(ll p, ll q) {
    ↪   // is (p, q) representable?
        if (g == 0) return p == 0 && (t ? q % t == 0
    ↪ : q == 0);
        if (p % g) return false;
        ll rem = q - (p / g) * s;
        return t ? rem % t == 0 : rem == 0;
    }
};
// WHY IT WORKS: extGCD writes (ng, ns) = x*(g, s) +
  ↪ y*(c, d) with ng = gcd(g, c), the smallest
// positive first coordinate achievable. What remains
  ↪ is the sublattice with first coordinate 0,
// generated by s*(c/ng) - d*(g/ng) - the determinant
  ↪ of the two old generators divided by ng.
// THE SAME OBJECT in higher dimension is the Hermite
  ↪ normal form of the generator matrix; with
// one row it degenerates to gcd, and 02 -
  ↪ LinearDiophantine.cpp is the single-equation
  ↪ version.
// TYPICAL USES: "can I reach (p, q) using these
  ↪ moves any integer number of times", reachability
// with two conserved quantities, and answering many
  ↪ (p, q) queries against a growing move set.
// NOTE this allows NEGATIVE x_i. Requiring x_i >= 0
  ↪ is a completely different (much harder)
// problem - see 08 - CoinReachability.cpp for the
  ↪ one-equation nonnegative version.
```

4.2 Primes and Divisors

4.2.1 Sieve

Linear Sieve & Smallest Prime Factor (SPF) - Time: O(N), Space: O(N)

```
// Computes primes list, SPF array, primality test,
  ↪ and O(log N) prime factorization
struct Sieve {
    int n;
    vector<int> primes;
    vector<int> spf;

    Sieve(int n = 1e7) : n(n), spf(n + 1, 0) {
        for (int i = 2; i <= n; i++) {
            if (spf[i] == 0) {
                spf[i] = i;
                primes.pb(i);
            }
            for (int p : primes) {
                if (p > spf[i] || (ll)i * p > n)
  ↪ break;
                spf[i * p] = p;
            }
        }
    }

    bool is_prime(int x) const {
        return x >= 2 && x <= n && spf[x] == x;
    }

    // O(log N) prime factorization using precomputed
  ↪ SPF array
    vector<int> get_prime_factors(int x) const {
        vector<int> factors;
        while (x > 1) {
            factors.pb(spf[x]);
            x /= spf[x];
        }
        return factors;
    }
};
```

4.2.2 Factorization

Trial Division Prime Factorization - Time: O(sqrt(N))

```
template <typename T = ll>
struct Factorization {
    static vector<T> get_prime_factors(T n) {
        vector<T> factors;
        for (T i = 2; i * i <= n; i++) {
            while (n % i == 0) {
                factors.pb(i);
                n /= i;
            }
        }
        if (n > 1) factors.pb(n);
        return factors;
    }
};
```

4.2.3 Divisors

Trial Division Divisor Generation - Time: O(sqrt(N))

```
template <typename T = ll>
struct Divisors {
    static vector<T> get_divisors(T n) {
        vector<T> divs;
        for (T i = 1; i * i <= n; i++) {
            if (n % i == 0) {
                divs.pb(i);
                if (n / i != i) divs.pb(n / i);
            }
        }
        return divs;
    }
};
```

4.2.4 Sieve1e9

Fast Bitset Sieve up to N = 10^9 - Time: ~0.3s for 10^9, Space: N / 16 bytes (~60MB)

```
// Stores odd numbers only (index i represents number
  ↪ 2*i + 1)
// WARNING: the bitset is ~60 MB - the instance MUST
  ↪ be global/static, never a local.
template <int MAXPR = (int)1e9>
struct Sieve1e9 {
    bitset<MAXPR / 2> is_prime_odd;

    Sieve1e9(int n = MAXPR) {
        is_prime_odd.set();
        is_prime_odd[0] = 0; // 1 is not prime
        for (int i = 1; (2 * i + 1) * (2 * i + 1) <=
  ↪ n; i++) {
            if (is_prime_odd[i]) {
                int p = 2 * i + 1;
                // bound is (n-1)/2, NOT n/2: n/2
  ↪ writes one index past the end of the bitset
```

```
                for (int j = 2 * i * (i + 1); j <= (n
  ↪ - 1) / 2; j += p) {
                    is_prime_odd[j] = 0;
                }
            }
        }
    }

    bool is_prime(int x) const {
        if (x == 2) return true;
        if (x < 2 || x % 2 == 0) return false;
        return is_prime_odd[x / 2];
    }
};
```

4.2.5 SegmentedSieve

Segmented Sieve for Range [L, R] - Time: O(sqrt(R) + (R - L) log log R), Space: O(sqrt(R) + R - L)

```
// Computes all prime numbers in range [L, R] where R
  ↪ <= 1e12 and (R - L) <= 1e7
struct SegmentedSieve {
    static vector<ll> get_primes_in_range(ll L, ll R)
  ↪ {
        ll limit = sqrtl((ld)R);
        while (limit * limit > R) limit--;
        while ((limit + 1) * (limit + 1) <= R)
  ↪ limit++;
        vector<bool> is_prime_small(limit + 1, true);
        vector<ll> primes;
        for (ll i = 2; i <= limit; i++) {
            if (is_prime_small[i]) {
                primes.pb(i);
                for (ll j = i * i; j <= limit; j +=
  ↪ i) is_prime_small[j] = false;
            }
        }

        vector<bool> is_prime_range(R - L + 1, true);
        for (ll p : primes) {
            ll start = max(p * p, (L + p - 1) / p *
  ↪ p);
            for (ll j = start; j <= R; j += p) {
                is_prime_range[j - L] = false;
            }
        }
        for (ll x = L; x <= min(R, 1LL); x++)
  ↪ is_prime_range[x - L] = false;   // 0 and 1

        vector<ll> result;
        for (ll i = 0; i <= R - L; i++) {
            if (is_prime_range[i]) result.pb(L + i);
        }
        return result;
    }
};
```

4.2.6 MillerRabinPollard

Primality + factorization for n up to 1e18. isPrime O(log^3 n), factor O(n^(1/4)).

```
// THE gate for any problem with a_i up to 1e18 -
  ↪ divisors, phi, tau, sigma all follow from
  ↪ factor().
typedef unsigned long long u64;
u64 mulmod(u64 a, u64 b, u64 m) { return (__int128)a
  ↪ * b % m; }
u64 powmod(u64 a, u64 e, u64 m) {
    u64 r = 1; a %= m;
    for (; e; e >>= 1, a = mulmod(a, a, m)) if (e &
  ↪ 1) r = mulmod(r, a, m);
    return r;
}
// Deterministic for all n < 3.3e24 with these 7
  ↪ bases (Miller-Rabin).
bool isPrime(u64 n) {
    if (n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
    u64 d = n - 1; int s = __builtin_ctzll(d); d >>=
  ↪ s;
    for (u64 a : {2, 325, 9375, 28178, 450775,
  ↪ 9780504, 1795265022}) {
        u64 p = powmod(a % n, d, n); int i = s;
        while (p != 1 && p != n - 1 && a % n && i--)
  ↪ p = mulmod(p, p, n);
        if (p != n - 1 && i != s) return false;
    }
    return true;
}
u64 pollard(u64 n) {
  ↪   // returns a non-trivial divisor (Brent)
    auto f = [n](u64 x) { return mulmod(x, x, n) + 1;
  ↪ };
    u64 x = 0, y = 0, t = 30, prd = 2, i = 1, q;
    while (t++ % 40 || gcd(prd, n) == 1) {
        if (x == y) x = ++i, y = f(x);
        if ((q = mulmod(prd, x > y ? x - y : y - x,
  ↪ n))) prd = q;
        x = f(x), y = f(f(y));
    }
    return gcd(prd, n);
}
vector<u64> factor(u64 n) {
  ↪   // unsorted multiset of primes
    if (n <= 1) return {};
  ↪   // guard: factor(0) would loop forever
    if (isPrime(n)) return {n};
    u64 x = pollard(n);
    auto l = factor(x), r = factor(n / x);
    l.insert(l.end(), all(r));
    return l;
}
// {prime, exponent} pairs, sorted
vector<pair<u64, int>> factorize(u64 n) {
    auto f = factor(n); sort(all(f));
    vector<pair<u64, int>> r;
    for (u64 p : f) (!r.empty() && r.back().first ==
  ↪ p) ? r.back().second++ : (r.push_back({p, 1}),
  ↪ 0);
    return r;
}
vector<u64> divisors(u64 n) {
  ↪   // all divisors, unsorted
    vector<u64> d = {1};
    for (auto [p, e] : factorize(n)) {
        int sz = d.size();
        for (u64 pe = 1, k = 0; k < (u64)e; k++) {
```

```
            pe *= p;
            for (int i = 0; i < sz; i++)
  ↪ d.push_back(d[i] * pe);
        }
    }
    return d;
}
u64 phi(u64 n) { u64 r = n; for (auto [p, e] :
  ↪ factorize(n)) r -= r / p; return r; }      //
  ↪ Euler totient
u64 tau(u64 n) { u64 r = 1; for (auto [p, e] :
  ↪ factorize(n)) r *= e + 1; return r; }      //
  ↪ #divisors
u64 sigma(u64 n) {
  ↪                                     // sum of
  ↪ divisors
    u64 r = 1;
    for (auto [p, e] : factorize(n)) { u64 t = 1, pe
  ↪ = 1; for (int i = 0; i < e; i++) t += (pe *= p);
  ↪ r *= t; }
    return r;
}
```

4.2.7 MultiplicativeSieve

Linear sieve computing ANY multiplicative function for all i <= n in O(N).

```
// Shipped: spf, phi, mu, tau (#divisors), sigma (sum
  ↪ of divisors).
// To add your own f: you need f(p), f(p^k) and
  ↪ f(a*b)=f(a)f(b) for coprime a,b.
struct MultSieve {
    int n;
    vector<int> spf, primes, phi, mu, tau, cnt;
  ↪   // cnt[i] = exponent of spf[i] in i
    vector<ll> sigma, pw;
  ↪   // pw[i] = spf[i]^cnt[i]

    MultSieve(int n) : n(n), spf(n + 1), phi(n + 1),
  ↪ mu(n + 1), tau(n + 1), cnt(n + 1),
                       sigma(n + 1), pw(n + 1) {
        phi[1] = mu[1] = tau[1] = sigma[1] = pw[1] =
  ↪ 1;
        for (int i = 2; i <= n; i++) {
            if (!spf[i]) {
                spf[i] = i, primes.push_back(i);
                phi[i] = i - 1, mu[i] = -1, tau[i] =
  ↪ 2, cnt[i] = 1, pw[i] = i, sigma[i] = i + 1LL;
            }
            for (int p : primes) {
                if (p > spf[i] || (ll)i * p > n)
  ↪ break;
                int j = i * p;
                spf[j] = p;
                if (p == spf[i]) {
  ↪   // p divides i: same prime, exponent grows
                    cnt[j] = cnt[i] + 1, pw[j] =
  ↪ pw[i] * p;
                    phi[j] = phi[i] * p;
                    mu[j] = 0;
                    tau[j] = tau[i] / (cnt[i] + 1) *
  ↪ (cnt[j] + 1);
                    sigma[j] = sigma[i] / ((pw[i] * p
  ↪ - 1) / (p - 1)) * ((pw[j] * p - 1) / (p - 1));
                } else {
  ↪   // coprime: multiply
                    cnt[j] = 1, pw[j] = p;
                    phi[j] = phi[i] * (p - 1);
                    mu[j] = -mu[i];
                    tau[j] = tau[i] * 2;
                    sigma[j] = sigma[i] * (p + 1);
                }
            }
        }
    }
    vector<int> factors(int x) const {
  ↪   // O(log x) via spf
        vector<int> f;
        while (x > 1) f.push_back(spf[x]), x /=
  ↪ spf[x];
        return f;
    }
};
```

4.2.8 PrimeEnumeration1e9

ENUMERATE EVERY PRIME UP TO 1e9 IN ~0.5s, and in a 30-smooth wheel that costs 1 BIT per 30

```
// integers (so ~33 MB at n = 1e9 versus ~60 MB for
  ↪ the odds-only bitset in 04 - Sieve1e9.cpp).
// Credit: min_25. Use THIS when you need the primes
  ↪ themselves at n >= 1e8; use 04 when you only
// need the primality bitmap and want six lines
  ↪ instead of sixty.
// HOW IT IS FAST: (1) a mod-30 wheel - of the 30
  ↪ residues only {1,7,11,13,17,19,23,29} can be
// prime, so one byte covers 30 integers; (2) the
  ↪ pattern of the small primes (<= Q) is computed
// ONCE into ‘pre‘ and memcpy’d over each block; (3)
  ↪ the sieve runs block by block so everything
// stays in L1/L2 cache, with each prime remembering
  ↪ where it left off in each of the 8 residues.
vector<int> sievePrimes(const int N, const int Q =
  ↪ 17, const int L = 1 << 15) {
    static const int rs[] = {1, 7, 11, 13, 17, 19,
  ↪ 23, 29};
    struct P { P(int p) : p(p) {} int p; int pos[8];
  ↪ };
    auto approxPrimeCount = [](const int N) -> int {
        return N > 60184 ? N / (log(N) - 1.1) :
  ↪ max(1., N / (log(N) - 1.11)) + 1;
    };
    const int v = sqrt(N), vv = sqrt(v);
    vector<bool> isp(v + 1, true);
    for (int i = 2; i <= vv; ++i) if (isp[i])
        for (int j = i * i; j <= v; j += i) isp[j] =
  ↪ false;
    const int rsize = approxPrimeCount(N + 30);
    vector<int> primes = {2, 3, 5};
    int psize = 3;
    primes.resize(rsize);
    vector<P> sp;
    size_t pbeg = 0;
    int prod = 1;
    for (int p = 7; p <= v; ++p) {
        if (!isp[p]) continue;
        if (p <= Q) prod *= p, ++pbeg,
  ↪ primes[psize++] = p;
        auto pp = P(p);
        for (int t = 0; t < 8; ++t) {
            int j = (p <= Q) ? p : p * p;
            while (j % 30 != rs[t]) j += p << 1;
            pp.pos[t] = j / 30;
        }
        sp.push_back(pp);
    }
    vector<unsigned char> pre(prod, 0xFF);
  ↪           // the small primes’ pattern, once
```

```
    for (size_t pi = 0; pi < pbeg; ++pi) {
        auto pp = sp[pi];
        const int p = pp.p;
        for (int t = 0; t < 8; ++t) {
            const unsigned char m = ~(1 << t);
            for (int i = pp.pos[t]; i < prod; i += p)
↪ pre[i] &= m;
        }
    }
    const int bs = (L + prod - 1) / prod * prod;
    vector<unsigned char> block(bs);
    unsigned char* pb = block.data();
    const int M = (N + 29) / 30;
    for (int beg = 0; beg < M; beg += bs, pb -= bs) {
        int end = min(M, beg + bs);
        for (int i = beg; i < end; i += prod)
↪ copy(pre.begin(), pre.end(), pb + i);
        if (beg == 0) pb[0] &= 0xFE;
↪           // 1 is not prime
        for (size_t pi = pbeg; pi < sp.size(); ++pi) {
            auto& pp = sp[pi];
            const int p = pp.p;
            for (int t = 0; t < 8; ++t) {
                int i = pp.pos[t];
                const unsigned char m = ~(1 << t);
                for (; i < end; i += p) pb[i] &= m;
                pp.pos[t] = i;
↪               // resume here in the next block
            }
        }
        for (int i = beg; i < end; ++i)
            for (int m = pb[i]; m > 0; m &= m - 1)
                primes[psize++] = i * 30 +
↪ rs[__builtin_ctz(m)];
    }
    while (psize > 0 && primes[psize - 1] > N)
↪ --psize;
    primes.resize(psize);
    return primes;
}
// pi(1e9) = 50847534, so the output vector alone is
↪ ~200 MB of int - if you only need to ITERATE
// the primes, sieve block by block and consume them
↪ inside the loop instead of storing them.
// Q = 17 means the wheel pre-pattern covers
↪ 7*11*13*17 = 17017 bytes; raising Q makes 'pre'
// bigger but the inner loop shorter. L = 1<<15 keeps
↪ a block inside L1.
// FOR PRIMALITY OF ONE NUMBER up to 1e18 use
↪ Miller-Rabin (06); for the FACTORISATION of one
// number use Pollard rho (06); for pi(n) without the
↪ primes use Lucy_Hedgehog (04 - Theorems/03).
```

4.2.9 Min25Sieve

MIN_25 SIEVE - prefix sum of ANY multiplicative f in O(n^(3/4) / log n), n up to ~1e10-1e11.

```
// Needs: Mint (06 - Math/01 - Modular Arithmetic/01
↪ - Mint.cpp).
// You must supply two things:
//   c[]  : f(p) written as a POLYNOMIAL in p, f(p) =
↪ c[0] + c[1]*p + c[2]*p^2 (degree <= 2 here,
//          extend pref() with more power sums for
↪ higher degree),
//   fpe  : f(p^e) for a prime power, given (p, e,
↪ p^e).
// PHASE 1 sieves sum of p^k over primes <= v for
↪ every v = floor(n/i) (this is exactly Lucy's
// method, 04 - Theorems/03 - PrimeCounting.cpp).
↪ PHASE 2 (S) walks every "smallest prime factor"
// class and re-adds the prime-power contributions.
↪ Memory O(sqrt n * deg).
// EXAMPLES: phi -> c = {-1, 1}, fpe = p^e - p^(e-1);
↪   mu -> c = {-1}, fpe = (e == 1 ? -1 : 0);
//           tau -> c = {2}, fpe = e + 1;
↪   Id -> c = {0, 1}, fpe = p^e;
//           f(p^e) = p^e * (p^e - 1) -> c = {0, -1,
↪ 1} (that is f(p) = p^2 - p).
// If f is COMPLETELY multiplicative, phase 1 alone
↪ is the answer (use Lucy directly).
struct Min25 {
    ll n, rt;
    int np;
    vector<ll> pr, w;
↪   // primes <= sqrt(n); values floor(n/i)
    vector<int> id1, id2;
↪   // position of v in w, for v <= rt / v > rt
    vector<vector<Mint>> g, sp;
↪   // g[k][j] over w[j]; sp[k][i] = prefix p^k
    vector<Mint> c;
    function<Mint(ll, int, ll)> fpe;

    int id(ll x) { return x <= rt ? id1[x] : id2[n /
↪ x]; }
    Mint pref(ll x, int k) {
↪   // sum_{i=2..x} i^k  (mod MOD)
        Mint z = x % MOD;
        if (k == 0) return z - 1;
        if (k == 1) return z * (z + 1) / 2 - 1;
        return z * (z + 1) * (2 * z + 1) / 6 - 1;
    }
    Min25(ll n, vector<Mint> c, function<Mint(ll,
↪ int, ll)> fpe) : n(n), c(c), fpe(fpe) {
        rt = (ll)sqrtl((ld)n);
        while (rt * rt > n) rt--;
        while ((rt + 1) * (rt + 1) <= n) rt++;
        int d = c.size();
        vector<char> comp(rt + 2, 0);
        for (ll i = 2; i <= rt; i++)
            if (!comp[i]) { pr.push_back(i); for (ll
↪ j = i * i; j <= rt; j += i) comp[j] = 1; }
        np = pr.size();
        sp.assign(d, vector<Mint>(np + 1, 0));
        for (int i = 0; i < np; i++) {
            Mint t = 1;
            for (int k = 0; k < d; k++) sp[k][i + 1]
↪ = sp[k][i] + t, t *= pr[i] % MOD;
        }
        id1.assign(rt + 2, 0), id2.assign(rt + 2, 0);
        for (ll i = 1, j; i <= n; i = j + 1) {
↪   // w is DECREASING: w[0] = n, back = 1
            ll x = n / i; j = n / x;
            w.push_back(x);
            (x <= rt ? id1[x] : id2[n / x]) =
↪ w.size() - 1;
        }
        g.assign(d, vector<Mint>(w.size()));
        for (int k = 0; k < d; k++)
            for (int j = 0; j < (int)w.size(); j++)
↪ g[k][j] = pref(w[j], k);
        for (int i = 0; i < np; i++)
↪   // sieve out multiples of pr[i]
            for (int j = 0; j < (int)w.size() &&
↪ pr[i] <= w[j] / pr[i]; j++) {
                int u = id(w[j] / pr[i]);
                Mint pk = 1;
                for (int k = 0; k < d; k++) g[k][j]
```

```
    -= pk * (g[k][u] - sp[k][i]), pk *= pr[i] % MOD;
            }
    }
    Mint S(ll x, int j) {
    // sum of f(i), i <= x, spf(i) > pr[j-1]
        if (j >= 1 && pr[j - 1] >= x) return 0;
        Mint res = 0;
        for (int k = 0; k < (int)c.size(); k++) res
    += c[k] * (g[k][id(x)] - sp[k][j]);
        for (int i = j; i < np && pr[i] <= x / pr[i];
    i++) {
            ll pw = pr[i];
            for (int e = 1; pw <= x; e++) {
                res += fpe(pr[i], e, pw) * (S(x / pw,
    i + 1) + Mint(e != 1));
                if (pw > x / pr[i]) break;
                pw *= pr[i];
            }
        }
        return res;
    }
    Mint solve() { return n < 1 ? Mint(0) : S(n, 0) +
    1; }   // the +1 is f(1)
};
// The "+ Mint(e != 1)" term is f(p^e) counted for
    the number p^e itself (only for e >= 2, since
// e = 1 is already inside the prime sum). No
    memoisation is needed anywhere - that is the
    point.
// SIBLINGS: 10 - PowerfulNumberSieve.cpp is usually
    FASTER (O(sqrt n)) when you can find a g with
// g(p) = f(p) and an easy prefix sum; 04 -
    Theorems/05 - DuSieve.cpp is the O(n^(2/3)) tool
    when
// a Dirichlet partner makes the convolution
    telescope.
```

4.2.10 PowerfulNumberSieve

POWERFUL NUMBER SIEVE - prefix sum of a multiplicative f in O(sqrt n) calls to an easy prefix

```
// Needs: Mint (06 - Math/01 - Modular Arithmetic/01
    - Mint.cpp).
// sum. Faster and shorter than Min_25 whenever you
    can find a helper g with
//   (1) g(p) = f(p) for every prime p, and (2) the
    prefix sum G(x) = sum_{i<=x} g(i) is cheap.
// Then f = g * h (Dirichlet), and h(p) = f(p) - g(p)
    = 0, so h is supported ONLY on POWERFUL
// numbers (every exponent >= 2) - and there are only
    O(sqrt n) of those below n. Hence
//   F(n) = sum_{d powerful, d <= n} h(d) * G(n / d),
// which this DFS evaluates. h(p^e) is derived on the
    fly from f and g by the convolution
// recurrence h(p^e) = f(p^e) - sum_{i<0..e-1} h(p^i)
    * g(p^(e-i)), so you never derive it by hand.
// PRECONDITION: g(p) = f(p) for ALL primes - check
    it, the algorithm is silently wrong otherwise.
// COST: O(sqrt n) nodes, each one call to G. With G
    in O(1) that is O(sqrt n) total; with G in
// O(sqrt x) it is still about O(n^(2/3)). Needs
    primes up to sqrt(n) (a plain sieve).
// EXAMPLE: f(p^e) = p^e + [e >= 2] pairs with g = Id
    (G(x) = x(x+1)/2). f = "phi times something"
// style problems usually pair with g = Id or g = 1
    or g = sigma (G(x) = sum d*floor(x/d)).
struct PowerfulSieve {
    ll n;
    vector<ll> pr;
    // primes up to sqrt(n)
    function<Mint(ll, int)> f, g;
    // f(p^e), g(p^e), e >= 1
    function<Mint(ll)> G;
    // prefix sum of g
    PowerfulSieve(ll n, vector<ll> pr,
    function<Mint(ll, int)> f, function<Mint(ll,
    int)> g,
                  function<Mint(ll)> G) : n(n),
    pr(pr), f(f), g(g), G(G) {}
    Mint dfs(int i, ll m, Mint h) {
    // m = a powerful number, h = h(m)
        Mint res = h * G(n / m);
        for (ll lim = n / m; i < (int)pr.size() &&
    pr[i] <= lim / pr[i]; i++) {
            ll p = pr[i], pw = p * p;
            vector<Mint> hp = {1, 0};
    // h(p^0) = 1, h(p^1) = 0
            for (int e = 2; pw <= lim; e++) {
                Mint v = f(p, e);
                for (int j = 0; j < e; j++) v -=
    hp[j] * g(p, e - j);
                hp.push_back(v);
                res += dfs(i + 1, m * pw, h * v);
                if (pw > lim / p) break;
                pw *= p;
            }
        }
        return res;
    }
    Mint solve() { return dfs(0, 1, 1); }
};
// COUNTING POWERFUL NUMBERS is itself a useful fact:
    every powerful number is uniquely a^2*b^3
// with b squarefree, so there are about
    2.173*sqrt(n) of them below n.
// WHEN g IS HARD TO GUESS: try g = Id^k for the k
    with f(p) ~ p^k, then fix the difference; or
// fall back to 09 - Min25Sieve.cpp, which needs no
    partner function at all.
```

4.2.11 DivisorSummatory

DIVISOR SUMMATORY FUNCTION D(n) = sum_{i=1..n} tau(i) = sum_{i=1..n} floor(n/i)

```
// in ~O(n^(1/3)) instead of the usual O(sqrt n)
    divisor blocks. About 100 ms at n = 1e18.
// Reach for it when n is 1e14 or more (blocks are
    fine up to ~1e12), or when many queries make
// the sqrt too slow. The result exceeds 64 bits
    around n ~ 4e17, hence __int128.
// HOW: the hyperbola x*y = n is walked with a
    STERN-BROCOT search over slopes. Starting from
    the
// point nearest the diagonal, we repeatedly take the
    longest lattice segment of the current slope
// that stays under the hyperbola (counting the
    trapezoid under it in O(1)), and refine the slope
// by mediants when the curvature no longer allows
    it. Below y ~ n^(1/3) the slopes get shorter
// than one step, so the tail is finished with a
    plain loop.
__int128 divisorSummatory(unsigned long long n) {
    if (n == 0) return 0;
    auto out = [n](unsigned long long x, unsigned y)
    { return x * y > n; };   // above the curve?
    auto cut = [n](unsigned long long x, unsigned dx,
    unsigned dy) {
        return (__uint128_t)x * x * dy >=
```

```
    (__uint128_t)n * dx;                // slope too steep
    };
    unsigned long long sn = (unsigned long long)sqrtl((ld)n);
    while (sn * sn > n) sn--;
    while ((sn + 1) * (sn + 1) <= n) sn++;
    unsigned long long cn = (unsigned long long)powl((ld)n, 0.34L);          // ~ n^(1/3)
    unsigned long long x = n / sn;
    unsigned y = (unsigned)(n / x + 1);
    __int128 ret = 0;
    stack<pair<unsigned, unsigned>> st;
    st.emplace(1, 0), st.emplace(1, 1);
    // the Stern-Brocot stack of slopes
    while (true) {
        unsigned lx, ly;
        tie(lx, ly) = st.top(); st.pop();
        while (out(x + lx, y - ly)) {
        // walk as long as the segment fits
            ret += (__int128)x * ly + (__int128)(ly + 1) * (lx - 1) / 2;
            x += lx, y -= ly;
        }
        if (y <= cn) break;
        unsigned rx = lx, ry = ly;
        while (true) {
        // pop until a usable slope is on top
            tie(lx, ly) = st.top();
            if (out(x + lx, y - ly)) break;
            rx = lx, ry = ly, st.pop();
        }
        while (true) {
        // mediant refinement
            unsigned mx = lx + rx, my = ly + ry;
            if (out(x + mx, y - my)) st.emplace(lx = mx, ly = my);
            else { if (cut(x + mx, lx, ly)) break; rx = mx, ry = my; }
        }
    }
    for (--y; y > 0; --y) ret += n / y;
    // the small-y tail, plain loop
    return ret * 2 - (__int128)sn * sn;
    // Dirichlet hyperbola symmetry
}
// EXACT VALUES for testing: D(10) = 27, D(100) = 482, D(1e6) = 13970034, D(1e9) = 20951330018,
// D(1e18) = 312644024778133769 29. Asymptotically D(n) = n ln n + (2*gamma - 1) n + O(n^(1/3)).
// RELATED SUMS with the same blocks: sum sigma(i) = sum i*floor(n/i), and any sum_{i} f(n/i);
// for those, plain O(sqrt n) blocks (03 - FloorSum.cpp) are the right tool.
```

4.2.12 MeisselLehmer

MEISSEL-LEHMER PRIME COUNTING - pi(n) in about O(n^(2/3) / log n), well under a second at

```
// n = 1e12 and a few seconds at 1e14. Use it when you need pi(n) MANY times, or for n past the
// comfort zone of Lucy's O(n^(3/4)) method (04 - Theorems/03 - PrimeCounting.cpp) - but Lucy
// stays the better choice when you also want the SUM of the primes, which this cannot give.
// phi(n, k) = count of i <= n free of the first k primes; the recursion phi(n, k) =
// phi(n, k-1) - phi(n/p_k, k-1) is memoised in a table for small arguments, and Lehmer's
// identity peels off the numbers with exactly two or three prime factors.
// CALL init() ONCE. Memory: LIM ints for the prefix table plus PN*PK ints (about 30 MB with the
// constants below) - shrink PN/PK if the limit is tight, raise LIM if n is huge.
namespace PCF {
    const int LIM = 5000000, PN = 40000, PK = 60;
    // sieve bound; memo table dimensions
    vector<int> primes, pref;
    // pref[i] = pi(i) for i < LIM
    vector<vector<int>> memo;
    ll isqrt(ll x) { ll r = (ll)sqrtl((ld)x); while (r * r > x) r--; while ((r+1)*(r+1) <= x) r++; return r; }
    ll icbrt(ll x) { ll r = (ll)cbrtl((ld)x); while (r * r * r > x) r--; while ((r+1)*(r+1)*(r+1) <= x) r++; return r; }
    void init() {
        vector<bool> comp(LIM, false);
        pref.assign(LIM, 0);
        for (int i = 2; i < LIM; i++) {
            if (!comp[i]) { primes.push_back(i); for (ll j = (ll)i * i; j < LIM; j += i) comp[j] = 1; }
            pref[i] = primes.size();
        }
        memo.assign(PN, vector<int>(PK));
        for (int i = 0; i < PN; i++) memo[i][0] = i;
        for (int k = 1; k < PK; k++)
            for (int i = 0; i < PN; i++)
                memo[i][k] = memo[i][k - 1] - memo[i / primes[k - 1]][k - 1];
    }
    ll phi(ll n, int k) {                                   // count i <= n free of the first k primes
        if (n < PN && k < PK) return memo[n][k];
        if (k == 1) return (n + 1) >> 1;
        if (primes[k - 1] >= n) return 1;
        return phi(n, k - 1) - phi(n / primes[k - 1], k - 1);
    }
    ll legendre(ll n) {
        // simplest version, O(n^(2/3) log n)-ish
        if (n < LIM) return pref[n];
        int k = pref[isqrt(n)];
        return phi(n, k) + k - 1;
    }
    ll lehmer(ll n) {
        // the fast one
        if (n < LIM) return pref[n];
        ll b = isqrt(n), c = lehmer(icbrt(n)), a = lehmer(isqrt(b));
        b = lehmer(b);
        ll res = phi(n, a) + (b + a - 2) * (b - a + 1) / 2;
        for (ll i = a; i < b; i++) {
            ll w = n / primes[i];
            res -= lehmer(w);
            if (i <= c) {
                ll lim = lehmer(isqrt(w));
                for (ll j = i; j < lim; j++) res += j - lehmer(w / primes[j]);
            }
        }
        return res;
    }
```

```
}
// SANITY VALUES: pi(1e6) = 78498, pi(1e9) =
  ↪ 50847534, pi(1e12) = 37607912018,
//                pi(1e13) = 346065536839, pi(1e15) =
  ↪ 29844570422669.
// pi(n) ~ n / ln n, and li(n) is a much better
  ↪ estimate; the k-th prime is ~ k(ln k + ln ln k).
// NEXT STEPS: the Lagarias-Miller-Odlyzko and
  ↪ Deleglise-Rivat refinements reach n^(2/3)/log^2
  ↪ and
// are what a real prime-counting program uses - not
  ↪ worth writing during a contest.
```

4.2.13 CountKDivisors

HOW MANY x <= n HAVE EXACTLY k DIVISORS? n up to ~1e12, k up to a few thousand.

```
// Needs: PCF::lehmer, PCF::primes (12 -
  ↪ MeisselLehmer.cpp).
// A number with exactly k divisors has the shape
  ↪ p1^(e1-1) * p2^(e2-1) * ... where e1*e2*... = k
// and (choosing the exponents in decreasing order)
  ↪ the primes can be taken increasing. So we
// backtrack over the FACTORISATIONS of k into
  ↪ factors > 1, assigning a prime to each factor.
// Two things make it fast: (a) the case k = 2 is
  ↪ just "count the primes <= n", answered by
// Meissel-Lehmer, and (b) a single remaining factor
  ↪ x means "count primes p with p^(x-1) <= n",
// answered by a binary search over the prime list.
// PRECONDITIONS: call PCF::init() first, and prep()
  ↪ once for the k range you need. Sieve limit
// LIM in PCF must exceed n^(1/2) for the binary
  ↪ searches to be safe (5e6 covers n <= 2.5e13).
// COMPLEXITY: hard to state - it is a search whose
  ↪ branching is bounded by the divisors of k;
// in practice a handful of milliseconds per query
  ↪ for n = 1e12.
namespace KDiv {
    const int MK = 2100;
  ↪   // max k + 1
    vector<vector<int>> dv;
  ↪   // dv[k] = divisors of k that are > 1, desc
    vector<int> minexp;
  ↪   // minexp[k] = sum (p-1) over primes p | k
    void prep() {
        dv.assign(MK, {}), minexp.assign(MK, 0);
        for (int i = 2; i < MK; i++) {
            for (int j = i; j < MK; j += i)
  ↪ dv[j].push_back(i);
            for (int p : PCF::primes) {
                if (p > i) break;
                int cur = i;
                while (cur % p == 0) cur /= p,
  ↪ minexp[i] += p - 1;
            }
        }
        for (int i = 2; i < MK; i++)
  ↪ reverse(all(dv[i]));   // try the big exponents
  ↪ first
    }
    ll powCap(ll b, int e, ll cap) {
  ↪   // b^e, saturating at cap + 1
        ll r = 1;
        for (int i = 0; i < e; i++) { if (r > cap /
  ↪ b) return cap + 1; r *= b; }
        return r;
    }
    // count of x <= n with exactly k divisors, using
  ↪ only primes from index ‘first‘ on
    ll go(ll n, int k, int first = 0) {
        if (n <= 0 || k >= MK) return 0;
        if (k == 1) return 1;
        if (n == 1) return 0;
        if (k == 2) return max(0LL, PCF::lehmer(n) -
  ↪ first);
        if (dv[k].size() == 1) {
  ↪   // k is prime: one prime power p^(k-1)
            int e = dv[k][0] - 1;
            if (powCap(PCF::primes[first], e, n) > n)
  ↪ return 0;
            int lo = first, hi =
  ↪ (int)PCF::primes.size() - 1, ans = first - 1;
            while (lo <= hi) {
  ↪   // last prime with p^e <= n
                int mid = (lo + hi) / 2;
                if (powCap(PCF::primes[mid], e, n) >
  ↪ n) hi = mid - 1;
                else lo = mid + 1, ans = mid;
            }
            return ans - first + 1;
        }
        ll res = 0;
        for (int x : dv[k])
  ↪   // this prime carries exponent x - 1
            for (int i = first; i <
  ↪ (int)PCF::primes.size(); i++) {
                ll p = powCap(PCF::primes[i], x - 1,
  ↪ n);
                if (p > n) break;
                if (k / x == 1) { res++; continue; }
                if (powCap(PCF::primes[i + 1],
  ↪ minexp[k / x], n) > n / p) break;
                res += go(n / p, k / x, i + 1);
            }
        return res;
    }
}
// RANGE QUERY: answer(l, r) = go(r, k) - go(l - 1,
  ↪ k).
// SMALL n: just sieve tau with a linear sieve (07 -
  ↪ MultiplicativeSieve.cpp) and count.
// SMALLEST number with exactly k divisors is a
  ↪ different search - 15 -
  ↪ SmallestWithKDivisors.cpp.
// FACTS: tau(x) is odd iff x is a perfect square;
  ↪ the count of x <= n with tau(x) = 2 is pi(n),
// with tau(x) = 3 it is pi(sqrt n), and with tau(x)
  ↪ = 4 it is pi(cbrt n) plus the number of
// products p*q of two distinct primes below n.
```

4.2.14 PrimeBasis

COPRIME (PRIME) BASIS - factor a set of numbers RELATIVE TO EACH OTHER, without ever factoring

```
// them. Given n values up to 1e18 (far past
  ↪ Pollard-rho comfort when n is large), it builds
  ↪ a set
// of PAIRWISE COPRIME basis elements such that every
  ↪ input is a product of basis powers. Then
// each input becomes an exponent vector, and gcd /
  ↪ lcm / divisibility / "product is a square" all
// become coordinatewise min / max / <= on those
  ↪ vectors.
// WHEN: "is a_i | prod a_j", "make all numbers equal
  ↪ by moving prime factors", "count subsets
// whose product is a perfect k-th power", any
  ↪ problem where only the RELATIVE factorisations of
```

```
// the given numbers matter and the numbers are too big to factor.
// SIZE: at most n * log log(max a) basis elements; build is O(n^2 log(max a)) worst case, and
// far less in practice. factorize(x) is O(|basis| * log x).
// NOTE the basis elements are not necessarily prime - they are only pairwise coprime, which is
// all that matters. They ARE prime whenever the inputs force it.
template <typename T>
struct PrimeBasis {
    vector<T> basis;
    void reducePair(T& x, T& y) {
        // strip the common part greedily
        bool sw = 0;
        if (x > y) sw ^= 1, swap(x, y);
        while (x > 1 && y % x == 0) { y /= x; if (x > y) sw ^= 1, swap(x, y); }
        if (sw) swap(x, y);
    }
    void split(int pos, T& val) {
        // basis[pos] and val share a factor
        while (basis[pos] % val == 0) basis[pos] /= val;
        vector<T> cur = {basis[pos], val};
        for (int ptr = 1; ptr < (int)cur.size(); ptr++)
            for (int i = 0; i < ptr; i++) {
                reducePair(cur[i], cur[ptr]);
                if (cur[ptr] == 1 || cur[i] == 1) break;
                T g = gcd(cur[ptr], cur[i]);
                if (g > 1) cur[ptr] /= g, cur[i] /= g, cur.push_back(g);
            }
        basis[pos] = cur[0], val = cur[1];
        for (int i = 2; i < (int)cur.size(); i++) if (cur[i] > 1) basis.push_back(cur[i]);
        if (basis[pos] == 1) swap(basis[pos], basis.back()), basis.pop_back();
    }
    void add(T val) {
        for (int i = 0; i < (int)basis.size(); i++) {
            reducePair(val, basis[i]);
            if (basis[i] == 1) { swap(basis[i], basis.back()); basis.pop_back(); break; }
            if (val == 1) return;
            if (gcd(basis[i], val) > 1) split(i, val);
        }
        if (val > 1) basis.push_back(val);
    }
    vector<int> factorize(T x) {
        // exponent vector of x over the basis
        vector<int> f(basis.size());
        for (int i = 0; i < (int)basis.size(); i++) while (x % basis[i] == 0) f[i]++, x /= basis[i];
        return f;
    }
};
// AFTER BUILDING: add() EVERY input first, then factorize() them - the basis must be final, or
// old vectors have the wrong length. Verify with: every factorize(a_i) must multiply back to a_i
// (i.e. dividing a_i by all basis powers leaves 1), and all pairs of basis elements are coprime.
// The vectors support: gcd = coordinatewise min, lcm = max, a | b iff vec(a) <= vec(b)
// coordinatewise, a*b = sum, a/b = difference, x is a perfect square iff all coordinates are even.
```

4.2.15 SmallestWithKDivisors

THE SMALLEST NUMBER WITH EXACTLY k DIVISORS. The answer explodes (k = 1e9 already needs ~100

```
// Needs: powmod (06 - MillerRabinPollard.cpp), divisors of k (03 - Divisors.cpp or factorize).
// digits), so it is compared by the SUM OF LOGARITHMS and reported modulo MOD.
// Structure: x = p_0^(e_0) * p_1^(e_1) * ... over the FIRST primes in order, with
// (e_0+1)(e_1+1)... = k and e_0 >= e_1 >= ... (swapping a larger exponent onto a smaller prime
// never hurts). So: walk the primes in order, and at prime i choose a divisor x | k for the
// factor (e_i + 1) = x. Memoise on (prime index, remaining k).
// PRECONDITIONS: pr must hold the first ~64 primes (2, 3, 5, ...); dvs = all divisors of k that
// are > 1, ASCENDING (that order is what makes the break-early pruning valid). k >= 1.
// COMPLEXITY: O(#states * tau(k)) with tiny constants - milliseconds for k up to 1e12.
struct SmallestKDiv {
    vector<ll> pr, dvs;
    vector<ld> lg;
    vector<map<ll, pair<ld, ll>>> memo;
    // memo[i][rem] = {log of x, x mod MOD}
    SmallestKDiv(vector<ll> pr, vector<ll> dvs) : pr(pr), dvs(dvs) {
        for (ll p : pr) lg.push_back(logl((ld)p));
        memo.assign(pr.size() + 1, {});
    }
    pair<ld, ll> go(int i, ll rem) {
        // use primes pr[i], pr[i+1], ... only
        if (rem == 1) return {0.0L, 1};
        auto it = memo[i].find(rem);
        if (it != memo[i].end()) return it->second;
        pair<ld, ll> best = {1e30L, 0};
        for (ll x : dvs) {
            if (rem % x) continue;
            ld z = lg[i] * (ld)(x - 1);
            // cost of putting exponent x-1 on pr[i]
            if (z > best.first) break;
            // dvs ascending => all later are worse
            auto cur = go(i + 1, rem / x);
            cur.first += z;
            cur.second = cur.second * powmod(pr[i], x - 1, MOD) % MOD;
            best = min(best, cur);
        }
        return memo[i][rem] = best;
    }
    ll solve(ll k) { return k == 1 ? 1 : go(0, k).second; }
};
// EXACT SMALL ANSWERS (for a sanity check): k = 1,2,3,4,5,6,7,8,9,10 -> 1, 2, 4, 6, 16, 12, 64,
// 24, 36, 48. k = 100 -> 45360. The sequence is OEIS A005179.
// LONG DOUBLE is enough here: two candidates tie only when the numbers are equal, and the log
// sums differ by more than 1e-9 otherwise for any k that fits in 64 bits. If paranoid, compare
// the exponent vectors exactly with big integers, or compare logs and break ties by the vectors.
// RELATED: the MAXIMUM number of divisors below n
```

```
  ↪ (highly composite numbers) is found by the
// same DFS with the objective flipped - keep the
  ↪ primes in order and try e_i = 0..60 with
// e_0 >= e_1 >= ...; tau maxes at 1344 for n <= 1e9,
  ↪ 6720 for 1e12, 103680 for 1e18.
```

4.3 Modular Arithmetic

4.3.1 CRT

```
// NOTE: extGCD below is the same one as in 01 -
  ↪ Euclidean Algorithm. Paste only one copy.
long long extGCD(long long a, long long b, long long
  ↪ &x, long long &y) {
    if (b == 0) {
        x = 1; y = 0;
        return a;
    }
    long long x1, y1;
    long long d = extGCD(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return d;
}

// Solves x = a_i (mod m_i). Returns {x, lcm(m_1,
  ↪ ..., m_k)}.
pair<long long, long long> CRT(const vector<long
  ↪ long>& a, const vector<long long>& m) {
    long long res = a[0], lcm = m[0];
    for (int i = 1; i < a.size(); i++) {
        long long x, y;
        long long g = extGCD(lcm, m[i], x, y);
        if ((a[i] - res) % g != 0) return {-1, -1};
  ↪ // No solution

        long long step = m[i] / g;
        long long k = ((a[i] - res) / g) % step * (x
  ↪ % step) % step;
        res = (long long)(((__int128)k * lcm + res) %
  ↪ ((__int128)lcm * step));
        lcm *= step;
        res = (res % lcm + lcm) % lcm;
    }
    return {res, lcm};
}
```

4.3.2 DiscreteLogAndRoots

Discrete log, modular square root, primitive root, discrete k-th root, multiplicative order.

```
// Needs: powmod (02 - Primes and Divisors/06 -
  ↪ MillerRabinPollard.cpp).
// All need mulmod/powmod from MillerRabinPollard.

// BSGS: smallest x >= 0 with a^x = b (mod m). Works
  ↪ for ANY m (handles gcd(a,m) != 1). O(sqrt m).
ll discreteLog(ll a, ll b, ll m) {
    a %= m, b %= m;
    ll k = 1, add = 0, g;
    while ((g = gcd(a, m)) > 1) {
  ↪ // strip common factors
        if (b == k) return add;
        if (b % g) return -1;
        b /= g, m /= g, add++, k = k * (a / g) % m;
    }
    ll n = (ll)sqrtl((ld)m) + 1, an = 1;
    for (ll i = 0; i < n; i++) an = an * a % m;
    unordered_map<ll, ll> vals;
    for (ll q = 0, cur = b; q <= n; q++) vals[cur] =
  ↪ q, cur = cur * a % m;
    for (ll p = 1, cur = k; p <= n; p++) {
        cur = cur * an % m;
        if (vals.count(cur)) { ll ans = n * p -
  ↪ vals[cur] + add; if (ans >= 0) return ans; }
    }
    return -1;
}

// Tonelli-Shanks: x with x^2 = a (mod p), p odd
  ↪ prime. Returns -1 if a is not a QR.
ll sqrtMod(ll a, ll p) {
    a %= p; if (a < 0) a += p;
    if (a == 0) return 0;
    if (powmod(a, (p - 1) / 2, p) != 1) return -1;
  ↪   // Euler's criterion
    if (p % 4 == 3) return powmod(a, (p + 1) / 4, p);
    ll s = p - 1, n = 2; int r = 0;
    while (s % 2 == 0) s /= 2, r++;
    while (powmod(n, (p - 1) / 2, p) != p - 1) n++;
  ↪   // find a non-residue
    ll x = powmod(a, (s + 1) / 2, p), b = powmod(a,
  ↪ s, p), g = powmod(n, s, p);
    for (int m = r;; m = r) {
        ll t = b;
        for (r = 0; r < m && t != 1; r++) t = t * t %
  ↪ p;
        if (r == 0) return x;
        ll gs = powmod(g, 1LL << (m - r - 1), p);
        g = gs * gs % p, x = x * gs % p, b = b * g %
  ↪ p;
    }
}

// Smallest primitive root of a PRIME p (generator of
  ↪ Z_p^*).
ll primitiveRoot(ll p) {
    if (p == 2) return 1;
    auto f = factorize(p - 1);
    for (ll g = 2; g < p; g++) {
        bool ok = true;
        for (auto [q, e] : f) if (powmod(g, (p - 1) /
  ↪ q, p) == 1) { ok = false; break; }
        if (ok) return g;
    }
    return -1;
}

// Multiplicative order of a mod m (smallest e > 0
  ↪ with a^e = 1). Requires gcd(a, m) = 1.
ll multOrder(ll a, ll m) {
    ll e = phi(m), r = e;
    for (auto [q, k] : factorize(e))
        while (r % q == 0 && powmod(a, r / q, m) ==
  ↪ 1) r /= q;
    return r;
}

// All x with x^k = a (mod p), p prime. Reduce to a
  ↪ discrete log in base g.
vector<ll> discreteRoot(ll k, ll a, ll p) {
    if (a == 0) return {0};
    ll g = primitiveRoot(p), y =
  ↪ discreteLog(powmod(g, k, p), a, p);
    if (y < 0) return {};
    ll x0 = powmod(g, y, p), d = gcd(k, p - 1), step
  ↪ = powmod(g, (p - 1) / d, p);
    vector<ll> r;
```

```
    for (ll i = 0, cur = x0; i < d; i++, cur = cur *
  ↪ step % p) r.push_back(cur);
    sort(all(r)), r.erase(unique(all(r)), r.end());
    return r;
}
```

4.3.3 LucasAndBinomialMod

C(n, r) mod m for HUGE n. Pick by what m is:

```
// Needs: Mint (06 - Math/01), and factorize (02 -
  ↪ Primes and Divisors/06) for the last section.
//   m prime and small          -> lucas
//   m = p^e                    -> binomPrimePower
  ↪ (Andrew Granville / Wilson generalisation)
//   m arbitrary (squarefree-ish)-> binomAnyMod (CRT
  ↪ over prime powers)

// LUCAS. p prime, needs fact/invFact built up to p.
  ↪ C(n,r) = prod C(n_i, r_i) over base-p digits.
Mint lucas(ll n, ll r) {
    if (r < 0 || r > n) return 0;
    Mint res = 1;
    while (n || r) {
        ll a = n % MOD, b = r % MOD;
        if (b > a) return 0;
        res *= fact[a] * invFact[b] * invFact[a - b];
        n /= MOD, r /= MOD;
    }
    return res;
}

// --- arbitrary modulus ---
ll pw(ll a, ll e, ll m) { ll r = 1; a %= m; for (; e;
  ↪ e >>= 1, a = a * a % m) if (e & 1) r = r * a %
  ↪ m; return r; }
ll inv_mod(ll a, ll m) {
  ↪    // requires gcd(a, m) = 1
    ll g = m, x = 0, x1 = 1, a1 = a % m;
    while (a1) { ll q = g / a1; g -= q * a1, swap(g,
  ↪ a1); x -= q * x1, swap(x, x1); }
    return (x % m + m) % m;
}
// n! with all factors of p removed, mod p^e.  O(p^e
  ↪ + log n).
ll factNoP(ll n, ll p, ll pe) {
    if (n == 0) return 1;
    vector<ll> f(pe);
    f[0] = 1;
    for (ll i = 1; i < pe; i++) f[i] = (i % p ? f[i -
  ↪ 1] * i : f[i - 1]) % pe;
    ll r = 1;
    for (ll m = n; m; m /= p) {
        r = r * pw(f[pe - 1], m / pe, pe) % pe;
        r = r * f[m % pe] % pe;
    }
    return r;
}
// C(n, r) mod p^e
ll binomPrimePower(ll n, ll r, ll p, ll pe) {
    if (r < 0 || r > n) return 0;
    ll k = 0;
  ↪    // exponent of p in C(n,r) (Kummer's theorem:
    for (ll m = n / p, a = r / p, b = (n - r) / p; m;
  ↪ m /= p, a /= p, b /= p) k += m - a - b;
    if (k >= 60 || pw(p, k, pe * p) % pe == 0) { ll t
  ↪ = pw(p, k, pe); if (t == 0) return 0; }
    ll num = factNoP(n, p, pe), d1 = factNoP(r, p,
  ↪ pe), d2 = factNoP(n - r, p, pe);
    ll res = num * inv_mod(d1, pe) % pe * inv_mod(d2,
  ↪ pe) % pe;
    return res * pw(p, k, pe) % pe;
}
// C(n, r) mod m for arbitrary m: factor m, solve mod
  ↪ each p^e, CRT.
ll binomAnyMod(ll n, ll r, ll m) {
    ll res = 0, M = m;
    for (auto [p, e] : factorize(m)) {
        ll pe = 1; for (int i = 0; i < e; i++) pe *=
  ↪ p;
        ll c = binomPrimePower(n, r, p, pe), t = M /
  ↪ pe;
        res = (res + c % pe * t % M * inv_mod(t % pe,
  ↪ pe)) % M;
    }
    return res;
}
// KUMMER'S THEOREM: the exponent of prime p in
  ↪ C(n,r) = number of CARRIES when adding
//                   r and n-r in base p.  (Used
  ↪ above to pull out the p-part.)
// WILSON: (p-1)! = -1 (mod p) for prime p.
```

4.3.4 CongruencesAndTowers

Linear congruences and power towers. Needs powmod / phi / gcd.

```
// Needs: powmod, phi (04 - NT/02 - Primes and
  ↪ Divisors/06 - MillerRabinPollard.cpp).

// Solve a*x = b (mod m). Returns all solutions in
  ↪ [0, m) (there are gcd(a,m) of them, or none).
vector<ll> linCongruence(ll a, ll b, ll m) {
    a = ((a % m) + m) % m, b = ((b % m) + m) % m;
    ll g = gcd(a, m);
    if (b % g) return {};
    ll m2 = m / g, a2 = a / g, b2 = b / g;
    // a2 is invertible mod m2
    ll x0 = 0, x1 = 1, g2 = m2, a3 = a2 % m2;
    while (a3) { ll q = g2 / a3; g2 -= q * a3,
  ↪ swap(g2, a3); x0 -= q * x1, swap(x0, x1); }
    ll inv = ((x0 % m2) + m2) % m2, x = b2 % m2 * inv
  ↪ % m2;
    vector<ll> r;
    for (ll i = 0; i < g; i++) r.push_back((x + i *
  ↪ m2) % m);
    return r;
}

// GENERALISED EULER:  a^k = a^(k mod phi(m) +
  ↪ phi(m))  (mod m)  whenever k >= log2(m).
// This is what makes power towers computable even
  ↪ when gcd(a, m) != 1.
ll capTower(const vector<ll>& a, int i, ll CAP = 64)
  ↪ {   // min(true tower value, CAP)
    if (i == (int)a.size()) return 1;
    if (a[i] == 0) return capTower(a, i + 1, CAP) ==
  ↪ 0 ? 1 : 0;   // 0^0 = 1, 0^(k>0) = 0
    if (a[i] == 1) return 1;
    ll e = capTower(a, i + 1, CAP), r = 1;
    for (ll k = 0; k < e; k++) { r *= a[i]; if (r >=
  ↪ CAP) return CAP; }
    return r;
}
// a[i] ^ a[i+1] ^ ... ^ a[last]   (mod m),
  ↪ right-associative.
ll powTower(const vector<ll>& a, int i, ll m) {
    if (m == 1) return 0;
    if (i == (int)a.size()) return 1 % m;
    ll ph = phi(m), t = capTower(a, i + 1);
```

```
    ll e = t < 64 ? t : powTower(a, i + 1, ph) + ph;
  ↪    // exact when tiny, +phi when large
    return powmod(a[i] % m, e, m);
}

// Extended CRT for NON-coprime moduli: x = a1 (mod
  ↪ m1), x = a2 (mod m2).
// Returns {x, lcm} or {-1, -1}. Chain it to merge
  ↪ many congruences.
pair<ll, ll> crt2(ll a1, ll m1, ll a2, ll m2) {
    ll g = gcd(m1, m2), l = m1 / g * m2;
    if ((a2 - a1) % g) return {-1, -1};
    ll m2g = m2 / g;
    ll x0 = 0, x1 = 1, gg = m2g, aa = (m1 / g) % m2g;
    while (aa) { ll q = gg / aa; gg -= q * aa,
  ↪ swap(gg, aa); x0 -= q * x1, swap(x0, x1); }
    ll inv = ((x0 % m2g) + m2g) % m2g;
    __int128 k = (__int128)((a2 - a1) / g % m2g) *
  ↪ inv % m2g;
    ll x = (ll)(((__int128)k * m1 + a1) % l);
    return {(x % l + l) % l, l};
}
```

4.3.5 Garner

GARNER'S ALGORITHM - reconstruct x from residues modulo PAIRWISE COPRIME m_i, in mixed-radix

```
// form, so you can emit x modulo a target MOD
  ↪ without ever building prod(m_i).
// This is what CRT cannot do once the product
  ↪ overflows 64 bits.
//   x = x1 + x2*m1 + x3*m1*m2 + ... ,   0 <= xi < mi
ll garner(vector<ll> a, vector<ll> m, ll MODOUT) {
    int k = a.size();
    vector<ll> x(k);
    auto inv = [](ll a, ll M) {
  ↪ // requires gcd(a, M) = 1
        ll g = M, X = 0, x1 = 1, A = ((a % M) + M) %
  ↪ M;
        while (A) { ll q = g / A; g -= q * A, swap(g,
  ↪ A); X -= q * x1, swap(X, x1); }
        return (X % M + M) % M;
    };
    for (int i = 0; i < k; i++) {
        x[i] = ((a[i] % m[i]) + m[i]) % m[i];
        ll mul = 1;
        for (int j = 0; j < i; j++) {
            x[i] = (x[i] - x[j] % m[i] * mul) % m[i];
            mul = mul * (m[j] % m[i]) % m[i];
        }
        x[i] = (x[i] % m[i] + m[i]) % m[i] * inv(mul,
  ↪ m[i]) % m[i];
    }
    ll res = 0, mul = 1;
    for (int i = 0; i < k; i++) {
        res = (res + x[i] % MODOUT * mul) % MODOUT;
        mul = mul * (m[i] % MODOUT) % MODOUT;
    }
    return (res % MODOUT + MODOUT) % MODOUT;
}
// MAIN USE: arbitrary-modulus convolution. Run NTT
  ↪ under three NTT primes, then Garner the
// three residues down to the target modulus.
//   NTT PRIMES:  998244353 = 119*2^23 + 1  (g=3)
  ↪ 167772161 =   5*2^25 + 1  (g=3)
//                469762049 =   7*2^26 + 1  (g=3)
  ↪ 1004535809 = 479*2^21 + 1  (g=3)
//                754974721 =  45*2^24 + 1  (g=11)
  ↪ <- note: NOT 9*2^23+1, a common misprint
// The 3-prime product 5.95e25 covers n * (MOD-1)^2
  ↪ for any MOD < 2^30 and n up to ~5e7.
```

4.3.6 Montgomery

MONTGOMERY MULTIPLICATION - modular multiplication for a FIXED ODD 64-bit modulus without any

```
// hardware division: 2-3x faster than (__int128)a*b
  ↪ % m. Use it in the hot loop of Miller-Rabin,
// Pollard rho, BSGS, matrix power or any
  ↪ modpow-heavy routine with a 64-bit modulus. For a
// COMPILE-TIME modulus the compiler already turns %
  ↪ into a multiply-shift, so this buys nothing;
// the win is for a modulus known only at run time.
// TRICK: work with the representative a*R mod m
  ↪ where R = 2^64. reduce(x) computes x*R^-1 mod m
// with two multiplications and no division, so
  ↪ (aR)(bR)*R^-1 = (ab)R stays in the same form.
// PRECONDITIONS: m must be ODD (the Newton iteration
  ↪ inverts m modulo 2^64) and m < 2^63.
// Values returned by get() are in [0, m); mul/add
  ↪ work on the Montgomery form.
typedef unsigned long long u64;
  ↪    // same typedef as 02/06 - keep one copy
struct Mont {
    u64 mod, minv, r2;
  ↪    // minv = m^-1 mod 2^64
    void set(u64 m) {
        mod = minv = m;
        for (int i = 0; i < 5; i++) minv *= 2 - minv
  ↪ * m;       // Newton: doubles the bits each time
        r2 = (u64)(-(__uint128_t)m % m);
  ↪           // R^2 mod m
    }
    u64 reduce(__uint128_t x) const {
        u64 y = (u64)(x >> 64) -
  ↪ (u64)(((__uint128_t)((u64)x * minv) * mod) >>
  ↪ 64);
        return (long long)y < 0 ? y + mod : y;
    }
    u64 in(u64 a) const { return
  ↪ reduce((__uint128_t)(a % mod) * r2); }   // to
  ↪ Montgomery form
    u64 get(u64 a) const { u64 y = reduce(a); return
  ↪ y >= mod ? y - mod : y; }   // back to normal
    u64 mul(u64 a, u64 b) const { return
  ↪ reduce((__uint128_t)a * b); }
    u64 add(u64 a, u64 b) const { a += b - mod;
  ↪ return (long long)a < 0 ? a + mod : a; }
    u64 sub(u64 a, u64 b) const { a -= b; return
  ↪ (long long)a < 0 ? a + mod : a; }
    u64 pow(u64 a, u64 e) const {
  ↪    // a already in Montgomery form
        u64 r = in(1);
        for (; e; e >>= 1, a = mul(a, a)) if (e & 1)
  ↪ r = mul(r, a);
        return r;
    }
};
// USAGE:  Mont M; M.set(n);  u64 x = M.in(3);  x =
  ↪ M.pow(x, d);  cout << M.get(x);
// Do NOT mix forms: everything inside the loop stays
  ↪ in Montgomery form, converting only at the
// boundaries. Comparisons against 1 and m-1
  ↪ (Miller-Rabin) must compare against M.in(1) and
// M.in(m-1), which you compute once.
// EVEN MODULUS: split m = 2^k * odd, run Montgomery
  ↪ on the odd part and the trivial bit-mask on
// the power of two, then CRT. Usually not worth it -
```

```
  ↪ just use __int128.
// ALTERNATIVE: BARRETT reduction works for even
  ↪ moduli too (precompute floor(2^64/m) and replace
// the division by a multiply-shift); it is a little
  ↪ slower but has no parity restriction.
```

4.3.7 QuadraticExtension

ARITHMETIC IN F_p[sqrt(D)] = { a + b*sqrt(D) : a, b in F_p } - the quadratic extension field

```
// Needs: powmod (02 - Primes and Divisors/06 -
  ↪ MillerRabinPollard.cpp).
// when D is a NON-RESIDUE mod p (then it is
  ↪ F_{p^2}), or a ring of "split" numbers when it
  ↪ is.
// WHAT IT BUYS YOU:
//   * CIPOLLA's square root mod p: always O(log p)
  ↪ multiplications, and unlike Tonelli-Shanks it
//     does not degrade when p - 1 has a huge power
  ↪ of two. (Tonelli is in 02 - DiscreteLog...)
//   * Closed forms that need an irrational:
  ↪ Fibonacci via ((1+sqrt5)/2)^n mod p, and ANY
  ↪ order-2
//     linear recurrence x_{n+1} = a x_n + b x_{n-1},
  ↪ even when the discriminant is a non-residue
//     mod p (no matrix needed - one field power).
//   * Lucas sequences / Frobenius primality tests,
  ↪ and n-th terms of (a + b sqrt(D))^n identities.
// PRECONDITIONS: QP an odd prime, all inputs reduced
  ↪ into [0, QP). QP up to ~4.6e18 (the __int128
// products need QP^2 < 2^127).
ll QP, QD;
  ↪   // the modulus and the radicand
ll qmul(ll x, ll y) { return (ll)((__int128)x * y %
  ↪ QP); }
struct Quad {
    ll a, b;
  ↪   // a + b*sqrt(QD)
    Quad(ll a = 0, ll b = 0) : a(a), b(b) {}
    Quad operator*(const Quad& o) const {
        return Quad((qmul(a, o.a) + qmul(QD, qmul(b,
  ↪ o.b))) % QP,
                    (qmul(a, o.b) + qmul(b, o.a)) %
  ↪ QP);
    }
    Quad operator+(const Quad& o) const { return
  ↪ Quad((a + o.a) % QP, (b + o.b) % QP); }
    Quad operator-(const Quad& o) const { return
  ↪ Quad((a - o.a + QP) % QP, (b - o.b + QP) % QP); }
    Quad conj() const { return Quad(a, (QP - b) %
  ↪ QP); }        // the Frobenius x -> x^p
    ll norm() const { return (qmul(a, a) - qmul(QD,
  ↪ qmul(b, b)) % QP + QP) % QP; }
};
Quad qpow(Quad x, ll e) {
  ↪   // O(log e) with 3 multiplications a step
    Quad r(1, 0);
    for (; e; e >>= 1, x = x * x) if (e & 1) r = r *
  ↪ x;
    return r;
}
// CIPOLLA: x with x^2 = a (mod p), p an odd prime.
  ↪ Returns -1 if a is not a quadratic residue.
ll cipolla(ll a, ll p) {
    a %= p; if (a < 0) a += p;
    if (a <= 1) return a;
    if (powmod(a, (p - 1) / 2, p) != 1) return -1;
  ↪   // Euler's criterion
    QP = p;
    ll t = 1;
    while (true) {
  ↪   // find t with t^2 - a a NON-residue
        QD = ((__int128)t * t - a) % p; QD = (QD + p)
  ↪ % p;
        if (QD && powmod(QD, (p - 1) / 2, p) == p -
  ↪ 1) break;
        t++;
    }
    return qpow(Quad(t, 1), (p + 1) / 2).a;
  ↪   // the sqrt(QD) part is provably 0
}
// The other root is p - x. Expected O(log p) tries
  ↪ for t (half the residues work).
// FIBONACCI mod p: set QD = 5, phi = (1 + sqrt 5)/2
  ↪ = Quad((p+1)/2, (p+1)/2); then
//   F(n) = (phi^n - (1 - phi)^n) / sqrt(5), i.e.
  ↪ qpow(phi, n).b * 2 (mod p) since the two
//   conjugate parts cancel. Works for every odd p,
  ↪ whether or not 5 is a residue.
// GENERAL ORDER-2 RECURRENCE x_{n+1} = A x_n + B
  ↪ x_{n-1}: the roots of z^2 = A z + B live in
// F_p[sqrt(A^2 + 4B)], so x_n is a linear
  ↪ combination of their n-th powers - one qpow call.
```

4.3.8 SqrtModPrimePower

factorize (02 - Primes and Divisors/06 - MillerRabinPollard.cpp).

```
// Needs: sqrtMod (02 - DiscreteLogAndRoots.cpp),
  ↪ extGCD (01 - Euclidean Algorithm), CRT (01),
//        factorize (02 - Primes and Divisors/06 -
  ↪ MillerRabinPollard.cpp).
// SQUARE ROOTS MODULO A PRIME POWER, AND MODULO A
  ↪ COMPOSITE - what Tonelli-Shanks alone cannot
// do. Everything here is HENSEL LIFTING: a root mod
  ↪ p^j lifts to a root mod p^(j+1) by one
// Newton step x <- x - f(x)/f'(x), valid whenever
  ↪ f'(x) = 2x is invertible mod p, i.e. p odd and
// p !| x. p = 2 is the exception (the derivative is
  ↪ even) and is handled separately below.
// PRECONDITION for both routines: gcd(a, p) = 1. If
  ↪ p | a, first pull out the largest p^(2t)
// dividing a (an ODD exponent of p in a means NO
  ↪ solution unless it is >= k), solve for a/p^(2t)
// modulo p^(k-2t), multiply the roots by p^t, and
  ↪ add every multiple of p^(k-t).
// COMPLEXITY: O(k log p) per lift; O(log^2) for the
  ↪ composite version plus the factorisation.
ll invMod(ll a, ll m) { ll x, y; extGCD(((a % m) + m)
  ↪ % m, m, x, y); return ((x % m) + m) % m; }
// one root of x^2 = a (mod p^k), p an ODD prime,
  ↪ gcd(a, p) = 1. -1 if a is a non-residue mod p.
ll sqrtPrimePower(ll a, ll p, int k) {
    ll x = sqrtMod(((a % p) + p) % p, p);
    if (x < 0) return -1;
    ll pk = p;
    for (int j = 1; j < k; j++) {
        ll np = pk * p;
        ll fx = (((__int128)x * x - a) % np + np) %
  ↪ np;
        x = (ll)((((__int128)x - (__int128)fx *
  ↪ invMod(2 * x % np, np)) % np + np) % np);
        pk = np;
    }
    return x;
  ↪   // the other root is p^k - x
}
// one root of x^2 = a (mod 2^k), a ODD. -1 if there
  ↪ is none.
```

```
ll sqrtPow2(ll a, int k) {
    a &= (1LL << k) - 1;
    if (k == 1) return 1;
    // a odd => x = 1
    if (k == 2) return (a & 3) == 1 ? 1 : -1;
    if ((a & 7) != 1) return -1;
    // a = 1 (mod 8) is necessary for k >= 3
    ll x = 1;
    for (int j = 3; j < k; j++) {
        // lift 2^j -> 2^(j+1); pick the right one
        ll nm = 1LL << (j + 1);
        if ((((__int128)x * x - a) % nm + nm) % nm) x += 1LL << (j - 1);
    }
    return x;
}
// NUMBER OF SOLUTIONS of x^2 = a (mod p^k) with gcd(a, p) = 1:
//   p odd: 2 if a is a residue mod p, else 0.  The roots are +-x.
//   p = 2: k = 1 -> 1 root; k = 2 -> 2 roots (a = 1 mod 4); k >= 3 -> 4 roots (a = 1 mod 8),
//          namely +-x and +-x + 2^(k-1).
// COMPOSITE m: factor m = prod p_i^k_i, solve each, and CRT every COMBINATION of signs - the
// solution count is the product of the per-prime-power counts, so it can be 2^omega(m) many.
// THE SPECIAL CASE a = 1 is worth memorising: the number of x with x^2 = 1 (mod m) is
//   prod over odd p^k || m of 2, times (1 if 2||m is 2^1, 2 if 4||m, 4 if 8|m).
ll numSqrt1(ll m) {
    // count of x in [0, m) with x^2 = 1
    ll res = 1;
    for (auto [p, e] : factorize(m))
        res *= (p != 2 ? 2 : (e == 1 ? 1 : e == 2 ? 2 : 4));
    return res;
}
// USES: counting self-inverse elements, "how many x with x^2 = 1" in group-theory counting,
// solving quadratic congruences ax^2 + bx + c = 0 (complete the square: (2ax + b)^2 = b^2 - 4ac,
// needs gcd(2a, m) = 1), and lifting solutions of ANY polynomial f - the same Newton step works
// whenever gcd(f'(x0), p) = 1.
```

4.3.9 ModularAPBlocks

THE SEQUENCE (a*k + b) mod m, FOR k = 0..n-1, AS O(sqrt m) ARITHMETIC PROGRESSIONS.

```
// Each returned block {start, diff, len} lists consecutive terms start, start+diff, ... that do
// NOT wrap past m - so a value-sorted / value-structured query over the whole sequence becomes
// O(sqrt m) range operations instead of n individual ones. This is the constructive form of the
// THREE-DISTANCE THEOREM: the points k*a mod m take at most three distinct gaps.
// USE IT FOR: sum of x^((a k + b) mod m), sum over k of f((a k + b) mod m) for an f with prefix
// sums, min/max of (a k + b) mod m over a range of k, counting terms in a value window when the
// generic O(log) count (01 - Euclidean Algorithm/06) does not fit the shape of the problem.
// PRECONDITIONS for decomposeModAP: gcd(a, m) = 1 and n <= m. The wrapper solveModAP below
// removes both restrictions: gcd is peeled off by rewriting the sequence, and n > m is split
// into floor(n/m) identical full cycles plus a remainder.
// COMPLEXITY: O(sqrt m log m) for the decomposition (the sort dominates).
struct APBlock { ll start, diff, len; };
// start + i*diff for 0 <= i < len
vector<APBlock> decomposeModAP(ll a, ll b, ll m, ll n) {
    vector<APBlock> res;
    ll w = (ll)sqrtl((ld)m) + 1;
    if (n <= w) {
        // too short to be worth the machinery
        for (ll i = 0; i < n; i++) res.push_back({(a * i + b) % m, 0, 1});
        return res;
    }
    vector<pair<ll, ll>> v(w + 1);
    // (residue, index) for the first w+1 terms
    for (ll i = 0; i <= w; i++) v[i] = {a * i % m, i};
    sort(all(v));
    ll d = 1, h = m;
    // a*d = +-h (mod m) with h <= m/w, |d| <= w
    for (ll i = 0; i < w; i++) {
        // pigeonhole: two residues within m/w
        ll diff = v[i + 1].first - v[i].first;
        if (diff < h) h = diff, d = v[i + 1].second - v[i].second;
    }
    if (d > 0) {
        // stepping k by d increases the value by h
        for (ll r = 0; r < d; r++)
            for (ll j = r; j < n;) {
                ll val = (a * j + b) % m;
                ll step = min((m - val - 1) / h + 1, (n - j - 1) / d + 1);
                res.push_back({val, h, step});
                j += step * d;
            }
    } else {
        // stepping k by -d decreases it by h
        for (ll r = n - 1; r >= n + d; r--)
            for (ll j = r; j >= 0;) {
                ll val = (a * j + b) % m;
                ll step = min((m - val - 1) / h + 1, j / -d + 1);
                res.push_back({val, h, step});
                j += step * d;
            }
    }
    return res;
}
// FULL WRAPPER for arbitrary a, b, m, n: reduce gcd, then full cycles, then one decomposition.
//   g = gcd(a, m) > 1: write b = b1*g + b2 with b2 < g; then (a k + b) mod m
//                      = b2 + g * ((a/g * k + b1) mod (m/g))  - recurse on the smaller modulus.
//   n >= m (after gcd = 1): each residue in [0, m) appears exactly once per cycle, so
//                      floor(n/m) full cycles contribute a constant, and n mod m is left over.
// EXAMPLE (sum of x^((a k + b) mod m) mod p): per block the answer is the geometric-like sum
// x^start * (1 + x^diff + ... + x^(diff*(len-1))), computable in O(log len) by the standard
// "return (x^n, 1 + x + ... + x^(n-1)) together"
```

```
  ↪ doubling recursion.
```

4.3.10 RootsOfUnityFilter

SUM OF BINOMIALS OVER A CONGRUENCE CLASS: r[j] = sum over k = j (mod m) of C(n, k), for every

```
// Needs: multiply (06 - Math/05 - Convolution/02 -
  ↪ NTT.cpp), modulus NMOD = 998244353.
// j at once, with n up to 1e18 and m up to ~1e5. O(m
  ↪ log m log n).
// THE IDEA: (1 + x)^n has C(n, k) as its
  ↪ coefficients, so the answer is that polynomial
  ↪ reduced
// MODULO x^m - 1 (i.e. exponents folded by k mod m).
  ↪ Binary-exponentiate (1 + x) in the ring
// Z_p[x] / (x^m - 1): one NTT product plus a fold
  ↪ per bit.
// PRECONDITIONS: works modulo NMOD (998244353); no
  ↪ condition on m at all - unlike the classical
// roots-of-unity filter, this needs no primitive
  ↪ m-th root to exist.
vector<ll> binomByResidue(ll n, int m) {
  ↪   // r[j] = sum_{k = j mod m} C(n, k)
    auto fold = [&](vector<ll> v) {
  ↪   // reduce modulo x^m - 1
        for (int i = (int)v.size() - 1; i >= m; i--)
  ↪ v[i - m] = (v[i - m] + v[i]) % NMOD;
        v.resize(min<int>(v.size(), m));
        return v;
    };
    vector<ll> res = {1}, base = fold({1, 1});
    for (; n; n >>= 1) {
        if (n & 1) res = fold(multiply(res, base));
        base = fold(multiply(base, base));
    }
    res.resize(m);
    return res;
}
// THE CLASSICAL ROOTS-OF-UNITY FILTER, which is what
  ↪ you use when m is TINY (2, 3, 4, ...) and
// you want a closed form instead of a convolution.
  ↪ With w a primitive m-th root of unity:
//   sum_{k = r (mod m)} a_k  =  (1/m) *
  ↪ sum_{j=0}^{m-1} w^(-j r) * A(w^j),     A(x) =
  ↪ sum a_k x^k
// so for a_k = C(n, k), A(x) = (1+x)^n and
//   sum_{k = r (mod m)} C(n, k)  =  (1/m) *
  ↪ sum_{j=0}^{m-1} w^(-j r) * (1 + w^j)^n.
// Examples: m = 2 gives 2^(n-1) for both classes (n
  ↪ >= 1); m = 4 gives the classic
//   (2^n + 2^((n+2)/2) * cos(n pi / 4)) / 4  style
  ↪ answers.
// WHERE TO GET w: if m | p - 1 use g^((p-1)/m) for a
  ↪ primitive root g mod p; otherwise work in
// the extension F_p[x]/(cyclotomic), or just use
  ↪ this polynomial version, which sidesteps it.
// SAME TECHNIQUE, other sequences: replace (1 + x)
  ↪ by ANY generating function and you filter its
// coefficients by residue - e.g. "number of subsets
  ↪ with sum divisible by m", "number of paths of
// length n whose displacement is 0 mod m". Reduce
  ↪ modulo x^m - 1 and exponentiate.
```

4.3.11 BinomialPrefixQueries

PARTIAL SUMS OF A BINOMIAL ROW: f(n, k) = C(n,0) + C(n,1) + ... + C(n,k), for q queries.

```
// Needs: Mint, buildFact, C (06 - Math/01 - Modular
  ↪ Arithmetic/01 - Mint.cpp).
// There is NO closed form for f(n, k), but both
  ↪ coordinates move in O(1):
//   f(n+1, k) = 2*f(n, k) - C(n, k)        f(n, k+1)
  ↪ = f(n, k) + C(n, k+1)
//   f(n-1, k) = (f(n, k) + C(n-1, k)) / 2  f(n, k-1)
  ↪ = f(n, k) - C(n, k)
// so treat the queries as points (n, k) and run MO'S
  ↪ ALGORITHM on them: O((n + q) sqrt n) with a
// tiny constant, versus O(n) per query naively.
// REACH FOR IT when a problem asks "probability that
  ↪ at most k of n coins came up heads" or
// "number of subsets of size <= k" for many (n, k)
  ↪ pairs offline.
// PRECONDITIONS: MOD prime, buildFact() called, all
  ↪ n < N. Queries must be answered OFFLINE.
vector<Mint> binomPrefixQueries(vector<pair<int,
  ↪ int>> qs) {
    int q = qs.size(), B = max(1, (int)(N / max(1.0,
  ↪ sqrt((double)q))));
    vector<int> ord(q);
    iota(all(ord), 0);
    sort(all(ord), [&](int x, int y) {
        int bx = qs[x].first / B, by = qs[y].first /
  ↪ B;
        if (bx != by) return bx < by;
        return (bx & 1) ? qs[x].second > qs[y].second
  ↪ : qs[x].second < qs[y].second;
    });
    vector<Mint> ans(q);
    Mint cur = 1, inv2 = Mint((MOD + 1) / 2);
  ↪   // f(1, 0) = C(1, 0) = 1
    int n = 1, k = 0;
    for (int i : ord) {
        auto [L, R] = qs[i];
        while (n < L) cur = cur * 2 - C(n, k), n++;
        while (n > L) --n, cur = (cur + C(n, k)) *
  ↪ inv2;
        while (k < R) ++k, cur += C(n, k);
        while (k > R) cur -= C(n, k), k--;
        ans[i] = cur;
    }
    return ans;
}
// EXACT VALUES you can use as identities instead,
  ↪ when they apply:
//   f(n, n) = 2^n;  f(n, k) = 2^n - f(n, n-k-1);
  ↪ f(2m, m) = (2^(2m) + C(2m, m)) / 2.
// A SINGLE query with n up to 1e18 and k small is
  ↪ just Cbig summed k times.
// The same Mo trick works for any quantity with O(1)
  ↪ moves in both indices - partial sums of
// C(n,i)*x^i, of Catalan-like rows, or of a fixed
  ↪ row of any Pascal-style triangle.
```

4.3.12 IntersectAP

INTERSECTION OF TWO ARITHMETIC PROGRESSIONS {a1 + d1*i : i >= 0} and {a2 + d2*j : j >= 0}.

```
// Needs: crt2 (04 - CongruencesAndTowers.cpp) or CRT (01 - CRT.cpp).
// The intersection is itself an AP with difference lcm(d1, d2), whose first term is the smallest
// common value that is >= max(a1, a2). Empty exactly when the CRT system is unsolvable, i.e.
// when (a1 - a2) is not divisible by gcd(d1, d2).
// Returns {first, step}; step = 0 means EMPTY. O(log) via one CRT.
// WHEN: merging periodic events ("both traffic lights green"), intersecting the answer sets of
// several modular constraints, and range-counting over a set defined by many congruences.
// OVERFLOW: lcm(d1, d2) can reach d1*d2 - use __int128 inside CRT (the notebook version does).
pair<ll, ll> intersectAP(ll a1, ll d1, ll a2, ll d2) {
    auto [r, m] = crt2(((a1 % d1) + d1) % d1, d1, ((a2 % d2) + d2) % d2, d2);
    if (m == -1) return {0, 0};   // no common term at all
    ll start = max(a1, a2);
    if (r < start) r += (start - r + m - 1) / m * m;   // advance to the first term >= start
    return {r, m};
}
// COUNTING the common terms in [lo, hi] is then (hi - first)/step + 1 when first <= hi.
// MANY progressions: fold them left to right with this function; the difference multiplies up to
// the lcm of all of them, so it overflows fast - keep an eye on it, or cap at "beyond the range".
// THE INFINITE-IN-BOTH-DIRECTIONS version (i and j any integer) is just the CRT answer with no
// max(a1, a2) clamp - that is the subgroup a1 + gcd-shifted lattice, see also
// 01 - Euclidean Algorithm/10 - IntegerLattice2D.cpp for the two-coordinate analogue.
```

4.3.13 PohligHellman

discreteLog (02 - DiscreteLogAndRoots.cpp), CRT (01 - CRT.cpp).

```
// Needs: powmod, factorize (02 - Primes and Divisors/06 - MillerRabinPollard.cpp),
//        discreteLog (02 - DiscreteLogAndRoots.cpp), CRT (01 - CRT.cpp).
// POHLIG-HELLMAN DISCRETE LOG - solve g^x = h (mod p) in O(sum over q^e || n of e*(log n + sqrt q))
// instead of the O(sqrt p) of plain baby-step giant-step, where n = ord(g). It is a massive win
// exactly when n is SMOOTH (all prime factors small), which is common in constructed problems:
// p - 1 = 2^k * small stuff, or a subgroup of small order.
// THE IDEA: the discrete log is determined modulo each prime power q^e dividing n. Inside the
// order-q subgroup a log costs only O(sqrt q), and the q-adic digits of x are recovered one at a
// time; CRT glues the residues back together.
// PRECONDITIONS: p PRIME (the inverse below uses Fermat), g of multiplicative order exactly n
// (pass n = p - 1 when g is a primitive root), 0 < h < p. Returns x in [0, n), or -1 if h is not
// in the subgroup generated by g.
ll pohligHellman(ll g, ll h, ll p, ll n) {
    vector<ll> rs, ms;
    for (auto [q, e] : factorize(n)) {
        ll qe = 1;
        for (int i = 0; i < e; i++) qe *= q;
        ll gq = powmod(g, n / q, p), x = 0, cur = 1;
        // gq generates the subgroup of order q
        for (int k = 0; k < e; k++) {
        // recover the q-adic digits of x
            ll gx = powmod(powmod(g, x, p), p - 2, p);  // g^-x
            ll hk = powmod((ll)((__int128)h * gx % p), n / (cur * q), p);
            ll d = discreteLog(gq, hk, p);
            // a log inside a group of size q
            if (d < 0) return -1;
            x += d * cur, cur *= q;
        }
        rs.push_back(x % qe), ms.push_back(qe);
    }
    return CRT(rs, ms).first;
}
// WHEN TO USE WHICH
//   n smooth              -> this file (can be near-instant even for p ~ 1e18)
//   n has a large prime   -> plain BSGS, O(sqrt p) time AND memory (02 - DiscreteLogAndRoots.cpp)
//   composite modulus     -> the BSGS in 02 already strips gcd(a, m); Pohlig-Hellman needs a
//                            group, so factor m and CRT the per-prime-power answers yourself.
// SANITY: check powmod(g, n, p) == 1 and that n really is the ORDER of g (multOrder), otherwise
// the CRT reassembles a residue modulo the wrong number and the answer is silently wrong.
// SECURITY TRIVIA THAT IS ACTUALLY A HINT: this is why cryptographic primes are chosen so that
// p - 1 has a large prime factor. If a problem hands you a "random" prime whose p - 1 factors
// into small pieces, that is the intended solution.
```

4.4 Theorems

4.4.1 NumberTheoryFormulas

NUMBER THEORY - FORMULA SHEET

```
/* ================= NUMBER THEORY - FORMULA SHEET =================
 MULTIPLICATIVE FUNCTIONS  (f(ab)=f(a)f(b) for gcd(a,b)=1)
  tau(n)=#divisors=prod(e_i+1)   sigma(n)=sum of divisors=prod (p^(e+1)-1)/(p-1)
  phi(n)=n*prod(1-1/p)           mu(n)= 0 if NOT SQUAREFREE (some p^2 | n), else (-1)^#primes
  sum_{d|n} phi(d) = n           sum_{d|n} mu(d) = [n==1]        (this is the key identity)
  sum_{d|n} mu(d)/d = phi(n)/n   sum_{d|n} mu(d)*tau(n/d) = 1

 DIRICHLET CONVOLUTION  (f*g)(n) = sum_{d|n} f(d) g(n/d)
  1*1 = tau      Id*1 = sigma      phi*1 = Id
  mu*1 = eps(=[n==1])
  mu is the inverse of 1, so f = g*1  <=>  g = f*mu.

 MOBIUS INVERSION
  g(n) = sum_{d|n} f(d)        <=>  f(n) = sum_{d|n} mu(d) g(n/d)
  g(n) = sum_{n|d, d<=N} f(d)  <=>  f(n) = sum_{n|d, d<=N} mu(d/n) g(d)      (the "upward" form)
  Standard use: [gcd(i,j)==1] = sum_{d | gcd(i,j)}
```

```
  ↪ mu(d).

GCD / LCM SUMS  (all O(N log N) or better with a
  ↪ mu/phi sieve)
   #coprime pairs (i,j)<=n        = sum_{d=1..n}
  ↪ mu(d) * floor(n/d)^2
   sum_{i,j<=n} gcd(i,j)          = sum_{d=1..n}
  ↪ phi(d) * floor(n/d)^2
   sum_{d|n} d*phi(n/d)           = sum_{i=1..n}
  ↪ gcd(i,n)
   sum_{i=1..n} lcm(i,n)          = n/2 * (1 +
  ↪ sum_{d|n} d*phi(d))
   sum_{i,j<=n} lcm(i,j)          = sum_g g *
  ↪ sum_{a,b<=n/g, gcd(a,b)=1} a*b
   general: sum_{i,j} f(gcd(i,j)) = sum_d (f*mu)(d) *
  ↪ floor(n/d)^2

DIVISOR BLOCKING  - evaluates sum_{i=1..n}
  ↪ F(floor(n/i)) in O(sqrt n):
   for (ll l = 1, r; l <= n; l = r + 1) { r = n / (n
  ↪ / l); }    // floor(n/i) == n/l on [l, r]
   Gives sum tau(i), sum sigma(i), sum floor(n/i),
  ↪ and Min_25 / Lucy-style prefix sums.

MODULAR
   Fermat:   a^(p-1) = 1 (mod p), p prime, p !| a
  ↪ => a^-1 = a^(p-2)
   Euler:    a^phi(m) = 1 (mod m) if gcd(a,m)=1
   GENERALISED EULER (a and m NOT coprime), for n >=
  ↪ log2(m):
             a^n = a^(n mod phi(m) + phi(m))  (mod m)
  ↪           <- power towers
   Wilson:   (p-1)! = -1 (mod p)  iff p prime
   CRT:      x = a_i (mod m_i) has a unique solution
  ↪ mod lcm(m_i) iff a_i = a_j (mod gcd(m_i,m_j))
   Legendre: exponent of p in n!  = sum_{i>=1}
  ↪ floor(n / p^i)  = (n - s_p(n)) / (p - 1)
   Kummer:   exponent of p in C(n,r) = #carries when
  ↪ adding r and (n-r) in base p
   LTE (p odd, p | a-b, p !| a,b):  v_p(a^n - b^n) =
  ↪ v_p(a-b) + v_p(n)
       p=2 (a, b both ODD):
         n odd  : v_2(a^n - b^n) = v_2(a-b)
  ↪ <- the case everyone forgets
         n even : v_2(a^n - b^n) = v_2(a-b) +
  ↪ v_2(a+b) + v_2(n) - 1
         n odd  : v_2(a^n + b^n) = v_2(a+b)
         n even : v_2(a^n + b^n) = 1
   LTE for SUMS (p odd, p | a+b, p !| a,b):
             n odd  -> v_p(a^n + b^n) = v_p(a+b) +
  ↪ v_p(n)
             n even -> p does not divide a^n + b^n at
  ↪ all

QUADRATIC RESIDUES (p odd prime)
   Euler's criterion: a^((p-1)/2) = +1 if a is a QR,
  ↪ -1 if not
   #QRs = (p-1)/2.  x^2=a has 0 or 2 roots.  Use
  ↪ Tonelli-Shanks.
   -1 is a QR iff p = 1 (mod 4);  2 is a QR iff p =
  ↪ +-1 (mod 8)

PRIMITIVE ROOTS exist exactly for m = 1, 2, 4, p^k,
  ↪ 2p^k (p odd prime). Count = phi(phi(m)).

REPRESENTATIONS
   n is a sum of two squares  iff every prime p = 3
  ↪ (mod 4) has an EVEN exponent
   n is a sum of three squares iff n != 4^a (8b+7)
  ↪   every n is a sum of four squares
   Primitive Pythagorean triples: (m^2-k^2, 2mk,
  ↪ m^2+k^2), m>k>0, gcd(m,k)=1, m-k odd

 SIZE FACTS (for choosing an algorithm)
   max tau(n): 1344 for n<=1e9, 6720 for n<=1e12,
  ↪ 103680 for n<=1e18
   sum_{i<=n} tau(i) ~ n ln n ;  sum_{i<=n} phi(i) ~
  ↪ 3n^2/pi^2
   pi(n) ~ n/ln n ;  n-th prime ~ n ln n ;  Bertrand:
  ↪ a prime always lies in (n, 2n)
   gaps: for n<=1e18 the max prime gap is < 1500
   Pisano period pi(m) | ... : F(n) mod m repeats
  ↪ with period <= 6m
================================================================
  ↪ */
```

4.4.2 MobiusInversion

Working code for the identities on the formula sheet. Needs MultSieve (mu, phi).

```
// Needs: MultSieve (04 - NT/02 - Primes and
  ↪ Divisors/07 - MultiplicativeSieve.cpp) for
  ↪ mu/phi.

// #pairs (i,j) with 1<=i,j<=n and gcd(i,j)=1
  ↪ O(n)
ll coprimePairs(int n, const MultSieve& S) {
    ll r = 0;
    for (int d = 1; d <= n; d++) r += (ll)S.mu[d] *
  ↪ (n / d) * (n / d);
    return r;
}
// sum over all pairs of gcd(i,j)
  ↪ O(n)
ll gcdSum(int n, const MultSieve& S) {
    ll r = 0;
    for (int d = 1; d <= n; d++) r += (ll)S.phi[d] *
  ↪ (n / d) * (n / d);
    return r;
}
// General:  sum_{i,j<=n} f(gcd(i,j))  =  sum_d
  ↪ (f*mu)(d) * floor(n/d)^2
// Build g = f*mu once by sieving over multiples:
  ↪ for d: for m=d,2d,..: g[m] += mu[m/d]*f[d]
vector<ll> dirichletMu(const vector<ll>& f, const
  ↪ MultSieve& S) {   // g = f * mu   O(n log n)
    int n = f.size() - 1;
    vector<ll> g(n + 1, 0);
    for (int d = 1; d <= n; d++) if (f[d])
        for (int m = d; m <= n; m += d) g[m] +=
  ↪ (ll)S.mu[m / d] * f[d];
    return g;
}

// DIVISOR BLOCKING: sum_{i=1..n} floor(n/i) in
  ↪ O(sqrt n). Same skeleton for sum tau / sum sigma.
ll sumFloorDiv(ll n) {
    ll r = 0;
    for (ll l = 1, rr; l <= n; l = rr + 1) { rr = n /
  ↪ (n / l); r += (n / l) * (rr - l + 1); }
    return r;
}
// sum_{i=1..n} tau(i) = sum_{i=1..n} floor(n/i)
  ↪ (count multiples)
// sum_{i=1..n} sigma(i) = sum_{i=1..n} i * floor(n/i)

// COUNT MULTIPLES / INCLUSION-EXCLUSION OVER
```

```
  ↪ DIVISORS:
// "how many i<=n are divisible by at least one of
  ↪ p1..pk" -> IE over the 2^k squarefree products,
// sign = mu of the product. For k up to ~20
  ↪ enumerate submasks; beyond that, sieve mu.

// sum_{i=1..n} gcd(i, n) = sum_{d|n} d * phi(n/d)
ll gcdSumWithN(ll n) {
    ll r = 0;
    for (auto d : divisors(n)) r += (ll)d * phi(n /
  ↪ d);
    return r;
}
// sum_{i=1..n} lcm(i, n) = n/2 * (1 + sum_{d|n}
  ↪ d*phi(d))
ll lcmSumWithN(ll n) {
    ll s = 1;
    for (auto d : divisors(n)) s += (ll)d * phi(d);
    return n / 2 * s + (n % 2 ? s / 2 : 0);
}
```

4.4.3 PrimeCounting

LUCY_HEDGEHOG: pi(n) = #primes <= n, and the SUM of primes <= n, in O(n^(3/4)) time and

```
// O(sqrt n) memory. Handles n up to ~1e12 in well
  ↪ under a second; 1e13 in a few seconds.
// Idea: S(v, p) = (count or sum) of the integers in
  ↪ [2, v] left after sieving by all primes < p.
// Sieving by p only touches v >= p^2, so the whole
  ↪ table lives on the O(sqrt n) distinct
// values of floor(n / i).
ll primePi(ll n) {
    if (n < 2) return 0;
    ll r = (ll)sqrtl((ld)n);
    while (r * r > n) r--;
    while ((r + 1) * (r + 1) <= n) r++;
    vector<ll> S(r + 1), L(r + 1);
  ↪     // S[v] for v <= r, L[i] for v = n/i
    for (ll i = 1; i <= r; i++) S[i] = i - 1, L[i] =
  ↪ n / i - 1;
    for (ll p = 2; p <= r; p++) {
        if (S[p] == S[p - 1]) continue;
  ↪     // p is not prime
        ll sp = S[p - 1], p2 = p * p;
        for (ll i = 1; i <= min(r, n / p2); i++)
            L[i] -= (i * p <= r ? L[i * p] : S[n / (i
  ↪ * p)]) - sp;
        for (ll v = r; v >= p2; v--) S[v] -= S[v / p]
  ↪ - sp;
    }
    return L[1];
}
// Sum of all primes <= n. Same recurrence with f(x)
  ↪ = x(x+1)/2 - 1 and a factor of p.
ll primeSum(ll n) {
    if (n < 2) return 0;
    ll r = (ll)sqrtl((ld)n);
    while (r * r > n) r--;
    while ((r + 1) * (r + 1) <= n) r++;
    // __int128: n(n+1)/2 wraps for n > 4.29e9, and
  ↪ the ANSWER (~n^2/(2 ln n)) wraps at n ~ 3e10.
    auto f = [](ll x) { return x % 2 ? (lll)(x + 1) /
  ↪ 2 * x - 1 : (lll)x / 2 * (x + 1) - 1; };
    vector<ll> S(r + 1), L(r + 1);
    for (ll i = 1; i <= r; i++) S[i] = f(i), L[i] =
  ↪ f(n / i);
    for (ll p = 2; p <= r; p++) {
        if (S[p] == S[p - 1]) continue;
        ll sp = S[p - 1], p2 = p * p;
        for (ll i = 1; i <= min(r, n / p2); i++)
            L[i] -= p * ((i * p <= r ? L[i * p] : S[n
  ↪ / (i * p)]) - sp);
        for (ll v = r; v >= p2; v--) S[v] -= p * (S[v
  ↪ / p] - sp);
    }
    return L[1];
}
// SAME SKELETON computes the prefix sum of ANY
  ↪ completely-multiplicative f (change the two
// initialisations and the multiplier). For general
  ↪ multiplicative f use the Min_25 sieve, which
// runs this as its first phase and then re-adds the
  ↪ prime-power contributions.
// APPROXIMATIONS for sanity checks: pi(n) ~ n/ln n,
  ↪ the k-th prime ~ k ln k.
```

4.4.4 Representations

Representing integers: sums of two squares, Pythagorean triples, Legendre symbol.

```
// Needs: powmod, factorize (02 - Primes and
  ↪ Divisors/06 - MillerRabinPollard.cpp).
// Needs factorize / sqrtMod / isPrime.

int legendre(ll a, ll p) {
  ↪     // 1 QR, -1 non-residue, 0 if p | a
    a %= p; if (a < 0) a += p;
    if (!a) return 0;
    return powmod(a, (p - 1) / 2, p) == 1 ? 1 : -1;
}
// n is a sum of two squares  <=>  every prime p = 3
  ↪ (mod 4) appears to an EVEN power.
bool isSumOfTwoSquares(ll n) {
    if (n <= 0) return n == 0;
    for (auto [p, e] : factorize(n)) if (p % 4 == 3
  ↪ && (e & 1)) return false;
    return true;
}
// One representation n = x^2 + y^2 (Cornacchia).
  ↪ Returns {-1,-1} if none exists.
pair<ll, ll> sumOfTwoSquares(ll n) {
    if (n == 0) return {0, 0};
    ll t = n, pw2 = 1;
    while (t % 4 == 0) t /= 4, pw2 *= 2;
  ↪     // pull out squares of 2
    if (!isSumOfTwoSquares(t)) return {-1, -1};
    // Gaussian-integer product over the prime
  ↪ factorisation
    ll xr = 1, yr = 0;
  ↪     // current value as xr + yr*i
    for (auto [p, e] : factorize(t)) {
        if (p == 2) { for (int i = 0; i < e; i++) {
  ↪ ll nx = xr - yr, ny = xr + yr; xr = nx, yr = ny;
  ↪ } continue; }
        if (p % 4 == 3) { ll k = 1; for (int i = 0; i
  ↪ < e / 2; i++) k *= p; xr *= k, yr *= k;
  ↪ continue; }
        ll r = sqrtMod(p - 1, p);
  ↪     // r^2 = -1 (mod p)
        ll a = p, b = r;
        while (b * b > p) { ll c = a % b; a = b, b =
  ↪ c; } // Cornacchia descent: p = b^2 + c^2
        ll c = (ll)sqrtl((ld)(p - b * b));
        while (c * c < p - b * b) c++;
        while (c * c > p - b * b) c--;
        for (int i = 0; i < e; i++) { ll nx = xr * b
  ↪ - yr * c, ny = xr * c + yr * b; xr = nx, yr =
```

```
    ny; }
    }
    return {llabs(xr) * pw2, llabs(yr) * pw2};
}
// #representations of n as an ORDERED pair of squares (with signs) = 4 * (d_1(n) - d_3(n)),
// where d_i counts divisors = i (mod 4).   [Jacobi's two-square theorem]

// PRIMITIVE Pythagorean triples with hypotenuse <= L:  (m^2-k^2, 2mk, m^2+k^2),
// m > k > 0, gcd(m,k) = 1, m-k odd. Every triple is a multiple of exactly one primitive triple.
vector<array<ll, 3>> pythagorean(ll L) {
    vector<array<ll, 3>> r;
    for (ll m = 2; m * m <= L; m++)
        for (ll k = 1 + (m % 2); k < m; k += 2)
            // m-k must be ODD => opposite parity
            if (gcd(m, k) == 1 && m * m + k * k <= L)
                r.push_back({m * m - k * k, 2 * m * k, m * m + k * k});
    return r;
}
// n is a sum of THREE squares  <=>  n != 4^a * (8b + 7).   Every n is a sum of FOUR squares.
```

4.4.5 DuSieve

DU SIEVE (Dirichlet prefix sums) - prefix sums of a multiplicative f in SUBLINEAR time.

```
// Pick g with an easy prefix sum G and an easy (f*g) prefix sum; then
//     g(1)*S(n) = sum_{i=1..n} (f*g)(i)  - sum_{i=2..n} g(i) * S(floor(n/i))
// Sieve S(x) directly for x <= LIM (take LIM ~ n^(2/3)); recurse with memoisation above that.
// Total O(n^(2/3)) with the sieve, O(n^(3/4)) without.
//   mu * 1  = eps  ->  M(n)   = 1 - sum_{i=2..n} M(n/i)
//   phi * 1 = Id   ->  Phi(n) = n(n+1)/2 - sum_{i=2..n} Phi(n/i)
struct DuSieve {
    ll LIM;
    vector<ll> mu, phi;
    // prefix sums for x <= LIM
    unordered_map<ll, ll> memoM, memoP;

    DuSieve(ll n) {
        LIM = max((ll)1000, (ll)powl((ld)n, 2.0L / 3));
        vector<int> spf(LIM + 1, 0), primes, m(LIM + 1, 0), p(LIM + 1, 0);
        m[1] = p[1] = 1;
        for (ll i = 2; i <= LIM; i++) {
            if (!spf[i]) spf[i] = i, primes.push_back(i), m[i] = -1, p[i] = i - 1;
            for (int q : primes) {
                if (q > spf[i] || i * q > LIM) break;
                spf[i * q] = q;
                if (q == spf[i]) m[i * q] = 0, p[i * q] = p[i] * q;
                else m[i * q] = -m[i], p[i * q] = p[i] * (q - 1);
            }
        }
        mu.assign(LIM + 1, 0), phi.assign(LIM + 1, 0);
        for (ll i = 1; i <= LIM; i++) mu[i] = mu[i - 1] + m[i], phi[i] = phi[i - 1] + p[i];
    }
    ll M(ll n) {
    // sum_{i=1..n} mu(i)   (Mertens)
        if (n <= LIM) return mu[n];
        auto it = memoM.find(n);
        if (it != memoM.end()) return it->second;
        ll r = 1;
        for (ll l = 2, rr; l <= n; l = rr + 1) { rr = n / (n / l); r -= (rr - l + 1) * M(n / l); }
        return memoM[n] = r;
    }
    ll P(ll n) {
    // sum_{i=1..n} phi(i)
        if (n <= LIM) return phi[n];
        auto it = memoP.find(n);
        if (it != memoP.end()) return it->second;
        ll r = n % 2 ? (n + 1) / 2 * n : n / 2 * (n + 1);
        for (ll l = 2, rr; l <= n; l = rr + 1) { rr = n / (n / l); r -= (rr - l + 1) * P(n / l); }
        return memoP[n] = r;
    }
};
// USES: squarefree count up to n = sum_{d<=sqrt n} mu(d) * floor(n/d^2);
//       #coprime pairs up to n = 2*Phi(n) - 1;  sum of gcd/lcm over pairs at n ~ 1e10;
//       any "sum over 1..n of a multiplicative function" that a linear sieve cannot reach.
```

4.4.6 MoreFormulaCards

NUMBER THEORY, SECOND FORMULA SHEET

```
/* ============ NUMBER THEORY, SECOND FORMULA SHEET ============
 Everything here was machine-verified against brute force / sympy.

 QUADRATIC RECIPROCITY.  p, q distinct ODD primes.
   (p/q)(q/p) = (-1)^( (p-1)/2 * (q-1)/2 )
     => (p/q) = (q/p), UNLESS p = q = 3 (mod 4), where (p/q) = -(q/p)
   SUPPLEMENTS: (-1/p) = 1 iff p = 1 (mod 4);  (2/p) = 1 iff p = +-1 (mod 8);
                (3/p) = 1 iff p = +-1 (mod 12)
   EULER'S CRITERION (p odd prime AND p !| a): a^((p-1)/2) = (a/p) (mod p).
     If p | a the value is 0, not +-1 - state the hypothesis.
   #k-th ROOTS of a != 0 mod p:  d = gcd(k, p-1); the count is d if a^((p-1)/d) = 1, else 0.
 JACOBI SYMBOL (a/n) for ODD n > 0, n NOT necessarily prime - O(log n), no factoring: */
int jacobi(ll a, ll n) {
          // n odd, n > 0
    int r = 1;
    a = ((a % n) + n) % n;
          // (a/n) is periodic in a mod n
    while (a) {
        while (a % 2 == 0) { a /= 2; if (n % 8 == 3 || n % 8 == 5) r = -r; }
        swap(a, n);
        if (a % 4 == 3 && n % 4 == 3) r = -r;
        a %= n;
    }
    return n == 1 ? r : 0;
}
/* CAUTION: (a/n) = 1 does NOT prove a is a QR mod a composite n. (a/n) = -1 DOES prove it is
```

```
  not. Use it as a cheap filter, and to find a
  ↪ non-residue fast for Tonelli-Shanks.

FIBONACCI  (F_0 = 0, F_1 = 1)
  FAST DOUBLING, O(log n), no matrices:
    F(2k) = F(k) * (2 F(k+1) - F(k)),   F(2k+1) =
  ↪ F(k)^2 + F(k+1)^2
  gcd(F_m, F_n) = F_gcd(m,n)   =>   F_m | F_n iff m
  ↪ | n  (m >= 3)
  Cassini:  F_{n-1} F_{n+1} - F_n^2 = (-1)^n
  d'Ocagne: F_m F_{n+1} - F_{m+1} F_n = (-1)^n
  ↪ F_{m-n}
  sum_{i<=n} F_i = F_{n+2} - 1        sum_{i<=n}
  ↪ F_i^2 = F_n F_{n+1}
  F_92 is the last Fibonacci number fitting in a
  ↪ signed 64-bit integer.
PISANO PERIOD pi(m) = the period of F(n) mod m
  pi(1)=1, pi(2)=3, and pi(m) is EVEN for every m > 2
  pi(lcm(a,b)) = lcm(pi(a), pi(b))          pi(p^k)
  ↪ = p^(k-1) * pi(p)
  pi(m) <= 6m, with EQUALITY exactly at m = 2 * 5^k
  ↪  (verified: m = 10, 50, 250)
  p = +-1 (mod 5) => pi(p) | p-1;   p = +-2 (mod 5)
  ↪ => pi(p) | 2(p+1)
ZECKENDORF: every n >= 1 is a UNIQUE sum of
  ↪ NON-CONSECUTIVE Fibonacci numbers, found
  greedily from the top. The basis of "Fibonacci
  ↪ base" digit DP and of Wythoff's game.

PELL:  x^2 - D y^2 = 1, D > 0 non-square. Always
  ↪ infinitely many solutions.
  sqrt(D) = [a0; a1, ..., a_L] is periodic with a_L
  ↪ = 2 a0 and a palindromic middle.
  FUNDAMENTAL SOLUTION = the convergent h_k / k_k
  ↪ with
      k = L - 1   if the period L is EVEN,      k =
  ↪ 2L - 1  if L is ODD.
  ALL solutions: x_m + y_m sqrt(D) = (x1 + y1
  ↪ sqrt(D))^m, i.e.
      x_{m+1} = x1 x_m + D y1 y_m,   y_{m+1} = x1
  ↪ y_m + y1 x_m
  NEGATIVE PELL x^2 - D y^2 = -1 is solvable IFF the
  ↪ period L is ODD; then the fundamental
  solution is the convergent at k = L - 1.
  SIZE WARNING: solutions explode. D = 61 gives x =
  ↪ 1766319049, but D = 661 gives a
  39-digit x. 'll' is useless past D ~ 150 - use
  ↪ bignum, or work modulo the answer.
  CF MACHINERY (a0 = floor(sqrt D); m = 0, d = 1, a
  ↪ = a0), all exact integers:
      m' = d*a - m,    d' = (D - m'^2) / d,    a' =
  ↪ floor((a0 + m') / d')
  BEST RATIONAL APPROXIMATION: the closest p/q to x
  ↪ with q <= N is a convergent OR a
  semiconvergent (t h1 + h2)/(t k1 + k2) with 1 <= t
  ↪ <= a_next. Check both.
  FAREY MEDIANT: for neighbours a/b < c/d (bc - ad =
  ↪ 1), (a+c)/(b+d) is the unique fraction
  with the SMALLEST denominator strictly between
  ↪ them.

SUMS OF SQUARES
  TWO:   n = x^2 + y^2 possible IFF every prime p =
  ↪ 3 (mod 4) has an EVEN exponent.
        r_2(n) = 4 (d_1(n) - d_3(n)), d_i =
  ↪ #divisors congruent to i mod 4.
  THREE: n = x^2 + y^2 + z^2 possible IFF n is NOT
  ↪ of the form 4^a (8b + 7).
  FOUR:  ALWAYS possible (Lagrange).  r_4(n) = 8 *
  ↪ sum over d | n with 4 !| d of d.
  PRIMITIVE PYTHAGOREAN TRIPLES: (m^2 - k^2, 2mk,
  ↪ m^2 + k^2), m > k > 0, gcd(m,k) = 1,
    m - k odd.
  GAUSSIAN INTEGERS Z[i]: the norm N(a+bi) = a^2 +
  ↪ b^2 is multiplicative. Primes of Z[i]:
    1+i (over 2); p = 1 (mod 4) SPLITS as pi *
  ↪ conj(pi); p = 3 (mod 4) stays prime.
    gcd by rounded division: q = round(a conj(b) /
  ↪ N(b)), r = a - qb, then Euclid on norms.

FROBENIUS / CHICKEN MCNUGGET.  a, b >= 2, gcd(a,b) =
  ↪ 1:
  largest non-representable  g(a,b) = ab - a - b;
  ↪ #non-representable = (a-1)(b-1)/2.
  For k >= 3 coins there is NO closed form: run
  ↪ Dijkstra on residues mod a_1 (d[r] = the
  smallest representable value congruent to r), then
  ↪ g = max_r d[r] - a_1. O(a_1 k log a_1).
BEATTY / RAYLEIGH.  r, s > 1 IRRATIONAL with 1/r +
  ↪ 1/s = 1  =>  floor(n r) and floor(n s),
  n >= 1, PARTITION the positive integers exactly.
  ↪ (Wythoff's losing positions are exactly
  (floor(n phi), floor(n phi^2)).)
THREE-DISTANCE (Steinhaus). The points {a}, {2a},
  ↪ ..., {na} cut [0,1) into intervals of AT
  MOST 3 distinct lengths, and when there are 3 the
  ↪ largest is the sum of the other two.
  Use: min over k <= n of (k*a mod m) in O(log) via
  ↪ the continued fraction of a/m.

DETERMINISTIC MILLER-RABIN BASE SETS (all
  ↪ machine-verified)
  n < 2^32              : {2, 7, 61}
  ↪          (valid to 4759123141)
  n < 3.2e18            : the first 12 primes {2..37}
  n < 3.317e24 > 2^64   : {2, 325, 9375, 28178,
  ↪ 450775, 9780504, 1795265022}
  {2,3,5,7} is NOT enough even for 32 bits:
  ↪ 3215031751 = 151*751*28351 passes all four.
  Skip a base a when n divides a, or small primes
  ↪ are reported composite.

NTT PRIMES (form, primality and primitive root all
  ↪ machine-verified)
  p                   = c * 2^k + 1      g     max
  ↪ transform length
  998244353           = 119 * 2^23 + 1    3
  ↪ 2^23     <- the default
  167772161           =   5 * 2^25 + 1    3
  ↪ 2^25
  469762049           =   7 * 2^26 + 1    3
  ↪ 2^26
  754974721           =  45 * 2^24 + 1   11
  ↪ 2^24     <- NOT 9*2^23+1 (common misprint)
  1004535809          = 479 * 2^21 + 1    3
  ↪ 2^21
  2013265921          =  15 * 2^27 + 1   31
  ↪ 2^27
  1224736769          =  73 * 2^24 + 1    3
  ↪ 2^24
  3221225473          =   3 * 2^30 + 1    5
  ↪ 2^30
  9223372036737335297  = 549755813881 * 2^24 + 1, g
  ↪ = 3   (64-bit, needs __int128 mulmod)
  THE 3-PRIME TRIPLE for an arbitrary modulus:
  ↪ 167772161 * 469762049 * 754974721 = 5.95e25.
```

```
   THE BINDING CONSTRAINT IS LENGTH, NOT MAGNITUDE:
   ↪ 754974721 caps the transform at 2^24, so
   the input length is capped at 2^23. At that length
   ↪ the magnitude headroom is still 6x.

 64-BIT CEILINGS - memorise these
   a*b fits in signed ll        iff a, b < 3037000499
   ↪  ( = floor(sqrt(2^63 - 1)) )
   n(n+1)/2 fits in ll          iff n <= 4294967295
   sum of primes <= n fits      iff n <~ 3e10
   ↪  (the sum is ~ n^2 / (2 ln n))
   sum of phi(i), i <= n, fits  iff n <= 5508509576
   ↪  (the sum is ~ 0.3040 n^2)
   max tau(n): 1344 for n <= 1e9,  6720 for 1e12,
   ↪ 103680 for 1e18
   NEED __int128 FOR: any modmul with modulus >
   ↪ 3.04e9 (Pollard, BSGS, Tonelli, CRT, Garner);
     floorSum with n > 1e9; Cornacchia's b*b > p
   ↪ test; prime/totient SUMS at n >= 1e10.
   FFT PRECISION, the real criterion (not "|result| <
   ↪ 1e15"):
     safe iff (sum a_i^2 + sum b_i^2) * log2(N) <
   ↪ 9e14.
     Splitting at sqrt(M) additionally needs M^1.5 <
   ↪ 9.2e18, i.e. M <~ 4.4e12, with the
     inputs already reduced into [0, M).
   FWHT: the forward transform inflates magnitudes by
   ↪ up to N, so the pointwise product is
     bounded by (N * max|a|)^2. Under a modulus the
   ↪ XOR inverse must MULTIPLY BY inv(N) mod p -
     an integer `x /= n` is exact over Z and WRONG
   ↪ mod p.
================================================================↪ a != 0):
   ↪ */
```

4.4.7 MultiplicativeGroupStructure

THE STRUCTURE OF Z_n^* (the units mod n) - the facts that decide whether a modular equation

```
// Needs: factorize (02 - Primes and Divisors/06 -
   ↪ MillerRabinPollard.cpp).
// has solutions, how many, and what the true
   ↪ exponent of the group is.
//   Z_n^* = product over p^e || n of Z_(p^e)^*, and
//     p odd : Z_(p^e)^* is CYCLIC of order phi(p^e)
   ↪ = p^(e-1)(p-1)      -> primitive root exists
//     p = 2 : Z_2^* trivial, Z_4^* cyclic of order
   ↪ 2, and for e >= 3
//             Z_(2^e)^* = Z_2 x Z_(2^(e-2))  -> NOT
   ↪ cyclic, no primitive root
//   A PRIMITIVE ROOT mod n exists iff n = 1, 2, 4,
   ↪ p^e or 2*p^e for an odd prime p.
// CARMICHAEL LAMBDA = the true exponent of Z_n^*:
   ↪ the smallest L with a^L = 1 for every unit a.
// It divides phi(n) and is often much smaller - use
   ↪ it (not phi) to reduce exponents, and as the
// modulus bound in power towers.
ll carmichael(ll n) {
    ll res = 1;
    for (auto [p, e] : factorize(n)) {
        ll pe = 1;
        for (int i = 0; i < e; i++) pe *= p;
        ll l = pe / p * (p - 1);
  ↪    // phi(p^e)
        if (p == 2 && e >= 3) l /= 2;
  ↪    // the one exception
        res = res / gcd(res, l) * l;
  ↪    // lcm
    }
    return res;
}
// HOW MANY DISTINCT VALUES DOES a^k mod n TAKE, over
   ↪ ALL a in [0, n)? (Not just the units - the
// non-units contribute the recursive term, which is
   ↪ what makes this more than |Z_n^*| / gcd.)
// Multiplicative in n, so handle each prime power
   ↪ and multiply.
ll ipow(ll b, ll e) { ll r = 1; while (e-- > 0) r *=
   ↪ b; return r; }
ll kthPowersPrimePower(ll p, ll e, ll k) {
  ↪    // count of distinct a^k mod p^e
    if (e <= 0 || k == 0) return 1;
    if (p == 2) {
        if (e == 1) return 2;
        ll u = ipow(2, e - 2) / gcd(k, ipow(2, e -
   ↪ 2));
        if (k % 2) u *= 2;
        return u + kthPowersPrimePower(2, e - k, k);
    }
    ll ph = ipow(p, e) - ipow(p, e - 1);
    return ph / gcd(k, ph) + kthPowersPrimePower(p, e
   ↪ - k, k);
}
ll kthPowersMod(ll k, ll n) {
  ↪    // #distinct a^k mod n over all a
    ll res = 1;
    for (auto [p, e] : factorize(n)) res *=
   ↪ kthPowersPrimePower(p, e, k);
    return res;
}
// COMPANION FACTS (all about x^k = a mod p, p prime,
//   the k-th powers form the subgroup of index d =
   ↪ gcd(k, p-1), so there are (p-1)/d of them;
//   a is a k-th power iff a^((p-1)/d) = 1, and then
   ↪ it has exactly d roots.
//   For a MODULUS n, x^k = a is solved prime-power
   ↪ by prime-power and glued with CRT.
// ORDER OF AN ELEMENT: ord(a) | lambda(n); compute
   ↪ it by starting from lambda(n) and dividing
// out prime factors while a^(L/q) = 1 (that is
   ↪ multOrder in 03 - Modular Arithmetic/02).
// EXPONENT REDUCTION: a^e = a^(e mod lambda(n) +
   ↪ lambda(n)) (mod n) for e >= log2(n), which is
// the correct form even when gcd(a, n) > 1 - the
   ↪ same guard the power tower uses.
```

4.4.8 FourSquares

sumOfTwoSquares (04 - Representations.cpp).

```
// Needs: isPrime (02 - Primes and Divisors/06 -
   ↪ MillerRabinPollard.cpp),
//        sumOfTwoSquares (04 - Representations.cpp).
// LAGRANGE'S FOUR-SQUARE DECOMPOSITION - actually
   ↪ CONSTRUCT x, y, z, w with
//     n = x^2 + y^2 + z^2 + w^2
// for n up to 1e18, in expected O(log^2 n) primality
   ↪ tests. The existence theorem is famous; the
// construction is the part people cannot reproduce
   ↪ under pressure, and brute force over two of
// the four variables is O(n) - hopeless past 1e12.
// THE ALGORITHM (Rabin-Shallit, simplified):
//   1. strip factors of 4: n = 4^a * m, solve for m
   ↪ and multiply every answer by 2^a;
//   2. if m = 7 (mod 8) then m is NOT a sum of three
   ↪ squares, so spend one square: m -= 1;
//   3. now m IS a sum of three squares. Pick x at
   ↪ random and test whether m - x^2 is a PRIME
```

```
//      congruent to 1 mod 4 (or a trivial 0/1/2).
  ↪ Such x are dense, so a handful of tries suffice;
//   4. split that prime into two squares with
  ↪ Cornacchia (04 - Representations.cpp).
// PRECONDITION: n >= 0. Uses randomness, so it is
  ↪ expected-time, not worst-case.
array<ll, 4> fourSquares(ll n) {
    array<ll, 4> r{0, 0, 0, 0};
    if (n <= 0) return r;
    ll mul = 1;
    while (n % 4 == 0) n /= 4, mul *= 2;
  ↪   // 4^a factors out of every square
    int idx = 0;
    if (n % 8 == 7) r[idx++] = 1, n -= 1;
  ↪   // 7 mod 8 needs a fourth nonzero square
    if (n < 2000) {
  ↪   // small: brute force, and always succeeds
        for (ll x = 0; x * x <= n && idx < 4; x++)
            for (ll y = x; x * x + y * y <= n; y++) {
                ll t = n - x * x - y * y, z =
  ↪ (ll)sqrtl((ld)t);
                while (z * z > t) z--;
                while ((z + 1) * (z + 1) <= t) z++;
                if (z * z == t) { r[idx++] = x,
  ↪ r[idx++] = y, r[idx++] = z; break; }
            }
    } else {
        mt19937_64
  ↪ rng(chrono::steady_clock::now().time_since_epoch().count());
        ll lim = (ll)sqrtl((ld)n);
        while (lim * lim > n) lim--;
        while (true) {
            ll x = rng() % (lim + 1), m = n - x * x;
            if (m % 4 != 1 && m != 2) continue;
  ↪   // want a prime = 1 (mod 4), or 2
            if (m != 2 && !isPrime(m)) continue;
            auto [u, v] = sumOfTwoSquares(m);
            if (u < 0) continue;
            r[idx++] = x, r[idx++] = u, r[idx++] = v;
            break;
        }
    }
    for (ll& v : r) v *= mul;
    return r;
}
// FACTS WORTH KNOWING
//   THREE squares suffice unless n = 4^a * (8b + 7)
  ↪ (Legendre); TWO suffice iff every prime
//   3 mod 4 has an even exponent; ONE iff n is a
  ↪ perfect square.
//   r_4(n) = 8 * sum of the divisors of n that are
  ↪ not multiples of 4  (Jacobi) - so the number
//   of ORDERED signed representations is easy even
  ↪ though finding one is not.
//   Every n is a sum of at most 9 cubes and at most
  ↪ 19 fourth powers (Waring); the "at most 4
//   squares" bound is tight exactly on the numbers
  ↪ 4^a(8b+7).
// IF n IS ALREADY FACTORED, a deterministic route
  ↪ exists: multiply Hurwitz quaternions the way
// sumOfTwoSquares multiplies Gaussian integers. The
  ↪ randomized version above is far shorter.
```

4.5 Rational

4.5.1 ContinuedFractions

Continued fractions, convergents, best rational approximation, Stern-Brocot search.

```
// x = [a0; a1, a2, ...]. For a rational p/q this is
  ↪ just the Euclidean algorithm's quotients.
vector<ll> cfrac(ll p, ll q) {
    vector<ll> a;
    // FLOOR division: C++ truncates toward zero, so
  ↪ cfrac(-7,2) would give [-3,-2] instead of
    // the correct [-4,2] and every convergent would
  ↪ be wrong.
    while (q) { ll d = p / q - ((p % q) && ((p < 0) ^
  ↪ (q < 0))); a.push_back(d); ll r = p - d * q; p =
  ↪ q, q = r; }
    return a;
}
// Convergents h[i]/k[i] of a continued fraction: h =
  ↪ a*h1 + h2, k = a*k1 + k2.
// Every convergent is a BEST approximation: no
  ↪ fraction with a smaller denominator is closer.
vector<pair<ll, ll>> convergents(const vector<ll>& a)
  ↪ {
    vector<pair<ll, ll>> c;
    ll h1 = 1, h2 = 0, k1 = 0, k2 = 1;
    for (ll x : a) {
        ll h = x * h1 + h2, k = x * k1 + k2;
        c.push_back({h, k});
        h2 = h1, h1 = h, k2 = k1, k1 = k;
    }
    return c;
}
// Best rational approximation to p/q with
  ↪ denominator <= N.
// The answer is a convergent OR a SEMICONVERGENT
  ↪ (t*h1+h2)/(t*k1+k2) - you must compare both.
pair<ll, ll> bestApprox(ll p, ll q, ll N) {
    ll h1 = 1, h2 = 0, k1 = 0, k2 = 1, P = p, Q = q;
    pair<ll, ll> best = {p >= 0 ? 0 : -((-p + q - 1)
  ↪ / q), 1};      // 0/1 (or floor) is legal
    auto better = [&](pair<ll, ll> a, pair<ll, ll> b)
  ↪ {             // is a closer to p/q than b?
        __int128 da = (__int128)llabs(p * a.second -
  ↪ a.first * q) * b.second;
        __int128 db = (__int128)llabs(p * b.second -
  ↪ b.first * q) * a.second;
        return da != db ? da < db : a.second <
  ↪ b.second;
    };
    while (Q) {
        ll a = P / Q, r = P % Q;
        if (k1 && a * k1 + k2 > N) {
  ↪                 // cannot take the full step
            ll t = (N - k2) / k1;
            if (t > 0) { pair<ll, ll> c = {t * h1 +
  ↪ h2, t * k1 + k2}; if (better(c, best)) best = c;
  ↪ }
            break;
        }
        ll h = a * h1 + h2, k = a * k1 + k2;
        if (k > N) break;
        h2 = h1, h1 = h, k2 = k1, k1 = k;
        if (better({h1, k1}, best)) best = {h1, k1};
        P = Q, Q = r;
    }
    return best;
}
// STERN-BROCOT SEARCH: find the simplest fraction
```

```
  ↪ p/q with f(p/q) true, when f is monotone.
// Walk the mediant tree; take exponential jumps in
  ↪ each direction so it is O(log^2).
// Classic uses: "smallest denominator between two
  ↪ reals", recovering a fraction from a decimal,
// and answering "is the answer rational with small
  ↪ denominator" style problems.
// FAREY / mediant fact: if a/b < c/d are neighbours
  ↪ (bc - ad = 1) then their mediant
// (a+c)/(b+d) is the unique simplest fraction
  ↪ strictly between them.
```

4.5.2 PellEquation

PELL'S EQUATION x^2 - D*y^2 = 1 (D > 0 and NOT a perfect square) - the fundamental solution,

```
// then all of them. Every solution is a CONVERGENT
  ↪ of the continued fraction of sqrt(D), so walk
// the convergents and stop at the first one that
  ↪ satisfies the equation: O(sqrt D) steps, since
// the period of sqrt(D) is O(sqrt D log D).
// ALL SOLUTIONS from the fundamental (x1, y1):
  ↪ x_{m+1} = x1*x_m + D*y1*y_m,
//
  ↪ y_{m+1} = x1*y_m + y1*x_m,
// i.e. x_m + y_m sqrt(D) = (x1 + y1 sqrt(D))^m. So
  ↪ there are infinitely many, growing
// EXPONENTIALLY - D = 61 already needs x =
  ↪ 1766319049, and D = 661 needs 39 digits. __int128
// carries you to about D = 200; past that use
  ↪ bignum, or work modulo the required output.
// Returns {0, 0} when D is a square or <= 0 (only
  ↪ the trivial (1, 0) exists).
pair<__int128, __int128> pell(ll D) {
    ll a0 = (ll)sqrtl((ld)D);
    while (a0 * a0 > D) a0--;
    while ((a0 + 1) * (a0 + 1) <= D) a0++;
    if (D <= 0 || a0 * a0 == D) return {0, 0};
    ll m = 0, d = 1, a = a0;
  ↪   // the exact CF recurrence for sqrt(D)
    __int128 h1 = 1, h = a0, k1 = 0, k = 1;
  ↪   // convergents h/k
    while (h * h - (__int128)D * k * k != 1) {
        m = d * a - m;
        d = (D - m * m) / d;
        a = (a0 + m) / d;
        __int128 h2 = h1; h1 = h; h = a * h1 + h2;
        __int128 k2 = k1; k1 = k; k = a * k1 + k2;
    }
    return {h, k};
}
// NEGATIVE PELL  x^2 - D*y^2 = -1 is solvable IFF
  ↪ the period of the CF of sqrt(D) is ODD; the
// same loop finds it if you test for -1 instead (and
  ↪ it always appears before the +1 solution).
// Necessary condition: D has no prime factor
  ↪ congruent to 3 mod 4 (not sufficient: D = 34).
// GENERAL x^2 - D*y^2 = k: find one solution by
  ↪ brute force / Lagrange-Matthews-Mollin, then
// multiply by powers of the fundamental unit to walk
  ↪ the whole class; there can be several
// inequivalent classes, so collect every solution
  ↪ with y below the standard bound.
// CF MACHINERY, exact in integers (a0 = floor(sqrt
  ↪ D), start m = 0, d = 1, a = a0):
//   m' = d*a - m,   d' = (D - m'^2)/d,   a' =
  ↪ floor((a0 + m')/d')   - the period ends when
//   a' = 2*a0, and the a-sequence between is a
  ↪ palindrome.
// USES: minimal solutions of "x^2 - D y^2 = 1" style
  ↪ Diophantine puzzles, the fundamental unit of
// the ring Z[sqrt D], square-triangular numbers (D =
  ↪ 8), and any recurrence whose growth factor
// is a quadratic irrational.
```

4.5.3 RationalReconstruction

RATIONAL RECONSTRUCTION - recover the fraction p/q from its value r = p * q^-1 (mod m).

```
// The answer is UNIQUE as long as |p| and q are both
  ↪ below sqrt(m/2), and it is found by running
// the extended Euclidean algorithm on (m, r) and
  ↪ stopping halfway: the remainder is p and the
// cofactor is q. O(log m).
// WHY YOU WANT IT: do a whole computation in modular
  ↪ arithmetic (Gaussian elimination, a DP over
// fractions, an interpolation, a determinant) and
  ↪ then read off the EXACT rational answer -
// no bignum fractions, no gcd-ing at every step.
  ↪ Also the standard way to guess a closed form
// from modular data, and to answer "print the answer
  ↪ as an irreducible fraction" when the
// intermediate values would explode.
// PRECONDITIONS: m >= 2, gcd(q, m) = 1 (implicit in
  ↪ r existing). Returns {0, 0} on failure -
// which means no such small fraction exists, so take
  ↪ a bigger m (or several and CRT them).
pair<ll, ll> ratRecon(ll r, ll m) {
    ll a = m, b = ((r % m) + m) % m, x = 0, y = 1;
  ↪   // invariant: b = y * r (mod m)
    ll lim = (ll)sqrtl((ld)m / 2);
    while (b > lim) {
        ll q = a / b;
        ll t = a - q * b; a = b, b = t;
        t = x - q * y; x = y, y = t;
    }
    if (y < 0) b = -b, y = -y;
    if (y == 0 || gcd(b < 0 ? -b : b, y) != 1) return
  ↪ {0, 0};
    return {b, y};
  ↪   // b / y = r (mod m), in lowest terms
}
// CHECK YOUR RESULT: verify b * inverse(y) mod m ==
  ↪ r before trusting it - the reconstruction can
// return a wrong small fraction if the true one is
  ↪ not small enough. Verifying is O(log m).
// LARGER BOUNDS: if you know |p| <= P and q <= Q
  ↪ with 2*P*Q < m, stop the loop at b <= P instead
// and the same argument applies. Choosing m as a
  ↪ product of several NTT primes (Garner) is the
// usual way to make m big enough.
// RELATED: the same halfway-Euclid stopping rule is
  ↪ the Berlekamp-Massey / Pade approximant of a
// power series, and 01 - ContinuedFractions.cpp is
  ↪ the exact-arithmetic version of this walk.
```

4.6 Sequences

4.6.1 FibonacciAndPisano

FIBONACCI, FAST. The notebook had the identities on a formula sheet and no routine; this is it.

```
// F(0)=0, F(1)=1. Fast doubling is O(log n) with two multiplications per bit and NO matrices - shorter and ~2x faster than 2x2 matrix exponentiation.
//   F(2k)   = F(k) * (2*F(k+1) - F(k))
//   F(2k+1) = F(k)^2 + F(k+1)^2
pair<ll, ll> fibPair(ll n, ll mod) {
          // returns {F(n), F(n+1)} mod
    if (!n) return {0, 1 % mod};
    auto [a, b] = fibPair(n >> 1, mod);
    ll c = a * ((2 * b % mod - a % mod + mod) % mod) % mod;     // F(2k)
    ll d = (a * a + b * b) % mod;
           // F(2k+1)
    return (n & 1) ? make_pair(d, (c + d) % mod) : make_pair(c, d);
}
ll fib(ll n, ll mod) { return fibPair(n, mod).first; }
// LUCAS NUMBERS L(n) = F(n-1) + F(n+1) = 2F(n+1) - F(n).
// NEGATIVE index: F(-n) = (-1)^(n+1) F(n).
// The mod must be < ~3e9 for the ll products above, or use __int128 / Mint.

// --- PISANO PERIOD pi(m): the period of F(n) mod m. Needed to reduce a huge index. ---
// pi(1) = 1, pi(2) = 3, and pi(m) is EVEN for every m > 2.
// pi(lcm(a,b)) = lcm(pi(a), pi(b))      pi(p^k) = p^(k-1) * pi(p)
// pi(m) <= 6m, with equality exactly at m = 2 * 5^k.
// p = +-1 (mod 5) => pi(p) divides p-1;   p = +-2 (mod 5) => pi(p) divides 2(p+1).
// So: factor m, get pi for each prime power by testing the divisors of that bound, then lcm.
ll pisanoPrime(ll p) {
          // p prime
    if (p == 2) return 3;
    if (p == 5) return 20;
    ll cand = (p % 5 == 1 || p % 5 == 4) ? p - 1 : 2 * (p + 1);
    vector<ll> divs;
    for (ll d = 1; d * d <= cand; d++)
        if (cand % d == 0) divs.push_back(d), divs.push_back(cand / d);
    sort(all(divs));
    for (ll d : divs) {
        auto [a, b] = fibPair(d, p);
        if (a == 0 && b == 1 % p) return d;
    }
    return cand;
}
// Full pi(m): factorise m (Pollard rho), take pisanoPrime(p) * p^(e-1) per prime power, lcm them.
/* MORE FIBONACCI FACTS worth the page
   gcd(F_m, F_n) = F_gcd(m,n)      =>   F_m | F_n iff  m | n   (m >= 3)
   Cassini:   F_{n-1} F_{n+1} - F_n^2 = (-1)^n
   Catalan:   F_n^2 - F_{n-r} F_{n+r} = (-1)^{n-r} F_r^2
   d'Ocagne:  F_m F_{n+1} - F_{m+1} F_n = (-1)^n F_{m-n}
   sum_{i<=n} F_i = F_{n+2} - 1        sum_{i<=n} F_i^2 = F_n F_{n+1}
   sum_{i<=n} F_{2i-1} = F_{2n}        sum_{i<=n} F_{2i} = F_{2n+1} - 1
   F_n = (phi^n - psi^n)/sqrt5;  F_92 is the last one fitting in a signed 64-bit integer.
   ZECKENDORF: every n >= 1 is a UNIQUE sum of NON-CONSECUTIVE Fibonacci numbers, found greedily
     from the largest. This is the "Fibonacci base", and it is what makes Fibonacci-nim work.
   LCM of F_1..F_n and "which F are divisible by k" both follow from the gcd identity.
   Fibonacci mod p when 5 is a QR mod p: sqrt(5) exists, so the closed form works in F_p directly.
     When it is not, work in F_p[x]/(x^2-5) (a "Phi field") - two components, same O(log n). */

// --- TRIBONACCI / any linear recurrence: matrix exponentiation (06 - Math/03), or Kitamasa /
// Bostan-Mori for a long recurrence.
// Berlekamp-Massey RECOVERS the recurrence from ~2k terms.
// --- LUCAS SEQUENCES U_n, V_n with U_{n+1} = P U_n - Q U_{n-1} generalise all of this and have
// the same doubling identities; they are what Baillie-PSW primality is built on.
```

4.6.2 FibonacciLCM

LCM OF FIBONACCI NUMBERS lcm(F(a1), F(a2), ..., F(an)) mod p - the numbers themselves have

```
// Needs: fib (01 - FibonacciAndPisano.cpp), powmod (02 - Primes and Divisors/06).
// billions of digits, so everything is done modulo p and division is a modular inverse.
// THE KEY IDENTITY is gcd(F(i), F(j)) = F(gcd(i, j)), which turns the usual
//   lcm(S) = lcm(S') * x / gcd(lcm(S'), x)      (x = the last element)
// into
//            -- all indices, no big numbers --
//   lcm(F(A)) = lcm(F(A')) * F(x) / lcm( F(gcd(a_i, x)) for a_i in A' )
// i.e. the denominator is again an lcm of Fibonacci numbers, on the SMALLER index set of gcds.
// So recurse on sets of indices; memoise on the sorted, deduplicated index vector.
// COMPLEXITY: O(#distinct gcd-closed subsets * n * log) - bounded by the number of divisors of
// the a_i in practice, fast for n up to a few hundred with a_i up to 1e9.
// CAUTION: this DIVIDES modulo p, so p must be prime AND no F(k) involved may be 0 mod p. With
// p = 1e9+7 that is safe for the usual constraints; if you hit a zero, use a different prime.
map<vector<int>, ll> fibLcmMemo;
ll fibLcm(vector<int> a) {
    sort(a.rbegin(), a.rend());
    while (!a.empty() && a.back() <= 2) a.pop_back();
       // F(1) = F(2) = 1 contribute nothing
    a.erase(unique(all(a)), a.end());
    if (a.empty()) return 1;
    if (a.size() == 1) return fib(a[0], MOD);
    auto it = fibLcmMemo.find(a);
    if (it != fibLcmMemo.end()) return it->second;
    vector<int> b(a.begin(), a.end() - 1);
    ll res = fibLcm(b);
    for (int& x : b) x = gcd(x, a.back());
    res = res * fib(a.back(), MOD) % MOD * powmod(fibLcm(b), MOD - 2, MOD) % MOD;
    return fibLcmMemo[a] = res;
```

```
}
// SAME PATTERN, other sequences: any sequence with
  ↪ gcd(u_i, u_j) = u_gcd(i,j) (a "strong
// divisibility sequence") works verbatim - F(n), 2^n
  ↪ - 1 (lcm of Mersenne numbers), and the
// Lucas sequences U_n(P, Q) with gcd(P, Q) = 1 in
  ↪ general.
// USEFUL COMPANIONS: F(m) | F(n) iff m | n (for m >=
  ↪ 3); gcd(F(m), F(n)) = F(gcd(m,n));
// lcm over ALL i <= n of F(i) has a product formula
  ↪ via the primitive parts (Zsygmondy), but the
// recursion above is what you want in a contest.
```

5 | Combinatorics

5.1 StarsAndBars

Stars and Bars (Number of Solutions to x_1 + x_2 + ... + x_k = n) - O(1)

```
// 1. Non-negative integer solutions (x_i >= 0): C(n
  ↪ + k - 1, k - 1)
// 2. Positive integer solutions (x_i >= 1): C(n - 1,
  ↪ k - 1)
struct StarsAndBars {
    // Requires precomputed Combination C(n, k)
    static ll count_non_negative(ll n, ll k,
  ↪ function<ll(ll, ll)> nCr) {
        if (n < 0 || k <= 0) return 0;
        return nCr(n + k - 1, k - 1);
    }

    static ll count_positive(ll n, ll k,
  ↪ function<ll(ll, ll)> nCr) {
        if (n < k || k <= 0) return 0;
        return nCr(n - 1, k - 1);
    }
};
```

5.2 BurnsidePolya

Burnside / Polya - count colourings up to symmetry. #orbits = (1/|G|) sum_g |fix(g)|

```
// Needs: Mint (06 - Math/01); phi (02 - Primes and
  ↪ Divisors/06) for the necklace formulas.
// Needs Mint and phi() (single-value Euler totient).

// Necklaces: n beads in a circle, k colours,
  ↪ ROTATIONS only.
// A rotation by r has gcd(r,n) cycles -> k^gcd(r,n)
  ↪ fixed colourings; group by d = gcd.
Mint necklaces(ll n, ll k) {
    Mint s = 0;
    for (ll d = 1; d * d <= n; d++) if (n % d == 0) {
        s += Mint(phi(n / d)) * Mint(k).pow(d);
        if (d != n / d) s += Mint(phi(d)) *
  ↪ Mint(k).pow(n / d);
    }
    return s * Mint(n).inv();
}
// Bracelets: rotations AND reflections (dihedral
  ↪ group, |G| = 2n).
Mint bracelets(ll n, ll k) {
    Mint s = necklaces(n, k) * Mint(n);
  ↪   // = sum over the n rotations
    if (n & 1) s += Mint(n) * Mint(k).pow((n + 1) /
  ↪ 2);
    else s += Mint(n / 2) * (Mint(k).pow(n / 2) +
  ↪ Mint(k).pow(n / 2 + 1));
    return s * Mint(2 * n).inv();
}
// GENERAL RECIPE: for each group element g, count
  ↪ its CYCLES c(g) on the objects being
// coloured; it fixes k^c(g) colourings. Answer =
  ↪ (1/|G|) * sum_g k^c(g).
//   square grid n x n under the 4 rotations: cycles
  ↪ = n^2, ceil(n^2/4)... (work them out per problem)
//   cube faces under 24 rotations: cycle counts
  ↪ 6,3,3,4,2,2,... -> (k^6+3k^4+12k^3+8k^2)/24
// POLYA (colours have weights): replace k^c by prod
  ↪ over cycles of (sum of x_i^len).
Mint burnside(const vector<ll>& cyclesPerElement, ll
  ↪ k) {
    Mint s = 0;
    for (ll c : cyclesPerElement) s += Mint(k).pow(c);
    return s *
  ↪ Mint((ll)cyclesPerElement.size()).inv();
}
```

5.3 CombinatoricsFormulas

COMBINATORICS - FORMULA SHEET

```
/* ================= COMBINATORICS - FORMULA SHEET
  ↪ =================
 BASICS   C(n,k)=n!/(k!(n-k)!)   P(n,k)=n!/(n-k)!
  ↪ multinomial n!/(k1!..km!)
   STARS AND BARS:  x1+..+xk = n, xi>=0 -> C(n+k-1,
  ↪ k-1);  xi>=1 -> C(n-1, k-1)
                    with upper bounds xi<=c: IE over
  ↪ which variables overflow
   Compositions of n into k parts C(n-1,k-1); into
  ↪ any parts 2^(n-1)

 BINOMIAL IDENTITIES  (memorise these - recognising
  ↪ one is worth 30 minutes)
   Pascal        C(n,k) = C(n-1,k-1) + C(n-1,k)
   Symmetry      C(n,k) = C(n,n-k)
   Absorption    k*C(n,k) = n*C(n-1,k-1)
   Hockey stick  sum_{i=r..n} C(i,r) = C(n+1, r+1)
   Vandermonde   sum_k C(m,k) C(n,p-k) = C(m+n, p)
   sum_k C(n,k) = 2^n           sum_k (-1)^k C(n,k) =
  ↪ [n==0]
   sum_k k*C(n,k) = n*2^(n-1)   sum_k C(n,k)^2 =
  ↪ C(2n,n)
   sum_k C(k,r) x^k ... upper negation: C(-n,k) =
  ↪ (-1)^k C(n+k-1, k)
   sum_{k} C(n,k) C(k,m) = C(n,m) 2^(n-m)
   Binomial inversion: f(n)=sum_k C(n,k) g(k)  <=>
  ↪ g(n)=sum_k (-1)^(n-k) C(n,k) f(k)

 INCLUSION-EXCLUSION
   |A1 u..u An| = sum |Ai| - sum |Ai n Aj| + ...
   #surjections [n]->[k] = sum_{i=0..k} (-1)^i C(k,i)
  ↪ (k-i)^n = k! * S2(n,k)
   "exactly m" from "at least m":  E(m) = sum_{j>=m}
  ↪ (-1)^(j-m) C(j,m) A(j)
   derangements D(n) = n! sum_{k=0..n} (-1)^k/k! =
  ↪ (n-1)(D(n-1)+D(n-2)), D0=1,D1=0
   permutations with exactly k fixed points = C(n,k)
```

```
  ↪ * D(n-k)

CATALAN  Cat(n) = C(2n,n)/(n+1) = C(2n,n) -
  ↪ C(2n,n+1),  Cat(n+1) = sum Cat(i)Cat(n-i)
  1,1,2,5,14,42,132,429,1430,4862  - counts:
  ↪ balanced bracket strings of length 2n;
  binary trees with n nodes; triangulations of an
  ↪ (n+2)-gon; monotone lattice paths under
  the diagonal; stack-sortable permutations;
  ↪ non-crossing partitions/chords.
  BALLOT / CYCLE LEMMA: paths from 0 to k in n steps
  ↪ staying > 0 = (k/n) C(n, (n+k)/2)
  REFLECTION: #paths (0,0)->(a,b) touching the line
  ↪ y=x+c  =  #paths to the reflected point

STIRLING
  2nd kind S2(n,k) = #partitions of n LABELLED items
  ↪ into k non-empty UNLABELLED blocks
     S2(n,k) = k*S2(n-1,k) + S2(n-1,k-1);  S2(n,k) =
  ↪ (1/k!) sum_i (-1)^i C(k,i)(k-i)^n
     x^n = sum_k S2(n,k) * x_falling(k)
  ↪ surjections = k! S2(n,k)
  1st kind (unsigned) c(n,k) = #permutations of n
  ↪ with exactly k cycles
     c(n,k) = c(n-1,k-1) + (n-1) c(n-1,k);   sum_k
  ↪ c(n,k) x^k = x(x+1)..(x+n-1)
  BELL B(n) = sum_k S2(n,k) = #set partitions.
  ↪ B(n+1) = sum_k C(n,k) B(k).  Bell triangle.

PARTITIONS (unordered, into positive parts)
  p(n) via pentagonal number theorem, O(n sqrt n):
    p(n) = sum_{k>=1} (-1)^(k+1) [ p(n - k(3k-1)/2)
  ↪ + p(n - k(3k+1)/2) ]
  partitions into <= k parts = partitions into parts
  ↪ <= k (conjugate)

TWELVEFOLD WAY  (n balls into k boxes)
  balls  boxes  any                 injective
  ↪ surjective
  dist.  dist.  k^n                 k!/(k-n)!
  ↪ k! S2(n,k)
  ident. dist.  C(n+k-1, n)         C(k,n)
  ↪ C(n-1, n-k)
  dist.  ident. sum_{i<=k} S2(n,i)  [n<=k]
  ↪ S2(n,k)
  ident. ident. p_k(n)              [n<=k]
  ↪ p(n into exactly k parts)

BURNSIDE / POLYA   #orbits = (1/|G|) sum_{g in G}
  ↪ |fix(g)|
  necklaces (rotations), n beads, k colours:  (1/n)
  ↪ sum_{d|n} phi(n/d) k^d
  bracelets (+ reflections): (necklace sum +
  ↪ reflection sum) / (2n),
    reflection sum = n*k^((n+1)/2) if n odd,
  ↪ (n/2)(k^(n/2) + k^(n/2+1)) if n even

TREES / GRAPHS
  Cayley: n^(n-2) labelled trees on n vertices; with
  ↪ given degrees d_i: (n-2)! / prod (d_i-1)!
  Prufer code <-> labelled tree, bijective
  Matrix-Tree: #spanning trees = any cofactor of the
  ↪ Laplacian (D - A)
  LGV: #non-intersecting path systems = det( M )
  ↪ with M_ij = #paths a_i -> b_j

GENERATING FUNCTIONS
  OGF   1/(1-x) = sum x^n           1/(1-x)^k = sum
  ↪ C(n+k-1, k-1) x^n
        x/(1-x-x^2) = Fibonacci
  ↪ (1-sqrt(1-4x))/(2x) = Catalan
        prod 1/(1-x^i) = partitions  prod (1+x^i) =
  ↪ distinct-part partitions
   EGF   e^x = sum x^n/n!            EGF of set
  ↪ partitions = e^(e^x - 1)
   EXPONENTIAL FORMULA: if C(x) is the EGF of
  ↪ connected structures, e^C(x) is the EGF of all
   Coefficient extraction: [x^n] A(x)B(x) = sum_k a_k
  ↪ b_(n-k)  (a convolution -> NTT)
   Lagrange inversion, for F(0)=0, F'(0)!=0:  [x^n]
  ↪ G(F^-1)(x) = (1/n)[x^(n-1)] G'(x)(x/F(x))^n

 MISC
   hook length: #standard Young tableaux of shape
  ↪ lambda = n! / prod(hook lengths)
   Erdos-Ko-Rado, Dilworth (min chain cover = max
  ↪ antichain), Sperner (max antichain C(n,n/2))
============================================================
  ↪ */
```

5.4 CatalanStirlingBell

Catalan, Stirling (both kinds), Bell, derangements, partitions. Needs Mint + buildFact().

```
Mint catalan(int n) { return C(2 * n, n) * inv_[n +
  ↪ 1]; }
// paths from 0 to k in n steps (+-1) that stay
  ↪ strictly positive - ballot / cycle lemma
Mint ballot(int n, int k) {
    if (n == 0) return k == 0;
  ↪ // the empty path
    if (k <= 0 || k > n || (n + k) % 2) return 0;
  ↪ // k <= 0 was unguarded: Mint(-2) wraps
    return Mint(k) * inv_[n] * C(n, (n + k) / 2);
}

Mint derange(int n) {
  ↪   // D(n) = (n-1)(D(n-1)+D(n-2))
    Mint a = 1, b = 0;
  ↪   // D0=1, D1=0
    for (int i = 2; i <= n; i++) { Mint c = Mint(i -
  ↪ 1) * (a + b); a = b, b = c; }
    return n ? b : a;
}
// permutations of n with exactly k fixed points
Mint fixedPoints(int n, int k) { return C(n, k) *
  ↪ derange(n - k); }

// Stirling 2nd kind, single value: S2(n,k) = (1/k!)
  ↪ sum_i (-1)^i C(k,i) (k-i)^n     O(k log n)
Mint stirling2(ll n, int k) {
    Mint r = 0;
    for (int i = 0; i <= k; i++) {
        Mint t = C(k, i) * Mint(k - i).pow(n);
        (i & 1) ? r -= t : r += t;
    }
    return r * invFact[k];
}
Mint surjections(ll n, int k) { return stirling2(n,
  ↪ k) * fact[k]; }

// Whole ROW of Stirling 2nd (all k for fixed n) in
  ↪ O(n^2), or O(n log n) as a convolution of
// a_i = (-1)^i/i! and b_i = i^n/i!  (NTT).
vector<Mint> stirling2Row(int n) {
```

```
    vector<Mint> a(n + 1), b(n + 1), r(n + 1, 0);
    for (int i = 0; i <= n; i++) a[i] = (i & 1 ?
  ↪ Mint(0) - invFact[i] : invFact[i]), b[i] =
  ↪ Mint(i).pow(n) * invFact[i];
    for (int k = 0; k <= n; k++) for (int i = 0; i <=
  ↪ k; i++) r[k] += a[i] * b[k - i];
    return r;
}
// Stirling 1st kind (unsigned), whole row:
  ↪ prod_{i=0..n-1} (x + i)   O(n^2)
vector<Mint> stirling1Row(int n) {
    vector<Mint> p = {1};
    for (int i = 0; i < n; i++) {
        vector<Mint> q(p.size() + 1, 0);
        for (size_t j = 0; j < p.size(); j++) q[j +
  ↪ 1] += p[j], q[j] += p[j] * Mint(i);
        p = q;
    }
    return p;
  ↪   // p[k] = c(n, k)
}
// Bell numbers B(0..n) via the Bell triangle, O(n^2)
vector<Mint> bell(int n) {
    vector<Mint> B(n + 1), row = {1};
    B[0] = 1;
    for (int i = 1; i <= n; i++) {
        vector<Mint> nr = {row.back()};
        for (Mint x : row) nr.push_back(nr.back() +
  ↪ x);
        row = nr, B[i] = row[0];
    }
    return B;
}
// Integer partitions p(0..n) via the pentagonal
  ↪ number theorem, O(n sqrt n)
vector<Mint> partitions(int n) {
    vector<Mint> p(n + 1, 0);
    p[0] = 1;
    for (int i = 1; i <= n; i++)
        for (int k = 1;; k++) {
            ll g1 = (ll)k * (3 * k - 1) / 2, g2 =
  ↪ (ll)k * (3 * k + 1) / 2;
            if (g1 > i) break;
            Mint t = p[i - g1] + (g2 <= i ? p[i - g2]
  ↪ : Mint(0));
            (k & 1) ? p[i] += t : p[i] -= t;
        }
    return p;
}
```

5.5 InclusionExclusion

INCLUSION-EXCLUSION recipes. Needs Mint + buildFact().

```
// x1+..+xk = n with 0 <= xi <= ub[i].  IE over which
  ↪ variables overflow their bound.
// O(2^k) - fine for k <= ~20. With equal bounds use
  ↪ the closed form below instead.
Mint boundedStarsBars(ll n, const vector<ll>& ub) {
    int k = ub.size();
    Mint r = 0;
    for (int m = 0; m < (1 << k); m++) {
        ll rem = n;
        for (int i = 0; i < k; i++) if (m >> i & 1)
  ↪ rem -= ub[i] + 1;
        if (rem < 0) continue;
        Mint t = C(rem + k - 1, k - 1);
        (__builtin_popcount(m) & 1) ? r -= t : r += t;
    }
    return r;
}
// Same with a COMMON bound c: sum_j (-1)^j C(k,j)
  ↪ C(n - j(c+1) + k-1, k-1).   O(k)
Mint boundedStarsBarsEqual(ll n, ll k, ll c) {
    Mint r = 0;
    for (ll j = 0; j <= k && j * (c + 1) <= n; j++) {
        Mint t = C(k, j) * C(n - j * (c + 1) + k - 1,
  ↪ k - 1);
        (j & 1) ? r -= t : r += t;
    }
    return r;
}
// #surjections [n] -> [k]  (equivalently k! *
  ↪ S2(n,k))
Mint surj(ll n, ll k) {
    Mint r = 0;
    for (ll i = 0; i <= k; i++) {
        Mint t = C(k, i) * Mint(k - i).pow(n);
        (i & 1) ? r -= t : r += t;
    }
    return r;
}
// "AT LEAST" -> "EXACTLY" (binomial / Mobius
  ↪ inversion over the subset lattice):
//   if A(j) = #configurations having at least a
  ↪ chosen set of j properties (summed over sets),
//   then E(m) = sum_{j>=m} (-1)^(j-m) C(j,m) A(j)
  ↪ counts those with EXACTLY m.
Mint exactlyFromAtLeast(const vector<Mint>& A, int m)
  ↪ {
    Mint r = 0;
    for (int j = m; j < (int)A.size(); j++) {
        Mint t = C(j, m) * A[j];
        ((j - m) & 1) ? r -= t : r += t;
    }
    return r;
}
// BINOMIAL INVERSION:  f(n) = sum_k C(n,k) g(k)
  ↪ <=>   g(n) = sum_k (-1)^(n-k) C(n,k) f(k)
vector<Mint> binomialInvert(const vector<Mint>& f) {
    int n = f.size();
    vector<Mint> g(n, 0);
    for (int i = 0; i < n; i++)
        for (int k = 0; k <= i; k++) {
            Mint t = C(i, k) * f[k];
            ((i - k) & 1) ? g[i] -= t : g[i] += t;
        }
    return g;
}
// COUNTING COPRIME PAIRS / "gcd is exactly g": count
  ↪ multiples then Mobius-invert downwards:
//   exact[g] = cntMultiples[g] - sum_{g | d, d > g}
  ↪ exact[d]      (iterate g downwards)
vector<ll> exactGcdCounts(const vector<ll>&
  ↪ cntMultiples) {
    int n = cntMultiples.size() - 1;
    vector<ll> e(n + 1);
    for (int g = n; g >= 1; g--) {
        e[g] = cntMultiples[g];
        for (int d = 2 * g; d <= n; d += g) e[g] -=
  ↪ e[d];
    }
    return e;
}
```

5.6 TreeCounting

COUNTING TREES AND GRAPHS.

```
// Needs: Mint (06 - Math/01) and determinant (06 -
  ↪ Math/04 - Linear Algebra/02).

// PRUFER CODE: bijection between labelled trees on
  ↪ n>=2 vertices and sequences in [0,n)^(n-2).
// Immediate corollaries: Cayley n^(n-2) labelled
  ↪ trees; with prescribed degrees d_i the count is
// (n-2)! / prod (d_i - 1)!  (because vertex i
  ↪ appears exactly d_i - 1 times in the code).
vector<int> toPrufer(const vector<vector<int>>& adj)
  ↪ {     // repeatedly strip the SMALLEST leaf
    int n = adj.size();
    if (n <= 2) return {};
    vector<int> deg(n), code;
    vector<bool> gone(n, false);
    priority_queue<int, vector<int>, greater<int>> q;
    for (int i = 0; i < n; i++) if ((deg[i] =
  ↪ adj[i].size()) == 1) q.push(i);
    for (int it = 0; it < n - 2; it++) {
        int leaf = q.top(); q.pop();
        gone[leaf] = true;
        int nb = -1;
        for (int v : adj[leaf]) if (!gone[v]) nb = v;
        code.push_back(nb);
        if (--deg[nb] == 1) q.push(nb);
    }
    return code;
}
vector<pair<int, int>> fromPrufer(vector<int> code) {
    int n = code.size() + 2;
    vector<int> deg(n, 1);
    for (int x : code) deg[x]++;
    priority_queue<int, vector<int>, greater<int>> q;
    for (int i = 0; i < n; i++) if (deg[i] == 1)
  ↪ q.push(i);
    vector<pair<int, int>> e;
    for (int x : code) {
        int leaf = q.top(); q.pop();
        e.push_back({leaf, x});
        if (--deg[x] == 1) q.push(x);
    }
    int a = q.top(); q.pop();
    e.push_back({a, q.top()});
    return e;
}

// MATRIX-TREE THEOREM: #spanning trees of a graph =
  ↪ any cofactor of the Laplacian L = D - A.
// Multigraphs work (put the multiplicity in A).
  ↪ Needs determinant() from GaussianElimination.
Mint spanningTrees(int n, const vector<pair<int,
  ↪ int>>& edges) {
    if (n == 1) return 1;
    vector<vector<Mint>> L(n, vector<Mint>(n, 0));
    for (auto [u, v] : edges) {
        L[u][u] += 1, L[v][v] += 1;
        L[u][v] -= 1, L[v][u] -= 1;
    }
    vector<vector<Mint>> M(n - 1, vector<Mint>(n -
  ↪ 1));    // delete row/col 0
    for (int i = 1; i < n; i++) for (int j = 1; j <
  ↪ n; j++) M[i - 1][j - 1] = L[i][j];
    return determinant(M);
}
// DIRECTED version (Tutte): #arborescences rooted at
  ↪ r = cofactor of (Dout - A) deleting row/col r
// for in-trees, or (Din - A) for out-trees.
// WEIGHTED version: put edge weights in A and the
  ↪ weighted degree in D -> sum over spanning
// trees of the product of weights (useful mod p for
  ↪ "sum over all spanning trees" problems).

// EXPONENTIAL FORMULA: if C(x) is the EGF of
  ↪ CONNECTED structures then e^(C(x)) is the EGF of
// all structures.  Concretely, with g_n = #labelled
  ↪ graphs on n nodes = 2^C(n,2):
//   c_n = g_n - sum_{k=1}^{n-1} C(n-1, k-1) * c_k *
  ↪ g_{n-k}      (connected labelled graphs)
vector<Mint> connectedLabelledGraphs(int n) {
    vector<Mint> g(n + 1), c(n + 1, 0);
    for (int i = 0; i <= n; i++) g[i] =
  ↪ Mint(2).pow((ll)i * (i - 1) / 2);
    for (int i = 1; i <= n; i++) {
        c[i] = g[i];
        for (int k = 1; k < i; k++) c[i] -= C(i - 1,
  ↪ k - 1) * c[k] * g[i - k];
    }
    return c;
}
```

5.7 SubsetConvolution

TRANSFORMS OVER THE SUBSET LATTICE. n bits, arrays of size 2^n.

```
// Needs: Mint (06 - Math/01 - Modular Arithmetic/01
  ↪ - Mint.cpp).

// ZETA (sum over subsets) and its inverse MOBIUS.
  ↪ O(n 2^n). This is SOS DP.
void zetaSub(vector<Mint>& f, int n) {
  ↪     // f[m] := sum_{s subset of m} f[s]
    for (int i = 0; i < n; i++)
        for (int m = 0; m < (1 << n); m++) if (m >> i
  ↪ & 1) f[m] += f[m ^ (1 << i)];
}
void mobiusSub(vector<Mint>& f, int n) {
  ↪     // exact inverse of zetaSub
    for (int i = 0; i < n; i++)
        for (int m = 0; m < (1 << n); m++) if (m >> i
  ↪ & 1) f[m] -= f[m ^ (1 << i)];
}
void zetaSuper(vector<Mint>& f, int n) {
  ↪     // f[m] := sum_{m subset of s} f[s]
    for (int i = 0; i < n; i++)
        for (int m = 0; m < (1 << n); m++) if (!(m >>
  ↪ i & 1)) f[m] += f[m | (1 << i)];
}
void mobiusSuper(vector<Mint>& f, int n) {
    for (int i = 0; i < n; i++)
        for (int m = 0; m < (1 << n); m++) if (!(m >>
  ↪ i & 1)) f[m] -= f[m | (1 << i)];
}
// OR-convolution: h[m] = sum_{a|b = m} f[a] g[b]
  ↪  -> zetaSub, multiply, mobiusSub
// AND-convolution: h[m] = sum_{a&b = m} f[a] g[b]
  ↪  -> zetaSuper, multiply, mobiusSuper
vector<Mint> orConv(vector<Mint> f, vector<Mint> g,
  ↪ int n) {
    zetaSub(f, n), zetaSub(g, n);
    for (int m = 0; m < (1 << n); m++) f[m] *= g[m];
    return mobiusSub(f, n), f;
}
vector<Mint> andConv(vector<Mint> f, vector<Mint> g,
```

```
    ↪ int n) {
    zetaSuper(f, n), zetaSuper(g, n);
    for (int m = 0; m < (1 << n); m++) f[m] *= g[m];
    return mobiusSuper(f, n), f;
}

// SUBSET (disjoint) CONVOLUTION: h[m] = sum_{a|b =
  ↪ m, a&b = 0} f[a] g[b].   O(2^n n^2).
// The ranked-zeta trick: index by popcount, convolve
  ↪ the ranks like polynomials.
vector<Mint> subsetConv(const vector<Mint>& f, const
  ↪ vector<Mint>& g, int n) {
    int N = 1 << n;
    vector<vector<Mint>> F(n + 1, vector<Mint>(N,
  ↪ 0)), G = F, H = F;
    for (int m = 0; m < N; m++)
  ↪ F[__builtin_popcount(m)][m] = f[m],
  ↪ G[__builtin_popcount(m)][m] = g[m];
    for (int i = 0; i <= n; i++) zetaSub(F[i], n),
  ↪ zetaSub(G[i], n);
    for (int i = 0; i <= n; i++)
        for (int j = 0; i + j <= n; j++)
            for (int m = 0; m < N; m++) H[i + j][m]
  ↪ += F[i][m] * G[j][m];
    for (int i = 0; i <= n; i++) mobiusSub(H[i], n);
    vector<Mint> h(N);
    for (int m = 0; m < N; m++) h[m] =
  ↪ H[__builtin_popcount(m)][m];
    return h;
}
// USES: exact set cover / "partition the ground set
  ↪ into k valid groups" (repeated subset
// convolution, or exponentiate the EGF-like series);
  ↪ counting perfect matchings by subsets;
// "for each mask, best way to split it into two
  ↪ disjoint valid halves".
// SUBMASK ENUMERATION over all masks is O(3^n):  for
  ↪ (s = m; s; s = (s-1) & m)
```

5.8 SpecialSequences

Named sequences that keep showing up, with what each one counts. Needs Mint + buildFact().

```
// MOTZKIN M(n): lattice paths (0,0)->(n,0) with
  ↪ steps U/D/FLAT staying >= 0; also the number of
// ways to draw non-crossing chords on n points on a
  ↪ circle. M: 1,1,2,4,9,21,51,127
vector<Mint> motzkin(int n) {
    vector<Mint> m(n + 1, 0);
    m[0] = 1;
    if (n >= 1) m[1] = 1;
    for (int i = 2; i <= n; i++) {
  ↪     // M(i) = M(i-1) + sum M(k)M(i-2-k)
        m[i] = m[i - 1];
        for (int k = 0; k <= i - 2; k++) m[i] += m[k]
  ↪ * m[i - 2 - k];
    }
    return m;
}
// NARAYANA N(n,k): Dyck paths of length 2n with
  ↪ exactly k peaks. sum_k N(n,k) = Catalan(n).
Mint narayana(int n, int k) { return k < 1 || k > n ?
  ↪ Mint(0) : C(n, k) * C(n, k - 1) * inv_[n]; }
// SCHRODER: large R(n) counts lattice paths
  ↪ (0,0)->(n,n) with steps E/N/NE staying below y=x.
// R: 1,2,6,22,90,394.  R(n) = R(n-1) + sum_{k} R(k)
  ↪ R(n-1-k)  (little schroeder s(n) = R(n)/2 for n
  ↪ >= 1 only (s(0) = 1, not 1/2))
vector<Mint> schroder(int n) {
    vector<Mint> r(n + 1, 0);
    r[0] = 1;
    for (int i = 1; i <= n; i++) { r[i] = r[i - 1];
  ↪ for (int k = 0; k < i; k++) r[i] += r[k] * r[i -
  ↪ 1 - k]; }
    return r;
}
// EULERIAN A(n,k): permutations of n with exactly k
  ↪ ascents.
// A(n,k) = (k+1) A(n-1,k) + (n-k) A(n-1,k-1).
  ↪ sum_k A(n,k) = n!
vector<vector<Mint>> eulerian(int n) {
    vector<vector<Mint>> a(n + 1, vector<Mint>(n + 1,
  ↪ 0));
    a[0][0] = 1;
    for (int i = 1; i <= n; i++)
        for (int k = 0; k < i; k++)
            a[i][k] = Mint(k + 1) * a[i - 1][k] +
  ↪ Mint(i - k) * (k ? a[i - 1][k - 1] : Mint(0));
    return a;
}
// q-BINOMIAL (Gaussian) coefficient: #k-dim
  ↪ subspaces of an n-dim space over F_q; also
// #partitions fitting in a k x (n-k) box, weighted
  ↪ by q^area.
// PRECONDITION: ord_p(q) > k. The denominator is
  ↪ prod (q^i - 1), which vanishes in F_p
// whenever ord_p(q) <= k - NOT only at q == 1. (q =
  ↪ -1 mod 998244353 breaks every k >= 2.)
// Root-of-unity-safe fallback, O(nk): Pascal's rule
  ↪ [n,k]_q = [n-1,k-1]_q + q^k [n-1,k]_q.
Mint qBinom(int n, int k, Mint q) {
  ↪     // prod_{i=1..k} (q^(n-k+i)-1)/(q^i-1)
    if (k < 0 || k > n) return 0;
    if (q == Mint(1)) return C(n, k);
  ↪     // q=1 is the 0/0 limit
    Mint num = 1, den = 1;
    for (int i = 1; i <= k; i++) num *= q.pow(n - k +
  ↪ i) - 1, den *= q.pow(i) - 1;
    return num / den;
}
// HOOK LENGTH FORMULA: #standard Young tableaux of
  ↪ shape lambda = n! / prod(hook lengths).
// hook(i,j) = (arm to the right) + (leg below) + 1.
Mint youngTableaux(const vector<int>& lam) {
    int n = 0;
    for (int x : lam) n += x;
    vector<int> conj(lam.empty() ? 0 : lam[0], 0);
    for (int i = 0; i < (int)lam.size(); i++) for
  ↪ (int j = 0; j < lam[i]; j++) conj[j]++;
    Mint den = 1;
    for (int i = 0; i < (int)lam.size(); i++)
        for (int j = 0; j < lam[i]; j++) den *=
  ↪ Mint(lam[i] - j + conj[j] - i - 1);
    return fact[n] / den;
}
// LGV LEMMA: for sources a_1..a_k and sinks b_1..b_k
  ↪ in a DAG, the number of NON-INTERSECTING
// path systems equals det(M) with M[i][j] = #paths
  ↪ a_i -> b_j (when only the identity
// permutation can be non-intersecting). Counts plane
  ↪ partitions, rhombus tilings, etc.
```

5.9 EulerTransformAndSymbolic

SYMBOLIC METHOD: spec -> generating function

```
// Needs: Mint (06 - Math/01 - Modular Arithmetic/01 - Mint.cpp).
/* ============ SYMBOLIC METHOD: spec -> generating function ============
 Decide labelled (EGF) or unlabelled (OGF), then translate the construction:
  UNLABELLED (OGF), A built from atoms with GF B(z):
    SEQ  (ordered list)     A = 1 / (1 - B)
    MSET (unordered, rep)   A = exp( sum_{k>=1} B(z^k) / k )          <- Euler transform
    PSET (unordered, no rep)A = exp( sum_{k>=1} (-1)^(k-1) B(z^k)/k )
    CYC  (up to rotation)   A = sum_{k>=1} (phi(k)/k) * log(1/(1 - B(z^k)))
  LABELLED (EGF):
    SEQ  A = 1/(1-B)        SET  A = exp(B)
    CYC  A = log(1/(1-B))
 "SET of connected pieces = exp(connected)" is the exponential formula.
================================================================================*/

// EULER TRANSFORM: a[n] = #atoms of size n  ->  b[n] = #MULTISETS of atoms of total size n.
// prod_{n>=1} (1 - x^n)^(-a_n) = 1 + sum b_n x^n.   O(n log n) via the harmonic sum.
vector<Mint> eulerTransform(const vector<Mint>& a) {  // a[0] unused; returns b with b[0] = 1
    int n = a.size() - 1;
    vector<Mint> c(n + 1, 0), b(n + 1, 0);
    for (int d = 1; d <= n; d++)  // c_n = sum_{d | n} d * a_d
        for (int m = d; m <= n; m += d) c[m] += Mint(d) * a[d];
    b[0] = 1;
    for (int i = 1; i <= n; i++) {  // b_n = (c_n + sum_{k<n} c_k b_{n-k}) / n
        Mint s = c[i];
        for (int k = 1; k < i; k++) s += c[k] * b[i - k];
        b[i] = s * inv_[i];
    }
    return b;
}
int muSmall(int n) {  // Mobius of one small n, by trial division
    int r = 1;
    for (int p = 2; (ll)p * p <= n; p++) if (n % p == 0) {
        n /= p;
        if (n % p == 0) return 0;
        r = -r;
    }
    return n > 1 ? -r : r;
}
// INVERSE Euler transform: b -> a  (given multiset counts, recover the atom counts).
vector<Mint> eulerInverse(const vector<Mint>& b) {
    int n = b.size() - 1;
    vector<Mint> c(n + 1, 0), a(n + 1, 0);
    for (int i = 1; i <= n; i++) {  // c_n = n*b_n - sum_{d<n} c_d b_{n-d}
        Mint s = Mint(i) * b[i];
        for (int d = 1; d < i; d++) s -= c[d] * b[i - d];
        c[i] = s;
    }
    for (int i = 1; i <= n; i++) {  // a_n = (1/n) sum_{d|n} mu(n/d) c_d
        Mint s = 0;
        for (int d = 1; d <= i; d++) if (i % d == 0) s += Mint(muSmall(i / d)) * c[d];
        a[i] = s * inv_[i];
    }
    return a;
}
// UNLABELLED ROOTED TREES: r_1 = 1, and r_{n+1} = (Euler transform of r_1..r_n)[n]
// (a rooted tree = a root plus a MULTISET of rooted subtrees).
vector<Mint> rootedTrees(int n) {
    vector<Mint> r(n + 1, 0);
    if (n >= 1) r[1] = 1;
    for (int i = 1; i < n; i++) {
        vector<Mint> a(i + 1, 0);
        for (int j = 1; j <= i; j++) a[j] = r[j];
        auto b = eulerTransform(a);
        r[i + 1] = b[i];
    }
    return r;  // 0,1,1,2,4,9,20,48,115,286,719,...
}
// PARTITIONS INTO PARTS FROM A SET S = eulerTransform with a[s] = 1 for s in S.
// COMPOSITIONS (ordered) use SEQ instead: 1/(1 - sum x^s).
```

5.10 PermanentAndSymmetric

RYSER'S FORMULA - permanent of an n x n matrix in O(2^n * n).

```
// perm(A) counts PERFECT MATCHINGS of a bipartite graph (weighted: sum over matchings of the
// product of weights). Use for n <= ~20: "count ways to assign n tasks to n people",
// "count systems of distinct representatives", "count permutations avoiding forbidden cells".
Mint permanent(const vector<vector<Mint>>& a) {
    int n = a.size();
    vector<Mint> row(n, 0);  // row[i] = sum_{j in S} a[i][j]
    Mint total = 0;
    int prev = 0;
    for (int g = 1; g < (1 << n); g++) {  // Gray code: one column toggles per step
        int cur = g ^ (g >> 1), diff = cur ^ prev, j = __builtin_ctz(diff);
        if (cur & diff) for (int i = 0; i < n; i++) row[i] += a[i][j];   // column j entered S
        else for (int i = 0; i < n; i++) row[i] -= a[i][j];              // column j left S
        prev = cur;
        Mint p = 1;
        for (int i = 0; i < n; i++) p *= row[i];
        (__builtin_popcount(cur) & 1) ? total -= p : total += p;
    }
    return (n & 1) ? Mint(0) - total : total;  // perm = (-1)^n * sum_S (-1)^|S| prod_i rowSum
}
// PERMANENT vs DETERMINANT: identical formula without the signs. det is polynomial-time,
// perm is #P-hard - never expect an O(n^3) permanent.
```

```
// NEWTON'S IDENTITIES - convert between power sums
  ↪ p_k = sum x_i^k and elementary symmetric e_k.
// e_k = sum over k-subsets of the product; this is
  ↪ exactly "for every k, the sum over all
// k-subsets of the product of the chosen values".
//   e_0 = 1;   k * e_k = sum_{i=1..k} (-1)^(i-1) *
  ↪ e_(k-i) * p_i
vector<Mint> powerToElementary(const vector<Mint>& p,
  ↪ int n) {   // p[1..n] -> e[0..n]
    vector<Mint> e(n + 1, 0);
    e[0] = 1;
    for (int k = 1; k <= n; k++) {
        Mint s = 0;
        for (int i = 1; i <= k; i++) {
            Mint t = e[k - i] * p[i];
            (i & 1) ? s += t : s -= t;
        }
        e[k] = s * inv_[k];
    }
    return e;
}
//   p_k = (-1)^(k-1) * k * e_k + sum_{i=1..k-1}
  ↪ (-1)^(k-1+i) * e_(k-i) * p_i
vector<Mint> elementaryToPower(const vector<Mint>& e,
  ↪ int n) {   // e[0..n] -> p[1..n]
    vector<Mint> p(n + 1, 0);
    for (int k = 1; k <= n; k++) {
        Mint s = Mint(k) * e[k];
        if ((k - 1) & 1) s = Mint(0) - s;
  ↪  // (-1)^(k-1) * k * e_k
        for (int i = 1; i < k; i++) {
            Mint t = e[k - i] * p[i];
            ((k - 1 + i) & 1) ? s -= t : s += t;
        }
        p[k] = s;
    }
    return p;
}
// e_k are also the coefficients of prod (x + x_i):
  ↪ prod_{i} (x + x_i) = sum_k e_k * x^(n-k).
// So "sum over k-subsets of the product" =
  ↪ coefficient extraction from that product, which
  ↪ you
// can also get in O(n log^2 n) by divide-and-conquer
  ↪ multiplication of the linear factors.
```

5.11 BernoulliFaulhaber

BERNOULLI NUMBERS and the CLOSED-FORM FAULHABER polynomial.

```
// Use this (not Lagrange interpolation) when you
  ↪ need the POLYNOMIAL COEFFICIENTS of
// sum_{k=1..n} k^p, or when p is large and many n
  ↪ are queried.
// Convention here: B_1 = -1/2  (the "B^-" convention
  ↪ that the formula below expects).
//   B_0 = 1 ;   sum_{k=0}^{m} C(m+1, k) * B_k = [m
  ↪ == 0]
//   =>  B_m = -(1/(m+1)) * sum_{k=0}^{m-1} C(m+1, k)
  ↪ * B_k
vector<Mint> bernoulli(int m) {
  ↪  // B[0..m], O(m^2)
    vector<Mint> B(m + 1, 0);
    B[0] = 1;
    for (int i = 1; i <= m; i++) {
        Mint s = 0;
        for (int k = 0; k < i; k++) s += C(i + 1, k)
  ↪ * B[k];
        B[i] = Mint(0) - s * inv_[i + 1];
    }
    return B;
  ↪  // 1, -1/2, 1/6, 0, -1/30, 0, 1/42, ...
}
// O(m log m) alternative: B_m / m! = [t^m] inv(
  ↪ sum_{i>=0} t^i / (i+1)! )  -- one Poly::inv.

// FAULHABER:  sum_{k=1}^{n} k^p = (1/(p+1)) *
  ↪ sum_{r=0}^{p} (-1)^r * C(p+1, r) * B_r *
  ↪ n^(p+1-r)
Mint powerSumClosed(ll n, int p, const vector<Mint>&
  ↪ B) {
    Mint s = 0, np = Mint(n).pow(p + 1);
  ↪  // n^(p+1-r), decreasing r
    vector<Mint> pw(p + 2);
    pw[0] = 1;
    for (int i = 1; i <= p + 1; i++) pw[i] = pw[i -
  ↪ 1] * Mint(n);
    for (int r = 0; r <= p; r++) {
        Mint t = C(p + 1, r) * B[r] * pw[p + 1 - r];
        (r & 1) ? s -= t : s += t;
    }
    return s * inv_[p + 1];
}
// The COEFFICIENTS of the Faulhaber polynomial in n
  ↪ (degree p+1):
//   [n^(p+1-r)] = (-1)^r * C(p+1, r) * B_r / (p+1)
vector<Mint> faulhaberPoly(int p, const vector<Mint>&
  ↪ B) {
    vector<Mint> c(p + 2, 0);
    for (int r = 0; r <= p; r++) {
        Mint t = C(p + 1, r) * B[r] * inv_[p + 1];
        c[p + 1 - r] = (r & 1) ? Mint(0) - t : t;
    }
    return c;
  ↪  // c[i] = coefficient of n^i
}
// RELATED SUMS
//   sum_{k=0}^{n} C(k, r)      = C(n+1, r+1)
  ↪             (hockey stick)
//   sum_{k=1}^{n} k^p * r^k    : degree-p polynomial
  ↪ times r^n; solve by ansatz or by
//                                differentiating the
  ↪ geometric series p times.
//   Bernoulli also gives Euler-Maclaurin and the von
  ↪ Staudt-Clausen denominator facts.
```

5.12 CycleLemmaFussCatalan

CYCLE LEMMA (Dvoretzky-Motzkin) and the FUSS-CATALAN / rational-Catalan family.

```
// The general-k version of the ballot problem: it
  ↪ handles every "+1 / -k step" path problem.
//
// CYCLE LEMMA: a word with m a's and n b's, m >=
  ↪ k*n, has EXACTLY m - k*n cyclic shifts that
// are k-dominating (every non-empty prefix has #a >
  ↪ k * #b).
//   k = 1 ballot: p A's and q B's with p > q  ->
  ↪ exactly p - q dominating shifts,
//                  so the probability that A leads
  ↪ the whole count is (p - q) / (p + q).

// #paths with p up-steps and q down-steps that stay
  ↪ STRICTLY positive after the first step.
Mint ballotStrict(ll p, ll q) {
```

```
    // p > q >= 0
    if (p <= q) return 0;
    return C(p + q, q) * Mint(p - q) * inv_[(int)(p +
    q)];
}
// FUSS-CATALAN: A_m(p, r) = (r / (m*p + r)) * C(m*p
    + r, m)
Mint fussCatalan(ll m, ll p, ll r) {
    if (m < 0) return 0;
    return C(m * p + r, m) * Mint(r) * Mint(m * p +
    r).inv();
}
// #p-ary trees with m internal nodes = A_m(p, 1) =
    C(m*p + 1, m) / (m*p + 1)
Mint pAryTrees(ll m, ll p) { return fussCatalan(m, p,
    1); }
// Catalan is the case p = 2, r = 1.
// #sequences of m opens and (p-1)*m closes where
    every prefix keeps the stack non-negative
// is also A_m(p, 1).

// RATIONAL CATALAN: for gcd(a, b) = 1, the number of
    monotone lattice paths from (0,0) to
// (b, a) that never rise above the line a*x = b*y is
    C(a + b, a) / (a + b).
Mint rationalCatalan(ll a, ll b) { return C(a + b, a)
    * Mint(a + b).inv(); }

// CHUNG-FELLER: among the C(2n,n) balanced +-1 paths
    of length 2n, the number having exactly
// 2r steps above the axis is Catalan(n) for EVERY r
    in 0..n.  Hence C(2n,n) = (n+1)*Cat(n).
// Use it for "count sequences with exactly k flaws"
    - the answer does not depend on k.

// REFLECTION PRINCIPLE (the tool behind all of the
    above):
//   #paths (0,0)->(x,y) touching the line y = m  =
    #paths (0,0)->(x, 2m - y)
//   so "#paths avoiding the line" = C(total, up) -
    C(total, up shifted by the reflection).
// PRECONDITION: the barrier must SEPARATE the start
    (height 0) from the end y, i.e. m != 0
// and 0, y strictly on the same side of m. Without
    it the reflection subtracts too much:
// (x,y,m) = (2,2,1) returns -1 where the true count
    is 0.
Mint pathsAvoidingLine(ll x, ll y, ll m) {
    if (m == 0 || (m > 0 && y >= m) || (m < 0 && y <=
    m)) return 0;             // steps +1/-1, must
    never reach y = m
    if (((x + y) & 1) || y > x) return 0;
    ll up = (x + y) / 2, up2 = (x + 2 * m - y) / 2;
    return C(x, up) - ((x + 2 * m - y) % 2 || up2 < 0
    || up2 > x ? Mint(0) : C(x, up2));
}
```

5.13 LGVandPolya

LGV LEMMA and the POLYA CYCLE INDEX - the two theorems that were stated but never coded.

```
// Needs: Mint, determinant (06 - Math/04/02), phi
    (04 - NT/02/06), muSmall (09 -
    EulerTransform...).

// LINDSTROM-GESSEL-VIENNOT: with sources a_1..a_k
    and sinks b_1..b_k in a DAG,
//   det(M), M[i][j] = #paths a_i -> b_j
// counts NON-INTERSECTING path systems, PROVIDED the
    only permutation that admits a
// non-intersecting system is the identity (true for
    "sorted" sources/sinks in a grid).
// Grid instance: monotone lattice paths, so M[i][j]
    = C(dx + dy, dx).
Mint lgvGrid(const vector<pair<ll, ll>>& src, const
    vector<pair<ll, ll>>& dst) {
    int k = src.size();
    vector<vector<Mint>> M(k, vector<Mint>(k));
    for (int i = 0; i < k; i++)
        for (int j = 0; j < k; j++) {
            ll dx = dst[j].first - src[i].first, dy =
    dst[j].second - src[i].second;
            M[i][j] = (dx < 0 || dy < 0) ? Mint(0) :
    C(dx + dy, dx);
        }
    return determinant(M);
    // needs GaussianElimination.cpp
}
// USES: k non-crossing monotone paths,
    rhombus/domino tilings of a region, plane
    partitions,
// "k tokens walk on a grid and must never share a
    cell".

// POLYA CYCLE INDEX. Z(G)(a_1..a_n) = (1/|G|) sum_{g
    in G} prod_k a_k^(#k-cycles of g).
// The UNWEIGHTED orbit count is Z(G)(m, m, ..., m)
    for m colours - that is what necklaces()
// and bracelets() already give. The reason to carry
    the cycle index is the WEIGHTED case:
// "necklaces with exactly c_1 of colour 1, c_2 of
    colour 2, ..." which the numeric form cannot do.
//   Z(C_n) = (1/n) * sum_{d | n} phi(d) * a_d^(n/d)
//   Z(D_n) = (1/2) Z(C_n) + (1/2) a_1 a_2^((n-1)/2)
                          [n odd]
//   Z(D_n) = (1/2) Z(C_n) + (1/4) ( a_1^2
    a_2^((n-2)/2) + a_2^(n/2) )  [n even]
//   Z(S_n) = (1/n) * sum_{l=1..n} a_l * Z(S_(n-l)),
    Z(S_0) = 1
// Weighted PET: substitute a_k -> (t_1^k + t_2^k +
    ... + t_m^k) and read off the coefficient
// of t_1^c1 * ... * t_m^cm.
// Necklaces with a PRESCRIBED colour multiset, n
    beads, counts c[] summing to n (rotations only):
Mint necklacesWithCounts(int n, const vector<int>& c)
    {
    Mint total = 0;
    for (int d = 1; d <= n; d++) {
        if (n % d) continue;
        // rotations of order n/d have d cycles
        bool ok = true;
        for (int x : c) if (x % (n / d)) ok = false;
        // each colour must split evenly
        if (!ok) continue;
        Mint ways = fact[d];
```

```
    // multinomial over the d cycle slots
        for (int x : c) ways *= invFact[x / (n / d)];
        total += Mint(phi(n / d)) * ways;
    }
    return total * Mint(n).inv();
}
// APERIODIC necklaces (Lyndon words) with k colours:
  M(n,k) = (1/n) sum_{d|n} mu(d) k^(n/d).
Mint lyndonCount(ll n, ll k) {
    Mint s = 0;
    for (ll d = 1; d <= n; d++) if (n % d == 0) s +=
  Mint(muSmall(d)) * Mint(k).pow(n / d);
    return s * Mint(n).inv();
}

// TRANSFER MATRIX: count length-L walks in a finite
  automaton. Build A[s][t] = #transitions,
// then (A^L)[s][t]. The generating function sum_L
  (A^L)[i][j] x^L = ((I - xA)^-1)[i][j] is
// RATIONAL with denominator det(I - xA) of degree <=
  #states, so the sequence satisfies a linear
// recurrence of that order - feed a prefix to
  Berlekamp-Massey and finish in O(k^2 log L)
// instead of O(states^3 log L).
```

5.14 Hafnian

HAFNIAN - the number of PERFECT MATCHINGS of a GENERAL (non-bipartite) graph, weighted:

```
// Needs: Mint (06 - Math/01 - Modular Arithmetic/01
  - Mint.cpp).
//   haf(A) = sum over perfect matchings M of prod
  over {i,j} in M of A[i][j],
// for a symmetric A with a zero diagonal and n EVEN.
  O(2^(n/2) * n^2), n up to ~38 in a couple of
// seconds. This is the general-graph analogue of the
  permanent (which counts perfect matchings of
// a BIPARTITE graph, 10 -
  PermanentAndSymmetric.cpp); haf(A) is the
  "square root" of the permanent
// of the corresponding symmetric matrix.
// REACH FOR IT for "count the ways to pair up 2m
  items with forbidden or weighted pairs", domino
// tilings of an irregular board, "partition into
  pairs" counting, and any #matching problem where
// Blossom only gives you the MAXIMUM matching and
  you need the COUNT of perfect ones.
// COUNTING is much harder than finding: max matching
  is polynomial (Blossom), counting is #P-hard.
// For PLANAR graphs use the FKT algorithm (a
  Pfaffian orientation, then one determinant) -
  that is
// polynomial, and is how you count domino tilings of
  a grid.
// The recursion removes the last two rows/columns
  and tracks a truncated polynomial in a marker
// variable whose exponent counts how many pairs have
  been committed; only degree <= n/2 survives.
namespace Hafnian {
int H;
    // = n/2 + 1, the truncation length
void addTo(vector<Mint>& r, const vector<Mint>& a,
  const vector<Mint>& b) {
    for (int i = 0; i < H; i++) if (a[i].v)
        for (int j = 0; j + i + 1 < H; j++) r[i + j +
  1] += a[i] * b[j];
}
vector<Mint> rec(const vector<vector<vector<Mint>>>&
  v) {
    vector<Mint> r(H, 0);
    if (v.empty()) { r[0] = 1; return r; }
    int m = (int)v.size() - 2;
    auto V = v;
    V.resize(m);
    vector<Mint> zero = rec(V);
    // vertex pair (m, m+1) left unmatched here
    for (int i = 0; i < m; i++)
        for (int j = 0; j < i; j++) {
    // ... or routed through i and j
            addTo(V[i][j], v[m][i], v[m + 1][j]);
            addTo(V[i][j], v[m + 1][i], v[m][j]);
        }
    vector<Mint> one = rec(V);
    for (int i = 0; i < H; i++) r[i] += one[i] -
  zero[i];
    addTo(r, one, v[m + 1][m]);
    // ... or matched to each other
    return r;
}
Mint solve(const vector<vector<Mint>>& a) {
    // a symmetric, zero diagonal, n even
    int n = a.size();
    H = n / 2 + 1;
    vector<vector<vector<Mint>>> v(n);
    for (int i = 0; i < n; i++) {
        v[i].resize(i);
        for (int j = 0; j < i; j++) v[i][j].assign(H,
  0), v[i][j][0] = a[i][j];
    }
    return rec(v).back();
}
}
// SMALL n: a plain bitmask DP over "set of matched
  vertices", always matching the lowest unmatched
// vertex, is O(2^n * n) - simpler and faster up to n
  = 22. Use the Hafnian only past that.
// RELATED
//  PFAFFIAN: for a SKEW-symmetric matrix, pf(A)^2 =
  det(A) and pf is computable in O(n^3); the
//    Hafnian is its "no signs" version, which is
  exactly why one is easy and the other is not.
//  NUMBER OF MATCHINGS OF EVERY SIZE (not just
  perfect): that is the matching polynomial; the
  same
//    recursion with a longer truncation, or O(2^n)
  subset DP.
//  MODULUS: everything here is Mint arithmetic, so
  the count comes out mod MOD; for exact small
//    counts use unsigned long long and drop Mint.
```

5.15 SubsetSumCounts

NUMBER OF SUBSETS WITH EACH SUM, for every sum 0..m, in O(m log m) - not O(n * m).

```
// Needs: multiply / npow (02 - NTT.cpp), Poly / exp_
  / log_ / cut (05 - Convolution/04).
// Given a multiset of n positive values, the
  generating function is prod_i (1 + x^(a_i)),
  which
// naively costs n multiplications. Take logarithms:
  with cnt[v] = how many times v occurs,
//   log prod_v (1 + x^v)^cnt[v] = sum_v cnt[v] *
  sum_{j>=1} (-1)^(j+1) x^(v j) / j,
// and the double sum is a HARMONIC sum, O(m log m)
  terms in total. Then one exp brings it back.
// REACH FOR IT for Yosupo "#P subset sum", "for
```

```
    ↪ every k, how many subsets sum to k", partition
// counting with a bounded multiset of part sizes,
    ↪ and knapsack COUNTING when m is 1e5 and n is
// large. A plain O(n*m) knapsack (or the bitset one)
    ↪ is better when n is small or you only need
// feasibility rather than counts.
// PRECONDITION: all values >= 1 (handle a_i = 0
    ↪ separately - each zero doubles every count).
Poly subsetSumCounts(const vector<int>& a, int m) {
    ↪    // result[k] = #subsets summing to k
    vector<int> cnt(m + 1, 0);
    for (int v : a) if (v >= 1 && v <= m) cnt[v]++;
    Poly p(m + 1, 0);
    for (int v = 1; v <= m; v++) if (cnt[v])
        for (int j = 1; (ll)v * j <= m; j++) {
            ll t = (ll)cnt[v] % NMOD * npow(j, NMOD -
    ↪ 2) % NMOD;
            if (j & 1) p[v * j] = (p[v * j] + t) %
    ↪ NMOD;
            else p[v * j] = (p[v * j] - t + NMOD) %
    ↪ NMOD;
        }
    return exp_(p, m + 1);
}
// THE SAME MACHINE, DIFFERENT SIGN - this is the
    ↪ symbolic method (09 - EulerTransformAndSymbolic):
//   MULTISET (each value usable any number of times)
    ↪  = exp( sum_j  B(x^j) / j )
//   POWERSET (each value usable at most once)
    ↪  = exp( sum_j (-1)^(j+1) B(x^j) / j )
// with B(x) = sum_v cnt[v] x^v. Swapping the sign is
    ↪ the only difference between "partitions into
// parts from a set" and "subsets of a multiset".
// GOING BACKWARDS: given the counts, log_ recovers B
    ↪ and hence the multiset - that is the inverse
// Euler transform, the standard "recover the item
    ↪ sizes from the subset-sum counts" problem.
// ADD OR REMOVE ONE ITEM in O(m): multiply or divide
    ↪ the answer by (1 + x^v) directly (division by
// 1 + x^v is the recurrence r[k] -= r[k - v]) - much
    ↪ cheaper than rebuilding.
// K ITEMS ONLY: to also track the SIZE of the subset
    ↪ you need a second variable; do the same
// construction with x^(v) y and truncate in both, or
    ↪ run a 2-D convolution (05 - Convolution/08).
```

6 | Math

6.1 Modular Arithmetic

6.1.1 Mint

Modular integer (MOD must be PRIME for inv/division) + O(N) factorial tables.

```
struct Mint {
    int v;
    Mint(ll x = 0) { v = int(x % MOD); if (v < 0) v
    ↪ += MOD; }
    Mint& operator+=(Mint o) { if ((v += o.v) >= MOD)
    ↪ v -= MOD; return *this; }
    Mint& operator-=(Mint o) { if ((v -= o.v) < 0) v
    ↪ += MOD; return *this; }
    Mint& operator*=(Mint o) { v = int(1LL * v * o.v
    ↪ % MOD); return *this; }
    Mint& operator/=(Mint o) { return *this *=
    ↪ o.inv(); }
    friend Mint operator+(Mint a, Mint b) { return a
    ↪ += b; }
    friend Mint operator-(Mint a, Mint b) { return a
    ↪ -= b; }
    friend Mint operator*(Mint a, Mint b) { return a
    ↪ *= b; }
    friend Mint operator/(Mint a, Mint b) { return a
    ↪ /= b; }
    friend bool operator==(Mint a, Mint b) { return
    ↪ a.v == b.v; }
    Mint pow(ll e) const { Mint r = 1, b = *this; for
    ↪ (; e > 0; e >>= 1, b *= b) if (e & 1) r *= b;
    ↪ return r; }
    Mint inv() const { return pow(MOD - 2); }
    ↪   // Fermat: a^(p-2) = a^-1 (mod p)
    friend ostream& operator<<(ostream& o, Mint m) {
    ↪ return o << m.v; }
    friend istream& operator>>(istream& i, Mint& m) {
    ↪ ll x; i >> x; m = Mint(x); return i; }
};

const int N = 200005;
    ↪   // ALSO in 01 - Template.cpp: keep one
Mint fact[N], invFact[N], inv_[N];
void buildFact() {
    ↪   // O(N) total, not O(N log MOD)
    fact[0] = invFact[0] = inv_[1] = 1;
    for (int i = 1; i < N; i++) fact[i] = fact[i - 1]
    ↪ * i;
    invFact[N - 1] = fact[N - 1].inv();
    for (int i = N - 1; i > 0; i--) invFact[i - 1] =
    ↪ invFact[i] * i;
    for (int i = 2; i < N; i++) inv_[i] = Mint(MOD -
    ↪ MOD / i) * inv_[MOD % i];
}
Mint C(ll n, ll r) { if (r < 0 || r > n) return 0;
    ↪ return fact[n] * invFact[r] * invFact[n - r]; }
Mint Perm(ll n, ll r) { if (r < 0 || r > n) return 0;
    ↪ return fact[n] * invFact[n - r]; }
// C(n, r) for HUGE n and small r (no tables):
    ↪ prod_{i<r}(n-i) / r!
Mint Cbig(ll n, ll r) {
    if (r < 0 || n < 0 || r > n) return 0;
    Mint num = 1;
    for (ll i = 0; i < r; i++) num *= Mint(n - i);
    return num * invFact[r];
}
```

6.1.2 BinaryFieldGF2k

ARITHMETIC IN THE FINITE FIELD GF(2^K): elements are K-bit masks = polynomials over GF(2)

```
// reduced mod a fixed IRREDUCIBLE polynomial of
    ↪ degree K. Add = xor, multiply = carry-less
// multiply then reduce. mul is O(K), pow / inverse
    ↪ O(K log) - or O(1) with the CLMUL intrinsic.
// REACH FOR IT when a problem is "linear algebra
    ↪ over a field of characteristic 2 with more than
// two elements": random algebraic identity testing
    ↪ over GF(2^K) (Freivalds / Schwartz-Zippel with
// xor-shaped constraints), Reed-Solomon-flavoured
    ↪ coding, "assign field values to symbols and test
// whether a determinant vanishes", and Wiedemann
    ↪ over GF(2).
// PRECONDITION: POLY below must be irreducible over
    ↪ GF(2) and of degree exactly K.
//   K=8: 0x11B   K=16: 0x1002B   K=30: 0x40000003
    ↪ K=31: 0x80000009   K=32: 0x1000000AF
```

```
//   K=64: x^64 + x^4 + x^3 + x + 1 (constant 0x1B),
  ↪ the usual choice for unsigned long long.
namespace GF2K {
const int K = 30;
const ll POLY = (1LL << K) | 3;                 //
  ↪ x^30 + x + 1, irreducible

ll add(ll a, ll b) { return a ^ b; }            //
  ↪ subtraction is the same operation
ll mul(ll a, ll b) {
    ll r = 0;
    while (b) {
        if (b & 1) r ^= a;
        b >>= 1, a <<= 1;
        if (a >> K & 1) a ^= POLY;              //
  ↪ reduce as soon as the degree reaches K
    }
    return r;
}
ll pw(ll a, ll e) {
    ll r = 1;
    for (; e; e >>= 1, a = mul(a, a)) if (e & 1) r =
  ↪ mul(r, a);
    return r;
}
ll inv(ll a) { return pw(a, (1LL << K) - 2); }  //
  ↪ a^(2^K - 1) = 1, so a^-1 = a^(2^K - 2)
}
// FACTS YOU NEED WHEN USING IT
//  |GF(2^K)| = 2^K, the multiplicative group is
  ↪ CYCLIC of order 2^K - 1, and every element
//  satisfies a^(2^K) = a. Squaring is a field
  ↪ automorphism (FROBENIUS): (a + b)^2 = a^2 + b^2,
//  so squaring is LINEAR over GF(2) and can be
  ↪ tabulated as a matrix.
//  GF(2^K) contains GF(2^d) exactly when d | K.
//  x^(2^K) - x factors as the product of all
  ↪ irreducible polynomials whose degree divides K.
//  A random degree-K polynomial is irreducible with
  ↪ probability about 1/K; test with
//  gcd(x^(2^d) - x, f) == 1 for all d < K dividing K.
// NOT THE SAME AS NIMBERS. Nim-multiplication (10 -
  ↪ Game Theory/05 - NimbersAndSums.cpp) is a
// DIFFERENT field structure on the same set: it
  ↪ needs no irreducible polynomial, it is defined by
// the mex rules, and 2^(2^k) values form subfields.
  ↪ Use nimbers for game values, this for coding
// and linear-algebra style problems - they are
  ↪ isomorphic but NOT equal element by element.
// SPEED: on x86 with -mpclmul, _mm_clmulepi64_si128
  ↪ does the carry-less product in one
// instruction; you then only need the Barrett-style
  ↪ reduction. Worth it only in a hot loop.
```

6.2 Multiplicative Functions

6.2.1 EulerTotientPhi

EULER TOTIENT SIEVE - phiArr[i] = #{1<=k<=i : gcd(k,i)=1} for all i < N in O(N log log N).

```
// Needed for: a^b mod m with gcd(a,m)=1 (Euler's
  ↪ theorem), counting coprime pairs, Mobius
  ↪ identities
// (sum_{d|n} phi(d) = n), and the order of the
  ↪ multiplicative group mod n.
// NAME: phiArr, not phi - a global 'phi' collides
  ↪ with the phi() FUNCTION in
// 04 - Number Theory/02 - Primes and Divisors/06 -
  ↪ MillerRabinPollard.cpp.
int phiArr[N];

void calculatePhi() {
    for (int i = 0; i < N; ++i) phiArr[i] = i & 1 ? i
  ↪ : i / 2;
    for (int i = 3; i < N; i += 2)
        if (phiArr[i] == i)
            for (int j = i; j < N; j += i) phiArr[j]
  ↪ -= phiArr[j] / i;
}
```

6.2.2 MobiusFunction

Mobius Function mu(n) Sieve - Time: O(N), Space: O(N)

```
// mu(n) = 1 if n is square-free with an even number
  ↪ of prime factors
// mu(n) = -1 if n is square-free with an odd number
  ↪ of prime factors
// mu(n) = 0 if n has a squared prime factor
struct Mobius {
    int n;
    vector<int> mu, primes;
    vector<bool> is_prime;

    Mobius(int n = 1e7) : n(n), mu(n + 1, 0),
  ↪ is_prime(n + 1, true) {
        is_prime[0] = is_prime[1] = false;
        mu[1] = 1;

        for (int i = 2; i <= n; i++) {
            if (is_prime[i]) {
                primes.pb(i);
                mu[i] = -1;
            }
            for (int p : primes) {
                if ((ll)i * p > n) break;
                is_prime[i * p] = false;
                if (i % p == 0) {
                    mu[i * p] = 0;
                    break;
                } else {
                    mu[i * p] = -mu[i];
                }
            }
        }
    }
};
```

6.3 Matrices

6.3.1 MatrixExponentiation

Matrix over Mint. mul O(n^3) (cache-friendly i-k-j order), pow O(n^3 log e).

```
// Needs: Mint (01 - Modular Arithmetic/01 -
  ↪ Mint.cpp).
// Fix the size at compile time
  ↪ (array<array<Mint,K>,K>) if you need the last 2x
  ↪ speedup.
typedef vector<vector<Mint>> Mat;

Mat operator*(const Mat& a, const Mat& b) {
    int n = a.size(), k = b.size(), m = b[0].size();
    Mat c(n, vector<Mint>(m));
    for (int i = 0; i < n; i++)
        for (int t = 0; t < k; t++) if (a[i][t].v)
            for (int j = 0; j < m; j++) c[i][j] +=
  ↪ a[i][t] * b[t][j];
    return c;
}
```

```
Mat eye(int n) { Mat r(n, vector<Mint>(n)); for (int
  ↪ i = 0; i < n; i++) r[i][i] = 1; return r; }
Mat mpow(Mat a, ll e) {
    Mat r = eye(a.size());
    for (; e > 0; e >>= 1, a = a * a) if (e & 1) r =
  ↪ r * a;
    return r;
}
// RECIPES
//  Linear recurrence f(n) = c1*f(n-1) + ... +
  ↪ ck*f(n-k):
//    companion matrix, first row = (c1..ck),
  ↪ subdiagonal = 1.  f(n) = (M^(n-k) *
  ↪ [f(k-1)..f(0)]^T)[0]
//  Add a "+d" constant term: enlarge by one row/col
  ↪ with a 1 on the diagonal, put d in the first row.
//  Count walks of length L between u and v:
  ↪ (A^L)[u][v] where A is the adjacency matrix.
//  Shortest walk with exactly L edges: same, but
  ↪ replace (+,*) with (min,+) - "min-plus product".
```

6.3.2 CharacteristicPolynomial

CHARACTERISTIC POLYNOMIAL p(x) = det(x*I - A) of an n x n matrix in O(n^3). MOD must be PRIME.

```
// Needs: Mint (01 - Modular Arithmetic/01 -
  ↪ Mint.cpp), Mat (03 - Matrices/01).
// REACH FOR IT when you need det(A - t*I) for MANY t
  ↪ (n = 500, q = 1e5 queries: build p once,
// then each query is one Horner evaluation), or A^k
  ↪ for a huge k: by Cayley-Hamilton p(A) = 0, so
// A^k = (x^k mod p(x))(A), which costs O(n^2 log k)
  ↪ polynomial work plus O(n^3) to expand - much
// better than O(n^3 log k) when n is large. Also:
  ↪ det(A) = (-1)^n * p[0], trace(A) = -p[n-1].
// Gaussian elimination CANNOT do this directly
  ↪ because the entries are polynomials in x.

// STEP 1: reduce to upper HESSENBERG form (a[i][j] =
  ↪ 0 for i > j + 1) by SIMILARITY transforms.
// The row op "row i -= f * row (c+1)" is undone by
  ↪ "col (c+1) += f * col i", so A -> E A E^-1 and
// the whole spectrum (hence the characteristic
  ↪ polynomial) is unchanged. O(n^3).
Mat hessenberg(Mat a) {
    int n = a.size();
    for (int c = 0; c + 2 < n; c++) {
        int p = -1;
        for (int i = c + 1; i < n; i++) if
  ↪ (a[i][c].v) { p = i; break; }
        if (p < 0) continue;                        //
  ↪ column already clear below the subdiagonal
        if (p != c + 1) {                           //
  ↪ swapping rows AND cols keeps it a similarity;
            swap(a[c + 1], a[p]);                   //
  ↪ column c is untouched because c < c+1 <= p
            for (int i = 0; i < n; i++) swap(a[i][c +
  ↪ 1], a[i][p]);
        }
        for (int i = c + 2; i < n; i++) if
  ↪ (a[i][c].v) {
            Mint f = a[i][c] / a[c + 1][c];
            for (int j = 0; j < n; j++) a[i][j] -=
  ↪ a[c + 1][j] * f;
            for (int j = 0; j < n; j++) a[j][c + 1]
  ↪ += a[j][i] * f;   // uses the UPDATED col i
        }
    }
    return a;
}
// STEP 2: expand det(x*I - H) along the last row.
  ↪ With f[m] = char poly of the leading m x m block,
//   f[m] = (x - h[m-1][m-1]) f[m-1] - sum_{i=1..m-1}
  ↪ h[m-1-i][m-1] * (prod_{j=m-i..m-1} h[j][j-1])
//
  ↪                * f[m-1-i]
// Only Hessenberg matrices satisfy such a short
  ↪ recurrence; that is the whole point of step 1.
vector<Mint> charPoly(const Mat& a) {            //
  ↪ returns p[0..n], p[i] = coeff of x^i, p[n] = 1
    int n = a.size();
    Mat h = hessenberg(a);
    vector<vector<Mint>> f(n + 1);
    f[0] = {1};
    for (int m = 1; m <= n; m++) {
        f[m].assign(m + 1, 0);
        for (int i = 0; i < m; i++) {
            f[m][i + 1] += f[m - 1][i];
            f[m][i] -= h[m - 1][m - 1] * f[m - 1][i];
        }
        Mint prod = 1;
        for (int i = 1; i < m; i++) {
            prod *= h[m - i][m - i - 1];
            Mint c = prod * h[m - 1 - i][m - 1];
            if (!c.v) continue;
            for (int j = 0; j <= m - 1 - i; j++)
  ↪ f[m][j] -= c * f[m - 1 - i][j];
        }
    }
    return f[n];
}
// A^k VIA CAYLEY-HAMILTON: r(x) = x^k mod p(x)
  ↪ (O(n^2 log k) with schoolbook, O(n log n log k)
// with NTT), then A^k = sum r[i] A^i. If you only
  ↪ need u^T A^k v, that is sum r[i] * (u^T A^i v)
// and the A^i v are n matrix-vector products - O(n *
  ↪ nnz * n) for a sparse A, no n^3 anywhere.
// EIGENVALUES over F_p are the roots of p; over R
  ↪ use the QR algorithm (not a contest routine).
// det(A + t*B) as a polynomial in t: reduce (A, B)
  ↪ to Hessenberg/triangular pencil form, or just
// evaluate at n+1 values of t with plain Gaussian
  ↪ determinants and interpolate - O(n^4), fine to
// n = 100 and much easier to get right.
// SIMILAR-LOOKING BUT DIFFERENT: the MINIMAL
  ↪ polynomial divides p; get it from the Krylov
// sequence u^T A^k v with Berlekamp-Massey (see 03 -
  ↪ SparseDeterminant.cpp).
```

6.3.3 SparseDeterminant

rnd (01 - Miscellaneous/04 - random.cpp).

```
// Needs: Mint (01 - Modular Arithmetic/01 -
  ↪ Mint.cpp), berlekampMassey (06 -
  ↪ Interpolation/02),
//        rnd (01 - Miscellaneous/04 - random.cpp).
// WIEDEMANN: determinant and minimal polynomial of a
  ↪ SPARSE n x n matrix in O(n * nnz + n^2),
// using O(n) memory. MOD must be PRIME. Gaussian
  ↪ elimination is O(n^3) and fills in; this is the
// routine for n = 5e4 with 1e5 non-zeros (Yosupo
  ↪ "sparse matrix det"), for adjacency-like
// matrices, and for the Matrix-Tree determinant of a
  ↪ sparse graph.
// IDEA: for random vectors u, v the scalar sequence
  ↪ s_k = u^T A^k v obeys the same linear
// recurrence as A itself (Cayley-Hamilton), so
  ↪ Berlekamp-Massey on 2n+2 samples recovers the
```

```
// minimal polynomial. Each sample costs one sparse
   ↪ mat-vec, hence n * nnz total.
// PRECONDITIONER: right-multiplying by a random
   ↪ diagonal D makes minpoly(AD) = charpoly(AD) with
// probability >= 1 - O(n^2 / MOD), which is what
   ↪ lets us read the determinant off the last
// coefficient. det(AD) = det(A) * prod d_i, so
   ↪ divide the d_i back out.
struct Elem { int r, c; Mint v; };                //
   ↪ A[r][c] = v; all (r, c) distinct, rest zero

vector<Mint> sparseMinPoly(int n, const vector<Elem>&
   ↪ a) {
    vector<Mint> u(n), x(n), s;
    for (int i = 0; i < n; i++) u[i] = rnd(1, MOD -
   ↪ 1), x[i] = rnd(1, MOD - 1);
    for (int k = 0; k < 2 * n + 2; k++) {
        Mint t = 0;
        for (int i = 0; i < n; i++) t += u[i] * x[i];
        s.push_back(t);
        vector<Mint> y(n, 0);                     // y
   ↪ = A * x, one pass over the non-zeros
        for (const Elem& e : a) y[e.r] += e.v *
   ↪ x[e.c];
        x = y;
    }
    return berlekampMassey(s);                    //
   ↪ s[i] = sum_j C[j] * s[i-1-j]
}
Mint sparseDet(int n, vector<Elem> a) {
    vector<Mint> d(n);
    for (int i = 0; i < n; i++) d[i] = rnd(1, MOD -
   ↪ 1);
    for (Elem& e : a) e.v *= d[e.c];              // A
   ↪ -> A * diag(d)
    vector<Mint> c = sparseMinPoly(n, a);
    if ((int)c.size() < n) return 0;              //
   ↪ minpoly shorter than n => singular (whp)
    Mint r = c.back();                            //
   ↪ charpoly = x^n - c0 x^(n-1) - ... - c(n-1)
    if (n % 2 == 0) r = Mint(0) - r;              //
   ↪ det = (-1)^(n+1) * c[n-1]
    for (int i = 0; i < n; i++) r /= d[i];
    return r;
}
// RANK of a sparse matrix: same trick on a random
   ↪ projection; simpler in practice is to run
// blocked elimination only if the fill-in stays
   ↪ small.
// SOLVING A x = b sparsely: with the minimal
   ↪ polynomial m(x) = sum m_i x^i and m_0 != 0,
//   x = -(1/m_0) * sum_{i>=1} m_i A^(i-1) b    --
   ↪ again only sparse mat-vecs.
// FAILURE MODE: if the returned recurrence is
   ↪ shorter than n, either the matrix is singular or
// the random draw was unlucky; rerun once before
   ↪ believing the 0.
// INTEGER determinant with no modulus: run this
   ↪ under several primes and CRT, sizing with
// Hadamard's bound |det| <= prod of the row 2-norms.
```

6.3.4 CirculantDeterminant

DETERMINANT OF A CIRCULANT in O(n log^2 n) instead of O(n^3).

```
// Needs: resultant (05 - Convolution/11 -
   ↪ PolyGcdResultant.cpp), which needs NTT +
   ↪ Polynomial.
// C[i][j] = c[(j - i) mod n]  (every row is the
   ↪ previous one rotated right by 1). Then
//   det C = prod_{k=0}^{n-1} f(w^k) = Res(x^n - 1,
   ↪ f),   f(x) = c0 + c1 x + ... + c(n-1) x^(n-1),
// where w is a primitive n-th root of unity.
   ↪ Eigenvalue f(w^k) has eigenvector (1, w^k, w^2k,
   ↪ ...)
// - the Fourier modes diagonalise EVERY circulant
   ↪ simultaneously, which is why circulants commute
// and why multiplying two of them is just a cyclic
   ↪ convolution of their first rows.
// The resultant form is what you use when n does NOT
   ↪ divide MOD - 1, so the roots of unity do not
// exist in the field. Everything below is mod NMOD =
   ↪ 998244353 (the Poly modulus).
ll circulantDet(const vector<ll>& c) {
    int n = c.size();
    Poly xn(n + 1, 0);
    xn[0] = NMOD - 1, xn[n] = 1;                  //
   ↪ x^n - 1
    return resultant(xn, c);
}
// If n | (NMOD - 1) you do not need the resultant at
   ↪ all: evaluate f at the n-th roots of unity
// with one length-n NTT and multiply the results,
   ↪ O(n log n).
// PERMUTANT MATRIX (a circulant along a
   ↪ permutation): M[i][j] = a[k] where sigma^k(i) =
   ↪ j,
// for a permutation sigma that is a SINGLE n-cycle.
   ↪ Relabel the indices along the cycle
// (q[0] = 0, q[t] = sigma^t(0)) and M becomes an
   ↪ honest circulant; the determinant picks up the
// SIGN of that relabelling permutation. If sigma has
   ↪ more than one cycle the matrix is not a
// permutant and the trick does not apply.
// BLOCK-CIRCULANT (blocks of size m, n blocks): det
   ↪ = prod_k det(F(w^k)) with F the matrix-valued
// symbol - same argument, m x m determinants instead
   ↪ of scalars.
// OTHER DETERMINANTS YOU CAN DO WITHOUT ELIMINATION
//  MATRIX DETERMINANT LEMMA: det(A + u v^T) = det(A)
   ↪ * (1 + v^T A^-1 u). Rank-k update:
//    det(A + U V^T) = det(A) * det(I_k + V^T A^-1
   ↪ U). So det(x*I + a b^T) = x^(n-1) * (x + a.b),
//    which closes "M[i][j] = a_i b_j off the
   ↪ diagonal, x + a_i b_i on it" in O(n).
//  CAUCHY-BINET: for A (n x m) and B (m x n),
   ↪ det(AB) = sum over n-subsets S of [m] of
//    det(A[:,S]) * det(B[S,:]). This is the LGV
   ↪ lemma's algebraic twin.
//  VANDERMONDE: det = prod_{i<j} (x_j - x_i).
   ↪ CAUCHY: det(1/(x_i+y_j)) =
//    prod_{i<j}(x_i-x_j)(y_i-y_j) /
   ↪ prod_{i,j}(x_i+y_j).
//  TRIDIAGONAL: d_k = b_k d_{k-1} - a_k c_{k-1}
   ↪ d_{k-2}, O(n) (this is the Hessenberg
   ↪ recurrence).
```

6.3.5 PolyMatrixPower

MATRIX EXPONENTIATION WHERE THE ENTRIES ARE POWER SERIES, truncated mod x^K.

```
// Needs: multiply (05 - Convolution/02 - NTT.cpp),
  ↪ Poly / cut (05 - Convolution/04).
// M^e in O(n^3 * K log K * log e).  Everything is
  ↪ mod NMOD = 998244353.
// REACH FOR IT for a linear recurrence in n that
  ↪ also tracks a SECOND statistic you want the full
// distribution of: put x^(contribution) in the
  ↪ transition instead of 1, and the (i, j) entry of
// M^e becomes the generating function over that
  ↪ statistic. Classic: "count length-n walks by
// number of times a marked edge is used", "f(n) for
  ↪ every k = number of ..." (CF 755G), any DP
// whose answer is asked for ALL values of a
  ↪ parameter at once rather than one value.
// A plain Mat over Mint gives you one number; this
  ↪ gives you K numbers for a log K factor.
typedef vector<vector<Poly>> PMat;

PMat pmul(const PMat& a, const PMat& b, int K) {
    int n = a.size(), m = b[0].size(), k = b.size();
    PMat c(n, vector<Poly>(m, Poly(1, 0)));
    for (int i = 0; i < n; i++)
        for (int t = 0; t < k; t++) {
            if (a[i][t].size() == 1 && !a[i][t][0])
  ↪ continue;      // skip the zero series
            for (int j = 0; j < m; j++) c[i][j] =
  ↪ c[i][j] + cut(multiply(a[i][t], b[t][j]), K);
        }
    return c;
}
PMat peye(int n) {
    PMat r(n, vector<Poly>(n, Poly(1, 0)));
    for (int i = 0; i < n; i++) r[i][i] = Poly(1, 1);
    return r;
}
PMat ppow(PMat a, ll e, int K) {                  //
  ↪ a^e mod x^K
    PMat r = peye(a.size());
    for (; e > 0; e >>= 1, a = pmul(a, a, K)) if (e &
  ↪ 1) r = pmul(r, a, K);
    return r;
}
// EXAMPLE (Fibonacci refined by the number of "1"
  ↪ steps, CF 755G shape):
//   M = {{1 + x, x}, {1, 0}} and (M^(n-1) * v)[0] is
  ↪ the generating function you want.
// CHEAPER ALTERNATIVES, try these first:
//  - if K is tiny (<= 3), just hand-roll the
  ↪ truncated series as k separate Mint matrices.
//  - if the statistic is "how many times did we take
  ↪ transition t", one variable suffices; if it
//    is a pair of statistics you need a bivariate
  ↪ series - flatten it to x^(i*W + j).
//  - if n (the matrix size) is 1, this is just
  ↪ power-series exponentiation: use pow_ directly.
//  - if you want ONE coefficient rather than all K,
  ↪ Bostan-Mori on the rational function is
//    O(d log d log e) and much faster (06 -
  ↪ Interpolation/03 - BostanMori.cpp).
// COST WARNING: n^3 poly multiplications per
  ↪ squaring. n = 8, K = 1e5, e = 1e18 is already at
  ↪ the
// edge; keep the matrix small, and truncate (cut to
  ↪ K) after EVERY product, never at the end.
```

6.3.6 Freivalds

FREIVALDS: verify A * B == C in O(t * n^2) instead of computing the product in O(n^3).

```
// Needs: Mint (01 - Modular Arithmetic/01 -
  ↪ Mint.cpp), rnd (01 - Miscellaneous/04 -
  ↪ random.cpp).
// Error probability <= 2^-t (one-sided: it never
  ↪ rejects a correct product). t = 30..40 is plenty.
// REACH FOR IT when the problem is "is this claimed
  ↪ product right", when A or B changes between
// queries so recomputing is too slow, or as a
  ↪ subroutine inside a bigger randomised check.
bool freivalds(const vector<vector<Mint>>& a, const
  ↪ vector<vector<Mint>>& b,
               const vector<vector<Mint>>& c, int t =
  ↪ 40) {
    int n = a.size();
    vector<Mint> r(n), x(n), y(n);
    while (t--) {
        for (int i = 0; i < n; i++) r[i] = rnd(0, 1);
  ↪            // random 0/1 vector is enough
        for (int i = 0; i < n; i++) {
  ↪            // x = B r,  y = C r
            x[i] = y[i] = 0;
            for (int j = 0; j < n; j++) x[i] +=
  ↪ b[i][j] * r[j], y[i] += c[i][j] * r[j];
        }
        for (int i = 0; i < n; i++) {
  ↪            // check A x == y
            Mint s = 0;
            for (int j = 0; j < n; j++) s += a[i][j]
  ↪ * x[j];
            if (!(s == y[i])) return false;
        }
    }
    return true;
}
// WHY IT WORKS: D = AB - C is non-zero, so some row
  ↪ of D is non-zero; for a uniform 0/1 vector r,
// Pr[D r == 0] <= 1/2 (fix all coordinates but one -
  ↪ at most one of the two choices can cancel).
// Independent rounds multiply, hence 2^-t. Do the
  ↪ products RIGHT to LEFT: A(Br), never (AB)r.

// THE GENERAL PATTERN - RANDOMISED ALGEBRAIC
  ↪ IDENTITY TESTING
// SCHWARTZ-ZIPPEL: a non-zero polynomial of total
  ↪ degree d over a field, evaluated at a uniform
// point of S^n, is zero with probability <= d / |S|.
  ↪ Every check below is one application.
//  - "are these two expressions equal?" - evaluate
  ↪ both at a random point mod a large prime.
//  - PERFECT MATCHING in a general graph: build the
  ↪ TUTTE matrix T with T[i][j] = +x_ij and
//    T[j][i] = -x_ij for each edge i < j, substitute
  ↪ random values, and G has a perfect matching
//    iff det T != 0 (whp). O(n^3) existence check
  ↪ with no Blossom. rank(T) = 2 * (max matching).
//  - BIPARTITE matching: same with the Edmonds
  ↪ matrix (random value on each edge, 0 elsewhere).
//  - "do these two multisets/strings/trees agree?" -
  ↪ hash them as polynomial evaluations.
//  - CHECKING A PRODUCT OF SPARSE OBJECTS: a
  ↪ random-vector probe generalises far past matrices
//    (it is exactly the idea behind Wiedemann in 03
  ↪ - SparseDeterminant.cpp).
// Prefer a random FIELD element over 0/1 when the
  ↪ identity has degree > 1; the failure bound is
```

```
// then d / MOD rather than 1/2 per round.
```

6.3.7 MinPlusMatrixPower

TROPICAL (MIN-PLUS) MATRIX POWER: replace (+, *) by (min, +) and matrix exponentiation answers

```
// "cheapest walk using EXACTLY k edges" in O(n^3 log k). The min-plus semiring has additive
// identity llinf and multiplicative identity 0, and it is associative - which is all matrix
// exponentiation ever needed. Everything below works verbatim for MAX-PLUS by flipping min to max
// and llinf to -llinf.
// REACH FOR IT when the edge count is a hard constraint and k is large: "shortest path with
// exactly k edges", "cheapest k-step schedule", periodic/cyclic transport problems, and DPs whose
// transition is the same min-plus matrix every layer (n <= 100-200, k up to 1e18).
// If k is small, just do k layers of DP - O(k * n^2) beats O(n^3 log k) whenever k < n log k.
typedef vector<vector<ll>> TMat;

TMat tmul(const TMat& a, const TMat& b) {
    int n = a.size(), m = b[0].size(), p = b.size();
    TMat c(n, vector<ll>(m, llinf));
    for (int i = 0; i < n; i++)
        for (int t = 0; t < p; t++) {
            if (a[i][t] >= llinf) continue;
                // skip, and avoid overflow
            for (int j = 0; j < m; j++)
                if (b[t][j] < llinf) c[i][j] = min(c[i][j], a[i][t] + b[t][j]);
        }
    return c;
}
TMat teye(int n) {
                // min-plus identity
    TMat r(n, vector<ll>(n, llinf));
    for (int i = 0; i < n; i++) r[i][i] = 0;
    return r;
}
TMat tpow(TMat a, ll k) {
                // exactly k edges
    TMat r = teye(a.size());
    for (; k > 0; k >>= 1, a = tmul(a, a)) if (k & 1) r = tmul(r, a);
    return r;
}
// SET-UP: a[i][j] = weight of edge i->j, llinf when there is no edge. Then tpow(a, k)[u][v] is the
// minimum total weight over walks u->v with exactly k edges (llinf if none exists).
// AT MOST k EDGES: add a zero-weight self-loop at every vertex (a[i][i] = min(a[i][i], 0)).
// COUNTING instead of minimising: the same code over (+, *) is 03 - Matrices/01, and over
// (or, and) with bitsets it answers reachability in exactly k steps in O(n^3 / 64).
// LONGEST path with exactly k edges: max-plus, but ONLY if the graph is a DAG or k is bounded;
// otherwise a positive cycle makes it unbounded.
// OVERFLOW is the standard bug: guard both operands against llinf before adding, as above, or
// llinf + llinf silently wraps negative and poisons the whole matrix.
// FASTER SPECIAL CASES
//  - if the matrix is CONVEX in the Monge sense, min-plus products drop to O(n^2) via SMAWK, so
//    the whole power costs O(n^2 log k) (see 08 - DP/01/11 - MinPlusConvolution.cpp).
//  - if you only need one row (a fixed source), keep a VECTOR and multiply it by the precomputed
//    powers a^(2^i): O(n^2 log k) total instead of n^3 log k.
//  - for eventually-periodic behaviour, the tropical eigenvalue (the minimum mean cycle, Karp's
//    algorithm) gives the per-step growth rate and lets you skip the huge exponent entirely.
```

6.4 Linear Algebra

6.4.1 LinearXorBasis

XOR basis (vector space over GF(2)). insert / query O(B).

```
// span size = 2^sz. Use MERGE to keep a basis in each segment-tree node -> max-xor over a RANGE.
struct Basis {
    static const int B = 62;      // 60 silently drops bits 61-62 of an ll up to 9e18
    ll b[B + 1] = {};
    // b[i] has leading bit i
    int sz = 0;

    bool insert(ll x) {
    // false if x already in the span
        for (int i = B; i >= 0; i--) if (x >> i & 1) {
            if (!b[i]) { b[i] = x, sz++; return true; }
            x ^= b[i];
        }
        return false;
    }
    bool contains(ll x) const {
        for (int i = B; i >= 0; i--) if (x >> i & 1) { if (!b[i]) return false; x ^= b[i]; }
        return true;
    }
    ll maxXor(ll r = 0) const { for (int i = B; i >= 0; i--) r = max(r, r ^ b[i]); return r; }
    ll minXor(ll r = 0) const { for (int i = B; i >= 0; i--) if (r >> i & 1) r ^= b[i]; return r; }
    void reduce() {
    // reduced row echelon form
        for (int i = B; i >= 0; i--) if (b[i])
            for (int j = i - 1; j >= 0; j--) if (b[i] >> j & 1) b[i] ^= b[j];
    }
    ll kth(ll k) {
    // k-th SMALLEST value in the span, 0-indexed
        reduce();
    // kth(0) == 0; -1 if k >= 2^sz
        ll r = 0;
        for (int i = 0; i <= B; i++) if (b[i]) { if (k & 1) r ^= b[i]; k >>= 1; }
        return k ? -1 : r;
    }
    ll spanSize() const { return sz >= 63 ? -1 : 1LL << sz; }
    friend Basis merge(Basis a, const Basis& c) {
        for (int i = 0; i <= B; i++) if (c.b[i]) a.insert(c.b[i]);
        return a;
    }
};
```

6.4.2 GaussianElimination

Gaussian elimination mod a PRIME. Returns rank; solves A x = b; also det and inverse.

```
// Needs: Mint (01 - Modular Arithmetic/01 -
  ↪ Mint.cpp) for the modular routines.
// O(n^2 m). Pass the augmented matrix (n rows, m+1
  ↪ cols) to solve().
int gaussMod(vector<vector<Mint>>& a, int m) {
  ↪    // reduce first m columns to RREF
    int n = a.size(), rank = 0;
    for (int col = 0; col < m && rank < n; col++) {
        int piv = -1;
        for (int i = rank; i < n; i++) if
  ↪ (a[i][col].v) { piv = i; break; }
        if (piv < 0) continue;
        swap(a[rank], a[piv]);
        Mint iv = a[rank][col].inv();
        for (auto& x : a[rank]) x *= iv;
        for (int i = 0; i < n; i++) if (i != rank &&
  ↪ a[i][col].v) {
            Mint f = a[i][col];
            for (size_t j = 0; j < a[i].size(); j++)
  ↪ a[i][j] -= f * a[rank][j];
        }
        rank++;
    }
    return rank;
}
// Solve A x = b. Returns {} if inconsistent, else
  ↪ one solution (free vars = 0).
// ‘free‘ receives the indices of the free variables
  ↪ (solution space has MOD^|free| points).
vector<Mint> solveLinear(vector<vector<Mint>> a,
  ↪ vector<Mint> b, vector<int>* freeIdx = nullptr) {
    int n = a.size(), m = a[0].size();
    for (int i = 0; i < n; i++) a[i].push_back(b[i]);
    int rank = gaussMod(a, m);
    for (int i = rank; i < n; i++) if (a[i][m].v)
  ↪ return {};      // 0 = nonzero
    vector<Mint> x(m, 0);
    int r = 0;
    for (int c = 0; c < m; c++) {
  ↪ // must run over EVERY column: stopping at
        if (r < rank && a[r][c].v) x[c] = a[r][m],
  ↪ r++;   // r == rank drops the free columns
        else if (freeIdx) freeIdx->push_back(c);
  ↪     // that come after the last pivot
    }
    return x;
}
Mint determinant(vector<vector<Mint>> a) {
  ↪    // O(n^3), destroys a
    int n = a.size();
    Mint det = 1;
    for (int col = 0; col < n; col++) {
        int piv = -1;
        for (int i = col; i < n; i++) if
  ↪ (a[i][col].v) { piv = i; break; }
        if (piv < 0) return 0;
        if (piv != col) swap(a[col], a[piv]), det =
  ↪ Mint(0) - det;
        det *= a[col][col];
        Mint iv = a[col][col].inv();
        for (int i = col + 1; i < n; i++) if
  ↪ (a[i][col].v) {
            Mint f = a[i][col] * iv;
            for (int j = col; j < n; j++) a[i][j] -=
  ↪ f * a[col][j];
        }
    }
    return det;
}
vector<vector<Mint>> inverse(vector<vector<Mint>> a)
  ↪ {   // {} if singular
    int n = a.size();
    for (int i = 0; i < n; i++) { a[i].resize(2 * n,
  ↪ 0); a[i][n + i] = 1; }
    if (gaussMod(a, n) < n) return {};
    vector<vector<Mint>> r(n, vector<Mint>(n));
    for (int i = 0; i < n; i++) for (int j = 0; j <
  ↪ n; j++) r[i][j] = a[i][n + j];
    return r;
}
// MATRIX-TREE: #spanning trees = determinant of the
  ↪ Laplacian (deg on diagonal, -1 per edge)
// with any one row AND the matching column deleted.
```

6.4.3 GaussXorAndReal

XOR-equation systems over GF(2) with bitset - O(n*m/64). n equations, m variables,

```
// row[i][m] holds the RHS. Returns rank, or -1 if
  ↪ inconsistent.
template <int M>
int gaussXor(vector<bitset<M + 1>>& a, int m,
  ↪ bitset<M>* sol = nullptr) {
    int n = a.size(), rank = 0;
    vector<int> where(m, -1);
    for (int col = 0; col < m && rank < n; col++) {
        int piv = -1;
        for (int i = rank; i < n; i++) if (a[i][col])
  ↪ { piv = i; break; }
        if (piv < 0) continue;
        swap(a[rank], a[piv]);
        for (int i = 0; i < n; i++) if (i != rank &&
  ↪ a[i][col]) a[i] ^= a[rank];
        where[col] = rank++;
    }
    for (int i = rank; i < n; i++) if (a[i][m])
  ↪ return -1;
    if (sol) { sol->reset(); for (int c = 0; c < m;
  ↪ c++) if (where[c] >= 0) (*sol)[c] =
  ↪ a[where[c]][m]; }
    return rank;
  ↪   // #solutions = 2^(m - rank)
}
// USES: "light switch" puzzles, xor-equation
  ↪ counting, rank of a set of vectors over GF(2),
//       linear independence of xor values (dense
  ↪ sibling of the XOR basis).

// ---- Gaussian elimination over the reals, with
  ↪ partial pivoting ----
// Returns rank; a is n x (m+1) augmented. Solution
  ↪ in x (free vars = 0). eps-based.
int gaussReal(vector<vector<ld>> a, int m,
  ↪ vector<ld>& x) {
    int n = a.size(), rank = 0;
    vector<int> where(m, -1);
    for (int col = 0; col < m && rank < n; col++) {
        int piv = rank;
        for (int i = rank; i < n; i++) if
  ↪ (fabsl(a[i][col]) > fabsl(a[piv][col])) piv = i;
        if (fabsl(a[piv][col]) < 1e-12) continue;
        swap(a[rank], a[piv]);
        for (int i = 0; i < n; i++) if (i != rank) {
            ld f = a[i][col] / a[rank][col];
```

```
            for (int j = col; j <= m; j++) a[i][j] -=
  ↪ f * a[rank][j];
        }
        where[col] = rank++;
    }
    x.assign(m, 0);
    for (int c = 0; c < m; c++) if (where[c] >= 0)
  ↪ x[c] = a[where[c]][m] / a[where[c]][c];
    return rank;
}
// The classic use is EXPECTED-VALUE DP with cyclic
  ↪ state dependencies: write one equation per
// state, E[s] = 1 + sum p(s->t) E[t], and solve. n
  ↪ <= ~500 fits O(n^3).
```

6.4.4 WeightedXorBasis

WEIGHTED XOR BASIS - keep the MAXIMUM-WEIGHT independent set of vectors over GF(2). O(B) per

```
// insert, online, no sorting and no rollback needed.
// The vectors of a matroid's ground set with a "keep
  ↪ the heaviest" greedy: linear independence is
// a MATROID, so the greedy is optimal. Standard
  ↪ basis insertion keeps whichever vector arrives
// first; this one keeps the heavier and pushes the
  ↪ lighter down to be re-inserted, which is the
// same as running Kruskal's exchange argument
  ↪ implicitly.
// REACH FOR IT: "pick a maximum-weight subset of
  ↪ vectors that stays linearly independent",
// "maximum spanning forest over GF(2) constraints",
  ↪ cycle-space problems where each edge carries
// both an xor value and a cost, and any "add items
  ↪ online, report the best independent set now".
struct WBasis {
    static const int B = 62;
    ll b[B + 1] = {};
  ↪   // b[i] has leading bit i
    ll w[B + 1] = {};
  ↪   // weight of the vector stored in b[i]
    int sz = 0;

    void insert(ll x, ll wt) {
        for (int i = B; i >= 0; i--) if (x >> i & 1) {
            if (!b[i]) { b[i] = x, w[i] = wt, sz++;
  ↪ return; }
            if (w[i] < wt) swap(b[i], x), swap(w[i],
  ↪ wt);   // keep the heavier one at this pivot
            x ^= b[i];
        }
    }
    ll totalWeight() const { ll s = 0; for (int i =
  ↪ 0; i <= B; i++) s += w[i]; return s; }
    ll maxXor(ll r = 0) const { for (int i = B; i >=
  ↪ 0; i--) r = max(r, r ^ b[i]); return r; }
};
// MINIMUM-weight independent set: negate the weights
  ↪ (or compare with > instead of <).
// THE TRAP: you may NOT just sort by weight and
  ↪ insert greedily into a plain basis if the vectors
// arrive online - but if they are all known up
  ↪ front, sorting descending and using the ordinary
// Basis::insert gives exactly the same answer and is
  ↪ simpler. Use this version when insertions are
// interleaved with queries.
// MATROID INTERSECTION generalises this to
  ↪ "independent in TWO matroids at once" (e.g.
  ↪ linearly
// independent AND a forest); that needs augmenting
  ↪ paths in an exchange graph, not a greedy.
// LINEAR MATROID over a general field: same greedy
  ↪ with Gaussian row-reduction instead of xor.
// TYPICAL ENCODING TRICK: to force "at most one
  ↪ vector per group", append group-identifier bits
// above the value bits (a 128-bit word via __int128
  ↪ gives you 62 value bits plus 66 tag bits).
// That turns "spanning-forest + xor" constraints
  ↪ into one basis insert per edge.
```

6.4.5 XorSystemExtremes

LEXICOGRAPHICALLY EXTREME SOLUTION of a GF(2) system, on top of 03 - GaussXorAndReal.cpp.

```
// gaussXor returns SOME solution (free variables =
  ↪ 0), which is the lexicographically SMALLEST one
// when variables are compared in index order x[0],
  ↪ x[1], ... . Problems that ask for the
// lexicographically LARGEST assignment, or that
  ↪ weight the variables, need this wrapper.
// THE TRICK: reverse the variable order before
  ↪ eliminating and reverse the answer back. Reduced
// row echelon form always puts a pivot as early as
  ↪ possible, so "free variables = 0" minimises the
// reversed vector, i.e. maximises the original. O(n
  ↪ * m / 64), same as one elimination.
// Careful: "lexicographically largest" here means as
  ↪ a SEQUENCE x[0] > x[1] > ...; if the problem
// means "largest as a binary NUMBER with x[0] the
  ↪ most significant bit", that is the same thing.
template <int M>
int gaussXorLexMax(vector<bitset<M + 1>> a, int m,
  ↪ bitset<M>& sol) {
    int n = a.size();
    for (int i = 0; i < n; i++) {
  ↪     // reverse the m variable columns
        bitset<M + 1> t;
        for (int j = 0; j < m; j++) t[j] = a[i][m - 1
  ↪ - j];
        t[m] = a[i][m];
        a[i] = t;
    }
    bitset<M> s;
    int rank = gaussXor<M>(a, m, &s);
    if (rank < 0) return -1;
  ↪     // inconsistent
    for (int j = 0; j < m; j++) sol[j] = s[m - 1 -
  ↪ j];    // and reverse the answer back
    return rank;
  ↪     // #solutions = 2^(m - rank)
}
// PRIORITY ORDER other than "by index": permute the
  ↪ columns by the priority you want (most
// important variable first), solve, permute back.
  ↪ Only the ORDER of the columns matters.
// MAXIMISE A LINEAR OBJECTIVE sum w_i x_i over the
  ↪ solution set: that is NOT lexicographic and is
// NP-hard in general (max-XOR-SAT). It is easy only
  ↪ when all w_i >= 0 and you may take each pivot
// greedily - i.e. when the objective is itself
  ↪ lexicographic.
// MAXIMISE THE XOR of the solution vector viewed as
  ↪ a number: build a basis of the NULL SPACE
// (one vector per free variable: set that free
  ↪ variable to 1, back-substitute the pivots), then
// run Basis::maxXor starting from the particular
  ↪ solution (04 - Linear Algebra/01).
// COUNTING: the solution set is a coset, so every
  ↪ count is 2^(m - rank) or 0; "number of solutions
```

```
// with x[i] = 1" is either 0, the full count, or
   ↪ half of it, depending on whether e_i is forced.
```

6.4.6 GF2MatrixInverse

INVERSE OF A MATRIX OVER GF(2) with bitsets - O(n^3 / 64). n = 2000 runs in well under a second,

```
// where a Mint-based inverse at the same size is 8e9
   ↪ operations and hopeless.
// Returns {} if the matrix is singular. Rows are
   ↪ bitset<M> with bit j = entry (i, j).
// REACH FOR IT for xor-system problems that need the
   ↪ FULL inverse rather than one solution:
//  - "for each edge, is it in EVERY / SOME spanning
   ↪ structure" style queries, which reduce to a
//    single entry of the inverse (CF 736D: after
   ↪ inverting the incidence-style matrix, entry
//    inv[b][a] decides whether edge (a, b) is
   ↪ forced);
//  - answering many right-hand sides against one
   ↪ fixed matrix (x = A^-1 b in O(n^2 / 64) each);
//  - determinant over GF(2) (= 1 iff invertible),
   ↪ rank, and the null space in one pass.
template <int M>
vector<bitset<M>> inverseGF2(vector<bitset<M>> a) {
    int n = a.size();
    vector<bitset<M>> inv(n);
    for (int i = 0; i < n; i++) inv[i][i] = 1;
    for (int i = 0; i < n; i++) {
   ↪    // forward elimination
        int piv = -1;
        for (int j = i; j < n; j++) if (a[j][i]) {
   ↪ piv = j; break; }
        if (piv < 0) return {};
   ↪    // singular
        swap(a[i], a[piv]), swap(inv[i], inv[piv]);
        for (int j = i + 1; j < n; j++) if (a[j][i])
   ↪ a[j] ^= a[i], inv[j] ^= inv[i];
    }
    for (int i = n - 1; i >= 0; i--)
   ↪    // back substitution
        for (int j = i + 1; j < n; j++) if (a[i][j])
   ↪ inv[i] ^= inv[j];
    return inv;
}
// NO PIVOT VALUES to divide by: over GF(2) every
   ↪ non-zero entry is 1, so elimination is pure xor
// and there is no numerical anything. That is why
   ↪ this is the cleanest Gaussian elimination there
// is - and why you should reduce to GF(2) whenever a
   ↪ problem lets you.
// DETERMINANT over GF(2) is 1 iff the forward pass
   ↪ finds a pivot in every column; equivalently
// rank == n. For the rank alone, or a solution of A
   ↪ x = b, use gaussXor (04 - Linear Algebra/03)
// which is the same loop with an augmented column.
// NULL SPACE: run gaussXor to RREF; for each free
   ↪ column c, the vector with 1 at c and the pivot
// entries copied from column c is a basis vector of
   ↪ the kernel. Its size is n - rank, and the
// solution set of A x = b is (any particular
   ↪ solution) + (that span).
// PRODUCT over GF(2) in O(n^3 / 64): transpose b,
   ↪ then c[i][j] = ((a[i] & bt[j]).count() & 1).
// Or, faster, c[i] = xor of a's rows selected by b -
   ↪ that is O(n^2 / 64) per row with no popcounts.
// M IS A COMPILE-TIME BOUND: bitset<M> must be at
   ↪ least n wide; keep M a multiple of 64.
```

6.5 Convolution

6.5.1 FFT

FFT over complex doubles. conv() is exact while |result coefficient| < ~1e15.

```
// Prefer NTT when the answer is taken modulo an NTT
   ↪ prime; use convMod for arbitrary moduli.
typedef complex<double> C;
void fft(vector<C>& a) {
    int n = a.size(), L = 31 - __builtin_clz(n);
    static vector<complex<long double>> R(2, 1);
    static vector<C> rt(2, 1);
    for (static int k = 2; k < n; k *= 2) {
        R.resize(n), rt.resize(n);
        auto x = polar(1.0L, acosl(-1.0L) / k);
        for (int i = k; i < 2 * k; i++) rt[i] = R[i]
   ↪ = i & 1 ? R[i / 2] * x : R[i / 2];
    }
    vector<int> rev(n);
    for (int i = 0; i < n; i++) rev[i] = (rev[i / 2]
   ↪ | (i & 1) << L) / 2;
    for (int i = 0; i < n; i++) if (i < rev[i])
   ↪ swap(a[i], a[rev[i]]);
    for (int k = 1; k < n; k *= 2)
        for (int i = 0; i < n; i += 2 * k)
            for (int j = 0; j < k; j++) {
                C z = rt[j + k] * a[i + j + k];
                a[i + j + k] = a[i + j] - z;
                a[i + j] += z;
            }
}
// Real convolution. Result is rounded to the nearest
   ↪ integer.
vector<ll> conv(const vector<ll>& a, const
   ↪ vector<ll>& b) {
    if (a.empty() || b.empty()) return {};
    int rs = a.size() + b.size() - 1, n = 1;
    while (n < rs) n <<= 1;
    vector<C> in(n), out(n);
    for (size_t i = 0; i < a.size(); i++)
   ↪ in[i].real((double)a[i]);
    for (size_t i = 0; i < b.size(); i++)
   ↪ in[i].imag((double)b[i]);
    fft(in);
    for (C& x : in) x *= x;
   ↪    // (a+bi)^2 = a^2-b^2 + 2abi
    for (int i = 0; i < n; i++) out[i] = in[-i & (n -
   ↪ 1)] - conj(in[i]);
    fft(out);
    vector<ll> res(rs);
    for (int i = 0; i < rs; i++) res[i] =
   ↪ llround(imag(out[i]) / (4 * n));
    return res;
}
// Convolution modulo an ARBITRARY M, splitting
   ↪ coefficients at sqrt(M) to keep precision.
vector<ll> convMod(const vector<ll>& a, const
   ↪ vector<ll>& b, ll M) {
    if (a.empty() || b.empty()) return {};
    int rs = a.size() + b.size() - 1, B = 32 -
   ↪ __builtin_clz(rs), n = 1 << B;
    ll cut = (ll)sqrtl((ld)M);
    vector<C> L(n), R(n), os(n), ol(n);
    for (size_t i = 0; i < a.size(); i++) L[i] =
   ↪ C((double)(a[i] / cut), (double)(a[i] % cut));
    for (size_t i = 0; i < b.size(); i++) R[i] =
   ↪ C((double)(b[i] / cut), (double)(b[i] % cut));
    fft(L), fft(R);
```

```
    for (int i = 0; i < n; i++) {
        int j = -i & (n - 1);
        ol[j] = (L[i] + conj(L[j])) * R[i] / (2.0 *
  ↪ n);
        os[j] = (L[i] - conj(L[j])) * R[i] / (2.0 *
  ↪ n) / C(0, 1);
    }
    fft(ol), fft(os);
    vector<ll> res(rs);
    for (int i = 0; i < rs; i++) {
        ll av = llround(real(ol[i])), cv =
  ↪ llround(imag(os[i]));
        ll bv = llround(imag(ol[i])) +
  ↪ llround(real(os[i]));
        res[i] = ((av % M * cut + bv) % M * cut + cv)
  ↪ % M;
    }
    return res;
}
```

6.5.2 NTT

NTT modulo 998244353 (= 119*2^23 + 1, primitive root 3). O(n log n).

```
// multiply() is the entry point. MAX LENGTH 2^23 for
  ↪ 998244353 (see the assert in ntt()).
// For an ARBITRARY modulus: run this under three of
  ↪ the primes below and Garner them together
// (06 - Garner.cpp is written for exactly that), or
  ↪ split the coefficients at sqrt(M) and use
// convMod in 01 - FFT.cpp.
const ll NMOD = 998244353, NROOT = 3;
ll npow(ll a, ll e, ll m = NMOD) { ll r = 1; a %= m;
  ↪ for (; e; e >>= 1, a = a * a % m) if (e & 1) r =
  ↪ r * a % m; return r; }

void ntt(vector<ll>& a, bool inv) {
    int n = a.size();
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <<= 1) {
        assert((NMOD - 1) % len == 0);   // 998244352
  ↪ = 119*2^23: len > 2^23 truncates the division
        ll w = npow(NROOT, (NMOD - 1) / len);   //
  ↪ and returns GARBAGE with no other symptom
        if (inv) w = npow(w, NMOD - 2);
        for (int i = 0; i < n; i += len) {
            ll cur = 1;
            for (int j = 0; j < len / 2; j++) {
                ll u = a[i + j], v = a[i + j + len /
  ↪ 2] * cur % NMOD;
                a[i + j] = (u + v) % NMOD;
                a[i + j + len / 2] = (u - v + NMOD) %
  ↪ NMOD;
                cur = cur * w % NMOD;
            }
        }
    }
    if (inv) { ll ninv = npow(n, NMOD - 2); for (ll&
  ↪ x : a) x = x * ninv % NMOD; }
}
vector<ll> multiply(vector<ll> a, vector<ll> b) {
    if (a.empty() || b.empty()) return {};
    int rs = a.size() + b.size() - 1, n = 1;
    while (n < rs) n <<= 1;
    a.resize(n), b.resize(n);
    ntt(a, false), ntt(b, false);
    for (int i = 0; i < n; i++) a[i] = a[i] * b[i] %
  ↪ NMOD;
    ntt(a, true);
    a.resize(rs);
    return a;
}
// ARBITRARY MODULUS: split each coefficient as a =
  ↪ hi*2^15 + lo and combine three products
// (or run NTT under three NTT-friendly primes and
  ↪ CRT). Values must stay below ~1e18.
// COMMON NTT PRIMES: 998244353 (g=3), 167772161
  ↪ (g=3), 469762049 (g=3), 1004535809 (g=3).
// USES: polynomial multiplication; "count
  ↪ pairs/triples with a given sum"; convolving
// distributions; string matching with wildcards; sum
  ↪ of C(a_i, k) over all i for every k.
```

6.5.3 FWHT

Fast Walsh-Hadamard Transform (FWHT) - Time: O(N log N)

```
// Computes Bitwise Convolutions: XOR, AND, OR for
  ↪ array of size N = 2^k
struct FWHT {
    enum Type { XOR, AND, OR };

    static void transform(vector<ll>& a, bool invert,
  ↪ Type type = XOR) {
        int n = a.size();
        for (int len = 1; 2 * len <= n; len <<= 1) {
            for (int i = 0; i < n; i += 2 * len) {
                for (int j = 0; j < len; j++) {
                    ll u = a[i + j], v = a[i + len +
  ↪ j];
                    if (type == XOR) {
                        a[i + j] = u + v;
                        a[i + len + j] = u - v;
                    } else if (type == AND) {
                        a[i + j] = invert ? u - v : u
  ↪ + v;
                    } else if (type == OR) {
                        a[i + len + j] = invert ? v -
  ↪ u : u + v;
                    }
                }
            }
        }
        if (type == XOR && invert) {
            for (ll &x : a) x /= n;
        }
    }

    // Computes C[k] = sum_{i op j = k} (A[i] * B[j])
  ↪ where op in {XOR, AND, OR}
    static vector<ll> multiply(vector<ll> a,
  ↪ vector<ll> b, Type type = XOR) {
        int n = 1;
        while (n < max(a.size(), b.size())) n <<= 1;
        a.resize(n); b.resize(n);

        transform(a, false, type);
        transform(b, false, type);
        for (int i = 0; i < n; i++) a[i] *= b[i];
        transform(a, true, type);
        return a;
    }
};
```

6.5.4 Polynomial

FORMAL POWER SERIES over NMOD = 998244353, on top of multiply() from 02 - NTT.cpp.

```
// Every routine works "mod x^k".
   ↪ inv/log/exp/sqrt/pow are all Newton iteration:
   ↪ each step
// doubles the number of correct coefficients, so the
   ↪ cost is the cost of the last multiply.
typedef vector<ll> Poly;

Poly cut(Poly a, int k) { a.resize(k, 0); return a; }
Poly operator+(Poly a, const Poly& b) {
    a.resize(max(a.size(), b.size()), 0);
    for (size_t i = 0; i < b.size(); i++) a[i] =
   ↪ (a[i] + b[i]) % NMOD;
    return a;
}
Poly operator-(Poly a, const Poly& b) {
    a.resize(max(a.size(), b.size()), 0);
    for (size_t i = 0; i < b.size(); i++) a[i] =
   ↪ (a[i] - b[i] + NMOD) % NMOD;
    return a;
}
Poly operator*(const Poly& a, ll c) { Poly r = a; for
   ↪ (ll& x : r) x = x * (c % NMOD) % NMOD; return r;
   ↪ }

Poly deriv(const Poly& a) {
    if (a.size() <= 1) return {0};
    Poly r(a.size() - 1);
    for (size_t i = 1; i < a.size(); i++) r[i - 1] =
   ↪ a[i] * i % NMOD;
    return r;
}
Poly integ(const Poly& a) {
    Poly r(a.size() + 1, 0);
    for (size_t i = 0; i < a.size(); i++) r[i + 1] =
   ↪ a[i] * npow(i + 1, NMOD - 2) % NMOD;
    return r;
}
// 1/a mod x^k. Requires a[0] != 0.   g_{2m} = g_m *
   ↪ (2 - a * g_m)
Poly inv(const Poly& a, int k) {
    Poly g = {npow(a[0], NMOD - 2)};
    for (int m = 1; m < k; m <<= 1) {
        Poly t = multiply(cut(a, 2 * m), g);
        t = cut(t, 2 * m);
        for (ll& x : t) x = (NMOD - x) % NMOD;
        t[0] = (t[0] + 2) % NMOD;
        g = cut(multiply(g, t), 2 * m);
    }
    return cut(g, k);
}
// log(a) mod x^k. Requires a[0] == 1.   log a =
   ↪ integral(a' / a)
Poly log_(const Poly& a, int k) { return
   ↪ cut(integ(cut(multiply(deriv(a), inv(a, k)), k -
   ↪ 1)), k); }
// exp(a) mod x^k. Requires a[0] == 0.   g_{2m} = g_m
   ↪ * (1 + a - log g_m)
Poly exp_(const Poly& a, int k) {
    Poly g = {1};
    for (int m = 1; m < k; m <<= 1) {
        Poly t = cut(a, 2 * m) - log_(cut(g, 2 * m),
   ↪ 2 * m);
        t[0] = (t[0] + 1) % NMOD;
        g = cut(multiply(g, t), 2 * m);
    }
    return cut(g, k);
}
// sqrt(a) mod x^k, for a[0] == 1 (general a[0] needs
   ↪ a modular square root of a[0]).
Poly sqrt_(const Poly& a, int k) {
    Poly g = {1};
    ll i2 = npow(2, NMOD - 2);
    for (int m = 1; m < k; m <<= 1) {
        Poly t = cut(a, 2 * m) + multiply(g, g);
        t = cut(t, 2 * m);
        g = cut(multiply(t, inv(cut(g, 2 * m) * 2, 2
   ↪ * m)), 2 * m);
    }
    return cut(g, k);
}
// a^e mod x^k, for a[0] == 1:  exp(e * log a).  For
   ↪ a[0] != 1 factor out c*x^d first.
Poly pow_(const Poly& a, ll e, int k) { return
   ↪ exp_(log_(a, k) * (e % NMOD), k); }

// Polynomial DIVISION with remainder: a = q*b + r,
   ↪ deg r < deg b.  O(n log n)
pair<Poly, Poly> divmod(Poly a, Poly b) {
    while (a.size() > 1 && !a.back()) a.pop_back();
   ↪  // trim BOTH, or the reversal is wrong
    while (b.size() > 1 && !b.back()) b.pop_back();
    int n = a.size(), m = b.size();
    if (n < m) return {{0}, a};
    Poly ra(a.rbegin(), a.rend()), rb(b.rbegin(),
   ↪ b.rend());
    Poly q = cut(multiply(cut(ra, n - m + 1),
   ↪ inv(cut(rb, n - m + 1), n - m + 1)), n - m + 1);
    reverse(all(q));
    Poly r = cut(a - multiply(q, b), max(1, m - 1));
   ↪   // cut AFTER subtracting, not before
    return {q, r};
}
// USES: counting with generating functions (exp of a
   ↪ "connected" series = all structures),
// partition numbers via prod 1/(1-x^i) = exp(sum
   ↪ ...), Catalan-style algebraic equations via
// sqrt, k-th power of a GF, linear recurrence
   ↪ coefficients via inv, and Lagrange inversion.
```

6.5.5 CyclicConvolution

CYCLIC CONVOLUTION and CYCLIC CORRELATION of two length-n arrays, O(n log n).

```
// Needs: multiply (02 - NTT.cpp)  [or swap in conv()
   ↪ from 01 - FFT.cpp].
// Cyclic = multiplication in the ring modulo x^n -
   ↪ 1: indices wrap around instead of growing.
// REACH FOR IT whenever the index set is a CIRCLE
   ↪ rather than a line: necklaces and rotations,
// "for every rotation r, the score of aligning a
   ↪ against b rotated by r", counting pairs (i, j)
// with (i + j) mod n = k, and any DP on residues
   ↪ modulo n.
typedef vector<ll> Poly;
Poly cyclicConv(const Poly& a, const Poly& b, int n)
   ↪ {   // c[k] = sum_{(i+j) mod n == k} a[i]b[j]
    Poly p = multiply(a, b), c(n, 0);
    for (size_t i = 0; i < p.size(); i++) c[i % n] =
   ↪ (c[i % n] + p[i]) % NMOD;
    return c;
}
// c[k] = sum_i a[i] * b[(i + k) mod n]  -- "slide b
   ↪ by k over a and take the dot product".
// This is the one you actually want for rotations.
```

```
  ↪ Reverse a cyclically, then convolve.
Poly cyclicCorr(const Poly& a, const Poly& b, int n) {
    Poly ar(n);
    for (int i = 0; i < n; i++) ar[i] = a[(n - i) %
  ↪ n];
    return cyclicConv(ar, b, n);
}
// NEGACYCLIC convolution (modulo x^n + 1, "wrap with
  ↪ a sign flip"): same fold but subtract the
// wrapped part, c[i % n] += (i / n odd ? -p[i] :
  ↪ p[i]). Shows up in ring-LWE style problems and in
// the Schoenhage-Strassen recursion.
// FASTER when n | (NMOD - 1): a length-n NTT IS the
  ↪ cyclic convolution - transform, multiply
// pointwise, transform back, with no padding to a
  ↪ power of two and no fold. When n is arbitrary,
// Bluestein (06 - ChirpZ.cpp) gives you the length-n
  ↪ DFT anyway.
// USES
//  ROTATION SCORING: given two binary necklaces, the
  ↪ number of positions where both are 1 under
//  rotation r is exactly cyclicCorr(a, b)[r] - one
  ↪ call answers all n rotations.
//  RESIDUE DP: "number of pairs whose sum is
  ↪ congruent to k mod n" for all k at once.
//  CIRCULAR STRING MATCHING with wildcards: encode
  ↪ as sums of products (see the wildcard trick in
//  07 - Strings/01 - String Matching/04) and fold
  ↪ cyclically.
//  A circulant matrix times a vector IS a cyclic
  ↪ convolution, so this is also how you apply a
//  circulant in O(n log n) instead of O(n^2).
```

6.5.6 ChirpZ

CHIRP-Z / BLUESTEIN: evaluate a polynomial at a GEOMETRIC progression of points,

```
// Needs: multiply, npow (02 - NTT.cpp); Poly / cut
  ↪ (04 - Polynomial.cpp).
//   P(1), P(z), P(z^2), ..., P(z^(m-1))   in O((n +
  ↪ m) log(n + m)),
// where n = deg P. Multipoint evaluation at
  ↪ arbitrary points is O(n log^2 n); this is a full
  ↪ log
// factor cheaper and much shorter, and it covers
  ↪ most of the cases that actually come up.
// PRECONDITION: z must be INVERTIBLE mod NMOD (z !=
  ↪ 0). Any m and n are fine - no power of two.
// REACH FOR IT for: a DFT of ARBITRARY length (set z
  ↪ = a primitive m-th root of unity - this is
// how you do a length-7 or length-1000000 transform
  ↪ with an NTT that only likes powers of two);
// "evaluate f at 1, r, r^2, ..." which is what
  ↪ q-analogue and geometric-weight problems ask for;
// and cyclic convolution of a non-power-of-two
  ↪ length.
// IDENTITY: i*j = C(i+j, 2) - C(i, 2) - C(j, 2),
  ↪ which turns the exponent z^(i*j) into a product
// of three one-index factors, so the sum becomes a
  ↪ convolution ("chirp" = the z^C(k,2) sequence).
Poly chirpZ(const Poly& a, ll z, int m) {
    int n = a.size() - 1;
  ↪    // deg P
    if (m <= 0) return {};
    ll iz = npow(z, NMOD - 2);
    auto tri = [&](ll k) { return k * (k - 1) / 2 %
  ↪ (NMOD - 1); };       // C(k, 2) mod (p-1)
    Poly f(n + 1), g(n + m);
    for (int j = 0; j <= n; j++) f[n - j] = a[j] *
  ↪ npow(iz, tri(j)) % NMOD;   // reversed
    for (int t = 0; t < n + m; t++) g[t] = npow(z,
  ↪ tri(t));
    Poly h = multiply(f, g);
    Poly r(m);
    for (int k = 0; k < m; k++) r[k] = h[n + k] *
  ↪ npow(iz, tri(k)) % NMOD;
    return r;
}
// ARBITRARY-LENGTH DFT: with w a primitive m-th root
  ↪ of unity mod NMOD (needs m | NMOD - 1),
// chirpZ(a, w, m) is the length-m DFT of a; the
  ↪ inverse is chirpZ(., w^-1, m) scaled by 1/m.
// That gives cyclic convolution of ANY length m in
  ↪ O(m log m) without a fold.
// THE OTHER DIRECTION: given the m values P(z^k),
  ↪ recovering P is a chirp-z with z^-1 followed by
// a rescale - the transform is its own inverse up to
  ↪ the chirp factors, exactly like a DFT.
// EVALUATING AT AN ARITHMETIC progression instead
  ↪ (P(a), P(a+d), ...): that is a Taylor SHIFT plus
// a scale, see 10 - PolyShiftAndComposition.cpp,
  ↪ also O(n log n).
// OVERFLOW: tri(k) is taken mod (NMOD - 1) by
  ↪ Fermat, so k may be as large as you like; without
// that reduction k*(k-1)/2 overflows around k = 4e9.
// RELATED: the same "split the exponent" trick
  ↪ underlies baby-step giant-step for discrete log
  ↪ and
// the O(sqrt(n)) evaluation of n! mod p.
```

6.5.7 RelaxedConvolution

ONLINE (RELAXED) CONVOLUTION. You feed a[i] and b[i] one index at a time and immediately get

```
// Needs: multiply (02 - NTT.cpp).
// c[i] = sum_{j+k=i} a[j] b[k], BEFORE you have to
  ↪ supply a[i+1] - which is exactly what a
// self-referential DP needs. Total O(n log^2 n) for
  ↪ all n steps; ~1s at n = 2e5.
// REACH FOR IT whenever f[i] depends on a
  ↪ convolution of f with itself (or with something f
// determines) up to index i - 1: f[n] = sum f[k]
  ↪ f[n-1-k] (Catalan-shaped DPs), f[n] = sum
// g[k] f[n-k] where you only learn g[n] after
  ↪ computing f[n-1], probability DPs with a feedback
// loop, ABC315-H style. Divide and conquer over the
  ↪ recursion is the alternative and is usually
// harder to write; a plain O(n^2) double loop is the
  ↪ thing you are replacing.
// The blocks are scheduled so that the product of
  ↪ two length-w blocks is done exactly when both
// halves are known, and every coefficient is covered
  ↪ exactly once.
struct Relaxed {
    vector<ll> a, b, c;
    int k = 0;
    Relaxed(int n) : a(n), b(n), c(2 * n, 0) {}
    // supply a[i] = x and b[i] = y (i must equal the
  ↪ current index); returns c[i].
    ll extend(ll x, ll y) {
        a[k] = x % NMOD, b[k] = y % NMOD;
        int s = k + 2;
        for (int w = 1; s % w == 0 && w < s; w <<= 1)
  ↪ {
            for (int t = 0; t < 2; t++) {
                if (t == 0 || w * 2 != s) {
  ↪    // the symmetric block is done once only
                    vector<ll> f(a.begin() + w - 1,
```

```
    ↪ a.begin() + 2 * w - 1);
                    vector<ll> g(b.begin() + k - w +
    ↪ 1, b.begin() + k + 1);
                    f = multiply(f, g);
                    for (size_t i = 0; i < f.size()
    ↪ && k + i < c.size(); i++)
                        c[k + i] = (c[k + i] + f[i])
    ↪ % NMOD;
                }
                swap(a, b);
    ↪     // and again, so a/b are restored
            }
        }
        return c[k++];
    }
};
// SELF-CONVOLUTION shorthand: pass the same value
    ↪ twice, extend(f[i], f[i]).
// TYPICAL SHAPE
//   Relaxed R(n); f[0] = 1;
//   for (i = 1; i <= n; i++) { ll t =
    ↪ R.extend(f[i-1], f[i-1]); f[i] = /* uses t */; }
//   Note the OFF-BY-ONE: after the i-th extend you
    ↪ know coefficient i-1 of the product, so the
//   value you feed in must be the one that is
    ↪ already final.
// WHEN NOT TO: if the recurrence is f[n] = sum_{k<n}
    ↪ g[k] f[n-k] with g KNOWN in advance, the
// cheaper standard answer is divide and conquer over
    ↪ [l, r) (contribute [l, mid) to [mid, r) with
// one multiply), same O(n log^2 n) but a smaller
    ↪ constant and no bookkeeping.
// If instead the whole system is polynomial (f =
    ↪ F(f) as power series), Newton iteration on the
// series (04 - Polynomial.cpp: inv / log / exp /
    ↪ sqrt) is O(n log n) and beats both.
```

6.5.8 NTT2D

2-D CONVOLUTION of two bivariate polynomials mod NMOD = 998244353:

```
// Needs: ntt, npow (02 - NTT.cpp).
//   c[i][j] = sum_{p+r=i, q+s=j} a[p][q] * b[r][s],
    ↪  in O(R*C*log(R*C)) with R, C the padded
// dimensions. The transform SEPARATES: NTT every
    ↪ row, then every column, multiply pointwise,
// invert columns then rows. Same idea works for any
    ↪ number of dimensions.
// REACH FOR IT when the state has TWO additive
    ↪ coordinates: "count paths with a given (x, y)
// displacement", "polynomials in two variables" (a
    ↪ variable per statistic you must track jointly),
// 2-D pattern matching by sums of products, and grid
    ↪ DPs whose transition is a translation.
typedef vector<vector<ll>> Poly2;
void ntt2d(Poly2& a, bool inv) {
    int R = a.size(), C = a[0].size();
    for (int i = 0; i < R; i++) ntt(a[i], inv);
    vector<ll> col(R);
    for (int j = 0; j < C; j++) {
        for (int i = 0; i < R; i++) col[i] = a[i][j];
        ntt(col, inv);
        for (int i = 0; i < R; i++) a[i][j] = col[i];
    }
}
Poly2 multiply2D(const Poly2& a, const Poly2& b) {
    int rr = a.size() + b.size() - 1, cc =
    ↪ a[0].size() + b[0].size() - 1, R = 1, C = 1;
    while (R < rr) R <<= 1;
    while (C < cc) C <<= 1;
    Poly2 x(R, vector<ll>(C, 0)), y(R, vector<ll>(C,
    ↪ 0));
    for (size_t i = 0; i < a.size(); i++)
        for (size_t j = 0; j < a[0].size(); j++)
    ↪ x[i][j] = a[i][j];
    for (size_t i = 0; i < b.size(); i++)
        for (size_t j = 0; j < b[0].size(); j++)
    ↪ y[i][j] = b[i][j];
    ntt2d(x, false), ntt2d(y, false);
    for (int i = 0; i < R; i++)
        for (int j = 0; j < C; j++) x[i][j] = x[i][j]
    ↪ * y[i][j] % NMOD;
    ntt2d(x, true);
    x.resize(rr);
    for (auto& row : x) row.resize(cc);
    return x;
}
// CHEAPER ALTERNATIVE - FLATTENING. If one dimension
    ↪ is small (C0 columns in a and C1 in b), pack
// the 2-D array into a 1-D one with stride W >= C0 +
    ↪ C1 - 1 (index i*W + j) and do a SINGLE 1-D
// convolution: the strides are wide enough that the
    ↪ coordinates never interfere. One NTT of length
// R*W instead of R + C transforms - usually faster,
    ↪ and it is three lines. Use the 2-D routine
// when both dimensions are large and comparable.
// The same separability argument gives 2-D FWHT (xor
    ↪ in both coordinates) and 2-D subset-sum
// transforms; nothing about it is special to the NTT.
// MEMORY: R*C longs. 4096 x 4096 is already 128 MB -
    ↪ flatten instead when the product is large.
```

6.5.9 FWHTAnyBase

FWHT IN AN ARBITRARY BASE k: convolution where indices combine by DIGIT-WISE ADDITION MOD k in

```
// Needs: npow (02 - NTT.cpp).
// base k. For k = 2 this is exactly xor and the
    ↪ routine collapses to the usual FWHT; for k = 10
    ↪ it
// is "add the decimal digits without carrying"; for
    ↪ k = 3 it is the ternary "no-carry" addition
// that Nim-like and ternary-state problems produce.
// Length must be n = k^d. Cost O(n * k * log_k n) -
    ↪ the transform along each digit is a length-k
// DFT, done with the k x k matrix M[p][q] = w^(pq).
// PRECONDITION: a primitive k-th root of unity must
    ↪ exist mod NMOD, i.e. k | NMOD - 1.
// 998244352 = 2^23 * 7 * 17, so k in
    ↪ {2,4,8,...,2^23, 7, 14, 17, 34, 119, ...} all
    ↪ work; k = 3
// does NOT - see the note at the bottom. Use a
    ↪ different NTT prime if you need another k.
void fwhtBase(vector<ll>& a, int k, bool inv) {
    int n = a.size();
    ll w = npow(NROOT, (NMOD - 1) / k);
    ↪     // primitive k-th root of unity
    if (inv) w = npow(w, NMOD - 2);
    vector<ll> wp(k);
    wp[0] = 1;
    for (int i = 1; i < k; i++) wp[i] = wp[i - 1] * w
    ↪ % NMOD;
    vector<ll> v(k);
    for (int len = 1; len < n; len *= k)
        for (int i = 0; i < n; i += len * k)
            for (int j = 0; j < len; j++) {
                for (int p = 0; p < k; p++) v[p] =
    ↪ a[i + j + p * len];
```

```
                for (int p = 0; p < k; p++) {
                    ll s = 0;
                    for (int q = 0; q < k; q++) s =
  ↪ (s + v[q] * wp[p * q % k]) % NMOD;
                    a[i + j + p * len] = s;
                }
            }
    if (inv) { ll ni = npow(n, NMOD - 2); for (ll& x
  ↪ : a) x = x * ni % NMOD; }
}
vector<ll> multiplyBase(vector<ll> a, vector<ll> b,
  ↪ int k) {
    int n = 1;
    while (n < (int)max(a.size(), b.size())) n *= k;
  ↪     // n must be a power of k
    a.resize(n, 0), b.resize(n, 0);
    fwhtBase(a, k, false), fwhtBase(b, k, false);
    for (int i = 0; i < n; i++) a[i] = a[i] * b[i] %
  ↪ NMOD;
    fwhtBase(a, k, true);
    return a;
}
// k-TH POWER OF THE OPERATOR: to convolve one array
  ↪ with itself e times, transform once, raise
// every entry to the e-th power, transform back -
  ↪ O(n log n + n log e), not e convolutions.
// TERNARY when 3 does not divide MOD - 1: work in
  ↪ F_p[w] / (w^2 + w + 1), i.e. represent each
// value as a + b*w with w^2 = -1 - w. That is a
  ↪ genuine cube root of unity as long as 3 | p + 1.
// Multiplication is (a1+b1 w)(a2+b2 w) = (a1a2 -
  ↪ b1b2) + (a1b2 + a2b1 - b1b2) w. Everything else
// in the routine above is unchanged.
// PACKING TRICK: if your indices are BITMASKS but
  ↪ the operation is digit-wise addition mod 3
// (e.g. "each element used 0, 1 or 2 times"),
  ↪ rewrite each mask in base 3 first: the value with
// binary digits d_i becomes sum d_i 3^i.
// WHY NOT SUBSET-SUM/OR: OR-convolution is the
  ↪ zeta/Mobius transform (08 - DP/02 - SOS_DP.cpp);
// this file is only for the group Z_k^d, where the
  ↪ transform is invertible pointwise.
```

6.5.10 PolyShiftAndComposition

TAYLOR SHIFT p(x) -> p(x + c), and COMPOSITION p(g(x)). Both mod NMOD = 998244353.

```
// Needs: multiply / npow (02 - NTT.cpp), Poly / cut
  ↪ (04 - Polynomial.cpp).

// TAYLOR SHIFT in O(n log n).  [x^k] p(x+c) =
  ↪ sum_{j>=k} a_j * C(j, k) * c^(j-k), which is a
// correlation once you divide by the factorials,
  ↪ hence ONE multiplication.
// REACH FOR IT to re-centre an interpolation ("I
  ↪ have f sampled at 0..n, I want it at m..m+n"),
// to turn "evaluate at an ARITHMETIC progression a,
  ↪ a+d, a+2d, ..." into a shift plus a scale, and
// inside falling-factorial / Stirling conversions.
Poly taylorShift(Poly a, ll c) {
    int n = a.size();
    vector<ll> f(n), fi(n);
    f[0] = 1;
    for (int i = 1; i < n; i++) f[i] = f[i - 1] * i %
  ↪ NMOD;
    fi[n - 1] = npow(f[n - 1], NMOD - 2);
    for (int i = n - 1; i > 0; i--) fi[i - 1] = fi[i]
  ↪ * i % NMOD;
    Poly A(n), B(n);
    for (int i = 0; i < n; i++) A[n - 1 - i] = a[i] *
  ↪ f[i] % NMOD;      // reversed a_j * j!
    ll p = 1;
    for (int i = 0; i < n; i++) B[i] = p * fi[i] %
  ↪ NMOD, p = p * (c % NMOD + NMOD) % NMOD;
    Poly h = multiply(A, B);
    Poly r(n);
    for (int k = 0; k < n; k++) r[k] = h[n - 1 - k] *
  ↪ fi[k] % NMOD;
    return r;
}
// COMPOSITION p(g(x)) mod x^m in O(sqrt(n) * m log
  ↪ m) by the BLOCK (baby-step giant-step) method:
// write p(y) = sum_{i<K} y^(i*D) * q_i(y) with deg
  ↪ q_i < D, precompute g^0..g^D once, build each
// q_i(g) by linear combinations, then Horner over
  ↪ the K blocks with the single power g^D.
// REACH FOR IT for "substitute one generating
  ↪ function into another": exp/log of a series with
  ↪ a
// non-constant argument, EGF of a labelled
  ↪ construction, "apply a transform given as a
  ↪ series".
Poly composition(const Poly& p, const Poly& g, int m)
  ↪ {
    int n = p.size(), K = 1;
    while (K * K < n) K++;
  ↪     // K blocks of D = ceil(n / K) terms
    int D = (n + K - 1) / K;
    vector<Poly> pw(D + 1, Poly{1});
    for (int i = 1; i <= D; i++) pw[i] =
  ↪ cut(multiply(pw[i - 1], g), m);
    Poly res(m, 0), gd{1};
    for (int i = 0; i < K; i++) {
        Poly blk(m, 0);
        for (int j = 0; j < D && i * D + j < n; j++) {
            ll c = p[i * D + j];
            if (!c) continue;
            for (int t = 0; t < (int)pw[j].size() &&
  ↪ t < m; t++)
                blk[t] = (blk[t] + c * pw[j][t]) %
  ↪ NMOD;
        }
        blk = cut(multiply(blk, gd), m);
        for (int t = 0; t < m && t < (int)blk.size();
  ↪ t++) res[t] = (res[t] + blk[t]) % NMOD;
        gd = cut(multiply(gd, pw[D]), m);
    }
    return res;
}
// COMPOSITIONAL INVERSE f^(-1) (the series h with
  ↪ f(h(x)) = x), for f[0] = 0, f[1] != 0:
// LAGRANGE INVERSION gives a single coefficient
  ↪ directly,
//   [x^n] h = (1/n) * [x^(n-1)] (x / f(x))^n,   and
  ↪ more generally
//   [x^n] G(h(x)) = (1/n) * [x^(n-1)] G'(x) * (x /
  ↪ f(x))^n.
// One coefficient costs O(n log n); all n of them by
  ↪ this route costs O(n^2 log n), so use it when
// you need ONE answer (which is the usual case:
  ↪ Catalan, Fuss-Catalan, tree counts).
// K-TH ROOT of a series with a[0] = 1: pow_(a,
  ↪ inv(k) mod (NMOD - 1)) - i.e. exp(log(a)/k). For
// a[0] != 1 factor out c * x^d first; a k-th root
  ↪ exists only if k | d and c is a k-th power
// residue, and then it is unique up to a k-th root
  ↪ of unity.
```

```
// COST WARNING: composition is the expensive one. If g is a MONOMIAL c*x^t, substitute by hand; if
// p is short, plain Horner with truncated products is O(n * m log m) and simpler.
```

6.5.11 PolyGcdResultant

POLYNOMIAL GCD and RESULTANT over F_p (p = NMOD = 998244353, prime).

```
// Needs: multiply / npow (02 - NTT.cpp), Poly / divmod / cut (04 - Polynomial.cpp).
// deg gcd(a, b) = the number of COMMON ROOTS of a and b counted with multiplicity (over the
// algebraic closure), so gcd answers "do these two polynomials share a root" and "how many".
// Euclid with the fast divmod is O(n log n) per step and O(n) steps; the half-gcd algorithm brings
// it to O(n log^2 n) but is long - this is the version you write in a contest.
Poly trim(Poly a) { while (a.size() > 1 && !a.back()) a.pop_back(); return a; }

Poly polyGcd(Poly a, Poly b) {                    // monic gcd; returns {1} for coprime inputs
    a = trim(a), b = trim(b);
    while (b.size() > 1 || b[0]) { Poly r = trim(divmod(a, b).second); a = b, b = r; }
    if (a.size() == 1 && !a[0]) return a;    // gcd(0, 0)
    ll ic = npow(a.back(), NMOD - 2);    // normalise to monic
    for (ll& x : a) x = x * ic % NMOD;
    return a;
}
// RESULTANT Res(a, b) = lc(b)^deg(a) * prod over roots beta of b of a(beta)
//                     = (-1)^(deg a * deg b) * lc(a)^deg(b) * prod over roots alpha of a of b(alpha)
// It is 0 EXACTLY when a and b share a root, and it is the determinant of the Sylvester matrix -
// so it is the polynomial-time way to eliminate a variable from two equations.
// Euclidean recursion: with r = a mod b,
//   Res(a, b) = lc(b)^(deg a - deg r) * (-1)^(deg b * deg r) * Res(b, r).
// O(n log n) per step, O(n) steps.
ll resultant(Poly a, Poly b) {
    a = trim(a), b = trim(b);
    if ((a.size() == 1 && !a[0]) || (b.size() == 1 && !b[0])) return 0;
    ll res = 1;
    while (b.size() > 1) {
        int da = a.size() - 1, db = b.size() - 1;
        Poly r = trim(divmod(a, b).second);
        int dr = r.size() - 1;
        if (r.size() == 1 && !r[0]) return 0;    // common root
        res = res * npow(b.back(), da - dr) % NMOD;
        if ((ll)db * dr % 2) res = (NMOD - res) % NMOD;
        a = b, b = r;
    }
    return res * npow(b[0], a.size() - 1) % NMOD;
}    // Res(a, const c) = c^deg(a)
// USES
//  DISCRIMINANT of f: Res(f, f') / lc(f) - zero iff f has a repeated root. gcd(f, f') isolates the
//    squarefree part, which is the first step of any polynomial factorisation.
//  ELIMINATION: given P(x, y) = 0 and Q(x, y) = 0, Res_y(P, Q) is a polynomial in x alone whose
//    roots are exactly the x-coordinates of the common solutions.
//  det(A + x*B) style problems and "determinant of a circulant" (03 - Matrices/04) are resultants.
//  COMMON ROOT COUNTING mod p for two polynomials given as products, without factoring either.
// PRECONDITION on the Euclidean chain: p must be PRIME (we invert leading coefficients). Over a
// composite modulus use the Sylvester determinant with detMod (07 - Numerical/02) instead.
// DEGREE GOTCHA: divmod in 04 - Polynomial.cpp keeps a padded remainder, so trim() before reading
// any degree - the sign in the resultant recursion depends on the TRUE degree.
```

6.5.12 CyclotomicPolynomial

CYCLOTOMIC POLYNOMIALS Phi_n(x) = prod over primitive n-th roots of unity of (x - w).

```
// They are the IRREDUCIBLE factors of x^n - 1 over the integers:  x^n - 1 = prod_{d | n} Phi_d(x),
// deg Phi_n = phi(n), and every Phi_n has integer coefficients (tiny ones - the first n with a
// coefficient outside {-1, 0, 1} is n = 105).
// By Mobius inversion  Phi_n(x) = prod_{d | n} (x^d - 1)^mu(n/d), and both multiplying and
// dividing by (x^d - 1) are O(deg) in place, so one Phi_n costs O(sigma(n)) - linear, no FFT.
// REACH FOR IT when a problem factors x^n - 1 (roots-of-unity filters, "count sequences whose sum
// is 0 mod n", cyclic codes), when you need the minimal polynomial of a root of unity, or for
// determinants of circulants factored per divisor.
void mulXd(vector<ll>& a, int d) {                // a *= (x^d - 1)
    int n = a.size();
    a.insert(a.begin(), d, 0);
    for (int i = 0; i < n; i++) a[i] -= a[i + d];
}
void divXd(vector<ll>& a, int d) {                // a /= (x^d - 1), exact division assumed
    int n = a.size();
    for (int i = n - 1 - d; i >= 0; i--) a[i] += a[i + d];
    a.erase(a.begin(), a.begin() + d);
}
int muOne(int n) {                                // Mobius of a single n, by trial division
    int r = 1;
    for (int p = 2; (ll)p * p <= n; p++) if (n % p == 0) {
        n /= p;
        if (n % p == 0) return 0;
        r = -r;
    }
    return n > 1 ? -r : r;
}
vector<ll> cyclotomic(int n) {                    // coefficients, low degree first; size phi(n)+1
    vector<ll> a{1};
    for (int d = 1; d <= n; d++) if (n % d == 0 && muOne(n / d) == 1) mulXd(a, d);
    for (int d = 1; d <= n; d++) if (n % d == 0 && muOne(n / d) == -1) divXd(a, d);
```

```
    return a;                                   //
  ↪ do the multiplications FIRST, then the divisions
}
// FACTS
//  Phi_p(x) = 1 + x + ... + x^(p-1) for prime p;
  ↪ Phi_2n(x) = Phi_n(-x) for odd n > 1;
//  Phi_{n p}(x) = Phi_n(x^p) if p | n, and
  ↪ Phi_n(x^p) / Phi_n(x) otherwise.
//  Phi_n(1) = p if n is a prime power p^k, and 1 for
  ↪ any other n > 1.
//  ORDER OF 2 MOD n and friends: the irreducible
  ↪ factors of x^n - 1 over F_q are the factors of
//  the Phi_d, each splitting into phi(d) / ord_d(q)
  ↪ factors of degree ord_d(q).
//  x^n - 1 over F_p (p not dividing n) is
  ↪ squarefree; if p | n, x^n - 1 = (x^(n/p) - 1)^p.
// ROOTS-OF-UNITY FILTER (the usual reason you are
  ↪ here):
//   sum_{k = r mod m} [x^k] F(x) = (1/m) *
  ↪ sum_{j=0}^{m-1} w^(-jr) * F(w^j),  w a primitive
  ↪ m-th
//   root of unity. Mod p you need m | p - 1 for w to
  ↪ exist; otherwise work in F_p[x] / Phi_m(x),
//   where x IS a primitive m-th root by construction
  ↪ - which is exactly what this file gives you.
// OVERFLOW: coefficients stay small (bounded by a
  ↪ few hundred for n < 1e5) so ll is safe; if you
// want them mod p, reduce afterwards.
```

6.5.13 ConvAnyMod

NTT, because 02 - NTT.cpp hardcodes NMOD.

```
// Needs: npow (02 - NTT.cpp)  -- self-contained
  ↪ otherwise: it carries its own modulus-parametric
// CONVOLUTION MODULO AN ARBITRARY M (composite, not
  ↪ NTT-friendly): run the NTT under THREE
// NTT-friendly primes and recombine with Garner. O(n
  ↪ log n) with a constant of three transforms.
// The true coefficients are bounded by len *
  ↪ (M-1)^2; the three primes below multiply to
  ↪ ~9.6e25,
// so for M <= 1e9 and len <= 1e7 the CRT
  ↪ reconstruction is EXACT, and only then reduced
  ↪ mod M.
// PREFER THIS over convMod in 01 - FFT.cpp when M is
  ↪ large (> ~1e9), when the arrays are long
// (>= 1e6, where the double-based split loses
  ↪ precision), or when you do not want to reason
  ↪ about
// floating point at all. It is exact by
  ↪ construction; the FFT version is not.
const ll CP[3] = {167772161, 469762049, 1224736769};
  ↪         // all c*2^k + 1, primitive root 3
void nttP(vector<ll>& a, bool inv, ll p) {
    int n = a.size();
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <<= 1) {
        ll w = npow(3, (p - 1) / len, p);
        if (inv) w = npow(w, p - 2, p);
        for (int i = 0; i < n; i += len) {
            ll cur = 1;
            for (int j = 0; j < len / 2; j++) {
                ll u = a[i + j], v = a[i + j + len /
  ↪ 2] * cur % p;
                a[i + j] = (u + v) % p, a[i + j + len
  ↪ / 2] = (u - v + p) % p;
                cur = cur * w % p;
            }
        }
    }
    if (inv) { ll ni = npow(n, p - 2, p); for (ll& x
  ↪ : a) x = x * ni % p; }
}
vector<ll> convP(const vector<ll>& a, const
  ↪ vector<ll>& b, ll p) {
    int rs = a.size() + b.size() - 1, n = 1;
    while (n < rs) n <<= 1;
    vector<ll> x(n, 0), y(n, 0);
    for (size_t i = 0; i < a.size(); i++) x[i] =
  ↪ (a[i] % p + p) % p;
    for (size_t i = 0; i < b.size(); i++) y[i] =
  ↪ (b[i] % p + p) % p;
    nttP(x, false, p), nttP(y, false, p);
    for (int i = 0; i < n; i++) x[i] = x[i] * y[i] %
  ↪ p;
    nttP(x, true, p);
    x.resize(rs);
    return x;
}
vector<ll> convAnyMod(const vector<ll>& a, const
  ↪ vector<ll>& b, ll M) {
    if (a.empty() || b.empty()) return {};
    vector<ll> r0 = convP(a, b, CP[0]), r1 = convP(a,
  ↪ b, CP[1]), r2 = convP(a, b, CP[2]);
    ll i01 = npow(CP[0] % CP[1], CP[1] - 2, CP[1]);
    ll i02 = npow(CP[0] % CP[2], CP[2] - 2, CP[2]);
    ll i12 = npow(CP[1] % CP[2], CP[2] - 2, CP[2]);
    vector<ll> res(r0.size());
    for (size_t i = 0; i < r0.size(); i++) {
  ↪         // Garner: mixed-radix digits x0,x1,x2
        ll x0 = r0[i];
        ll x1 = (r1[i] - x0 % CP[1] + CP[1]) % CP[1]
  ↪ * i01 % CP[1];
        ll t = (r2[i] - x0 % CP[2] + CP[2]) % CP[2] *
  ↪ i02 % CP[2];
        ll x2 = (t - x1 % CP[2] + CP[2]) % CP[2] *
  ↪ i12 % CP[2];
        __int128 v = (__int128)x2 * CP[1] % M * CP[0]
  ↪ % M + (__int128)x1 * CP[0] % M + x0;
        res[i] = (ll)(v % M);
    }
    return res;
}
// PRIME MENU (all with primitive root 3; the 2-power
  ↪ in p-1 caps the transform length):
//   167772161 = 5*2^25 + 1    469762049 = 7*2^26 + 1
  ↪     754974721 = 45*2^24 + 1
//   998244353 = 119*2^23 + 1  1004535809 = 479*2^21
  ↪ + 1  1224736769 = 73*2^24 + 1
// TWO PRIMES are enough only when len * (M-1)^2 < P0
  ↪ * P1 (~7.9e16) - i.e. tiny M. Use three.
// SPLIT-COEFFICIENT ALTERNATIVE: convMod in 01 -
  ↪ FFT.cpp writes each coefficient as hi*2^15 + lo
// and does 3 complex FFTs - shorter, but its error
  ↪ grows with len and with M.
// EXACT INTEGER convolution with no modulus: pass an
  ↪ M larger than any possible coefficient, or
// drop the final reduction and reconstruct into
  ↪ __int128.
```

6.6 Interpolation

6.6.1 LagrangeAndFaulhaber

Lagrange interpolation + power sums. Needs Mint + buildFact().

```
// Needs: Mint (01 - Modular Arithmetic/01 -
  ↪ Mint.cpp).

// Given y[0..n] = f(0), f(1), ..., f(n) for a
  ↪ polynomial f of degree <= n, evaluate f(x). O(n).
// The consecutive-points case is the one you
  ↪ actually want; it kills a whole class of problems
// whose answer is "a polynomial in n of degree d"
  ↪ (guess d, sample d+1 values, interpolate).
Mint lagrangeConsecutive(const vector<Mint>& y, ll x)
  ↪ {
    int n = y.size() - 1;
    if (0 <= x && x <= n) return y[x];          // x
  ↪ < 0 would index y[negative]
    vector<Mint> pre(n + 2, 1), suf(n + 2, 1);
    for (int i = 0; i <= n; i++) pre[i + 1] = pre[i]
  ↪ * Mint(x - i);
    for (int i = n; i >= 0; i--) suf[i] = suf[i + 1]
  ↪ * Mint(x - i);
    Mint r = 0;
    for (int i = 0; i <= n; i++) {
        Mint t = y[i] * pre[i] * suf[i + 1] *
  ↪ invFact[i] * invFact[n - i];
        ((n - i) & 1) ? r -= t : r += t;
    }
    return r;
}
// Arbitrary sample points, O(n^2).
Mint lagrange(const vector<Mint>& xs, const
  ↪ vector<Mint>& ys, Mint x) {
    int n = xs.size();
    Mint r = 0;
    for (int i = 0; i < n; i++) {
        Mint num = ys[i], den = 1;
        for (int j = 0; j < n; j++) if (j != i) num
  ↪ *= x - xs[j], den *= xs[i] - xs[j];
        r += num / den;
    }
    return r;
}
// FAULHABER: sum_{i=1..n} i^k  is a polynomial in n
  ↪ of degree k+1 -> sample k+2 points. O(k log k).
Mint powerSum(ll n, int k) {
    vector<Mint> y(k + 2, 0);
    for (int i = 1; i <= k + 1; i++) y[i] = y[i - 1]
  ↪ + Mint(i).pow(k);
    return lagrangeConsecutive(y, n);
}
// Same trick works for: sum of C(i,k), sum of i^k *
  ↪ r^i (degree k in n after dividing by r^n),
// and any DP whose answer is eventually polynomial.
```

6.6.2 BerlekampMassey

Berlekamp-Massey: find the shortest linear recurrence fitting a sequence, O(n^2).

```
// Needs: Mint (01 - Modular Arithmetic/01 -
  ↪ Mint.cpp).
// Then jump to the k-th term in O(len^2 log k). THE
  ↪ weapon for "brute force small n, find the
// pattern, extrapolate" - works on any sequence that
  ↪ satisfies a constant-coefficient recurrence.
// Feed it ~2*len terms (60-100 is usually plenty).
vector<Mint> berlekampMassey(vector<Mint> s) {
    int n = s.size(), L = 0, m = 0;
    vector<Mint> C(n), B(n), T;
    C[0] = B[0] = 1;
    Mint b = 1;
    for (int i = 0; i < n; i++) {
        m++;
        Mint d = s[i];
        for (int j = 1; j <= L; j++) d += C[j] * s[i
  ↪ - j];
        if (d.v == 0) continue;
        T = C;
        Mint coef = d / b;
        for (int j = m; j < n; j++) C[j] -= coef *
  ↪ B[j - m];
        if (2 * L > i) continue;
        L = i + 1 - L, B = T, b = d, m = 0;
    }
    C.resize(L + 1), C.erase(C.begin());
    for (auto& x : C) x = Mint(0) - x;
    return C;
  ↪   // s[i] = sum_j C[j] * s[i-1-j]
}
// k-th term (0-indexed) of the recurrence tr with
  ↪ initial values S. O(|tr|^2 log k).
Mint linearRec(const vector<Mint>& S, const
  ↪ vector<Mint>& tr, ll k) {
    if (tr.empty()) return 0;                   //
  ↪ e[1] would be out of bounds
    int n = tr.size();
    auto comb = [&](vector<Mint> a, vector<Mint> b) {
        vector<Mint> r(2 * n + 1, 0);
        for (int i = 0; i <= n; i++) if (a[i].v)
            for (int j = 0; j <= n; j++) r[i + j] +=
  ↪ a[i] * b[j];
        for (int i = 2 * n; i > n; i--)
            for (int j = 0; j < n; j++) r[i - 1 - j]
  ↪ += r[i] * tr[j];
        r.resize(n + 1);
        return r;
    };
    vector<Mint> pol(n + 1, 0), e(pol);
    pol[0] = e[1] = 1;
    for (++k; k; k /= 2) {
        if (k % 2) pol = comb(pol, e);
        e = comb(e, e);
    }
    Mint r = 0;
    for (int i = 0; i < n; i++) r += pol[i + 1] *
  ↪ S[i];
    return r;
}
```

6.6.3 BostanMori

BOSTAN-MORI: the k-th term of a linear recurrence of order d in O(d log d log k), mod NMOD.

```
// Needs: multiply / npow (02 - NTT.cpp).
// This is the fast replacement for Kitamasa (O(d^2 log k), 06 - Interpolation/02 - linearRec) and
// for companion-matrix exponentiation (O(d^3 log k)). At d = 500, k = 1e18 it is the only one that
// runs. Pair it with Berlekamp-Massey: guess the recurrence from brute-forced terms, then jump.
// INPUT: a[0..d-1] the first d terms, c[0..d-1] with a[n] = sum_i c[i] * a[n-1-i] for n >= d.
// IDEA: the generating function is RATIONAL, sum a_n x^n = P(x) / Q(x) with
//   Q = 1 - c0 x - ... - c(d-1) x^d,   P = (A * Q) mod x^d.
// Then [x^k] P/Q: multiply top and bottom by Q(-x). The new denominator V(x) = Q(x)Q(-x) is EVEN,
// so V(x) = W(x^2); keeping the even or odd part of the numerator (by the parity of k) halves k.
ll bostanMori(vector<ll> a, const vector<ll>& c, ll k) {
    int d = c.size();
    if (k < (ll)a.size() && k < d) return a[k] % NMOD;
    vector<ll> Q(d + 1, 0);
    Q[0] = 1;
    for (int i = 0; i < d; i++) Q[i + 1] = (NMOD - c[i] % NMOD) % NMOD;
    a.resize(d, 0);
    vector<ll> P = multiply(a, Q);
    P.resize(d);
    // P = (A * Q) mod x^d
    while (k) {
        vector<ll> Qm = Q;
        for (size_t i = 1; i < Qm.size(); i += 2) Qm[i] = (NMOD - Qm[i]) % NMOD;
        vector<ll> U = multiply(P, Qm), V = multiply(Q, Qm);
        P.assign((U.size() + 1) / 2, 0);
        for (size_t i = k & 1; i < U.size(); i += 2) P[i / 2] = U[i];
        Q.assign((V.size() + 1) / 2, 0);
        for (size_t i = 0; i < V.size(); i += 2) Q[i / 2] = V[i];
        P.resize(d), Q.resize(d + 1);
        k >>= 1;
    }
    return P[0] * npow(Q[0], NMOD - 2) % NMOD;
}
// THE RATIONAL GENERATING FUNCTION IS THE POINT, not just the k-th term:
//  sum_n a_n = P(1) / Q(1)   (formally; use it when the series really does terminate/converge,
//    e.g. absorbing-state probabilities where a_n is the chance of stopping at step n).
//  sum_n n * a_n = (P'(1) Q(1) - P(1) Q'(1)) / Q(1)^2 - this is the EXPECTED STOPPING TIME, the
//    reason this trick shows up in probability problems: brute-force 2d terms, run Berlekamp-
//    Massey, build P/Q, differentiate at x = 1. Beware Q(1) = 0, which means the series does not
//    sum and you must handle the pole separately.
//  PARTIAL FRACTIONS of P/Q give the closed form a_n = sum c_i r_i^n over the roots of Q.
//  PREFIX SUMS: sum_{n<=k} a_n is the k-th coefficient of P / ((1-x) Q) - just multiply Q by
//    (1 - x) and run the same routine.
// PRECONDITION: NMOD prime, Q[0] = 1 (automatic here). The recurrence must be exact; if you got it
// from Berlekamp-Massey, feed BM at least 2*d terms or the answer is silently wrong.
```

6.6.4 MultipointEval

MULTIPOINT EVALUATION and FAST INTERPOLATION at ARBITRARY points, both O(n log^2 n), mod NMOD.

```
// Needs: multiply (02 - NTT.cpp), Poly / divmod / deriv / cut / operator+ (04 - Polynomial.cpp).
// Evaluating a degree-n polynomial at n points naively is O(n^2); interpolating from n arbitrary
// pairs naively is O(n^2). These are the log-squared versions, built on the same PRODUCT TREE:
// node [l, r] stores prod_{i in [l,r]} (x - x_i).
// REACH FOR IT when n is 1e5 and the points are NOT consecutive integers (if they are, use
// lagrangeConsecutive, O(n)) and NOT a geometric progression (then use chirp-z, O(n log n)).
// Typical: "evaluate this generating function at n given points", "recover the polynomial behind
// n scattered samples", shifting a sampled polynomial to a new set of arguments.
vector<Poly> ptree;    // 4n nodes; call buildTree first
void buildTree(int v, int l, int r, const vector<ll>& x) {
    if (l == r) { ptree[v] = Poly{(NMOD - x[l] % NMOD) % NMOD, 1}; return; }
    int m = (l + r) / 2;
    buildTree(2 * v, l, m, x), buildTree(2 * v + 1, m + 1, r, x);
    ptree[v] = multiply(ptree[2 * v], ptree[2 * v + 1]);
}
void evalRec(int v, int l, int r, Poly f, vector<ll>& out) {
    f = divmod(f, ptree[v]).second;
    // f mod prod (x - x_i) keeps the values
    if (l == r) { out[l] = f.empty() ? 0 : f[0]; return; }
    int m = (l + r) / 2;
    evalRec(2 * v, l, m, f, out), evalRec(2 * v + 1, m + 1, r, f, out);
}
vector<ll> evalAll(const Poly& f, const vector<ll>& x) {
    int n = x.size();
    if (!n) return {};
    ptree.assign(4 * n, {});
    buildTree(1, 0, n - 1, x);
    vector<ll> out(n);
    evalRec(1, 0, n - 1, f, out);
    return out;
}
// INTERPOLATION: with M(x) = prod (x - x_i), the Lagrange basis denominator at x_i is M'(x_i), so
// one multipoint evaluation of M' gives every denominator; then merge the leaves bottom-up with
// result = A * (right subtree product) + B * (left subtree product).
Poly interpRec(int v, int l, int r, const vector<ll>& d) {
    if (l == r) return Poly{d[l]};
    int m = (l + r) / 2;
    Poly A = interpRec(2 * v, l, m, d), B = interpRec(2 * v + 1, m + 1, r, d);
    return multiply(A, ptree[2 * v + 1]) +
```

```
    multiply(B, ptree[2 * v]);
}
Poly interpolate(const vector<ll>& x, const vector<ll>& y) {      // all x_i must be DISTINCT
    int n = x.size();
    if (!n) return {};
    ptree.assign(4 * n, {});
    buildTree(1, 0, n - 1, x);
    Poly root = ptree[1];
    vector<ll> out(n);
    evalRec(1, 0, n - 1, deriv(root), out);
    // out[i] = M'(x_i) != 0 since x_i distinct
    vector<ll> d(n);
    for (int i = 0; i < n; i++) d[i] = y[i] % NMOD * npow(out[i], NMOD - 2) % NMOD;
    return cut(interpRec(1, 0, n - 1, d), n);
}
// COMPLEXITY: each level of the tree does O(n) total polynomial work with logarithmic factors, so
// n log^2 n; the constant is big - expect ~1s at n = 1e5, and prefer the specialised routines.
// PRECONDITIONS: NMOD prime; the x_i must be pairwise distinct for interpolate (duplicates make
// M'(x_i) = 0 and the inverse blows up).
// deriv/divmod come from 04 - Polynomial.cpp.
// APPLICATIONS BEYOND THE OBVIOUS
//  Composing "evaluate then re-interpolate" implements ANY linear map given by its action on
//  points, e.g. shifting a sampled polynomial by a constant in O(n log^2 n) (or O(n log n) with
//  the Taylor shift in 05 - Convolution/10).
//  The product tree alone is worth having: prod (x - x_i) in O(n log^2 n) is how you build the
//  characteristic polynomial of a set of roots, the elementary symmetric polynomials of n values,
//  and "the number of ways to pick k of these items" for every k at once.
```

6.6.5 InterpolationCoefficients

GETTING THE COEFFICIENTS, not just one value. 01 - LagrangeAndFaulhaber.cpp evaluates an

```
// Needs: Mint (01 - Modular Arithmetic/01 - Mint.cpp); binomialBasis needs 02 - NTT.cpp instead.
// interpolating polynomial at a point; sometimes the problem wants the polynomial itself - to
// integrate it, to compose it, to read off a single coefficient, or to answer queries at points
// that are not known in advance.

// ARBITRARY sample points, explicit coefficients, O(n^2). x_i must be pairwise DISTINCT.
// Build M(x) = prod (x - x_i) once, then divide out each (x - x_i) by synthetic division.
vector<Mint> interpCoeffs(const vector<Mint>& x, const vector<Mint>& y) {
    int n = x.size();
    vector<Mint> M(n + 1, 0), r(n, 0);
    M[0] = 1;
    for (int i = 0; i < n; i++)
        // M *= (x - x_i)
        for (int j = n; j >= 0; j--) { M[j] *= Mint(0) - x[i]; if (j) M[j] += M[j - 1]; }
    for (int i = 0; i < n; i++) {
        vector<Mint> q(n);
        // q = M / (x - x_i), synthetic division
        Mint rem = M[n];
        for (int j = n - 1; j >= 0; j--) q[j] = rem, rem = M[j] + rem * x[i];
        Mint den = 1;
        for (int j = 0; j < n; j++) if (j != i) den *= x[i] - x[j];
        Mint sc = y[i] / den;
        for (int j = 0; j < n; j++) r[j] += q[j] * sc;
    }
    return r;
    // r[j] = coefficient of x^j
}
// THE INVERSE VANDERMONDE MATRIX is exactly this: V[i][j] = x_i^j, and (V^-1)[j][i] is the
// coefficient of x^j in the i-th Lagrange basis polynomial - the vector interpCoeffs produces when
// y = e_i. So "solve V a = y" IS interpolation, in O(n^2) instead of O(n^3), and
//   det V = prod_{i < j} (x_j - x_i).
// If the first column starts at x_i instead of 1 (i.e. V[i][j] = x_i^(j+1)), divide row i by x_i.

// BINOMIAL (Newton forward-difference) BASIS, from CONSECUTIVE samples p(0..n), in O(n log n):
//   p(x) = sum_i a_i C(x, i),  a_i = i-th finite difference at 0 = sum_j (-1)^(i-j) C(i,j) p(j)
// which is one convolution of p[j]/j! with (-1)^j/j!. The a_i are integers whenever p maps
// integers to integers, so this is THE basis for "p has integer values" arguments, and evaluating
// p at a HUGE x is then sum a_i * C(x, i) with C(x, i) built by a running product - O(n) and no
// division by anything that could vanish.
// Written mod NMOD on top of multiply() from 02 - NTT.cpp (Poly = vector<ll>); if your modulus is
// not NTT-friendly, run the same two lines with an O(n^2) convolution or with convMod.
vector<ll> binomialBasis(const vector<ll>& p) {
    // p[j] = value at j, j = 0..n
    int n = p.size();
    vector<ll> f(n), fi(n), u(n), v(n);
    f[0] = 1;
    for (int i = 1; i < n; i++) f[i] = f[i - 1] * i % NMOD;
    fi[n - 1] = npow(f[n - 1], NMOD - 2);
    for (int i = n - 1; i > 0; i--) fi[i - 1] = fi[i] * i % NMOD;
    for (int i = 0; i < n; i++) {
        u[i] = p[i] % NMOD * fi[i] % NMOD;
        v[i] = (i & 1) ? (NMOD - fi[i]) % NMOD : fi[i];
    }
    vector<ll> w = multiply(u, v);
    vector<ll> a(n);
    for (int i = 0; i < n; i++) a[i] = w[i] * f[i] % NMOD;
    return a;
}
// FALLING FACTORIAL basis x^(i) = x(x-1)...(x-i+1) is the same thing scaled: a_i * i! are its
// coefficients. Converting between the monomial and falling-factorial bases is exactly the
// STIRLING transform (Stirling numbers of the first/second kind are the two change-of-basis
// matrices), which is how "sum of i^k" becomes "sum of C(i, j)" and telescopes.
// WHICH ROUTINE WHEN
//   one value, consecutive samples .....
```

```
  ↪ lagrangeConsecutive, O(n)
//   one value, arbitrary samples ......... lagrange,
  ↪ O(n^2)
//   coefficients, arbitrary samples ......
  ↪ interpCoeffs, O(n^2)  (or 04 - MultipointEval)
//   coefficients in the binomial basis ...
  ↪ binomialBasis, O(n log n)
```

6.6.6 PRecursive

LINEAR RECURRENCE WITH POLYNOMIAL COEFFICIENTS (a "P-recursive" or holonomic sequence):

```
// Needs: Mint (01 - Modular Arithmetic/01 -
  ↪ Mint.cpp). MOD must be PRIME.
//   sum_{i=0..order} P_i(m - i) * a[m - i] = 0   for
  ↪ all large m,   deg P_i <= deg.
// This is the strictly stronger sibling of
  ↪ Berlekamp-Massey. BM only finds
  ↪ CONSTANT-coefficient
// recurrences, so it fails on n!, C(2n, n), Motzkin,
  ↪ derangements, Hertzsprung, and essentially
// every sequence built from binomials or factorials.
  ↪ Those are all P-recursive.
// REACH FOR IT in the classic contest workflow:
  ↪ brute-force the first ~60-100 terms, guess the
// recurrence here, then extend it to n = 1e7 in O(n
  ↪ * order * deg). Also tells you the answer IS
// holonomic, which is itself a strong hint about the
  ↪ intended solution.
// COST: guessing is O(n^3 / deg^2)-ish Gaussian
  ↪ elimination on an R x C matrix; extending is
// linear. You must GUESS deg (try 0, 1, 2, ... until
  ↪ it succeeds) - order comes out automatically.
// You need roughly (deg + 2) * (order + 1) terms;
  ↪ give it more and the verification is meaningful.

// coeffs[i][d] = coefficient of m^d in P_i. Returns
  ↪ {} if no such recurrence fits the data.
vector<vector<Mint>> findPRec(const vector<Mint>& t,
  ↪ int deg) {
    int n = t.size(), B = (n + 2) / (deg + 2), C = B
  ↪ * (deg + 1), R = n - (B - 1);
    if (B < 2 || R < C - 1) return {};
    vector<vector<Mint>> mat(R, vector<Mint>(C));
    for (int y = 0; y < R; y++)
        for (int b = 0; b < B; b++) {
            Mint v = t[y + b];
            for (int d = 0; d <= deg; d++) mat[y][b *
  ↪ (deg + 1) + d] = v, v *= Mint(y + b);
        }
    int rank = 0;
    for (int x = 0; x < C && rank < R; x++) {
  ↪             // RREF
        int piv = -1;
        for (int y = rank; y < R; y++) if
  ↪ (mat[y][x].v) { piv = y; break; }
        if (piv < 0) continue;
        swap(mat[rank], mat[piv]);
        Mint iv = Mint(1) / mat[rank][x];
        for (int j = x; j < C; j++) mat[rank][j] *=
  ↪ iv;
        for (int y = 0; y < R; y++) if (y != rank &&
  ↪ mat[y][x].v) {
            Mint c = mat[y][x];
            for (int j = x; j < C; j++) mat[y][j] -=
  ↪ c * mat[rank][j];
        }
        rank++;
    }
    if (rank == C) return {};
  ↪             // no non-trivial kernel vector
    // column 'rank' is free: set it to 1 and read
  ↪ the pivot columns off (the matrix is in RREF)
    int order = rank / (deg + 1);
    vector<vector<Mint>> r(order + 1,
  ↪ vector<Mint>(deg + 1, 0));
    r[0][rank % (deg + 1)] = 1;
    for (int y = rank - 1; y >= 0; y--)
        r[order - y / (deg + 1)][y % (deg + 1)] =
  ↪ Mint(0) - mat[y][rank];
    return r;
}
// Extend the sequence to index n. 'terms' must hold
  ↪ at least 'order' correct initial values.
// PRECONDITION: the leading polynomial P_0(m) must
  ↪ be non-zero mod MOD at every m you generate -
// if it vanishes the recurrence genuinely does not
  ↪ determine a[m] (a "singularity" of the
// recurrence); shift the start or pick a different
  ↪ MOD.
vector<Mint> extendPRec(int n, const
  ↪ vector<vector<Mint>>& c, vector<Mint> terms) {
    int order = c.size() - 1, deg = c[0].size() - 1;
    vector<Mint> a = terms;
    a.resize(max<int>(n + 1, terms.size()), 0);
    for (int m = terms.size(); m <= n; m++) {
        Mint s = 0;
        for (int i = 1; i <= order; i++) {
            Mint p = 1, k = Mint(m - i);
            for (int d = 0; d <= deg; d++) s += a[m -
  ↪ i] * c[i][d] * p, p *= k;
        }
        Mint den = 0, p = 1;
        for (int d = 0; d <= deg; d++) den += c[0][d]
  ↪ * p, p *= Mint(m);
        a[m] = (Mint(0) - s) / den;
    }
    return a;
}
// EXAMPLES that come out immediately (deg, order):
//   n!  -> (1, 1):  a[m] - m * a[m-1] = 0.
  ↪ Catalan -> (1, 1).      Motzkin -> (1, 2).
//   derangements -> (1, 2).  C(2n, n) -> (1, 1).
  ↪ Hertzsprung -> (1, 4).  sum C(n,k)^3 -> (2, 2).
// ALWAYS SANITY-CHECK: re-run extendPRec on the
  ↪ initial terms and compare against the input; a
// recurrence that fits by accident is the main
  ↪ failure mode when you feed too few terms.
// THEORY: holonomic sequences are closed under +,
  ↪ Cauchy product, and Hadamard product, and the
// generating function satisfies a linear ODE with
  ↪ polynomial coefficients. That is why "sum of
// products of binomials" is essentially always
  ↪ P-recursive (Zeilberger's algorithm proves it).
// If you also need the k-th term for k = 1e18,
  ↪ P-recursive sequences admit an O(sqrt(k) log k)
// evaluation by the factorial-style
  ↪ baby-step/giant-step product of matrices - far
  ↪ more code.
```

6.6.7 ExpPolynomialSum

SUM OF A GEOMETRIC TIMES A POLYNOMIAL: S(n) = sum_{i=0}^{n} a^i * f(i) for n up to 1e18,

```
// Needs: Mint + buildFact (01 - Mint.cpp),
   ↪ lagrangeConsecutive (06 - Interpolation/01).
// where f is a polynomial of degree <= k given by
   ↪ its values f(0), f(1), ..., f(k). O(k log MOD).
// REACH FOR IT for "sum i^k r^i", weighted prefix
   ↪ sums in a linear recurrence, expected values
// where each step is discounted by a factor a, and
   ↪ anything of the shape sum a^i * (polynomial).
// WHY IT WORKS: for a != 1 the answer has the closed
   ↪ form S(n) = a^n * g(n) + c with g a
// polynomial of degree exactly k and c a constant.
   ↪ Apply the operator (D h)(n) = h(n+1) - a*h(n):
// it kills a^n * g one degree at a time, so D^(k+1)
   ↪ annihilates the a^n g(n) part completely and
// turns the constant into c * (1 - a)^(k+1). That
   ↪ single equation gives c; then g is sampled and
// Lagrange-interpolated. For a == 1 the sum is just
   ↪ a polynomial of degree k+1 in n.
Mint expPolySum(vector<Mint> y, Mint a, ll n) {
  ↪    // y[i] = f(i) for i = 0..k
    int k = y.size() - 1;
    if (n < 0) return 0;
    y.push_back(lagrangeConsecutive(y, k + 1));
  ↪    // need f(k+1) too, so k+2 samples of S
    vector<Mint> S(k + 2);
    Mint p = 1;
    for (int i = 0; i <= k + 1; i++) {
        S[i] = (i ? S[i - 1] : Mint(0)) + p * y[i];
        p *= a;
    }
    if (a == Mint(1)) return lagrangeConsecutive(S,
  ↪ n);  // S is a polynomial of degree k+1
    Mint c = 0;
  ↪    // c * (1-a)^(k+1) = D^(k+1) S at 0
    for (int i = 0; i <= k + 1; i++) {
        Mint t = C(k + 1, i) * (Mint(0) - a).pow(k +
  ↪ 1 - i) * S[i];
        c += t;
    }
    c /= (Mint(1) - a).pow(k + 1);
    vector<Mint> g(k + 1);
  ↪    // g(i) = (S(i) - c) / a^i
    Mint ia = Mint(1) / a, q = 1;
    for (int i = 0; i <= k; i++) g[i] = (S[i] - c) *
  ↪ q, q *= ia;
    return lagrangeConsecutive(g, n) * a.pow(n) + c;
}
// SPECIAL CASES WORTH KNOWING
//   f == 1:      sum_{i<=n} a^i = (a^(n+1) - 1) / (a
  ↪ - 1), and n+1 when a == 1. Do NOT route a
//                trivial geometric series through
  ↪ this - and watch a == 1 mod MOD, which happens
//                for a = MOD + 1 style inputs and
  ↪ makes the closed form divide by zero.
//   a == 1:      pure Faulhaber, powerSum in 06 -
  ↪ Interpolation/01.
//   INFINITE sum with |a| < 1 formally: sum_{i>=0}
  ↪ a^i f(i) = (applied k+1 times) - or read it off
//                the rational generating function,
  ↪ see 03 - BostanMori.cpp.
// PRECONDITION: MOD prime and larger than k + 2 (we
  ↪ invert factorials and (1 - a)^(k+1)); a != 1
// for the second branch, which the code checks. n
  ↪ may be as large as you like: a.pow(n) and
// lagrangeConsecutive both take a ll argument.
// SAMPLING f: if you are handed the COEFFICIENTS of
  ↪ f instead of its values, evaluate it at
// 0..k+1 by Horner in O(k^2) or by one multipoint
  ↪ evaluation.
```

6.6.8 PowerSumsAtOnce

ALL POWER SUMS AT ONCE: s[i] = sum_{p=0}^{t} p^i for EVERY i = 0..k, in O(k log k).

```
// Needs: multiply / npow (02 - NTT.cpp), Poly / inv
   ↪ / cut (04 - Polynomial.cpp).
// (Convention 0^0 = 1, so s[0] = t + 1.) powerSum in
   ↪ 06 - Interpolation/01 gives one exponent in
// O(k); this gives all k+1 of them for the same
   ↪ asymptotic cost, which is what you need when the
// answer is "for each k, output sum i^k" or when a
   ↪ Newton-identity / symmetric-function step needs
// the whole vector of power sums.
// EGF IDENTITY:  sum_i s[i] x^i / i! = sum_{p=0}^{t}
   ↪ e^(p x) = (e^(n x) - 1) / (e^x - 1)
//                                   = [(e^(n x) -
   ↪ 1) / x] * [x / (e^x - 1)],   n = t + 1.
// The right factor is the BERNOULLI generating
   ↪ function, B[i] = i! * [x^i] x/(e^x - 1), and
// x/(e^x - 1) = 1 / sum_{i>=0} x^i / (i+1)! - one
   ↪ power-series inverse, O(k log k). That beats the
// O(k^2) Bernoulli recurrence in 05 -
   ↪ Combinatorics/11.
Poly bernoulliEGF(int m) {                          //
   ↪ returns B[i] / i! for i = 0..m-1
    Poly d(m);
    d[0] = 1;
    for (int i = 1; i < m; i++) d[i] = d[i - 1] *
   ↪ npow(i + 1, NMOD - 2) % NMOD;   // 1 / (i+1)!
    return inv(d, m);
}
Poly powerSums(ll t, int k) {                       //
   ↪ s[i] = sum_{p=0}^{t} p^i, i = 0..k
    Poly B = bernoulliEGF(k + 2), P(k + 2, 0);
    ll n = (t + 1) % NMOD, cur = 1;
    for (int i = 1; i <= k + 1; i++) {
   ↪              // P[i] = n^i / i!
        cur = cur * n % NMOD * npow(i, NMOD - 2) %
   ↪ NMOD;
        P[i] = cur;
    }
    Poly Q = cut(multiply(P, B), k + 2);
    Poly s(k + 1);
    ll f = 1;
    for (int i = 0; i <= k; i++) { s[i] = Q[i + 1] *
   ↪ f % NMOD; f = f * (i + 1) % NMOD; }
    return s;
}
// RELATED ONE-LINERS BUILT FROM THE SAME PIECES
//  SUM OVER A LIST rather than an interval: sum_{j}
   ↪ a_j^i for all i = 0..k in O((n + k) log^2 n) -
//    build the rational function sum_j 1 / (1 - a_j
   ↪ x) by divide and conquer over the list (keep
//    numerator and denominator, merge with up = A*D
   ↪ + B*C, down = B*D), then expand it mod x^(k+1).
//    Those coefficients ARE the power sums; feed
   ↪ them to Newton's identities (05 - Comb/10)
//    to get the elementary symmetric polynomials.
//  sum_{p=0}^{t} p^i * r^p for all i: same shape,
   ↪ use 07 - ExpPolynomialSum.cpp per i, or convolve
//    with the EGF of the geometric series.
//  The Faulhaber POLYNOMIAL in n (coefficients
   ↪ rather than a value) is faulhaberPoly in
//    05 - Combinatorics/11 - BernoulliFaulhaber.cpp.
```

```
// PRECONDITION: NMOD prime and > k + 2 (we invert
   ↪ 1..k+1 and take a series inverse whose constant
// term is 1). t may be huge - only t + 1 mod NMOD
   ↪ enters.
```

6.6.9 ReedsSloane

REEDS-SLOANE: the shortest linear recurrence fitting a sequence MODULO A COMPOSITE m.

```
// Needs: modInverse / extGCD (04 - Number
   ↪ Theory/01), CRT (04 - Number Theory/03/01 -
   ↪ CRT.cpp).
// Berlekamp-Massey (06 - Interpolation/02) divides
   ↪ by the discrepancy, so it needs a FIELD and
// silently produces garbage whenever a leading
   ↪ coefficient is not invertible mod m. This is the
// replacement: it factors m into prime powers, runs
   ↪ a shift-register synthesis over each Z/p^e
// (tracking, for every 2-adic-style valuation g, the
   ↪ best approximation whose discrepancy has that
// valuation), and CRTs the coefficient vectors back
   ↪ together.
// REACH FOR IT when the problem's modulus is 1e9,
   ↪ 2^32, 10^k or any other composite and you still
// want the "brute force a few terms, guess the
   ↪ recurrence, extrapolate" attack. O(n^2 log m).
// OUTPUT: A[0..L] with A[0] = 1 and sum_j A[j] *
   ↪ s[i-j] = 0 mod m, i.e.
//   s[i] = -(A[1] s[i-1] + ... + A[L] s[i-L])  mod
   ↪ m.   Feed 2L or more terms.
namespace RS {
int deg(const vector<ll>& a) {                    //
   ↪ -inf encoded as a large negative number
    return (a.size() > 1 || (a.size() == 1 && a[0]))
   ↪ ? (int)a.size() - 1 : -40000;
}
int L(const vector<ll>& a, const vector<ll>& b) {
   ↪ return max(deg(a), deg(b) + 1); }
void ext(vector<ll>& a, size_t d) { if (d > a.size())
   ↪ a.resize(d, 0); }

// shift-register synthesis modulo p^e (m = p^e);
   ↪ returns the connection polynomial
vector<ll> primePower(const vector<ll>& s, ll m, ll
   ↪ p, int e) {
    vector<vector<ll>> a(e), b(e), an(e), bn(e),
   ↪ ao(e), bo(e);
    vector<ll> t(e), u(e), r(e), to(e, 1), uo(e),
   ↪ pw(e + 1, 1);
    for (int i = 1; i <= e; i++) pw[i] = pw[i - 1] *
   ↪ p;
    for (int i = 0; i < e; i++) {
        a[i] = an[i] = {pw[i]};
        b[i] = {0}, bn[i] = {s[0] * pw[i] % m};
        t[i] = s[0] * pw[i] % m;
        if (!t[i]) t[i] = 1, u[i] = e;
        else for (u[i] = 0; t[i] % p == 0; t[i] /= p)
   ↪ u[i]++;
    }
    for (size_t k = 1; k < s.size(); k++) {
        for (int g = 0; g < e; g++) if (L(an[g],
   ↪ bn[g]) > L(a[g], b[g])) {
            int h = e - 1 - u[g];
            ao[g] = a[h], bo[g] = b[h], to[g] = t[h],
   ↪ uo[g] = u[h], r[g] = k - 1;
        }
        a = an, b = bn;
        for (int o = 0; o < e; o++) {
            ll d = 0;
            for (size_t i = 0; i < a[o].size() && i
   ↪ <= k; i++) d = (d + a[o][i] * s[k - i]) % m;
            if (!d) { t[o] = 1, u[o] = e; continue; }
            for (u[o] = 0, t[o] = d; t[o] % p == 0;
   ↪ t[o] /= p) u[o]++;
            int g = e - 1 - u[o];
            if (L(a[g], b[g]) == 0) { ext(bn[o], k +
   ↪ 1); bn[o][k] = (bn[o][k] + d) % m; continue; }
            ll c = t[o] % m * modInverse(to[g] % m,
   ↪ m) % m * pw[u[o] - uo[g]] % m;
            int sh = k - r[g];
            ext(an[o], ao[g].size() + sh), ext(bn[o],
   ↪ bo[g].size() + sh);
            for (size_t i = 0; i < ao[g].size(); i++)
                an[o][i + sh] = (an[o][i + sh] - c *
   ↪ ao[g][i]) % m;
            for (size_t i = 0; i < bo[g].size(); i++)
                bn[o][i + sh] = (bn[o][i + sh] - c *
   ↪ bo[g][i]) % m;
            for (ll& x : an[o]) x = (x % m + m) % m;
            for (ll& x : bn[o]) x = (x % m + m) % m;
            while (an[o].size() > 1 && !an[o].back())
   ↪ an[o].pop_back();
            while (bn[o].size() > 1 && !bn[o].back())
   ↪ bn[o].pop_back();
        }
    }
    return an[0];
}
}
vector<ll> reedsSloane(vector<ll> s, ll m) {
    vector<array<ll, 3>> fac;                     //
   ↪ {p^e, p, e}
    ll t = m;
    for (ll i = 2; i * i <= t; i++) if (t % i == 0) {
        ll c = 0, q = 1;
        while (t % i == 0) t /= i, c++, q *= i;
        fac.push_back({q, i, c});
    }
    if (t > 1) fac.push_back({t, t, 1});
    vector<vector<ll>> per;
    size_t n = 0;
    for (auto& f : fac) {
        vector<ll> ss = s;
        for (ll& x : ss) x = (x % f[0] + f[0]) % f[0];
        per.push_back(RS::primePower(ss, f[0], f[1],
   ↪ (int)f[2]));
        n = max(n, per.back().size());
    }
    vector<ll> res(n);
    for (size_t i = 0; i < n; i++) {
        vector<ll> c(fac.size()), md(fac.size());
        for (size_t j = 0; j < fac.size(); j++)
            md[j] = fac[j][0], c[j] = i <
   ↪ per[j].size() ? per[j][i] : 0;
        res[i] = CRT(c, md).first;
    }
    return res;                                   //
   ↪ res[0] == 1; s[i] = -sum_{j>=1} res[j]*s[i-j]
}
// AFTER YOU HAVE THE RECURRENCE: extending it is
   ↪ trivial (the leading coefficient is 1, so no
// inversion is ever needed) - just iterate s[i] =
   ↪ -(sum_{j>=1} A[j] s[i-j]) mod m.
// For the k-th term with k = 1e18 mod a COMPOSITE,
   ↪ do Kitamasa-style polynomial exponentiation
// mod the connection polynomial: it only multiplies,
   ↪ never divides, so a composite modulus is fine.
```

```
// CHEAPER FIRST TRIES, in order:
//  - if m is prime, use Berlekamp-Massey; this file is strictly heavier.
//  - if m is squarefree, run BM under each prime factor separately and CRT. That is 10 lines and
//    covers m = 1e9+6-style inputs; the prime-POWER case is the only one that really needs this.
//  - if the sequence is not constant-coefficient at all (factorials, binomials), you want
//    06 - PRecursive.cpp instead, and it also works over composite moduli when the leading
//    polynomial happens to be invertible.
// PRECONDITION: m >= 1 and the factorisation loop is trial division, so m up to ~1e18 needs
// Pollard rho (04 - Number Theory/02/06) instead of the loop above.
```

6.7 Numerical

6.7.1 Integration

ADAPTIVE SIMPSON - numeric integral of a black-box f on [a,b] to a requested absolute error.

```
// Simpson approximates f by a parabola through the two ends and the midpoint; "adaptive" means it
// splits an interval only where the parabola does not yet agree with the two half-parabolas.
// Reach for it when the answer is an integral you cannot do in closed form: area of a union of
// circles/ellipses cut by a line, area swept by a moving shape, probability over a continuous
// parameter, arc lengths. Cost is proportional to how wiggly f is, not to (b-a).
template <class F> ld simpson(F f, ld a, ld b) {
    return (b - a) * (f(a) + 4 * f((a + b) / 2) + f(b)) / 6;
}
template <class F> ld asr(F f, ld a, ld b, ld eps, ld S, int dep) {
    ld c = (a + b) / 2, L = simpson(f, a, c), R = simpson(f, c, b);
    if (dep <= 0 || fabsl(L + R - S) <= 15 * eps) return L + R + (L + R - S) / 15;
    return asr(f, a, c, eps / 2, L, dep - 1) + asr(f, c, b, eps / 2, R, dep - 1);
}
template <class F> ld integrate(F f, ld a, ld b, ld eps = 1e-9) {
    return asr(f, a, b, eps, simpson(f, a, b), 50);
}
// TRAPS. (1) If f has a KINK or a jump, split the interval at that point by hand and integrate the
// pieces - adaptive Simpson will burn its depth budget trying to resolve it. (2) The (L+R-S)/15
// term is Richardson extrapolation; keeping it makes the result much more accurate for free.
// (3) A too-loose eps on a nearly flat region can stop early and miss a spike: if f can be zero on
// most of the range and huge somewhere, integrate the pieces separately.
//
// OTHER NUMERICAL TOOLS WORTH THE PAGE:
// TERNARY SEARCH on a strictly unimodal f over reals: 100 iterations of
//   m1 = l + (r-l)/3, m2 = r - (r-l)/3;  f(m1) < f(m2) ? r = m2 : l = m1;   (minimising)
//   fixes the answer to ~1e-30 relative. On INTEGERS use binary search on the difference
//   f(m+1) - f(m) instead - the three-way split has off-by-one traps at the last two points.
// NEWTON'S METHOD for a root of g: x -= g(x) / g'(x). Doubles the correct digits per step but
//   needs a good start; bisect first if the sign changes, then Newton.
// BISECTION on a monotone predicate: 100 iterations of (l+r)/2 is always safe and never diverges.
// GOLDEN-SECTION SEARCH: like ternary search but one function evaluation per step instead of two -
//   worth it only when f is expensive.
// SOLVING f(x) = 0 WHERE f IS A POLYNOMIAL: find all real roots by recursing on the derivative
//   (roots of f' split the line into monotone pieces; bisect inside each).
// SIMULATED ANNEALING / HILL CLIMBING when the objective is not unimodal: start from several
//   random points, take a step of size T, accept worse solutions with probability exp(-dE/T),
//   cool T geometrically (T *= 0.995). Good for geometric-median / facility-location style
//   problems where an exact method is out of reach; always seed the RNG from the clock.
// WEISZFELD for the geometric median (minimise sum of Euclidean distances): iterate
//   x <- (sum p_i / |x - p_i|) / (sum 1 / |x - p_i|), guarding the case x == p_i. Converges
//   linearly; nested ternary search is the safer contest answer (the objective is convex).
```

6.7.2 LinearSolvers

SPECIAL-SHAPE LINEAR SOLVERS. Gaussian elimination is O(n^3); these are the shapes where the

```
// structure buys you a factor of n or removes a precondition.

// --- THOMAS ALGORITHM: TRIDIAGONAL system in O(n). ---
//   a[i] x[i-1] + b[i] x[i] + c[i] x[i+1] = d[i], with a[0] = c[n-1] = 0.
// This is THE closer for random walks on a path ("expected steps to escape [0, n]"), 1-D heat /
// diffusion DPs, and any DP whose state depends on its two neighbours. Gaussian elimination on
// the same system is O(n^3) and will TLE at n = 1e5; this is 8 lines.
// STABLE without pivoting when the system is diagonally dominant (|b| > |a| + |c|), which every
// hitting-time system is.
vector<ld> thomas(vector<ld> a, vector<ld> b, vector<ld> c, vector<ld> d) {
    int n = b.size();
    for (int i = 1; i < n; i++) {
        ld m = a[i] / b[i - 1];
        b[i] -= m * c[i - 1], d[i] -= m * d[i - 1];
    }
    vector<ld> x(n);
    x[n - 1] = d[n - 1] / b[n - 1];
    for (int i = n - 2; i >= 0; i--) x[i] = (d[i] - c[i] * x[i + 1]) / b[i];
    return x;
}
// BANDED systems of half-width w (state depends on w neighbours) generalise this at O(n w^2) -
// the shape of "robot on a grid with m columns", where w = m.
// CYCLIC tridiagonal (x[0] and x[n-1] also coupled): Sherman-Morrison - solve two tridiagonal
```

```
// systems and combine, still O(n).

// --- DETERMINANT MODULO AN ARBITRARY m (composite allowed), O(n^3 log m). ---
// Gaussian elimination inverts the pivot, so it needs a PRIME modulus. This version never
// divides: it runs the Euclidean algorithm between two rows, swapping them, until the lower
// entry is 0. Use it when the modulus is 2^k, 1e9, or anything you cannot invert in.
ll detMod(vector<vector<ll>> a, ll m) {
    int n = a.size();
    ll det = 1 % m;
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++)
            while (a[j][i]) {
                // Euclid on the two rows
                ll t = a[i][i] / a[j][i];
                for (int k = i; k < n; k++) a[i][k] = (a[i][k] - t * a[j][k]) % m;
                swap(a[i], a[j]), det = -det;
            }
        det = det * a[i][i] % m;
        if (!a[i][i]) return 0;
    }
    return (det % m + m) % m;
}
// MATRIX RANK over a field: the usual elimination, counting non-zero pivot rows.
// RANK over GF(2): use bitsets - O(n^3 / 64), see 04 - Linear Algebra/03.
// INTEGER determinant with no modulus: compute it modulo several primes and CRT. Size it with
// HADAMARD'S BOUND |det| <= prod of the row 2-norms <= n^(n/2) * max|a_ij|^n, and take enough
// primes that the product exceeds 2x that.
// SPARSE determinant (n = 1e5 with few non-zeros): Wiedemann - pick random u, v, feed the Krylov
// sequence u^T A^k v to Berlekamp-Massey to get the minimal polynomial, whose constant term is
// +-det after a random diagonal preconditioner. O(n * nnz).
// CHARACTERISTIC POLYNOMIAL in O(n^3): reduce to upper HESSENBERG form by similarity transforms
// (so det(xI - H) satisfies a short recurrence in the leading principal minors), then read the
// coefficients off. With Cayley-Hamilton that gives A^k in O(n^3 + n^2 log k) instead of
// O(n^3 log k) - worth it only for large k and small n.
```

6.7.3 Romberg

ROMBERG INTEGRATION - the other numerical quadrature. Repeatedly halve the step (trapezoid rule)

```
// and RICHARDSON-EXTRAPOLATE the error away: T(h) has an error expansion in h^2 only, so combining
// two step sizes cancels the h^2 term, three cancel h^4, and so on. Row k of the table is accurate
// to O(h^(2k)). Cost: 2^k function evaluations, each sample reused across the whole table.
// REACH FOR IT over adaptive Simpson (07 - Numerical/01) when the integrand is SMOOTH and cheap
// and you want many correct digits: Romberg reaches 1e-12 in ~10 rows where Simpson needs a deep
// adaptive recursion. Use adaptive Simpson instead when the function has kinks, spikes, or a
// singular endpoint - Romberg assumes the error expansion exists and quietly lies when it does not.
template <class F> ld romberg(F f, ld a, ld b, ld tol = 1e-10, int maxIter = 25) {
    vector<ld> t;
    ld h = b - a;
    t.push_back(h * (f(a) + f(b)) / 2);    // T(h), one trapezoid
    for (int k = 1, m = 1; k <= maxIter; k++, m <<= 1) {
        ld s = 0, x = a + h / 2;
        for (int j = 0; j < m; j++) s += f(x), x += h;   // only the NEW midpoints
        h /= 2;
        ld cur = t[0] / 2 + h * s;
        // refined trapezoid, previous row's T
        ld prev = t.back();
        for (int j = 0; j < k; j++) {
            // Richardson: t[j] holds the old row
            ld nxt = (powl(4, j + 1) * cur - t[j]) / (powl(4, j + 1) - 1);
            t[j] = cur, cur = nxt;
        }
        t.push_back(cur);
        if (k > 3 && fabsl(prev - cur) < tol) break;
    }
    return t.back();
}
// THE TABLE: R[k][0] = trapezoid with 2^k intervals, R[k][j] = (4^j R[k][j-1] - R[k-1][j-1])
// / (4^j - 1). R[k][1] is exactly Simpson's rule; R[k][2] is Boole's. The code above keeps only
// one row, which is all you need.
// PRACTICAL NOTES
//  - convergence check on |R[k][k] - R[k-1][k-1]| is a heuristic; require a few rows first (the
//    k > 3 guard) or a flat integrand will make it stop after one step.
//  - INFINITE range: substitute x = t/(1-t) or integrate to a large cut-off and bound the tail.
//  - ENDPOINT SINGULARITY (1/sqrt(x) at 0): substitute it away, or use the tanh-sinh (double
//    exponential) rule, which handles endpoint singularities and converges even faster.
//  - OSCILLATORY integrands: split at the zeros of the oscillating factor and sum, or Romberg will
//    happily converge to nonsense.
//  - the same Richardson idea accelerates ANY sequence with a known error expansion, e.g.
//    numerical derivatives (central differences) and extrapolating a DP in a step size.
```

6.8 Linear Programming

6.8.1 Simplex

TWO-PHASE SIMPLEX. maximise c.x subject to A x <= b, x >= 0.

```
// Returns the optimum, or +inf if unbounded, or -inf if infeasible; x is filled with a solution.
// Exponential in theory, O(n*m) per pivot and instant at contest sizes (n, m up to a few hundred).
// REACH FOR IT when the problem is "optimise a linear objective under linear constraints" and no
// combinatorial structure presents itself - resource allocation, blending, fractional covering,
// zero-sum matrix games, and any min-cost-flow-with-side-constraints that flow
```

```
  ↪ cannot express.
typedef vector<ld> vd;
struct Simplex {
    int m, n;
    vector<int> B, N;
    vector<vd> D;
    Simplex(const vector<vd>& A, const vd& b, const
  ↪ vd& c)
        : m(b.size()), n(c.size()), B(m), N(n + 1),
  ↪ D(m + 2, vd(n + 2, 0)) {
        for (int i = 0; i < m; i++) for (int j = 0; j
  ↪ < n; j++) D[i][j] = A[i][j];
        for (int i = 0; i < m; i++) B[i] = n + i,
  ↪ D[i][n] = -1, D[i][n + 1] = b[i];
        for (int j = 0; j < n; j++) N[j] = j, D[m][j]
  ↪ = -c[j];
        N[n] = -1, D[m + 1][n] = 1;
    }
    void pivot(int r, int s) {
        ld inv = 1 / D[r][s];
        for (int i = 0; i < m + 2; i++) if (i != r &&
  ↪ fabsl(D[i][s]) > 1e-9) {
            ld f = D[i][s] * inv;
  ↪           // the pivot COLUMN is fixed up by
            for (int j = 0; j < n + 2; j++) if (j !=
  ↪ s) D[i][j] -= f * D[r][j];   // the loop below
        }
        for (int j = 0; j < n + 2; j++) if (j != s)
  ↪ D[r][j] *= inv;
        for (int i = 0; i < m + 2; i++) if (i != r)
  ↪ D[i][s] *= -inv;
        D[r][s] = inv;
        swap(B[r], N[s]);
    }
    bool simplex(int phase) {
        int x = m + phase - 1;
        for (;;) {
            int s = -1;
            for (int j = 0; j <= n; j++)
                if (N[j] != -phase && (s < 0 ||
  ↪ make_pair(D[x][j], N[j]) < make_pair(D[x][s],
  ↪ N[s]))) s = j;
            if (D[x][s] >= -1e-9) return true;
            int r = -1;
            for (int i = 0; i < m; i++) {
                if (D[i][s] <= 1e-9) continue;
                if (r < 0 || make_pair(D[i][n + 1] /
  ↪ D[i][s], B[i]) < make_pair(D[r][n + 1] /
  ↪ D[r][s], B[r])) r = i;
            }
            if (r < 0) return false;
  ↪           // unbounded
            pivot(r, s);
        }
    }
    ld solve(vd& x) {
        int r = 0;
        for (int i = 1; i < m; i++) if (D[i][n + 1] <
  ↪ D[r][n + 1]) r = i;
        if (D[r][n + 1] < -1e-9) {
  ↪           // phase 1: find any feasible point
            pivot(r, n);
            if (!simplex(2) || D[m + 1][n + 1] <
  ↪ -1e-9) return -1.0L / 0.0L;   // infeasible
            for (int i = 0; i < m; i++) if (B[i] ==
  ↪ -1) {
                int s = 0;
                for (int j = 1; j <= n; j++)
                    if (make_pair(D[i][j], N[j]) <
  ↪ make_pair(D[i][s], N[s])) s = j;
                pivot(i, s);
            }
        }
        bool ok = simplex(1);
        x.assign(n, 0);
        for (int i = 0; i < m; i++) if (B[i] < n)
  ↪ x[B[i]] = D[i][n + 1];
        return ok ? D[m][n + 1] : 1.0L / 0.0L;
  ↪           // +inf = unbounded
    }
};
/* MODELLING NOTES - the part that actually costs you
  ↪ time
 MINIMISE instead: negate c and negate the answer.
 >= CONSTRAINT: multiply the row by -1.   EQUALITY:
  ↪ add it twice, as <= and as >=.
 FREE VARIABLE (no x >= 0): substitute x = u - v with
  ↪ u, v >= 0. Doubles that column.
 UPPER BOUND x_i <= U: just another row; simplex has
  ↪ no special handling and does not need any.
 LINEAR-FRACTIONAL objective (c.x + a) / (d.x + b):
  ↪ CHARNES-COOPER - substitute y = x/(d.x+b),
   t = 1/(d.x+b) and maximise c.y + a t subject to A
  ↪ y <= b t, d.y + b t = 1, t >= 0. Linear again.
 INTEGER answers are NOT guaranteed. If the
  ↪ constraint matrix is TOTALLY UNIMODULAR (network
   matrices, interval matrices, bipartite incidence)
  ↪ the optimum IS integral - that is exactly why
   flow problems have integral optima. Otherwise you
  ↪ need ILP, which is NP-hard.
 ZERO-SUM MATRIX GAME: shift A so every entry is > 0,
  ↪ then min sum(y) s.t. A y >= 1, y >= 0 gives
   value v = 1/sum(y) and the column player's mixed
  ↪ strategy y*v; the DUAL gives the row player's.
   Von Neumann: max_x min_y x^T A y = min_y max_x x^T
  ↪ A y.
 DUALITY, which is often the actual solution: max
  ↪ c.x, Ax <= b, x >= 0  is dual to
   min b.y, A^T y >= c, y >= 0; the two optima are
  ↪ equal. COMPLEMENTARY SLACKNESS at the optimum:
   x_j > 0 => the j-th dual constraint is tight, and
  ↪ y_i > 0 => the i-th primal row is tight.
   The combinatorial specialisations you already know
  ↪ are max-flow/min-cut and Konig.
 PRECISION: this is floating point. If the data is
  ↪ integral and small, and you need an exact
   answer, prefer a combinatorial model (flow,
  ↪ matching) over simplex. */
```

7 | Strings

7.1 String Matching

7.1.1 KMP

KMP. pi[i] = length of the longest proper border of s[0..i] (prefix that is also a suffix).

```
// Gives O(n+m) substring search, the period of a
  ↪ prefix (i+1 - pi[i]), and the automaton for DP.
vector<int> compute_pi(const string &s) {
    int n = s.size();
    vector<int> pi(n);
    for (int i = 1; i < n; i++) {
```

```
        int j = pi[i - 1];
        while (j > 0 && s[i] != s[j])
            j = pi[j - 1];
        if (s[i] == s[j])
            j++;
        pi[i] = j;
    }
    return pi;
}

vector<int> find_matches(const string &text, const
  ↪ string &pat) {
    int n = pat.length(), m = text.length();
    string s = pat + "#" + text;
    vector<int> pi = compute_pi(s), ans;
    for (int i = n + 1; i <= n + m; i++) { /* n + 1
  ↪ is where the text starts */
        if (pi[i] == n) {
            ans.push_back(i - 2 * n); /* i - (n - 1)
  ↪ - (n + 1) */
        }
    }
    return ans;
}

void compute_automaton(string s, vector<vector<int>>
  ↪ &aut) {
    s += '#';
    int n = s.size();
    vector<int> pi = compute_pi(s);
    aut.assign(n, vector<int>(26));
    for (int i = 0; i < n; i++) {
        for (int c = 0; c < 26; c++) {
            if (i > 0 && 'a' + c != s[i])
                aut[i][c] = aut[pi[i - 1]][c];
            else
                aut[i][c] = i + ('a' + c == s[i]);
        }
    }
}
```

7.1.2 RabinKarp

RABIN-KARP - rolling-hash substring search in O(n+m) expected. Prefer 05 - Hashing for anything

```
// serious (this uses a FIXED base, which is
  ↪ hackable); keep this for one-off matching.
vector<int> rabin_karp(string const &s, string const
  ↪ &t) {
    const int p = 31;
    const int m = 1e9 + 9;
    int S = s.size(), T = t.size();

    vector<ll> p_pow(max({S, T, 1}));            //
  ↪ max(0,0) would leave it empty
    p_pow[0] = 1;
    for (int i = 1; i < (int) p_pow.size(); i++)
        p_pow[i] = (p_pow[i - 1] * p) % m;

    vector<ll> h(T + 1, 0);
    for (int i = 0; i < T; i++)
        h[i + 1] = (h[i] + (t[i] - 'a' + 1) *
  ↪ p_pow[i]) % m;
    ll h_s = 0;
    for (int i = 0; i < S; i++)
        h_s = (h_s + (s[i] - 'a' + 1) * p_pow[i]) % m;

    vector<int> occurrences;
    for (int i = 0; i + S - 1 < T; i++) {
        ll cur_h = (h[i + S] + m - h[i]) % m;
        if (cur_h == h_s * p_pow[i] % m)
            occurrences.push_back(i);
    }
    return occurrences;
}
```

7.1.3 ZFunction

z[i] is the length of the longest substring starting from s[i] which is also a prefix of s

```
vector<int> z_function(string s) {
    int n = s.length();
    vector<int> z(n);
    int l = 0, r = 0;
    for (int i = 1; i < n; i++) {
        if (i < r) {
            z[i] = min(r - i, z[i - l]);
        }
        while (i + z[i] < n && s[z[i]] == s[i +
  ↪ z[i]]) {
            z[i]++;
        }
        if (i + z[i] > r) {
            l = i;
            r = i + z[i];
        }
    }
    return z;
}
```

7.1.4 WildcardMatching

STRING MATCHING WITH WILDCARDS in O(n log n). A wildcard matches any character, on EITHER side.

```
// Needs: conv (06 - Math/05 - Convolution/01 -
  ↪ FFT.cpp).
// Map every real character to a positive value and
  ↪ every wildcard to 0. Then position i matches iff
//        sum_j  ( p[j] - t[i+j] )^2 * p[j] * t[i+j]
  ↪ ==  0,
// because each term is >= 0 and vanishes exactly
  ↪ when the characters agree or one of them is 0.
// Expanding gives three correlations: sum p^3 t - 2
  ↪ sum p^2 t^2 + sum p t^3, and a correlation is a
// convolution with the pattern reversed.
vector<int> wildcardMatch(const string& t, const
  ↪ string& p, char wild = '?') {
    int n = t.size(), m = p.size();
    if (m > n) return {};
    // The identity needs values >= 0 and wildcard ==
  ↪ 0. Compress the characters that actually
    // occur to 1..k - with a raw `c - 'a' + 1` any
  ↪ character below 'a' is NEGATIVE and the terms
    // can cancel, which reports matches that do not
  ↪ exist (e.g. text "_b" against pattern "Z]").
    ll id[256] = {}, k = 0;
  ↪          // a flat table, not a map: the
    for (char c : t) if (c != wild) id[(unsigned
  ↪ char)c] = 1;   // conversion is O(n), not O(n
  ↪ log n)
    for (char c : p) if (c != wild) id[(unsigned
  ↪ char)c] = 1;
    for (int c = 0; c < 256; c++) if (id[c]) id[c] =
  ↪ ++k;
    auto val = [&](char c) { return c == wild ? 0LL :
  ↪ id[(unsigned char)c]; };
    vector<ll> a(m), b(n);
    for (int i = 0; i < m; i++) a[i] = val(p[m - 1 -
```

```
    ↪ i]);       // reversed pattern
    for (int i = 0; i < n; i++) b[i] = val(t[i]);
    auto pw = [](vector<ll> v, int k) { for (ll& x :
    ↪ v) { ll y = 1; for (int i = 0; i < k; i++) y *=
    ↪ x; x = y; } return v; };
    auto c1 = conv(pw(a, 3), b), c2 = conv(pw(a, 2),
    ↪ pw(b, 2)), c3 = conv(a, pw(b, 3));
    vector<int> res;
    for (int i = 0; i + m <= n; i++)
        if (c1[i + m - 1] - 2 * c2[i + m - 1] + c3[i
    ↪ + m - 1] == 0) res.push_back(i);
    return res;
    ↪          // 0-based start positions
}
// PRECISION: the largest intermediate is about k^4 *
  ↪ m, where k is the number of DISTINCT real
// characters (which is why compressing is free).
  ↪ With k = 26 and m = 1e5 that is
// ~4.6e10, comfortable for double FFT. For a big
  ↪ alphabet, compress the characters that actually
// occur first, or run the three convolutions under
  ↪ two NTT primes and CRT.
// COUNT MISMATCHES at every shift (no wildcards,
  ↪ small alphabet): for each letter c convolve the
// 0/1 indicator of c in p (reversed) with the
  ↪ indicator of c in t; summing gives the number of
// MATCHES per shift, and mismatches = m - matches.
  ↪ O(|sigma| n log n).
// k-MISMATCH: same counts, keep shifts with matches
  ↪ >= m - k.
// SUBSET / "at most one character class" matching:
  ↪ encode each position as a bitmask over the
// alphabet and convolve the complement indicators; a
  ↪ shift is valid iff the total is 0.
```

7.1.5 PrefixFunctionApplications

WHAT THE PREFIX FUNCTION ANSWERS once you have pi[] - every one of these is O(n) and every one

```
// Needs: 07 - Strings/01 - String Matching/01 -
  ↪ KMP.cpp   (compute_pi)
// of them is a contest problem on its own. INDEX
  ↪ CONVENTION: s is 0-indexed and
//     pi[i] = length of the longest PROPER border of
  ↪ the prefix s[0..i]  (so pi[0] = 0).
// A "border" is a string that is both a proper
  ↪ prefix and a proper suffix.
// WHEN: any question about periodicity, repetition,
  ↪ "shortest string whose k-th power is s",
// "how many times does every prefix occur", "which
  ↪ prefixes are also suffixes".

// --- ALL BORDERS of the whole string, longest
  ↪ first. The border of a border is a border, so the
// chain pi[n-1] -> pi[pi[n-1]-1] -> ... enumerates
  ↪ them ALL, in O(number of borders) <= O(n).
vector<int> allBorders(const vector<int>& pi) {
    vector<int> b;
    for (int k = pi.empty() ? 0 : pi.back(); k > 0; k
  ↪ = pi[k - 1]) b.push_back(k);
    return b;
  ↪          // lengths, decreasing
}
// --- PERIODS. p is a period of s (s[i] == s[i+p]
  ↪ wherever both exist) IFF n - p is a border.
// So the periods are exactly { n - b : b a border },
  ↪ and the SMALLEST period is n - pi[n-1].
int smallestPeriod(int n, const vector<int>& pi) {
  ↪ return n - (n ? pi[n - 1] : 0); }
// s is a perfect power t^k IFF n % smallestPeriod ==
  ↪ 0, and then k = n / smallestPeriod.
// (Careful: "aabaa" has period 3 but 5 % 3 != 0, so
  ↪ it is NOT a power - the test is required.)

// --- OCCURRENCE COUNT OF EVERY PREFIX inside s.
  ↪ cnt[len] = number of occurrences of s[0..len-1].
// Every position ends the prefix of length i+1, and
  ↪ an occurrence of a prefix implies an
// occurrence of all of ITS borders - so push the
  ↪ counts down the pi chain in reverse order.
vector<int> prefixOccurrences(const string& s, const
  ↪ vector<int>& pi) {
    int n = s.size();
    vector<int> cnt(n + 1, 0);
    for (int i = 0; i < n; i++) cnt[pi[i]]++;
    for (int i = n - 1; i > 0; i--) cnt[pi[i - 1]] +=
  ↪ cnt[i];
    for (int i = 0; i <= n; i++) cnt[i]++;
  ↪          // the prefix itself
    cnt[0] = 0;
    return cnt;
}
// The same routine on  p + '#' + t  counts
  ↪ occurrences of every prefix of p inside t: read
// cnt[|p|] for the number of matches, and cnt[k] -
  ↪ (contribution inside p) for shorter prefixes.

// --- BORDER TREE: parent(i) = pi[i-1] over nodes
  ↪ 0..n (node i = the prefix of length i). Then
// "the borders of the prefix of length i" = the
  ↪ ancestors of i, so
//   is u a border of v?            -> u is an
  ↪ ancestor of v (Euler-tour interval test, O(1))
//   longest common border of two   -> LCA
//   count borders of prefix i      -> depth(i)
vector<int> borderTree(const vector<int>& pi) {
    int n = pi.size();
    vector<int> par(n + 1, 0);
    par[0] = -1;
    for (int i = 1; i <= n; i++) par[i] = pi[i - 1];
    return par;
}
/* MORE PREFIX-FUNCTION FACTS worth having on the page
 pi vs Z: z[i] = length of the lcp of s and s[i..].
  ↪ Either converts to the other in O(n), but pi
   is the one that extends ONLINE (append a
  ↪ character, update in O(1) amortised) and the one
  ↪ that
   turns into an automaton; z is the one that answers
  ↪ "match at offset i" directly.
 AUTOMATON aut[i][c] (see 01 - KMP.cpp) makes the DP
  ↪ "count strings of length L avoiding p"
   trivial; without it that DP is O(L n |A|) with the
  ↪ pi chain walked per step.
 DISTINCT SUBSTRINGS incrementally: append c to s,
  ↪ run the prefix function of the REVERSED new
   string; the new distinct substrings number (i+1) -
  ↪ max(z-values) - O(n^2) total, and the right
   answer whenever the input arrives online and a
  ↪ suffix automaton is overkill.
 FINE AND WILF: if p and q are periods of s and p + q
  ↪ - gcd(p,q) <= n, then gcd(p,q) is a period.
   This is why the border lengths of any prefix form
  ↪ O(log n) ARITHMETIC PROGRESSIONS - the fact
   that makes "sum over all borders" DPs O(n log n)
  ↪ instead of O(n^2).
 COUNTING STRINGS with a given pi array, or with a
```

```
   ↪ given set of borders, is Fine and Wilf plus
    inclusion-exclusion over the progressions; do NOT
   ↪ try to enumerate borders one by one. */
```

7.2 Hashing

7.2.1 StringHashing

Double-mod polynomial hashing. RANDOM bases => not anti-hackable.

```
// No modular inverses: h(l,r) is compared already
   ↪ scaled, so get() is 2 mults.
ll HB1, HB2;
struct Hash {
    static const ll M1 = 1000000007, M2 = 998244353;
    int n;
    vector<ll> h1, h2, p1, p2;

    Hash(const string& s) : n(s.size()), h1(n + 1,
   ↪ 0), h2(n + 1, 0), p1(n + 1, 1), p2(n + 1, 1) {
        if (!HB1) {
            mt19937_64
   ↪ rng(chrono::steady_clock::now().time_since_epoch().count());
            HB1 = 256 + rng() % (M1 - 300), HB2 = 256
   ↪ + rng() % (M2 - 300);   // FULL range: a base
   ↪ drawn
        // from only 1e6 values gives collision
   ↪ probability n/1e6 per modulus (0.1 at n = 1e5)
        }
        for (int i = 0; i < n; i++) {
            h1[i + 1] = (h1[i] * HB1 + s[i]) % M1,
   ↪ p1[i + 1] = p1[i] * HB1 % M1;
            h2[i + 1] = (h2[i] * HB2 + s[i]) % M2,
   ↪ p2[i + 1] = p2[i] * HB2 % M2;
        }
    }
    pair<ll, ll> get(int l, int r) const {
   ↪   // [l, r] inclusive, 0-based
        ll a = (h1[r + 1] - h1[l] * p1[r - l + 1]) %
   ↪ M1;
        ll b = (h2[r + 1] - h2[l] * p2[r - l + 1]) %
   ↪ M2;
        return {a < 0 ? a + M1 : a, b < 0 ? b + M2 :
   ↪ b};
    }
    bool eq(int l1, int r1, int l2, int r2) const {
        return r1 - l1 == r2 - l2 && get(l1, r1) ==
   ↪ get(l2, r2);
    }
    // concat(A over lenA, B): h = hA * base^lenB +
   ↪ hB   (works per modulus)
};
// USES: substring equality O(1) * longest common
   ↪ prefix of two suffixes by binary search
//       * dedup of substrings * 2D grid patterns
   ↪ (hash rows, then hash the row-hashes)
//       * tree isomorphism (hash the sorted multiset
   ↪ of child hashes)
```

7.2.2 Hashing2D

2D HASHING - O(1) hash of any submatrix, for grid pattern matching.

```
// 2D prefix hash with random bases b1 (down) and b2
   ↪ (right), single 62-bit modulus via __int128.
ll HB2D1 = 0, HB2D2 = 0;
   ↪ // ONE base pair per RUN, at file scope.
struct Hash2D {
   ↪ // Per-object bases would make two grids
    static const ll M = (ll)1e18 + 9;
   ↪ // incomparable - and "find this pattern in
    int n, m;
   ↪ // that grid" needs exactly two objects.
    ll b1, b2;
    vector<ll> P1, P2;
    vector<vector<ll>> h;

    static ll mul(ll a, ll b) { return
   ↪ (ll)((__int128)a * b % M); }
    Hash2D(const vector<string>& g) : n(g.size()),
   ↪ m(n ? g[0].size() : 0),
                                      P1(n + 1, 1),
   ↪ P2(m + 1, 1), h(n + 1, vector<ll>(m + 1, 0)) {
        if (!HB2D1) {
            mt19937_64
   ↪ rng(chrono::steady_clock::now().time_since_epoch().c
            HB2D1 = 256 + rng() % (M - 300), HB2D2 =
   ↪ 256 + rng() % (M - 300);
        }
        b1 = HB2D1, b2 = HB2D2;
        for (int i = 1; i <= n; i++) P1[i] = mul(P1[i
   ↪ - 1], b1);
        for (int j = 1; j <= m; j++) P2[j] = mul(P2[j
   ↪ - 1], b2);
        for (int i = 1; i <= n; i++)
            for (int j = 1; j <= m; j++) {
                __int128 v = (__int128)mul(h[i -
   ↪ 1][j], b1) + mul(h[i][j - 1], b2)
                           - mul(mul(h[i - 1][j - 1],
   ↪ b1), b2) + g[i - 1][j - 1];
                h[i][j] = (ll)((v % M + M) % M);
            }
    }
    // hash of the submatrix with top-left (r, c) and
   ↪ size dr x dc, 0-based
    ll get(int r, int c, int dr, int dc) const {
        int r2 = r + dr, c2 = c + dc;
        __int128 v = (__int128)h[r2][c2] -
   ↪ mul(h[r][c2], P1[dr]) - mul(h[r2][c], P2[dc])
                   + mul(mul(h[r][c], P1[dr]),
   ↪ P2[dc]);
        return (ll)((v % M + M) % M);
    }
};
// USES: count occurrences of a pattern grid *
   ↪ largest common square submatrix (binary search
   ↪ the
// side, hash all windows of both grids) * count
   ↪ distinct k x k submatrices.
// CHEAPER when the pattern is fixed: 1D-hash each
   ↪ row of the pattern, then run KMP /
// Aho-Corasick over the rows treated as single
   ↪ symbols.
```

7.2.3 HashingWithUpdates

HASHING UNDER POINT UPDATES, forward AND backward - the structure that answers "is s[l..r] a

```
// palindrome" while characters keep changing. Static
   ↪ prefix hashes cannot survive an update, and
// Manacher / eertree cannot be updated at all, so
   ↪ this is the only tool for that query shape.
// Two Fenwick trees over the modular values s[i]*p^i
   ↪ and s[i]*p^(n+1-i): point update O(log n),
// substring hash O(log n), palindrome test O(log n),
   ↪ longest palindrome at a centre O(log^2 n).
// MODULUS 2^61 - 1 with 128-bit multiplication: one
   ↪ modulus of that size is safer than two
// 32-bit ones (birthday bound ~ q^2 / 2^61) and
```

```
  ↪ shorter to write.
// INDEX CONVENTION: 1-INDEXED, s[1..n], and every l,
  ↪ r below is inclusive 1-based. get(l, r) is
// scaled so that equal substrings of EQUAL LENGTH
  ↪ compare equal - never compare across lengths.
struct DynHash {
    static const unsigned long long MD = (1ULL << 61)
  ↪ - 1;
    static unsigned long long mul(unsigned long long
  ↪ a, unsigned long long b) {
        __int128 c = (__int128)a * b;
        unsigned long long lo = (unsigned long
  ↪ long)(c & MD), hi = (unsigned long long)(c >>
  ↪ 61);
        lo += hi;
        return lo >= MD ? lo - MD : lo;
    }
    static unsigned long long ad(unsigned long long
  ↪ a, unsigned long long b) {
        return a + b >= MD ? a + b - MD : a + b;
    }
    static unsigned long long sb(unsigned long long
  ↪ a, unsigned long long b) {
        return a >= b ? a - b : a + MD - b;
    }
    static unsigned long long pw(unsigned long long
  ↪ b, unsigned long long e) {
        unsigned long long r = 1;
        for (; e; e >>= 1, b = mul(b, b)) if (e & 1)
  ↪ r = mul(r, b);
        return r;
    }
    int n;
    string s;
  ↪           // s[1..n]
    vector<unsigned long long> P, IP, ft[2];
    DynHash(const string& in) {
        n = in.size(), s = string(n + 1, 0);
  ↪            // s[0] unused; filled by upd below
        unsigned long long base = 131 + (unsigned
  ↪ long long)chrono::steady_clock::now()

  ↪ .time_since_epoch().count() % 1000000;
        P.resize(n + 2), IP.resize(n + 2),
  ↪ ft[0].assign(n + 1, 0), ft[1].assign(n + 1, 0);
        P[0] = IP[0] = 1;
        unsigned long long ib = pw(base, MD - 2);
        for (int i = 1; i <= n + 1; i++) P[i] =
  ↪ mul(P[i - 1], base), IP[i] = mul(IP[i - 1], ib);
        for (int i = 1; i <= n; i++) upd(i, in[i -
  ↪ 1]);
    }
    void add(int k, int i, unsigned long long v) {
        for (; i <= n; i += i & -i) ft[k][i] =
  ↪ ad(ft[k][i], v);
    }
    unsigned long long qr(int k, int i) {
        unsigned long long r = 0;
        for (; i > 0; i -= i & -i) r = ad(r,
  ↪ ft[k][i]);
        return r;
    }
    unsigned long long rng(int k, int l, int r) {
  ↪ return sb(qr(k, r), qr(k, l - 1)); }
    void upd(int i, char c) {
  ↪           // set s[i] = c
        unsigned long long d = sb(c, s[i]);
        add(0, i, mul(d, P[i])), add(1, i, mul(d, P[n
  ↪ + 1 - i]));
        s[i] = c;
    }
    unsigned long long get(int l, int r) { return
  ↪ mul(rng(0, l, r), IP[l]); }
    unsigned long long rev(int l, int r) { return
  ↪ mul(rng(1, l, r), IP[n + 1 - r]); }
    bool isPal(int l, int r) { return get(l, r) ==
  ↪ rev(l, r); }
    int oddRadius(int c) {
  ↪           // longest s[c-k..c+k] palindrome
        int lo = 0, hi = min(c - 1, n - c), res = 0;
        while (lo <= hi) {
            int m = (lo + hi) / 2;
            isPal(c - m, c + m) ? (res = m, lo = m +
  ↪ 1) : hi = m - 1;
        }
        return res;
  ↪           // length 2 * res + 1
    }
    int evenRadius(int c) {
  ↪           // longest s[c-k+1..c+k] palindrome
        int lo = 1, hi = min(c, n - c), res = 0;
        while (lo <= hi) {
            int m = (lo + hi) / 2;
            isPal(c - m + 1, c + m) ? (res = m, lo =
  ↪ m + 1) : hi = m - 1;
        }
        return res;
  ↪           // length 2 * res
    }
};
/* NOTES
 THE SCALING IS THE WHOLE GAME. Forward stores s[i] *
  ↪ p^i, so the raw range sum of [l, r] is
  p^l times the hash of the substring - multiply by
  ↪ p^-l. Backward stores s[i] * p^(n+1-i), whose
  range sum is p^(n+1-r) times the reversed
  ↪ substring's hash. Get one exponent wrong and
  ↪ every
  answer is silently false; test isPal on a known
  ↪ palindrome first.
 A SEGMENT TREE does the same with range ASSIGN ("set
  ↪ s[l..r] to c") lazily, which a Fenwick
  cannot: keep (hash, length) per node and lazily
  ↪ set hash = c * (p^len - 1) / (p - 1).
 SUBSTRING COMPARISON / SORTING under updates: binary
  ↪ search the first differing position with
  get(), O(log^2 n) per comparison - the standard
  ↪ "compare two suffixes with updates".
 LONGEST COMMON EXTENSION under updates is the same
  ↪ binary search, and is what makes this a
  drop-in replacement for a suffix array in dynamic
  ↪ problems.
 ANTI-HASH: the base is randomised at construction.
  ↪ Never hard-code a base in an open-hacking
  contest; with a fixed base and mod 2^61-1 the
  ↪ string is still forgeable by a prepared test. */
```

7.3 Aho Corasick

7.3.1 AhoCorasick

AHO-CORASICK - a trie of all patterns plus suffix (fail) links, so one pass over the text finds

```
// every occurrence of every pattern in O(|text| + total pattern length + #matches).
// The goto-automaton form (nxt filled in by BFS) is also the transition table for DP over strings.
struct Mapper {
    int operator()(char c) const { return c - 'a'; }
};

template <int ALPHABET, typename Mapper>
struct Aho {
    struct Node {
        array<int, ALPHABET> nxt;
        int link = 0;
        int terminal_idx = -1; // -1 if not a terminal, else idx of string insertion
        int exit_link = 0; // Points to the nearest terminal suffix
        int depth = 0; // Size of the prefix the node represents
        Node() { nxt.fill(0); }
    };

    vector<Node> t;
    vector<int> bfs_order;
    Mapper get_idx; // Instance of the mapping function
    int word_count = 0; // Tracks the order of inserted strings

    // Constructor optionally accepts a custom mapper instance
    Aho(Mapper mapper = Mapper()) : get_idx(mapper) {
        t.emplace_back();
    }

    int insert(const string& p) {
        int u = 0;
        for (char c : p) {
            int v = get_idx(c);
            if (!t[u].nxt[v]) {
                t[u].nxt[v] = t.size();
                t.emplace_back();
                t.back().depth = t[u].depth + 1; // New node's depth is parent's depth + 1
            }
            u = t[u].nxt[v];
        }

        // If this exact string hasn't been inserted before, assign it the current index
        if (t[u].terminal_idx == -1) {
            t[u].terminal_idx = word_count;
        }
        word_count++; // Increment for the next insertion

        return u;
    }

    void build() {
        queue<int> q;
        for (int c = 0; c < ALPHABET; ++c) {
            if (t[0].nxt[c]) {
                q.push(t[0].nxt[c]);
            }
        }

        while (!q.empty()) {
            int u = q.front();
            q.pop();
            bfs_order.push_back(u);

            for (int c = 0; c < ALPHABET; ++c) {
                int v = t[u].nxt[c];
                if (v) {
                    t[v].link = t[t[u].link].nxt[c];

                    // Compute the exit link:
                    // If the immediate suffix link is a terminal, point to it.
                    // Otherwise, copy its exit link.
                    t[v].exit_link = (t[t[v].link].terminal_idx != -1) ? t[v].link : t[t[v].link].exit_link;

                    q.push(v);
                }
                else {
                    t[u].nxt[c] = t[t[u].link].nxt[c];
                }
            }
        }
    }

    int advance(int u, char c) {
        return t[u].nxt[get_idx(c)];
    }
};
```

7.3.2 AhoCorasickDP

DP OVER THE AHO-CORASICK AUTOMATON - the high-value use of Aho, more than matching itself.

```
// Needs: Mint (06 - Math/01) and AhoCorasick (01 - AhoCorasick.cpp).
// After build(), nxt[] is a total transition function, so the automaton is a DFA over states
// 0..S-1 and you can run any DP on (position, state, extra).
// "bad" marks states that contain a forbidden pattern as a suffix; propagate it along exit links.
struct AhoDP {
    static const int A = 26;                                    // trie fan-out
    int alpha = 26;                                             // alphabet the DP counts over (set smaller!)
    vector<array<int, A>> nxt;
    vector<int> link, term;
    vector<char> bad;
    vector<int> order;  // BFS order of states

    AhoDP() { newNode(); }
    int newNode() { nxt.push_back({}); nxt.back().fill(0); link.push_back(0); term.push_back(0); bad.push_back(0); return nxt.size() - 1; }
    void insert(const string& p, int id = 1) {
        int u = 0;
        for (char c : p) {
            int v = c - 'a';
            if (!nxt[u][v]) nxt[u][v] = newNode();
            u = nxt[u][v];
```

```
        }
        term[u] = id, bad[u] = 1;
    }
    void build() {
        queue<int> q;
        for (int c = 0; c < A; c++) if (nxt[0][c])
↪ q.push(nxt[0][c]);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            order.push_back(u);
            bad[u] = bad[u] || bad[link[u]];
↪  // a state is bad if any SUFFIX is a pattern
            for (int c = 0; c < A; c++) {
                int v = nxt[u][c];
                if (v) link[v] = nxt[link[u]][c],
↪ q.push(v);
                else nxt[u][c] = nxt[link[u]][c];
↪  // total transition function
            }
        }
    }
    // #strings of length L over the first ‘alpha‘
↪ letters containing NO pattern. O(L*S*alpha).
    Mint countAvoiding(int L) {
        int S = nxt.size();
        vector<Mint> dp(S, 0), nd;
        dp[0] = 1;
        for (int i = 0; i < L; i++) {
            nd.assign(S, 0);
            for (int u = 0; u < S; u++) if (!bad[u]
↪ && dp[u].v)
                for (int c = 0; c < alpha; c++) {
                    int v = nxt[u][c];
                    if (!bad[v]) nd[v] += dp[u];
                }
            dp = nd;
        }
        Mint r = 0;
        for (int u = 0; u < S; u++) if (!bad[u]) r +=
↪ dp[u];
        return r;
    }
    // #occurrences of every pattern in a text: walk
↪ the text, count visits per state, then push
    // the counts DOWN the suffix-link tree (reverse
↪ BFS order).
    vector<ll> occurrences(const string& s) {
        vector<ll> cnt(nxt.size(), 0);
        int u = 0;
        for (char c : s) u = nxt[u][c - 'a'],
↪ cnt[u]++;
        for (int i = order.size() - 1; i >= 0; i--)
↪ cnt[link[order[i]]] += cnt[order[i]];
        return cnt;
↪  // cnt[state of pattern p] = #occurrences
    }
};
// SHORTEST / LEXICOGRAPHICALLY SMALLEST string
↪ avoiding all patterns: BFS over (state) with
// the alphabet in order. If a cycle among good
↪ states is reachable, arbitrarily long strings
// exist - detect it with a cycle check on the
↪ good-state subgraph.
```

7.3.3 DynamicAhoCorasick

DYNAMIC (ONLINE) AHO-CORASICK - the pattern set grows while queries arrive. A plain

```
// Aho-Corasick must be rebuilt from scratch after
↪ every insertion, which is O(total length) per
// insert; here the patterns are kept in O(log P)
↪ STATIC automata of sizes 1, 2, 4, 8, ... and
// inserting merges the small ones the way binary
↪ counter increments carry.
// AMORTISED O(|p| log P) per insert, O(|s| log P)
↪ per query (log P <= ~20 automata to feed).
// WHEN: "add a word, then count occurrences of all
↪ words in this text", online dictionary
// problems (CF 710F), and any offline-impossible
↪ interleaving of insert and query.
// DELETION: keep a SECOND identical structure of
↪ deleted patterns and subtract its answer.
// INDEX CONVENTION: node 0 is the root; cnt[v] is
↪ already the SUM over the whole suffix-link
// chain, so a query adds cnt[cur] once per text
↪ position and never walks exit links.
struct StaticAho {
↪          // rebuilt wholesale, never patched
    vector<array<int, 26>> nxt;
    vector<int> link, cnt;
    StaticAho() { clear(); }
    void clear() { nxt.assign(1, {}), nxt[0].fill(0),
↪ link.assign(1, 0), cnt.assign(1, 0); }
    void insert(const string& p) {
        int u = 0;
        for (char ch : p) {
            int c = ch - 'a';
            if (!nxt[u][c]) {
                nxt[u][c] = nxt.size();
                nxt.push_back({}),
↪ nxt.back().fill(0), link.push_back(0),
↪ cnt.push_back(0);
            }
            u = nxt[u][c];
        }
        cnt[u]++;
    }
    void build() {
↪          // goto-automaton + cumulative cnt
        queue<int> q;
        for (int c = 0; c < 26; c++) if (nxt[0][c])
↪ q.push(nxt[0][c]);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            cnt[u] += cnt[link[u]];
            for (int c = 0; c < 26; c++) {
                int v = nxt[u][c];
                if (v) link[v] = nxt[link[u]][c],
↪ q.push(v);
                else nxt[u][c] = nxt[link[u]][c];
            }
        }
    }
    ll count(const string& s) const {
↪          // total occurrences of all patterns
        ll r = 0;
        int u = 0;
        for (char ch : s) u = nxt[u][ch - 'a'], r +=
↪ cnt[u];
        return r;
    }
};
struct DynamicAho {
```

```
    static const int B = 20;
  ↪          // supports 2^20 patterns
    vector<string> words[B];
    StaticAho aho[B];
    void insert(const string& p) {
        int lvl = 0;
        while (lvl < B && !words[lvl].empty()) lvl++;
  ↪           // the carry of a binary counter
        words[lvl].push_back(p);
        for (int i = 0; i < lvl; i++) {
            for (auto& w : words[i])
  ↪ words[lvl].push_back(w);
            words[i].clear(), aho[i].clear();
        }
        aho[lvl].clear();
        for (auto& w : words[lvl]) aho[lvl].insert(w);
        aho[lvl].build();
    }
    ll count(const string& s) {
  ↪          // occurrences of all patterns in s
        ll r = 0;
        for (int i = 0; i < B; i++) if
  ↪ (!words[i].empty()) r += aho[i].count(s);
        return r;
    }
};
/* NOTES AND VARIANTS
 WHY THE COST IS AMORTISED: a pattern is copied into
  ↪ a bigger automaton at most log P times, so
   the total rebuild work is O(sum |p| * log P) -
  ↪ exactly the same argument as binary lifting on
   a knapsack, or as the "logarithmic method" for
  ↪ making any static structure insert-only.
 THE SAME WRAPPER makes ANY static structure dynamic
  ↪ (suffix arrays, convex hulls, kd-trees):
   keep buckets of size 2^i and merge on carry. That
  ↪ is the reusable idea here.
 PER-PATTERN counts instead of a total: store an id
  ↪ per terminal and walk the exit-link chain, or
   put the query positions on the fail tree of each
  ↪ bucket - the total stays O(matches log P).
 MEMORY: 26 ints per node in every bucket. With big
  ↪ alphabets use a hash map per node or the
   sorted-children trick, otherwise 20 rebuilt copies
  ↪ will not fit.
 IF ALL PATTERNS ARE KNOWN OFFLINE, do not use this -
  ↪ build one static automaton. */
```

7.3.4 PatternContainmentPoset

ALL-PAIR OCCURRENCE RELATION between the patterns themselves: for every ordered pair (i, j),

```
// does pattern i occur as a SUBSTRING of pattern j?
  ↪ That relation is a partial order (a poset),
// and problems then ask for its longest chain, its
  ↪ maximum antichain (Dilworth = minimum path
// cover = k - maximum bipartite matching, see 03 -
  ↪ Graph Theory), or its transitive reduction.
// CF 590E "Birthday" is the canonical use: pick the
  ↪ largest subset with no containments.
// NAIVE is O(k^2 * L) with a matcher per pair. This
  ↪ is O(sum |p| * A) for the automaton plus
// O(k^3 / 64) for the closure, and the closure is
  ↪ the whole trick:
// during one pass over p_j you record ONLY the
  ↪ NEAREST terminal suffix at each position (the
// exit link). Everything else follows by
  ↪ TRANSITIVITY, because "occurs in" is transitive.
// INDEX CONVENTION: patterns are 0-indexed in
  ↪ insertion order and must be DISTINCT (dedup
  ↪ first,
// otherwise p_i occurs in p_j and p_j in p_i and the
  ↪ poset degenerates).
// Needs: 07 - Strings/03 - Aho Corasick/01 -
  ↪ AhoCorasick.cpp
template <int K>
  ↪ // K = compile-time bound on the #patterns
vector<bitset<K>> containment(const vector<string>&
  ↪ p) {
    Aho<26, Mapper> ac;
    vector<int> node(p.size());
    for (size_t i = 0; i < p.size(); i++) node[i] =
  ↪ ac.insert(p[i]);
    ac.build();
    int k = p.size();
    vector<bitset<K>> in(k);
  ↪          // in[j][i] = p_i occurs in p_j
    for (int j = 0; j < k; j++) {
        int u = 0;
        for (char c : p[j]) {
            u = ac.advance(u, c);
            // the nearest terminal suffix - skipping
  ↪ p_j itself, which ends at its own last
            // position and would otherwise hide the
  ↪ shorter terminal suffixes underneath it
            int tu = ac.t[u].terminal_idx;
            int v = (tu != -1 && tu != j) ? u :
  ↪ ac.t[u].exit_link;
            if (v && ac.t[v].terminal_idx != -1 &&
  ↪ ac.t[v].terminal_idx != j)
                in[j][ac.t[v].terminal_idx] = 1;
        }
    }
    vector<int> ord(k);
  ↪          // shorter patterns first: a proper
    iota(ord.begin(), ord.end(), 0);
  ↪          // substring is strictly shorter
    sort(ord.begin(), ord.end(), [&](int a, int b) {
  ↪ return p[a].size() < p[b].size(); });
    for (int j : ord) {
  ↪          // transitive closure
        auto cur = in[j];
        for (int i = 0; i < k; i++) if (cur[i]) in[j]
  ↪ |= in[i];
    }
    return in;
}
/* USING THE POSET
 LARGEST SUBSET WITH NO CONTAINMENT (maximum
  ↪ antichain) = k - (minimum chain cover), and by
   Dilworth on the TRANSITIVELY CLOSED relation,
  ↪ minimum chain cover = k - maximum matching in
   the bipartite graph "i on the left, j on the
  ↪ right, edge if p_i occurs in p_j". Recover the
   actual antichain from the Konig vertex cover: keep
  ↪ the vertices NOT in the cover.
 LONGEST CHAIN = longest path in the DAG = the usual
  ↪ O(k^2) DP on the closed relation.
 COUNTING occurrences instead of deciding: replace
  ↪ the bitset with a fail-tree subtree sum -
   push +1 at every visited node of p_j and read the
  ↪ subtree sum of each terminal, O((L + k) log).
 MEMORY: k * K / 8 bytes. k = 750 (the CF 590E bound)
  ↪ is 70 KB; k = 1e5 is not viable, and at
   that size the question is always something the
  ↪ fail tree answers directly.
 IF THE PATTERNS REPEAT, dedup and remember the
```

```
   ↪ multiplicity; the poset is on DISTINCT strings.
   ↪ */
```

7.4 Suffix Structures

7.4.1 SuffixArray

SUFFIX ARRAY + LCP + sparse table for O(1) LCE. Build is O(n log n) (prefix doubling with a radix

```
// sort), Kasai gives the LCP array in O(n).
   ↪ Everything you can ask about substrings reduces
   ↪ to a
// range on this array - see 03 -
   ↪ SuffixArrayApplications.cpp.
struct SuffixArray {
    string S;
    // sa is the suffix array with the empty suffix
   ↪ being sa[0]
    // lcp[i] holds the lcp between sa[i], sa[i - 1]
    vector<int> logs, sa, lcp, rank;
    vector<vector<int>> table;

    SuffixArray() {};

    SuffixArray(string &s, int lim = 256) {
        S = s;
        int n = s.size() + 1, k = 0, a, b;
        vector<int> c(s.begin(), s.end() + 1),
   ↪ tmp(n), frq(max(n, lim));
        c.back() = 0; //0 is less than any character
        sa = lcp = rank = tmp, iota(sa.begin(),
   ↪ sa.end(), 0);
        for (int j = 0, p = 0; p < n; j = max(1, j *
   ↪ 2), lim = p) {
            p = j, iota(tmp.begin(), tmp.end(), n -
   ↪ j);
            for (int i = 0; i < n; i++) {
                if (sa[i] >= j)
                    tmp[p++] = sa[i] - j;
            }

            fill(frq.begin(), frq.end(), 0);

            for (int i = 0; i < n; i++) frq[c[i]]++;
            for (int i = 1; i < lim; i++) frq[i] +=
   ↪ frq[i - 1];
            for (int i = n; i--;)
   ↪ sa[--frq[c[tmp[i]]]] = tmp[i];

            swap(c, tmp), p = 1, c[sa[0]] = 0;
            for (int i = 1; i < n; i++)
                a = sa[i - 1], b = sa[i], c[b] =
   ↪ (tmp[a] == tmp[b] && tmp[a + j] == tmp[b + j]) ?
   ↪ p - 1 : p++;
        }

        for (int i = 1; i < n; i++) rank[sa[i]] = i;
        for (int i = 0, j; i < n - 1; lcp[rank[i++]]
   ↪ = k)
            for (k &&k--, j = sa[rank[i] - 1];
                    s[i + k] == s[j + k];
        k++);
    }

    void preLcp() {
        int n = S.size() + 1;
        logs = vector<int>(n + 5);
        for (int i = 2; i < n + 5; ++i) {
            logs[i] = logs[i / 2] + 1;
        }
        // level-major: sized by logs[n], not a
   ↪ hard-coded 20 (which is OUT OF BOUNDS at
        // |s| >= 2^20 - and Codeforces routinely
   ↪ gives |s| up to 1e6), and no dead columns.
        table = vector<vector<int>>(logs[n] + 1,
   ↪ vector<int>(n));
        for (int i = 0; i < n; ++i) table[0][i] =
   ↪ lcp[i];
        for (int j = 1; j <= logs[n]; ++j)
            for (int i = 0; i + (1 << j) <= n; ++i)
                table[j][i] = min(table[j - 1][i],
   ↪ table[j - 1][i + (1 << (j - 1))]);
    }

    // ARGUMENTS ARE SUFFIX-ARRAY INDICES, not string
   ↪ positions. For two string positions
    // p and q use  lce(p, q)  below, which is what
   ↪ every application actually wants.
    int queryLcp(int i, int j) {
        if (i == j) return (int)S.size() - sa[i];
        if (i > j) swap(i, j);
        i++;
        int len = logs[j - i + 1];
        return min(table[len][i], table[len][j - (1
   ↪ << len) + 1]);
    }
    int lce(int p, int q) { return queryLcp(rank[p],
   ↪ rank[q]); }   // longest common extension
};
```

7.4.2 SuffixAutomaton

SUFFIX AUTOMATON - O(n) states/transitions, accepts exactly the substrings of s.

```
// st[v].len   = longest string in v's endpos class
   ↪    link = suffix link (v's parent)
// cnt[v]      = |endpos(v)| = #occurrences of every
   ↪ string in v
// distinct substrings = sum over v != root of
   ↪ (len[v] - len[link[v]])
struct SAM {
    struct S { int len = 0, link = -1; array<int, 26>
   ↪ nxt; S() { nxt.fill(-1); } };
    vector<S> st;
    vector<ll> cnt;
   ↪ // occurrence counts (after countOcc)
    vector<int> firstPos;
   ↪ // end position of the first occurrence
    int last = 0;
    bool counted = false;
   ↪ // countOcc() is idempotent

    SAM(int reserve = 0) { st.reserve(2 * reserve),
   ↪ st.push_back(S()), cnt.push_back(0),
   ↪ firstPos.push_back(0); }

    void extend(char ch) {
        int c = ch - 'a', cur = st.size();
        st.push_back(S()), cnt.push_back(1),
   ↪ firstPos.push_back(0);
        st[cur].len = st[last].len + 1, firstPos[cur]
   ↪ = st[cur].len - 1;
        int p = last;
        while (p != -1 && st[p].nxt[c] == -1)
   ↪ st[p].nxt[c] = cur, p = st[p].link;
        if (p == -1) st[cur].link = 0;
        else {
            int q = st[p].nxt[c];
```

```
            if (st[p].len + 1 == st[q].len)
↪ st[cur].link = q;
            else {
                int cl = st.size();
                st.push_back(st[q]),
↪ cnt.push_back(0),
↪ firstPos.push_back(firstPos[q]);
                st[cl].len = st[p].len + 1;
                while (p != -1 && st[p].nxt[c] == q)
↪ st[p].nxt[c] = cl, p = st[p].link;
                st[q].link = st[cur].link = cl;
            }
        }
        last = cur;
    }
    void build(const string& s) { for (char c : s)
↪ extend(c); }

    vector<int> byLen() const {
↪ // states sorted by len (counting sort)
        int n = st.size(), mx = st[0].len;
        for (auto& x : st) mx = max(mx, x.len);
        vector<int> c(mx + 2, 0), ord(n);
        for (auto& x : st) c[x.len]++;
        for (int i = 1; i <= mx; i++) c[i] += c[i -
↪ 1];
        for (int v = n - 1; v >= 0; v--)
↪ ord[--c[st[v].len]] = v;
        return ord;
↪ // increasing len; reverse for link-tree order
    }
    void countOcc() {
↪ // push counts down the suffix-link tree
        if (counted) return;
↪ // NOT idempotent without this guard
        counted = true;
        auto ord = byLen();
        for (int i = (int)ord.size() - 1; i > 0; i--)
↪ {
            int v = ord[i];
            if (st[v].link >= 0) cnt[st[v].link] +=
↪ cnt[v];
        }
        cnt[0] = 0;
    }
    ll distinctSubstrings() const {
↪ // number of DISTINCT non-empty substrings
        ll r = 0;
        for (int v = 1; v < (int)st.size(); v++) r +=
↪ st[v].len - st[st[v].link].len;
        return r;
    }
    ll totalOccurrences() {
↪ // sum over distinct substrings of #occurrences
        countOcc();
        ll r = 0;
        for (int v = 1; v < (int)st.size(); v++) r +=
↪ cnt[v] * (st[v].len - st[st[v].link].len);
        return r;
    }
    int countOf(const string& p) {
↪ // #occurrences of p in s (call countOcc first)
        int v = 0;
        for (char ch : p) { v = st[v].nxt[ch - 'a'];
↪ if (v < 0) return 0; }
        return cnt[v];
    }
    // K-th SMALLEST distinct substring (k is
↪ 1-indexed). Needs `paths` = #substrings from v.
    vector<ll> paths;
    void buildPaths() {
        paths.assign(st.size(), 1);
↪ // 1 counts the empty continuation
        auto ord = byLen();
        for (int i = (int)ord.size() - 1; i >= 0;
↪ i--) {
            int v = ord[i];
            paths[v] = 1;
            for (int c = 0; c < 26; c++) if
↪ (st[v].nxt[c] >= 0) paths[v] +=
↪ paths[st[v].nxt[c]];
        }
    }
    string kthSubstring(ll k) {
↪ // "" if fewer than k distinct substrings
        string r;
        int v = 0;
        while (k > 0) {
            int c = 0;
            for (; c < 26; c++) {
                int u = st[v].nxt[c];
                if (u < 0) continue;
                if (k <= paths[u]) { r += char('a' +
↪ c), v = u, k--; break; }
                k -= paths[u];
            }
            if (c == 26) break;
        }
        return r;
    }
};
// LONGEST COMMON SUBSTRING of s and t: build the SAM
↪ of s, then walk t keeping the current
// length; on a missing transition follow suffix
↪ links and clamp the length to st[v].len.
int longestCommonSubstring(SAM& A, const string& t) {
    int v = 0, l = 0, best = 0;
    for (char ch : t) {
        int c = ch - 'a';
        while (v && A.st[v].nxt[c] < 0) v =
↪ A.st[v].link, l = A.st[v].len;
        if (A.st[v].nxt[c] >= 0) v = A.st[v].nxt[c],
↪ l++;
        best = max(best, l);
    }
    return best;
}
// GENERALISED SAM (several strings): reset `last =
↪ 0` before each string and, in extend(),
// if a transition already exists reuse/clone it
↪ instead of creating a new state.
```

7.4.3 SuffixArrayApplications

WHAT YOU ACTUALLY EXTRACT FROM A SUFFIX ARRAY.

```
// Convention of 01 - SuffixArray.cpp: sa has n+1
↪ entries (sa[0] = the empty suffix),
// and lcp[i] = LCP(sa[i], sa[i-1]).  Everything
↪ below assumes that.

// 1) NUMBER OF DISTINCT SUBSTRINGS = sum over i of
↪ (len of suffix i) - lcp with the previous.
ll distinctSubstrings(const SuffixArray& S) {
    ll n = S.S.size(), r = 0;
    for (ll i = 1; i <= n; i++) r += (n - S.sa[i]) -
↪ S.lcp[i];
    return r;
```

```
}
// 2) LONGEST REPEATED SUBSTRING = max lcp; the
   ↪ substring starts at sa[argmax].
pair<int, int> longestRepeated(const SuffixArray& S)
   ↪ {         // {length, start position}
    int n = S.S.size(), bi = 0, best = 0;
    for (int i = 1; i <= n; i++) if (S.lcp[i] > best)
   ↪ best = S.lcp[i], bi = S.sa[i];
    return {best, bi};
}
// 3) OCCURRENCES of a pattern = the contiguous SA
   ↪ block whose suffixes start with it.
//    Two binary searches, O(|P| log n).
pair<int, int> patternRange(const SuffixArray& S,
   ↪ const string& p) {   // [lo, hi], empty if lo>hi
    int n = S.S.size();
    auto cmp = [&](int idx, const string& q) { return
   ↪ S.S.compare(idx, q.size(), q) < 0; };
    int lo = 1, hi = n, L = n + 1, R = n;
    while (lo <= hi) { int m = (lo + hi) / 2;
   ↪ cmp(S.sa[m], p) ? lo = m + 1 : (L = m, hi = m -
   ↪ 1); }
    lo = L, hi = n, R = L - 1;
   ↪          // R MUST be reset here:

   ↪          // otherwise an absent pattern

   ↪          // returns the range [L, n]
    while (lo <= hi) {
        int m = (lo + hi) / 2;
        S.S.compare(S.sa[m], p.size(), p) == 0 ? (R =
   ↪ m, lo = m + 1) : hi = m - 1;
    }
    return {L, R};
   ↪          // count = R - L + 1
}
// 4) LONGEST COMMON SUBSTRING of two strings (named
   ↪ lcsViaSuffixArray to avoid clashing
//    with the SAM version in 02 -
   ↪ SuffixAutomaton.cpp): build SA of a + sep + b
   ↪ (sep smaller than every
//    real character), then take the max lcp over
   ↪ ADJACENT pairs coming from different sides.
int lcsViaSuffixArray(const string& a, const string&
   ↪ b) {
    string t = a + char(1) + b;
    SuffixArray S(t);
    int n = t.size(), best = 0, cut = a.size();
    auto side = [&](int p) { return p < cut ? 0 : (p
   ↪ == cut ? -1 : 1); };
    for (int i = 2; i <= n; i++) {
        int s1 = side(S.sa[i]), s2 = side(S.sa[i -
   ↪ 1]);
        if (s1 >= 0 && s2 >= 0 && s1 != s2) best =
   ↪ max(best, S.lcp[i]);
    }
    return best;
}
// 5) K-TH SMALLEST DISTINCT SUBSTRING: SA entry i
   ↪ contributes (n - sa[i]) - lcp[i] new
//    substrings, all sharing the prefix of length
   ↪ lcp[i]. Prefix-sum those counts and binary
//    search for k; inside the block the answer is a
   ↪ prefix of suffix sa[i].
// 6) LCS OF K STRINGS: concatenate with distinct
   ↪ separators, then slide a window over the SA
//    that contains at least one suffix of every
   ↪ string; the answer is the max over windows of
//    the minimum lcp inside (monotonic deque), O(n)
   ↪ after the SA.
// 7) COMPARE TWO SUBSTRINGS in O(1): let l =
   ↪ S.lce(i, j) (STRING positions - queryLcp takes
//    SUFFIX-ARRAY indices, do not mix them up); if l
   ↪ >= both
//    lengths they are equal, otherwise compare
   ↪ s[i+l] with s[j+l].
```

7.4.4 GeneralizedSAM

GENERALIZED SUFFIX AUTOMATON - one automaton accepting every substring of a SET of strings.

```
// Same size bound O(total length). The only change
   ↪ vs a plain SAM: when extending, the transition
// may ALREADY exist (a previous string walked here),
   ↪ so handle that case instead of blindly adding.
// Feeding strings one after another into a plain SAM
   ↪ is WRONG - it creates junk states.
struct GenSAM {
    struct S { int len = 0, link = -1; array<int, 26>
   ↪ nxt; S() { nxt.fill(-1); } };
    vector<S> st;
    GenSAM() { st.push_back(S()); }
    int clone(int q, int len) { int c = st.size();
   ↪ st.push_back(st[q]), st[c].len = len; return c; }
    int extend(int last, int c) {
   ↪          // returns the new "last"
        if (st[last].nxt[c] != -1) {
   ↪          // the edge is already there
            int q = st[last].nxt[c];
            if (st[q].len == st[last].len + 1) return
   ↪ q;
            int cl = clone(q, st[last].len + 1);
            for (int p = last; p != -1 &&
   ↪ st[p].nxt[c] == q; p = st[p].link) st[p].nxt[c]
   ↪ = cl;
            st[q].link = cl;
            return cl;
        }
        int cur = st.size();
        st.push_back(S()), st[cur].len = st[last].len
   ↪ + 1;
        int p = last;
        while (p != -1 && st[p].nxt[c] == -1)
   ↪ st[p].nxt[c] = cur, p = st[p].link;
        if (p == -1) st[cur].link = 0;
        else {
            int q = st[p].nxt[c];
            if (st[p].len + 1 == st[q].len)
   ↪ st[cur].link = q;
            else {
                int cl = clone(q, st[p].len + 1);
                for (; p != -1 && st[p].nxt[c] == q;
   ↪ p = st[p].link) st[p].nxt[c] = cl;
                st[q].link = st[cur].link = cl;
            }
        }
        return cur;
    }
    void add(const string& s) { int last = 0; for
   ↪ (char ch : s) last = extend(last, ch - 'a'); }
    vector<int> byLen() const {
   ↪          // states in increasing len
        int n = st.size(), mx = 0;
        for (auto& x : st) mx = max(mx, x.len);
        vector<int> c(mx + 2, 0), ord(n);
        for (auto& x : st) c[x.len]++;
        for (int i = 1; i <= mx; i++) c[i] += c[i -
```

```
    ↪ 1];
        for (int v = n - 1; v >= 0; v--)
    ↪ ord[--c[st[v].len]] = v;
        return ord;
    }
    // HOW MANY OF THE k STRINGS CONTAIN each
    ↪ substring: mark the states each string walks
    ↪ through,
    // then push the marks up the link tree. k <= 64
    ↪ here; for larger k use small-to-large sets, or
    // a bitset<K>, or process the strings in batches
    ↪ of 64 and add up the popcounts.
    vector<int> countStringsContaining(const
    ↪ vector<string>& v) {   // requires v.size() <= 64
        vector<unsigned long long> mask(st.size(), 0);
        for (int i = 0; i < (int)v.size(); i++)
            for (int p = 0, j = 0; j <
    ↪ (int)v[i].size(); j++)
                p = st[p].nxt[v[i][j] - 'a'], mask[p]
    ↪ |= 1ULL << i;
        auto ord = byLen();
        for (int i = ord.size() - 1; i > 0; i--)
            mask[st[ord[i]].link] |= mask[ord[i]];
        vector<int> r(st.size());
        for (size_t i = 0; i < st.size(); i++) r[i] =
    ↪ __builtin_popcountll(mask[i]);
        return r;
    ↪          // r[v] for every state v
    }
};
// LONGEST COMMON SUBSTRING OF k STRINGS: build the
    ↪ generalized SAM, take the deepest state with
// count == k; its longest string is len[v] (walk
    ↪ suffix links for the actual string).
// DISTINCT SUBSTRINGS OF THE UNION = sum over v != 0
    ↪ of (len[v] - len[link[v]]), same formula.
// SUBSTRING PRESENT IN EXACTLY ONE STRING / IN AT
    ↪ LEAST k: filter by the same counts.
// ALTERNATIVE when the strings are few: concatenate
    ↪ with distinct separators and use one SUFFIX
// ARRAY + a sliding window over lcp. Shorter to
    ↪ write, O(n log n), but no online extension.
```

7.4.5 SuffixArraySAIS

SUFFIX ARRAY IN O(n) - SA-IS. Use it instead of 01 - SuffixArray.cpp (O(n log n) doubling)

```
// when n is 1e6 or more, when the input is a
    ↪ vector<int> with a huge alphabet (doubling needs
    ↪ a
// radix sort per round), or when the doubling
    ↪ version is the bottleneck of a heavy solution.
// It is ~4x faster than doubling at n = 1e6 and
    ↪ needs no sorting of pairs at all.
// INDEX CONVENTION - DIFFERENT FROM 01 -
    ↪ SuffixArray.cpp, and mixing them up is a wrong
    ↪ answer:
//   sa has EXACTLY n entries (no empty suffix);
    ↪ sa[i] = start of the i-th smallest suffix.
//   lcp has n entries with lcp[0] = 0 and lcp[i] =
    ↪ LCP(sa[i-1], sa[i]).
// HOW IT WORKS (enough to debug it): classify every
    ↪ position as S-type (its suffix is smaller
// than the next one) or L-type. The LMS positions
    ↪ (an S right after an L) split the string into
// blocks; sort those blocks by recursing on a
    ↪ shorter string, then INDUCE the whole order from
// them with two linear scans. The recursion halves
    ↪ the length, so the total is O(n).
vector<int> saIs(const vector<int>& s, int up) {
    ↪          // values in [0, up], up >= 0
    int n = s.size();
    if (n == 0) return {};
    if (n == 1) return {0};
    if (n == 2) return s[0] < s[1] ? vector<int>{0,
    ↪ 1} : vector<int>{1, 0};
    vector<int> sa(n);
    vector<char> ls(n, 0);
    ↪          // 1 = S-type
    for (int i = n - 2; i >= 0; i--) ls[i] = s[i] ==
    ↪ s[i + 1] ? ls[i + 1] : s[i] < s[i + 1];
    vector<int> sumL(up + 2, 0), sumS(up + 2, 0);
    for (int i = 0; i < n; i++) ls[i] ? sumL[s[i] +
    ↪ 1]++ : sumS[s[i]]++;
    for (int i = 0; i <= up; i++) { sumS[i] +=
    ↪ sumL[i]; if (i < up) sumL[i + 1] += sumS[i]; }
    auto induce = [&](const vector<int>& lms) {
        fill(sa.begin(), sa.end(), -1);
        vector<int> buf = sumS;
        for (int d : lms) if (d != n) sa[buf[s[d]]++]
    ↪ = d;
        buf = sumL;
        sa[buf[s[n - 1]]++] = n - 1;
        for (int i = 0; i < n; i++) {
            int v = sa[i];
            if (v >= 1 && !ls[v - 1]) sa[buf[s[v -
    ↪ 1]]++] = v - 1;
        }
        buf = sumL;
        for (int i = n - 1; i >= 0; i--) {
            int v = sa[i];
            if (v >= 1 && ls[v - 1]) sa[--buf[s[v -
    ↪ 1] + 1]] = v - 1;
        }
    };
    vector<int> lmsMap(n + 1, -1), lms;
    int m = 0;
    for (int i = 1; i < n; i++) if (!ls[i - 1] &&
    ↪ ls[i]) lmsMap[i] = m++, lms.push_back(i);
    induce(lms);
    if (m) {
        vector<int> sorted;
        for (int v : sa) if (lmsMap[v] != -1)
    ↪ sorted.push_back(v);
        vector<int> rec(m);
        int upRec = 0;
        rec[lmsMap[sorted[0]]] = 0;
        for (int i = 1; i < m; i++) {
            int l = sorted[i - 1], r = sorted[i];
            int el = lmsMap[l] + 1 < m ?
    ↪ lms[lmsMap[l] + 1] : n;
            int er = lmsMap[r] + 1 < m ?
    ↪ lms[lmsMap[r] + 1] : n;
            bool same = el - l == er - r;
            if (same) {
                while (l < el && s[l] == s[r]) l++,
    ↪ r++;
                if (l == n || s[l] != s[r]) same =
    ↪ false;
            }
            if (!same) upRec++;
            rec[lmsMap[sorted[i]]] = upRec;
        }
        vector<int> recSa = saIs(rec, upRec);
        for (int i = 0; i < m; i++) sorted[i] =
    ↪ lms[recSa[i]];
        induce(sorted);
```

```
    }
    return sa;
}
vector<int> suffixArray(const string& s) {
  ↪          // n entries, no empty suffix
    vector<int> a(s.begin(), s.end());
    for (int& x : a) x = (unsigned char)x;
    return saIs(a, 255);
}
// KASAI: lcp[i] = LCP(suffix sa[i-1], suffix sa[i]),
  ↪ lcp[0] = 0. O(n).
vector<int> kasai(const string& s, const vector<int>&
  ↪ sa) {
    int n = s.size(), k = 0;
    vector<int> rnk(n), lcp(n, 0);
    for (int i = 0; i < n; i++) rnk[sa[i]] = i;
    for (int i = 0; i < n; i++) {
        if (!rnk[i]) { k = 0; continue; }
        int j = sa[rnk[i] - 1];
        while (i + k < n && j + k < n && s[i + k] ==
  ↪ s[j + k]) k++;
        lcp[rnk[i]] = k;
        if (k) k--;
    }
    return lcp;
}
/* NOTES
 GENERAL ALPHABETS: call saIs directly on a
  ↪ vector<int> after coordinate-compressing the
  ↪ values
   to [0, up]; every value must be >= 0 and the LAST
  ↪ element does NOT have to be a sentinel -
   this implementation handles the end of string
  ↪ internally.
 CONCATENATING MANY STRINGS (generalised suffix
  ↪ array): join with distinct separators SMALLER
   than every real character, or - better with SA-IS
  ↪ - compress to ints and use 0, 1, 2, ... as
   separators, shifting real characters up.
 EVERYTHING ELSE (distinct substrings, longest
  ↪ repeated, pattern range, LCE sparse table) is in
   03 - SuffixArrayApplications.cpp; shift its
  ↪ formulas by one because that file's sa[0] is the
   empty suffix and this one has no empty suffix.
 MEMORY is ~3n ints at the top level plus the
  ↪ recursion, which sums to < 2x that. Doubling
  ↪ needs
   5 arrays of n and a radix sort - SA-IS also wins
  ↪ on memory.
 DC3 / SKEW is the other linear construction; it is
  ↪ shorter to write but slower in practice and
   needs 3n extra memory, so SA-IS is the one to
  ↪ carry. */
```

7.4.6 IsomorphicSuffixArray

PARAMETERIZED / ISOMORPHIC SUFFIX ARRAY. Two strings are ISOMORPHIC (p-match) when some

```
// Needs: 07 - Strings/04 - Suffix Structures/05 -
  ↪ SuffixArraySAIS.cpp   (saIs, kasai)
// BIJECTION of the alphabet turns one into the
  ↪ other: "abab" ~ "cdcd" ~ "baba", but not "aabb".
// WHEN: "how many substrings of s are isomorphic to
  ↪ s[l..r]", detecting renamed patterns,
// register-allocation style matching. Plain hashing
  ↪ cannot do this - the equality is not on
// characters but on the PATTERN OF REPEATS.
// THE ENCODING that makes it work: a[i] = (i -
  ↪ previous position of s[i]) + 1, or 1 when s[i] is
// the first occurrence of its character. Two
  ↪ substrings are isomorphic iff their encodings
  ↪ match
// AFTER CAPPING: at offset L inside a substring the
  ↪ value is a[x] if a[x] <= L + 1, else 1 (a
// reference that points outside the window becomes
  ↪ "first occurrence"). The cap is why this is
// not just an ordinary suffix array on a[].
// INDEX CONVENTION: sa has n entries, no empty
  ↪ suffix; sa[i] = start of the i-th smallest suffix
// UNDER ISOMORPHISM, and lcp[i] = longest common
  ↪ ISOMORPHIC prefix of sa[i] and sa[i+1] (note:
// indexed by the LEFT of the pair, unlike 01 -
  ↪ SuffixArray.cpp - one entry per adjacent pair).
// COMPLEXITY: O(n log n * A) worst case (A =
  ↪ alphabet), because one comparison may restart at
// every distinct character; in practice each
  ↪ comparison is a couple of jumps.
struct IsoSuffixArray {
    int n, LG;
    vector<int> a, sa, rnk, lcp, hsa, hrnk, hlcp, lg;
    vector<vector<int>> sp;
  ↪          // sparse table over hlcp
    IsoSuffixArray(const string& s, int A = 26, char
  ↪ base = 'a') {
        n = s.size(), a.resize(n);
        vector<int> last(A, -1);
        for (int i = 0; i < n; i++)
            a[i] = last[s[i] - base] < 0 ? 1 : i -
  ↪ last[s[i] - base] + 1, last[s[i] - base] = i;
        hsa = saIs(a, n + 1);
  ↪          // helper: plain SA of the encoding
        hrnk.resize(n);
        for (int i = 0; i < n; i++) hrnk[hsa[i]] = i;
        hlcp.assign(n, 0);
        for (int i = 0, k = 0; i < n; i++) {
  ↪          // Kasai on the integer array
            if (!hrnk[i]) { k = 0; continue; }
            int j = hsa[hrnk[i] - 1];
            while (i + k < n && j + k < n && a[i + k]
  ↪ == a[j + k]) k++;
            hlcp[hrnk[i]] = k;
            if (k) k--;
        }
        lg.assign(n + 1, 0);
        for (int i = 2; i <= n; i++) lg[i] = lg[i /
  ↪ 2] + 1;
        LG = lg[n] + 1;
        sp.assign(LG, vector<int>(n, 0));
        sp[0] = hlcp;
        for (int k = 1; k < LG; k++)
            for (int i = 0; i + (1 << k) <= n; i++)
                sp[k][i] = min(sp[k - 1][i], sp[k -
  ↪ 1][i + (1 << (k - 1))]);
        sa.resize(n), iota(sa.begin(), sa.end(), 0);
        sort(sa.begin(), sa.end(), [&](int i, int j)
  ↪ { return cmp(i, j); });
        rnk.resize(n);
        for (int i = 0; i < n; i++) rnk[sa[i]] = i;
        lcp.assign(max(0, n - 1), 0);
        for (int i = 0; i + 1 < n; i++) lcp[i] =
  ↪ lce(sa[i], sa[i + 1]);
    }
    int rawLce(int i, int j) {
  ↪          // lcp of a[i..] and a[j..]
        if (i == j) return n - i;
        int l = hrnk[i], r = hrnk[j];
        if (l > r) swap(l, r);
```

```
        int k = lg[r - l];
        return min(sp[k][l + 1], sp[k][r - (1 << k) +
    ↪ 1]);
    }
    int lce(int i, int j) {
    ↪           // longest common ISOMORPHIC prefix
        for (int L = 0;;) {
            int x = i + L, y = j + L;
            if (x >= n || y >= n) return L;
            int vx = a[x] > L + 1 ? 1 : a[x], vy =
    ↪ a[y] > L + 1 ? 1 : a[y];
            if (vx != vy) return L;
            L += max(1, rawLce(x, y));
    ↪           // raw equality implies iso equality
        }
    }
    bool cmp(int i, int j) {
        int L = lce(i, j);
        if (i + L >= n) return true;
        if (j + L >= n) return false;
        int vx = a[i + L] > L + 1 ? 1 : a[i + L], vy
    ↪ = a[j + L] > L + 1 ? 1 : a[j + L];
        return vx < vy;
    }
    // Number of substrings isomorphic to s[p ..
    ↪ p+len-1]: the SA block of suffixes whose
    // isomorphic prefix of that length matches.
    ↪ O(log^2 n) with the lcp array as a min-sparse
    // table; the linear scan below is fine when len
    ↪ is large or queries are few.
    pii occurrences(int p, int len) {
        int lo = rnk[p], hi = rnk[p];
        while (lo > 0 && lcp[lo - 1] >= len) lo--;
        while (hi + 1 < n && lcp[hi] >= len) hi++;
        return {lo, hi};
    ↪           // count = hi - lo + 1
    }
};
/* NOTES
 THE OTHER ENCODINGS you may meet: "previous
  ↪ occurrence index" (0 if none) is the same idea;
  "order of first appearance" (abcab -> 01201) is
  ↪ NOT enough for substrings, because a substring
  restarts the numbering - that is exactly what the
  ↪ cap fixes here.
 HASHING ALTERNATIVE: hash the capped encoding with a
  ↪ segment tree over "distance to previous
  occurrence" and answer isomorphism of two
  ↪ substrings in O(log n) - shorter if you only need
  EQUALITY tests and not an order.
 STRUCTURAL / ORDER-ISOMORPHISM (the same RELATIVE
  ↪ ORDER of numbers, not equality pattern) is a
  different problem: encode each position by (rank
  ↪ among the previous window) and use a similar
  suffix structure, or hash the pair (previous
  ↪ smaller, previous larger) positions.
 THE COMPARATOR IS NOT TRANSITIVE-SAFE if the cap is
  ↪ wrong; test on "aabb" vs "bbaa" (isomorphic)
  and "aabb" vs "abab" (not) before trusting a
  ↪ modified version. */
```

7.4.7 DistinctSubstringsInRange

NUMBER OF DISTINCT SUBSTRINGS OF s[l..r], for q queries, OFFLINE in O((n + q) log n).

```
// Needs: 07 - Strings/04 - Suffix Structures/02 -
  ↪ SuffixAutomaton.cpp
// For the whole string this is one line off a suffix
  ↪ automaton or a suffix array; for an
// arbitrary WINDOW it is a genuinely hard problem,
  ↪ and this is the standard solution
// (HackerRank "How Many Substrings", CF gym 102129I).
// INDEX CONVENTION: 0-based inclusive queries {l,
  ↪ r}; internally everything is 1-based over
// positions 1..n, and LCT node id = SAM state id + 1
  ↪ (id 0 is the null node).
// THE INVARIANT: sweep r from left to right and
  ↪ keep, in a Fenwick with range add and range sum,
//     val[p] = number of distinct substrings whose
  ↪ LAST occurrence in s[1..r] STARTS at p.
// Then the answer for [l, r] is sum of val[l..r] -
  ↪ every distinct substring of the window is
// counted at the start position of its last
  ↪ occurrence, which is inside the window exactly
  ↪ when
// the substring occurs in the window at all.
// MAINTAINING IT is the trick: appending s[r]
  ↪ creates r new occurrences (the suffixes of the
// prefix), so add +1 to every position 1..r, and
  ↪ then REMOVE the previous last-occurrence entries
// of exactly those substrings. Those substrings are
  ↪ the ones on the suffix-link path from the
// state of prefix r to the root - and their old
  ↪ "last occurrence" positions are constant along
// each PREFERRED PATH of a link-cut tree. So
  ↪ maintain the suffix-link tree in an LCT where
// access(state, r) PAINTS the path with the colour
  ↪ r: every preferred-child change reports one
// maximal group (a range of lengths that shared an
  ↪ old colour), which is one range update.
// That "access as painting" pattern is reusable: any
  ↪ "assign a colour to a root path, and pay
// only for the O(log n) amortised colour groups you
  ↪ destroy" problem is this code.
struct DistinctSubstringsInRange {
    int n = 0, sz = 0;
    vector<int> par, lz, col, len;
    vector<array<int, 2>> ch;
    vector<pii> mods;
  ↪           // {segment top len, old colour}
    vector<ll> b1, b2;
  ↪           // range-add range-sum Fenwick
    void bAdd(vector<ll>& b, int i, ll v) { for (; i
  ↪ <= n; i += i & -i) b[i] += v; }
    ll bSum(const vector<ll>& b, int i) const {
        ll r = 0;
        for (; i > 0; i -= i & -i) r += b[i];
        return r;
    }
    void rangeAdd(int l, int r, ll v) {
  ↪           // add v to positions [l, r]
        if (l > r) return;
        bAdd(b1, l, v), bAdd(b2, l, v * (l - 1));
        if (r + 1 <= n) bAdd(b1, r + 1, -v), bAdd(b2,
  ↪ r + 1, -v * r);
    }
    ll pref(int i) const { return bSum(b1, i) * i -
  ↪ bSum(b2, i); }
    ll rangeSum(int l, int r) const { return l > r ?
  ↪ 0 : pref(r) - pref(l - 1); }
```

```
    void mark(int x, int v) { lz[x] = col[x] = v; }
    void push(int x) {
        if (!lz[x]) return;
        if (ch[x][0]) mark(ch[x][0], lz[x]);
        if (ch[x][1]) mark(ch[x][1], lz[x]);
        lz[x] = 0;
    }
    bool isRoot(int x) { return ch[par[x]][0] != x &&
↪ ch[par[x]][1] != x; }
    void rotate(int x) {
        int y = par[x], z = par[y], k = ch[y][1] ==
↪ x, w = ch[x][!k];
        if (!isRoot(y)) ch[z][ch[z][1] == y] = x;
        ch[x][!k] = y, ch[y][k] = w;
        if (w) par[w] = y;
        par[y] = x, par[x] = z;
    }
    void splay(int x) {
        static vector<int> st;
        st.clear(), st.push_back(x);
        for (int i = x; !isRoot(i); i = par[i])
↪ st.push_back(par[i]);
        while (!st.empty()) push(st.back()),
↪ st.pop_back();
        while (!isRoot(x)) {
            int y = par[x], z = par[y];
            if (!isRoot(y)) rotate((ch[y][1] == x) ==
↪ (ch[z][1] == y) ? y : x);
            rotate(x);
        }
    }
    void access(int x, int v) {
↪           // paint root..x with colour v
        mods.clear();
        for (int z = 0; x; z = x, x = par[x]) {
            splay(x);
            mods.push_back({len[x - 1], col[x]});
↪             // this preferred group dies now
            ch[x][1] = z;
            mark(x, v);
        }
    }
    vector<ll> solve(const string& s, const
↪ vector<pii>& queries) {
        n = s.size();
        SAM sam(n);
        vector<int> pos(n + 1, 1);
        for (int i = 0; i < n; i++) sam.extend(s[i]),
↪ pos[i + 1] = sam.last + 1;
        sz = sam.st.size();
        par.assign(sz + 1, 0), lz.assign(sz + 1, 0),
↪ col.assign(sz + 1, 0);
        ch.assign(sz + 1, {0, 0}), len.assign(sz, 0);
        for (int i = 0; i < sz; i++) len[i] =
↪ sam.st[i].len;
        for (int i = 1; i <= sz; i++) par[i] =
↪ sam.st[i - 1].link + 1;
        b1.assign(n + 2, 0), b2.assign(n + 2, 0);
        vector<vector<pii>> byR(n + 1);
↪             // {l, query id}, 1-based r
        for (int i = 0; i < (int)queries.size(); i++)
            byR[queries[i].second +
↪ 1].push_back({queries[i].first + 1, i});
        vector<ll> ans(queries.size(), 0);
        for (int r = 1; r <= n; r++) {
            rangeAdd(1, r, 1);
↪             // every suffix of s[1..r] is new
            access(pos[r], r);
            int prevLen = 0;
            for (int j = (int)mods.size() - 1; j >=
↪ 1; j--) {  // root-most group first
                auto [topLen, oldCol] = mods[j];
                if (!topLen) continue;
↪               // the root state (empty string)
                if (oldCol) rangeAdd(oldCol - topLen
↪ + 1, oldCol - prevLen, -1);
                prevLen = topLen;
            }
            for (auto [l, id] : byR[r]) ans[id] =
↪ rangeSum(l, r);
        }
        return ans;
    }
};
/* NOTES
 WHY THE LENGTHS LINE UP: a preferred group is a set
↪ of suffix-link-tree nodes whose strings all
  had the same last-occurrence end position
↪ ‘oldCol‘, and the group covers exactly the
↪ lengths
  (prevLen, topLen]. Those substrings therefore
↪ started at positions oldCol - topLen + 1 ..
  oldCol - prevLen, which is the single range you
↪ cancel. Get the direction of prevLen wrong and
  the answers drift slowly - test against a brute
↪ force on n <= 12 before using it.
 THE SAM IS BUILT FIRST, in full, and the LCT is
↪ built over the FINAL suffix-link tree; no links
  or cuts ever happen, only access(). Clones created
↪ later do not break the sweep because the
  path from the state of prefix r to the root always
↪ lists exactly the suffixes of that prefix.
 COMPLEXITY: O(n log n) amortised for the accesses
↪ (the number of preferred-child changes is
  O(n log n)), each producing one O(log n) Fenwick
↪ range update.
 ONLINE variant: replace the Fenwick with a
↪ PERSISTENT one, keeping a root per r; queries
↪ then
  need no sorting. Memory becomes O(n log^2 n).
 RELATED, MUCH EASIER: distinct substrings of the
↪ WHOLE string (sum of len - len[link] over SAM
  states), and distinct VALUES in a range (the
↪ classic BIT-over-last-occurrence sweep, which is
  this same idea with one position per value instead
↪ of a whole path). */
```

7.5 Palindromes

7.5.1 Manacher

MANACHER - the radius of the longest palindrome centred at every position, in O(n).

```
// Answers "is s[l..r] a palindrome" in O(1) and
↪ counts all palindromic substrings in one pass.
struct Manacher {
    vector<int> p;

    // Returns a vector where p[i] is the radius of
↪ the longest palindrome around center i
    // Centers are interleaved: 0 is before string, 1
↪ is first char, 2 is between 1st & 2nd, etc.
    Manacher(string s) {
        string t = "#";
        for (char c : s) {
            t += c;
            t += "#";
```

```
        }
        int n = t.size();
        p.assign(n, 0);
        int c = 0, r = 0;
        for (int i = 0; i < n; i++) {
            int mirror = 2 * c - i;
            if (i < r) p[i] = min(r - i, p[mirror]);
            while (i - 1 - p[i] >= 0 && i + 1 + p[i]
↪ < n && t[i - 1 - p[i]] == t[i + 1 + p[i]]) {
                p[i]++;
            }
            if (i + p[i] > r) {
                c = i;
                r = i + p[i];
            }
        }
    }

    // Check if substring [l, r] (0-based) is a
↪ palindrome in O(1)
    bool is_palindrome(int l, int r) {
        int len = r - l + 1;
        int center = l + r + 1;
        return p[center] >= len;
    }
};
```

7.5.2 PalindromicTree

PALINDROMIC TREE (eertree) - O(n) nodes, one per DISTINCT palindromic substring, built online in

```
// O(n). Gives the count of distinct palindromes,
↪ occurrence counts, and the longest palindromic
// suffix at every prefix - which is what
↪ palindromic-factorization DP needs.
struct PalindromeTree {
    int n, id, cur, tot;
    vector<array<int, 26>> go;

    // Core structural links
    vector<int> suflink, len, cnt;

    // Extentions for Hard DP (Palindrome
↪ Partitioning)
    vector<int> diff;   // Difference in length to
↪ the suffix link
    vector<int> slink;  // Series link (skips
↪ arithmetic progressions of suffix links)

    // Extensions for counting
    vector<int> num;    // Number of palindromic
↪ suffixes ending at this node
    long long total_overlapping; // Running sum of
↪ total palindromes

    PalindromeTree() {}

    PalindromeTree(const string &s) {
        build(s);
    }

    void init(int max_len) {
        n = max_len;
        go.assign(n + 2, {});
        suflink.assign(n + 2, 0);
        len.assign(n + 2, 0);
        cnt.assign(n + 2, 0);
        diff.assign(n + 2, 0);
        slink.assign(n + 2, 0);
        num.assign(n + 2, 0);

        suflink[0] = suflink[1] = 1;
        len[1] = -1;
        id = 2;
        cur = 0;
        tot = 0;
        total_overlapping = 0;
    }

    int get(const string &s, int i, int v) {
        while (i - len[v] - 1 < 0 || s[i - len[v] -
↪ 1] != s[i]) {
            v = suflink[v];
        }
        return v;
    }

    void add(const string &s, int i) {
        int ch = s[i] - 'a';
        cur = get(s, i, cur);

        if (go[cur][ch] == 0) {
            len[id] = 2 + len[cur];
            suflink[id] = go[get(s, i,
↪ suflink[cur])][ch];

            // 1. Number of palindromes ending
↪ exactly here
            num[id] = num[suflink[id]] + 1;

            // 2. Arithmetic Progression / Series
↪ Links for DP
            diff[id] = len[id] - len[suflink[id]];
            if (diff[id] == diff[suflink[id]]) {
                slink[id] = slink[suflink[id]];
            } else {
                slink[id] = suflink[id];
            }

            tot++;
            go[cur][ch] = id++;
        }
        cur = go[cur][ch];
        cnt[cur]++;
        total_overlapping += num[cur];
    }

    // Standard build
    void build(const string &s) {
        init(s.length());
        for (int i = 0; i < n; i++) {
            add(s, i);
        }
        countAll();
    }

    // Call this to push frequencies down the suffix
↪ links
    void countAll() {
        for (int i = id - 1; i >= 2; --i) {
            cnt[suflink[i]] += cnt[i];
        }
    }

    // --- Black-Box Solvers ---

    // 1. Returns number of unique palindromes
```

```
    int cntDistinct() {
        return tot;
    }

    // 2. Returns total number of palindromes
↪ (including overlaps/duplicates)
    long long cntTotal() {
        return total_overlapping;
    }

    // 3. Solves Minimum Palindrome Partitioning in
↪ O(N log N)
    // Rebuilds the tree internally to compute the DP
↪ concurrently.
    int min_partitions(const string &s) {
        init(s.length());
        const int INF = 1e9;
        vector<int> dp(s.length() + 1, INF);
        vector<int> series_ans(s.length() + 2, INF);

        dp[0] = 0;

        for (int i = 0; i < s.length(); ++i) {
            add(s, i);

            for (int v = cur; v > 1; v = slink[v]) {
                series_ans[v] = dp[i + 1 -
↪ (len[slink[v]] + diff[v])];

                if (diff[v] == diff[suflink[v]]) {
                    series_ans[v] =
↪ min(series_ans[v], series_ans[suflink[v]]);
                }

                dp[i + 1] = min(dp[i + 1],
↪ series_ans[v] + 1);
            }
        }
        return dp[s.length()];
    }

    // 4. Returns an array of frequencies for string
↪ B's palindromes inside this tree (String A)
    // Note: You must `build()` the tree with string
↪ A before calling this with B.
    vector<int> count_string(const string& B) {
        vector<int> out_cnt(id, 0);
        int curr = 0;

        for (int i = 0; i < B.length(); ++i) {
            int ch = B[i] - 'a';

            while (true) {
                if (i - len[curr] - 1 >= 0 && B[i -
↪ len[curr] - 1] == B[i]) {
                    if (go[curr][ch]) {
                        curr = go[curr][ch];
                        break;
                    }
                }
                if (curr == 1) {
                    curr = 0;
                    break;
                }
                curr = suflink[curr];
            }
            out_cnt[curr]++;
        }

        for (int i = id - 1; i >= 2; --i) {
            out_cnt[suflink[i]] += out_cnt[i];
        }

        return out_cnt;
    }
};
```

7.5.3 PalindromicTreeRollback

PALINDROMIC TREE WITH ROLLBACK - same structure, but every extend() can be undone, so you can DFS

```
// NOTE: 02 - PalindromicTree.cpp declares the same
  ↪ 'struct PalindromeTree'. Paste one.
// a trie/tree of characters and maintain the
  ↪ palindromic structure of the current
  ↪ root-to-node string.
struct PalindromeTree {
    int n, id, cur, tot;
    vector<array<int, 26>> go;

    // Core structural links
    vector<int> suflink, len, cnt;

    // Extensions for Hard DP (Palindrome
  ↪ Partitioning)
    vector<int> diff;   // Length difference to
  ↪ suffix link
    vector<int> slink;  // Series link

    // Extensions for counting
    vector<int> num;    // Number of palindromic
  ↪ suffixes ending at this node
    long long total_overlapping; // Running total of
  ↪ all palindromic substrings

    // --- ROLLBACK EXTENSIONS ---
    string S;           // Maintains current active
  ↪ string
    struct History {
        int old_cur;
        int added_node;  // ID of created node, or 0
  ↪ if edge already existed
        int parent_node; // Node where
  ↪ go[parent_node][ch] was updated
        int ch;          // Character index (0..25)
    };
    vector<History> history;

    PalindromeTree() {}

    PalindromeTree(int max_len) {
        init(max_len);
    }

    void init(int max_len) {
        n = max_len;
        go.assign(n + 2, {});
        suflink.assign(n + 2, 0);
        len.assign(n + 2, 0);
        cnt.assign(n + 2, 0);
        diff.assign(n + 2, 0);
        slink.assign(n + 2, 0);
        num.assign(n + 2, 0);

        suflink[0] = suflink[1] = 1;
        len[1] = -1;
        id = 2;
```

```
        cur = 0;
        tot = 0;
        total_overlapping = 0;

        S.clear();
        history.clear();
    }

    int get(int i, int v) {
        while (i - len[v] - 1 < 0 || S[i - len[v] -
↪ 1] != S[i]) {
            v = suflink[v];
        }
        return v;
    }

    // 1. PUSH / ADD CHARACTER (WITH ROLLBACK HISTORY)
    void push_back(char c) {
        S.push_back(c);
        int i = (int)S.length() - 1;
        int ch = c - 'a';

        int old_cur = cur;
        cur = get(i, cur);

        int added_node = 0;
        int parent_node = 0;

        if (go[cur][ch] == 0) {
            added_node = id;
            parent_node = cur;

            len[id] = 2 + len[cur];
            suflink[id] = go[get(i,
↪ suflink[cur])][ch];

            num[id] = num[suflink[id]] + 1;

            diff[id] = len[id] - len[suflink[id]];
            if (diff[id] == diff[suflink[id]]) {
                slink[id] = slink[suflink[id]];
            } else {
                slink[id] = suflink[id];
            }

            tot++;
            go[cur][ch] = id++;
        }

        cur = go[cur][ch];
        cnt[cur]++;
        total_overlapping += num[cur];

        // Save state transition into history stack
        history.push_back({old_cur, added_node,
↪ parent_node, ch});
    }

    // 2. POP / ROLLBACK LAST CHARACTER
    void pop_back() {
        if (history.empty()) return;

        History h = history.back();
        history.pop_back();

        // Revert occurrence count & running
↪ palindrome count
        total_overlapping -= num[cur];
        cnt[cur]--;

        // If a new node was created during this
↪ push_back, delete it
        if (h.added_node != 0) {
            int u = h.added_node;
            go[h.parent_node][h.ch] = 0; //
↪ Disconnect child edge

            // Reset node u's properties
            go[u].fill(0);
            suflink[u] = len[u] = cnt[u] = diff[u] =
↪ slink[u] = num[u] = 0;

            id--;
            tot--;
        }

        // Restore previous active node and string
↪ length
        cur = h.old_cur;
        S.pop_back();
    }

    // --- Black-Box Solvers ---

    int cntDistinct() {
        return tot;
    }

    long long cntTotal() {
        return total_overlapping;
    }

    // Call this if you need to push end-occurrence
↪ counts down suffix links
    void countAll() {
        for (int i = id - 1; i >= 2; --i) {
            cnt[suflink[i]] += cnt[i];
        }
    }
};
```

7.5.4 PalindromicFactorization

COUNTING PALINDROMIC FACTORISATIONS - the number of ways to cut s into palindromic pieces,

```
// Needs: 07 - Strings/05 - Palindromes/02 -
↪ PalindromicTree.cpp
// in O(n log n). The tree file already has
↪ min_partitions (the MINIMUM number of pieces);
↪ this is
// the same series-link machinery with (min, +1)
↪ replaced by (sum, *1), plus the even-length
// variant that CF 932G needs. Doing it naively is
↪ O(n^2): "for each i, for each palindromic
// suffix" - and a string like a^n has n palindromic
↪ suffixes at every position.
// WHY IT IS O(n log n): the palindromic suffixes of
↪ any prefix fall into O(log n) ARITHMETIC
// PROGRESSIONS by length (Fine and Wilf). diff[v] =
↪ len[v] - len[link[v]] is the common
// difference of v's progression and slink[v] jumps
↪ to the previous progression, so walking
// v = cur, slink[v], slink[slink[v]], ... visits
↪ O(log n) nodes. series[v] aggregates the whole
// progression of v in O(1) by reusing
↪ series[link[v]] from the PREVIOUS position -
↪ that reuse is
```

```
// the entire trick and is why series[] must be indexed by NODE and not by position.
// INDEX CONVENTION: dp is 1-indexed over prefixes, dp[i] = answer for s[0..i-1], dp[0] = 1.
ll countPalindromicFactorizations(const string& s, bool evenLengthPiecesOnly = false) {
    int n = s.size();
    PalindromeTree t;
    t.init(n);
    vector<ll> dp(n + 1, 0), series(n + 2, 0);
    dp[0] = 1;
    for (int i = 0; i < n; i++) {
        t.add(s, i);
        for (int v = t.cur; t.len[v] > 0; v = t.slink[v]) {
            series[v] = dp[i + 1 - (t.len[t.slink[v]] + t.diff[v])];
            int sl = t.suflink[v];
            if (t.diff[v] == t.diff[sl]) series[v] = (series[v] + series[sl]) % MOD;
            dp[i + 1] = (dp[i + 1] + series[v]) % MOD;
        }
        if (evenLengthPiecesOnly && (i + 1) % 2) dp[i + 1] = 0;   // an odd prefix cannot be filled
    }
    return dp[n];
}
/* THE VARIANTS THIS UNLOCKS - all the same loop, only the semiring changes
 MINIMUM number of pieces: (min, +1) - already in 02 - PalindromicTree.cpp as min_partitions.
 NUMBER of factorisations: (sum, *1) - the code above.
 FACTORISATIONS INTO AN EVEN NUMBER OF PIECES, or with a parity constraint: carry two dp arrays
  dp[i][0], dp[i][1] and one series[] per parity, exactly as min_partition does for (even, odd).
 CF 932G "PALINDROME PARTITION" (split s into k pieces with p_1 = p_k, p_2 = p_{k-1}, ...):
  build t = s[0] s[n-1] s[1] s[n-2] ... and count factorisations of t into EVEN-LENGTH
  palindromes - that is the call with evenLengthPiecesOnly = true. n must be even.
 WEIGHTED pieces (cost per palindrome length): the series aggregate must then be a min/sum over
  the progression WITH the weight, which only stays O(1) if the weight is affine in the length -
  otherwise you are back to O(n^2) and should look for a different decomposition.
 SANITY: countPalindromicFactorizations("aaa") = 4 (aaa | a+aa | aa+a | a+a+a), and any string of
  distinct characters gives exactly 1. */
```

7.5.5 PalindromesInRange

HOW MANY PALINDROMIC SUBSTRINGS LIE ENTIRELY INSIDE s[l..r]? Offline, O((n + q) log n).

```
// Needs: 07 - Strings/05 - Palindromes/01 - Manacher.cpp
// The whole-string count is one Manacher pass; the RANGE version is the hard part, because a
// centre's contribution is CLIPPED by both ends of the query.
// INDEX CONVENTION (0-based, inclusive): with the interleaved radii p[] of 01 - Manacher.cpp,
//   d1[i] = (p[2i + 1] + 1) / 2 = number of odd palindromes centred AT i  (longest = 2*d1-1)
//   d2[i] = p[2i] / 2           = number of even palindromes centred BETWEEN i-1 and i
// Then the answer for [l, r] is
//   sum over i in [l, r]     of min(d1[i], i - l + 1, r - i + 1)
// + sum over i in [l+1, r]   of min(d2[i], i - l,  r - i + 1).
// SPLIT AT THE MIDPOINT so that only ONE of the two clips can bind: on the left half the binding
// clip is the distance to l, on the right half it is the distance to r. Each half becomes
//   sum over i in a range of ( key[i] >= T ? affine(i) : v[i] )
// with key[i] = v[i] - i (left) or v[i] + i (right) - a threshold sweep with three Fenwicks.
struct PalinRange {
    int n;
    vector<ll> d1, d2;
    PalinRange(const string& s) {
        Manacher M(s);
        n = s.size(), d1.assign(n, 0), d2.assign(n, 0);
        for (int i = 0; i < n; i++) d1[i] = (M.p[2 * i + 1] + 1) / 2, d2[i] = M.p[2 * i] / 2;
    }
    struct BIT {
        vector<ll> f;
        BIT(int n = 0) : f(n + 1, 0) {}
        void add(int i, ll v) { for (i++; i < (int)f.size(); i += i & -i) f[i] += v; }
        ll qr(int i) { ll r = 0; for (i++; i > 0; i -= i & -i) r += f[i]; return r; }
        ll rng(int l, int r) { return l > r ? 0 : qr(r) - qr(l - 1); }
    };
    // sum over i in [ql, qr] of ( key[i] >= T ? c0 + c1 * i : v[i] ), for many queries at once.
    vector<ll> sweep(const vector<ll>& v, const vector<ll>& key,
                     const vector<array<ll, 5>>& qs) {          // {ql, qr, T, c0, c1}
        int q = qs.size();
        vector<ll> res(q, 0), pre(n + 1, 0);
        for (int i = 0; i < n; i++) pre[i + 1] = pre[i] + v[i];
        vector<int> ord(n), qo(q);
        iota(ord.begin(), ord.end(), 0), iota(qo.begin(), qo.end(), 0);
        sort(ord.begin(), ord.end(), [&](int a, int b) { return key[a] > key[b]; });
        sort(qo.begin(), qo.end(), [&](int a, int b) { return qs[a][2] > qs[b][2]; });
        BIT cnt(n), sIdx(n), sVal(n);
        int ptr = 0;
        for (int id : qo) {
            auto [ql, qr, T, c0, c1] = qs[id];
            while (ptr < n && key[ord[ptr]] >= T) {
                int i = ord[ptr++];
                cnt.add(i, 1), sIdx.add(i, i), sVal.add(i, v[i]);
            }
            if (ql > qr) continue;
            ll c = cnt.rng(ql, qr), si = sIdx.rng(ql, qr), sv = sVal.rng(ql, qr);
            res[id] = c0 * c + c1 * si + (pre[qr + 1] - pre[ql] - sv);
        }
        return res;
    }
    vector<ll> solve(const vector<pii>& queries) {              // {l, r} 0-based inclusive
```

```
        int q = queries.size();
        vector<ll> km(n), kp(n), res(q, 0);
        vector<array<ll, 5>> a(q), b(q), c(q), d(q);
        for (int i = 0; i < n; i++) km[i] = d1[i] -
    ↪ i, kp[i] = d1[i] + i;
        for (int j = 0; j < q; j++) {
            ll l = queries[j].first, r =
    ↪ queries[j].second, m = (l + r) / 2;
            a[j] = {l, m, 1 - l, 1 - l, 1};
    ↪           // odd, left half:  i - l + 1
            b[j] = {m + 1, r, r + 1, r + 1, -1};
    ↪           // odd, right half: r - i + 1
        }
        auto ra = sweep(d1, km, a), rb = sweep(d1,
    ↪ kp, b);
        for (int i = 0; i < n; i++) km[i] = d2[i] -
    ↪ i, kp[i] = d2[i] + i;
        for (int j = 0; j < q; j++) {
            ll l = queries[j].first, r =
    ↪ queries[j].second, m = (l + r + 1) / 2;
            c[j] = {l + 1, m, -l, -l, 1};
    ↪           // even, left half:  i - l
            d[j] = {m + 1, r, r + 1, r + 1, -1};
    ↪           // even, right half: r - i + 1
        }
        auto rc = sweep(d2, km, c), rd = sweep(d2,
    ↪ kp, d);
        for (int j = 0; j < q; j++) res[j] = ra[j] +
    ↪ rb[j] + rc[j] + rd[j];
        return res;
    }
};
/* NOTES
 WHOLE-STRING COUNT is just sum(d1) + sum(d2) - no
   ↪ sweep needed, and the number of DISTINCT
   palindromic substrings is the node count of an
   ↪ eertree (05 - Palindromes/02).
 ONLINE queries: replace the sweep with a WAVELET
   ↪ TREE (02 - DS/13) that also stores prefix SUMS
   per node, or with a merge-sort tree - same
   ↪ formulas, O(log^2 n) per query, no sorting of
   queries. The offline version above is shorter and
   ↪ faster; use it unless the queries are forced
   to be online.
 PALINDROMES STARTING IN [l, r] (rather than
   ↪ contained in it) is a different and easier sweep:
   each centre contributes a contiguous range of
   ↪ start positions.
 COUNTING DISTINCT palindromes inside a range is much
   ↪ harder (offline eertree + BIT over the
   "last occurrence" trick, like distinct values in a
   ↪ range); do not confuse the two.
 CHECK THE SPLIT POINT: the odd half uses m = (l+r)/2
   ↪ and the even half m = (l+r+1)/2. Both come
   from solving i - l + 1 <= r - i + 1 and i - l <= r
   ↪ - i + 1; an off-by-one there double counts
   the centre and is the only bug this code ever has.
   ↪ */
```

7.6 Rotations

7.6.1 LyndonAndRotations

LYNDON FACTORISATION (Duval) and MINIMAL CYCLIC ROTATION (Duval), both O(n), O(1) memory.

```
// A Lyndon word is strictly smaller than all of its
   ↪ proper suffixes. Every string factors
// UNIQUELY as w1 >= w2 >= ... >= wk with each wi
   ↪ Lyndon.
vector<string> duval(const string& s) {
    int n = s.size(), i = 0;
    vector<string> f;
    while (i < n) {
        int j = i + 1, k = i;
        while (j < n && s[k] <= s[j]) s[k] < s[j] ? k
   ↪ = i : k++, j++;
        while (i <= k) f.push_back(s.substr(i, j -
   ↪ k)), i += j - k;
    }
    return f;
}
// Smallest cyclic rotation: run Duval on s+s and
   ↪ take the factor starting before n.
int minRotation(string s) {
    int n = s.size();
    s += s;
    int i = 0, ans = 0;
    while (i < n) {
        ans = i;
        int j = i + 1, k = i;
        while (j < (int)s.size() && s[k] <= s[j])
   ↪ s[k] < s[j] ? k = i : k++, j++;
        while (i <= k) i += j - k;
    }
    return ans;                                    //
   ↪ index in the ORIGINAL string
}
// USES: canonical form of a cyclic string (necklace
   ↪ dedup -> pairs with Burnside);
//       "is A a rotation of B" (compare minRotation
   ↪ forms, or search A in B+B);
//       lexicographically smallest result of
   ↪ rotating; runs / repetitions via Lyndon roots.
```

7.6.2 RunsAndTandemRepeats

ALL RUNS (maximal repetitions) of s in O(n log n) - O(n) if the LCE structure is linear.

```
// Needs: 07 - Strings/04 - Suffix Structures/05 -
   ↪ SuffixArraySAIS.cpp   (saIs, kasai)
// A RUN is a triple (l, r, p): s[l..r] has period p,
   ↪ r - l + 1 >= 2p, and it cannot be extended
// left or right with the same period. Every tandem
   ↪ repeat, every square, every "x repeated k
// times" substring lives inside exactly one run, so
   ↪ runs are THE compact encoding of all
// repetitions - and by the RUNS THEOREM there are
   ↪ FEWER THAN n of them, with total exponent
// sum(r - l + 1)/p < 3n. Enumerating squares
   ↪ directly is quadratic; enumerating runs is not.
// WHEN: count/locate squares or k-th powers,
   ↪ "longest substring that is a repetition",
   ↪ periodic
// structure of a string, and as the input to
   ↪ Lempel-Ziv style factorisations.
// INDEX CONVENTION: 0-based, l and r INCLUSIVE, p is
   ↪ the SMALLEST period of s[l..r].
// HOW (Bannai et al.): a run's period is a LYNDON
   ↪ ROOT - for one of the two alphabet orders, the
// run contains a Lyndon word of length p starting at
```

```
  ↪ some i. So take every i, let l[i] be the
// longest Lyndon prefix of the suffix i (that is
  ↪ "next smaller suffix" minus i), and try to
// extend the pair (i, i + l[i]) both ways with LCE.
  ↪ Do it for BOTH orders (< and its reverse).
struct RunsFinder {
    struct LceSA {
  ↪          // suffix array + O(1) LCE
        int n, LG;
        vector<int> sa, rnk, lcp, lg;
        vector<vector<int>> sp;
        void build(const string& s) {
            n = s.size(), sa = suffixArray(s), lcp =
  ↪ kasai(s, sa), rnk.assign(n, 0);
            for (int i = 0; i < n; i++) rnk[sa[i]] =
  ↪ i;
            lg.assign(n + 1, 0);
            for (int i = 2; i <= n; i++) lg[i] = lg[i
  ↪ / 2] + 1;
            LG = lg[n] + 1, sp.assign(LG,
  ↪ vector<int>(n, 0)), sp[0] = lcp;
            for (int k = 1; k < LG; k++)
                for (int i = 0; i + (1 << k) <= n;
  ↪ i++)
                    sp[k][i] = min(sp[k - 1][i], sp[k
  ↪ - 1][i + (1 << (k - 1))]);
        }
        int lce(int i, int j) {
  ↪          // lcp of the suffixes i and j
            if (i >= n || j >= n) return 0;
            if (i == j) return n - i;
            int l = rnk[i], r = rnk[j];
            if (l > r) swap(l, r);
            int k = lg[r - l];
            return min(sp[k][l + 1], sp[k][r - (1 <<
  ↪ k) + 1]);
        }
    };
    int n;
    LceSA F, B;
  ↪          // forward, and on the reversal
    vector<int> crnk;
  ↪          // suffix ranks under the FLIPPED
    RunsFinder(const string& s) {
  ↪          // alphabet order
        n = s.size();
        string r(s.rbegin(), s.rend()), c = s;
        for (char& ch : c) ch = (char)(255 -
  ↪ (unsigned char)ch);
        F.build(s), B.build(r);
        vector<int> csa = suffixArray(c);
        crnk.assign(n, 0);
        for (int i = 0; i < n; i++) crnk[csa[i]] = i;
        // NOT the reverse of F.rnk: flipping the
  ↪ alphabet flips every comparison EXCEPT the
        // prefix rule ("abc" < "abcd" in both
  ↪ orders), so the second order needs its own array.
    }
    int lceF(int i, int j) { return F.lce(i, j); }
  ↪           // common prefix of s[i..], s[j..]
    int lceB(int i, int j) {
  ↪          // common suffix of s[..i], s[..j]
        if (i < 0 || j < 0) return 0;
        return B.lce(n - 1 - i, n - 1 - j);
    }
    vector<array<int, 3>> runs() {
        vector<array<int, 3>> res;
        for (int ord = 0; ord < 2; ord++) {
  ↪          // both alphabet orders
            const vector<int>& rk = ord ? crnk :
  ↪ F.rnk;
            vector<int> nxt(n, n), st;
  ↪          // next suffix smaller in this order
            for (int i = n - 1; i >= 0; i--) {
                while (!st.empty() && rk[st.back()] >
  ↪ rk[i]) st.pop_back();
                nxt[i] = st.empty() ? n : st.back();
                st.push_back(i);
            }
            for (int i = 0; i < n; i++) {
                int p = nxt[i] - i, j = i + p;
  ↪          // p = Lyndon root length
                if (j > n) continue;
                int b = j < n ? lceF(i, j) : 0, a =
  ↪ lceB(i - 1, j - 1);
                if (a + b < p || p == 0) continue;
                int l = i - a, r = j + b - 1;
                if (r - l + 1 >= 2 * p)
  ↪ res.push_back({l, r, p});
            }
        }
        sort(res.begin(), res.end());
  ↪          // (l, r, p) with p increasing
        vector<array<int, 3>> out;
  ↪          // keep the SMALLEST p per (l, r)
        for (auto& x : res)
            if (out.empty() || out.back()[0] != x[0]
  ↪ || out.back()[1] != x[1]) out.push_back(x);
        return out;
  ↪          // < n runs
    }
};
/* WHAT TO DO WITH THE RUNS
 NUMBER OF SQUARES (as occurrences, x x with |x| =
  ↪ q): a run (l, r, p) with length L = r - l + 1
  contains, for every multiple q of ... no: for
  ↪ every q with p <= q and 2q <= L AND q a multiple
  of p, exactly L - 2q + 1 occurrences of a square
  ↪ of half-length q. Summing over runs is O(n)
  amortised by the runs theorem (total exponent <
  ↪ 3n).
 NUMBER OF DISTINCT SQUARES is at most n (the "three
  ↪ squares lemma"); get them by taking, per
  run, the primitively rooted squares and
  ↪ deduplicating with the run's root.
 K-TH POWERS: a run (l, r, p) contains a k-th power
  ↪ iff L >= k p, and then L - k p + 1 of them.
 LONGEST REPETITION = the run maximising L / p, read
  ↪ straight off the list.
 MAIN-LORENTZ is the alternative: divide & conquer
  ↪ with z-functions, O(n log n), ~40 lines, and
  it reports tandem repeats as O(n log n) grouped
  ↪ triples instead of < n runs. Prefer runs.
 PRIMITIVITY of s[l..r] (is it a proper power?) is a
  ↪ prefix-function question, not a runs one:
  see 01 - String Matching/05, "n % (n - pi) == 0".
 TESTING: on a^n there is exactly ONE run (0, n-1,
  ↪ 1); on "abababa" exactly one, (0, 6, 2). If
  you get duplicates with different p, the
  ↪ Lyndon-root loop ran with only one order. */
```

7.7 Sequences

7.7.1 DeBruijnAndBitap

DE BRUIJN SEQUENCES and BIT-PARALLEL MATCHING.

```
// --- DE BRUIJN SEQUENCE B(k, n): a CYCLIC string over an alphabet of size k, of length k^n, in
// which every one of the k^n words of length n appears exactly once as a (cyclic) substring.
// USES: "open the safe with the fewest keypresses" (the classic), minimum-length probe sequences,
// and hashing tricks (a de Bruijn constant is how you find the index of the lowest set bit).
// CONSTRUCTION (FKM / Lyndon): concatenate, in lexicographic order, every Lyndon word whose
// length DIVIDES n. That is exactly what this O(k^n) recursion emits.
string deBruijn(int k, int n) {
    string s;
    vector<int> a(k * n, 0);
    function<void(int, int)> db = [&](int t, int p) {
        if (t > n) {
            if (n % p == 0) for (int i = 1; i <= p; i++) s += char('0' + a[i]);
        } else {
            a[t] = a[t - p];
            db(t + 1, p);
            for (int j = a[t - p] + 1; j < k; j++) a[t] = j, db(t + 1, t);
        }
    };
    db(1, 1);
    return s;
}          // length k^n, read cyclically
// The same object is an EULERIAN CIRCUIT in the de Bruijn graph (vertices = words of length n-1,
// edges = words of length n), so Hierholzer on that graph is the alternative construction - and
// the BEST theorem then counts how many distinct de Bruijn sequences exist: (k!)^(k^(n-1)) / k^n.

// --- BITAP / SHIFT-AND: exact matching in O(n*m/64) with bitsets, and the natural home for
// approximate matching. Build one bitmask per character marking where it occurs in the pattern;
// then one pass over the text ANDs shifted copies together.
// Beats KMP only when you need the extra flexibility (k mismatches, wildcards, "any of these
// patterns"), or when you want ALL occurrences as a bitset to intersect with a range.
vector<int> bitap(const string& t, const string& p) {
    int n = t.size(), m = p.size();
    if (!m || m > n) return {};
    vector<unsigned long long> mask(256, 0);
    // Works for m <= 64 with a single word; for longer patterns use bitset<M> and shift it.
    if (m > 64) return {};
    for (int i = 0; i < m; i++) mask[(unsigned char)p[i]] |= 1ULL << i;
    unsigned long long st = 0, hit = 1ULL << (m - 1);
    vector<int> res;
    for (int i = 0; i < n; i++) {
        st = ((st << 1) | 1) & mask[(unsigned char)t[i]];
        if (st & hit) res.push_back(i - m + 1);
    }
    return res;
}
/* WHAT BITAP GENERALISES TO, which is the reason to carry it
 k MISMATCHES: keep k+1 state words R0..Rk, where Rj = "matched a prefix with at most j errors".
  Rj_new = ((Rj << 1) & mask[c]) | Rj-1 | (Rj-1 << 1) | (Rj-1_old << 1) | 1
  covering substitution, insertion and deletion respectively - that is Wu-Manber, O(k n m / 64).
 WILDCARDS in the PATTERN: set mask[c] bits for every character the class accepts. A '?' just
  means "every mask has that bit". This is free - no FFT needed - and is the right tool when the
  alphabet is small and the pattern is short. Use the FFT method (01 - String Matching/04) when
  the pattern is long or wildcards appear in the TEXT too.
 MULTIPLE PATTERNS of the same length: OR their masks and check which one actually matched.
 OCCURRENCES INSIDE [l, r]: keep the whole occurrence set as a bitset and popcount a range - that
  is what makes "how many times does p occur in t[l..r]" O(n/64) per query with no suffix
  structure at all.
 MYERS' BIT-VECTOR ALGORITHM computes full edit distance in O(n m / 64) with the same idea
  (carry-save arithmetic on the DP's difference vectors); it is the fastest practical exact edit
  distance and the escape hatch when the O(nm) DP is 1e10.  */
```

7.7.2 EditDistance

EDIT DISTANCE (Levenshtein): fewest single-character insertions, deletions and substitutions

```
// turning a into b. dp[i][j] = distance between the prefixes a[0..i-1] and b[0..j-1], so the
// table is (n+1) x (m+1) and the answer is dp[n][m].
// O(n m) time, O(min(n,m)) memory here.
// WHEN: fuzzy matching, "minimum operations to make equal", DNA-style alignment. If you only need
// insert+delete (no substitution) the answer is n + m - 2*LCS(a, b) - use the LCS DP instead.
int editDistance(const string& a, const string& b) {
    int n = a.size(), m = b.size();
    vector<int> prv(m + 1), cur(m + 1);
    iota(prv.begin(), prv.end(), 0);
    for (int i = 1; i <= n; i++) {
        cur[0] = i;
        for (int j = 1; j <= m; j++)
            cur[j] = min({prv[j] + 1, cur[j - 1] + 1, prv[j - 1] + (a[i - 1] != b[j - 1])});
        swap(prv, cur);
    }
    return prv[m];
}
// --- BANDED: when you only care whether the distance is <= k. Cells off the diagonal by more
// than k can never be on an optimal path (each step off the diagonal costs >= 1), so evaluate
// only |i - j| <= k. O(n k). Returns k + 1 as "more than k".
int editDistanceAtMost(const string& a, const string& b, int k) {
    int n = a.size(), m = b.size(), BIG = k + 1;
    if (abs(n - m) > k) return BIG;
    vector<int> prv(m + 2, BIG), cur(m + 2, BIG);
    for (int j = 0; j <= min(m, k); j++) prv[j] = j;          // row 0
    for (int i = 1; i <= n; i++) {
```

```
        int lo = max(0, i - k), hi = min(m, i + k);
        cur[lo] = lo ? min({prv[lo] + 1, prv[lo - 1]
    ↪ + (a[i - 1] != b[lo - 1]), BIG}) : min(i, BIG);
        for (int j = lo + 1; j <= hi; j++)
    ↪               // cur[lo-1] is OUTSIDE the band
            cur[j] = min({prv[j] + 1, cur[j - 1] + 1,
    ↪ prv[j - 1] + (a[i - 1] != b[j - 1]), BIG});
        if (hi + 1 <= m) cur[hi + 1] = BIG;
    ↪               // the cell the next row reads
        swap(prv, cur);
    }
    return min(prv[m], BIG);
}
// --- DAMERAU-LEVENSHTEIN, RESTRICTED (optimal
    ↪ string alignment): also allows swapping two
// ADJACENT characters at cost 1, but forbids editing
    ↪ a substring twice. This is the version
// contests mean; the unrestricted one needs the
    ↪ "last row/column per character" table.
int damerau(const string& a, const string& b) {
    int n = a.size(), m = b.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1));
    for (int i = 0; i <= n; i++) dp[i][0] = i;
    for (int j = 0; j <= m; j++) dp[0][j] = j;
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++) {
            dp[i][j] = min({dp[i - 1][j] + 1, dp[i][j
    ↪ - 1] + 1,
                            dp[i - 1][j - 1] + (a[i -
    ↪ 1] != b[j - 1])});
            if (i > 1 && j > 1 && a[i - 1] == b[j -
    ↪ 2] && a[i - 2] == b[j - 1])
                dp[i][j] = min(dp[i][j], dp[i - 2][j
    ↪ - 2] + 1);
        }
    return dp[n][m];
}
// --- MYERS' BIT-VECTOR ALGORITHM: exact Levenshtein
    ↪ in O(n * m / 64), the escape hatch when the
// O(n m) table is 1e10. It stores the VERTICAL
    ↪ differences dp[i][j] - dp[i-1][j] (always in
// {-1,0,+1}) as two bitmasks Pv/Mv over the rows,
    ↪ and advances a whole column with 12 word ops.
// Here a is the "pattern" and must satisfy |a| <=
    ↪ 64; loop over blocks of 64 rows for longer a.
int myersEditDistance(const string& a, const string&
    ↪ b) {
    int m = a.size();
    if (!m) return b.size();
    unsigned long long peq[256] = {};
    for (int i = 0; i < m; i++) peq[(unsigned
    ↪ char)a[i]] |= 1ULL << i;
    unsigned long long Pv = ~0ULL, Mv = 0, high =
    ↪ 1ULL << (m - 1);
    int score = m;
    for (char ch : b) {
        unsigned long long Eq = peq[(unsigned
    ↪ char)ch];
        unsigned long long Xv = Eq | Mv;
        unsigned long long Xh = (((Eq & Pv) + Pv) ^
    ↪ Pv) | Eq;
        unsigned long long Ph = Mv | ~(Xh | Pv), Mh =
    ↪ Pv & Xh;
        if (Ph & high) score++;
        if (Mh & high) score--;
        Ph = (Ph << 1) | 1;
    ↪           // dp[0][j] = j  (GLOBAL alignment)
        Mh <<= 1;
        Pv = Mh | ~(Xv | Ph);
        Mv = Ph & Xv;
    }
    return score;
}
/* VARIANTS AND TRAPS
 SHIFT IN 0 INSTEAD OF 1 ('Ph <<= 1') and Myers
  ↪ computes, for every j, the distance from a to the
   BEST-MATCHING substring of b ending at j - i.e.
  ↪ approximate pattern matching, k-differences.
 RECONSTRUCTING the alignment needs the table; with
  ↪ O(n) memory use Hirschberg
   (08 - DP/04 - Classics), which is the same divide
  ↪ & conquer for any such 2-string DP.
 WEIGHTED costs (ins != del != sub) work in the plain
  ↪ DP but break Myers and the LCS identity.
 LONGEST COMMON SUBSTRING (contiguous) is NOT this
  ↪ DP: use hashing + binary search, a suffix
   automaton, or dp[i][j] = dp[i-1][j-1] + 1 with the
  ↪ answer read as a max over all cells.
 EDIT DISTANCE TO A REGULAR LANGUAGE / to any string
  ↪ of a trie: same DP with the automaton state
   as the second dimension, O(n * states).
 LOWER BOUND: |n - m| <= dist <= max(n, m), and dist
  ↪ >= (number of characters whose counts
   differ) / 2 style bounds - cheap filters before
  ↪ running the real DP on many pairs. */
```

7.7.3 BitParallelLCS

BIT-PARALLEL LCS - the length of the longest common subsequence of a and b in O(n m / 64),

```
// where n = |a|. The classic DP is O(n m) and dies
  ↪ at n = m = 5e4; this survives n = m = 1e5.
// IDEA: the LCS row is monotone and its DIFFERENCE
  ↪ vector is a bit string over the positions of
// a (bit i = "the LCS value increases at position i
  ↪ of a"). One row update is then a handful of
// word operations, of which the carry-propagating
  ↪ ADD is what does the actual work.
// PRECONDITION: none - this is exact, for arbitrary
  ↪ alphabets (map characters to 0..A-1).
// The answer is the POPCOUNT of the final vector;
  ↪ the bit positions themselves are NOT an LCS.
struct BitLCS {
    int n, W;
  ↪          // W = number of 64-bit words
    vector<vector<unsigned long long>> match;
  ↪          // match[c] = positions of c in a
    BitLCS(const string& a, int A = 26, char base =
  ↪ 'a') {
        n = a.size(), W = n / 64 + 1;
        match.assign(A, vector<unsigned long long>(W,
  ↪ 0));
        for (int i = 0; i < n; i++) match[a[i] -
  ↪ base][i >> 6] |= 1ULL << (i & 63);
    }
    int lcs(const string& b, char base = 'a') {
        vector<unsigned long long> row(W, 0), x(W);
        for (char ch : b) {
            const auto& mc = match[ch - base];
            for (int i = 0; i < W; i++) x[i] = row[i]
  ↪ | mc[i];  // x = row | match[c]
            unsigned long long carry = 1;
  ↪           // row = (row << 1) | 1
            for (int i = 0; i < W; i++) {
                unsigned long long nc = row[i] >> 63;
                row[i] = (row[i] << 1) | carry;
                carry = nc;
```

```
            }
            unsigned long long borrow = 0;
  ↪           // row = x - row
            for (int i = 0; i < W; i++) {
                unsigned long long d = x[i] - row[i]
  ↪ - borrow;
                borrow = (x[i] < row[i] + borrow) ||
  ↪ (borrow && row[i] == ~0ULL);
                row[i] = d;
            }
            for (int i = 0; i < W; i++) row[i] = x[i]
  ↪ & ~row[i];// row = x & ~(x - row)
        }
        int res = 0;
        for (int i = 0; i < W; i++) res +=
  ↪ __builtin_popcountll(row[i]);
        return res;
    }
};
// USAGE: build once on the SHORTER string (that is
  ↪ the one that costs memory and words),
//        then call lcs() for every other string -
  ↪ the build is reused.
//   BitLCS L(a); int k = L.lcs(b);
/* NOTES
 WHY POPCOUNT: after processing all of b, bit i of
  ↪ 'row' is set iff LCS(a[0..i], b) is strictly
   greater than LCS(a[0..i-1], b) - the difference
  ↪ vector of a monotone row of the DP table.
 EDIT DISTANCE has the same treatment (Myers, 07 -
  ↪ Sequences/02) with two vectors instead of one,
   because its differences live in {-1, 0, +1} while
  ↪ the LCS row only ever steps by 0 or 1.
 LCS ITSELF (the subsequence, not its length) still
  ↪ needs the O(n m) table or Hirschberg
   (08 - DP/04 - Classics) - bit-parallelism gives
  ↪ you the LENGTH only.
 LCS OF PREFIXES: read the popcount after each
  ↪ character of b to get LCS(a, b[0..j]) for all j
  ↪ at
   no extra cost. That is what makes this the
  ↪ practical answer to "all-prefix" LCS queries.
 SPARSE ALTERNATIVE: when the number of matching
  ↪ pairs r is small, the Hunt-Szymanski algorithm
   is O(r log n) - better on near-distinct alphabets,
  ↪ worse on binary strings.
 ALPHABET: 'match' is A * W words. For a large
  ↪ alphabet, map to the distinct characters of a
   first (everything else can never match and can be
  ↪ dropped from b). */
```

7.7.4 AllSubstringLCS

ALL-SUBSTRING LCS - after ONE O(n m) pass you can answer lcs(s, t[i..j]) for EVERY one of the

```
// m^2 substrings of t. The classic DP gives one pair
  ↪ per O(n m); this gives all of them at once.
// INDEX CONVENTION: s and t are 1-indexed below; the
  ↪ returned array F is indexed 1..m and
//     lcs(s, t[i..j]) = #{ k in [i, j] : F[k] < i }.
// So one array of m integers encodes all m^2
  ↪ answers, and a single query is a RANGE COUNT of
// values below a threshold - BIT offline, wavelet
  ↪ tree, or merge-sort tree, O(log m) each.
// WHEN: "for every substring of t, how similar is it
  ↪ to s", edit-distance-to-a-window sweeps,
// and problems asking for the shortest/longest
  ↪ window of t whose LCS with s reaches a target.
// WHY IT WORKS: for a fixed left end i the row of
  ↪ the LCS table is monotone with steps of 0/1;
// F[k] is the largest left end for which position k
  ↪ still contributes a step. f/g below track
// the row and column "critical points" of the table
  ↪ - that is the Alves-Caceres-Song theory.
vector<int> allSubstringLCS(const string& s, const
  ↪ string& t) {
    int n = s.size(), m = t.size();
    vector<int> f(m + 1), g(m + 1, 0), nf(m + 1, 0);
    for (int j = 0; j <= m; j++) f[j] = j;
  ↪           // row 0 of the table
    for (int i = 1; i <= n; i++) {
        nf[0] = 0;
        for (int j = 1; j <= m; j++) {
            int gp = g[j - 1], fp = f[j];
  ↪             // g[i][j-1] and f[i-1][j]
            if (s[i - 1] == t[j - 1]) nf[j] = gp,
  ↪ g[j] = fp;
            else nf[j] = max(fp, gp), g[j] = min(gp,
  ↪ fp);
        }
        swap(f, nf);
    }
    return f;
  ↪           // F[1..m], F[0] is unused
}
// DIRECT USE - all answers for one left end i in
  ↪ O(m):
//   int cnt = 0; for (int j = i; j <= m; j++) { if
  ↪ (F[j] < i) cnt++; /* lcs(s, t[i..j]) = cnt */ }
// MANY (i, j) QUERIES: sort by i, sweep i from m
  ↪ down to 1 adding position k when F[k] < i to a
// BIT, then the answer is a prefix-sum query on [i,
  ↪ j]. O((m + q) log m) total.
/* RELATED SHAPES
 ALL-SUBSTRING LCS OF s AGAINST ITS OWN SUBSTRINGS is
  ↪ the same call with t = s.
 THE TRANSPOSE (all substrings of s against the whole
  ↪ t) is the same routine with the arguments
   swapped - the construction is NOT symmetric, so
  ↪ swap deliberately.
 SEMI-LOCAL LCS is the full theory this is a corner
  ↪ of: it also answers "s[a..b] vs t[i..j]"
   using a permutation matrix and range counting, in
  ↪ O(n m log) preprocessing. Rarely needed.
 EDIT DISTANCE has no such compact all-substring
  ↪ encoding; only the +1/-1 step structure of LCS
   makes the answers a range count.
 SANITY CHECK: lcs(s, t[i..i]) is 1 exactly when t[i]
  ↪ appears in s, and F[k] < i is then the test
   "the first position of t[k]'s character in s is
  ↪ early enough" - verify on a tiny case before
   trusting the direction of the inequality. */
```

7.7.5 CyclicLCS

CYCLIC LCS - max over all n cyclic rotations of s of lcs(rotation, t), in O(n m) TOTAL.

```
// The naive answer is n separate LCS computations,
  ↪ O(n^2 m). WHEN: the two sequences are
// necklaces / circular arrangements and the cut
  ↪ point is yours to choose.
// HOW: build the LCS table of s+s against t once,
  ↪ then RE-ROOT it n times. Deleting the first
// row of the table only ever cancels ONE unit of the
  ↪ answer, and the path that loses it can be
// followed in O(m) - so all n rotations cost O(n m)
  ↪ together, not O(n^2 m).
// INDEX CONVENTION: rows 1..2n of dp read the
```

```
    ↪ character s[(row - 1) % n]; dp[i][j] is the LCS
    ↪ of
// the first i characters of the doubled s with
    ↪ t[0..j-1]. Rotation r reads dp[r + n][m].
// from[i][j] records WHICH transition was taken: 0 =
    ↪ came from the left, 1 = diagonal match,
// 2 = came from above. The re-rooting walk needs it,
    ↪ so it is not optional bookkeeping.
int cyclicLCS(const string& s, const string& t) {
    int n = s.size(), m = t.size();
    if (!n || !m) return 0;
    vector<vector<int>> dp(2 * n + 1, vector<int>(m +
    ↪ 1, 0));
    vector<vector<char>> from(2 * n + 1,
    ↪ vector<char>(m + 1, 0));
    auto eq = [&](int i, int j) { return s[(i - 1) %
    ↪ n] == t[j - 1]; };
    for (int i = 0; i <= 2 * n; i++)
        for (int j = 0; j <= m; j++) {
            dp[i][j] = 0, from[i][j] = 0;
            if (j && dp[i][j - 1] > dp[i][j])
    ↪ dp[i][j] = dp[i][j - 1], from[i][j] = 0;
            if (i && j && eq(i, j) && dp[i - 1][j -
    ↪ 1] + 1 > dp[i][j])
                dp[i][j] = dp[i - 1][j - 1] + 1,
    ↪ from[i][j] = 1;
            if (i && dp[i - 1][j] > dp[i][j])
    ↪ dp[i][j] = dp[i - 1][j], from[i][j] = 2;
        }
    int res = 0;
    for (int r = 0; r < n; r++) {
        res = max(res, dp[r + n][m]);
    ↪           // rotation starting at s[r]
        int x = r + 1, y = 0;
    ↪           // now delete row r + 1
        while (y <= m && from[x][y] == 0) y++;
    ↪           // find where that row is used
        for (; y <= m && x <= 2 * n; x++) {
            from[x][y] = 0, dp[x][m]--;
            if (x == 2 * n) break;
            for (; y <= m; y++) {
    ↪             // follow the path downward
                if (from[x + 1][y] == 2) break;
                if (y + 1 <= m && from[x + 1][y + 1]
    ↪ == 1) { y++; break; }
            }
        }
    }
    return res;
}
/* WHAT THIS DOES AND DOES NOT COMPUTE
 IT ROTATES ONLY s. That is the standard definition
    ↪ of CLCS and it is enough: rotating BOTH
    strings is a strictly larger maximum in general,
    ↪ and needs n * m separate cut pairs - if a
    problem really wants both, rotate t in an outer
    ↪ loop (O(n m^2)) or think again.
 MEMORY is O(n m) for two tables - at n = m = 2000
    ↪ that is ~32 MB for dp plus ~8 MB for from.
    Make s the SHORTER string; the doubled dimension
    ↪ is the expensive one.
 THE ANSWER IS NOT MONOTONE in the rotation, so
    ↪ binary searching the cut point is wrong.
 CYCLIC EDIT DISTANCE is the same re-rooting idea
    ↪ (Maes' algorithm) but is O(n m log n) via
    divide & conquer on non-crossing optimal paths -
    ↪ do not expect this code to adapt.
 SANITY: cyclicLCS(s, s) == |s| and cyclicLCS(s, t)
    ↪ >= lcs(s, t) always. */
```

7.7.6 SubsequenceAutomaton

SUBSEQUENCE AUTOMATON (a.k.a. the "next occurrence" table). O(n * A) build, O(1) transition.

```
// INDEX CONVENTION: nxt[i][c] = the SMALLEST j >= i
    ↪ with s[j] == c, or n if there is none.
// There are n + 1 rows, i = 0..n; row i is the state
    ↪ "we have consumed s[0..i-1]".
// After matching a character at j you move to row j
    ↪ + 1 - never to row j.
// WHEN: everything of the form "is t a subsequence
    ↪ of s", counting or ranking DISTINCT
// subsequences, and shortest strings that are NOT
    ↪ subsequences. Build it from the RIGHT.
vector<array<int, 26>> subseqAutomaton(const string&
    ↪ s) {
    int n = s.size();
    vector<array<int, 26>> nxt(n + 1);
    for (int c = 0; c < 26; c++) nxt[n][c] = n;
    for (int i = n - 1; i >= 0; i--) nxt[i] = nxt[i +
    ↪ 1], nxt[i][s[i] - 'a'] = i;
    return nxt;
}
// --- IS t A SUBSEQUENCE OF s? O(|t|) per query
    ↪ after the build (the reason to build at all:
// the two-pointer scan is O(|s| + |t|) per query,
    ↪ which loses the moment there are many queries).
bool isSubsequence(const string& t, const
    ↪ vector<array<int, 26>>& nxt) {
    int n = nxt.size() - 1, i = 0;
    for (char ch : t) {
        int j = nxt[i][ch - 'a'];
        if (j == n) return false;
        i = j + 1;
    }
    return true;
}
// --- NUMBER OF DISTINCT NON-EMPTY SUBSEQUENCES, O(n
    ↪ * A). Each distinct subsequence is read by
// exactly ONE path in the automaton (always take the
    ↪ earliest occurrence), so just count paths.
// f[i] = number of distinct non-empty subsequences
    ↪ of s[i..], counted as "first letter is c".
ll distinctSubsequences(const string& s, const
    ↪ vector<array<int, 26>>& nxt, ll mod = MOD) {
    int n = s.size();
    vector<ll> f(n + 2, 0);
    for (int i = n; i >= 0; i--) {
        f[i] = 0;
        for (int c = 0; c < 26; c++) if (nxt[i][c] <
    ↪ n) f[i] = (f[i] + f[nxt[i][c] + 1] + 1) % mod;
    }
    return f[0];
}
// --- K-TH DISTINCT SUBSEQUENCE in lexicographic
    ↪ order (1-indexed, non-empty). Use the UNCAPPED
// counts f[] above (saturating at k, not modular - a
    ↪ modular count cannot be compared).
string kthSubsequence(const string& s, const
    ↪ vector<array<int, 26>>& nxt, ll k) {
    int n = s.size();
    vector<ll> f(n + 2, 0);
    ↪           // f[i] saturated at 2e18
    for (int i = n; i >= 0; i--)
        for (int c = 0; c < 26; c++) if (nxt[i][c] <
    ↪ n)
            f[i] = min((ll)2e18, f[i] + f[nxt[i][c] +
```

```
    ↪ 1] + 1);
    if (k > f[0]) return "";
    ↪          // fewer than k subsequences
    string res;
    int i = 0;
    while (k > 0) {
        for (int c = 0; c < 26; c++) {
            int j = nxt[i][c];
            if (j == n) continue;
            ll cur = f[j + 1] + 1;
    ↪          // subsequences starting with c
            if (k <= cur) { res += char('a' + c), i =
    ↪ j + 1, k--; break; }
            k -= cur;
        }
    }
    return res;
}
// --- SHORTEST STRING OVER THE ALPHABET THAT IS NOT
    ↪ A SUBSEQUENCE of s, O(n * A).
// g[i] = that length for s[i..]; a missing letter
    ↪ gives 1, otherwise pay one letter and recurse.
int shortestNonSubsequence(const string& s, const
    ↪ vector<array<int, 26>>& nxt, int A = 26) {
    int n = s.size();
    vector<int> g(n + 2, 0);
    for (int i = n; i >= 0; i--) {
        g[i] = inf;
        for (int c = 0; c < A; c++)
            g[i] = min(g[i], nxt[i][c] == n ? 1 : 1 +
    ↪ g[nxt[i][c] + 1]);
    }
    return g[0];
    ↪          // reconstruct by replaying the min
}
/* VARIANTS
 DISTINCT COMMON SUBSEQUENCES of s and t: run both
    ↪ automata in lockstep,
    f[i][j] = sum over c of (f[nxt1[i][c] +
    ↪ 1][nxt2[j][c] + 1] + 1), O(n m A).
 COUNTING SUBSEQUENCES EQUAL TO t (occurrences, not
    ↪ distinct) is the other classic DP,
    dp[i][j] = dp[i-1][j] + (s[i] == t[j]) *
    ↪ dp[i-1][j-1], O(n m) - the automaton does not
    ↪ help.
 DISTINCT SUBSEQUENCES OF LENGTH EXACTLY k: add the
    ↪ length as a dimension, O(n k A).
 LARGE ALPHABET: store nxt as a vector of sorted
    ↪ position lists per character and binary search;
    that is O(n + A) memory and O(log n) per
    ↪ transition, which is the right trade when A is
    ↪ 1e5.
 DO NOT CONFUSE with the SUFFIX automaton (04 -
    ↪ Suffix Structures/02), which recognises
    SUBSTRINGS. This one recognises SUBSEQUENCES and
    ↪ is a DAG with no suffix links. */
```

7.8 Theorems

7.8.1 StringTheorems

STRING THEOREMS - the facts that turn an O(n^2) idea into an O(n) one. No code: these are the

```
/* STRING THEOREMS - the facts that turn an O(n^2)
    ↪ idea into an O(n) one. No code: these are the
    statements you need to REMEMBER, because none of
    ↪ them is derivable at the table.

PERIODICITY
 BORDER-PERIOD DUALITY: p is a period of s (s[i] =
    ↪ s[i+p] whenever both exist) IFF n - p is the
    length of a border. So the periods and the borders
    ↪ are the same set read backwards, and the
    prefix function gives all of them (07 -
    ↪ Strings/01/05).
 FINE AND WILF: if p and q are both periods of s and
    ↪ p + q - gcd(p, q) <= n, then gcd(p, q) is
    also a period. The bound is TIGHT (Fibonacci
    ↪ words). Consequences:
    - the borders of any string form O(log n)
    ↪ ARITHMETIC PROGRESSIONS (each progression is one
      "period class"); this is what makes
    ↪ palindromic-factorisation and border-sum DPs O(n
    ↪ log n)
      instead of O(n^2) - see 05 - Palindromes/04.
    - two occurrences of a pattern P closer than |P| /
    ↪ 2 force P to be periodic; hence in any text
      window of length 2|P| a non-periodic pattern
    ↪ occurs at most twice (the basis of constant-space
      matching and of "the occurrences form O(log)
    ↪ arithmetic progressions").
 CRITICAL FACTORISATION: every string s with period p
    ↪ has a position i < p such that the local
    period at i equals the global period. Splitting
    ↪ there makes the two halves comparable
    independently - the basis of the two-way
    ↪ (Crochemore-Perrin) O(n) constant-space matcher.
 PRIMITIVITY: s is a power t^k with k >= 2 IFF n % (n
    ↪ - pi[n-1]) == 0 and n - pi[n-1] < n. A
    primitive word has exactly n distinct rotations; a
    ↪ non-primitive one has n / k.
 THE CYCLE LEMMA / rotations: s and t are rotations
    ↪ of each other IFF t is a substring of s + s
    IFF minRotation(s) == minRotation(t) (06 -
    ↪ Rotations/01).

REPETITIONS
 RUNS THEOREM (Bannai et al. 2015): a string of
    ↪ length n has FEWER THAN n runs (maximal
    repetitions), and the sum of their exponents is <
    ↪ 3n. Both bounds are what make "enumerate all
    repetitions" linear rather than quadratic.
    ↪ Construction: 06 - Rotations/02.
 THREE SQUARES LEMMA: if three squares start at the
    ↪ same position and u is a prefix of v is a
    prefix of w, then |u| + |v| <= |w|. Hence at most
    ↪ O(log n) squares start at any position, and a
    string of length n contains fewer than n DISTINCT
    ↪ squares.
 CHEN-FOX-LYNDON: every string factors uniquely as w1
    ↪ >= w2 >= ... >= wk with each wi Lyndon, and
    Duval computes it in O(n) with O(1) memory. The
    ↪ last factor is the smallest suffix; the first
    is the longest Lyndon prefix. Lyndon roots are the
    ↪ pivot of the runs algorithm.
 A LYNDON WORD is strictly smaller than all of its
    ↪ proper suffixes; equivalently it is the unique
    minimum among its own rotations, and it is
```

```
    ↪ primitive. There are (1/n) * sum over d | n of
    mu(d) * A^(n/d) Lyndon words of length exactly n
    ↪ over an alphabet of size A (Witt's formula).

SUBSTRINGS AND SUFFIX STRUCTURES
 ENDPOS EQUIVALENCE (suffix automaton): two
    ↪ substrings are equivalent when they occur at
    ↪ exactly
    the same set of END positions. The classes form a
    ↪ TREE under "is a suffix of" (the suffix-link
    tree), there are at most 2n - 1 states and 3n - 4
    ↪ transitions, and one class contains exactly
    the strings of lengths (len[link[v]], len[v]] - so
    ↪ a class contributes len[v] - len[link[v]]
    distinct substrings. Summing that is the count of
    ↪ distinct substrings.
 THE SUFFIX-LINK TREE of a SAM built on the REVERSED
    ↪ string IS the suffix tree of the string.
 A STRING OF LENGTH n HAS AT MOST n DISTINCT
    ↪ PALINDROMIC SUBSTRINGS (eertree node count), and
    ↪ the
    palindromic suffixes of any prefix again form
    ↪ O(log n) arithmetic progressions.
 DISTINCT SUBSTRINGS = n(n+1)/2 - sum of the LCP
    ↪ array; adding one character to the end changes
    that by (new length) - (longest suffix that
    ↪ already occurred), which is the SAM's len[last].

MATCHING BOUNDS
 KNUTH-MORRIS-PRATT does at most 2n character
    ↪ comparisons; Boyer-Moore is sublinear on average
    but O(nm) in the worst case without Galil's rule;
    ↪ Z and prefix functions are exactly linear.
 A PATTERN WITH WILDCARDS is not solvable by automata
    ↪ in linear time; the FFT convolution
    sum (p_i - t_j)^2 p_i t_j is the standard O(n log
    ↪ n) route (01 - String Matching/04).
 EDIT DISTANCE cannot be computed in O(n^(2-eps))
    ↪ unless SETH fails - so the O(n m / 64)
    bit-parallel version (07 - Sequences/02) is
    ↪ essentially the last word.
 LCS OF k STRINGS is NP-hard in k; for k = 2 it is
    ↪ O(n m / 64) and no truly subquadratic
    algorithm is known (same SETH barrier).

COUNTING
 NUMBER OF DISTINCT NECKLACES (rotation classes) of
    ↪ length n over A letters:
    (1/n) * sum over d | n of phi(d) * A^(n/d)
    ↪                          (Burnside)
 NUMBER OF BRACELETS adds the reflections; see 05 -
    ↪ Combinatorics/02 - BurnsidePolya.cpp.
 DE BRUIJN SEQUENCES B(A, n): there are
    ↪ (A!)^(A^(n-1)) / A^n distinct ones (07 -
    ↪ Sequences/01). */
```

8 | Dynamic Programming (DP)

8.1 DP Optimizations

8.1.1 DivideAndConquerDP

Divide & Conquer DP Optimization - Time: O(K * N log N), Space: O(N)

```
// Applies when DP transition is dp[k][i] = min_{j <
   ↪ i} (dp[k-1][j] + cost(j+1, i))
// Condition: opt(i, j) <= opt(i, j + 1)
   ↪ (Monotonicity of optimal split points)
const int N = 200005;                                  //
   ↪ ALSO in 01 - Template.cpp: keep one copy
int a[N];
ll curCost;
int curL = 0, curR = -1, freq[N];
pair<int, ll> pre[N], cur[N];
// Modify at will :
int lo(int ind) { return 1; }
int hi(int ind) { return ind; }

void add(int val) {
    curCost += freq[val]++;
}

void remove(int val) {
    curCost -= --freq[val];
}

ll Cost(int l, int r) {
    while (curR < r)
        add(a[++curR]);
    while (curL > l)
        add(a[--curL]);
    while (curR > r)
        remove(a[curR--]);
    while (curL < l)
        remove(a[curL++]);
    return curCost;
}

ll f(int ind, int k) { return pre[k - 1].second +
   ↪ Cost(k, ind); }
void store(int ind, int k, ll v) { cur[ind] = pair(k,
   ↪ v); }

void rec(int L, int R, int LO, int HI) {
    if (L > R) return;
    int mid = (L + R) >> 1;
    pair best(llinf, LO);
    for (int k = max(LO, lo(mid)); k <= min(HI,
   ↪ hi(mid)); ++k)
        best = min(best, pair(f(mid, k), k));
    store(mid, best.second, best.first);
    rec(L, mid - 1, LO, best.second);
    rec(mid + 1, R, best.second, HI);
}

void solve(int L, int R) { rec(L, R, -inf, inf); }
```

8.1.2 CHT

DYNAMIC CONVEX HULL TRICK (UPPER hull -> MAXIMUM of m*x + b). Lines may be inserted in ANY order

```
// and queried at any x, O(log n) each. If your slopes AND queries are both monotone, the deque version at the bottom of this file is O(1) amortised and much shorter - prefer it.
// For a MINIMUM, insert (-m, -b) and negate the answer.
// To maximise over all x in [l, r], query only l and r: the upper envelope is convex.
struct Line {
    ll m, b;
    mutable function<const Line *()> succ;

    bool operator<(const Line &other) const {
        return m < other.m;
    }

    bool operator<(const ll &x) const {
        const Line *s = succ();
        if (!s)
            return 0;
        return (lll)(b - s->b) < (lll)(s->m - m) * x;
    // __int128: |dm|*|x| overflows ll
    }
};

// will maintain upper hull for maximum
struct HullDynamic : public multiset<Line, less<>> {
    bool bad(iterator y) {
        auto z = next(y);
        if (y == begin()) {
            if (z == end())
                return 0;
            return y->m == z->m && y->b <= z->b;
        }
        auto x = prev(y);
        if (z == end())
            return y->m == x->m && y->b <= x->b;
        // __int128, NOT long double: each factor reaches ~2e18, the product needs ~123 bits,
        // and this test is consulted exactly when three lines are nearly collinear.
        return (lll)(x->b - y->b) * (z->m - y->m) >= (lll)(y->b - z->b) * (y->m - x->m);
    }

    void insert_line(ll m, ll b) {
//         for minimum
//        m *= -1;
//        b *= -1;
        auto y = insert({m, b});
        y->succ = [=] { return next(y) == end() ? 0 : &*next(y); };
        if (bad(y)) {
            erase(y);
            return;
        }
        while (next(y) != end() && bad(next(y)))
            erase(next(y));
        while (y != begin() && bad(prev(y)))
            erase(prev(y));
    }

    ll query(ll x) {
    // UB if empty - insert at least one line first
        auto l = *lower_bound(x);
//         for minimum
//         return -(l.m * x + l.b);
        return l.m * x + l.b;
    }
};
//
// ------------------------------------------------------------
// MONOTONE CHT (deque) - the version you actually want when the DP feeds lines in slope order.
// MINIMISES m*x + b. Requires slopes added in DECREASING order; queries in increasing x are O(1)
// amortised (move the head), arbitrary x is O(log n) by binary search on the hull.
// For a MAXIMUM, negate m and b on the way in and negate the answer on the way out.
struct MonoCHT {
    vector<ll> M, B;
    int head = 0;
    bool bad(int a, int b, int c) {
    // is line b unnecessary between a and c?
        return (lll)(B[c] - B[a]) * (M[a] - M[b]) <= (lll)(B[b] - B[a]) * (M[a] - M[c]);
    }
    void add(ll m, ll b) {
    // m must be <= the last slope added
        M.push_back(m), B.push_back(b);
        while (M.size() >= 3 && bad(M.size() - 3, M.size() - 2, M.size() - 1))
            M.erase(M.end() - 2), B.erase(B.end() - 2);
        head = min(head, (int)M.size() - 1);
    }
    ll f(int i, ll x) { return M[i] * x + B[i]; }
    ll queryInc(ll x) {
    // x non-decreasing across calls
        while (head + 1 < (int)M.size() && f(head + 1, x) <= f(head, x)) head++;
        return f(head, x);
    }
    ll query(ll x) {
    // any x, O(log n)
        int lo = 0, hi = M.size() - 1;
        while (lo < hi) { int m = (lo + hi) / 2; f(m + 1, x) <= f(m, x) ? lo = m + 1 : hi = m; }
        return f(lo, x);
    }
};
// WHEN EACH ONE: monotone slopes + monotone queries -> MonoCHT::queryInc, O(n).
//                monotone slopes, arbitrary queries -> MonoCHT::query, O(n log n).
//                arbitrary slopes -> HullDynamic above, or Li Chao (04),
//                    which needs no ordering at all.
//                lines inserted and removed -> CHT with rollback (03) or Li Chao merge.
```

8.1.3 CHTRollback

Rollback Convex Hull Trick - Space: O(N), Add: O(log N), Query: O(log N), Rollback: O(1)

```
// Supports inserting monotonic lines with history stack rollback for tree/dsu DP
// Template parameters:
// IS_MAX: Set to true for Maximum queries, false for Minimum queries.
// IS_INC: Set to true if slopes added are strictly Increasing, false for Decreasing.
template<bool IS_MAX = false, bool IS_INC = false>
struct RollbackCHT {
    struct Line {
        ll m, c;
        Line() : m(0), c(llinf) {}
        Line(ll m, ll c) : m(m), c(c) {}
        ll eval(ll x) const { return m * x + c; }
    };

    struct History {
        int pos;
        int sz;
        Line old_line;
    };

    vector<Line> st;
    vector<History> hist;
    int sz;

    RollbackCHT(int max_nodes) {
        st.resize(max_nodes);
        sz = 0;
    }

    bool redundant(Line l1, Line l2, Line l3) {
        __int128 dx1 = l1.m - l2.m;
        __int128 dc1 = l2.c - l1.c;
        __int128 dx2 = l2.m - l3.m;
        __int128 dc2 = l3.c - l2.c;
        return dc1 * dx2 >= dc2 * dx1;
    }

    void add(ll m, ll c) {
        // The magic happens here: automatically formats internal state
        ll m_int = IS_INC ? -m : m;
        ll c_int = IS_MAX ? -c : c;

        Line l_new(m_int, c_int);
        int pos = sz;
        int low = 1, high = sz - 1;

        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (redundant(st[mid - 1], st[mid], l_new)) {
                pos = mid;
                high = mid - 1;
            } else {
                low = mid + 1;
            }
        }

        hist.push_back({pos, sz, st[pos]});
        st[pos] = l_new;
        sz = pos + 1;
    }

    void rollback() {
        if (hist.empty()) return;
        History h = hist.back();
        hist.pop_back();
        sz = h.sz;
        st[h.pos] = h.old_line;
    }

    ll query(ll x) {
        if (sz == 0) return IS_MAX ? -llinf : llinf;

        // The magic happens here: matches x to the internal geometry
        ll x_int = (IS_INC != IS_MAX) ? -x : x;

        int low = 0, high = sz - 2;
        int best = sz - 1;

        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (st[mid].eval(x_int) <= st[mid + 1].eval(x_int)) {
                best = mid;
                high = mid - 1;
            } else {
                low = mid + 1;
            }
        }

        ll ans = st[best].eval(x_int);
        return IS_MAX ? -ans : ans;
    }
};
```

8.1.4 LiChaoTree

LI CHAO TREE - MAXIMUM of a set of lines y = m*x + b at any query x, with NO order requirement

```
// on the slopes or on the queries (that is the whole point; CHT needs monotone slopes).
// O(log X) per line insert and per query, O(log^2 X) to insert a line only on a SEGMENT [lx, rx].
// Space O(n log X). Works for any family of functions where two members cross AT MOST ONCE -
// lines, and also a*x^2 + b*x + c with a FIXED a.
// EXACT INTEGER ARITHMETIC: never compute the crossing abscissa. Compare the two candidates at
// ns, mid and ne only - a division in double is off by ~1e3 in x when the coefficients reach 1e18,
// which silently sends the loser down the wrong half.
// SET the domain [X0, X1] before use; it must cover every x you will ever query.
struct LiChao {
    struct Line { ll m = 0, b = LLONG_MIN / 4; };
                // neutral = -infinity
    ll X0, X1;
    vector<Line> t;
    vector<int> lf, rt;
    LiChao(ll X0, ll X1) : X0(X0), X1(X1) { newNode(); }
    int newNode() { t.push_back({}), lf.push_back(-1), rt.push_back(-1); return t.size() - 1; }
    static ll f(const Line& L, ll x) { return L.m == 0 && L.b == LLONG_MIN / 4 ? L.b : L.m * x + L.b; }
    void add(Line nw, int v, ll l, ll r) {
                // insert on the whole node range
        while (true) {
```

```
            ll m = l + (r - l) / 2;
            // NOT (l+r)/2: negative l+r truncates up
            bool lef = f(nw, l) > f(t[v], l), mid = f(nw, m) > f(t[v], m);
            if (mid) swap(t[v], nw);
            if (l == r) return;
            if (lef != mid) {
                if (lf[v] < 0) lf[v] = newNode();
                v = lf[v], r = m;
            } else {
                if (f(nw, r) > f(t[v], r)) {
                    if (rt[v] < 0) rt[v] = newNode();
                    v = rt[v], l = m + 1;
                } else return;
            }
        }
    }
    void addLine(ll m, ll b) { add({m, b}, 0, X0, X1); }
    void addSegment(ll m, ll b, ll lx, ll rx, int v = 0, ll l = LLONG_MIN, ll r = LLONG_MIN) {
        if (l == LLONG_MIN) l = X0, r = X1;
        if (rx < l || r < lx) return;
        if (lx <= l && r <= rx) { add({m, b}, v, l, r); return; }
        ll md = l + (r - l) / 2;
        if (lf[v] < 0) lf[v] = newNode();
        if (rt[v] < 0) rt[v] = newNode();
        addSegment(m, b, lx, rx, lf[v], l, md), addSegment(m, b, lx, rx, rt[v], md + 1, r);
    }
    ll query(ll x) {
        ll r = LLONG_MIN, l = X0, rr = X1;
        for (int v = 0; v >= 0;) {
            r = max(r, f(t[v], x));
            if (l == rr) break;
            ll m = l + (rr - l) / 2;
            if (x <= m) v = lf[v], rr = m;
            else v = rt[v], l = m + 1;
        }
        return r;
        // LLONG_MIN/4 if no line covers x
    }
};
// FOR A MINIMUM: insert (-m, -b) and negate the answer.
// PERSISTENT / MERGEABLE: see 05 - PersistentLiChaoTree.cpp. MERGING two Li Chao trees costs
// O(n log X) amortised in total (push the loser line down into the other subtree, exactly like
// segment-tree merging) - that is how you do convex-hull DP on a tree without small-to-large.
```

8.1.5 PersistentLiChaoTree

Persistent Li Chao Tree - Space: O(N log X), Insert Line: O(log X), Query: O(log X)

```
// Version-persistent Li Chao Segment Tree for tree / history DP queries
const ll maxN = 1e7 + 5;

struct Line {
    ll m, c;

    Line() : m(0), c(llinf) {}

    Line(ll m, ll c) : m(m), c(c) {}
};

ll sub(ll x, Line l) {
    return x * l.m + l.c;
}

// Persistent Li Chao
struct Node {
// range I am responsible for
    Line line;
    Node *left, *right;

    Node() {
        left = right = NULL;
    }

    Node(ll m, ll c) {
        line = Line(m, c);
        left = right = NULL;
    }

    void extend(int l, int r) {
        if (left == NULL && l != r) {
            left = new Node();
            right = new Node();
        }
    }

    Node *copy(Node *node) {
        Node *newNode = new Node;
        newNode->left = node->left;
        newNode->right = node->right;
        newNode->line = node->line;
        return newNode;
    }

    Node *add(Line toAdd, int l, int r) {
        int mid = l + (r - l) / 2;    // NOT (l+r)/2: that truncates toward ZERO, so on a negative
                                      // interval the left child equals the parent -> infinite recursion
        Node *cur = copy(this);
        if (l == r) {
            if (sub(l, toAdd) < sub(l, cur->line))
                swap(toAdd, cur->line);
            return cur;
        }
        bool lef = sub(l, toAdd) < sub(l, cur->line);
        bool midE = sub(mid + 1, toAdd) < sub(mid + 1, cur->line);
        if (midE)
            swap(cur->line, toAdd);
        cur->extend(l, r);
        if (lef != midE) {
            cur->left = cur->left->add(toAdd, l, mid);
        } else {
            if (mid + 1 > r)
                return cur;
            cur->right = cur->right->add(toAdd, mid + 1, r);
        }
        return cur;
    }

    Node *add(Line toAdd) {
        return add(toAdd, -maxN + 1, maxN - 1);
    }
```

```
    ll query(ll x, int l, int r) {
        int mid = l + (r - l) / 2;    // NOT (l+r)/2:
    ↪ that truncates toward ZERO, so on a negative
                                      // interval the
    ↪ left child equals the parent -> infinite
    ↪ recursion
        if (l == r || left == NULL)
            return sub(x, line);
        extend(l, r);
        if (x <= mid)
            return min(sub(x, line), left->query(x,
    ↪ l, mid));
        else
            return min(sub(x, line), right->query(x,
    ↪ mid + 1, r));
    }

    ll query(ll x) {
        return query(x, -maxN + 1, maxN - 1);
    }

    void clear() {
        if (left != NULL) {
            left->clear();
            right->clear();
        }
        delete this;
    }
};

const int N = 200005;                          //
    ↪ #versions; also in 01 - Template.cpp
Node *tree[N];
```

8.1.6 KnuthDP

KNUTH OPTIMIZATION - interval DP dp[i][j] = min_{i<=k<j}(dp[i][k] + dp[k+1][j]) + cost(i,j)

```
// from O(n^3) to O(n^2), by restricting k to
    ↪ [opt[i][j-1], opt[i+1][j]].
// THE HYPOTHESIS IS ON cost, NOT ON opt.
    ↪ ‘opt[i][j-1] <= opt[i][j] <= opt[i+1][j]‘ is the
// CONCLUSION - you cannot test it before running.
    ↪ What you must check is BOTH of:
//    (1) QUADRANGLE INEQUALITY (Monge):  cost(a,c) +
    ↪ cost(b,d) <= cost(a,d) + cost(b,c)
//        for all a <= b <= c <= d.  Equivalent local
    ↪ form, and the only one checkable by hand:
//        cost(a,c) + cost(a+1,c+1) <= cost(a+1,c) +
    ↪ cost(a,c+1).
//    (2) MONOTONE ON INTERVALS:  a <= b <= c <= d  =>
    ↪  cost(b,c) <= cost(a,d).
// Both hold for: sum of a non-negative array over
    ↪ [i,j]; (S_j - S_i)^2; f(S_j - S_i) with f convex
// and a[] >= 0; "number of equal pairs inside
    ↪ [i,j]". They FAIL for anything built from max,
    ↪ and
// monotonicity fails as soon as a[] can be negative
    ↪ (QI can still hold - D&C needs only QI,
// Knuth needs both). Applied without them there is
    ↪ no crash and no TLE, just a wrong answer.
// Classic uses: optimal BST, merging n piles of
    ↪ stones, CF 1101F-style interval costs.
struct KnuthDP {
    int n;
    vector<vector<ll>> dp;
    vector<vector<int>> opt;

    KnuthDP(int n = 0) : n(n), dp(n + 2, vector<ll>(n
    ↪ + 2, 0)), opt(n + 2, vector<int>(n + 2, 0)) {}

    // User-defined cost function cost(i, j)
    ll solve(function<ll(int, int)> cost) {
        for (int i = 1; i <= n; i++) {
            opt[i][i] = i;
        }

        for (int len = 2; len <= n; len++) {
            for (int i = 1; i + len - 1 <= n; i++) {
                int j = i + len - 1;
                dp[i][j] = llinf;
                int L = opt[i][j - 1], R = opt[i +
    ↪ 1][j];
                for (int k = L; k <= min(R, j - 1);
    ↪ k++) {
                    ll cur = dp[i][k] + dp[k + 1][j]
    ↪ + cost(i, j);
                    if (cur < dp[i][j]) {
                        dp[i][j] = cur;
                        opt[i][j] = k;
                    }
                }
            }
        }
        return dp[1][n];
    }
};
```

8.1.7 AliensTrick

ALIENS TRICK (Lagrangian relaxation / wqs binary search) - O(n log(range)).

```
// Drops an "exactly k pieces" constraint: charge
    ↪ lambda per piece, solve the UNCONSTRAINED DP,
// and binary search lambda until the optimum uses k
    ↪ pieces. Answer = cost(lambda) - k*lambda.
// PRECONDITION: f(k), the optimum with exactly k
    ↪ pieces, must be CONVEX in k (for a minimum;
// concave for a maximum). NOT "strictly" -
    ↪ strictness is never needed and rarely true.
// THREE THINGS THAT MAKE IT CORRECT, all easy to get
    ↪ wrong:
//  (1) dp_solver MUST return the LARGEST optimal
    ↪ count for that lambda. With f convex, the
//      difference d(k) = f(k) - f(k-1) is
    ↪ non-decreasing; the search below finds the
    ↪ largest
//      lambda with count >= k, and then d(k) <=
    ↪ -lambda <= d(k+1), so k really is optimal at
//      that lambda and cost - k*lambda = f(k). Any
    ↪ other tie-break makes the last line wrong.
//  (2) An integer lambda suffices ONLY if all costs
    ↪ are integers.
//  (3) [min_penalty, max_penalty] must straddle
    ↪ every slope of f; assert it.
// CONVEX families: "k pairwise non-adjacent items",
    ↪ "exactly k segments with a Monge cost",
// "k disjoint intervals", "min-cost flow of value k"
    ↪ (flow cost is always convex in the value),
// matroid-rank-constrained optima. TRAPS: two
    ↪ coupled constraints, or a cost rewarding a parity
// of k. CHECK IT: brute-force f(k) for n <= 60 and
    ↪ assert f(k+1)-f(k) >= f(k)-f(k-1).
struct AliensTrick {
    // Binary searches optimal penalty lambda to
    ↪ choose exactly K items
    // Returns minimum cost to choose exactly K items
```

```
    static ll solve(int n, int k, ll min_penalty, ll
↪ max_penalty, function<pair<ll, int>(ll)>
↪ dp_solver) {
        ll l = min_penalty, r = max_penalty,
↪ best_lambda = 0;

        while (l <= r) {
            ll mid = l + (r - l) / 2;
            auto [cost, count] = dp_solver(mid);
            if (count >= k) {
                best_lambda = mid;
                l = mid + 1; // Increase penalty if
↪ we used too many items
            } else {
                r = mid - 1; // Decrease penalty if
↪ we used too few items
            }
        }

        // True cost = cost_with_penalty - (k *
↪ best_lambda)
        auto [cost, count] = dp_solver(best_lambda);
        return cost - (ll)k * best_lambda;
    }
};
```

8.1.8 AliensTrickExample

ALIENS TRICK (Lagrangian / wqs binary search) - drops the "exactly k pieces" constraint by charging

```
// a penalty lambda per piece, solving the
↪ unconstrained DP, and binary searching lambda
↪ until the
// optimal solution uses k. REQUIRES the optimum as a
↪ function of k to be CONVEX; check that first.
// 1. DP State Structure
struct DPState {
    long long cost;
    long long count; // Number of items/segments
↪ picked

    // TIE-BREAKER LOGIC:
    // If costs are equal, we maximize the count to
↪ handle collinear points on the convex hull.
    // If you are writing a MINIMIZATION problem with
↪ min(), change this to:
    // if (cost != other.cost) return cost >
↪ other.cost; (since priority queue / max
↪ naturally takes the "biggest")
    // Actually, when using standard >, < operators
↪ for min/max DP:
    bool operator<(const DPState& other) const {
        if (cost != other.cost) return cost <
↪ other.cost;
        return count < other.count; // Always prefer
↪ taking MORE items if costs are tied
    }
};

void solve_aliens() {
    int n, k;
    cin >> n >> k;
    // Read ABC 218 H input: A array is of size N-1
    vector<long long> A(n);
    for(int i = 1; i < n; i++) cin >> A[i];
    // If K > N / 2, swap colors so K <= N / 2 (the
↪ score is symmetric)
    if (k > n / 2) k = n - k;
    // Construct 1-indexed values array 'a' where
↪ a[i] = A[i-1] + A[i]
    vector<long long> a(n + 1);
    for (int i = 1; i <= n; i++) {
        a[i] = A[i - 1] + (i < n ? A[i] : 0);
    }
    // 2. The Unconstrained DP
    // Returns the optimal {cost, count} for a given
↪ penalty.
    auto check_dp = [&](long long penalty) -> DPState
↪ {
        // dp[i][0]: Max score in prefix i, ending
↪ completely outside a segment
        // dp[i][1]: Max score in prefix i, ending
↪ inside an active segment
        vector<vector<DPState>> dp(n + 1,
↪ vector<DPState>(2, {0, 0}));

        for (int i = 1; i <= n; i++) {
            // Transition 0: We don't pick item i
            dp[i][0] = max(dp[i-1][0], dp[i-1][1]);

            // Transition 1: We pick item i
            // Rule: ONLY apply penalty when STARTING
↪ a new pick/segment!
            // Type B logic: To ensure non-adjacency,
↪ we must transition from dp[i-1][0].

            // --- IF MAXIMIZING PROFIT ---
            DPState start_new = {dp[i-1][0].cost +
↪ a[i] - penalty, dp[i-1][0].count + 1};

            // Replaced max(start_new, continue_old)
↪ with just start_new
            dp[i][1] = start_new;
        }
        return max(dp[n][0], dp[n][1]);
    };

    // 3. WQS Binary Search
    // The bounds must be large enough to cover the
↪ maximum possible penalty.
    // FIXED: Dropped bounds to 2e9 to prevent k *
↪ mid_penalty from overflowing long long
    long long low = -2e9 - 7, high = 2e9 + 7;
    long long best_ans = 0;

    while (low <= high) {
        long long mid_penalty = low + (high - low) /
↪ 2;

        DPState res = check_dp(mid_penalty);

        // We want EXACTLY k items.
        // If we picked >= k items, it means the
↪ penalty wasn't harsh enough to stop us.
        // Even if res.count > k, we save the answer
↪ because of collinear points on the hull.
        if (res.count >= k) {

            // --- IF MAXIMIZING PROFIT ---
            // We subtracted the penalty K times, so
↪ we add it back.
            best_ans = res.cost + (1LL * k *
↪ mid_penalty);

            // --- IF MINIMIZING COST ---
            // best_ans = res.cost - (1LL * k *
↪ mid_penalty);
```

```
            // Penalty was too weak, increase it to force picking fewer items next time
            low = mid_penalty + 1;
        } else {
            // We picked too few items. The penalty was too harsh.
            high = mid_penalty - 1;
        }
    }

    // Output the final answer
    cout << best_ans << "\n";
}
```

8.1.9 MaxAverageSubarray

UPPER hull (maximum) and this one is the LOWER hull (minimum) - they are NOT

```
// NOTE: 02 - CHT.cpp also declares `struct Line` / `struct HullDynamic`, but that one is the
//       UPPER hull (maximum) and this one is the LOWER hull (minimum) - they are NOT
//       interchangeable. If you need both, rename one pair to UpperHull / LowerHull.
/*
 * DYNAMIC CONVEX HULL TRICK (CHT) - AVERAGE SUBARRAY TEMPLATE
 *
 * HOW TO USE THE 4 VARIATIONS:
 *
 * 1. MAX Average, LONGEST length  (Default)
 *    - Code: Use `lhs < rhs` in Query::operator<
 *    - Call: hull.insert_line(i, S[i]); hull.query_tangent_index(i, S[i]);
 *
 * 2. MAX Average, SHORTEST length
 *    - Code: Use `lhs <= rhs` in Query::operator<
 *    - Call: hull.insert_line(i, S[i]); hull.query_tangent_index(i, S[i]);
 *
 * 3. MIN Average, LONGEST length
 *    - Code: Use `lhs < rhs` in Query::operator<
 *    - Call: hull.insert_line(i, -S[i]); hull.query_tangent_index(i, -S[i]); // Negated
 *
 * 4. MIN Average, SHORTEST length
 *    - Code: Use `lhs <= rhs` in Query::operator<
 *    - Call: hull.insert_line(i, -S[i]); hull.query_tangent_index(i, -S[i]); // Negated
 *
 */

struct Query {
    ll i, Si;
};

struct Line {
    ll m, b;
    mutable function<const Line *()> succ;

    bool operator<(const Line &other) const {
        return m < other.m;
    }

    bool operator<(const Query &q) const {
        const Line *s = succ();
        if (!s) return false;

        // CORRECTED ALGEBRA: Direct cross-multiplication of slopes
        __int128_t lhs = (__int128_t)(q.Si - b) * (q.i - s->m);
        __int128_t rhs = (__int128_t)(q.Si - s->b) * (q.i - m);

        // --- TIE-BREAKING CONTROL ---
        // Change this return statement based on length requirements:
        // return lhs < rhs;  // <-- Use for LONGEST subarray
        // return lhs <= rhs; // <-- Use for SHORTEST subarray
        return lhs < rhs;
    }
};

struct HullDynamic : public multiset<Line, less<>> {
    bool bad(iterator y) {
        auto z = next(y);
        if (y == begin()) {
            if (z == end()) return false;
            // CORRECTED: Lower hull keeps smallest y
            return y->m == z->m && y->b >= z->b;
        }
        auto x = prev(y);
        if (z == end()) return y->m == x->m && y->b >= x->b;

        return (__int128_t)(x->b - y->b) * (z->m - y->m) <= (__int128_t)(y->b - z->b) * (y->m - x->m);
    }

    void insert_line(ll m, ll b) {
        auto y = insert({m, b});
        y->succ = [=] { return next(y) == end() ? 0 : &*next(y); };
        if (bad(y)) {
            erase(y);
            return;
        }
        while (next(y) != end() && bad(next(y))) erase(next(y));
        while (y != begin() && bad(prev(y))) erase(prev(y));
    }

    int query_tangent_index(ll i, ll Si) {
        auto l = *lower_bound(Query{i, Si});
        return l.m;
    }
};

// USAGE (longest subarray with average >= 0, after subtracting the target from every element):
//   S[0..n] = prefix sums; the answer for each i is i - (tangent index from the hull of (j, S[j])).
//   HullDynamic hull; hull.insert_line(0, 0);
//   for (int i = 1; i <= n; i++) {
//       int k = hull.query_tangent_index(i, S[i]);
//           // best j < i
//       ans[i] = i - k;
```

```
//       hull.insert_line(i, S[i]);
//   }
// WHY IT WORKS: avg(j+1..i) >= 0  <=>  S[i] - S[j]
  ↪ >= 0, and among all valid j you want the
// SMALLEST, which is the point of tangency from (i,
  ↪ S[i]) to the lower hull of the points (j, S[j]).
// Binary search on the answer + prefix sums is the
  ↪ shorter O(n log n) alternative; this is O(n).
// THE FOUR VARIATIONS, all from the same hull - only
  ↪ the tie-break and the sign change:
//   MAX average, LONGEST  segment: insert(i, S[i]),
  ↪ tangent(i, S[i]),  strict '<'  in the compare
//   MAX average, SHORTEST segment: insert(i, S[i]),
  ↪ tangent(i, S[i]),  non-strict '<='
//   MIN average, LONGEST  segment: insert(i, -S[i]),
  ↪ tangent(i, -S[i]), strict '<'
//   MIN average, SHORTEST segment: insert(i, -S[i]),
  ↪ tangent(i, -S[i]), non-strict '<='
// (Minimising is just maximising the negated prefix
  ↪ sums; the tie-break decides which of two
// equally-good segments you keep, which is what
  ↪ "longest" versus "shortest" means here.)
```

8.1.10 SlopeTrick

SLOPE TRICK - maintain a CONVEX piecewise-linear f(x) as two heaps of its breakpoints.

```
// L = left slopes (max-heap), R = right slopes
  ↪ (min-heap), minf = current minimum value.
// Every operation below is O(log n). Classic use:
  ↪ "minimum total |change| to make an array
// non-decreasing", scheduling with
  ↪ earliness/tardiness penalties, and any DP where
//   dp_i(x) = min over y<=x of dp_{i-1}(y)  + |x -
  ↪ a_i|.
struct SlopeTrick {
    priority_queue<ll> L;
  ↪        // left breakpoints (max-heap)
    priority_queue<ll, vector<ll>, greater<ll>> R;
  ↪        // right breakpoints (min-heap)
    ll minf = 0, addL = 0, addR = 0;
  ↪        // lazy shifts of each side

    ll topL() { return L.top() + addL; }
    ll topR() { return R.top() + addR; }
    void pushL(ll x) { L.push(x - addL); }
    void pushR(ll x) { R.push(x - addR); }

    void addConst(ll c) { minf += c; }
    void addRampR(ll a) {
  ↪        // f(x) += max(0, x - a)
        if (!L.empty()) minf += max(0LL, topL() - a);
        pushL(a);
        pushR(topL()); L.pop();
    }
    void addRampL(ll a) {
  ↪        // f(x) += max(0, a - x)
        if (!R.empty()) minf += max(0LL, a - topR());
        pushR(a);
        pushL(topR()); R.pop();
    }
    void addAbs(ll a) { addRampL(a), addRampR(a); }
  ↪        // f(x) += |x - a|
    void prefixMin() { while (!R.empty()) R.pop(); }
  ↪        // f(x) = min_{y<=x} f(y)
    void suffixMin() { while (!L.empty()) L.pop(); }
  ↪        // f(x) = min_{y>=x} f(y)
    void shift(ll dl, ll dr) { addL += dl, addR +=
  ↪ dr; }     // f(x) -> min over [x-dr, x-dl]
    ll minVal() const { return minf; }
};
// RECIPE for "make a[] non-decreasing with min sum
  ↪ |b_i - a_i|":
//   for each a_i:  st.prefixMin();  st.addAbs(a_i);
  ↪  answer = st.minVal()
// For "strictly increasing", first replace a_i by
  ↪ a_i - i.
```

8.1.11 MinPlusConvolution

MIN-PLUS / MAX-PLUS CONVOLUTION: c[k] = min_i (a[i] + b[k-i]).

```
// In general this is quadratic and CONJECTURED to
  ↪ stay that way (the MinConv hypothesis:
// MinConv, MaxConv, unbounded knapsack and 0/1
  ↪ knapsack are all equivalent under subquadratic
// reductions). So the ONLY fast versions rely on
  ↪ convexity.

// BOTH CONVEX -> O(n + m). The difference sequence
  ↪ of the answer is the sorted MERGE of the two
// difference sequences: greedily always take the
  ↪ smaller next slope.
vector<ll> minPlusConvexConvex(const vector<ll>& a,
  ↪ const vector<ll>& b) {
    int n = a.size(), m = b.size();
    vector<ll> c(n + m - 1);
    c[0] = a[0] + b[0];
    int i = 0, j = 0;
    for (int k = 1; k < n + m - 1; k++) {
        ll da = (i + 1 < n) ? a[i + 1] - a[i] :
  ↪ LLONG_MAX;
        ll db = (j + 1 < m) ? b[j + 1] - b[j] :
  ↪ LLONG_MAX;
        if (da <= db) c[k] = c[k - 1] + da, i++;
        else c[k] = c[k - 1] + db, j++;
    }
    return c;
}
// MAX-PLUS with both CONCAVE: same code, take the
  ↪ LARGER slope first (or negate and reuse).
vector<ll> maxPlusConcaveConcave(vector<ll> a,
  ↪ vector<ll> b) {
    for (ll& x : a) x = -x;
    for (ll& x : b) x = -x;
    auto c = minPlusConvexConvex(a, b);
    for (ll& x : c) x = -x;
    return c;
}
// ONLY ONE CONVEX -> the matrix X[k][i] = a[i] +
  ↪ b[k-i] is totally monotone (Monge), so the row
// minima can be found with SMAWK in O(n + m), or
  ↪ with divide & conquer in O(n log n):
//   solve(kl, kr, il, ir): take k = mid, scan i in
  ↪ [il, ir] for the best, recurse with the
//   split at the optimum. Same skeleton as
  ↪ DivideAndConquerDP.
// 'valid(k)' must return the legal range of i for
  ↪ output index k - for the convolution above
// that is [max(0, k-m+1), min(n-1, k)]. Intersecting
  ↪ with it is what keeps 'best' well defined;
// without it an empty candidate set leaves best = -1
  ↪ and poisons both recursive calls.
template <class F, class V>
void dcOpt(int kl, int kr, int il, int ir,
  ↪ vector<ll>& c, F cost, V valid) {
    if (kl > kr) return;
    int km = (kl + kr) / 2, best = -1;
```

```
    ll bv = LLONG_MAX;
    auto [lo, hi] = valid(km);
    for (int i = max(il, lo); i <= min(ir, hi); i++) {
        ll v = cost(km, i);
        if (v < bv) bv = v, best = i;
    }
    c[km] = bv;
    if (best < 0) best = max(il, lo);
    // no candidate: keep the bounds sane
    dcOpt(kl, km - 1, il, best, c, cost, valid);
    dcOpt(km + 1, kr, best, ir, c, cost, valid);
}
// NEITHER CONVEX -> O(n*m) is the best you should plan for, and that is not laziness: min-plus
// convolution, 0/1 knapsack, unbounded knapsack and tree knapsack are subquadratic-EQUIVALENT,
// so a general fast merge would break all four. Look for convexity, a small value range, or a
// bitset instead of a cleverer convolution.
// WHERE THIS APPEARS: merging two "cost as a function of how many you take" curves - group
// knapsack, tree DP where each child contributes a convex cost curve, and combining the
// per-part curves produced by Alien's trick.
```

8.1.12 OneDOneD

1D/1D DP OPTIMISATION - the self-referential case that divide & conquer structurally cannot do.

```
//     dp[i] = min over j < i of ( dp[j] + w(j, i) )
// D&C optimisation needs the LAYERED form dp[k][i] = min(dp[k-1][j] + w) so that the whole
// previous layer is already known. Here dp[j] is produced by the very loop that consumes it, so
// you must decide dp[i] before you may use it. That is what this file solves.
// PRECONDITION: w satisfies the QUADRANGLE INEQUALITY (Monge)
//     w(a,c) + w(b,d) <= w(a,d) + w(b,c)   for all a <= b <= c <= d
// checkable in the local form  w(a,c) + w(a+1,c+1) <= w(a+1,c) + w(a,c+1).
// Then the optimal j for i is NON-DECREASING in i, so the array of "who is currently best for
// each future i" is a sequence of INTERVALS, each owned by one j - and finalising dp[i] can only
// append one new interval and overwrite a suffix of the old ones. Keep them on a stack.
// O(n log n) (the binary search for the crossover); O(n) if the crossover is O(1) to find.
template <class F>
vector<ll> onedOned(int n, F w) {
    // w(j, i) for j < i
    vector<ll> dp(n + 1, llinf);
    dp[0] = 0;
    vector<array<int, 3>> st;
    // {owner j, from, to} intervals
    st.push_back({0, 1, n});
    int head = 0;
    for (int i = 1; i <= n; i++) {
        while (head < (int)st.size() && st[head][2] < i) head++;
        dp[i] = dp[st[head][0]] + w(st[head][0], i);
        // the owner of position i
        st[head][1] = i;
        // now insert i as a candidate owner for the suffix it wins
        auto better = [&](int a, int b, int x) {
            // is a better than b at x?
            return dp[a] + w(a, x) <= dp[b] + w(b, x);
        };
        while (!st.empty() && st.back()[1] > i && better(i, st.back()[0], st.back()[1]))
            st.pop_back();
        if (st.empty() || head >= (int)st.size())
            st.push_back({i, i + 1, n});
        else {
            auto [j, l, r] = st.back();
            if (better(i, j, r)) {
                // i wins part of this interval
                int lo = max(l, i + 1), hi = r, pos = r + 1;
                while (lo <= hi) {
                // first x where i beats j
                    int m = (lo + hi) / 2;
                    better(i, j, m) ? (pos = m, hi = m - 1) : lo = m + 1;
                }
                st.back()[2] = pos - 1;
                if (pos <= n) st.push_back({i, pos, n});
            } else if (r < n) st.push_back({i, r + 1, n});
        }
    }
    return dp;
}
/* WHEN YOU NEED WHICH OPTIMISATION - the whole family on one card
   dp[i] = min_j (dp[j] + w(j,i)),   w Monge, SELF-REFERENTIAL       -> THIS FILE, O(n log n)
   dp[k][i] = min_j (dp[k-1][j] + w(j,i)),  w Monge, LAYERED          -> D&C opt (01), O(n k log n)
   dp[i][j] = min_k (dp[i][k] + dp[k+1][j]) + w(i,j),  w Monge AND
              monotone on intervals                  -> Knuth (06), O(n^2)
   dp[i] = min_j (m_j * x_i + b_j)                   -> CHT (02) / Li Chao (04)
   "exactly k pieces", optimum convex in k           -> Aliens (07)
   min-plus convolution of a CONVEX sequence with anything            -> SMAWK / D&C (11)
   the query point moves monotonically and lines are range-updated    -> kinetic segment tree
   VERIFY the quadrangle inequality numerically on random quadruples before trusting any of them:
   a DP optimisation applied without its precondition is a silent wrong answer, never a crash.
   COMMON MONGE COSTS: (S_j - S_i)^2; f(S_j - S_i) with f convex and a[] >= 0; c * (j - i);
   "number of equal pairs inside [i,j]"; a_i * b_j with a non-increasing and b non-decreasing.
   NOT MONGE: anything built from max, and anything with negative a[] where you also need the
   interval-monotonicity half (D&C only needs QI; Knuth needs both). */
```

8.1.13 KineticSegmentTree

KINETIC SEGMENT TREE ("segment tree beats" for lines). Each position i owns a LINE

```
// f_i(x) = a_i * x + b_i and a current parameter x; the tree supports
//     addX(l, r, d)   : x += d for every position in [l, r]   (d >= 0, MONOTONE)
//     queryMax(l, r)  : max over the range of a_i * x_i + b_i
```

```
//     setLine(i, a, b): replace one line
// in AMORTISED O(log^2 n) per operation (O(n log^2
  ↪ n) total), and that amortisation is the point:
// a Li Chao tree answers "many lines, one query
  ↪ point" while THIS answers "one line per position,
// query points that drift". 12 - OneDOneD.cpp lists
  ↪ it as the tool for "the query point moves
// monotonically and the lines are range-updated" -
  ↪ this is that tool.
// PRECONDITION FOR THE COMPLEXITY (not for
  ↪ correctness): x only ever INCREASES. Each node
  ↪ keeps
// ‘melt‘ = how much further x may advance before the
  ↪ argmax inside it changes; a decrease would
// invalidate every melt and destroy the
  ↪ potential-function argument (the total number of
  ↪ argmax
// changes over the whole run is O(n log^2 n) only
  ↪ for monotone x).
// INDEX CONVENTION: 0-based positions, inclusive
  ↪ ranges, x starts at 0 for every position.
struct KineticSegTree {
    struct Node { ll val, slope, melt; };
  ↪          // melt = INF when nothing can flip
    int n;
    vector<Node> t;
    vector<ll> lz;
  ↪          // pending x-advance for a subtree
    KineticSegTree(const vector<ll>& a, const
  ↪ vector<ll>& b) {
        n = a.size(), t.assign(4 * n, {0, 0, 0}),
  ↪ lz.assign(4 * n, 0);
        build(1, 0, n - 1, a, b);
    }
    Node merge(const Node& L, const Node& R) {
        const Node &w = L.val >= R.val ? L : R, &o =
  ↪ L.val >= R.val ? R : L;
        ll cross = llinf;
        if (o.slope > w.slope) cross = (w.val -
  ↪ o.val) / (o.slope - w.slope) + 1;
        return {w.val, w.slope, min({L.melt, R.melt,
  ↪ cross})};
    }
    void build(int v, int l, int r, const vector<ll>&
  ↪ a, const vector<ll>& b) {
        if (l == r) { t[v] = {b[l], a[l], llinf};
  ↪ return; }
        int m = (l + r) / 2;
        build(2 * v, l, m, a, b), build(2 * v + 1, m
  ↪ + 1, r, a, b);
        t[v] = merge(t[2 * v], t[2 * v + 1]);
    }
    void apply(int v, ll d) {
  ↪          // legal only when d < t[v].melt
        t[v].val += t[v].slope * d;
        if (t[v].melt != llinf) t[v].melt -= d;
        lz[v] += d;
    }
    void push(int v) {
        if (!lz[v]) return;
        apply(2 * v, lz[v]), apply(2 * v + 1, lz[v]),
  ↪ lz[v] = 0;
    }
    void addX(int v, int l, int r, int ql, int qr, ll
  ↪ d) {
        if (qr < l || r < ql) return;
        if (ql <= l && r <= qr && d < t[v].melt) {
  ↪ apply(v, d); return; }
        if (l == r) { t[v].val += t[v].slope * d;
  ↪ return; }    // a leaf never "melts"
        push(v);
        int m = (l + r) / 2;
        addX(2 * v, l, m, ql, qr, d), addX(2 * v + 1,
  ↪ m + 1, r, ql, qr, d);
        t[v] = merge(t[2 * v], t[2 * v + 1]);
  ↪          // the RECURSION is the beats step
    }
    ll query(int v, int l, int r, int ql, int qr) {
        if (qr < l || r < ql) return LLONG_MIN;
        if (ql <= l && r <= qr) return t[v].val;
        push(v);
        int m = (l + r) / 2;
        return max(query(2 * v, l, m, ql, qr),
  ↪ query(2 * v + 1, m + 1, r, ql, qr));
    }
    void setLine(int v, int l, int r, int i, ll a, ll
  ↪ b, ll x) {
        if (l == r) { t[v] = {a * x + b, a, llinf};
  ↪ return; }
        push(v);
        int m = (l + r) / 2;
        i <= m ? setLine(2 * v, l, m, i, a, b, x) :
  ↪ setLine(2 * v + 1, m + 1, r, i, a, b, x);
        t[v] = merge(t[2 * v], t[2 * v + 1]);
    }
    void addX(int l, int r, ll d) { addX(1, 0, n - 1,
  ↪ l, r, d); }
    ll queryMax(int l, int r) { return query(1, 0, n
  ↪ - 1, l, r); }
    void setLine(int i, ll a, ll b, ll x) {
  ↪ setLine(1, 0, n - 1, i, a, b, x); }
};
/* USING IT
 THE MODEL PROBLEM: process items left to right; item
  ↪ i contributes a_i * (how long it has been
  ↪ open) + b_i, and you repeatedly ask for the best
  ↪ open item. Every "time step" is one range
  ↪ addX, every answer one queryMax.
 SLOPE TRICK / APERIODIC DPs of the form dp[i] =
  ↪ max_j (dp[j] + w_j * (i - j)) become: insert a
  ↪ line at position j when it becomes available,
  ↪ addX(0, n-1, 1) each step, query the max. That
  ↪ is the self-referential case D&C optimisation
  ↪ cannot do (see 12 - OneDOneD.cpp).
 THE MELT FORMULA is integer division; with
  ↪ real-valued slopes use long double and a small
  ↪ epsilon, or scale to integers. ‘cross‘ must be the
  ↪ FIRST x at which the loser strictly wins,
  ↪ which is why it is (diff / dslope) + 1 - dropping
  ↪ the +1 makes the tree do nothing and the
  ↪ answers silently stale.
 DECREASING x, or arbitrary jumps, still give CORRECT
  ↪ answers (the recursion just triggers more
  ↪ often) but the complexity degrades to O(n) per
  ↪ operation in the worst case.
 SIMPLER ALTERNATIVES FIRST: if all lines are
  ↪ inserted once and queries are offline-sortable,
  ↪ use
  ↪ CHT (02) or Li Chao (04). Reach for this only when
  ↪ positions are updated in RANGES. */
```

8.2 Bitmask DP

8.2.1 SOS_DP

Computes the sum of all subsets for every bitmask in O(N * 2^N)

```
vector<long long> SOS_DP(int n_bits, const
  ↪ vector<long long>& a) {
    int max_mask = 1 << n_bits;
    vector<long long> dp = a; // dp[mask] initially
  ↪ holds exactly the value for mask

    for (int i = 0; i < n_bits; ++i) {
        for (int mask = 0; mask < max_mask; ++mask) {
            if (mask & (1 << i)) {
                dp[mask] += dp[mask ^ (1 << i)];
            }
        }
    }
    return dp; // dp[mask] now contains sum of all
  ↪ submasks of mask
}
```

8.2.2 BitmaskDP

BITMASK DP - the n <= 20 toolkit.

```
// MEMORY: vector<vector<ll>> dp(1<<20,
  ↪ vector<ll>(20)) is ~193 MB (24 B of vector
  ↪ header per row).
// Flatten to ONE vector<ll> of size (1<<n)*n (168
  ↪ MB) or vector<int> (84 MB), or iterate masks in
// increasing popcount and keep only two layers. At n
  ↪ = 20, 2^n * n^2 = 4e8 - usually not intended.

// TSP: shortest Hamiltonian cycle from 0 back to 0.
  ↪ O(2^n * n^2).
ll tsp(const vector<vector<ll>>& d) {
    int n = d.size(), N = 1 << n;
    vector<vector<ll>> dp(N, vector<ll>(n, llinf));
    dp[1][0] = 0;
    for (int m = 1; m < N; m++)
        for (int u = 0; u < n; u++) if ((m >> u & 1)
  ↪ && dp[m][u] < llinf)
            for (int v = 0; v < n; v++) if (!(m >> v
  ↪ & 1) && d[u][v] < llinf)
                dp[m | 1 << v][v] = min(dp[m | 1 <<
  ↪ v][v], dp[m][u] + d[u][v]);
    ll best = llinf;
    for (int u = 1; u < n; u++) if (dp[N - 1][u] <
  ↪ llinf && d[u][0] < llinf)
        best = min(best, dp[N - 1][u] + d[u][0]);
    return n == 1 ? 0 : best;
  ↪ // drop the +d[u][0] for a Hamiltonian PATH
}

// MINIMUM SET COVER / partition into the fewest
  ↪ valid groups. O(3^n) via submask enumeration.
// ok[m] = true if the set m is a legal single group.
int minPartition(int n, const vector<char>& ok) {
    int N = 1 << n;
    vector<int> dp(N, inf);
    dp[0] = 0;
    for (int m = 1; m < N; m++) {
        int low = m & -m;
  ↪ // fix the lowest element: kills the n! factor
        for (int s = m; s; s = (s - 1) & m) if ((s &
  ↪ low) && ok[s])
            dp[m] = min(dp[m], dp[m ^ s] + 1);
    }
    return dp[N - 1];
}
// COUNTING the partitions instead: same loop with
  ↪ dp[m] += dp[m ^ s].
// If groups are UNORDERED, fixing the lowest element
  ↪ (above) already prevents double counting.

// ASSIGNMENT (n tasks to n workers, minimum cost),
  ↪ O(2^n * n). dp[m] = best cost using the
// first popcount(m) workers on exactly the task set
  ↪ m.
ll assignment(const vector<vector<ll>>& c) {
    int n = c.size(), N = 1 << n;
    vector<ll> dp(N, llinf);
    dp[0] = 0;
    for (int m = 0; m < N - 1; m++) if (dp[m] <
  ↪ llinf) {
        int i = __builtin_popcount(m);
  ↪ // next worker
        for (int j = 0; j < n; j++) if (!(m >> j & 1))
            dp[m | 1 << j] = min(dp[m | 1 << j],
  ↪ dp[m] + c[i][j]);
    }
    return dp[N - 1];
  ↪ // n > 20 -> use the Hungarian algorithm
}

// COUNTING HAMILTONIAN PATHS / permutations with
  ↪ adjacency constraints: same shape as tsp but
// dp[m][u] += dp[m ^ (1<<u)][v] over allowed v.
// SUBSET-SUM STYLE: when the state is "which items
  ↪ used" AND the order does not matter, prefer
// dp over masks; when only a total matters, use a
  ↪ knapsack - 2^n is not always necessary.
// MEET IN THE MIDDLE: n <= 40 -> split in half,
  ↪ enumerate 2^20 each side, sort + two pointers.
```

8.2.3 SubsetConvolution

ZETA / MOBIUS OVER THE SUBSET LATTICE, and SUBSET CONVOLUTION. 01 - SOS_DP.cpp has the plain

```
// "sum over submasks"; this file adds the INVERSE
  ↪ (which is what turns a sum-over-subsets answer
// back into an exact-set answer) and the convolution
  ↪ that requires the parts to be DISJOINT.
//   OR-convolution   c[k] = sum over i | j = k of
  ↪ a[i] b[j]      -> zeta, multiply, mobius
//   AND-convolution  c[k] = sum over i & j = k of
  ↪ a[i] b[j]      -> superZeta, mul, superMobius
//   XOR-convolution  c[k] = sum over i ^ j = k of
  ↪ a[i] b[j]      -> FWHT, see 06 - Math/05/03
//   SUBSET conv      c[k] = sum over i | j = k AND i
  ↪ & j = 0     -> THIS FILE, O(2^n n^2)
// WHEN: "partition a set into groups" DPs (count
  ↪ ways to split into k connected pieces, minimum
// cost exact cover, chromatic number), where the
  ↪ naive "for each mask, for each submask" is
// O(3^n) - subset convolution replaces that with
  ↪ O(2^n n^2), which at n = 20 is 4e8 vs 3.5e9.
void zeta(vector<ll>& f, int n) {
  ↪          // g[s] = sum over t subset of s
    for (int i = 0; i < n; i++)
        for (int s = 0; s < (1 << n); s++)
            if (s >> i & 1) f[s] = (f[s] + f[s ^ (1
  ↪ << i)]) % MOD;
}
void mobius(vector<ll>& f, int n) {
  ↪          // exact inverse of zeta
    for (int i = 0; i < n; i++)
        for (int s = 0; s < (1 << n); s++)
            if (s >> i & 1) f[s] = (f[s] - f[s ^ (1
  ↪ << i)] % MOD + MOD) % MOD;
```

```
}
void superZeta(vector<ll>& f, int n) {
    for (int i = 0; i < n; i++)
        for (int s = 0; s < (1 << n); s++)
            if (!(s >> i & 1)) f[s] = (f[s] + f[s | (1 << i)]) % MOD;
}
void superMobius(vector<ll>& f, int n) {
    for (int i = 0; i < n; i++)
        for (int s = 0; s < (1 << n); s++)
            if (!(s >> i & 1)) f[s] = (f[s] - f[s | (1 << i)] % MOD + MOD) % MOD;
}
// SUBSET CONVOLUTION. Rank by POPCOUNT: after the zeta transform, a product of the rank-i and
// rank-j layers counts pairs with i | j SUBSET of s and |i| + |j| = k; reading the answer at
// k = popcount(s) forces i | j = s and |i| + |j| = |s|, which for a union is exactly i & j = 0.
// That ranking is the entire idea - without it, disjointness is not expressible as a transform.
vector<ll> subsetConvolution(const vector<ll>& A, const vector<ll>& B, int n) {
    int N = 1 << n;
    vector<vector<ll>> f(n + 1, vector<ll>(N, 0)), g = f, h = f;
    for (int s = 0; s < N; s++) {
        int p = __builtin_popcount(s);
        f[p][s] = A[s], g[p][s] = B[s];
    }
    for (int k = 0; k <= n; k++) zeta(f[k], n), zeta(g[k], n);
    for (int k = 0; k <= n; k++)
        for (int i = 0; i <= k; i++)
            for (int s = 0; s < N; s++) h[k][s] = (h[k][s] + f[i][s] * g[k - i][s]) % MOD;
    for (int k = 0; k <= n; k++) mobius(h[k], n);
    vector<ll> c(N);
    for (int s = 0; s < N; s++) c[s] = h[__builtin_popcount(s)][s];
    return c;
}
/* USING IT
 SET PARTITION / EXACT COVER: let f[s] = 1 if s is a legal piece. The number of ways to cover the
  full set with exactly k pieces is the k-th subset-convolution POWER of f at s = full; dividing
  by k! makes the pieces unordered. Chromatic number = smallest k with (independent sets)^k > 0.
 SUBSET EXP / LOG exist too (O(2^n n^2)) and turn "connected pieces" into "all pieces" and back -
  the standard route for counting CONNECTED labelled structures on subsets.
 MIN-PLUS version (minimum cost exact cover) does NOT work: min-plus has no inverse, so the
  mobius step is invalid. Use the O(3^n) submask enumeration for min/max objectives.
 O(3^n) SUBMASK ENUMERATION is `for (int t = s; t; t = (t - 1) & s)` and remains the right answer
  for n <= 15 or when the operation is not a ring - it is shorter and has no modulus.
 XOR needs FWHT, not this; AND/OR need only the transform pair with no ranking.
 OVERFLOW: everything here is mod MOD. Without a modulus use __int128 or accept unsigned
  wraparound
  for OR/AND (the mobius step is exact in Z, so wraparound cancels). */
```

8.2.4 DynamicSubmaskCount

DYNAMIC SUBMASK COUNTING - a multiset of masks with INSERT, ERASE and the query

```
//     "how many stored masks a satisfy  a & s == a (a is a submask of s)"
// in O(2^(B/2)) per operation, B = number of bits. SOS DP answers this in O(1) but only for a
// STATIC multiset (any insert costs a full O(B 2^B) rebuild); enumerating submasks of s per query
// is O(2^B). This is the meet-in-the-middle between the two, and the technique generalises to any
// "precompute forwards for heavy items, backwards for light items" split.
// THE SPLIT: call a mask HEAVY if popcount > B/2, LIGHT otherwise.
//   LIGHT masks are stored twice: once at their own index, and once at EVERY SUBMASK of theirs
//     (<= 2^(B/2) of them) in sup[], so sup[t] = #light masks that CONTAIN t.
//   HEAVY masks are stored at every SUPERMASK of theirs (<= 2^(B/2) of them) in sub[],
//     so sub[t] = #heavy masks that are CONTAINED in t.
// A query with a light s only needs light answers (a submask of s has popcount <= popcount(s)),
// and is a submask enumeration over s, <= 2^(B/2) steps. A query with a heavy s reads sub[s] for
// the heavy part and inclusion-exclusion over the ZERO bits of s (there are < B/2 of them) for
// the light part: #light masks meeting ~s = sum over non-empty T in ~s of (-1)^(|T|+1) sup[T].
template <int B>
struct SubmaskCounter {
    static const int SZ = 1 << B, H = B / 2;
    vector<int> own, sup, sub;
    int lightTotal = 0;
    SubmaskCounter() : own(SZ, 0), sup(SZ, 0), sub(SZ, 0) {}
    void modify(int m, int d) {
        if (__builtin_popcount(m) <= H) {
            own[m] += d, lightTotal += d;
            for (int t = m;; t = (t - 1) & m) { sup[t] += d; if (!t) break; }
        } else {
            int zeros = ((SZ - 1) ^ m);
            for (int t = zeros;; t = (t - 1) & zeros) { sub[m | t] += d; if (!t) break; }
        }
    }
    void insert(int m) { modify(m, 1); }
    void erase(int m) { modify(m, -1); }
    int count(int s) {
        if (__builtin_popcount(s) <= H) {
            int r = 0;
            for (int t = s;; t = (t - 1) & s) { r += own[t]; if (!t) break; }
            return r;
        }
        int zeros = ((SZ - 1) ^ s), bad = 0;
        for (int t = zeros; t; t = (t - 1) & zeros)
            bad += (__builtin_popcount(t) & 1) ?
```

```
    ↪ sup[t] : -sup[t];
        return sub[s] + lightTotal - bad;
    }
};
/* NOTES
 THE THRESHOLD H = B/2 balances 2^H against 2^(B-H);
  ↪ with skewed workloads (many more queries
   than inserts) move it - all four loops stay
  ↪ correct for any H.
 MEMORY is 3 * 2^B ints: B = 20 is 12 MB, B = 22 is
  ↪ 50 MB. Above that, drop 'own' (it equals the
   light part of a separate map) or switch to a hash
  ↪ map for the sparse arrays.
 SUPERMASK COUNTING ("how many stored a with a & s ==
  ↪ s") is the same code with the roles of sup
   and sub swapped, or just complement everything: a
  ↪ superset of s is a submask of ~s complemented.
 EXACTLY-EQUAL, OR "how many a with a & s == 0"
  ↪ (disjoint) = count(~s) - both fall out for free.
 STATIC ALTERNATIVE: if all inserts precede all
  ↪ queries, run SOS DP once and answer in O(1).
  ↪ Check
   that first; this structure is only for genuinely
  ↪ interleaved updates and queries. */
```

8.3 Knapsack

8.3.1 BoundedKnapsackBitset

BOUNDED KNAPSACK, FEASIBILITY ONLY, with a bitset: O(n W / 64). Binary-group the counts

```
// (1,2,4,...) so "cnt copies of w" becomes O(log
  ↪ cnt) shift-ors. Use when you only need which sums
// are reachable, not the best value.
const int MAX_W = 100005;
bitset<MAX_W> dp;

void multiset_bitset_dp() {
    // Example: We have 13 items, each with weight 5
    int count = 13;
    int weight = 5;

    vector<int> bundled_weights;

    // Binary grouping logic
    int current_power = 1;
    while (count >= current_power) {
        bundled_weights.push_back(current_power *
  ↪ weight);
        count -= current_power;
        current_power *= 2;
    }
    // Add whatever is left over
    if (count > 0) {
        bundled_weights.push_back(count * weight);
    }

    // Now run the ultra-fast Bitset DP on the bundles
    dp[0] = 1;
    for (int bundle_w : bundled_weights) {
        dp |= (dp << bundle_w);
    }
}
```

8.3.2 KnapsackFamily

KNAPSACK FAMILY. W = capacity. All O(n*W) unless noted.

```
// 0/1: iterate capacity DOWNWARD so each item is
  ↪ used once.
ll knap01(const vector<int>& w, const vector<ll>& v,
  ↪ int W) {
    vector<ll> dp(W + 1, 0);
    for (size_t i = 0; i < w.size(); i++)
        for (int c = W; c >= w[i]; c--) dp[c] =
  ↪ max(dp[c], dp[c - w[i]] + v[i]);
    return dp[W];
}
// UNBOUNDED: iterate capacity UPWARD.
ll knapInf(const vector<int>& w, const vector<ll>& v,
  ↪ int W) {
    vector<ll> dp(W + 1, 0);
    for (size_t i = 0; i < w.size(); i++)
        for (int c = w[i]; c <= W; c++) dp[c] =
  ↪ max(dp[c], dp[c - w[i]] + v[i]);
    return dp[W];
}
// BOUNDED (item i available cnt[i] times) via BINARY
  ↪ GROUPING: split cnt into 1,2,4,...,rest
// and run 0/1 on the groups. O(W * sum log cnt).
ll knapBounded(const vector<int>& w, const
  ↪ vector<ll>& v, const vector<int>& cnt, int W) {
    vector<int> W2; vector<ll> V2;
    for (size_t i = 0; i < w.size(); i++) {
        int c = cnt[i];
        for (int k = 1; c > 0; k <<= 1) {
            int t = min(k, c); c -= t;
            W2.push_back(w[i] * t), V2.push_back(v[i]
  ↪ * t);
        }
    }
    return knap01(W2, V2, W);
}
// FEASIBILITY ONLY (which sums are reachable) ->
  ↪ bitset, O(n*W/64). Massively faster.
bitset<100001> reachableSums(const vector<int>& w) {
    bitset<100001> dp;
    dp[0] = 1;
    for (int x : w) dp |= dp << x;
    return dp;
}
// COUNTING ways (mod p): same loops, dp[c] += dp[c -
  ↪ w[i]].
// Order matters: item-outer/capacity-inner counts
  ↪ COMBINATIONS,
//                capacity-outer/item-inner counts
  ↪ PERMUTATIONS (ordered sequences).
```

8.3.3 BoundedKnapsackDeque

BOUNDED KNAPSACK IN O(n * W) - item i may be used at most cnt[i] times. The binary-grouping

```
// version in 02 - KnapsackFamily.cpp is O(W * sum
  ↪ log cnt), which is the one to write by default;
// reach for this when sum log cnt is too big (huge
  ↪ counts, or W already at the limit).
// THE REWRITE: fix item i and split the capacities
  ↪ by RESIDUE mod w[i]. Along one residue class,
//     dp'[t] = max over k in [0, cnt] of dp[t - k] +
  ↪ k * val
//            = ( max over j in [t - cnt, t] of dp[j]
  ↪ - j * val ) + t * val
// which is a SLIDING WINDOW MAXIMUM of width cnt + 1
  ↪ - a monotone deque, O(1) amortised per cell.
// PRECONDITION: none beyond w[i] >= 1; this is an
  ↪ exact rewrite, not an approximation. (The
// subtracted term j * val is what makes the window
  ↪ comparable - do not forget to add it back.)
ll boundedKnapsackDeque(const vector<int>& w, const
  ↪ vector<ll>& val,
                        const vector<int>& cnt, int W)
  ↪ {
    int n = w.size();
    vector<ll> dp(W + 1, 0), nd(W + 1, 0);
    for (int i = 0; i < n; i++) {
        int wi = w[i], m = cnt[i];
        ll v = val[i];
        if (wi <= 0 || m <= 0) continue;
  ↪           // weight 0 items: add m * v to all
        if (m > W / wi) m = W / wi;
  ↪           // more copies than fit
        for (int s = 0; s < wi && s <= W; s++) {
            deque<pair<int, ll>> dq;
  ↪           // {t, dp[s + t*wi] - t*v}
            for (int t = 0; s + (ll)t * wi <= W; t++)
  ↪ {
                ll cur = dp[s + t * wi] - (ll)t * v;
                while (!dq.empty() &&
  ↪ dq.back().second <= cur) dq.pop_back();
                dq.push_back({t, cur});
                while (dq.front().first < t - m)
  ↪ dq.pop_front();
                nd[s + t * wi] = dq.front().second +
  ↪ (ll)t * v;
            }
        }
        dp.swap(nd);
    }
    return dp[W];
  ↪           // dp[c] = best with weight <= c
}
/* THE FAMILY, so you pick the right one under time
  ↪ pressure
 cnt[i] = 1                        -> 0/1 knapsack, O(n
  ↪ W)                 (02 - KnapsackFamily)
 cnt[i] = infinity                 -> unbounded, O(n
  ↪ W), forward loop      (02 - KnapsackFamily)
 general cnt[i], small sum log     -> binary grouping,
  ↪ O(W sum log cnt)    (02 - KnapsackFamily)
 general cnt[i], W large           -> THIS FILE, O(n W)
 FEASIBILITY only (no values)      -> bitset, O(n W /
  ↪ 64)                  (02 - KnapsackFamily)
 feasibility, many equal weights -> O(S sqrt S) by
  ↪ distinct value        (04 - SubsetSumSqrt)
 COUNTING the number of ways with bounded
  ↪ multiplicity: the deque does NOT apply (sums, not
   maxima) - use prefix sums along the same residue
  ↪ classes instead, dp'[t] = P[t] - P[t-cnt-1],
   which is the same O(n W) with the same residue
  ↪ decomposition.
 RECONSTRUCTION: store which t the deque front came
  ↪ from, or re-run the DP layer by layer keeping
   all n rows (O(n W) memory) and walk back.
 IF THE VALUES ARE THE WEIGHTS (subset sum), the
  ↪ reachability bitset beats everything by a factor
   of 64 - check that first, it is one line. */
```

8.3.4 SubsetSumSqrt

SUBSET SUM IN O(S sqrt S) WITH RECONSTRUCTION, where S is the sum of all values (or the target,

```
// whichever is smaller). The bitset DP is O(n S /
  ↪ 64) and usually wins, but it does NOT
// reconstruct the subset and it degrades when n is
  ↪ large; this one is O(S sqrt S) REGARDLESS of
// n and hands back the item indices.
// THE COUNTING ARGUMENT that gives sqrt: only
  ↪ DISTINCT values matter, and if the values sum to
  ↪ S
// there are at most O(sqrt S) distinct ones. For a
  ↪ fixed value v with multiplicity c, "reachable
// with at most c copies of v" is computed in ONE
  ↪ O(S) pass: walk each residue class mod v and
// carry a counter of how many copies are still
  ↪ available - a reachable cell refills the counter
// to c, an unreachable cell consumes one copy if any
  ↪ remain. That is the bounded-knapsack
// feasibility trick without a deque.
// PRECONDITION: all values > 0 (a zero value makes
  ↪ the residue walk spin).
// RETURNS the indices of a subset summing to exactly
  ↪ 'target', or an empty vector if impossible.
vector<int> subsetSumSqrt(const vector<int>& a, int
  ↪ target) {
    if (target < 0) return {};
    vector<char> dp(target + 1, 0);
    vector<int> last(target + 1, -1), cnt(target + 1,
  ↪ 0);
    vector<vector<int>> id(target + 1);
    for (int i = 0; i < (int)a.size(); i++)
        if (a[i] > 0 && a[i] <= target)
  ↪ id[a[i]].push_back(i), cnt[a[i]]++;
    dp[0] = 1;
    for (int v = 1; v <= target; v++) {
        if (!cnt[v]) continue;
        for (int r = 0; r < v; r++) {
            int left = 0;
  ↪           // copies of v still usable
            for (int k = r; k <= target; k += v) {
                if (dp[k]) left = cnt[v];
  ↪           // reachable without any copy here
                else if (left) dp[k] = 1, left--,
  ↪ last[k] = id[v][left];
            }
        }
    }
    if (!dp[target]) return {};
    vector<int> res;
    for (int s = target; s > 0; s -= a[last[s]])
  ↪ res.push_back(last[s]);
    return res;
}
/* WHEN TO USE WHAT
 n small, S small            -> plain O(n S) DP,
  ↪ easiest to modify.
 feasibility only            -> bitset  dp |= dp <<
```

```
  ↪ a[i],  O(n S / 64). Fastest by far.
 need the SUBSET back       -> THIS FILE, or the
  ↪ bitset plus O(n) "peel one item at a time" with
                              a per-item bitset
  ↪ snapshot (O(n S / 64) memory - usually too much).
 many equal values          -> THIS FILE is optimal:
  ↪ it is O(S) per DISTINCT value.
 counting the number of subsets, not feasibility ->
  ↪ the residue walk does not apply; use prefix
                              sums per residue class
  ↪ (bounded) or plain O(n S) (unbounded).
 CLASSIC USES: "split the array into two halves with
  ↪ a given sum", partitioning tree levels
   (CF 1481F), and any "choose a subset of the
  ↪ multiset of subtree sizes" argument - those all
   have sum of values = n, hence O(n sqrt n).
 THE SAME sqrt ARGUMENT gives the O(n sqrt n / 64)
  ↪ bound for "subset sums of an array whose
   values sum to n" with bitsets - group equal
  ↪ values, binary-group each group, and the total
   number of groups is O(sqrt n log n). */
```

8.4 Classics

8.4.1 DigitDP

DIGIT DP - count numbers in [L, R] with a digit property. f(R) - f(L-1).

```
// State: (position, tight, started, problem state).
  ↪ ‘started‘ handles leading zeros.
// THIS CODE IS BASE 10 (to_string, digit cap 9,
  ↪ memo[20] positions). For another base, write the
// digits yourself, change the 9, and size memo by
  ↪ the digit count (base 2 needs 64 positions).
// L - 1 IS NOT ALWAYS AVAILABLE: if the bounds are
  ↪ given as huge decimal STRINGS you cannot
// subtract one, so carry two tight flags (tightLo,
  ↪ tightHi) and count [L, R] in a single pass.
string DIG;
ll memo[20][2][2][200];
bool vis[20][2][2][200];

ll go(int p, int tight, int started, int st) {
    if (p == (int)DIG.size()) return started ? /*
  ↪ ACCEPT? */ (st == 0) : 0;   // <- edit
    if (vis[p][tight][started][st]) return
  ↪ memo[p][tight][started][st];
    vis[p][tight][started][st] = true;
    ll res = 0;
    int hi = tight ? DIG[p] - '0' : 9;
    for (int d = 0; d <= hi; d++) {
        int nStarted = started || d > 0;
        int nSt = nStarted ? (st + d) % 10 : 0;
  ↪                          // <- edit
        res += go(p + 1, tight && d == hi, nStarted,
  ↪ nSt);
    }
    return memo[p][tight][started][st] = res;
}
bool qualifiesZero = 1;                          //
  ↪ does the number 0 satisfy the predicate?
ll countUpTo(ll x) {
    if (x < 0) return 0;
    DIG = to_string(x);
    memset(vis, 0, sizeof vis);
    return go(0, 1, 0, 0) + qualifiesZero;       //
  ↪ set qualifiesZero = 1 only if 0 itself counts
}
ll countRange(ll L, ll R) { return countUpTo(R) -
  ↪ countUpTo(L - 1); }
// VARIANTS: track (sum of digits), (value mod k),
  ↪ (last digit), (a bitmask of used digits),
// (whether we already saw "13"), (count so far) -
  ↪ anything with a small state.
// For "k-th number with the property", binary search
  ↪ on countUpTo.
```

8.4.2 LIS

LIS in O(n log n), with reconstruction. STRICT increasing by default.

```
// Non-decreasing: change lower_bound -> upper_bound.
// Longest DEcreasing: negate the array. Longest
  ↪ non-increasing: negate + upper_bound.
vector<int> lis(const vector<ll>& a) {
    int n = a.size();
    vector<ll> tail;                             //
  ↪ tail[k] = smallest possible tail of an LIS of
  ↪ length k+1
    vector<int> pos(n), prv(n, -1), idx;
    for (int i = 0; i < n; i++) {
        int k = lower_bound(all(tail), a[i]) -
  ↪ tail.begin();
        if (k == (int)tail.size())
  ↪ tail.push_back(a[i]), idx.push_back(i);
        else tail[k] = a[i], idx[k] = i;
        pos[i] = k;
        prv[i] = k ? idx[k - 1] : -1;
    }
    vector<int> res;
    for (int i = idx.empty() ? -1 : idx.back(); i >=
  ↪ 0; i = prv[i]) res.push_back(i);
    reverse(all(res));
    return res;                                  //
  ↪ indices of one LIS; length = tail.size()
}
// SAME SHAPE SOLVES: longest chain of pairs (sort by
  ↪ x, LIS on y); maximum non-overlapping
// intervals; "minimum number of non-increasing
  ↪ subsequences covering the array" = LIS length
// (Dilworth). For LIS with an extra weight, replace
  ↪ ‘tail‘ with a max-BIT over compressed values.
```

8.4.3 IntervalDP

INTERVAL DP - dp over [l, r], processed by increasing length. O(n^3) (O(n^2) with Knuth).

```
// This is the base pattern that Knuth's optimisation
  ↪ speeds up; write this first, optimise later.

// 1) SPLIT form: dp[l][r] = min over k of dp[l][k] +
  ↪ dp[k+1][r] + cost(l, r)
//    matrix chain multiplication, merging stones,
  ↪ optimal BST, "cost to combine a range"
template <class Cost> ll intervalSplit(int n, Cost
  ↪ cost) {
    vector<vector<ll>> dp(n, vector<ll>(n, 0));
    for (int len = 2; len <= n; len++)
        for (int l = 0, r = len - 1; r < n; l++, r++)
  ↪ {
            dp[l][r] = llinf;
            for (int k = l; k < r; k++) dp[l][r] =
  ↪ min(dp[l][r], dp[l][k] + dp[k + 1][r]);
            dp[l][r] += cost(l, r);
        }
    return dp[0][n - 1];
}
```

```
// 2) ENDPOINT form: decide about a[l] or a[r] and shrink. Palindromes, "remove from either end".
// Minimum insertions to make s a palindrome:
int minInsertPalindrome(const string& s) {
    int n = s.size();
    vector<vector<int>> dp(n, vector<int>(n, 0));
    for (int len = 2; len <= n; len++)
        for (int l = 0, r = len - 1; r < n; l++, r++)
            dp[l][r] = s[l] == s[r] ? dp[l + 1][r - 1] : 1 + min(dp[l + 1][r], dp[l][r - 1]);
    return n ? dp[0][n - 1] : 0;
}

// 3) "BURST BALLOONS" form: choose the LAST element removed in the range, so the neighbours are
//    the (still intact) boundaries l-1 and r+1.
ll burst(vector<ll> a) {
    int n = a.size();
    a.insert(a.begin(), 1), a.push_back(1);
    vector<vector<ll>> dp(n + 2, vector<ll>(n + 2, 0));
    for (int len = 1; len <= n; len++)
        for (int l = 1, r = len; r <= n; l++, r++)
            for (int k = l; k <= r; k++)
                dp[l][r] = max(dp[l][r], dp[l][k - 1] + a[l - 1] * a[k] * a[r + 1] + dp[k + 1][r]);
    return dp[1][n];
}
// WHEN KNUTH APPLIES (form 1): cost is monotone on intervals and satisfies the quadrangle
// inequality; then opt[l][r-1] <= opt[l][r] <= opt[l+1][r] and the k-loop becomes amortised O(1).
// COUNTING variants (e.g. #ways to parenthesise) replace min with + and cost with 1.
```

8.4.4 BrokenProfileDP

BROKEN PROFILE DP - sweep the grid cell by cell carrying a bitmask of the "broken" frontier.

```
// O(n * m * 2^m); always make m the SMALLER dimension.
// mask bit j = "cell (i, j) is already covered by something placed earlier".

// Count tilings of an n x m grid by 1x2 dominoes.
ll dominoTilings(int n, int m) {
    if (m > n) swap(n, m);
    if (((ll)n * m) & 1) return 0;
    int M = 1 << m;
    vector<ll> cur(M, 0), nxt;
    cur[0] = 1;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++) {
            nxt.assign(M, 0);
            for (int mask = 0; mask < M; mask++) if (cur[mask]) {
                if (mask >> j & 1) nxt[mask ^ (1 << j)] += cur[mask];           // already covered
                else {
                    if (i + 1 < n) nxt[mask | (1 << j)] += cur[mask];           // vertical domino
                    if (j + 1 < m && !(mask >> (j + 1) & 1))
                        nxt[mask | (1 << (j + 1))] += cur[mask];                // horizontal domino
                }
            }
            cur = nxt;
        }
    return cur[0];
}
// BLOCKED CELLS: keep a grid `bad[i][j]`; a blocked cell must arrive already "covered"
// (skip it by clearing bit j and forbidding any placement on it).
// OTHER PIECES: add one transition per shape; the mask must record every cell the shape
// occupies in a LATER position of the sweep.
// COUNTING INDEPENDENT SETS / no-two-adjacent colourings: mask = which cells of the previous
// row are used; transition over masks with (a & b) == 0 and no two adjacent bits in b.
// AS WRITTEN this is O(n * 4^m) (nested mask loops), not O(n * m * 2^m): at m = 14 that is 2.7e8
// per row. To get O(n * 2^m), note sum over a with (a & b) == 0 of cur[a] = zetaSub(cur)[~b],
// so one subset-zeta pass per row replaces the inner loop (see 05 - Combinatorics/07).
ll independentSets(int n, int m) {
    // #ways to pick cells, none orthogonally adjacent
    if (m > n) swap(n, m);
    int M = 1 << m;
    vector<ll> cur(M, 0), nxt;
    for (int b = 0; b < M; b++) if (!(b & (b << 1))) cur[b] = 1;
    for (int i = 1; i < n; i++) {
        nxt.assign(M, 0);
        for (int a = 0; a < M; a++) if (cur[a])
            for (int b = 0; b < M; b++) if (!(b & (b << 1)) && !(a & b)) nxt[b] += cur[a];
        cur = nxt;
    }
    ll r = 0;
    for (ll x : cur) r += x;
    return r;
}
// CONNECTED-COMPONENT DP ("plug DP") is the heavy version: the state stores, for each frontier
// cell, WHICH connected component its plug belongs to (a bracket-like labelling). Use it for
// "count Hamiltonian cycles / single-closed-loop tilings"; it is long - only write it if the
// problem clearly needs connectivity, not just coverage.
```

8.4.5 MaximalRectangle

LARGEST ALL-ZERO (or all-one) RECTANGLE in a binary grid, O(n*m).

```
// Row by row, keep the histogram of consecutive zeros ending at this row, then run the
// largest-rectangle-in-histogram monotonic stack.
ll maximalRectangle(const vector<vector<int>>& g, int target = 0) {
    int n = g.size(), m = n ? g[0].size() : 0;
    vector<int> h(m, 0);
    ll best = 0;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) h[j] = (g[i][j] == target) ? h[j] + 1 : 0;
        vector<int> st;
        // indices, increasing height
        for (int j = 0; j <= m; j++) {
            int cur = (j == m) ? -1 : h[j];
            while (!st.empty() && h[st.back()] >= cur) {
```

```
                int ht = h[st.back()]; st.pop_back();
                int left = st.empty() ? -1 :
    ↪ st.back();
                best = max(best, (ll)ht * (j - left -
    ↪ 1));
            }
            st.push_back(j);
        }
        st.clear();
    }
    return best;
}
// LARGEST SQUARE instead: dp[i][j] = min(dp[i-1][j],
    ↪ dp[i][j-1], dp[i-1][j-1]) + 1 when free.
// COUNTING all all-zero submatrices: for each row,
    ↪ for each column j keep ‘run[j]‘ = the number
// of rectangles ending at (i, j); with a monotonic
    ↪ stack, run[j] = run[prevSmaller] + h[j] *
// (j - prevSmaller), and the total is the sum of
    ↪ run[j] over all cells. O(n*m).
ll countAllZeroSubmatrices(const vector<vector<int>>&
    ↪ g) {
    int n = g.size(), m = n ? g[0].size() : 0;
    vector<ll> h(m, 0), run(m, 0);
    ll total = 0;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) h[j] = g[i][j] ==
    ↪ 0 ? h[j] + 1 : 0;
        vector<int> st;
        for (int j = 0; j < m; j++) {
            while (!st.empty() && h[st.back()] >=
    ↪ h[j]) st.pop_back();
            int p = st.empty() ? -1 : st.back();
            run[j] = (p < 0 ? 0 : run[p]) + h[j] * (j
    ↪ - p);
            total += run[j];
            st.push_back(j);
        }
    }
    return total;
}
```

8.4.6 Hirschberg

HIRSCHBERG - reconstruct the ANSWER of a two-sequence DP (LCS, alignment, edit script) in

```
// O(n m) time but only O(min(n, m)) memory instead
    ↪ of the O(n m) table. WHEN: n = m = 1e5 and you
// need the actual subsequence, not its length - the
    ↪ table would be 1e10 cells.
// THE IDEA (divide & conquer on the ROW, not on the
    ↪ value): the last row of the DP for the top
// half of a, and the last row of the REVERSED DP for
    ↪ the bottom half, meet at the middle row.
// The column k maximising L[k] + R[m - k] is a point
    ↪ the optimal path passes through, so the
// problem splits into (top half of a vs b[0..k)) and
    ↪ (bottom half of a vs b[k..m)).
// COST: T(n, m) = O(n m) + T(n/2, k) + T(n/2, m - k)
    ↪ = O(n m) total - the halves of the second
// level together cost half of the first level, so
    ↪ the series is 2 n m.
// PRECONDITION: the DP must be reversible - dp
    ↪ computed left-to-right on the reversed strings
// must equal the suffix DP. True for LCS, edit
    ↪ distance and affine-gap alignment; NOT true for
// DPs whose transition depends on the direction
    ↪ (e.g. "no two chosen positions adjacent" is fine,
// but anything reading a prefix STATE such as a
    ↪ running remainder is not).
vector<int> lcsLastRow(const string& a, const string&
    ↪ b) {      // O(|b|) memory
    int m = b.size();
    vector<int> prv(m + 1, 0), cur(m + 1, 0);
    for (char ca : a) {
        for (int j = 1; j <= m; j++)
            cur[j] = ca == b[j - 1] ? prv[j - 1] + 1
    ↪ : max(prv[j], cur[j - 1]);
        swap(prv, cur);
    }
    return prv;
}
string hirschbergLCS(const string& a, const string&
    ↪ b) {
    int n = a.size(), m = b.size();
    if (n == 0 || m == 0) return "";
    if (n == 1) return b.find(a[0]) != string::npos ?
    ↪ string(1, a[0]) : "";
    int i = n / 2;
    string a1 = a.substr(0, i), a2 = a.substr(i);
    string ra(a2.rbegin(), a2.rend()), rb(b.rbegin(),
    ↪ b.rend());
    vector<int> L = lcsLastRow(a1, b), R =
    ↪ lcsLastRow(ra, rb);
    int best = -1, k = 0;
    for (int j = 0; j <= m; j++) if (L[j] + R[m - j]
    ↪ > best) best = L[j] + R[m - j], k = j;
    return hirschbergLCS(a1, b.substr(0, k)) +
    ↪ hirschbergLCS(a2, b.substr(k));
}
/* ADAPTING IT
 EDIT DISTANCE / ALIGNMENT: replace lcsLastRow with
    ↪ the edit-distance row (initialised to
    0,1,2,... instead of zeros) and take the MINIMUM
    ↪ of L[j] + R[m-j] instead of the maximum. The
    split column is then a cell the optimal alignment
    ↪ passes through; emit the operations.
 AFFINE GAP COSTS need three rows (match / gap-in-a /
    ↪ gap-in-b) and the split must record WHICH
    of the three states the path is in at the middle
    ↪ row - otherwise the two halves disagree.
 MEMORY: O(m) for the rows plus O(m log n) for the
    ↪ substrings taken at each recursion level; pass
    index ranges instead of substrings if that matters.
 RECURSION DEPTH is O(log n) - safe.
 IF YOU ONLY NEED THE LENGTH, do not use this: one
    ↪ rolling row is enough (and for LCS, the
    bit-parallel version in 07 - Strings/07 -
    ↪ Sequences/03 is O(n m / 64)).
 THE SAME TRICK reconstructs paths in any layered DAG
    ↪ DP where a middle layer is cheap to
    evaluate from both ends - it is the DP analogue of
    ↪ "meet in the middle". */
```

8.4.7 MaximumSubarray

MAXIMUM SUBARRAY (Kadane) and its dimensional relatives. All exact, all one pass per dimension.

```
// KADANE: the best subarray ending at i either
    ↪ extends the best one ending at i-1 or restarts.
// O(n). Returns {sum, l, r} with the segment
    ↪ NON-EMPTY (if every element is negative the
    ↪ answer
// is the largest single element - the empty-segment
    ↪ convention gives 0 instead, so decide which
// one the problem wants BEFORE writing the loop;
    ↪ that is the only bug this ever has).
array<ll, 3> maxSubarray(const vector<ll>& a) {
```

```
    int n = a.size();
    ll best = LLONG_MIN, cur = 0;
    int bl = 0, br = 0, start = 0;
    for (int i = 0; i < n; i++) {
        if (cur <= 0) cur = a[i], start = i;
  ↪          // restart: the prefix only hurts
        else cur += a[i];
        if (cur > best) best = cur, bl = start, br =
  ↪ i;
    }
    return {best, bl, br};
}
// CIRCULAR (the array wraps): the answer is either
  ↪ an ordinary subarray, or the complement of the
// MINIMUM subarray. Guard the all-negative case -
  ↪ "total - min" would return the empty segment.
ll maxCircularSubarray(const vector<ll>& a) {
    ll total = 0, mx = LLONG_MIN, mn = LLONG_MAX, cx
  ↪ = 0, cn = 0;
    for (ll x : a) {
        total += x;
        cx = max(x, cx + x), mx = max(mx, cx);
        cn = min(x, cn + x), mn = min(mn, cn);
    }
    return mx < 0 ? mx : max(mx, total - mn);
}
// MAXIMUM SUM SUBMATRIX, O(rows^2 * cols): fix the
  ↪ top and bottom rows, compress the columns
// between them into a 1D array by prefix sums, and
  ↪ run Kadane on it. Make ROWS the smaller side.
ll maxSubmatrix(const vector<vector<ll>>& g) {
    int n = g.size(), m = n ? g[0].size() : 0;
    ll best = LLONG_MIN;
    for (int top = 0; top < n; top++) {
        vector<ll> col(m, 0);
        for (int bot = top; bot < n; bot++) {
            for (int j = 0; j < m; j++) col[j] +=
  ↪ g[bot][j];
            best = max(best, maxSubarray(col)[0]);
        }
    }
    return best;
}
/* THE VARIANTS, and which structure each one needs
 LENGTH AT LEAST k: prefix sums S, answer = max over
  ↪ i >= k of S[i] - min(S[0..i-k]) - one running
   minimum, O(n).
 LENGTH AT MOST k: S[i] - min over a SLIDING WINDOW
  ↪ of S, so a monotone deque, O(n).
 AT MOST k NEGATIVE ELEMENTS / at most k "skips":
  ↪ dp[i][j] = best ending at i having used j skips.
 MAXIMUM AVERAGE subarray (with a length constraint):
  ↪ binary search the average and run the
   "length at least k" version on a[i] - mid, or the
  ↪ tangent trick in 01 - DP Opt/09.
 MAXIMUM PRODUCT subarray: carry BOTH the running max
  ↪ and the running min (a negative times a
   negative flips them), O(n).
 K-TH LARGEST subarray sum: binary search the value
  ↪ plus a two-pointer/BIT count of pairs.
 MAXIMUM SUBARRAY WITH UPDATES / on a range: segment
  ↪ tree storing (total, best prefix, best
   suffix, best inside) - see 02 - Data Structures;
  ↪ this file is the static case.
 ALL-NEGATIVE ARRAYS are the standard trap in every
  ↪ one of these. */
```

8.4.8 ConnectedComponentDP

CONNECTED-COMPONENT DP (a.k.a. the "broken interval" or profile-free connectivity DP).

```
// SHAPE OF THE PROBLEM: you must build one object
  ↪ out of n items processed in SORTED order, the
// cost depends on how far apart consecutive items
  ↪ end up, and the final object must be CONNECTED
// (one block, one path, one permutation with a
  ↪ cyclic condition). The state is NOT "which items
// are used" (2^n) but "how many disjoint fragments
  ↪ exist so far", because the fragments are
// interchangeable - only their COUNT matters, and
  ↪ every fragment can be glued later.
// STATE: dp[i][c][cost][ends] = first i items
  ↪ placed, c open fragments, accumulated cost, and
// `ends` = how many of the two outer boundaries have
  ↪ been fixed (0, 1 or 2) when the object is a
// PATH rather than a cycle. Drop `ends` for objects
  ↪ with no distinguished endpoints.
// THE COST TRICK, which is the whole point: when
  ↪ moving from item i to item i+1, EVERY fragment
// boundary that is still open must eventually cross
  ↪ the gap (a[i+1] - a[i]). There are
// (2c - ends) such open boundaries, so add (a[i+1] -
  ↪ a[i]) * (2c - ends) to the cost NOW, once,
// instead of trying to know later which items are
  ↪ adjacent. That is what removes the 2^n.
// COMPLEXITY: O(n^2 * K) states with O(1)
  ↪ transitions, K = cost budget. The classic
  ↪ instance is
// JOI Skyscraper: count permutations of a[] with sum
  ↪ |a[i+1] - a[i]| <= K.
// PRECONDITION: sort a[] first - the cost accounting
  ↪ above is only valid for sorted items.
ll skyscraper(vector<ll> a, int K) {
    int n = a.size();
    sort(a.begin(), a.end());
    if (n == 1) return 1;
    // dp[c][cost][ends]
    vector<vector<array<ll, 3>>> dp(n + 2,
  ↪ vector<array<ll, 3>>(K + 1, {0, 0, 0}));
    dp[0][0][0] = 1;
    for (int i = 0; i < n; i++) {
        vector<vector<array<ll, 3>>> nd(n + 2,
  ↪ vector<array<ll, 3>>(K + 1, {0, 0, 0}));
        ll gap = i ? a[i] - a[i - 1] : 0;
        for (int c = 0; c <= i; c++)
            for (int cost = 0; cost <= K; cost++)
                for (int e = 0; e < 3; e++) {
                    ll v = dp[c][cost][e];
                    if (!v) continue;
                    ll nc = cost + gap * (2 * c - e);
  ↪                 // pay for every open boundary
                    if (nc > K) continue;
                    auto add = [&](int c2, int e2, ll
  ↪ ways) {
                        // after placing an item at
  ↪ least one fragment MUST exist: without this
                        // guard the state (c = 0,
  ↪ ends = 1) survives and inflates the count
                        if (c2 < 1 || c2 > n || ways
  ↪ <= 0) return;
                        nd[c2][nc][e2] =
  ↪ (nd[c2][nc][e2] + v % MOD * (ways % MOD)) % MOD;
                    };
                    add(c + 1, e, c + 1 - e);
  ↪                 // new fragment, interior
                    add(c + 1, e + 1, e < 2 ? 2 - e :
```

```
    ↪ 0);       // new fragment, at an end
                    add(c, e, 2 * c - e);
    ↪           // append to one fragment's side
                    add(c, e + 1, e < 2 ? 2 - e : 0);
    ↪           // append and make it an end
                    if (c >= 2) add(c - 1, e, c - 1);
    ↪           // merge two fragments
                }
            dp = nd;
        }
        ll res = 0;
        for (int cost = 0; cost <= K; cost++) res = (res
    ↪ + dp[1][cost][2]) % MOD;
        return res;
    ↪          // one fragment, both ends fixed
}
/* READING THE TRANSITIONS (they are always these
    ↪ five, in every problem of this family)
 NEW FRAGMENT: the item starts a block of its own.
    ↪ Interior blocks can be created in
    (c + 1 - ends) ways (the gaps between existing
    ↪ blocks); if it is to become an outer end of the
    final object, that is one of the (2 - ends)
    ↪ remaining end slots.
 EXTEND: glue the item to one of the (2c - ends) open
    ↪ sides of an existing block, or spend it as
    an outer end of that block (again 2 - ends ways).
 MERGE: join two adjacent blocks with this item, (c -
    ↪ 1) ordered choices of which pair.
 ACCEPT at the end only c == 1 (connected!) and ends
    ↪ == 2 (both boundaries closed).
 OTHER PROBLEMS IN THIS FAMILY: "arrange n numbers in
    ↪ a row minimising the sum of adjacent
    differences with constraints", counting linear
    ↪ extensions with gap costs, "cut a rod into
    pieces so that ..." and any permutation problem
    ↪ whose cost telescopes over sorted values.
 IF THE OBJECT IS A CYCLE, delete the `ends`
    ↪ dimension entirely and accept c == 1.
 WATCH THE OVERCOUNTING: `add(c, e + 1, 2 - e)` and
    ↪ `add(c + 1, e + 1, 2 - e)` both consume an end
    slot - forgetting one of them, or letting e exceed
    ↪ 2, silently changes the answer. */
```

8.4.9 PlugDP

PLUG DP WITH CONNECTIVITY (the bracket / minimum-representation profile DP). This is the

```
// heavyweight cousin of 04 - BrokenProfileDP.cpp:
    ↪ broken profile only remembers WHICH frontier
// cells are occupied, which is enough for tilings;
    ↪ here the frontier must also remember WHICH
// PARTIAL PATHS ARE CONNECTED TO WHICH, so that
    ↪ closing a loop early can be forbidden.
// ENCODING: the frontier has m + 1 "plugs". Each
    ↪ plug is 0 (nothing), 1 (a LEFT bracket) or
// 2 (a RIGHT bracket), packed two bits per plug.
    ↪ Because the paths live in a planar grid they
// can never cross, so the connection pattern is a
    ↪ balanced bracket sequence - matching brackets
// = "these two plug ends belong to the same partial
    ↪ path". THAT is why 3 states per plug suffice
// instead of a general set partition of the frontier.
// COMPLEXITY: O(n * m * S) where S = number of
    ↪ reachable states, empirically a few thousand for
// m <= 12 (the Motzkin-like count, far below
    ↪ 3^(m+1)); keep the states in a hash map.
// INDEX CONVENTION (get this wrong and nothing
    ↪ works): cells are 0-based and scanned row by row.
// BEFORE processing cell (i, j) the state holds, in
    ↪ plug index order 0..m:
//   indices < j  : the BOTTOM edges of the cells (i,
    ↪ 0..j-1) already placed in this row
//   index    j   : the LEFT edge of (i, j)
    ↪ <- p below
//   index    j+1 : the TOP edge of (i, j)
    ↪ <- q below
//   indices > j+1: the TOP edges of (i, j+1..m-1),
    ↪ inherited from the previous row
// AFTER processing, plug j becomes the BOTTOM edge
    ↪ of (i, j) and plug j+1 its RIGHT edge, which
// is exactly the invariant for cell (i, j+1). At the
    ↪ end of a row plug m must be 0 (no path may
// leave the grid) and the state is SHIFTED LEFT by
    ↪ one plug. Plug k uses bits 2k and 2k+1.
// THE PROBLEM SOLVED HERE: count HAMILTONIAN CYCLES
    ↪ in a grid with blocked cells (URAL 1519
// "Formula 1"). Every other plug-DP problem is this
    ↪ loop with different accept/reject rules.
struct PlugDP {
    int n, m;
    vector<string> g;
    ↪          // '.' free, '*' blocked
    PlugDP(const vector<string>& grid) : g(grid) { n
    ↪ = g.size(), m = n ? g[0].size() : 0; }
    static int get(ll s, int i) { return (s >> (2 *
    ↪ i)) & 3; }
    static ll set(ll s, int i, int v) { return (s &
    ↪ ~(3LL << (2 * i))) | ((ll)v << (2 * i)); }
    int matchRight(ll s, int i) {
    ↪          // find the ')' matching '(' at i
        int cnt = 0;
        for (int j = i; j <= m; j++) {
            int v = get(s, j);
            if (v == 1) cnt++;
            else if (v == 2 && --cnt == 0) return j;
        }
        return -1;
    }
    int matchLeft(ll s, int i) {
    ↪          // find the '(' matching ')' at i
        int cnt = 0;
        for (int j = i; j >= 0; j--) {
            int v = get(s, j);
            if (v == 2) cnt++;
            else if (v == 1 && --cnt == 0) return j;
        }
        return -1;
    }
    ll solve() {
        int li = -1, lj = -1;
    ↪          // the LAST free cell
        for (int i = 0; i < n; i++)
            for (int j = 0; j < m; j++) if (g[i][j]
    ↪ == '.') li = i, lj = j;
        if (li < 0) return 0;
        ll ans = 0;
        map<ll, ll> cur, nxt;
        cur[0] = 1;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                nxt.clear();
                for (auto& [s, w] : cur) {
                    int p = get(s, j), q = get(s, j +
    ↪ 1);
                    if (g[i][j] == '*') {
    ↪          // blocked: no plug may touch it
```

```
                    if (!p && !q) nxt[s] += w;
                    continue;
                }
                if (!p && !q) {
    ↪           // open a new path: down and right
                    if (i + 1 < n && j + 1 < m)
    ↪ nxt[set(set(s, j, 1), j + 1, 2)] += w;
                } else if (!p || !q) {
    ↪           // one plug: continue down or right
                    int v = p ? p : q;
                    if (i + 1 < n) nxt[set(set(s,
    ↪ j, v), j + 1, 0)] += w;    // go down
                    if (j + 1 < m) nxt[set(set(s,
    ↪ j, 0), j + 1, v)] += w;    // go right
                } else if (p == 1 && q == 1) {
    ↪           // two '(' : merge, retag the match
                    int k = matchRight(s, j + 1);
                    nxt[set(set(set(s, k, 1), j,
    ↪ 0), j + 1, 0)] += w;
                } else if (p == 2 && q == 2) {
    ↪           // two ')' : merge, retag the match
                    int k = matchLeft(s, j);
                    nxt[set(set(set(s, k, 2), j,
    ↪ 0), j + 1, 0)] += w;
                } else if (p == 2 && q == 1) {
    ↪           // different paths: just join them
                    nxt[set(set(s, j, 0), j + 1,
    ↪ 0)] += w;
                } else if (i == li && j == lj) {
    ↪           // p == 1 && q == 2: closes a loop
                    ll t = set(set(s, j, 0), j +
    ↪ 1, 0);    // legal ONLY at the last free cell
                    if (t == 0) ans += w;
                }
            }
            cur.swap(nxt);
        }
        nxt.clear();
    ↪       // end of row: shift the profile
        for (auto& [s, w] : cur) if (!get(s, m))
    ↪ nxt[s << 2] += w;
        cur.swap(nxt);
    }
    return ans;
  }
};
/* ADAPTING IT - the five cases above never change,
  ↪ only what you do with them
 COUNT HAMILTONIAN PATHS (not cycles): allow the
  ↪ "close" case at any cell but require exactly one
  ↪ open path at the end, or add a virtual start/end
  ↪ cell.
 SIMPLE CYCLE THAT NEED NOT COVER EVERY CELL (max
  ↪ weight cycle): let a free cell be SKIPPED (the
  ↪ `!p && !q` case also allows keeping the state
  ↪ unchanged), and accept the closure at any cell,
  ↪ taking the max instead of the sum.
 PARTITION INTO CLOSED LOOPS: accept every closure
  ↪ and multiply, no "last cell" restriction.
 THE THREE-STATE ENCODING ONLY WORKS FOR NON-CROSSING
  ↪ (planar) connections. For general
  ↪ connectivity (Steiner-tree profile DPs, spanning
  ↪ forests) you need MINIMUM REPRESENTATION:
  ↪ store, per frontier cell, the id of its component
  ↪ normalised to 0,1,2,... left to right - same
  ↪ loop, bigger state, and a canonicalisation step
  ↪ after every move.
 ALWAYS make m the SMALLER dimension (transpose if
  ↪ needed) - the state count is exponential in m.
 DEBUGGING: a 4x4 empty grid has 6 Hamiltonian
  ↪ cycles, 6x6 has 1072, 8x8 has 425;  a grid with
  ↪ an
  ↪ odd number of free cells has none. Check those
  ↪ before running anything bigger. */
```

8.4.10 DPOverDivisors

DP OVER THE DIVISOR LATTICE OF ONE NUMBER - the exact analogue of SOS DP, with "submask"

```
// replaced by "divisor". The divisors of n form a
  ↪ lattice isomorphic to a product of chains (one
// per prime, of length e_p), so the same "one
  ↪ dimension at a time" trick works:
//     for each prime p:  for divisors in increasing
  ↪ order:  g[d] += g[d / p]
// gives g[d] = sum over e | d of f[e] in O(D *
  ↪ omega(n)) instead of O(D^2) pair testing, where
// D = number of divisors (<= 1344 below 1e9) and
  ↪ omega = number of distinct primes (<= 9).
// WHEN: "for every divisor d of n, count the array
  ↪ elements whose gcd with n is exactly d" (the
// mobius direction), "sum over all divisors of a
  ↪ divisor", multiplicative-function DPs restricted
// to one n, and the CF-style "count subsets whose
  ↪ gcd is exactly d".
// INDEX CONVENTION: divs is SORTED ASCENDING and f
  ↪ is indexed by POSITION in that array; pos maps
// a divisor value to its index. Sorted order
  ↪ guarantees d / p comes before d, which is what
  ↪ makes
// the in-place update legal - do not shuffle divs.
struct DivisorLattice {
    vector<ll> divs;
    vector<ll> primes;
    map<ll, int> pos;
    DivisorLattice(ll n, const vector<ll>&
  ↪ primeFactors) : primes(primeFactors) {
        for (ll i = 1; i * i <= n; i++)
            if (n % i == 0) { divs.push_back(i); if
  ↪ (i != n / i) divs.push_back(n / i); }
        sort(divs.begin(), divs.end());
        for (int i = 0; i < (int)divs.size(); i++)
  ↪ pos[divs[i]] = i;
    }
    void zeta(vector<ll>& f) {
  ↪          // g[d] = sum over e | d of f[e]
        for (ll p : primes)
            for (int i = 0; i < (int)divs.size(); i++)
                if (divs[i] % p == 0) f[i] +=
  ↪ f[pos[divs[i] / p]];
    }
    void mobius(vector<ll>& f) {
  ↪          // exact inverse of zeta
        for (ll p : primes)
            for (int i = (int)divs.size() - 1; i >=
  ↪ 0; i--)
                if (divs[i] % p == 0) f[i] -=
  ↪ f[pos[divs[i] / p]];
    }
    void superZeta(vector<ll>& f) {
  ↪          // g[d] = sum over d | e of f[e]
        for (ll p : primes)
            for (int i = (int)divs.size() - 1; i >=
  ↪ 0; i--)
                if (divs[i] % p == 0) f[pos[divs[i] /
  ↪ p]] += f[i];
    }
```

```
    void superMobius(vector<ll>& f) {
  ↪          // exact inverse of superZeta
        for (ll p : primes)
            for (int i = 0; i < (int)divs.size(); i++)
                if (divs[i] % p == 0) f[pos[divs[i] /
  ↪ p]] -= f[i];
    }
};
/* THE CANONICAL USE
 cnt[d] = number of array elements divisible by d
  ↪ (easy: for each element, add 1 at gcd(a, n));
 then superZeta turns "exactly d" counts into
  ↪ "divisible by d" counts and superMobius turns
  ↪ them
 back. Counting subsets with gcd exactly d is then:
  ↪ ways[d] = 2^(divisible by d) - 1, followed by
 superMobius over the divisor lattice.
 OVER 1..N INSTEAD OF THE DIVISORS OF ONE n: the same
  ↪ transforms are the harmonic loops
   for (d = 1..N) for (m = 2d, 3d, ...) f[m] += f[d]
  ↪  (divisor zeta,   O(N log N))
   for (d = N..1) for (m = 2d, 3d, ...) f[d] += f[m]
  ↪  (multiple zeta,  O(N log N))
 and their mobius inverses reverse the inner
  ↪ direction. See 04 - Number Theory/04/02.
 GCD CONVOLUTION c[k] = sum over gcd(i,j) = k of a[i]
  ↪ b[j]: multipleZeta both, multiply pointwise,
 multipleMobius the result. LCM convolution is the
  ↪ same with the divisor transforms.
 WHY NOT JUST TEST ALL PAIRS: D^2 is 1.8e6 for D =
  ↪ 1344, fine once - but inside a loop over
 queries or over n it is not, and the lattice version
  ↪ is 12000 operations.
 THE PRIME LIST must be the DISTINCT primes of n
  ↪ (factorise with 04 - NT/02/06 Pollard). Passing a
 prime twice double-counts and silently corrupts
  ↪ every value. */
```

8.4.11 XorEquationCount

COUNTING SOLUTIONS OF x_1 xor x_2 xor ... xor x_n = X with 0 <= x_i <= a_i, in O(n * B).

```
// The bounds are DIFFERENT per variable, which is
  ↪ what makes this hard: with a common bound
// 2^B - 1 the answer is just 2^(B(n-1)) [X < 2^B].
  ↪ Digit DP over the bits with an "is this
// variable still tight" flag per variable would need
  ↪ 2^n states - the trick below avoids that.
// THE TRICK: classify a solution by the HIGHEST bit
  ↪ k at which some x_i first drops below a_i.
// Above bit k every variable equals a_i exactly, so
  ↪ that prefix is forced. At bit k, at least one
// variable with a_i's bit set chooses 0 and becomes
  ↪ FREE below k; every other variable is either
// still tight (its low bits are then bounded by
  ↪ a_i's low bits) or already free (2^k choices).
// One designated free variable absorbs the xor
  ↪ constraint on the low bits - it has 2^k choices
// and exactly ONE of them fixes the remaining xor -
  ↪ which is why the first freed variable
// contributes a factor of 1 and every later one a
  ↪ factor of 2^k.
// If NOBODY drops at bit k, the parity of the k-th
  ↪ bits is forced to popcount-parity, so the
// recursion continues to bit k-1 only when that
  ↪ parity matches X's k-th bit.
// PRECONDITION: a_i >= 0 and X >= 0; B must cover
  ↪ every a_i and X (bit B-1 is the top).
ll countXorSolutions(const vector<ll>& a, ll X, int B
  ↪ = 60) {
    int n = a.size();
    ll res = 0;
    for (int k = B - 1; k >= 0; k--) {
        ll low = (1LL << k) % MOD;
        // dp[freed][parity of the k-th bits chosen
  ↪ so far]
        ll dp[2][2] = {{1, 0}, {0, 0}};
        for (int i = 0; i < n; i++) {
            ll lo = a[i] & ((1LL << (k + 1)) - 1);
  ↪          // bits k..0 of a_i
            ll nd[2][2] = {{0, 0}, {0, 0}};
            if (a[i] >> k & 1) {
                ll keep = (lo - (1LL << k) + 1) %
  ↪ MOD;          // x_i bit k = 1, low bits <= a_i
                for (int f = 0; f < 2; f++)
                    for (int p = 0; p < 2; p++) {
                        if (!dp[f][p]) continue;
                        nd[f][p ^ 1] = (nd[f][p ^ 1]
  ↪ + dp[f][p] * keep) % MOD;   // stay tight
                        ll fac = f ? low : 1;
  ↪          // the FIRST freed one absorbs xor
                        nd[1][p] = (nd[1][p] +
  ↪ dp[f][p] * fac) % MOD;            // drop to 0
  ↪ here
                    }
            } else {
                ll ch = (lo + 1) % MOD;
  ↪          // x_i bit k = 0, low bits <= a_i
                for (int f = 0; f < 2; f++)
                    for (int p = 0; p < 2; p++)
  ↪ nd[f][p] = dp[f][p] * ch % MOD;
            }
            memcpy(dp, nd, sizeof dp);
        }
        res = (res + dp[1][(X >> k) & 1]) % MOD;
        int par = 0;
  ↪          // nobody dropped at bit k
        for (int i = 0; i < n; i++) par ^= (a[i] >>
  ↪ k) & 1;
        if (par != ((X >> k) & 1)) return res;
  ↪          // the forced prefix already fails
    }
    return (res + 1) % MOD;
  ↪          // the all-tight tuple x_i = a_i
}
/* VARIANTS
 SUM OF THE SOLUTIONS, or counting with x_i in [lo_i,
  ↪ hi_i]: inclusion-exclusion over the lower
  bounds (subtract the count with x_i <= lo_i - 1) -
  ↪ 2^n terms, so only for tiny n; otherwise
  carry a second "already below lo_i" flag, which
  ↪ doubles the state, not the exponent.
 AND / OR CONSTRAINTS instead of xor: the same "first
  ↪ variable that drops" decomposition works,
  with the parity condition replaced by the AND/OR
  ↪ of the k-th bits.
 XOR EQUAL TO AT MOST X: sum the answers over the
  ↪ prefixes of X, i.e. for each bit where X has a
  1, count solutions matching X above that bit and 0
  ↪ there - one extra outer loop.
 LINEAR ALGEBRA ALTERNATIVE: if the bounds are all
  ↪ 2^B - 1 (or the variables range over a
  subspace), this is a rank computation over GF(2),
  ↪ not a DP - see 06 - Math/07 XOR basis.
 SANITY: n = 1 gives [X <= a_1]; all a_i = 2^B - 1
  ↪ gives 2^(B(n-1)). */
```

8.4.12 TransferMatrixDP

TRANSFER-MATRIX DP - when the DP has a HUGE time dimension (n up to 1e18) but few states.

```
// Needs: 06 - Math/03 - Matrices/01 -
  ↪ MatrixExponentiation.cpp   (Mat, operator*, mpow)
// RECOGNITION TEST: dp[t] is a vector over S states,
  ↪ and dp[t+1] = M * dp[t] with M INDEPENDENT
// of t. Then dp[n] = M^n * dp[0], computed in O(S^3
  ↪ log n). If M depends on t only through a
// small period p, multiply the p matrices once and
  ↪ exponentiate the product.
// WHEN: count strings of length 1e18 avoiding a
  ↪ pattern, count walks of a given length in a
// graph, count tilings of a 4 x 1e18 strip, any
  ↪ linear recurrence with n astronomically large.
// PRECONDITIONS, all checkable: (1) the transition
  ↪ is LINEAR in the dp values (sums of products
// with CONSTANTS - "max" is not linear, see min-plus
  ↪ below); (2) the state set does not grow with
// t; (3) the same matrix applies at every step (or
  ↪ with a fixed period).
// Build the matrix by RUNNING ONE STEP of the DP
  ↪ from each unit vector - that is mechanical and
// far less error-prone than deriving the entries by
  ↪ hand.
template <class Step>
Mat transferMatrix(int S, Step step) {
  ↪          // step(from, add) enumerates edges
    Mat M(S, vector<Mint>(S, 0));
    for (int i = 0; i < S; i++) step(i, [&](int j,
  ↪ Mint w) { M[j][i] += w; });
    return M;
  ↪          // COLUMN convention: dp' = M * dp
}
// EXAMPLE - count strings of length n over an
  ↪ alphabet of size A that avoid a set of patterns:
// states = the nodes of the Aho-Corasick automaton
  ↪ with terminal nodes deleted, and the step is
//   for each node u, for each character c: v =
  ↪ advance(u, c); if v is not terminal, add 1 edge.
// dp[0] = e_root, and the answer is the sum of the
  ↪ entries of M^n * dp[0].     O(S^3 log n)
/* THE VARIANTS
 MIN-PLUS ("shortest walk with exactly n edges",
  ↪ "minimum cost after n steps"): replace (+, *)
  with (min, +) and exponentiate the same way - see
  ↪ 06 - Math/03/07 - MinPlusMatrixPower.cpp.
  The matrix product is still associative, which is
  ↪ ALL that exponentiation needs; only the
  O(S^3) per multiply survives, there is no Strassen
  ↪ and no inverse.
 KITAMASA / BOSTAN-MORI (06 - Math/06/03) computes
  ↪ the n-th term of a linear recurrence of order
  k in O(k log k log n) instead of O(k^3 log n). If
  ↪ your matrix is a companion matrix - i.e. the
  DP is a plain linear recurrence - use that
  ↪ instead; k = 1000 is hopeless for matrices and
  ↪ easy
  for Bostan-Mori.
 BERLEKAMP-MASSEY finds the recurrence for you:
  ↪ generate 2k terms with the slow DP, recover the
  order-k recurrence, then jump to n. This is the
  ↪ practical route when the state space is
  structured but you cannot describe the matrix
  ↪ cleanly.
 PERIODIC / TIME-DEPENDENT transitions: if M_t has
  ↪ period p, exponentiate (M_{p-1} ... M_0).
 PROFILE DPs on a k x n grid become transfer matrices
  ↪ over the 2^k profiles - that is how "count
  tilings of a 5 x 1e18 board" is solved (and 2^k
  ↪ must stay under ~200 for the cube to fit).
 EIGENVALUES / CLOSED FORMS are almost never worth it
  ↪ in a contest: the matrix power is O(S^3
  log n) with no numerical risk, and a closed form
  ↪ needs distinct eigenvalues you cannot verify.
 SANITY: check M^1 and M^2 against the slow DP for
  ↪ small n before trusting M^1e18. */
```

8.5 Probability

8.5.1 ExpectedValueDP

EXPECTED VALUE / PROBABILITY DP.

```
// Needs: Mint (06 - Math/01) and solveLinear (06 -
  ↪ Math/04 - Linear Algebra/02 -
  ↪ GaussianElimination.cpp).
// LINEARITY OF EXPECTATION FIRST: E[sum of
  ↪ indicators] = sum of P(indicator). Most "expected
// number of X" problems collapse to summing
  ↪ per-element probabilities with no DP at all.
// Only when states depend on each other CYCLICALLY
  ↪ do you need a linear system.

// E[i] = 1 + sum_j P[i][j] * E[j], with E[absorbing]
  ↪ = 0.
// Move the unknowns left:  E[i] - sum_j P[i][j] E[j]
  ↪ = 1  ->  (I - P) E = 1. O(n^3).
// Needs solveLinear from GaussianElimination.cpp
  ↪ (values are Mint, so probabilities must be
// given as modular fractions p * inv(q)).
vector<Mint> hittingTimes(const vector<vector<Mint>>&
  ↪ P, const vector<char>& absorbing) {
    int n = P.size();
    vector<vector<Mint>> A(n, vector<Mint>(n, 0));
    vector<Mint> b(n, 0);
    for (int i = 0; i < n; i++) {
        if (absorbing[i]) { A[i][i] = 1, b[i] = 0;
  ↪ continue; }
        A[i][i] = 1;
        for (int j = 0; j < n; j++) A[i][j] -=
  ↪ P[i][j];
        b[i] = 1;
    }
    return solveLinear(A, b);
}
/* ABSORBING MARKOV CHAIN, canonical form P = [Q R ;
  ↪ 0 I]:
    fundamental matrix N = (I - Q)^-1 ;  expected
  ↪ steps before absorption = N * 1
    absorption probabilities B = N * R  (B[i][j] =
  ↪ P(absorbed in j | started in i))

   TRIDIAGONAL SYSTEMS (grid random walks that only
  ↪ move left/right within a row) - O(n) with
   the Thomas algorithm instead of O(n^3):
     a_i x_{i-1} + b_i x_i + c_i x_{i+1} = d_i , a_1
  ↪ = c_n = 0
     forward:  c'_i = c_i / (b_i - a_i c'_{i-1}) ,
  ↪ d'_i = (d_i - a_i d'_{i-1}) / (b_i - a_i
  ↪ c'_{i-1})
     back:     x_n = d'_n ,  x_i = d'_i - c'_i x_{i+1}
   Diagonally dominant (always true for hitting
  ↪ times), so it is stable.

   TREES: substitute E_v = A_v * E_parent + B_v, push
  ↪ A and B up in one DFS, then resolve at the
   root and push back down. O(n). (Folklore - no
```

```
↪ canonical reference; derive it, do not trust it.)

 COMMON TRAPS
   - "expected value of a maximum" is NOT the
↪ maximum of expected values; sum over thresholds:
     E[max] = sum_{t>=1} P(max >= t).
   - Probabilities mod p: store p*inv(q); never mix
↪ doubles and Mint.
   - Infinite processes: E exists only if
↪ absorption has probability 1; check reachability
↪ first.
   - Conditional expectation: E[X] = sum_y P(Y=y) *
↪ E[X | Y=y]  - condition on the FIRST step.  */
```

9 | Trees

9.1 Lowest Common Ancestor (LCA)

9.1.1 LCA

LCA by Euler tour + sparse table: O(n log n) build, O(1) query. 0-based, rooted at 'root'.

```
// For k-th ancestor / "jump while predicate" use
↪ BinaryLifting instead (02 - BinaryLifting.cpp).
struct LCA {
    static const int B = 21;
    vector<array<int, 2>> tour;
    vector<array<array<int, 2>, B>> T;
    vector<int> d, in, lg;

    template <class G>
    LCA(int n, const G& adj, int root = 0) : d(n),
↪ in(n), lg(2 * n + 1) {
        tour.reserve(2 * n);
        dfs(root, -1, adj);
        int m = tour.size();                    // ==
↪ 2n - 1
        T.resize(m);
        for (int i = 2; i <= 2 * n; i++) lg[i] = lg[i
↪ / 2] + 1;
        for (int i = 0; i < m; i++) T[i][0] = tour[i];
        for (int j = 1; j < B; j++)
            for (int i = 0; i + (1 << j) - 1 < m; i++)
                T[i][j] = min(T[i][j - 1], T[i + (1
↪ << (j - 1))][j - 1]);
    }
    template <class G>
    void dfs(int u, int p, const G& adj) {
        in[u] = tour.size();
        tour.push_back({d[u], u});
        for (int v : adj[u]) if (v != p) {
            d[v] = d[u] + 1;
            dfs(v, u, adj);
            tour.push_back({d[u], u});
        }
    }
    int get(int u, int v) const {
        int l = min(in[u], in[v]), r = max(in[u],
↪ in[v]), j = lg[r - l + 1];
        return min(T[l][j], T[r - (1 << j) +
↪ 1][j])[1];
    }
    int dist(int u, int v) const { return d[u] + d[v]
↪ - 2 * d[get(u, v)]; }
};
```

9.1.2 BinaryLifting

BINARY LIFTING - k-th ancestor, LCA, and "jump while a predicate holds", all O(log n).

```
// This is NOT only a tree tool: the same table jumps
↪ 2^k steps in ANY functional graph
// (i -> f(i)), which is how you do "apply the
↪ operation 1e18 times", permutation powers,
// and "min lamps to cover [L,R]" (jump by
↪ farthest-reach).
struct Lift {
    int n, B;
    vector<vector<int>> up;                     //
↪ up[k][v] = 2^k-th ancestor (-1 above the root)
    vector<int> d;

    template <class G>
    Lift(int n, const G& adj, int root = 0) : n(n),
↪ B(1), d(n, 0) {
        while ((1 << B) < n) B++;
        B++;
        up.assign(B, vector<int>(n, -1));
        dfs(root, -1, adj);
        for (int k = 1; k < B; k++)
            for (int v = 0; v < n; v++)
                up[k][v] = up[k - 1][v] < 0 ? -1 :
↪ up[k - 1][up[k - 1][v]];
    }
    template <class G>
    void dfs(int u, int p, const G& adj) {
        up[0][u] = p;
        for (int v : adj[u]) if (v != p) d[v] = d[u]
↪ + 1, dfs(v, u, adj);
    }
    int kth(int v, ll k) const {                //
↪ k-th ancestor, -1 if it does not exist
        for (int i = 0; i < B && v >= 0; i++) if (k
↪ >> i & 1) v = up[i][v];
        return k >> B ? -1 : v;
    }
    int lca(int u, int v) const {
        if (d[u] < d[v]) swap(u, v);
        u = kth(u, d[u] - d[v]);
        if (u == v) return u;
        for (int i = B - 1; i >= 0; i--) if (up[i][u]
↪ != up[i][v]) u = up[i][u], v = up[i][v];
        return up[0][u];
    }
    int dist(int u, int v) const { return d[u] + d[v]
↪ - 2 * d[lca(u, v)]; }
    int onPath(int u, int v, int k) const {     //
↪ k-th vertex on the path u -> v (k = 0 gives u)
        int a = lca(u, v), du = d[u] - d[a];
        return k <= du ? kth(u, k) : kth(v, d[u] +
↪ d[v] - 2 * d[a] - k);
    }
    // highest ancestor of v still satisfying pred
↪ (monotone: true near v, false near the root)
    template <class F>
    int jumpWhile(int v, F pred) const {
        for (int i = B - 1; i >= 0; i--) if (up[i][v]
↪ >= 0 && pred(up[i][v])) v = up[i][v];
        return v;
    }
};
```

```
// STANDALONE version for a functional graph f (no tree needed):
//   up[0][v] = f(v); up[k][v] = up[k-1][up[k-1][v]]; then apply k steps by the bits of k.
```

9.1.3 LCAConstantTime

LCA IN O(1) after O(n log n) preprocessing - Euler tour + sparse table on DEPTH.

```
// Binary lifting (02) is O(log n) per query and needs O(n log n) memory too, so reach for this
// when the query count dominates: O(1) LCA turns "distance between every pair in a list" and
// "is u an ancestor of v" from n log n into n. It is also the primitive behind O(1) LCE on a
// suffix tree and behind virtual trees built in bulk.
// The tour has 2n-1 entries; the LCA of u and v is the SHALLOWEST vertex between their first
// occurrences, and since consecutive tour depths differ by exactly +-1 a plain sparse table over
// (depth, vertex) pairs is enough - min over an overlapping range is idempotent.
struct LCA {
    int n, LOG;
    vector<int> first, depth, tour;
    vector<vector<pair<int, int>>> sp;
                // (depth, vertex)
    LCA(int n, const vector<vector<int>>& adj, int root = 0)
        : n(n), first(n, -1), depth(n, 0) {
        tour.reserve(2 * n);
        vector<pair<int, int>> st{{root, -1}};
                // iterative: no stack overflow
        vector<int> it(n, 0);
        depth[root] = 0;
        while (!st.empty()) {
            auto& [u, p] = st.back();
            if (first[u] < 0) first[u] = tour.size();
            if (it[u] < (int)adj[u].size()) {
                int v = adj[u][it[u]++];
                if (v == p) continue;
                tour.push_back(u), depth[v] = depth[u] + 1;
                st.push_back({v, u});
            } else {
                tour.push_back(u);
                st.pop_back();
            }
        }
        int m = tour.size();
        LOG = 1;
        while ((1 << LOG) <= m) LOG++;
        sp.assign(LOG, vector<pair<int, int>>(m));
        for (int i = 0; i < m; i++) sp[0][i] = {depth[tour[i]], tour[i]};
        for (int k = 1; k < LOG; k++)
            for (int i = 0; i + (1 << k) <= m; i++)
                sp[k][i] = min(sp[k - 1][i], sp[k - 1][i + (1 << (k - 1))]);
    }
    int lca(int u, int v) const {
        int a = first[u], b = first[v];
        if (a > b) swap(a, b);
        int k = 31 - __builtin_clz(b - a + 1);
        return min(sp[k][a], sp[k][b - (1 << k) + 1]).second;
    }
    int dist(int u, int v) const { return depth[u] + depth[v] - 2 * depth[lca(u, v)]; }
    bool isAncestor(int u, int v) const { return lca(u, v) == u; }
};
// TRUE O(n) PREPROCESSING is possible (Farach-Colton-Bender): split the +-1 depth array into
// blocks of size log(n)/2, precompute every possible block pattern (there are only sqrt(n) of
// them), and put a sparse table over the block minima. It is ~60 lines and only worth it at
// n >= 5e5, where the O(n log n) table's memory is the real problem.

// --- SUBTREE HASHING: are these two subtrees the same shape? O(n log n). ---
// Canonical form: sort the children's ids and look the sorted vector up in a map. Two subtrees
// get the same id iff they are isomorphic as ROOTED trees.
// Shorter than AHU (09 - Isomorphism) and merges naturally into other tree DPs; AHU is still the
// one to use when you need the UNROOTED comparison (root at the centres and take the minimum).
map<vector<int>, int> subtreeId;
vector<int> subHash;
int hashSubtree(int u, int p, const vector<vector<int>>& adj) {
    vector<int> ch;
    for (int v : adj[u]) if (v != p) ch.push_back(hashSubtree(v, u, adj));
    sort(all(ch));
    auto it = subtreeId.find(ch);
    if (it == subtreeId.end()) it = subtreeId.emplace(ch, subtreeId.size()).first;
    return subHash[u] = it->second;
}
// RANDOMISED VARIANT, O(n) and mergeable: h(u) = XOR over children of f(h(child)) with f a fixed
// random 64-bit mixing function (splitmix64). No map, no sorting, and it composes under rerooting
// - which is what you want when you need the hash of every subtree AND of every "rest of tree".

// --- MERGING TWO TREE PATHS. A segment tree node stores the MINIMAL PATH CONTAINING all the
// vertices in its range, as that path's two endpoints ({-1,-1} = empty, {-2,-2} = they do not fit
// on any path). Merging is O(1) given an O(1) LCA: the covering path of two paths, if it exists,
// has its endpoints among the four given ones, so test each of the four candidate pairs (u, v)
// with dist(u,v) == dist(u,w) + dist(w,v) for every endpoint w.
// NOTE THE SEMANTIC: this is the smallest path CONTAINING both, not their union - two disjoint
// pieces of one long path merge into that long path, which is what a covering query wants.
// USES: "is this set of vertices contained in one path", "the smallest path covering a range of
// queries", and segment-tree-over-queries problems on trees.
struct PathNode { int x = -1, y = -1; };
PathNode mergePaths(PathNode a, PathNode b, const LCA& L) {
    if (a.x < 0) return b;
    if (b.x < 0) return a;
    int e[4] = {a.x, a.y, b.x, b.y};
```

```
    // ALL SIX pairs, not just four:
    for (int i = 0; i < 4; i++) for (int j = i; j < 4; j++) {   // (a.x,b.x) and (a.y,b.y) are spanning pairs too
        int u = e[i], v = e[j];
        bool ok = true;
        for (int w : e) if (L.dist(u, v) != L.dist(u, w) + L.dist(w, v)) ok = false;
        if (ok) return {u, v};
    }
    return {-2, -2};          // not a path: mark it invalid
}
```

9.2 DSU on Tree

9.2.1 DSUOnTree

DSU ON TREE (small to large) - answers offline subtree queries in O(n log n) by keeping the heavy child's data and re-adding only the light subtrees. Each vertex is added O(log n) times. Use it for "most frequent colour in each subtree" style problems where merging maps would be O(n^2).

```
class DSUOnTree {
private:
    struct Query {
        int idx, x;
    };

    int n;
    vector<vector<int>> adj;
    vector<int> sz, depth;
    vector<bool> big;
    vector<vector<Query>> queries;
    vector<int> ans;

    // ========================================
    //       PROBLEM SPECIFIC STATE VARIABLES
    // ========================================
    vector<char> c; // Example: Array of characters per node

    // Example state variables
    // int distinct_count = 0;
    // vector<int> freq;

    void add(int u, int p) {
        // --- ADD LOGIC HERE ---
        // Example: freq[c[u] - 'a']++;

        for (int v : adj[u]) {
            if (v == p || big[v]) continue; // Skip parent and heavy child
            add(v, u);
        }
    }

    void remove(int u, int p) {
        // --- REMOVE LOGIC HERE ---
        // Example: freq[c[u] - 'a']--;

        for (int v : adj[u]) {
            if (v == p || big[v]) continue; // Skip parent and heavy child
            remove(v, u);
        }
    }

    int get_answer(int u, int x = 0) {
        // --- QUERY ANSWER LOGIC HERE ---
        return 0;
    }
    // ========================================

    void pre(int u, int p = 0) {
        sz[u] = 1;
        depth[u] = depth[p] + 1;
        for (int v : adj[u]) {
            if (v == p) continue;
            pre(v, u);
            sz[u] += sz[v]; // Compute subtree size
        }
    }

    void dfs(int u, int p = 0, bool keep = false) {
        int mx = -1, bigChild = -1;

        // Find the heavy child
        for (int v : adj[u]) {
            if (v == p) continue;
            if (sz[v] > mx) {
                mx = sz[v];
                bigChild = v;
            }
        }

        // Process light children and clear them
        for (int v : adj[u]) {
            if (v == p || v == bigChild) continue;
            dfs(v, u, false);
        }

        // Process heavy child and keep it
        if (bigChild != -1) {
            dfs(bigChild, u, true);
            big[bigChild] = true;
        }

        // Add current node and its light children
        add(u, p);

        // Answer all queries for the current node
        for (const auto& q : queries[u]) {
            ans[q.idx] = get_answer(u, q.x);
        }

        // Reset the big child flag
        if (bigChild != -1) {
            big[bigChild] = false;
        }

        // Clear state if this node is not meant to be kept
        if (!keep) {
            remove(u, p);
        }
    }

public:
    // Initialize with 1-based indexing in mind
    DSUOnTree(int nodes, const vector<char>& chars) : n(nodes), c(chars) {
        adj.assign(n + 1, vector<int>());
        sz.assign(n + 1, 0);
        depth.assign(n + 1, 0);
```

```
        big.assign(n + 1, false);
        queries.assign(n + 1, vector<Query>());
        // Initialize problem-specific arrays here
    ↪ (e.g., freq.assign(26, 0);)
    }

    void add_edge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    void add_query(int node, int query_idx, int x =
    ↪ 0) {
        queries[node].push_back({query_idx, x});
    }

    vector<int> solve(int root, int num_queries) {
        ans.assign(num_queries, 0);
        pre(root, 0); // Precalculate sizes and depths
        dfs(root, 0, false); // Run the sack algorithm
        return ans;
    }
};
```

9.3 Heavy Light Decomposition (HLD)

9.3.1 HLD

HEAVY-LIGHT DECOMPOSITION - splits the tree into chains so any root-to-node path crosses O(log n)

```
// of them; combined with a segment tree it gives
    ↪ path update/query in O(log^2 n).
// Subtrees are contiguous in tin[], so subtree
    ↪ queries are a single range on the same tree.
class HLD {
public:
    vector<int> par, sz, head, tin, tout, who, depth;

    int dfs1(int u, vector<vector<int> > &adj) {
        for (int &v: adj[u]) {
            if (v == par[u])continue;
            depth[v] = depth[u] + 1;
            par[v] = u;
            sz[u] += dfs1(v, adj);
            if (sz[v] > sz[adj[u][0]] || adj[u][0] ==
    ↪ par[u]) swap(v, adj[u][0]);
        }
        return sz[u];
    }

    void dfs2(int u, int &timer, const
    ↪ vector<vector<int> > &adj) {
        tin[u] = timer++;
        for (int v: adj[u]) {
            if (v == par[u])continue;
            head[v] = (timer == tin[u] + 1 ? head[u]
    ↪ : v);
            dfs2(v, timer, adj);
        }
        tout[u] = timer - 1;
    }

    HLD(vector<vector<int> > adj, int r = 0)
        : par(adj.size(), -1), sz(adj.size(), 1),
    ↪ head(adj.size(), r), tin(adj.size()),
    ↪ tout(adj.size()),
          who(adj.size()),
          depth(adj.size()) {
        dfs1(r, adj);
        int x = 0;
        dfs2(r, x, adj);
        for (int i = 0; i < adj.size(); ++i)
    ↪ who[tin[i]] = i;
    }

    vector<pair<int, int> > path(int u, int v) {
        vector<pair<int, int> > res;
        for (;; v = par[head[v]]) {
            if (depth[head[u]] >
    ↪ depth[head[v]])swap(u, v);
            if (head[u] != head[v]) {
                res.emplace_back(tin[head[v]],
    ↪ tin[v]);
            } else {
                if (depth[u] > depth[v])swap(u, v);
                res.emplace_back(tin[u], tin[v]);
                return res;
            }
        }
    }

    pair<int, int> subtree(int u) {
        return {tin[u], tout[u]};
    }

    int dist(int u, int v) {
        return depth[u] + depth[v] - 2 * depth[lca(u,
    ↪ v)];
    }

    int lca(int u, int v) {
        for (;; v = par[head[v]]) {
            if (depth[head[u]] >
    ↪ depth[head[v]])swap(u, v);
            if (head[u] == head[v]) {
                if (depth[u] > depth[v])swap(u, v);
                return u;
            }
        }
    }

    bool isAncestor(int u, int v) {
        return tin[u] <= tin[v] && tout[u] >= tout[v];
    }
};
```

9.4 Centroid Decomposition

9.4.1 Centroid

CENTROID DECOMPOSITION - recursively remove the centroid (the vertex whose largest remaining

```
// component is <= n/2), giving a decomposition tree
    ↪ of depth O(log n). Every path in the original
// tree passes through the centroid of some level,
    ↪ which turns path-counting problems into
// "for each centroid, combine the distance lists of
    ↪ its components" in O(n log n).
class CentroidDecomposition {
private:
    int n;
    vector<vector<int>> adj;
    vector<int> sz, par, nodes;
    vector<bool> vis;

    int dfsSz(int u, int p) {
        sz[u] = 1;
        for (int v : adj[u]) {
```

```
            if (vis[v] || v == p) continue;
            sz[u] += dfsSz(v, u);
        }
        return sz[u];
    }

    int getCentroid(int u, int p, int total_nodes) {
        for (int v : adj[u]) {
            if (vis[v] || v == p) continue;
            if (sz[v] * 2 > total_nodes) {
                return getCentroid(v, u, total_nodes);
            }
        }
        return u;
    }

    void collect(int u, int p) {
        nodes.push_back(u);
        for (int v : adj[u]) {
            if (vis[v] || v == p) continue;
            collect(v, u);
        }
    }

    void build(int u, int p) {
        int total_nodes = dfsSz(u, p);
        int centroid = getCentroid(u, p, total_nodes);

        if (p != -1) {
            par[centroid] = p;
        } else {
            par[centroid] = centroid;
        }

        vis[centroid] = true;

        // ========================================
        //       PROBLEM SPECIFIC PROCESSING
        // ========================================
        // Process paths going through the 'centroid'
  ↪ here
        for (int v : adj[centroid]) {
            if (vis[v]) continue;
            nodes.clear();
            collect(v, centroid);

            // e.g., Update answers using the nodes
  ↪ collected from this subtree branch
        }
        // ========================================

        // Recursively decompose the remaining
  ↪ components
        for (int v : adj[centroid]) {
            if (vis[v]) continue;
            build(v, centroid);
        }
    }

public:
    // Initialize with 1-based indexing
    CentroidDecomposition(int nodes_count) :
  ↪ n(nodes_count) {
        adj.assign(n + 1, vector<int>());
        sz.assign(n + 1, 0);
        par.assign(n + 1, 0);
        vis.assign(n + 1, false);
    }

    void add_edge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    // Start decomposition, usually from node 1
    void decompose(int root = 1) {
        build(root, -1);
    }

    // Returns the parent of a node in the centroid
  ↪ tree
    int get_centroid_parent(int u) const {
        return par[u];
    }
};
```

9.4.2 PathLengthCounts

NUMBER OF PATHS OF EVERY LENGTH IN A TREE - cnt[L] = how many unordered pairs {u, v} have

```
// Needs: conv (06 - Math/05 - Convolution/01 -
  ↪ FFT.cpp).
// dist(u, v) == L, for ALL L at once, in O(n log^2
  ↪ n). 0-indexed, unweighted tree.
// The naive "for each length, count" is O(n^2); a
  ↪ plain centroid decomposition answers "paths of
// length <= K" in O(n log n) but not all lengths
  ↪ simultaneously. The fix is to convolve the depth
// HISTOGRAMS instead of merging them pairwise: at
  ↪ each centroid, square the histogram of all
// distances and subtract the square of each child
  ↪ component's histogram (inclusion-exclusion for
// the pairs that never actually cross the centroid).
// Reach for it for "for every k, how many pairs are
  ↪ at distance k", "sum over pairs of f(dist)"
// when f is arbitrary, and distance-distribution /
  ↪ diameter-profile problems.
vector<ll> pathLengthCounts(int n, const
  ↪ vector<vector<int>>& adj) {
    vector<ll> cnt(n + 1, 0);
    vector<int> sub_(n, 0), dep;
    vector<char> done(n, 0);
    int tot = 0;
    function<void(int, int)> calcSz = [&](int u, int
  ↪ p) {
        tot++, sub_[u] = 1;
        for (int v : adj[u]) if (v != p && !done[v])
  ↪ calcSz(v, u), sub_[u] += sub_[v];
    };
    function<int(int, int)> centroid = [&](int u, int
  ↪ p) {
        for (int v : adj[u]) if (v != p && !done[v]
  ↪ && sub_[v] * 2 > tot) return centroid(v, u);
        return u;
    };
    function<void(int, int, int)> gather = [&](int u,
  ↪ int p, int d) {
        dep.push_back(d);
        for (int v : adj[u]) if (v != p && !done[v])
  ↪ gather(v, u, d + 1);
    };
    function<void(int, int, int, int)> add = [&](int
  ↪ u, int p, int d0, int sign) {
        dep.clear(), gather(u, p, d0);
        vector<ll> h(*max_element(all(dep)) + 1, 0);
        for (int d : dep) h[d]++;
        vector<ll> r = conv(h, h);
```

```
        // ordered pairs, diagonal included
        for (int i = 0; i < (int)r.size() && i <= n;
    i++) cnt[i] += sign * r[i];
      };
      function<void(int, int)> decomp = [&](int u, int
    p) {
        tot = 0, calcSz(u, p);
        int c = centroid(u, p);
        done[c] = 1;
        add(c, p, 0, 1);
        // all pairs inside this component
        for (int v : adj[c]) if (!done[v]) add(v, c,
    1, -1);    // minus the ones inside one child
        for (int v : adj[c]) if (!done[v]) decomp(v,
    c);
      };
      if (n) decomp(0, -1);
      cnt[0] = 0;
      for (ll& x : cnt) x /= 2;
        // ordered -> unordered
      return cnt;
        // cnt[L] for L = 1..n-1
}
// WEIGHTED trees: the same decomposition works, but
    the histogram is indexed by weight, so it is
// only practical when the total weight is small.
    Otherwise sort the distance lists and use two
// pointers - that answers "pairs with dist <= K" but
    not all lengths at once.
// SUM OVER PAIRS OF f(dist): convolve as above and
    dot with f. Any f works, which is the whole
// point of getting the full distribution.
// ONE LENGTH ONLY, or "count paths with dist <= K":
    plain centroid decomposition with sorting,
// O(n log^2 n) and no FFT at all - see 01 - Centroid.
```

9.5 Tree Diameter

9.5.1 TreeDiameter

Tree Diameter via 2-BFS - Time: O(V + E), Space: O(V + E)

```
// Computes diameter length, endpoints (u, v), and
    the full diameter path
struct TreeDiameter {
    int n;
    vector<vector<int>> adj;
    vector<int> dist, par;

    TreeDiameter(int n = 0) : n(n), adj(n + 1),
    dist(n + 1), par(n + 1) {}

    void add_edge(int u, int v) {
        adj[u].pb(v);
        adj[v].pb(u);
    }

    int bfs(int src) {
        fill(all(dist), -1);
        queue<int> q;
        q.push(src);
        dist[src] = 0;
        par[src] = -1;

        int farthest = src;
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            if (dist[u] > dist[farthest]) farthest =
    u;
            for (int v : adj[u]) {
                if (dist[v] == -1) {
                    dist[v] = dist[u] + 1;
                    par[v] = u;
                    q.push(v);
                }
            }
        }
        return farthest;
    }

    // Returns {diameter_length, {endpoint_u,
    endpoint_v}}
    pair<int, pair<int, int>> get_diameter() {
        int u = bfs(1);
        int v = bfs(u);
        return {dist[v], {u, v}};
    }

    // Returns the sequence of vertices on the
    diameter path from u to v
    vector<int> get_path() {
        auto [len, endpoints] = get_diameter();
        auto [u, v] = endpoints;
        vector<int> path;
        for (int curr = v; curr != -1; curr =
    par[curr]) {
            path.pb(curr);
        }
        reverse(all(path));
        return path;
    }
};
```

9.6 Euler Tour

9.6.1 EulerTour

Euler Tour Technique (Tree Flattening into 1D Array Intervals) - O(V + E)

```
// Subtree of node u corresponds to contiguous range
    [in_time[u], out_time[u]] in flat_tree
struct EulerTour {
    int n, timer = 0;
    vector<vector<int>> adj;
    vector<int> in_time, out_time, flat_tree;

    EulerTour(int n = 0) : n(n), adj(n + 1),
    in_time(n + 1), out_time(n + 1) {}

    void add_edge(int u, int v) {
        adj[u].pb(v);
        adj[v].pb(u);
    }

    void dfs(int u, int p = -1) {
        in_time[u] = ++timer;
        flat_tree.pb(u);
        for (int v : adj[u]) {
            if (v != p) dfs(v, u);
        }
        out_time[u] = timer;
    }

    void build(int root = 1) {
        timer = 0;
        flat_tree.clear();
        dfs(root);
```

```
    }

    // Returns [L, R] 1-based index range for subtree queries on node u
    pair<int, int> get_subtree_range(int u) const {
        return {in_time[u], out_time[u]};
    }
};

/* THE EULER-TOUR TOOLBOX - four different tours, each answering a different query family.
 CONVENTION HERE: in_time is 1-based (in_time[u] = ++timer) but flat_tree is a 0-based vector,
 so flat_tree[in_time[u] - 1] == u. Keep that straight when you index a BIT.

 1) SUBTREE = a contiguous range [in[v], out[v]].
      point update, subtree query  -> BIT over in[]: add at in[u], query [in[v], out[v]]
      subtree update, point query  -> DIFFERENCE BIT: +x at in[v], -x at out[v]+1,
                                      then the value at u is the prefix sum up to in[u]
 2) PATH TO THE ROOT, using the SAME difference trick in the other direction:
      subtree update, path-to-root query  <-> path update, point query (they are duals).
      PATH UPDATE u..v by +x, point query:  +x at in[u], +x at in[v], -x at in[lca],
                                            -x at in[parent(lca)]
      PATH QUERY u..v, point update:
      query(in[u]) + query(in[v]) - 2*query(in[lca])  (+ the LCA's own value if vertices carry weights)
 3) THE 2n TOUR (+1 on entry, -1 on exit) turns ancestor tests and LCA into RMQ:
      u is an ancestor of v  iff  in[u] <= in[v] && out[v] <= out[u]
      LCA(u,v) = the vertex of minimum DEPTH in the tour range [in[u], in[v]] - and consecutive
      depths differ by exactly 1, which is what the O(n)/O(1) +-1 RMQ exploits.
 4) THE EDGE TOUR (each edge appears twice) is what Mo's-on-trees uses: a vertex appearing TWICE
      inside the range cancels, so a path becomes one contiguous range plus the LCA by hand.
      See 02 - Data Structures/06 - Square Root Decomposition/03 - MoOnPaths.cpp.

 EDGE WEIGHTS instead of vertex weights: push each edge's weight onto its DEEPER endpoint, then
 every path formula above drops the LCA term (the LCA's incoming edge is not on the path). */
```

9.7 Rerooting

9.7.1 Rerooting

REROOTING DP - compute f(v as root) for EVERY v in O(n).

```
// Four hooks:  E() identity of merge * self(v) contribution of v itself *
//              merge(a,b) associative+commutative * lift(a, child) push a child's value
//              across one edge into the parent's frame.
// Shipped example: ans[v].first = sum of distances from v to every other node.
struct Reroot {
    typedef pair<ll, ll> T;                                  // {sum of distances, #nodes}
    static T E() { return {0, 0}; }
    static T self(int v) { return {0, 1}; }
    static T merge(T a, T b) { return {a.first + b.first, a.second + b.second}; }
    static T lift(T a, int child) { return {a.first + a.second, a.second}; }

    int n;
    vector<vector<int>> adj;
    vector<T> down, up, ans;
    vector<int> par, ord;

    Reroot(int n) : n(n), adj(n), down(n, E()), up(n, E()), ans(n, E()), par(n, -1) {}
    void add_edge(int u, int v) { adj[u].push_back(v), adj[v].push_back(u); }

    void build(int root = 0) {
        ord.clear(), ord.reserve(n);
        vector<int> st = {root};
        par[root] = -1;
        while (!st.empty()) {                                // iterative order: no recursion depth limit
            int u = st.back(); st.pop_back();
            ord.push_back(u);
            for (int v : adj[u]) if (v != par[u])
                par[v] = u, st.push_back(v);
        }
        for (int i = (int)ord.size() - 1; i >= 0; i--) {   // pass 1: leaves -> root
            int u = ord[i];
            down[u] = self(u);
            for (int v : adj[u]) if (v != par[u])
                down[u] = merge(down[u], lift(down[v], v));
        }
        up[root] = E();
        for (int u : ord) {                                  // pass 2: root -> leaves
            vector<int> ch;
            for (int v : adj[u]) if (v != par[u])
                ch.push_back(v);
            int k = ch.size();
            vector<T> pre(k + 1, E()), suf(k + 1, E());
            for (int i = 0; i < k; i++) pre[i + 1] = merge(pre[i], lift(down[ch[i]], ch[i]));
            for (int i = k - 1; i >= 0; i--) suf[i] = merge(suf[i + 1], lift(down[ch[i]], ch[i]));
            ans[u] = merge(down[u], up[u]);
            for (int i = 0; i < k; i++)
                up[ch[i]] = lift(merge(self(u), merge(pre[i], suf[i + 1]))), u);
        }
    }
};
// The prefix/suffix trick ("aggregate over all children EXCEPT one") is reusable on its own:
// use it whenever a DP needs "everything but this branch" and merge has no inverse.
// OTHER INSTANTIATIONS: max depth from each node (T = ll, lift = a+1, self = 0);
// count of nodes at even distance; product of subtree answers; tree diameter through each node.
```

9.8 Virtual Tree

9.8.1 VirtualTree

VIRTUAL (AUXILIARY) TREE - compress k marked vertices plus their pairwise LCAs into a tree

```
// with at most 2k-1 nodes, in O(k log k). Run the
   ↪ real DP on that instead of on all n nodes.
// THE pattern for "q queries, each with k_i marked
   ↪ vertices, sum of k_i bounded".
// Needs tin/tout from a DFS and an LCA (binary
   ↪ lifting or Euler+sparse table).
struct VirtualTree {
    int n;
    vector<int> tin, tout, dep;
    vector<vector<int>> adj, vadj;
   ↪ // vadj = the compressed tree
    int timer = 0;

    VirtualTree(int n) : n(n), tin(n), tout(n),
   ↪ dep(n, 0), adj(n), vadj(n) {}
    void add_edge(int u, int v) {
   ↪ adj[u].push_back(v), adj[v].push_back(u); }
    void dfs(int u, int p) {
        tin[u] = timer++;
        for (int v : adj[u]) if (v != p) dep[v] =
   ↪ dep[u] + 1, dfs(v, u);
        tout[u] = timer++;
    }
    bool isAnc(int u, int v) const { return tin[u] <=
   ↪ tin[v] && tout[v] <= tout[u]; }

    // `lca` must be a working LCA callable. Returns
   ↪ the roots' node list; edges land in vadj.
    template <class LCA>
    vector<int> build(vector<int> key, LCA lca) {
        if (key.empty()) return {};
        sort(all(key), [&](int a, int b) { return
   ↪ tin[a] < tin[b]; });
        key.erase(unique(all(key)), key.end());
        int k = key.size();
        for (int i = 0; i + 1 < k; i++)
   ↪ key.push_back(lca(key[i], key[i + 1]));
        sort(all(key), [&](int a, int b) { return
   ↪ tin[a] < tin[b]; });
        key.erase(unique(all(key)), key.end());
        for (int u : key) vadj[u].clear();
        vector<int> st = {key[0]};
        for (size_t i = 1; i < key.size(); i++) {
            int u = key[i];
            while (!isAnc(st.back(), u))
   ↪ st.pop_back();
            vadj[st.back()].push_back(u);
   ↪ // parent -> child (directed is enough)
            st.push_back(u);
        }
        return key;
   ↪ // key[0] is the root of the virtual tree
    }
};
// ALWAYS clear vadj for exactly the nodes you used
   ↪ (done above) - never clear all n per query,
// or the total cost becomes O(q*n) and defeats the
   ↪ whole point.
// The compressed edge u -> v represents the real
   ↪ path between them; its "weight" is usually
// dep[v] - dep[u], or a path aggregate you can query
   ↪ in O(1) with prefix sums / HLD.
```

9.9 Isomorphism

9.9.1 TreeIsomorphism

TREE ISOMORPHISM (AHU canonical form). O(n log n) with a map, O(n) with per-level bucketing.

```
// Gives every rooted subtree a canonical ID: two
   ↪ subtrees are isomorphic iff their IDs match.
// Exact - unlike random subtree hashing, which can
   ↪ collide.
struct TreeIso {
    map<vector<int>, int> id;
   ↪  // canonical child-multiset -> id
    // canonical id of the subtree rooted at u
    int rootedId(int u, int p, const
   ↪ vector<vector<int>>& adj) {
        vector<int> ch;
        for (int v : adj[u]) if (v != p)
   ↪ ch.push_back(rootedId(v, u, adj));
        sort(all(ch));
        auto it = id.find(ch);
        if (it != id.end()) return it->second;
        int nid = id.size();
        return id[ch] = nid;
    }
    // CENTERS of a tree: 1 or 2 vertices (Jordan).
   ↪ Root there to canonicalise an UNROOTED tree.
    vector<int> centers(int n, const
   ↪ vector<vector<int>>& adj) {
        if (n == 1) return {0};
        vector<int> deg(n), leaves;
        for (int i = 0; i < n; i++) if ((deg[i] =
   ↪ adj[i].size()) <= 1) leaves.push_back(i);
        int removed = leaves.size();
        while (removed < n) {
            if (leaves.empty()) break;
   ↪  // guard: not a tree / disconnected
            vector<int> nxt;
            for (int u : leaves)
                for (int v : adj[u]) if (--deg[v] ==
   ↪ 1) nxt.push_back(v);
            leaves = nxt, removed += leaves.size();
        }
        return leaves;
   ↪  // 1 or 2 vertices, adjacent if 2
    }
    // Canonical id of an UNROOTED tree: min over its
   ↪ centers.
    int unrootedId(int n, const vector<vector<int>>&
   ↪ adj) {
        auto c = centers(n, adj);
        int best = rootedId(c[0], -1, adj);
        if (c.size() == 2) best = min(best,
   ↪ rootedId(c[1], -1, adj));
        return best;
    }
};
// Two unrooted trees are isomorphic iff unrootedId
   ↪ is equal (use ONE shared TreeIso object so
// the id space is common). Two rooted trees: compare
   ↪ rootedId at their roots.
// USES: "how many distinct tree shapes", dedupe
   ↪ trees, count isomorphism classes, group
// identical subtrees for a DP over shapes.
// JORDAN: a tree has exactly one or two centers; if
   ↪ two, they are adjacent.
```

9.10 Tree DP

9.10.1 TreeDP

TREE DP - the most common tree shape in a contest. Root the tree, compute a value per subtree

```
// bottom-up, and combine children. If the answer must be "for every root", see 07 - Rerooting.

// --- MAXIMUM WEIGHT INDEPENDENT SET / MINIMUM VERTEX COVER on a tree, O(n). ---
// dp[v][0] = best for v's subtree with v NOT taken, dp[v][1] = with v taken.
// A tree is bipartite, so alpha = n - nu and (Konig) min vertex cover = maximum matching - but
// this DP is shorter than building a matching, and it handles weights, which Konig does not.
pair<ll, ll> misTree(int v, int p, const vector<vector<int>>& adj, const vector<ll>& w) {
    ll take = w[v], skip = 0;
    for (int u : adj[v]) if (u != p) {
        auto [s, t] = misTree(u, v, adj, w);
        skip += max(s, t), take += s;
        // if v is taken, no child may be
    }
    return {skip, take};
        // answer = max(skip, take)
}
// MINIMUM VERTEX COVER (unweighted) = n - MIS. MAXIMUM MATCHING on a tree = the greedy "match
// every leaf to its parent, delete both, repeat" - and that greedy is optimal (exchange argument).

// --- TREE KNAPSACK, O(n^2) - and the one character that decides n^2 vs n^3. ---
// dp[v][j] = best value using exactly j vertices of v's subtree. Merge child by child, capping
// BOTH loops by the ALREADY ACCUMULATED size, not by n. Each unordered pair (a, b) is then
// charged exactly once, at their LCA, giving O(n^2) total. Writing `u <= n` instead of
// `u <= acc` is O(n^3) and looks identical.
vector<ll> knapTree(int v, int p, const vector<vector<int>>& adj, const vector<ll>& w) {
    vector<ll> dp = {0, w[v]};
        // dp[0] = take nothing at all
    for (int u : adj[v]) if (u != p) {
        vector<ll> c = knapTree(u, v, adj, w);
        // c[0] = skip this child entirely
        vector<ll> nd(dp.size() + c.size() - 1, LLONG_MIN);
        nd[0] = 0;
        for (size_t i = 1; i < dp.size(); i++)
            // i >= 1: v IS taken, so a child
            for (size_t j = 0; j < c.size(); j++)
                // may attach and stay connected
                if (dp[i] != LLONG_MIN && c[j] != LLONG_MIN) nd[i + j] = max(nd[i + j], dp[i] + c[j]);
        dp = nd;
    }
    return dp;
        // dp[j] = best CONNECTED set of j vertices containing v (dp[0] = 0)
}
// Dropping the `i >= 1` restriction gives the UNCONSTRAINED version: best j vertices anywhere in
// the subtree, no connectivity - that is the group-knapsack-on-a-tree variant.
// WITH A CAP k: allocate min(size, k) + 1 slots instead, giving O(n * k).

// --- DIAMETER / LONGEST PATH by DP, O(n). The version you reuse inside other tree DPs. ---
// down1[v], down2[v] = the two longest downward paths from v through DIFFERENT children.
// The answer is max over v of down1[v] + down2[v]; the two-BFS trick gives the same on an
// unweighted or POSITIVELY weighted tree, but only this DP survives negative weights.

/* MORE TREE-DP SHAPES, all O(n) unless noted
   COUNT SUBTREES / connected subgraphs containing v: prod over children of (1 + f(child)).
   COUNT MATCHINGS / independent sets: same [0/1] split, sum instead of max.
   SUM OF DISTANCES from every vertex: rerooting with (subtree size, subtree distance sum).
   K-COLOURINGS with adjacent-different: k * (k-1)^(n-1). With a colour list per vertex, DP
     dp[v][c] and take the product over children of (sum over c' != c of dp[u][c']) - keep the
     total per child and subtract the forbidden term to stay O(n * k).
   LONGEST PATH WITH AT MOST K EDGES / paths of length exactly L: dp[v][d] merged small-to-large
     over the depth arrays, or long-path decomposition for O(n).
   BINARY LIFTING ON A DP: when the transition is associative you can jump 2^k ancestors at once
     (max edge on a path, "the first ancestor with property P").
   TREE + BITMASK: dp[v][mask] for k <= 20 marked vertices (Steiner tree on a tree).
   REROOTING is the answer to "compute f(v) for EVERY root" - do not run the DP n times.
   AUXILIARY / VIRTUAL TREE (09 - Trees/08) collapses q marked vertices into an O(q) tree, so a
     per-query tree DP costs O(q log q) instead of O(n).
   DSU ON TREE (09 - Trees/02) is the answer when the per-subtree state is a MULTISET you cannot
     merge algebraically (most frequent colour, k-th value), O(n log n).
   SEGMENT TREE MERGING (02 - DS/16) does the same in O(n log n) while keeping the state
     queryable, at the cost of memory.
   RECURSION DEPTH: a path-shaped tree with n = 2e5 overflows the default stack once frames get
     fat. Move big locals out of the recursive function or convert to the iterative order used
     in 07 - Rerooting.cpp. */
```

9.11 Dynamic Trees

9.11.1 LinkCutTree

LINK-CUT TREE - a FOREST that changes shape, with path queries. O(log n) amortised per op.

```
// This is dynamic HLD: link, cut, re-root, and aggregate over the path u..v, all online.
// If the tree is STATIC use HLD (09 - Trees/03) - it is faster and far shorter. Reach for an LCT
// only when edges are added/removed, or when you need makeRoot.
// HOW IT WORKS: each vertex has at most one "preferred" child; preferred paths are stored as
// splay trees keyed by DEPTH; a non-preferred edge is a path-parent pointer the splay does not
```

```
// know about. access(v) makes root..v one preferred path, and the same heavy/light potential that
// bounds HLD bounds the number of preferred-child changes, hence the O(log n) amortised.
struct LCT {
    struct Node { int ch[2] = {0, 0}, p = 0; ll val = 0, sum = 0; bool rev = 0; };
    vector<Node> t;
    LCT(int n) : t(n + 1) {}          // 1-indexed; node 0 is the null
    bool isRoot(int x) { return t[t[x].p].ch[0] != x && t[t[x].p].ch[1] != x; }
    void pull(int x) { t[x].sum = t[t[x].ch[0]].sum ^ t[x].val ^ t[t[x].ch[1]].sum; }
    void flip(int x) { if (x) swap(t[x].ch[0], t[x].ch[1]), t[x].rev ^= 1; }
    void push(int x) { if (t[x].rev) flip(t[x].ch[0]), flip(t[x].ch[1]), t[x].rev = 0; }
    void rot(int x) {
        int p = t[x].p, g = t[p].p, d = t[p].ch[1] == x;
        if (!isRoot(p)) t[g].ch[t[g].ch[1] == p] = x;
        t[x].p = g;
        t[p].ch[d] = t[x].ch[!d], t[t[x].ch[!d]].p = p;
        t[x].ch[!d] = p, t[p].p = x;
        pull(p), pull(x);
    }
    void splay(int x) {
        vector<int> st{x};
        for (int y = x; !isRoot(y); y = t[y].p) st.push_back(t[y].p);
        for (int i = st.size() - 1; i >= 0; i--) push(st[i]);
        while (!isRoot(x)) {
            int p = t[x].p, g = t[p].p;
            if (!isRoot(p)) rot((t[g].ch[1] == p) == (t[p].ch[1] == x) ? p : x);
            rot(x);
        }
    }
    int access(int x) {
        int last = 0;          // root..x becomes one splay tree
        for (int y = x; y; y = t[y].p) splay(y), t[y].ch[1] = last, pull(y), last = y;
        splay(x);
        return last;           // = the LCA on the previous access
    }
    void makeRoot(int x) { access(x), flip(x), push(x); }
    int findRoot(int x) { access(x); while (t[x].ch[0]) push(x), x = t[x].ch[0]; splay(x); return x; }
    bool connected(int x, int y) { return findRoot(x) == findRoot(y); }
    void link(int x, int y) { makeRoot(x); if (findRoot(y) != x) t[x].p = y; }
    void cut(int x, int y) {
        makeRoot(x);           // no-op if the edge is absent
        if (findRoot(y) == x && t[y].p == x && !t[y].ch[0]) t[y].p = t[x].ch[1] = 0, pull(x);
    }
    ll query(int x, int y) { makeRoot(x), access(y); return t[y].sum; }    // path aggregate
    void update(int x, ll v) { access(x), t[x].val = v, pull(x); }
    int lca(int r, int x, int y) { makeRoot(r), access(x); return access(y); }
};
// THE AGGREGATE is XOR here purely because it is self-inverse and needs no reversal handling.
// For SUM/MIN/MAX just change pull(); for a NON-COMMUTATIVE aggregate (matrix product, max
// prefix sum) you must store BOTH directions in the node and swap them inside flip().
// EDGE WEIGHTS: give each edge its own vertex (n + edgeId) with the weight, and link both ends
// to it. Vertex values then stay 0 and every path query still works.
// SUBTREE aggregates need "virtual subtree" bookkeeping: keep a second sum over the non-preferred
// children, updated in access() when you swap t[y].ch[1]. That doubles the code - if the tree is
// static, an Euler tour (09 - Trees/06) is O(log n) and ten lines.
/* WHAT LCT UNLOCKS
   ONLINE DYNAMIC CONNECTIVITY ON A FOREST: connected() in O(log n). For general graphs (cycles)
     you need the offline segment-tree-on-time trick (02 - Data Structures/14) or Holm-de
     Lichtenberg-Thorup at O(log^2 n).
   DYNAMIC MST: to add edge (u,v,w), if u and v are connected find the MAXIMUM edge on the path;
     if it exceeds w, cut it and link the new edge. O(log n) per update.
   MAXIMUM FLOW (Dinic with link-cut) improves the blocking-flow step to O(VE log V) - almost
     never needed at contest sizes.
   OFFLINE "PAINT THE ROOT PATH": access() itself is the operation "recolour root..v with a new
     colour"; the number of colour changes is O(n log n) total, so if each change costs a segment
     tree update you get O(n log^2 n). This is the standard trick for "count distinct substrings
     over all substrings of s" and for tree-path-colouring problems. */
```

9.12 Long Path Decomposition

9.12.1 LongPathDecomposition

LONG PATH DECOMPOSITION - split the tree into vertical paths by always following the child with

```
// the DEEPEST subtree (HLD follows the child with the BIGGEST subtree instead). 0-indexed, rooted.
// TWO THINGS IT BUYS YOU THAT HLD DOES NOT.
//  1. O(1) K-TH ANCESTOR after O(n log n) preprocessing. Binary lifting is O(log n) per query;
//     this is the standard constant-time answer, and it is only ~30 lines.
//  2. DEPTH-INDEXED TREE DP IN O(n) TOTAL ("dp[v][d] = something about the vertices at depth d
//     below v"). Give each path head an array of its length and let every vertex on the path SHARE
//     that array with an offset - the heavy child inherits the parent's array for free, and only
//     the O(sqrt n)-ish path heads allocate. The classic use is "for each v, the most common depth
//     in its subtree" and "count vertices at
```

```
  ↪ distance k below v" for all v.
// The number of paths equals the number of LEAVES,
  ↪ and the crucial property is: the path containing
// v has length at least (height of v), so jumping to
  ↪ the head of v's path already covers a lot.
// K-TH ANCESTOR IN O(1). For every path head h of
  ↪ length L, store the L ancestors above h as well
// as the L vertices of the path itself ("up" and
  ↪ "down" tables of size 2L, total O(n)). To answer
// kth(v, k): let j = highest power of two <= k, jump
  ↪ v to its 2^j-th ancestor with a lifting table
// (one step), and the remaining k - 2^j < 2^j <=
  ↪ (height of the new vertex), so the rest of the
// jump lands inside the new vertex's own path table
  ↪ - one array lookup.
struct LongPath {
    int n, B;
    vector<int> par, dep, hgt, heavy, head;
    vector<vector<int>> jmp, tab;
  ↪          // tab[h] = ladder of head h
    vector<int> posInTab, headOf, logs;
    LongPath(int n, const vector<vector<int>>& adj,
  ↪ int root = 0)
        : n(n), B(1), par(n, -1), dep(n, 0), hgt(n,
  ↪ 0), heavy(n, -1), head(n, -1),
          posInTab(n, 0), headOf(n, -1), logs(n + 1,
  ↪ 0) {
        while ((1 << B) <= n) B++;
        for (int i = 2; i <= n; i++) logs[i] = logs[i
  ↪ / 2] + 1;
        vector<int> ord;
  ↪           // iterative DFS -> preorder
        vector<int> st{root};
        par[root] = -1;
        while (!st.empty()) {
            int u = st.back(); st.pop_back();
            ord.push_back(u);
            for (int v : adj[u]) if (v != par[u])
  ↪ par[v] = u, dep[v] = dep[u] + 1, st.push_back(v);
        }
        for (int i = n - 1; i >= 0; i--) {
  ↪           // heights, bottom up
            int u = ord[i];
            for (int v : adj[u]) if (v != par[u] &&
  ↪ hgt[v] + 1 > hgt[u])
                hgt[u] = hgt[v] + 1, heavy[u] = v;
        }
        jmp.assign(B, vector<int>(n, -1));
        for (int u = 0; u < n; u++) jmp[0][u] =
  ↪ par[u];
        for (int k = 1; k < B; k++) for (int u = 0; u
  ↪ < n; u++)
            jmp[k][u] = jmp[k - 1][u] < 0 ? -1 :
  ↪ jmp[k - 1][jmp[k - 1][u]];
        for (int u : ord) head[u] = (par[u] >= 0 &&
  ↪ heavy[par[u]] == u) ? head[par[u]] : u;
        tab.assign(n, {});
        for (int h = 0; h < n; h++) if (head[h] == h)
  ↪ {         // build the ladder of this path
            int L = hgt[h] + 1;
            vector<int> down;
            for (int v = h; v >= 0; v = heavy[v])
  ↪ down.push_back(v);
            vector<int> upv;
  ↪       // L ancestors above h, if they exist
            for (int x = par[h], c = 0; x >= 0 && c <
  ↪ L; x = par[x], c++) upv.push_back(x);
            reverse(all(upv));
            for (int v : down) upv.push_back(v);
  ↪           // [far ancestors ... h ... deepest]
            tab[h] = upv;
            int off = (int)tab[h].size() -
  ↪ (int)down.size();
            for (int i = 0; i < (int)down.size(); i++)
                posInTab[down[i]] = off + i,
  ↪ headOf[down[i]] = h;
        }
    }
    int kth(int v, int k) const {
  ↪           // k-th ancestor, -1 if too high
        if (k == 0) return v;
        if (k > dep[v]) return -1;
        int j = logs[k];
        v = jmp[j][v], k -= 1 << j;
  ↪           // one lifting step
        int h = headOf[v];
        return tab[h][posInTab[v] - k];
  ↪           // the rest is inside the ladder
    }
};
// WHY THE LADDER IS LONG ENOUGH: after the single
  ↪ 2^j jump, the remainder k' < 2^j and the vertex
// we are on has height >= 2^j - 1, so its path (and
  ↪ hence its ladder's upper half) covers k'.
// MEMORY is O(n log n) for the lifting table and
  ↪ O(n) for the ladders (each path of length L keeps
// 2L entries, and the path lengths sum to n).
// LEVEL ANCESTOR can also be done in O(n)
  ↪ preprocessing (Euler tour + the ladder trick
  ↪ without the
// lifting table), but the version above is the one
  ↪ you can write from memory.
// SMALL-TO-LARGE ALTERNATIVE for depth-indexed DP:
  ↪ merge maps/multisets child by child, O(n log^2)
// - shorter but slower; use it when the DP state is
  ↪ not a plain array indexed by depth.
```

9.13 Path Operations

9.13.1 PathIntersectionAndUnion

INTERSECTION AND UNION OF TWO TREE PATHS, O(log n) each. 0-indexed, rooted tree.

```
// Needs: Lift (09 - Trees/01 - Lowest Common
  ↪ Ancestor (LCA)/02 - BinaryLifting.cpp).
// A path is the pair of its endpoints {a, b}
  ↪ (unordered); {-1, -1} is the EMPTY path and
// {-2, -2} is "these two paths do not union into a
  ↪ path".
// Reach for it for "how many of these q paths pass
  ↪ through a common vertex", "the longest prefix
// of the queries whose paths all intersect" (the
  ↪ intersection is associative, so put it in a
// segment tree or binary-search it), "merge the
  ↪ constraints u..v must be inside one path", and
// road/route-overlap problems.
// KEY DECOMPOSITION: a path a..b splits at l =
  ↪ lca(a, b) into two VERTICAL chains l..a and l..b.
// The intersection of two paths is the union of the
  ↪ (at most four) pairwise chain intersections,
// and each of those is again a vertical chain that
  ↪ is trivial to compute from depths and lca.
struct PathOps {
    const Lift& L;
    typedef pair<int, int> P;
    PathOps(const Lift& L) : L(L) {}
    bool isAnc(int u, int a) const {
```

```
        // is a an ancestor of u (or == u)?
      return L.d[u] >= L.d[a] && L.kth(u, L.d[u] -
↪ L.d[a]) == a;
    }
    int len(P p) const { return p.first < 0 ? 0 :
↪ L.dist(p.first, p.second) + 1; }   // #vertices
    // The common part of the vertical chains u..a
↪ and v..c, where u is an ancestor of a and v of c.
    P chainMeet(int u, int a, int v, int c) const {
      if (!isAnc(a, v)) return {-1, -1};
↪         // v must lie on the chain u..a
      int x = L.lca(a, c);
      if (L.d[v] < L.d[u]) { if (isAnc(x, u))
↪ return {u, x}; }
      else if (isAnc(x, v)) return {v, x};
      return {-1, -1};
    }
    P joinChains(P x, P y) const {
↪         // union, ASSUMED to be a path
      if (x.first < 0) return y;
      if (y.first < 0) return x;
      int can[4] = {x.first, x.second, y.first,
↪ y.second}, a = can[0], b = -1;
      for (int u : can) if (L.d[u] > L.d[a]) a = u;
↪         // deepest endpoint
      for (int u : can) if (!isAnc(a, u) && (b < 0
↪ || L.d[b] < L.d[u])) b = u;
      if (b < 0) { b = can[0]; for (int u : can) if
↪ (L.d[u] < L.d[b]) b = u; }
      return {a, b};
    }
    P intersect(P p, P q) const {
↪         // {-1,-1} when they are disjoint
      if (p.first < 0 || q.first < 0) return {-1,
↪ -1};
      int u = L.lca(p.first, p.second), v =
↪ L.lca(q.first, q.second);
      P r1 = joinChains(chainMeet(u, p.first, v,
↪ q.first), chainMeet(u, p.first, v, q.second));
      P r2 = joinChains(chainMeet(u, p.second, v,
↪ q.first), chainMeet(u, p.second, v, q.second));
      return joinChains(r1, r2);
    }
    P unite(P x, P y) const {
↪         // {-2,-2} if the union is not a path
      if (x.first == -2 || y.first == -2) return
↪ {-2, -2};
      if (x.first < 0) return y;
      if (y.first < 0) return x;
      int e[4] = {x.first, x.second, y.first,
↪ y.second};
      int need = len(x) + len(y) - len(intersect(x,
↪ y));     // |union| by inclusion-exclusion
      for (int i = 0; i < 4; i++) for (int j = i; j
↪ < 4; j++) {
        P r{e[i], e[j]};
↪         // candidate covering path
        bool ok = len(r) == need;
↪         // no branch and no gap
        for (int w : e)
          if (L.dist(r.first, w) + L.dist(w,
↪ r.second) != L.dist(r.first, r.second)) ok = 0;
        if (ok) return r;
      }
      return {-2, -2};
    }
};
// BOTH OPERATIONS ARE ASSOCIATIVE, so they drop
↪ straight into a segment tree over an array of
// paths: "the intersection of a range of queries" or
↪ "the longest prefix whose union is a path"
// (descend the tree greedily, which is exactly the
↪ classic "sort a permutation along a tree" task).
// SMALLEST PATH CONTAINING two given paths is a
↪ THIRD operation, and it is not the union - see
// mergePaths in 01 - LCA/03 - LCAConstantTime, which
↪ returns the minimal COVERING path.
// DOES VERTEX w LIE ON PATH a..b? dist(a,w) +
↪ dist(w,b) == dist(a,b). That one-liner replaces
↪ this
// whole file when the query is that simple.
// PATH INTERSECTION LENGTH is len(intersect(p, q))
↪ in VERTICES; subtract one for edges.
```

10 | Game Theory

10.1 GrundyAndNim

SPRAGUE-GRUNDY. Every FINITE, ACYCLIC (progressively bounded) IMPARTIAL game under normal

```
/* SPRAGUE-GRUNDY.  Every FINITE, ACYCLIC
↪ (progressively bounded) IMPARTIAL game under
↪ normal
   play (last to move wins) is equivalent to a nim
↪ pile of size grundy(position).
   ACYCLIC IS A HYPOTHESIS, NOT A DETAIL: on a graph
↪ with cycles the mex recursion is not
   well-founded and grundy() below returns silently
↪ wrong values - use retrograde BFS instead.
   grundy(p) = mex{ grundy(q) : q reachable from p }.
   A position is LOSING for the player to move  iff
↪ grundy = 0.
   SUM of independent games: grundy = XOR of the
↪ parts.                                          */
int mex(const vector<int>& v) {
    vector<bool> seen(v.size() + 1, false);
    for (int x : v) if (x >= 0 && x <= (int)v.size())
↪ seen[x] = true;
    int m = 0;
    while (m < (int)seen.size() && seen[m]) m++;
    return m;
}
// Grundy over a DAG of at most n positions, memoised.
vector<int> g_;
int grundy(int u, const vector<vector<int>>& nxt) {
    if (g_[u] != -1) return g_[u];
    g_[u] = 0;
↪   // guard (DAG => no real cycles)
    vector<int> s;
    for (int v : nxt[u]) s.push_back(grundy(v, nxt));
    return g_[u] = mex(s);
}
/* NIM AND FRIENDS
   NIM (take any number from one pile). Losing iff
↪ XOR of pile sizes == 0.
     Winning move: find a pile with (x ^ X) < x where
↪ X is the total xor; reduce it to x ^ X.
   MISERE NIM (last to move LOSES). If every pile is
↪ size 1: first player wins iff #piles is EVEN.
     Otherwise identical to normal nim (win iff xor
↪ != 0).
   NIM_k / MOORE'S NIM (may take from up to k piles).
```

```
   ↪ Losing iff, for every bit, the number of
     piles with that bit set is divisible by (k+1).
    STAIRCASE NIM. INDEX 0 IS THE GROUND - coins there
   ↪ are DEAD and have no move.
     Losing iff XOR of the counts at ODD indices ==
   ↪ 0.  (If instead a coin on index 0 may still
     slide OFF the board, the ground is a virtual
   ↪ index -1 and you must XOR the EVEN indices.
     Getting this backwards is wrong on 96 of the 256
   ↪ four-position states.)
     Recognise it as "move something toward a sink,
   ↪ and moving it to an even slot is reversible".
    SUBTRACTION GAME, allowed set S: grundy is
   ↪ eventually PERIODIC. Compute a prefix, find the
     period, extrapolate - the standard route for n
   ↪ up to 1e9.
     S = {1..k}: grundy(n) = n mod (k+1).
    WYTHOFF (two piles, take from one or equally from
   ↪ both). Losing positions are
     (floor(k*phi), floor(k*phi^2)), phi =
   ↪ (1+sqrt5)/2, k >= 0.
    FIBONACCI NIM (first move any amount < pile; then
   ↪ at most 2x the previous move).
     First player loses iff the pile size is a
   ↪ Fibonacci number.
    TURNING / COIN GAMES: grundy of a set of coins =
   ↪ XOR of the grundy of each single coin.
    GREEN HACKENBUSH (all edges neutral): grundy of a
   ↪ path of length n is n; FUSION principle -
     a cycle of odd length fuses to a single edge, an
   ↪ even cycle to nothing; sum branches by XOR.
    COLON PRINCIPLE (trees): grundy of a node = XOR
   ↪ over children of (grundy(child) + 1).      */

// RETROGRADE ANALYSIS - when the game graph HAS
   ↪ CYCLES (draws possible) or the game is partisan,
// Grundy does not apply. BFS backwards from terminal
   ↪ positions counting out-degrees.
// win[u] = 1 first player to move wins, 0 loses, -1
   ↪ draw.
vector<int> retrograde(int n, const
   ↪ vector<vector<int>>& radj, vector<int> deg,
                       const vector<int>&
   ↪ terminalLose) {
    vector<int> res(n, -1);
    queue<int> q;
    for (int u : terminalLose) res[u] = 0, q.push(u);
    while (!q.empty()) {
        int v = q.front(); q.pop();
        for (int u : radj[v]) {
            if (res[u] != -1) continue;
            if (res[v] == 0) res[u] = 1, q.push(u);
   ↪     // can move to a losing position
            else if (--deg[u] == 0) res[u] = 0,
   ↪ q.push(u);   // all moves lead to wins
        }
    }
    return res;
   ↪     // remaining -1 are draws
}
```

10.2 GameAlgorithms

Concrete game solvers. All impartial games under NORMAL play (last move wins) unless stated.

```
// SUBTRACTION GAME with allowed move set S: grundy
   ↪ is eventually PERIODIC. Compute a prefix,
// detect the period, extrapolate to n up to 1e18.
   ↪ This is the standard route for huge n.
struct Periodic {
    vector<int> g;
    int start = -1, per = -1;
    Periodic(const vector<int>& S, int prefix = 2000)
   ↪ {
        g.assign(prefix, 0);
        for (int i = 1; i < prefix; i++) {
            vector<bool> seen(S.size() + 2, false);
            for (int s : S) if (s <= i && g[i - s] <
   ↪ (int)seen.size()) seen[g[i - s]] = true;
            while (g[i] < (int)seen.size() &&
   ↪ seen[g[i]]) g[i]++;
        }
        detect();
    }
    void detect() {
   ↪     // smallest (start, per) that fits
        int n = g.size();
        for (int p = 1; p <= n / 3; p++) {
            bool ok = true;
            for (int i = n / 3; i + p < n; i++) if
   ↪ (g[i] != g[i + p]) { ok = false; break; }
            if (!ok) continue;
            int st = 0;
   ↪     // the LAST break, not the first match:
            for (int i = n - p - 1; i >= 0; i--)
   ↪     // periodicity must hold from start ON
                if (g[i] != g[i + p]) { st = i + 1;
   ↪ break; }
            if (n - p - st < 2 * p + 20) continue;
   ↪     // demand real evidence before believing
            start = st, per = p;
            return;
        }
    }
    int at(ll n) const {
        if (n < (ll)g.size()) return g[n];
        assert(per > 0);
   ↪     // no period found: do NOT extrapolate
        return g[start + (n - start) % per];
    }
};
// S = {1..k}  =>  grundy(n) = n mod (k+1)  (no need
   ↪ to compute anything)

// WYTHOFF'S GAME (two piles; take any amount from
   ↪ one pile, or the SAME amount from both).
// Losing (P-)positions: (floor(k*phi),
   ↪ floor(k*phi^2)) for k >= 0, phi = (1+sqrt5)/2.
// Since phi^2 - phi = 1, the two coordinates differ
   ↪ by EXACTLY k, so k = b - a.
// Tested exactly, no floating point:  a ==
   ↪ floor(k*phi)  <=>  d <= k*sqrt5 < d+2, d = 2a-k.
bool wythoffLosing(ll a, ll b) {
    if (a > b) swap(a, b);
    ll k = b - a, d = 2 * a - k;
    return d >= 0 && (__int128)d * d <= (__int128)5 *
   ↪ k * k
                && (__int128)5 * k * k <
```

```
    ↪ (__int128)(d + 2) * (d + 2);
}
// P-positions: (0,0) (1,2) (3,5) (4,7) (6,10) (8,13)
    ↪ (9,15) (11,18) (12,20) (14,23) ...

// MOORE'S NIM_k: you may take from up to k piles in
    ↪ one move.
// Losing iff, for EVERY bit, the number of piles
    ↪ having that bit set is divisible by (k+1).
bool mooreLosing(const vector<ll>& piles, int k) {
    for (int b = 0; b < 63; b++) {
        int c = 0;
        for (ll x : piles) c += (x >> b) & 1;
        if (c % (k + 1)) return false;
    }
    return true;
}

// STAIRCASE NIM: coins on positions 0..n-1, a move
    ↪ slides some coins from i to i-1.
// INDEX 0 IS THE GROUND: coins there are DEAD and
    ↪ cannot move. Then only ODD indices matter
// (a move from an even index is reversible, so it is
    ↪ worth nothing).
// If your problem instead lets a coin at index 0
    ↪ slide OFF the board, the ground is a virtual
// index -1 and you must XOR the EVEN indices. Brute
    ↪ force over 4 positions: the wrong convention
// is wrong on 96 of 256 states, e.g. (0,0,0,1).
bool staircaseLosing(const vector<ll>& cnt) {
    ll x = 0;
    for (size_t i = 1; i < cnt.size(); i += 2) x ^=
    ↪ cnt[i];
    return x == 0;
}

// GREEN HACKENBUSH (all edges neutral), grundy of a
    ↪ rooted tree = COLON PRINCIPLE:
//   g(v) = XOR over children c of (g(c) + 1)
int hackenbushTree(int u, int p, const
    ↪ vector<vector<int>>& adj) {
    int r = 0;
    for (int v : adj[u]) if (v != p) r ^=
    ↪ hackenbushTree(v, u, adj) + 1;
    return r;
}
// FUSION PRINCIPLE for graphs: all vertices of a
    ↪ cycle can be fused into one; an ODD cycle
// leaves a single loop (= one edge, grundy 1), an
    ↪ EVEN cycle leaves nothing (grundy 0).

// COIN TURNING GAMES: a position is a set of
    ↪ "heads". The grundy value of the whole position
    ↪ is
// the XOR of the grundy values of each single head
    ↪ considered alone (Mock Turtles, Ruler, ...).

// MISERE NIM (last to move LOSES): if EVERY pile is
    ↪ 1, the first player wins iff #piles is even;
// otherwise the normal-play rule (xor != 0 => win)
    ↪ applies unchanged.
```

10.3 MinimaxAndMatching

Games that Sprague-Grundy does NOT cover: partisan games, scored games, draws.

```
// Needs: Blossom (03 - Graph Theory/07 - Matching/04
    ↪ - Blossom.cpp) for the matching part.

// MINIMAX with ALPHA-BETA pruning. Returns the value
    ↪ in PLAYER 0's units (a fixed sign), not
// "the value for the player to move" - with maxing =
    ↪ false the result is still player-0 scored.
// 'moves(state)' lists successors; 'eval(state)'
    ↪ scores terminal/leaf states from player 0's view.
template <class S, class Moves, class Eval>
ll alphabeta(const S& s, int depth, ll alpha, ll
    ↪ beta, bool maxing, Moves moves, Eval eval) {
    auto nxt = moves(s);
    if (!depth || nxt.empty()) return eval(s);
    if (maxing) {
        ll best = LLONG_MIN;
        for (auto& t : nxt) {
            best = max(best, alphabeta(t, depth - 1,
    ↪ alpha, beta, false, moves, eval));
            alpha = max(alpha, best);
            if (beta <= alpha) break;
    ↪       // beta cutoff
        }
        return best;
    }
    ll best = LLONG_MAX;
    for (auto& t : nxt) {
        best = min(best, alphabeta(t, depth - 1,
    ↪ alpha, beta, true, moves, eval));
        beta = min(beta, best);
        if (beta <= alpha) break;
    ↪       // alpha cutoff
    }
    return best;
}
// Order moves best-first: pruning is only good if
    ↪ you try strong moves early.
// Memoise on (state) when states repeat; use
    ↪ iterative deepening when depth is unbounded.

// MATCHING GAME ON A GRAPH: a token sits on vertex
    ↪ s; players alternately move it along an
// edge to an unvisited vertex; a player who cannot
    ↪ move loses.
// THEOREM: the FIRST player wins iff s is in EVERY
    ↪ maximum matching, i.e. iff removing s
// decreases the size of the maximum matching.
// This works on ANY graph, and you get EVERY start
    ↪ vertex at once for the price of one matching.
// After a maximum matching is fixed, the vertices
    ↪ left exposed by SOME maximum matching are
// exactly those reachable from an already-exposed
    ↪ vertex by an EVEN-length alternating path -
// which is precisely the set an unsuccessful blossom
    ↪ search paints as OUTER. So run the matching,
// then one failing search from each exposed vertex,
    ↪ and the unpainted vertices are the wins.
// O(V^3) for the whole table, versus O(V^4) for
    ↪ "recompute the matching without s" per vertex.
// Needs Blossom (03 - Graph Theory/07 - Matching/04
    ↪ - Blossom.cpp) - see the wrapper below.
vector<char> matchingGameWins(Blossom& B) {
    ↪       // B already has all edges added
    B.solve();
```

```
    vector<char> miss(B.n, 0);
    for (int v = 0; v < B.n; v++) if (B.match[v] < 0)
  ↪ {
        B.augment(v);
  ↪     // always fails; it is here for used[]
        for (int u = 0; u < B.n; u++) miss[u] |=
  ↪ B.used[u];
    }
    for (int v = 0; v < B.n; v++) miss[v] = !miss[v];
  ↪     // in every maximum matching -> P1 wins
    return miss;
}
// A vertex with NO edges is exposed in every
  ↪ matching, so the first player loses there -
  ↪ which is
// right, they cannot move. ISOLATED vertices
  ↪ therefore need no special case.
// GAMES WITH SCORES (not just win/lose): plain
  ↪ minimax DP over states,
//   dp[state] = best over moves of (gain +
  ↪ (-dp[next]))  for a zero-sum symmetric game,
//   or keep two arrays (dp0, dp1) when the players'
  ↪ objectives differ.
// GAMES WITH DRAWS / CYCLES: retrograde BFS (see 01
  ↪ - GrundyAndNim.cpp), never Grundy.
```

10.4 NamedGames

NAMED GAMES: the values you cannot re-derive during a contest

```
/* ====== NAMED GAMES: the values you cannot
  ↪ re-derive during a contest ======
 Two different families live here: TAKE-AND-BREAK
  ↪ (octal) games, and COIN-TURNING games.
 They have different theory - octal games are the
  ↪ splitting rule below, coin-turning games are
 XOR of single-head values and, in 2-D, nim
  ↪ MULTIPLICATION. Do not mix them up.
 Every value on this card was brute-force verified.
 All are impartial, NORMAL play. Grundy of a sum =
  ↪ XOR of parts.

 SPLITTING RULE (the general take-and-break move):
   g(v) = mex{ g(a1) XOR g(a2) XOR ... : v -> (a1,
  ↪ a2, ...) }

 LASKER'S NIM  (take any amount from a pile, OR split
  ↪ a pile of size >= 2 into two non-empty)
   g(4k+1) = 4k+1     g(4k+2) = 4k+2     g(4k+3) =
  ↪ 4k+4     g(4k+4) = 4k+3

 KAYLES  (remove 1 or 2 adjacent pins, possibly
  ↪ splitting the row)
   Periodic with period 12 from n = 71 onward
  ↪ (Winning Ways says 72; 71 is tight).
     block for n >= 71:  7 4 1 2 8 1 4 7 2 1 8 2
     14 exceptions (n, G):
  ↪ (0,0)(3,3)(6,3)(9,4)(11,6)(15,7)(18,3)(21,4)(22,6)(28,5)(34,6)
                          (39,3)(57,4)(70,6)
  ↪  last exception n = 70

 DAWSON'S CHESS  (octal game .137)
   Period 34 from n = 52 onward.
     block for n >= 52: 3 3 0 1 1 3 0 2 1 1 0 4 5 3 7
  ↪ 4 8 1 1 2 0 3 1 1 0 3 3 2 2 4 4 5 5 9
     7 exceptions (n, G):
  ↪ (0,0)(14,0)(16,2)(17,2)(31,2)(34,0)(51,2)
  ↪ last exception n = 51
 DAWSON'S KAYLES (.07)  period 34 from n = 53, same
  ↪ block, 8 exceptions, last at n = 52.

 GRUNDY'S GAME  (split a pile into two UNEQUAL
  ↪ non-empty piles)
   Periodicity is an OPEN PROBLEM - more than 2*10^10
  ↪ values computed and no period found.
   Do NOT assume a period here; compute the prefix
  ↪ you need.

 GUY-SMITH PERIODICITY THEOREM (how to PROVE a
  ↪ period, not just guess it)
   For an octal game whose largest non-zero code
  ↪ digit has index k: if
     G(n + p) = G(n) for all n with n0 <= n < 2*n0 +
  ↪ p + k
   then the period holds for ALL n >= n0. So checking
  ↪ up to 2*n0 + p + k is a proof.

 TURNING GAMES  (a position is a set of heads; the
  ↪ whole position's Grundy value is the XOR of
 the values of each single head considered alone)
   Twins       (turn exactly 2 coins), 0-indexed:
  ↪ g(x) = x                    [plain nim]
   Mock Turtles(turn 1, 2 or 3 coins), 0-indexed:
  ↪ g(x) = the x-th ODIOUS number
               (odious = odd popcount): 1, 2, 4, 7,
  ↪ 8, 11, 13, 14, 16, ...
               g(x) = 2x if 2x is odious, else 2x + 1
   Ruler       (turn any number of CONSECUTIVE
  ↪ coins), 1-indexed:
               g(x) = largest power of two dividing x
  ↪ = x & (-x)  -> 1,2,1,4,1,2,1,8,...
   TRAP: Ruler is 1-INDEXED while Twins and Mock
  ↪ Turtles are 0-INDEXED. Mixing them up gives a
   silently wrong answer.

 NIM WITH ONE PASS (Guy's model: the pass may be used
  ↪ once, at any NON-terminal position)
   1 heap:  G(0) = 0,  G(n) = n+1 for n odd,  G(n) =
  ↪ n-1 for n even >= 2.
   2 heaps: P-positions are exactly (0,0) and (a,
  ↪ a+1) with a ODD.
   3+ heaps: OPEN. No pattern; 34 P-positions with
  ↪ a<=b<=c<=13 and neither xor==0 nor xor==1
     comes close. Never guess.
   If the pass is ALSO legal at a terminal position
  ↪ (a weaker rule), the game collapses:
     G = (XOR of piles) XOR 1, so P-positions are
  ↪ exactly XOR == 1.
================================================================
  ↪ */
bool odious(ll x) { return __builtin_parityll(x); }
ll mockTurtles(ll x) { return odious(2 * x) ? 2 * x :
  ↪ 2 * x + 1; }   // 0-indexed
ll ruler(ll x) { return x & -x; }
  ↪                 // 1-indexed
ll lasker(ll n) {
  ↪ // g(0) = 0, NOT -1
    ll r = n % 4;
    if (r == 0) return n ? n - 1 : 0;
    return r == 3 ? n + 1 : n;
}
// Grundy of a sum of turning-game heads = XOR of the
  ↪ single-head values.
ll turningGameValue(const vector<ll>& heads, ll
  ↪ (*g)(ll)) {
    ll r = 0;
    for (ll h : heads) r ^= g(h);
```

```
    return r;
}
```

10.5 NimbersAndSums

NIMBERS, OCTAL GAMES, AND THE SUM VARIANTS.
Everything on this page was brute-force verified.

```
// --- NIM PRODUCT. [0, 2^(2^k)) is a FIELD under (XOR, nimMul). ---
// Definition: a (*) b = mex{ a'(*)b ^ a(*)b' ^ a'(*)b' : a' < a, b' < b }.
// Computed with the two identities: for a Fermat 2-power F = 2^(2^k),
//   x (*) F = x * F for x < F,   and   F (*) F = 3F/2.
// USES: Turning Corners (turn the 4 corners of a rectangle) has g(x,y) = x (*) y, and the TARTAN
// THEOREM says the 2-D product of two coin-turning games has g(x,y) = g1(x) (*) g2(y). Also
// CF 1310F, which asks you to solve a^(*)x = b in the nimber field of size 2^64.
typedef unsigned long long u64;      // nimbers are UNSIGNED, always
map<pair<u64, u64>, u64> nmMemo;
u64 nimMul(u64 a, u64 b) {
    if (a < b) swap(a, b);
    if (b == 0) return 0;
    if (b == 1) return a;
    auto it = nmMemo.find({a, b});
    if (it != nmMemo.end()) return it->second;
    int L = 0;
    while (L < 6 && (a >> (1u << L))) L++;
    u64 F = 1ULL << (1u << (L - 1));          // largest Fermat 2-power <= a
    u64 ah = a / F, al = a % F, bh = b / F, bl = b % F;
    u64 A = nimMul(ah, bh), B = nimMul(ah, bl), C = nimMul(al, bh), D = nimMul(al, bl);
    return nmMemo[{a, b}] = ((A ^ B ^ C) * F) ^ D ^ nimMul(A, F / 2);
}

// --- OCTAL (TAKE-AND-BREAK) GAME GRUNDY GENERATOR. ---
// The code is ".d1d2d3...": digit d_k governs "remove exactly k tokens from one heap".
//   bit 1 (value 1): you may take the WHOLE heap (i.e. leave zero heaps)
//   bit 2 (value 2): you may leave ONE non-empty heap
//   bit 4 (value 4): you may SPLIT the remainder into TWO non-empty heaps
// So .333... = Nim, .77 = Kayles, .07 = Dawson's Kayles, .137 = Dawson's chess.
// Being able to read a statement into a code string turns "invent a Grundy recurrence" into a
// function call. O(N^2 * digits) because of the split term; O(N * digits) without it.
vector<int> octal(const string& code, int N) {          // code = the digits AFTER the dot
    vector<int> d;
    for (char c : code) d.push_back(isdigit(c) ? c - '0' : c - 'a' + 10);
    vector<int> g(N + 1, 0);
    for (int n = 1; n <= N; n++) {
        set<int> s;
        for (int i = 0; i < (int)d.size(); i++) {
            int k = i + 1;
            if (k > n) break;
            int rem = n - k;
            if ((d[i] & 1) && rem == 0) s.insert(0);
            if ((d[i] & 2) && rem >= 1) s.insert(g[rem]);
            if (d[i] & 4) for (int a = 1; a < rem; a++) s.insert(g[a] ^ g[rem - a]);
        }
        int m = 0;
        while (s.count(m)) m++;
        g[n] = m;
    }
    return g;
}
// GUY-SMITH PERIODICITY THEOREM turns a guess into a PROOF: with k = the index of the last
// non-zero code digit, if G(n + p) = G(n) for all n0 <= n < 2*n0 + p + k, the period holds for
// ALL n >= n0. See 02 - GameAlgorithms.cpp for the (fixed) period detector.

// --- EUCLID'S GAME. (a, b); subtract a positive multiple of the smaller from the larger. ---
// FIRST PLAYER WINS iff a == b OR max/min > phi. The usual phrasing ">= phi" is WRONG on the
// whole diagonal: (a, a) is always an immediate win.
bool euclidWin(ll a, ll b) {
    if (a < b) swap(a, b);
    if (b == 0) return false;
    if (a == b || a >= 2 * b) return true;
    return !euclidWin(b, a - b);
}

// --- ANTI-SG / the SJ theorem: MISERE play on a SUM of impartial games. ---
// PRECONDITION, and it is not optional: in every component, SG(u) == 0 must imply u is TERMINAL.
// (Without it this rule is wrong on ~1.3% of random impartial games; with it, exact.)
// Misere nim satisfies it, which is why the classic misere-nim rule looks so clean.
bool antiSGFirstWins(const vector<int>& g) {
    int x = 0, mx = 0;
    for (int v : g) x ^= v, mx = max(mx, v);
    return (x && mx > 1) || (!x && mx <= 1);
}

// --- EVERY-SG: you must move in EVERY unfinished component. Not a function of the XOR at all. ---
// step(v) = 0                                                  if v is terminal
// step(v) = 1 + max{ step(u) : SG(u) == 0 }                    if SG(v) > 0   (drag it out - you will win it)
// step(v) = 1 + min{ step(u) }                                 if SG(v) == 0  (end it fast - you will lose it)
// First player wins iff max over components of step(root) is ODD.
// The mirrored max/min assignment is WRONG; this orientation is the verified one.
void everySGStep(const vector<vector<int>>& succ, const vector<int>& sg, vector<int>& step) {
    int n = succ.size();
    step.assign(n, 0);
    for (int u = 0; u < n; u++) {          // u in REVERSE topological order
        if (succ[u].empty()) { step[u] = 0; continue; }
        if (sg[u] > 0) {
```

```
            int b = 0;
            for (int v : succ[u]) if (!sg[v]) b =
  ↪ max(b, step[v] + 1);
            step[u] = b;
        } else {
            int b = INT_MAX;
            for (int v : succ[u]) b = min(b, step[v]
  ↪ + 1);
            step[u] = b;
        }
    }
}
/* ============ VERIFIED CLOSED FORMS AND FACTS
  ↪ ============
 COIN-TURNING FAMILY (a position is a set of heads;
  ↪ the whole position = XOR of single-head
 values). THE INDEXING IS LOAD-BEARING - mixing 0-
  ↪ and 1-indexed values is silently wrong.
   Twins           turn exactly 2          0-idx
  ↪ g(x) = x
   TurningTurtles  turn 1 or 2             1-idx
  ↪ g(x) = x
   MockTurtles     turn 1, 2 or 3          0-idx
  ↪ g(x) = 2x or 2x+1, whichever is ODIOUS
                                                (
  ↪ = the x-th odious number: 1 2 4 7 8 11 13 )
   Ruler           any CONSECUTIVE run     1-idx
  ↪ g(x) = x & -x
   Triplets        turn exactly 3          0-idx
  ↪ g(x) = 0 for x < 2, MockTurtles(x-2) otherwise
   Mogul           turn 1..5                     NO
  ↪ closed form (1 2 4 8 16 31 32 64 103 ...)
   TurningCorners  turn 4 rectangle corners
  ↪ g(x, y) = x (*) y     [nim product]
   TARTAN THEOREM: for the 2-D product of two
  ↪ coin-turning games, g(x,y) = g1(x) (*) g2(y).

 k-WYTHOFF (Fraenkel): take from one pile, or k1, k2
  ↪ > 0 from both with |k1 - k2| < k.
   P-positions: a_n = mex{a_i, b_i : i < n}, b_n =
  ↪ a_n + k*n.
   Closed form: (floor(n*alpha), floor(n*beta)) with
  ↪ alpha = (2 - k + sqrt(k*k + 4)) / 2,
   beta = alpha + k.  k = 1 is ordinary Wythoff.

 SILVER DOLLAR (coins at p_1 < ... < p_k on a strip,
  ↪ slide one left onto an empty square, no
   jumping): with gaps g_i = p_i - p_{i-1} - 1 and
  ↪ p_0 = -1, the GRUNDY VALUE is
   g_k XOR g_{k-2} XOR g_{k-4} XOR ...  - staircase
  ↪ nim on alternate gaps counted FROM THE RIGHT.

 CHOMP: every m x n board is a FIRST-PLAYER WIN
  ↪ except 1x1, by strategy stealing - and there is
   NO known constructive winning move. Two-row Chomp:
  ↪ P-positions are exactly (b+1, b).
   Strategy stealing is a pure EXISTENCE proof; if
  ↪ the problem wants the move, it is useless.

 BLUE-RED HACKENBUSH (partisan!): a string of edges
  ↪ from the ground has a NUMBER value, not a
   nimber. The first run of m equal colours
  ↪ contributes +-m (Blue positive), and every later
  ↪ edge
   contributes +-1/2, +-1/4, ... halving. This is the
  ↪ cleanest concrete model of surreal numbers.
   Partisan values in general: 0 = second player
  ↪ wins, * = {0|0} = first player wins,
   up = {0|*} is positive but smaller than every
  ↪ positive number, a SWITCH {a|b} with a > b is
   "hot" (both players want to move there), and the
  ↪ TEMPERATURE is what it costs the opponent to
   let you move first. SIMPLICITY RULE: if some
  ↪ number lies strictly between the Left and Right
   options, the game IS the simplest such number.
  ↪ NUMBER AVOIDANCE: never move in a number
   component while a non-number component exists.

 MISERE THEORY, honestly: misere nim's clean rule is
  ↪ an ACCIDENT. In general the misere outcome
   of a sum is NOT a function of the Grundy values;
  ↪ the correct invariant is the GENUS, and only
   "tame" games (genus matching nim's) obey nim-like
  ↪ rules. Anti-SG above is exactly the tame
   case, which is why it carries a precondition.

 COMPLEXITY LANDSCAPE - when to stop looking for a
  ↪ formula:
   UNDIRECTED vertex geography is in P (the
  ↪ maximum-matching theorem in 03 -
  ↪ MinimaxAndMatching).
   DIRECTED vertex geography, Node Kayles,
  ↪ generalized geography and generalized Hex are all
   PSPACE-complete; generalized chess and Go are
  ↪ EXPTIME-complete. So a 1e5-sized instance of
   directed geography MUST have extra structure -
  ↪ find it instead of searching.

 BOGUS NIM / REVERSIBLE MOVES: adding a move that the
  ↪ opponent can immediately undo does not
 change the Grundy value. That is precisely why the
  ↪ even indices in staircase nim are worth
 nothing, and it is the impartial case of the "bypass
  ↪ a reversible option" simplification. */
```

10.6 Hackenbush

BLUE-RED HACKENBUSH ON A TREE - the one PARTISAN game you can actually compute. The board is a

```
// tree rooted at the ground; every edge is Blue
  ↪ (only Left may delete it) or Red (only Right may
// delete it), and deleting an edge also deletes
  ↪ everything no longer connected to the root. The
// value of such a position is a DYADIC RATIONAL (a
  ↪ surreal NUMBER, never a nimber), and the usual
// number rules then decide everything: value > 0
  ↪ means Left wins whoever moves, value < 0 means
// Right wins whoever moves, value == 0 means the
  ↪ SECOND player wins.
// THE RECURSION. For a stalk (a path from the
  ↪ ground) the value is easy and worth remembering:
// the first run of m equal colours is worth +-m, and
  ↪ every edge after it is worth +-1/2, +-1/4,
// halving each time. For a tree, evaluate each
  ↪ subtree, then push its value v through the edge
// that joins it to its parent:  Blue edge -> (v + p)
  ↪ / 2^(p-1) with p the smallest positive
// integer making v + p > 1;  Red edge -> (v - p) /
  ↪ 2^(p-1) with p smallest making v - p < -1.
// Sum the results over the children. That is the
  ↪ "one edge above a game" simplification, and it
// is what makes the whole tree computable in
  ↪ near-linear time.
// WHEN: exactly the CodeChef GERALD08 shape - a tree
  ↪ of black/white edges, two players delete
// their own colour, print who wins moving first and
```

```
  ↪ who wins moving second. More generally, any
// partisan game whose components are trees of
  ↪ forced-order moves.
// COMPLEXITY: O(n log^2 n) with small-to-large
  ↪ merging of the fraction maps. The value needs
// Theta(n) bits in the worst case, which is why it
  ↪ is stored as an integer part plus a map of
// set fraction bits rather than as a long double - a
  ↪ double loses the answer at depth 53.
// DEGENERATE: a single vertex (no edges) has value
  ↪ 0, so the second player wins. All-Blue trees
// have value equal to the number of edges'
  ↪ contribution and are a Left win; the routine
  ↪ handles
// them with no special case. EXACT - all arithmetic
  ↪ is integer bit shuffling.
struct Surreal {
  ↪   // value = val + sum of frac bits, in [0,1)
    int val = 0, w = 0;
  ↪   // absolute bit position = key + w
    map<int, int> frac;
  ↪   // only positions < 0 survive normalisation
    void clear() { frac.clear(), val = w = 0; }
    void add(int x) {
  ↪   // set bit at key x, then carry upward
        frac[x]++;
        auto it = frac.find(x);
        while (it != frac.end() && it->se != 1) {
            if (it->se > 1) frac[it->fi + 1] +=
  ↪ it->se >> 1, it->se &= 1;
            auto d = it++;
            if (!d->se) frac.erase(d);
        }
    }
    void divide(int x) {
  ↪   // value /= 2^x
        for (int i = 0; i < x; i++) {
            if (!val) break;
            if (val & 1) add(i - w);
  ↪   // bit i of val lands at position i - x
            val >>= 1;
        }
        w -= x;
    }
    void operator+=(Surreal& o) {
        val += o.val;
        for (auto& [k, b] : o.frac) if (b) add(k +
  ↪ o.w - w);
        while (!frac.empty() && frac.rbegin()->fi + w
  ↪ >= 0) {           // promote carries into val
            if (frac.rbegin()->se) val += 1 <<
  ↪ (frac.rbegin()->fi + w);
            frac.erase(prev(frac.end()));
        }
    }
    void blueEdge() {
  ↪   // v -> (v + p) / 2^(p-1)
        int p = max(1, 1 - val);
        if (frac.empty() && p + val == 1) p++;
  ↪    // v + p > 1 is STRICT
        val += p, divide(p - 1);
    }
    void redEdge() {
  ↪   // v -> (v - p) / 2^(p-1)
        int p = max(1, 1 + val);
        if (val - p == -1) p++;
        val -= p, divide(p - 1);
    }
    int sign() { return val > 0 ? 1 : val < 0 ? -1 :
  ↪ !frac.empty(); }
};
// g[u] = {v, colour}, colour 0 = Blue (Left / first
  ↪ player's edges), 1 = Red. Tree rooted at 0.
// Returns the sign of the value: +1 Left wins
  ↪ always, -1 Right wins always, 0 second player
  ↪ wins.
int hackenbush(int n, const vector<vector<pii>>& g) {
    vector<Surreal> a(n);
    vector<int> id(n), vis(n, 0);
    iota(all(id), 0);
    function<void(int)> dfs = [&](int u) {
        vis[u] = 1;
        for (auto& [v, c] : g[u]) {
            if (vis[v]) continue;
            dfs(v);
            c ? a[id[v]].redEdge() :
  ↪ a[id[v]].blueEdge();
            if (sz(a[id[v]].frac) >
  ↪ sz(a[id[u]].frac)) swap(id[v], id[u]);   //
  ↪ small to large
            a[id[u]] += a[id[v]], a[id[v]].clear();
        }
    };
    dfs(0);
    return a[id[0]].sign();
}
// READING THE ANSWER: sign > 0 -> Left wins moving
  ↪ first AND second; sign < 0 -> Right wins both;
// sign == 0 -> whoever moves second wins. There is
  ↪ no "first player wins whoever they are" case,
// because a Hackenbush tree value is always a number
  ↪ and numbers are never fuzzy with 0.
// GREEN HACKENBUSH (every edge deletable by both) is
  ↪ impartial and is a NIMBER instead: the
// colon principle, grundy(u) = XOR over children of
  ↪ (grundy(child) + 1) - see 02 - GameAlgorithms.
// A tree with edges of all three colours has no
  ↪ closed form; you are back to computing surreal
// values of general games, which is exponential.
// THE PARTISAN TOOLKIT this file is the concrete
  ↪ instance of (05 - NimbersAndSums has the rest):
// 0 = second player wins, * = {0|0} = first player
  ↪ wins, up = {0|*} beats every 0 but loses to
// every positive number, a SWITCH {a|b} with a > b
  ↪ is hot, TEMPERATURE measures how much the move
// is worth, SIMPLICITY RULE picks the simplest
  ↪ number strictly between the Left and Right
  ↪ options,
// and NUMBER AVOIDANCE says never move in a number
  ↪ component while a non-number one exists.
```

11 | Geometry

11.1 Geometry 2D

11.1.1 Core

Point / vector core. P<ll> = exact (use for all predicates), P<ld> = real answers.

```
// cross(a,b) > 0  <=>  b is counter-clockwise from a.
// a.cross(b,c) > 0 <=> c lies strictly LEFT of the
  ↪ directed line a->b.   [= orient(a,b,c)]
int sgn(ll x) { return (x > 0) - (x < 0); }
int sgn(ld x) { return (x > eps) - (x < -eps); }
int sgn(int x) { return (x > 0) - (x < 0); }
  ↪ // without these two overloads a bare int or
int sgn(double x) { return sgn((ld)x); }
  ↪ // double argument is AMBIGUOUS: hard error
ll gcdll(ll a, ll b) { return b ? gcdll(b, a % b) :
  ↪ a; }   // std::__gcd is libstdc++-only

template <class T>
struct P {
    typedef P Pt; T x = 0, y = 0;
    P(T x = 0, T y = 0) : x(x), y(y) {}
    Pt operator+(Pt p) const { return {x + p.x, y +
  ↪ p.y}; }
    Pt operator-(Pt p) const { return {x - p.x, y -
  ↪ p.y}; }
    Pt operator*(T d) const { return {x * d, y * d}; }
    Pt operator/(T d) const { return {x / d, y / d}; }
    T dot(Pt p) const { return x * p.x + y * p.y; }
    T cross(Pt p) const { return x * p.y - y * p.x; }
    T cross(Pt a, Pt b) const { return (a -
  ↪ *this).cross(b - *this); }  // orient
    T dist2() const { return x * x + y * y; }
    ld dist() const { return sqrtl((ld)dist2()); }
    ld angle() const { return atan2l(y, x); }
  ↪    // (-pi, pi]
    Pt perp() const { return {-y, x}; }
  ↪    // rotate +90 deg
    Pt unit() const { return *this / dist(); }
    Pt rot(ld a) const { return {x * cosl(a) - y *
  ↪ sinl(a), x * sinl(a) + y * cosl(a)}; }
    bool operator<(Pt p) const { return tie(x, y) <
  ↪ tie(p.x, p.y); }
    bool operator==(Pt p) const { return tie(x, y) ==
  ↪ tie(p.x, p.y); }
    friend istream& operator>>(istream& i, Pt& p) {
  ↪ return i >> p.x >> p.y; }
    friend ostream& operator<<(ostream& o, Pt p) {
  ↪ return o << p.x << ' ' << p.y; }
};
typedef P<ll> Pi;
typedef P<ld> Pd;

// Signed angle from a to b in (-pi, pi]. Sign tells
  ↪ you the rotation direction.
template <class T> ld angleTo(P<T> a, P<T> b) {
  ↪ return atan2l(a.cross(b), a.dot(b)); }
// Angle a-O-b in [0, 2pi).
template <class T> ld angleAt(P<T> a, P<T> O, P<T> b)
  ↪ {
    ld r = angleTo(a - O, b - O);
    return r < 0 ? r + 2 * PI : r;
}

// EXACT polar sort, no atan2. Orders by angle in [0,
  ↪ 2pi) starting at +x axis.
// sort(all(v), angCmp<ll>);   ties (same direction)
  ↪ keep input order.
template <class T> bool half(P<T> p) { return p.y < 0
  ↪ || (p.y == 0 && p.x < 0); }
template <class T> bool angCmp(P<T> a, P<T> b) {
    return half(a) != half(b) ? half(a) < half(b) :
  ↪ a.cross(b) > 0;
}

// OVERFLOW: |coord| <= 1e9  =>  cross fits in ll
  ↪ (max ~4e18, ll max 9.2e18).
//           dist2 of a difference fits too. Anything
  ↪ squared twice needs __int128.
```

11.1.2 Lines

Lines, segments, rays. "line ab" = infinite line through a and b.

```
// Predicates are exact on P<ll>; anything returning
  ↪ a point needs P<ld>.

template <class P> ld lineDist(P p, P a, P b) {
  ↪    // distance point -> line
    return fabsl((ld)(b - a).cross(p - a)) / (b -
  ↪ a).dist();
}
template <class P> P proj(P p, P a, P b) {
  ↪    // foot of perpendicular
    return a + (b - a) * ((ld)(p - a).dot(b - a) / (b
  ↪ - a).dist2());
}
template <class P> P refl(P p, P a, P b) { return
  ↪ proj(p, a, b) * 2 - p; }

template <class P> bool onSeg(P p, P a, P b) {
  ↪    // inclusive of endpoints
    return a.cross(b, p) == 0 && (a - p).dot(b - p)
  ↪ <= 0;
}
template <class P> ld segDist(P p, P a, P b) {
  ↪    // distance point -> segment (exact-safe)
    if (a == b) return (p - a).dist();
    ld d = (ld)(b - a).dist2(), t = (ld)(p - a).dot(b
  ↪ - a);
    if (t <= 0) return (p - a).dist();
    if (t >= d) return (p - b).dist();
    return fabsl((ld)(b - a).cross(p - a)) / sqrtl(d);
}
template <class P> P closestOnSeg(P p, P a, P b) {
    if ((p - a).dot(b - a) <= 0) return a;
    if ((p - b).dot(b - a) >= 0) return b;
    return proj(p, a, b);
}

// LINE x LINE. first = 1 unique, 0
  ↪ parallel-disjoint, -1 same line.
template <class P> pair<int, P> lineInter(P a, P b, P
  ↪ c, P d) {
    auto x = (b - a).cross(d - c);
    if (sgn(x) == 0) return {-(a.cross(b, c) == 0),
  ↪ P()};
    auto p = c.cross(b, d), q = c.cross(d, a);
  ↪     // p + q == x  (affine weights)
    return {1, (a * p + b * q) / x};
  ↪     // P must be floating
}

// SEGMENT x SEGMENT. Returns 0, 1, or 2 points (2 =
  ↪ collinear overlap endpoints).
```

```
// Exact on P<ll> when the answer has 0 or 2 points.
  ↪ The 1-point case DIVIDES and the numerator
// is ~2*C^3, so it is only exact for |coordinate| <=
  ↪ ~1.6e6. Above that use segCross for the
// boolean, or Frac (01 - Miscellaneous/13) for an
  ↪ exact rational point.
template <class P> vector<P> segInter(P a, P b, P c,
  ↪ P d) {
    auto oa = c.cross(d, a), ob = c.cross(d, b), oc =
  ↪ a.cross(b, c), od = a.cross(b, d);
    if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) <
  ↪ 0)
        return {(a * ob - b * oa) / (ob - oa)};
    set<P> s;
    if (onSeg(c, a, b)) s.insert(c);
    if (onSeg(d, a, b)) s.insert(d);
    if (onSeg(a, c, d)) s.insert(a);
    if (onSeg(b, c, d)) s.insert(b);
    return {all(s)};
}
// Just "do they touch?" - fully exact, NO division
  ↪ and no allocation, so it is both safe at
// |c| = 1e9 and ~30x faster than asking segInter.
  ↪ Use this whenever you only need a bool.
template <class P> bool segCross(P a, P b, P c, P d) {
    auto oa = c.cross(d, a), ob = c.cross(d, b), oc =
  ↪ a.cross(b, c), od = a.cross(b, d);
    if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) <
  ↪ 0) return true;
    return onSeg(c, a, b) || onSeg(d, a, b) ||
  ↪ onSeg(a, c, d) || onSeg(b, c, d);
}

template <class P> ld segSegDist(P a, P b, P c, P d) {
    if (segCross(a, b, c, d)) return 0;
    return min({segDist(a, c, d), segDist(b, c, d),
  ↪ segDist(c, a, b), segDist(d, a, b)});
}

// RAY from s through e.
template <class P> bool onRay(P p, P s, P e) {
    return s.cross(e, p) == 0 && (p - s).dot(e - s)
  ↪ >= 0;
}
template <class P> ld rayDist(P p, P s, P e) {
    return (p - s).dot(e - s) <= 0 ? (p - s).dist() :
  ↪ lineDist(p, s, e);
}

// Perpendicular bisector of ab, returned as two
  ↪ points on it.
// FLOATING P ONLY: (a+b)/2 truncates on P<ll> and
  ↪ returns a DIFFERENT line (for a=(0,0),
// b=(1,1) it gives y = -x instead of y = -x + 1).
  ↪ For an exact integer version work on the
// doubled lattice: the line through (a+b) and (a+b)
  ↪ + (b-a).perp().
template <class P> pair<P, P> perpBisector(P a, P b) {
    P m = (a + b) / 2;
    return {m, m + (b - a).perp()};
}
// Interior angle bisector direction at B for angle
  ↪ A-B-C (unit vectors summed).
template <class P> P bisectorDir(P a, P b, P c) {
  ↪ return (a - b).unit() + (c - b).unit(); }

// Line as a*x + b*y = c  ->  two points on it (needs
  ↪ floating P).
template <class P> pair<P, P> fromABC(ld A, ld B, ld
  ↪ C) {
    P n(A, B), p = n * (C / n.dist2());
    return {p, p + n.perp()};
}
```

11.1.3 Polygon

Polygons. Vertices in order (CW or CCW); convex routines want CCW, no collinear points.

```
template <class P> auto area2(const vector<P>& v) {
  ↪     // SIGNED 2*area. >0 <=> CCW.
    decltype(v[0].cross(v[0])) a = 0;
  ↪     // exact on P<ll>; safe on empty
    for (int i = 0, n = v.size(); i < n; i++) a +=
  ↪ v[i].cross(v[(i + 1) % n]);
    return a;
}
template <class P> ld area(const vector<P>& v) {
  ↪ return fabsl((ld)area2(v)) / 2; }
template <class P> ld perimeter(const vector<P>& v) {
    ld r = 0;
    for (int i = 0, n = v.size(); i < n; i++) r +=
  ↪ (v[i] - v[(i + 1) % n]).dist();
    return r;
}
template <class P> P centroid(const vector<P>& v) {
  ↪     // area centroid (needs floating P)
    P r; ld A = 0;
    for (int i = 0, j = v.size() - 1, n = v.size(); i
  ↪ < n; j = i++) {
        auto c = v[j].cross(v[i]);
        r = r + (v[i] + v[j]) * c; A += c;
    }
    return r / A / 3;
}
// A completely collinear (zero-area) polygon reports
  ↪ TRUE here - guard with area2 != 0 if that
// matters. Bow-ties and spikes are correctly
  ↪ rejected.
template <class P> bool isConvex(const vector<P>& v) {
    int n = v.size(), pos = 0, neg = 0;
    for (int i = 0; i < n; i++) {
        int s = sgn(v[i].cross(v[(i + 1) % n], v[(i +
  ↪ 2) % n]));
        s > 0 ? pos++ : s < 0 ? neg++ : 0;
    }
    return !pos || !neg;
}

// Point in ANY simple polygon, O(n), exact.
  ↪ strict=true -> boundary counts as outside.
template <class P> bool inPoly(const vector<P>& v, P
  ↪ a, bool strict = true) {
    int cnt = 0, n = v.size();
    for (int i = 0; i < n; i++) {
        P q = v[(i + 1) % n];
        if (onSeg(a, v[i], q)) return !strict;
        cnt ^= ((a.y < v[i].y) - (a.y < q.y)) *
  ↪ a.cross(v[i], q) > 0;
    }
    return cnt;
}

// Convex hull, CCW, strictly convex (collinear
  ↪ points dropped). O(n log n).
// Change <= 0 to < 0 in the pop test to KEEP
  ↪ collinear points on the hull.
```

```
template <class P> vector<P> hull(vector<P> p) {
    if (p.size() <= 1) return p;
    sort(all(p));
    vector<P> h(p.size() + 1);
    int s = 0, t = 0;
    for (int it = 2; it--; s = --t, reverse(all(p)))
        for (P q : p) {
            while (t >= s + 2 && h[t - 2].cross(h[t -
  ↪ 1], q) <= 0) t--;
            h[t++] = q;
        }
    return {h.begin(), h.begin() + t - (t == 2 &&
  ↪ h[0] == h[1])};
}

// Point in CONVEX polygon (CCW, strict), O(log n). 1
  ↪ inside, 0 on boundary, -1 outside.
// h must be a strictly convex CCW hull with n >= 1.
  ↪ Returns 1 inside, 0 on the boundary,
// -1 outside.
template <class P> int inConvex(const vector<P>& h, P
  ↪ p) {
    if (h.empty()) return -1;
    int n = h.size();
    if (n == 1) return p == h[0] ? 0 : -1;
    if (n == 2) return onSeg(p, h[0], h[1]) ? 0 : -1;
    if (h[0].cross(h[1], p) < 0 || h[0].cross(h[n -
  ↪ 1], p) > 0) return -1;
    int l = 1, r = n - 1;
    while (r - l > 1) { int m = (l + r) / 2;
  ↪ (h[0].cross(h[m], p) >= 0 ? l : r) = m; }
    int s = sgn(h[l].cross(h[l + 1], p));
    if (s < 0) return -1;
    if (s == 0) return 0;
    return (h[0].cross(h[1], p) == 0 ||
  ↪ h[0].cross(h[n - 1], p) == 0) ? 0 : 1;
}

// Index of the hull vertex furthest in direction
  ↪ `dir`, O(log n). CCW hull, no collinear.
// Use to get tangents from a far-away direction, or
  ↪ max/min dot product.
#define _cmp(i, j) sgn(dir.perp().cross(h[(i) % n] -
  ↪ h[(j) % n]))
#define _ext(i) (_cmp(i + 1, i) >= 0 && _cmp(i, i - 1
  ↪ + n) < 0)
template <class P> int extreme(const vector<P>& h, P
  ↪ dir) {
    int n = h.size(), lo = 0, hi = n;
    if (_ext(0)) return 0;
    while (lo + 1 < hi) {
        int m = (lo + hi) / 2;
        if (_ext(m)) return m;
        int ls = _cmp(lo + 1, lo), ms = _cmp(m + 1,
  ↪ m);
        (ls < ms || (ls == ms && ls == _cmp(lo, m)) ?
  ↪ hi : lo) = m;
    }
    return lo;
}

// Cut polygon by the line s->e, KEEP the part on the
  ↪ LEFT (boundary included). O(n).
// Needs floating P. Boundary-inclusive matters: with
  ↪ a strict test, vertices lying exactly on
// the line are dropped from BOTH halves and you lose
  ↪ area. Repeat over many lines = half-plane
// intersection. cut(v,e,s) gives the other half; the
  ↪ two areas sum to area(v).
// Keeps the part LEFT of the directed line s->e,
  ↪ boundary INCLUSIVE. Vertices exactly on the
// line are kept, so a line through a vertex produces
  ↪ a DUPLICATE point - fine for area, but
// dedup before feeding the result to hull / isConvex
  ↪ / minkowski / diameter2.
template <class P> vector<P> cut(const vector<P>& v,
  ↪ P s, P e) {
    vector<P> r;
    for (int i = 0, n = v.size(); i < n; i++) {
        P cur = v[i], prv = v[(i + n - 1) % n];
        bool side = sgn(s.cross(e, cur)) >= 0;
        if (side != (sgn(s.cross(e, prv)) >= 0))
  ↪ r.push_back(lineInter(s, e, cur, prv).second);
        if (side) r.push_back(cur);
    }
    return r;
}

// Minkowski sum of two CONVEX CCW polygons. O(n+m).
  ↪ Result is convex CCW.
// BOTH polygons must be CONVEX and
  ↪ COUNTER-CLOCKWISE. On clockwise input the merge
  ↪ below makes
// no progress and loops forever (an unexplained
  ↪ TLE/MLE, not a wrong answer) - so normalise.
template <class P> vector<P> minkowski(vector<P> A,
  ↪ vector<P> B) {
    if (area2(A) < 0) reverse(all(A));
    if (area2(B) < 0) reverse(all(B));
    auto low = [](vector<P>& v) {
        auto lo = [](P a, P b) { return tie(a.y, a.x)
  ↪ < tie(b.y, b.x); };
        rotate(v.begin(), min_element(all(v), lo),
  ↪ v.end());
    };
    low(A); low(B);
    A.push_back(A[0]); A.push_back(A[1]);
    B.push_back(B[0]); B.push_back(B[1]);
    vector<P> r; size_t i = 0, j = 0;
    while (i + 2 < A.size() || j + 2 < B.size()) {
        r.push_back(A[i] + B[j]);
        auto c = (A[i + 1] - A[i]).cross(B[j + 1] -
  ↪ B[j]);
        if (c >= 0 && i + 2 < A.size()) i++;
        if (c <= 0 && j + 2 < B.size()) j++;
    }
    return r;
}

// Rotating calipers. Hull must be CCW strictly
  ↪ convex.
template <class P> auto diameter2(const vector<P>& h)
  ↪ {   // max squared distance
    int n = h.size(), j = n < 2 ? 0 : 1;
    decltype(h[0].dist2()) r = 0;
    for (int i = 0; i < j; i++)
        for (;; j = (j + 1) % n) {
            r = max(r, (h[i] - h[j]).dist2());
            if ((h[(j + 1) % n] - h[j]).cross(h[i +
  ↪ 1] - h[i]) >= 0) break;
        }
    return r;
}
template <class P> ld width(const vector<P>& h) {
  ↪      // min distance between parallel supports
    int n = h.size(); if (n <= 2) return 0;
```

```
    ld r = 1e18; int j = 1;
    for (int i = 0; i < n; i++) {
        while ((h[(i + 1) % n] - h[i]).cross(h[(j +
  ↪ 1) % n] - h[j]) >= 0) j = (j + 1) % n;
        r = min(r, lineDist(h[j], h[i], h[(i + 1) %
  ↪ n]));
    }
    return r;
}
// Minimum-area enclosing rectangle. One side always
  ↪ lies on a hull edge (that's the theorem),
// so try every edge. O(n^2) on the HULL only - fine,
  ↪ and immune to caliper tie bugs.
template <class P> ld minRectArea(const vector<P>& h)
  ↪ {
    int n = h.size(); if (n < 3) return 0;
    ld best = 1e18;
    for (int i = 0; i < n; i++) {
        P d = h[(i + 1) % n] - h[i];
        ld L = d.dist(), lo = 1e18, hi = -1e18, ht =
  ↪ 0;
        for (P q : h) {
            ld u = (ld)d.dot(q - h[i]) / L, v =
  ↪ (ld)d.cross(q - h[i]) / L;
            lo = min(lo, u); hi = max(hi, u); ht =
  ↪ max(ht, fabsl(v));
        }
        best = min(best, (hi - lo) * ht);
    }
    return best;
}

// PICK'S THEOREM. Lattice polygon: A = I + B/2 - 1.
template <class P> ll boundaryPts(const vector<P>& v)
  ↪ {    // B
    ll b = 0;
    for (int i = 0, n = v.size(); i < n; i++) {
        P d = v[(i + 1) % n] - v[i];
        b += gcdll(llabs(d.x), llabs(d.y));
    }
    return b;
}
// PICK'S THEOREM, and it needs a SIMPLE polygon with
  ↪ integer vertices. On a self-intersecting
// or degenerate (zero-area) polygon the result is
  ↪ meaningless, not merely approximate.
template <class P> ll interiorPts(const vector<P>& v)
  ↪ {    // I = A - B/2 + 1
    return (llabs(area2(v)) - boundaryPts(v)) / 2 + 1;
}
```

11.1.4 Circles

Circles. All of these want a floating P (Pd).

```
// --- circle x line: 0, 1 (tangent) or 2 points on
  ↪ segment/line ab ---
template <class P> vector<P> circleLine(P o, ld r, P
  ↪ a, P b) {
    P p = proj(o, a, b);
    ld h2 = r * r - (p - o).dist2();
    if (h2 < -eps) return {};
    if (h2 < eps) return {p};
    P h = (b - a).unit() * sqrtl(h2);
    return {p - h, p + h};
}
template <class P> vector<P> circleSeg(P o, ld r, P
  ↪ a, P b) {
    vector<P> out;
    for (P p : circleLine(o, r, a, b)) if (segDist(p,
  ↪ a, b) < eps) out.push_back(p);
    return out;
}

// --- circle x circle: the 0/1/2 intersection points
  ↪ ---
template <class P> vector<P> circleCircle(P a, ld r1,
  ↪ P b, ld r2) {
    P d = b - a; ld d2 = d.dist2();
    if (d2 < eps) return {};
  ↪     // concentric (equal => infinite)
    ld p = (d2 + r1 * r1 - r2 * r2) / (2 * d2), h2 =
  ↪ r1 * r1 - p * p * d2;
    if (r1 + r2 < sqrtl(d2) - eps || fabsl(r1 - r2) >
  ↪ sqrtl(d2) + eps) return {};
    P mid = a + d * p, per = d.perp() *
  ↪ sqrtl(max((ld)0, h2) / d2);
    if (h2 < eps) return {mid};
    return {mid + per, mid - per};
}

// --- AREA of the lens (intersection region) of two
  ↪ circles ---
template <class P> ld circleInterArea(P a, ld r1, P
  ↪ b, ld r2) {
    ld d = (a - b).dist();
    if (d >= r1 + r2) return 0;
    if (d + min(r1, r2) <= max(r1, r2)) return PI *
  ↪ min(r1, r2) * min(r1, r2);
    ld A = acosl((d * d + r1 * r1 - r2 * r2) / (2 * d
  ↪ * r1));
    ld B = acosl((d * d + r2 * r2 - r1 * r1) / (2 * d
  ↪ * r2));
    return r1 * r1 * (A - sinl(2 * A) / 2) + r2 * r2
  ↪ * (B - sinl(2 * B) / 2);
}

// --- TANGENTS. Each result is a pair (touch point
  ↪ on c1, touch point on c2).
// inner=false -> the 2 external tangents; inner=true
  ↪ -> the 2 internal (crossing) ones.
// Tangent lines from a POINT p to circle (c,r):
  ↪ tangents(p, 0, c, r, false).
template <class P> vector<pair<P, P>> tangents(P c1,
  ↪ ld r1, P c2, ld r2, bool inner) {
    if (inner) r2 = -r2;
    P d = c2 - c1;
    ld dr = r1 - r2, d2 = d.dist2(), h2 = d2 - dr *
  ↪ dr;
    if (d2 < eps || h2 < -eps) return {};
    vector<pair<P, P>> out;
    for (ld s : {-1.0L, 1.0L}) {
        P v = (d * dr + d.perp() * sqrtl(max((ld)0,
  ↪ h2)) * s) / d2;
        out.push_back({c1 + v * r1, c2 + v * r2});
    }
    if (h2 < eps) out.pop_back();
    return out;
}

// --- circumcircle / incircle of a triangle ---
template <class P> P circumcenter(P a, P b, P c) {
    P B = c - a, C = b - a;
    return a + (B * C.dist2() - C * B.dist2()).perp()
  ↪ / B.cross(C) / 2;
}
template <class P> ld circumradius(P a, P b, P c) {
```

```
    return (b - a).dist() * (c - b).dist() * (a -
  ↪ c).dist() / (2 * fabsl((ld)(b - a).cross(c -
  ↪ a)));
}
template <class P> P incenter(P a, P b, P c) {
    ld A = (b - c).dist(), B = (c - a).dist(), C = (a
  ↪ - b).dist();
    return (a * A + b * B + c * C) / (A + B + C);
}
template <class P> ld inradius(P a, P b, P c) {
    ld s = ((b - a).dist() + (c - b).dist() + (a -
  ↪ c).dist()) / 2;
    return fabsl((ld)(b - a).cross(c - a)) / 2 / s;
}
template <class P> P orthocenter(P a, P b, P c) {
  ↪ return a + b + c - circumcenter(a, b, c) * 2; }

// --- MINIMUM ENCLOSING CIRCLE, expected O(n)
  ↪ (Welzl, incremental) ---
template <class P> pair<P, ld> mec(vector<P> p) {
    if (p.empty()) return {P(), 0};
    shuffle(all(p),
  ↪ mt19937(chrono::steady_clock::now().time_since_epoch().count()));
    P o = p[0]; ld r = 0, E = 1 + 1e-9;
    for (int i = 0; i < (int)p.size(); i++) if ((o -
  ↪ p[i]).dist() > r * E) {
        o = p[i], r = 0;
        for (int j = 0; j < i; j++) if ((o -
  ↪ p[j]).dist() > r * E) {
            o = (p[i] + p[j]) / 2, r = (o -
  ↪ p[i]).dist();
            for (int k = 0; k < j; k++) if ((o -
  ↪ p[k]).dist() > r * E)
                o = circumcenter(p[i], p[j], p[k]), r
  ↪ = (o - p[i]).dist();
        }
    }
    return {o, r};
}

// --- AREA of (circle centered at c, radius r)
  ↪ INTERSECT polygon ps ---
template <class P> ld circlePolyArea(P c, ld r,
  ↪ vector<P> ps) {
    auto arg = [](P p, P q) { return
  ↪ atan2l(p.cross(q), p.dot(q)); };
    auto tri = [&](P p, P q) -> ld {
        ld r2 = r * r / 2;
        P d = q - p;
        ld a = d.dot(p) / d.dist2(), b = (p.dist2() -
  ↪ r * r) / d.dist2(), det = a * a - b;
        if (det <= 0) return arg(p, q) * r2;
        ld s = max((ld)0, -a - sqrtl(det)), t =
  ↪ min((ld)1, -a + sqrtl(det));
        if (t < 0 || 1 <= s) return arg(p, q) * r2;
        P u = p + d * s, v = p + d * t;
        return arg(p, u) * r2 + u.cross(v) / 2 +
  ↪ arg(v, q) * r2;
    };
    ld sum = 0;
    for (int i = 0, n = ps.size(); i < n; i++) sum +=
  ↪ tri(ps[i] - c, ps[(i + 1) % n] - c);
    return fabsl(sum);
}

// --- power of a point / radical axis ---
// pow(p) = |p-c|^2 - r^2 : <0 inside, 0 on circle,
  ↪ >0 outside; equals (tangent length)^2.
template <class P> ld power(P p, P c, ld r) { return
  ↪ (p - c).dist2() - r * r; }
// Radical axis of two non-concentric circles, as two
  ↪ points on it.
template <class P> pair<P, P> radicalAxis(P a, ld r1,
  ↪ P b, ld r2) {
    P d = b - a; ld t = ((r1 * r1 - r2 * r2) /
  ↪ d.dist2() + 1) / 2;
    P m = a + d * t;
    return {m, m + d.perp()};
}
```

11.1.5 Sweep

Global / sweep-line geometry: problems about a whole SET of objects.

```
// --- CLOSEST PAIR of points, O(n log n). Returns
  ↪ the two points. Exact on P<ll>. ---
template <class P> pair<P, P> closestPair(vector<P>
  ↪ v) {
    assert(v.size() > 1);
  ↪      // n < 2 has no pair to return
    set<P> S;
  ↪     // ordered by (x, y)
    sort(all(v), [](P a, P b) { return a.y < b.y; });
    pair<ll, pair<P, P>> best{LLONG_MAX, {P(), P()}};
    int j = 0;
    for (P p : v) {
        P d{1 + (ll)sqrtl((ld)best.first), 0};
        while (v[j].y <= p.y - d.x) S.erase(v[j++]);
        auto lo = S.lower_bound(p - d), hi =
  ↪ S.upper_bound(p + d);
        for (; lo != hi; ++lo) best = min(best, {(*lo
  ↪ - p).dist2(), {*lo, p}});
        S.insert(p);
    }
    return best.second;
}

// --- AREA OF UNION OF AXIS-ALIGNED RECTANGLES, O(n
  ↪ log n). ---
// rects: {x1, y1, x2, y2} half-open [x1,x2) x
  ↪ [y1,y2). Sweep x, segment tree counts covered y.
struct CoverTree {
  ↪     // "count>0 length" segment tree
    int n; vector<int> cnt; vector<ll> len;
  ↪ vector<ll> ys;
    CoverTree(vector<ll> y) : ys(y) {
        n = ys.size() - 1, cnt.assign(4 * n, 0),
  ↪ len.assign(4 * n, 0);
    }
    void upd(int v, int l, int r, int ql, int qr, int
  ↪ d) {
        if (qr <= l || r <= ql) return;
        if (ql <= l && r <= qr) cnt[v] += d;
        else {
            int m = (l + r) / 2;
            upd(2 * v, l, m, ql, qr, d); upd(2 * v +
  ↪ 1, m, r, ql, qr, d);
        }
        len[v] = cnt[v] ? ys[r] - ys[l] : (r - l == 1
  ↪ ? 0 : len[2 * v] + len[2 * v + 1]);
    }
    void upd(int l, int r, int d) { upd(1, 0, n, l,
  ↪ r, d); }
    ll get() { return len[1]; }
};
ll rectUnionArea(vector<array<ll, 4>> r) {
```

```
    vector<ll> ys;
    for (auto& q : r) ys.push_back(q[1]),
  ↪ ys.push_back(q[3]);
    sort(all(ys)); ys.erase(unique(all(ys)),
  ↪ ys.end());
    if (ys.size() < 2) return 0;
    auto id = [&](ll y) { return
  ↪ (int)(lower_bound(all(ys), y) - ys.begin()); };
    vector<array<ll, 4>> ev;
  ↪     // {x, +-1, y1, y2}
    for (auto& q : r) ev.push_back({q[0], 1, q[1],
  ↪ q[3]}), ev.push_back({q[2], -1, q[1], q[3]});
    sort(all(ev));
    CoverTree T(ys);
    ll area = 0, px = ev.empty() ? 0 : ev[0][0];
    for (auto& e : ev) {
        area += T.get() * (e[0] - px); px = e[0];
        T.upd(id(e[2]), id(e[3]), (int)e[1]);
    }
    return area;
}
// PERIMETER of the union: run the same sweep twice
  ↪ (once per axis) accumulating
// |len(after) - len(before)| at each event x, and
  ↪ add the vertical contributions.

// --- MAX POINTS ON A LINE, O(n^2 log n) ---
template <class P> int maxOnLine(vector<P> p) {
    int n = p.size(), best = min(n, 1);
    for (int i = 0; i < n; i++) {
        map<pair<ll, ll>, int> c; int dup = 0;
        for (int j = 0; j < n; j++) if (j != i) {
            ll dx = p[j].x - p[i].x, dy = p[j].y -
  ↪ p[i].y;
            if (!dx && !dy) { dup++; continue; }
  ↪         // duplicate points
            ll g = gcdll(llabs(dx), llabs(dy)); dx /=
  ↪ g; dy /= g;
            if (dx < 0 || (dx == 0 && dy < 0)) dx =
  ↪ -dx, dy = -dy;
            c[{dx, dy}]++;
        }
        best = max(best, dup + 1);
        for (auto& [k, v] : c) best = max(best, v +
  ↪ dup + 1);
    }
    return best;
}

// --- COUNT PAIRS OF INTERSECTING SEGMENTS, O(n^2)
  ↪ exact. ---
// For "do ANY two of them cross", use Shamos-Hoey
  ↪ (17), which is O(n log n) and returns the pair.
// For O((n+k) log n) reporting of ALL of them use
  ↪ Bentley-Ottmann; almost never needed.
template <class P> ll countSegInter(vector<pair<P,
  ↪ P>> s) {
    ll c = 0;
    for (int i = 0; i < (int)s.size(); i++)
        for (int j = i + 1; j < (int)s.size(); j++)
            c += segCross(s[i].first, s[i].second,
  ↪ s[j].first, s[j].second);
    return c;
}

// --- CONVEX LAYERS (onion peeling): repeatedly
  ↪ strip the hull. O(n^2 log n) naive. ---
// Fine to n ~ 3000; past that use the decremental
  ↪ hull in 22 - OnionDecomposition.cpp,
// O(n log^2 n). Note this version DROPS points lying
  ↪ inside a hull edge, 22 keeps them.
template <class P> vector<vector<P>> onion(vector<P>
  ↪ p) {
    vector<vector<P>> L;
    while (!p.empty()) {
        auto h = hull(p);
        L.push_back(h);
        for (P q : h) p.erase(find(all(p), q));
    }
    return L;
}
```

11.1.6 HalfPlaneIntersection

HALF-PLANE INTERSECTION - O(n log n). A half-plane is "everything LEFT of the directed

```
// line a -> b". Returns the CCW vertices of the
  ↪ intersection, or {} if it is empty.
// A big bounding box is added, so the result is
  ↪ always bounded.
// 2D LINEAR PROGRAMMING is exactly this: constraints
  ↪ -> half-planes, then optimise over the
// resulting convex polygon (ternary search on the
  ↪ boundary, or check all vertices).
struct HP {
    Pd p, d;                                        //
  ↪ keep the side LEFT of p -> p + d
    HP() {}
    HP(Pd a, Pd b) : p(a), d(b - a) {}
    bool out(Pd r) const { return d.cross(r - p) <
  ↪ -eps; }
};
Pd hpInter(const HP& a, const HP& b) {              //
  ↪ assumes a.d and b.d are not parallel
    return a.p + a.d * ((b.p - a.p).cross(b.d) /
  ↪ a.d.cross(b.d));
}
vector<Pd> halfPlaneInter(vector<HP> h, ld BOX = 1e9)
  ↪ {
    Pd box[4] = {Pd(-BOX, -BOX), Pd(BOX, -BOX),
  ↪ Pd(BOX, BOX), Pd(-BOX, BOX)};
    for (int i = 0; i < 4; i++)
  ↪ h.push_back(HP(box[i], box[(i + 1) % 4]));
    sort(all(h), [](const HP& a, const HP& b) {
        if (half(a.d) != half(b.d)) return half(a.d)
  ↪ < half(b.d);
        ld c = a.d.cross(b.d);
        if (fabsl(c) > eps) return c > 0;
        return a.out(b.p);                          //
  ↪ same direction: most restrictive first
    });
    deque<HP> dq;
    for (auto& x : h) {
        if (!dq.empty() &&
  ↪ fabsl(dq.back().d.cross(x.d)) <= eps &&
            dq.back().d.dot(x.d) > 0) continue;
        while (dq.size() > 1 &&
  ↪ x.out(hpInter(dq[dq.size() - 1], dq[dq.size() -
  ↪ 2]))) dq.pop_back();
        while (dq.size() > 1 && x.out(hpInter(dq[0],
  ↪ dq[1]))) dq.pop_front();
        dq.push_back(x);
    }
    while (dq.size() > 2 &&
  ↪ dq[0].out(hpInter(dq[dq.size() - 1],
  ↪ dq[dq.size() - 2]))) dq.pop_back();
    while (dq.size() > 2 &&
```

```
    dq.back().out(hpInter(dq[0], dq[1])))
    dq.pop_front();
    if (dq.size() < 3) return {};
    vector<Pd> r;
    for (size_t i = 0; i < dq.size(); i++)
        r.push_back(hpInter(dq[i], dq[(i + 1) %
        dq.size()]));
    return r;
}
// If the constraints arrive ONLINE, or you must
    report after each one, use 21 - DynamicHalfPlane.
// If you only have a handful of half-planes,
    repeated cut() (03 - Polygon.cpp) on a big box is
// shorter to retype and just as correct - O(n^2) but
    n is small.
```

11.1.7 Unions

AREA OF UNIONS. The three shapes that actually show up.

```
// --- UNION OF POLYGONS (any simple polygons, may
    overlap), O(n^2 log n). ---
// Works by, for every edge, measuring how much of it
    is "on the boundary of the union" and
// accumulating the shoelace contribution with the
    right multiplicity.
ld edgeRatio(Pd a, Pd b) { return sgn(b.x) ? a.x /
    b.x : a.y / b.y; }
ld polyUnion(vector<vector<Pd>>& ps) {
    //     // every polygon must be CCW
    ld ret = 0;
    for (size_t i = 0; i < ps.size(); i++)
        for (size_t v = 0; v < ps[i].size(); v++) {
            Pd A = ps[i][v], B = ps[i][(v + 1) %
            ps[i].size()];
            vector<pair<ld, int>> seg = {{0, 0}, {1,
            0}};
            for (size_t j = 0; j < ps.size(); j++) {
                if (i == j) continue;
                for (size_t u = 0; u < ps[j].size();
                u++) {
                    Pd C = ps[j][u], D = ps[j][(u +
                    1) % ps[j].size()];
                    int sc = sgn(A.cross(B, C)), sd =
                    sgn(A.cross(B, D));
                    if (sc != sd) {
                        ld sa = C.cross(D, A), sb =
                        C.cross(D, B);
                        if (min(sc, sd) < 0)
                        seg.push_back({sa / (sa - sb), sc > sd ? 1 :
                        -1});
                    } else if (!sc && !sd && j < i &&
                    sgn((B - A).dot(D - C)) > 0) {
                        seg.push_back({edgeRatio(C -
                        A, B - A), 1});
                        seg.push_back({edgeRatio(D -
                        A, B - A), -1});
                    }
                }
            }
            sort(all(seg));
            for (auto& s : seg) s.first =
            min(max(s.first, (ld)0), (ld)1);
            ld sum = 0;
            int cnt = seg[0].second;
            for (size_t j = 1; j < seg.size(); j++) {
                //     // fraction of AB on the union boundary
                if (!cnt) sum += seg[j].first - seg[j
                - 1].first;
                cnt += seg[j].second;
            }
            ret += A.cross(B) * sum;
        }
    return ret / 2;
}

// --- UNION OF CIRCLES, O(n^2 log n) by integrating
    each arc that is on the outer boundary. ---
ld circleUnion(vector<Pd> c, vector<ld> r) {
    int n = c.size();
    ld total = 0;
    for (int i = 0; i < n; i++) {
        vector<pair<ld, int>> ev;
        int cover = 0;
        for (int j = 0; j < n; j++) {
            if (i == j) continue;
            ld d = (c[i] - c[j]).dist();
            if (d + r[i] <= r[j] + eps &&
                (r[i] < r[j] || (fabsl(r[i] - r[j]) <
                eps && j < i))) { cover = -1; break; }
            if (d >= r[i] + r[j] - eps || d + r[j] <=
            r[i] + eps) continue;
            ld a = (c[j] - c[i]).angle();
            ld w = acosl((d * d + r[i] * r[i] - r[j]
            * r[j]) / (2 * d * r[i]));
            ld L = a - w, R = a + w;
            if (L < -PI) L += 2 * PI;
            if (R > PI) R -= 2 * PI;
            if (L > R) ev.push_back({L, 1}),
            ev.push_back({PI, -1}),
                       ev.push_back({-PI, 1}),
            ev.push_back({R, -1});
            else ev.push_back({L, 1}),
            ev.push_back({R, -1});
        }
        if (cover < 0) continue;
        ev.push_back({-PI, 0}), ev.push_back({PI, 0});
        sort(all(ev));
        int cnt = 0; ld last = -PI;
        for (auto [a, t] : ev) {
            if (!cnt && a > last) {
                //     // arc [last, a] is on the boundary
                Pd p1 = c[i] + Pd(cosl(last),
                sinl(last)) * r[i];
                Pd p2 = c[i] + Pd(cosl(a), sinl(a)) *
                r[i];
                total += p1.cross(p2) / 2 + r[i] *
                r[i] * ((a - last) - sinl(a - last)) / 2;
            }
            cnt += t;
            last = max(last, a);
        }
    }
    return total;
}

// --- PERIMETER OF A UNION OF AXIS-ALIGNED
    RECTANGLES ---
// Run the rectUnionArea sweep TWICE (once per axis).
    At each event x, the vertical contribution
// is |coveredLen(after) - coveredLen(before)|; sum
    those, then repeat with x/y swapped.
```

11.1.8 Transforms

COORDINATE TRANSFORMS - the tricks that turn a hard metric into an easy one.

```
// CHEBYSHEV <-> MANHATTAN.  rot45(p) = (x+y, x-y).
//   Manhattan distance in the original = Chebyshev distance after rot45  (and vice versa).
// Use it for: "max Manhattan distance over a set" (becomes max over 4 coordinate extremes),
// "count pairs within Manhattan distance k", chessboard-king problems.
template <class P> P rot45(P p) { return P(p.x + p.y, p.x - p.y); }
template <class P> P unrot45(P p) { return P((p.x + p.y) / 2, (p.x - p.y) / 2); }
// Max Manhattan distance between any two of the points, O(n).
ll maxManhattan(const vector<Pi>& v) {
    ll best = 0;
    for (int s = 0; s < 2; s++) {
        // the 2^1 sign patterns for (x +- y)
        ll mn = LLONG_MAX, mx = LLONG_MIN;
        for (auto& p : v) { ll t = s ? p.x - p.y : p.x + p.y; mn = min(mn, t), mx = max(mx, t); }
        best = max(best, mx - mn);
    }
    return best;
}
// In d dimensions the same idea uses all 2^(d-1) sign patterns.

// MANHATTAN MST - the MST under L1 distance, O(n log n) instead of O(n^2).
// Key fact: every point only needs edges to its nearest neighbour in each of 8 octants; by
// symmetry 4 sweeps suffice (rotate/reflect the plane between sweeps).
vector<array<ll, 3>> manhattanEdges(vector<Pi> ps) {
    // {w, i, j}; indices stay ORIGINAL
    int n = ps.size();
    vector<int> id(n);
    iota(all(id), 0);
    vector<array<ll, 3>> e;
    for (int s = 0; s < 4; s++) {
        sort(all(id), [&](int a, int b) { return ps[a].x + ps[a].y < ps[b].x + ps[b].y; });
        map<ll, int> act;
        // key = -y, value = index
        for (int i : id) {
            for (auto it = act.lower_bound(-ps[i].y); it != act.end(); act.erase(it++)) {
                int j = it->second;
                Pi d = ps[i] - ps[j];
                if (d.y > d.x) break;
                e.push_back({d.x + d.y, (ll)i, (ll)j});
            }
            act[-ps[i].y] = i;
        }
        for (auto& p : ps) { if (s & 1) p.x = -p.x; else swap(p.x, p.y); }   // next octant pair
    }
    return e;
    // feed these to Kruskal
}

// LINEAR TRANSFORMATION mapping p0->q0 and p1->q1 (rotation + scaling + translation).
Pd linTrans(Pd p0, Pd p1, Pd q0, Pd q1, Pd r) {
    Pd dp = p1 - p0, dq = q1 - q0, num(dp.cross(dq), dp.dot(dq));
    return q0 + Pd((r - p0).cross(num), (r - p0).dot(num)) / dp.dist2();
}

// POINT <-> LINE DUALITY:  point (a, b)  <->  line y = a*x - b.
//   "max points on a common line" becomes "max lines through a common point"
//   convex hull of points  <->  upper/lower envelope of lines (this IS the CHT correspondence)
//   incidences are preserved, above/below is preserved.
```

11.1.9 KDTree

KD-TREE over 2D points: nearest-neighbour and "count points in a rectangle".

```
// Build O(n log n); NN query O(log n) expected, O(n) worst. The O(sqrt n) bound is for
// RECTANGLE queries; a rectangle count costs O(sqrt n + k).
// Use when coordinates are large/sparse and a grid or sort-sweep will not do.
struct KD {
    struct Node {
        Pi p;
        // the splitting point
        ll x0 = LLONG_MAX, x1 = LLONG_MIN, y0 = LLONG_MAX, y1 = LLONG_MIN;   // bounding box
        int l = -1, r = -1, cnt = 0;
    };
    vector<Node> t;
    vector<Pi> pts;

    KD(vector<Pi> v) : pts(v) { if (!v.empty()) build(0, v.size(), false); }

    int build(int lo, int hi, bool divX) {
        if (lo >= hi) return -1;
        int id = t.size();
        t.push_back({});
        int mid = (lo + hi) / 2;
        nth_element(pts.begin() + lo, pts.begin() + mid, pts.begin() + hi,
                    [&](Pi a, Pi b) { return divX ? a.x < b.x : a.y < b.y; });
        Node nd;
        nd.p = pts[mid], nd.cnt = hi - lo;
        for (int i = lo; i < hi; i++)
            nd.x0 = min(nd.x0, pts[i].x), nd.x1 = max(nd.x1, pts[i].x),
            nd.y0 = min(nd.y0, pts[i].y), nd.y1 = max(nd.y1, pts[i].y);
        t[id] = nd;
        int L = build(lo, mid, !divX), R = build(mid + 1, hi, !divX);
        t[id].l = L, t[id].r = R;
        return id;
    }
    ll boxDist(int i, Pi q) const {
        // squared distance from q to the box
        const Node& n = t[i];
        ll dx = max({(ll)0, n.x0 - q.x, q.x - n.x1}), dy = max({(ll)0, n.y0 - q.y, q.y - n.y1});
        return dx * dx + dy * dy;
    }
```

```
    void nn(int i, Pi q, ll& best, Pi& bp, bool
  ↪ skipSelf) const {
        if (i < 0 || boxDist(i, q) >= best) return;
        ll d = (t[i].p - q).dist2();
        if (d < best && !(skipSelf && d == 0)) best =
  ↪ d, bp = t[i].p;
        int a = t[i].l, b = t[i].r;
        if (a >= 0 && b >= 0 && boxDist(b, q) <
  ↪ boxDist(a, q)) swap(a, b);
        nn(a, q, best, bp, skipSelf), nn(b, q, best,
  ↪ bp, skipSelf);
    }
    // skipSelf skips EVERY point at distance 0, not
  ↪ just q itself - with duplicate input points
    // that is wrong for "nearest OTHER point". If
  ↪ duplicates are possible, count multiplicity
    // separately and treat a repeated point as its
  ↪ own nearest neighbour at distance 0.
    pair<ll, Pi> nearest(Pi q, bool skipSelf = false)
  ↪ const {
        if (t.empty()) return {llinf, Pi()};
        ll best = LLONG_MAX; Pi bp;
        nn(0, q, best, bp, skipSelf);
        return {best, bp};
  ↪     // {squared distance, point}
    }
    int count(int i, ll x0, ll y0, ll x1, ll y1)
  ↪ const { // #points in [x0,x1] x [y0,y1]
        if (i < 0 || t[i].x1 < x0 || t[i].x0 > x1 ||
  ↪ t[i].y1 < y0 || t[i].y0 > y1) return 0;
        if (x0 <= t[i].x0 && t[i].x1 <= x1 && y0 <=
  ↪ t[i].y0 && t[i].y1 <= y1) return t[i].cnt;
        int r = (x0 <= t[i].p.x && t[i].p.x <= x1 &&
  ↪ y0 <= t[i].p.y && t[i].p.y <= y1);
        return r + count(t[i].l, x0, y0, x1, y1) +
  ↪ count(t[i].r, x0, y0, x1, y1);
    }
    int count(ll x0, ll y0, ll x1, ll y1) const {
  ↪ return t.empty() ? 0 : count(0, x0, y0, x1, y1);
  ↪ }
};
```

11.1.10 Triangulation

EAR CLIPPING - triangulates any SIMPLE polygon (convex or not, no self-intersections, no holes)

```
// Needs: Core.cpp, Polygon.cpp (area2).
// into n-2 triangles, O(n^2). Exact with P<ll>.
  ↪ Returns index triples into the ORIGINAL vector.
// An "ear" is a convex vertex whose triangle
  ↪ contains no other vertex of the polygon.
// USES: area/integral of anything over a polygon,
  ↪ rendering, splitting a region for a sweep,
// point-location preprocessing, computing a
  ↪ polygon's centre of mass with a weight function.
template <class P>
vector<array<int, 3>> triangulate(const vector<P>&
  ↪ poly) {
    int n = poly.size();
    vector<array<int, 3>> tri;
    if (n < 3) return tri;
    vector<int> id(n);
    iota(all(id), 0);
    if (area2(poly) < 0) reverse(all(id));
  ↪          // work counter-clockwise
    auto inTri = [&](int a, int b, int c, int q) {
  ↪          // closed triangle
        return poly[a].cross(poly[b], poly[q]) >= 0
  ↪ && poly[b].cross(poly[c], poly[q]) >= 0 &&
              poly[c].cross(poly[a], poly[q]) >= 0;
    };
    while (id.size() > 2) {
        int m = id.size(), cut = -1, flat = -1;
        for (int i = 0; i < m && cut < 0; i++) {
            int a = id[(i + m - 1) % m], b = id[i], c
  ↪ = id[(i + 1) % m];
            auto o = poly[a].cross(poly[b], poly[c]);
            if (o == 0) { flat = i; continue; }
  ↪           // collinear vertex: just drop it
            if (o < 0) continue;
  ↪           // reflex
            bool ok = true;
            for (int j = 0; j < m && ok; j++)
                if (id[j] != a && id[j] != b && id[j]
  ↪ != c && inTri(a, b, c, id[j])) ok = false;
            if (ok) cut = i, tri.push_back({a, b, c});
        }
        if (cut < 0) cut = flat;
        if (cut < 0) break;
  ↪           // not a simple polygon
        id.erase(id.begin() + cut);
    }
    return tri;
}
// FASTER: monotone decomposition (sweep, O(n log n))
  ↪ then triangulate each monotone piece in O(n).
// Only worth writing for n beyond ~2000; ear
  ↪ clipping handles everything a contest throws at
  ↪ you.
// DELAUNAY TRIANGULATION (13 - Delaunay.cpp, O(n log
  ↪ n)) maximises the minimum angle instead, and
// its dual is the VORONOI DIAGRAM (14 -
  ↪ Voronoi.cpp). Cheap derivation to remember: lift
  ↪ each
// point to the paraboloid (x, y, x^2+y^2) and take
  ↪ the LOWER hull of the 3D convex hull - its
// faces project to the Delaunay triangles.
//   Delaunay contains: the Euclidean minimum
  ↪ spanning tree, the nearest-neighbour graph, and
  ↪ the
//   closest pair. So "EMST of points in the plane" =
  ↪ Delaunay (O(n log n) edges) + Kruskal.
//   In-circle test (a,b,c counter-clockwise): d is
  ↪ strictly inside the circumcircle of a,b,c iff
//   the 3x3 determinant of rows (a-d, |a-d|^2),
  ↪ (b-d, |b-d|^2), (c-d, |c-d|^2) is > 0.
  ↪ [__int128]
// ART GALLERY THEOREM: floor(n/3) guards always
  ↪ suffice for a simple n-gon, and are sometimes
// necessary (comb polygon). Proof = triangulate,
  ↪ 3-colour the triangulation graph, take the
// smallest colour class. The triangulation above is
  ↪ exactly what that construction needs.
// POLYGON WITH HOLES: connect each hole to the outer
  ↪ boundary with a "bridge" edge (pick the hole's
// rightmost vertex, shoot a ray right, split the hit
  ↪ edge) to get one simple polygon, then clip.
```

11.1.11 PolygonOps

POLYGON AGAINST POLYGON: clipping, intersection, containment, distance, and the O(log n)

```
// Needs: Core.cpp, Lines.cpp, Polygon.cpp.
// queries you get once one side is a convex hull.

// --- IS THIS POLYGON SIMPLE? O(n^2), exact on P<ll>. ---
// Every polygon routine (inPoly, area, triangulate, Pick, polyUnion) silently returns nonsense on
// a self-intersecting polygon, so this is also the oracle you stress-test them with.
template <class P> bool isSimple(const vector<P>& v) {
    int n = v.size();
    if (n < 3) return false;
    for (int i = 0; i < n; i++) {
        if (v[i] == v[(i + 1) % n]) return false;
                // zero-length edge
        for (int j = i + 1; j < n; j++) {
            int a = i, b = (i + 1) % n, c = j, d = (j + 1) % n;
            if (a == c || a == d || b == c || b == d) {
                // adjacent edges: they may only
                if (a == d || b == c) {
                    // share the one common vertex
                    P sh = (a == d) ? v[a] : v[b];
                    for (P q : {v[a], v[b], v[c], v[d]})
                        if (!(q == sh) && onSeg(q, v[a], v[b]) &&
                            onSeg(q, v[c], v[d]))
                            return false;
                }
                continue;
            }
            if (segCross(v[a], v[b], v[c], v[d])) return false;
        }
    }
    return true;
}
// --- WINDING NUMBER. inPoly uses the EVEN-ODD rule, which is wrong for a self-intersecting
// outline and for "polygon with holes given as one vertex list". The nonzero rule uses this. ---
template <class P> int windingNumber(const vector<P>& v, P p) {    // 0 = outside (nonzero rule)
    int n = v.size(), w = 0;
    for (int i = 0; i < n; i++) {
        P a = v[i], b = v[(i + 1) % n];
        if (onSeg(p, a, b)) return 0;
            // on the boundary: caller decides
        if (a.y <= p.y) { if (b.y > p.y && a.cross(b, p) > 0) w++; }
        else if (b.y <= p.y && a.cross(b, p) < 0) w--;
    }
    return w;
}
// --- SUTHERLAND-HODGMAN: clip any polygon by a CONVEX window, O(n * m). ---
// The window must be convex and counter-clockwise; the subject may be concave.
// This is the general "intersect with a convex region" and it is just ‘cut‘ in a loop.
template <class P> vector<P> clipByConvex(vector<P> subject, const vector<P>& win) {
    int m = win.size();
    for (int i = 0; i < m && !subject.empty(); i++)
        subject = cut(subject, win[i], win[(i + 1) % m]);
    return subject;
}
// --- CONVEX intersect CONVEX in O(n + m) is the classic O’Rourke walk, but clipByConvex is
// O(nm) and fits on one line - at contest sizes (n, m <= a few thousand) prefer it. Use
// halfPlaneInter (06) when the polygons come as half-plane constraints, not vertex lists.

// --- MINIMUM DISTANCE BETWEEN TWO CONVEX POLYGONS, O(n * m) as written. ---
// 0 if they touch or overlap. The O(n+m) rotating-calipers version exists but this is 6 lines and
// the distance is a floating answer anyway. Equivalent trick: dist(origin, minkowski(A, -B)).
template <class P> ld convexDist(const vector<P>& A, const vector<P>& B) {
    for (P p : A) if (inPoly(B, p, false)) return 0;
    for (P p : B) if (inPoly(A, p, false)) return 0;
    ld best = 1e18;
    int n = A.size(), m = B.size();
    for (int i = 0; i < n; i++) for (int j = 0; j < m; j++) {
        P a = A[i], b = A[(i + 1) % n], c = B[j], d = B[(j + 1) % m];
        if (segCross(a, b, c, d)) return 0;
        best = min(best, segSegDist(a, b, c, d));
    }
    return best;
}
// --- TANGENTS FROM A POINT TO A CONVEX POLYGON, O(n) here (O(log n) with a binary search). ---
// Returns the two vertex indices l, r such that the whole polygon lies to one side of p->h[l]
// and of p->h[r]: the visible arc is h[r], h[r+1], ..., h[l]. p must be strictly OUTSIDE.
// USES: visibility ("how much of the wall can I see"), shadow/silhouette, and "add p to the hull".
template <class P> pair<int, int> pointTangents(const vector<P>& h, P p) {
    int n = h.size(), l = 0, r = 0;
    for (int i = 1; i < n; i++) {
        if (p.cross(h[i], h[l]) > 0) l = i;
                // h[l] is the CCW-most
        if (p.cross(h[i], h[r]) < 0) r = i;
                // h[r] is the CW-most
    }
    return {l, r};
}
// --- LINE x CONVEX POLYGON in O(log n): which two edges does the line a->b cross? ---
// Returns {-1,-1} for no hit; {i,-1} for a single touched corner i; {i,j} when the line crosses
// the edges (i,i+1) and (j,j+1). h must be a strictly convex CCW hull (no collinear vertices).
// USES: "length of the segment inside the polygon" per query, in O(log n) instead of O(n).
template <class P> pair<int, int> lineHull(const vector<P>& h, P a, P b) {
    int n = h.size();
    int endA = extreme(h, (a - b).perp()), endB = extreme(h, (b - a).perp());
    auto cmpL = [&](int i) { return sgn(a.cross(h[i], b)); };
```

```
    if (cmpL(endA) < 0 || cmpL(endB) > 0) return {-1,
  ↪ -1};          // whole hull on one side
    auto bs = [&](int lo, int hi) {
  ↪               // last index still >= 0
        int step = (hi - lo + n) % n;
        for (int s = step; s > 0; s /= 2)
            while (s && cmpL((lo + s) % n) >= 0) lo =
  ↪ (lo + s) % n, step -= s;
        return lo;
    };
    int i = bs(endB, endA), j = bs(endA, endB);
    if (i == j) return {i, -1};
    return {i, j};
}
/* MORE POLYGON FACTS AND RECIPES
 AREA OF INTERSECTION of two CONVEX polygons:
  ↪ clipByConvex then area(). For two ARBITRARY
  ↪ simple
   polygons, triangulate one (10 - Triangulation.cpp)
  ↪ and clip each triangle by the other.
 AREA OF THE UNION: |A| + |B| - |A n B|, or polyUnion
  ↪ (07 - Unions.cpp) for many polygons at once.
 SYMMETRIC DIFFERENCE: |A| + |B| - 2|A n B|.
 POINT IN A POLYGON WITH HOLES: outer boundary CCW,
  ↪ every hole CW, then the sign of the total
   winding number is the answer (this is exactly why
  ↪ windingNumber exists).
 CONVEX POLYGON CONTAINS CONVEX POLYGON: every vertex
  ↪ of B is in A (inConvex, O(m log n)).
 CENTROID OF THE PERIMETER (not of the area): sum of
  ↪ edge midpoints weighted by edge LENGTH.
   The two differ - "balance the wire frame" vs
  ↪ "balance the plate".
 POLYGON DIAMETER / WIDTH / MIN RECTANGLE: 03 -
  ↪ Polygon.cpp (rotating calipers).
 REGULAR n-GON of circumradius R centred at c: c +
  ↪ R*(cos(2*pi*k/n + t), sin(2*pi*k/n + t)).
   Its area is (n/2) R^2 sin(2 pi/n); with INRADIUS r
  ↪ it is n r^2 tan(pi/n).
 SPLIT A POLYGON BY A LINE into both halves: cut(v,
  ↪ s, e) and cut(v, e, s).
 LARGEST INSCRIBED / SMALLEST ENCLOSING: smallest
  ↪ enclosing circle is mec (04 - Circles.cpp);
   smallest enclosing rectangle is minRectArea;
  ↪ largest inscribed CIRCLE in a convex polygon is
   the Chebyshev centre, 18 - InscribedCircle.cpp (a
  ↪ nested ternary search, no LP needed);
   largest inscribed AXIS-ALIGNED rectangle needs a
  ↪ sweep and is genuinely hard - do not improvise.
 OFFSET / BUFFER a convex polygon by d: push every
  ↪ edge outward by d along its normal and
   intersect the half-planes (06 -
  ↪ HalfPlaneIntersection.cpp). The exact offset of
  ↪ a NON-convex
   polygon has circular arcs at the reflex corners -
  ↪ approximate them.
 TRIANGLE CONTAINS POINT, exact: the three
  ↪ orientations all have the same sign (or one is 0
  ↪ for
   the boundary). Faster than inPoly for n = 3 and it
  ↪ is the inner loop of triangulate. */
```

11.1.12 Primitives

THE SMALL-FUNCTION LAYER. Individually trivial, collectively the thing you actually spend

```
// Needs: Core.cpp, Lines.cpp, Circles.cpp.
// contest minutes re-deriving. Everything here is
  ↪ one to six lines.

// --- ANGLE BISECTORS OF TWO LINES. Two lines have
  ↪ TWO bisectors, perpendicular to each other.
// Direction of the internal one = the sum of the
  ↪ unit direction vectors; external = difference.
// (Take the pair whose sign matches the side you
  ↪ want; if the lines are parallel the bisector is
// the parallel line halfway between them.)
template <class P> pair<P, P> lineBisectors(P d1, P
  ↪ d2) {          // returns the two DIRECTIONS
    P u = d1.unit(), v = d2.unit();
    return {u + v, u - v};
}
// Internal bisector of the angle at b in triangle
  ↪ a-b-c, as a direction from b:
template <class P> P angleBisector(P a, P b, P c) {
  ↪ return (a - b).unit() + (c - b).unit(); }
// The internal bisector from A meets BC at the point
  ↪ dividing it in ratio AB : AC.
template <class P> P bisectorFoot(P a, P b, P c) {
    ld x = (a - b).dist(), y = (a - c).dist();
    return (b * y + c * x) / (x + y);
}
// --- ROTATE / SCALE ABOUT AN ARBITRARY CENTRE
  ↪ (rot() in Core is about the origin). ---
template <class P> P rotAbout(P p, P c, ld a) {
  ↪ return c + (p - c).rot(a); }
template <class P> P scaleAbout(P p, P c, ld k) {
  ↪ return c + (p - c) * k; }
// Reflect p across the POINT c (a 180-degree
  ↪ rotation):
template <class P> P reflPoint(P p, P c) { return c *
  ↪ 2 - p; }
// --- SIGNED AREA OF A TRIANGLE, and the barycentric
  ↪ coordinates of p in a-b-c. ---
template <class P> auto triArea2(P a, P b, P c) {
  ↪ return a.cross(b, c); }         // 2 * signed
template <class P> array<ld, 3> barycentric(P a, P b,
  ↪ P c, P p) {
    ld s = a.cross(b, c);
    return {(ld)p.cross(b, c) / s, (ld)a.cross(p, c)
  ↪ / s, (ld)a.cross(b, p) / s};
}
// p is inside the triangle iff all three coordinates
  ↪ are >= 0 (they always sum to 1).
template <class P> bool inTriangle(P a, P b, P c, P
  ↪ p) {           // exact, boundary included
    int s1 = sgn(a.cross(b, p)), s2 = sgn(b.cross(c,
  ↪ p)), s3 = sgn(c.cross(a, p));
    return (s1 >= 0 && s2 >= 0 && s3 >= 0) || (s1 <=
  ↪ 0 && s2 <= 0 && s3 <= 0);
}
// --- SEGMENT x CIRCLE: clip the segment a-b to the
  ↪ disc, returning the surviving piece. ---
template <class P> vector<P> segCircle(P a, P b, P c,
  ↪ ld r) {
    auto v = circleLine(c, r, a, b);
  ↪              // 0, 1 or 2 points
    vector<P> res;
    for (P p : v) if (onSeg(p, a, b))
  ↪ res.push_back(p);
    return res;
```

```
}
// --- CIRCLE THROUGH THREE POINTS = circumcenter (04 - Circles.cpp); its radius = circumradius.
// CIRCLE THROUGH TWO POINTS WITH A GIVEN RADIUS: the two centres are m +- h * (b-a).perp().unit()
// where m is the midpoint and h = sqrt(r^2 - |b-a|^2/4). None if 2r < |b-a|.
template <class P> vector<P> circlesThrough(P a, P b, ld r) {
    ld d2 = (b - a).dist2(), h2 = r * r - d2 / 4;
    if (d2 < eps || h2 < 0) return {};
                // a == b: infinitely many
    P m = (a + b) / 2, dir = (b - a).perp().unit() * sqrtl(h2);
    return h2 < eps ? vector<P>{m} : vector<P>{m - dir, m + dir};
}
// --- CIRCLE OF RADIUS r THROUGH p AND TANGENT TO THE LINE a-b. Its centre is at distance r from
// the line, so on one of the two PARALLELS at offset r, and at distance r from p, so on the
// circle (p, r). Intersect the two: 0, 1 or 2 centres. With d = distance from p to the line,
// there are 2 for d < 2r (both, tangentially, when d == 0), 1 when d == 2r, and none beyond.
template <class P> vector<P> circleTangentLine(P a, P b, P p, ld r) {
    vector<P> res;
    P n = (b - a).perp().unit() * r;
    for (int s = -1; s <= 1; s += 2)
        for (P c : circleLine(p, r, a + n * s, b + n * s)) res.push_back(c);
    return res;
}
// --- APOLLONIUS CIRCLE: the locus of p with |pa| / |pb| = k (k != 1) is a circle whose diameter
// endpoints are the two points dividing ab internally and externally in ratio k : 1.
template <class P> pair<P, ld> apollonius(P a, P b, ld k) {
    P u = (b * k + a) / (k + 1), v = (b * k - a) / (k - 1);        // internal, external
    return {(u + v) / 2, (u - v).dist() / 2};
}
// --- IS THE QUADRILATERAL a,b,c,d CONCYCLIC? Exact on P<ll> with __int128 (in-circle test). ---
// > 0: d strictly inside the circumcircle of a CCW triangle a,b,c;  == 0: concyclic.
template <class P> int inCircle(P a, P b, P c, P d) {
    auto f = [&](P p) { return (__int128)(p - d).dist2(); };
    __int128 det = (__int128)(a.x - d.x) * ((b.y - d.y) * f(c) - (c.y - d.y) * f(b))
                 - (__int128)(a.y - d.y) * ((b.x - d.x) * f(c) - (c.x - d.x) * f(b))
                 + f(a) * ((__int128)(b.x - d.x) * (c.y - d.y)
                         - (__int128)(c.x - d.x) * (b.y - d.y));
    return (det > 0) - (det < 0);
}
// --- MAXIMUM POINTS COVERED BY A CIRCLE OF RADIUS r, O(n^2 log n). ---
// For each i, every j within 2r gives an ARC of centre-angles that cover both; sweep the arcs.
template <class P> int maxCovered(const vector<P>& p, ld r) {
    int n = p.size(), best = 1;
    for (int i = 0; i < n; i++) {
        vector<pair<ld, int>> ev;
        int base = 1;
                // p[i], plus its duplicates
        for (int j = 0; j < n; j++) if (j != i) {
            ld d = (p[j] - p[i]).dist();
            if (d > 2 * r + eps) continue;
            if (d < eps) { base++; continue; }
                // DUPLICATE of p[i]: the

                // division below would be 0/0

                // and it is always covered
            ld th = (p[j] - p[i]).angle(), ph = acosl(min((ld)1, max((ld)-1, d / (2 * r))));
            ld s = th - ph, e = th + ph;
                // the arc of valid centres
            while (s < -PI) s += 2 * PI;
                // normalise, then SPLIT the
            e = s + 2 * ph;
                // arcs that wrap past +pi -
            // 0 = arc opens, 1 = arc closes.
Pair-sorting then puts the OPEN first at an equal
            // angle, which is what makes a
TANGENTIAL arc (d == 2r, so the arc is a single
point)
            // still count - with {+1}/{-1} the close
sorts first and the point is lost.
            if (e <= PI) ev.push_back({s, 0}),
ev.push_back({e, 1});
            else {
                ev.push_back({s, 0}),
ev.push_back({PI, 1});
                ev.push_back({-PI, 0}),
ev.push_back({e - 2 * PI, 1});
            }
        }
        sort(all(ev));
        int cur = base;
        best = max(best, cur);
        for (auto& [a, t] : ev) best = max(best, cur += t ? -1 : 1);
    }
    return best;
                // the circle passes through p[i]
}
// --- GEOMETRIC MEDIAN (minimise the SUM of distances). Nested ternary search - the objective is
// convex, so this is exact to any eps and cannot get stuck. Weiszfeld iteration is faster but
// breaks when the iterate lands exactly on a sample point.
template <class P> P geoMedian(const vector<P>& p) {
    auto f = [&](ld x, ld y) { ld s = 0; for (auto& q : p) s += (P{x, y} - q).dist(); return s; };
    auto fy = [&](ld x) { ld lo = -2e9, hi = 2e9;
        for (int it = 0; it < 100; it++) { ld a = lo + (hi - lo) / 3, b = hi - (hi - lo) / 3;
            f(x, a) < f(x, b) ? hi = b : lo = a; }
        return (lo + hi) / 2; };
    ld lo = -2e9, hi = 2e9;
    for (int it = 0; it < 100; it++) {
        ld a = lo + (hi - lo) / 3, b = hi - (hi - lo) / 3;
        f(a, fy(a)) < f(b, fy(b)) ? hi = b : lo = a;
    }
    ld x = (lo + hi) / 2;
```

```
    return {x, fy(x)};
}
/* THE REST OF THE SMALL LAYER, as one-liners you can
  ↪ reconstruct instantly
 COLLINEAR a, b, c              a.cross(b, c) == 0
 PARALLEL / PERPENDICULAR       d1.cross(d2) == 0   /
  ↪ d1.dot(d2) == 0
 SAME SIDE OF A LINE            sgn(a.cross(b, p)) ==
  ↪ sgn(a.cross(b, q))
 POINT ON A RAY / SEGMENT       onRay, onSeg (02 -
  ↪ Lines.cpp)
 DISTANCE point-line/segment    lineDist, segDist
  ↪ ray: rayDist    seg-seg: segSegDist
 PROJECT / REFLECT over a line  proj, refl (02 -
  ↪ Lines.cpp)
 ANGLE between two vectors      atan2(cross, dot) ->
  ↪ signed, in (-pi, pi]. NEVER acos(dot/|a||b|):
                                it loses all precision
  ↪ near 0 and pi and cannot give you a sign.
 ROTATE 90 DEGREES              p.perp() = (-y, x).
  ↪ Twice = negation. This is the whole of "turn
                                left"; you almost never
  ↪ need a general rot() for a right angle.
 SORT BY ANGLE                  angCmp (01 - Core.cpp),
  ↪ NOT atan2 - exact and 5x faster.
 AREA OF A TRIANGLE             |cross| / 2. Heron only
  ↪ if you are given the three SIDES, and then
                                use the sorted stable
  ↪ form: sort a >= b >= c and
                                A =
  ↪ sqrt((a+(b+c))(c-(a-b))(c+(a-b))(a+(b-c))) / 4.
 TRIANGLE CENTRES               centroid (a+b+c)/3,
  ↪ circumcenter, incenter, orthocenter (04).
                                EULER LINE: O, G, N, H
  ↪ are collinear with OG:GN:NH = 2:1:3, and
                                H = A + B + C - 2O.
  ↪ Nine-point centre = midpoint of OH, radius R/2.
 TRIANGLE IDENTITIES            A = rs = abc/(4R);  R =
  ↪ abc/(4A);  r = A/s;  r_a = A/(s-a)
                                law of cosines c^2 =
  ↪ a^2 + b^2 - 2ab cos C
                                law of sines a/sin A =
  ↪ 2R
                                OI^2 = R(R - 2r)  =>
  ↪ EULER'S INEQUALITY R >= 2r
                                Stewart (cevian d to BC
  ↪ split m + n): b^2 m + c^2 n = a(d^2 + mn)
                                median m_a = sqrt(2b^2
  ↪ + 2c^2 - a^2)/2
 CYCLIC QUADRILATERAL           PTOLEMY: AC*BD = AB*CD
  ↪ + BC*AD (in cyclic order). For ANY four
                                points AC*BD <= AB*CD +
  ↪ BC*AD, equality iff concyclic in that order.
                                BRAHMAGUPTA area =
  ↪ sqrt((s-a)(s-b)(s-c)(s-d)).
                                General quadrilateral
  ↪ (BRETSCHNEIDER, angle-free):
                                A = sqrt(4p^2q^2 -
  ↪ (b^2+d^2-a^2-c^2)^2)/4 with p, q the diagonals.
 POWER OF A POINT               pow(p) = |pc|^2 - r^2;
  ↪ for ANY line through p meeting the circle at
                                A and B the signed
  ↪ product PA*PB equals it. Outside: = tangent^2.
                                RADICAL AXIS = equal
  ↪ power, a line perpendicular to the centre line.
                                RADICAL CENTRE of three
  ↪ circles with non-collinear centres: the
                                three pairwise axes are
  ↪ concurrent.
 DESCARTES CIRCLE THEOREM       four mutually tangent
  ↪ circles, curvatures k_i = +-1/r_i (negative
                                for the enclosing one):
  ↪  (k1+k2+k3+k4)^2 = 2(k1^2+k2^2+k3^2+k4^2),
                                so k4 = k1+k2+k3 +- 2
  ↪ sqrt(k1k2 + k2k3 + k3k1).
 LATTICE POINTS                 on a segment: gcd(|dx|,
  ↪ |dy|) + 1 endpoints included.
                                inside a polygon: PICK,
  ↪ I = A - B/2 + 1 (simple polygons only).
                                under a line, in
  ↪ O(log): floorSum (04 - Number Theory/01/03).
 HELLY (2D)                     if every THREE of a
  ↪ family of convex sets meet, all of them meet.
 RADON (2D)                     any FOUR points split
  ↪ into two sets whose hulls intersect.
 CARATHEODORY (2D)              a point in the hull of
  ↪ S is in the hull of at most THREE points of S
                                - which is exactly why
  ↪ the minimum enclosing circle needs only 3.
 ART GALLERY                    floor(n/3) guards
  ↪ always suffice for a simple n-gon (triangulate,
                                3-colour, take the
  ↪ smallest class). Orthogonal polygon: floor(n/4).
 EULER, PLANAR                  V - E + F = 1 + C. n
  ↪ lines in general position cut the plane into
                                1 + n + C(n,2) regions,
  ↪ C(n-1,2) of them bounded; n circles in
                                general position give
  ↪ n^2 - n + 2 regions.
 DUALITY                        point (a,b) <-> line y
  ↪ = ax - b. Incidence and above/below are
                                preserved, so "max
  ↪ collinear points" becomes "max concurrent lines"
                                and a convex hull
  ↪ becomes an upper envelope. */
```

11.1.13 Delaunay

DELAUNAY TRIANGULATION, O(n log n), divide and conquer over the quad-edge structure.

```
// Needs: Core.cpp.
// Returns index triples into the input, each one
  ↪ COUNTER-CLOCKWISE. The defining property, and
// the only one you ever use: the circumcircle of
  ↪ every returned triangle is EMPTY of input points.
// WHEN: EUCLIDEAN MST of points in the plane - the
  ↪ EMST is a subgraph of the Delaunay graph, so
// Delaunay (O(n) edges) + Kruskal replaces the
  ↪ O(n^2) complete graph. Also the nearest-neighbour
// graph, the closest pair, the largest empty circle,
  ↪ "maximise the minimum angle" meshing, and
// the VORONOI DIAGRAM, which is exactly the dual (14
  ↪ - Voronoi.cpp).
// COMPLEXITY: O(n log n) time. At most 2n - 2 - h
  ↪ triangles and 3n - 3 - h edges, h = hull size.
// Allocates quad-edges with new and never frees
  ↪ them; budget ~200 bytes per point.
// DEGENERATE: DUPLICATE POINTS ARE NOT ALLOWED -
  ↪ dedupe before calling. Fewer than three points,
// or ALL POINTS COLLINEAR, returns {} - there is
  ↪ genuinely no triangle, handle it upstream.
// Four or more CONCYCLIC points make the
  ↪ triangulation non-unique; you get one legal
  ↪ answer.
// EXACT on P<ll> for |coord| <= 1e9: orient fits in
  ↪ ll (<= 8e18) and inCircle is __int128.
typedef struct QuadEdge* QE;
```

```
Pi qArb(LLONG_MAX, LLONG_MAX);
  ↪ // a sentinel unequal to any real point
struct QuadEdge {
    QE rot, o; Pi p = qArb; bool mark = 0;
    Pi& F() { return r()->p; }
  ↪ // the far endpoint of this directed edge
    QE& r() { return rot->rot; }
  ↪ // the same edge reversed
    QE prev() { return rot->o->rot; }
    QE next() { return r()->prev(); }
};
bool inCirc(Pi p, Pi a, Pi b, Pi c) {
  ↪ // is p strictly inside circumcircle(a, b, c)?
    __int128 p2 = p.dist2(), A = a.dist2() - p2, B =
  ↪ b.dist2() - p2, C = c.dist2() - p2;
    return p.cross(a, b) * C + p.cross(b, c) * A +
  ↪ p.cross(c, a) * B > 0;
}
QE qeEdge(Pi orig, Pi dest) {
    QE q[] = {new QuadEdge{0, 0, orig}, new
  ↪ QuadEdge{0, 0, qArb}, new QuadEdge{0, 0, dest},
            new QuadEdge{0, 0, qArb}};
    for (int i = 0; i < 4; i++) q[i]->o = q[-i & 3],
  ↪ q[i]->rot = q[(i + 1) & 3];
    return *q;
}
void qeSplice(QE a, QE b) { swap(a->o->rot->o,
  ↪ b->o->rot->o), swap(a->o, b->o); }
QE qeConnect(QE a, QE b) {
    QE q = qeEdge(a->F(), b->p);
    qeSplice(q, a->next()), qeSplice(q->r(), b);
    return q;
}
QE dNext(QE e, bool lf) { return lf ? e->o :
  ↪ e->prev(); }        // walk left / right of the
  ↪ base
bool dValid(QE e, QE base) { return
  ↪ e->F().cross(base->F(), base->p) > 0; }
QE dDel(QE init, QE base, bool lf) {
  ↪  // drop edges that the new base invalidates
    QE e = dNext(init, lf);
    if (dValid(e, base))
        while (inCirc(dNext(e, lf)->F(), base->F(),
  ↪ base->p, e->F())) {
            QE t = dNext(e, lf);
            qeSplice(e, e->prev()), qeSplice(e->r(),
  ↪ e->r()->prev());
            e = t;
        }
    return e;
}
pair<QE, QE> dRec(const vector<Pi>& s) {        //
  ↪ {ccw hull edge at left end, cw at right}
    if (s.size() <= 3) {
        QE a = qeEdge(s[0], s[1]), b = qeEdge(s[1],
  ↪ s.back());
        if (s.size() == 2) return {a, a->r()};
        qeSplice(a->r(), b);
        auto side = s[0].cross(s[1], s[2]);
        QE c = side ? qeConnect(b, a) : 0;
  ↪  // side == 0: three collinear points, no face
        return {side < 0 ? c->r() : a, side < 0 ? c :
  ↪ b->r()};
    }
    QE A, B, ra, rb;
    int half = s.size() / 2;
    tie(ra, A) = dRec({s.begin(), s.end() - half});
    tie(B, rb) = dRec({s.begin() + (s.size() - half),
  ↪ s.end()});
    while ((B->p.cross(A->F(), A->p) < 0 && (A =
  ↪ A->next())) ||     // find the lower common
  ↪ tangent
           (A->p.cross(B->F(), B->p) > 0 && (B =
  ↪ B->r()->o)));
    QE base = qeConnect(B->r(), A);
    if (A->p == ra->p) ra = base->r();
    if (B->p == rb->p) rb = base;
    for (;;) {
  ↪ // zip the two hulls together, bottom to top
        QE lc = dDel(base->r(), base, 1), rc =
  ↪ dDel(base, base, 0);
        if (!dValid(lc, base) && !dValid(rc, base))
  ↪ break;
        if (!dValid(lc, base) || (dValid(rc, base) &&
  ↪ inCirc(rc->F(), rc->p, lc->F(), lc->p)))
            base = qeConnect(rc, base->r());
        else base = qeConnect(base->r(), lc->r());
    }
    return {ra, rb};
}
vector<array<int, 3>> delaunay(const vector<Pi>& pts)
  ↪ {
    vector<Pi> p = pts;
    sort(all(p));
    vector<array<int, 3>> res;
    if (p.size() < 3) return res;
    bool line = true;
  ↪ // guard collinear input: dRec spins on it
    for (int i = 2; i < sz(p) && line; i++) line =
  ↪ p[0].cross(p[1], p[i]) == 0;
    if (line) return res;
    QE e = dRec(p).first;
    vector<QE> q = {e};
    while (e->o->F().cross(e->F(), e->p) < 0) e =
  ↪ e->o;
    vector<Pi> out;
    auto walk = [&](QE e) {
  ↪ // emit one face, queue its neighbours
        QE c = e;
        do { c->mark = 1, out.pb(c->p), q.pb(c->r()),
  ↪ c = c->next(); } while (c != e);
    };
    walk(e), out.clear();
  ↪ // the first face walked is the OUTER one
    for (int i = 0; i < sz(q); i++) if (!(e =
  ↪ q[i])->mark) walk(e);
    map<pair<ll, ll>, int> id;
    for (int i = 0; i < sz(pts); i++) id[{pts[i].x,
  ↪ pts[i].y}] = i;
    for (int i = 0; i + 2 < sz(out); i += 3)
        res.pb({id[{out[i].x, out[i].y}], id[{out[i +
  ↪ 1].x, out[i + 1].y}],
                id[{out[i + 2].x, out[i + 2].y}]});
    return res;
}
// EUCLIDEAN MST: collect the 3 edges of every
  ↪ triangle, dedupe, sort by squared length,
  ↪ Kruskal.
//   vector<array<ll,3>> E; for (auto& t :
  ↪ delaunay(p)) for (int k = 0; k < 3; k++)
//       E.pb({(p[t[k]] - p[t[(k+1)%3]]).dist2(),
  ↪ t[k], t[(k+1)%3]});
// LARGEST EMPTY CIRCLE whose centre is inside the
  ↪ hull: its centre is a Delaunay circumcentre or
// a point on the hull boundary - check all O(n)
  ↪ circumcentres, then the hull edges.
```

```
// The DELAUNAY GRAPH also contains the
   ↪ nearest-neighbour graph and the Gabriel graph,
   ↪ so any
// "for each point, its closest other point" question
   ↪ is answered by scanning its Delaunay edges.
// FLIP ALGORITHM (any triangulation -> Delaunay by
   ↪ flipping illegal edges) is O(n^2) and simple,
// but you already have this; the paraboloid lift (x,
   ↪ y, x^2+y^2) + LOWER 3D hull is the other
// derivation and is the one to remember when you
   ↪ need the connection to convexity.
```

11.1.14 Voronoi

VORONOI DIAGRAM. The cell of site i is the set of points closer to site i than to any other

```
// Needs: Core.cpp, HalfPlaneIntersection.cpp
   ↪ (Delaunay.cpp for the fast variant).
// site; it is a convex polygon (unbounded for sites
   ↪ on the convex hull, so everything here is
// clipped to a box of half-width BOX - make BOX
   ↪ comfortably larger than any coordinate you care
// about, and remember that a vertex of the clipped
   ↪ cell may be an artefact of the box).
// The cell is just the intersection of the
   ↪ half-planes "closer to i than to j" over all j
   ↪ != i,
// and each of those is the side of the perpendicular
   ↪ bisector of s_i s_j containing s_i.
// WHEN: nearest-site queries with a planar point
   ↪ location structure on top; largest empty circle;
// "which post office serves this house"; any
   ↪ facility-location or growth/flood simulation
   ↪ where
// regions expand at equal speed. If you only need
   ↪ nearest neighbours, a KD-tree (09) is shorter.
// COMPLEXITY: O(n^2 log n) for all n cells as
   ↪ written (O(n log n) per cell). That is fine up to
// n around 2000. For O(n log n) total, build the
   ↪ DELAUNAY dual instead - see the tail.
// DEGENERATE: duplicate sites give two identical
   ↪ cells that are both empty ({} is returned);
// dedupe first. n == 1 gives the whole box.
   ↪ Collinear sites give slab cells, handled
   ↪ correctly.
// FLOATING POINT: the bisectors are exact only if
   ↪ you keep sites integral, but halfPlaneInter is
// an eps algorithm, so the cell vertices carry the
   ↪ usual 1e-9 relative error. Four or more
// cocircular sites make a Voronoi vertex of degree >
   ↪ 3 and the cell may pick up a zero-length
// edge; strip edges shorter than eps if you are
   ↪ counting them.
vector<Pd> voronoiCell(const vector<Pd>& s, int i, ld
   ↪ BOX = 1e9) {
    vector<HP> h;
    for (int j = 0; j < sz(s); j++) if (j != i) {
        Pd m = (s[i] + s[j]) / 2;
   ↪    // the perpendicular bisector, directed so
        h.pb(HP(m, m + (s[j] - s[i]).perp()));
   ↪    // that the LEFT side contains s[i]
    }
    return halfPlaneInter(h, BOX);
}
vector<vector<Pd>> voronoi(const vector<Pd>& s, ld
   ↪ BOX = 1e9) {
    vector<vector<Pd>> c;
    for (int i = 0; i < sz(s); i++)
   ↪ c.pb(voronoiCell(s, i, BOX));
    return c;
}
// O(n log n) VIA THE DELAUNAY DUAL (13 -
   ↪ Delaunay.cpp). One Voronoi vertex per Delaunay
   ↪ triangle,
// namely its CIRCUMCENTRE; one Voronoi edge per
   ↪ Delaunay edge, joining the circumcentres of the
// two triangles that share it. Recipe: run
   ↪ delaunay(p); for every triangle t store
// circumcenter(p[t[0]], p[t[1]], p[t[2]]); for each
   ↪ site collect the circumcentres of the
// triangles incident to it and sort them by angle
   ↪ around the site - that is its cell, in CCW
// order. A site on the CONVEX HULL has an unbounded
   ↪ cell: its two hull edges contribute rays
// along their outward perpendicular bisectors, so
   ↪ clip them to the box by hand.
// NEAREST SITE for a query point = the cell
   ↪ containing it: build the diagram, then run planar
// point location (15). For a handful of queries just
   ↪ scan all sites.
// LARGEST EMPTY CIRCLE with centre inside the hull:
   ↪ its centre is a Voronoi vertex (= a Delaunay
// circumcentre) or a point where a Voronoi edge
   ↪ crosses the hull boundary. Check both families.
// DELAUNAY EDGE <-> VORONOI EDGE, and the MST /
   ↪ nearest-neighbour consequences, are all in 13.
// WEIGHTED (POWER / LAGUERRE) DIAGRAM: replace the
   ↪ bisector by the RADICAL AXIS of the two
// circles (04 - Circles.cpp) - the cells stay convex
   ↪ polygons, so this exact code works with the
// bisector swapped out. The MULTIPLICATIVELY
   ↪ weighted (Apollonius) diagram has circular arcs
   ↪ for
// boundaries and none of this applies.
```

11.1.15 PointLocation

PLANAR POINT LOCATION, offline, by a left-to-right sweep. Input: a set of segments that are

```
// Needs: Core.cpp.
// pairwise NON-CROSSING (they may share endpoints) -
   ↪ i.e. the edges of a planar subdivision - and
// a list of query points. Output, for each query,
   ↪ the index of the segment DIRECTLY BELOW it, or
// -1 if the query is below everything. That single
   ↪ answer is the whole primitive: label every
// segment with the id of the face lying immediately
   ↪ ABOVE it and the query's face falls out.
// WHEN: "n disjoint polygons, q points, which
   ↪ polygon contains each" with n and q both 1e5, so
// the O(nq) scan is dead. Also "which cell of an
   ↪ arrangement", and nearest-site queries once you
// have built the Voronoi diagram (14). For ONE
   ↪ convex polygon use inConvex (03) instead; for a
// handful of polygons just call inPoly (03) on each.
// COMPLEXITY: O((n + q) log n) time, O(n + q)
   ↪ memory. Everything is offline - all queries must
   ↪ be
// known up front. (Online needs a persistent segment
   ↪ tree over the vertical slabs, which is
// O(n log n) memory and rarely worth typing.)
// DEGENERATE: NO VERTICAL SEGMENTS - the comparator
   ↪ below assumes every segment points strictly
// rightwards. If you have them, shear the plane with
   ↪ (x, y) -> (x * (2C + 1) + y, y), which is
// exact on integers and makes all x distinct, but
   ↪ squares your coordinates (use __int128 cross).
// A query exactly ON a segment reports that segment
```

```
  ↪ (endpoints included), because segments
// starting at x are inserted before the queries at x
  ↪ and segments ending at x are removed after.
// Zero-length segments must be filtered out first.
// EXACT on P<ll> with |coord| <= 1e9; the comparator
  ↪ is pure orientation signs, no division.
vector<int> locateBelow(vector<pair<Pi, Pi>> s, const
  ↪ vector<Pi>& qs) {
    int n = sz(s), m = sz(qs);
    for (auto& e : s) if (e.second < e.first)
  ↪ swap(e.first, e.second);
    s.pb({Pi(), Pi()});
  ↪   // slot n: the current query, as a point
    // Two non-crossing segments are comparable
  ↪ wherever their x-ranges overlap: ask which side
  ↪ of
    // a's line BOTH endpoints of b fall on, and if
  ↪ that is inconclusive (shared endpoint) ask the
    // mirror question. Only ever consulted while
  ↪ both are in the sweep set, so this is a valid
    // strict weak ordering at every instant even
  ↪ though it changes as the sweep advances.
    auto below = [&](int i, int j) {
        auto& [a, b] = s[i]; auto& [c, d] = s[j];
        int v = sgn(a.cross(b, c)) + sgn(a.cross(b,
  ↪ d));
        if (v) return v > 0;
        return sgn(c.cross(d, a)) + sgn(c.cross(d,
  ↪ b)) < 0;
    };
    set<int, decltype(below)> t(below);
    vector<int> add(n), del(n), qi(m), res(m, -1);
    iota(all(add), 0), iota(all(del), 0),
  ↪ iota(all(qi), 0);
    sort(all(add), [&](int i, int j) { return
  ↪ s[i].first.x < s[j].first.x; });
    sort(all(del), [&](int i, int j) { return
  ↪ s[i].second.x < s[j].second.x; });
    sort(all(qi), [&](int i, int j) { return qs[i].x
  ↪ < qs[j].x; });
    int ia = 0, id = 0, iq = 0;
    while (ia < n || id < n || iq < m) {
  ↪   // sweep EVERY event x, not just query x:
        ll x = LLONG_MAX;
  ↪   // the set order is only valid while all
        if (ia < n) x = min(x, s[add[ia]].first.x);
  ↪   // members still span the sweep line
        if (id < n) x = min(x, s[del[id]].second.x);
        if (iq < m) x = min(x, qs[qi[iq]].x);
        while (ia < n && s[add[ia]].first.x == x)
  ↪ t.insert(add[ia++]);       // open,
        while (iq < m && qs[qi[iq]].x == x) {
  ↪                          // then answer,
            int k = qi[iq++];
            s[n] = {qs[k], qs[k]};
  ↪   // a zero-length segment = the query point
            auto it = t.lower_bound(n);
            if (it != t.begin()) res[k] = *prev(it);
        }
        while (id < n && s[del[id]].second.x == x)
  ↪ t.erase(del[id++]);       // then close
    }
    return res;
}
// ATTACHING FACES. Build the subdivision as
  ↪ half-edges: every segment carries the face id
  ↪ above
// it and the face id below it (the unbounded face is
  ↪ -1). Then:
//   f = locateBelow(...)[k];  answer = f < 0 ? -1 :
  ↪ faceAbove[f];
// and the query is ON the boundary iff
  ↪ qs[k].cross(s[f].first, s[f].second) == 0.
// For DISJOINT SIMPLE POLYGONS the labelling is
  ↪ free: walk each polygon in CCW order and give
// every edge whose direction points LEFT (a.x > b.x)
  ↪ the polygon's id as its "above" face, and
// every rightward edge the id of whatever the
  ↪ polygon sits in (-1 for a top-level polygon).
// ARRANGEMENT OF LINES: n lines cut the plane into 1
  ↪ + n + C(n,2) faces; build the arrangement by
// sorting the intersections along each line, then
  ↪ run this on the resulting O(n^2) segments.
// A different, shorter answer to "point in one of
  ↪ many disjoint polygons": if the polygons are
// convex and you only need containment, sort by
  ↪ x-range and binary search, or build the union
// hull. Reach for point location only when the
  ↪ subdivision is genuinely a subdivision.
```

11.1.16 DynamicHull

DYNAMIC CONVEX HULL - insert points ONLINE, ask "is this point inside the hull so far" in

```
// Needs: Core.cpp.
// O(log n), amortised O(log n) per insert. The
  ↪ structure below is the UPPER hull only, stored as
// a map x -> max y; run a second copy fed the
  ↪ negated points for the lower hull and you have
  ↪ the
// full hull. Storing one y per x is what makes this
  ↪ short: the upper chain has strictly
// increasing x by definition, and if two points
  ↪ share an x only the higher one can be on it.
// WHEN: points arrive one at a time and you must
  ↪ answer containment / support / area queries
// between insertions; "add point, print hull area";
  ↪ maximising a linear function over a growing
// set (that special case is CHT / Li Chao, 08 - DP
  ↪ Optimizations, and is simpler - use it if the
// query is always a dot product). Deletions are NOT
  ↪ supported; for those, use offline divide and
// conquer over time, or Kinetic /
  ↪ segment-tree-on-time rebuilding.
// COMPLEXITY: O(log n) amortised insert (each point
  ↪ is erased at most once), O(log n) query,
// O(n) memory.
// DEGENERATE: COLLINEAR points are dropped (the
  ↪ chain keeps only strict turns), so a set of
// collinear points collapses to its two extremes and
  ↪ under() still answers correctly. Inserting a
// point already inside is a no-op. An empty
  ↪ structure answers false to everything.
// EXACT on integers: the only arithmetic is a cross
  ↪ product, so |coord| <= 1e9 keeps it in ll.
struct UpHull {
  ↪   // the upper chain, left to right
    map<ll, ll> h;
    typedef map<ll, ll>::iterator it_t;
    Pi at(it_t i) { return Pi(i->fi, i->se); }
    bool under(ll x, ll y) {
  ↪   // is (x, y) on or below the chain?
        auto it = h.lower_bound(x);
        if (it == h.end() || (it == h.begin() &&
  ↪ it->fi != x)) return false;   // outside the span
        if (it->fi == x) return y <= it->se;
        return at(prev(it)).cross(at(it), Pi(x, y))
```

```
    <= 0;                       // not strictly left
    }
    bool bad(it_t it) {
    // does *it sit on or below its neighbours?
        if (it == h.begin() || next(it) == h.end())
return false;
        return at(prev(it)).cross(at(next(it)),
at(it)) <= 0;
    }
    void add(ll x, ll y) {
        if (under(x, y)) return;
        h[x] = y;
        auto it = h.find(x);
        while (next(it) != h.end() && bad(next(it)))
h.erase(next(it));
        while (it != h.begin() && bad(prev(it)))
h.erase(prev(it));
    }
};
// FULL HULL: UpHull up, dn;  insert p as up.add(p.x,
  p.y) and dn.add(-p.x, -p.y);
//   inside(p) = up.under(p.x, p.y) && dn.under(-p.x,
  -p.y).
// EXTREME POINT in direction d, O(log n): binary
  search the upper chain for the first edge whose
// direction turns from d-positive to d-negative,
  i.e. the last vertex with
// dot(next - cur, d) > 0. With a map you have to
  walk, so keep the chain in a
// vector-backed balanced BST (or an ordered set with
  order_of_key) if you need this a lot.
// AREA UNDER INSERTIONS, the usual companion query:
  keep ll s2 = sum of cross(p_i, p_{i+1}) over
// consecutive chain vertices. Every erase of a
  middle vertex b between a and c does
//   s2 -= cross(a,b) + cross(b,c) - cross(a,c);
// and every insert of b between a and c does the
  same with a plus sign; erases and inserts at an
// end touch only the single adjacent term. The
  hull's area is |s2_up + s2_down| / 2 with the
// lower chain's points negated, since negating both
  coordinates leaves every cross product alone.
// OFFLINE ALTERNATIVE, almost always easier: if you
  know all the points up front and the queries
// are interleaved, sort by time and rebuild the hull
  in O(n log n) once per sqrt-block, or run
// divide and conquer on the time axis. Only type
  this file when the input is genuinely online.
```

11.1.17 ShamosHoey

SHAMOS-HOEY: do ANY TWO of n segments intersect? O(n log n), and it returns the witness pair.

```
// Needs: Core.cpp, Lines.cpp (segCross).
// The sweep keeps the active segments in a set
  ordered by their y at the sweep line. Two
  segments
// can only be the FIRST crossing pair if they are
  adjacent in that order immediately before they
// cross, so it is enough to test a segment against
  its two neighbours when it is inserted, and to
// test the two segments that become neighbours when
  one is removed.
// WHEN: "is this polygon simple", "do these n
  roads/walls cross", "is this a planar graph
// drawing" - anything where you only need YES/NO or
  one example, with n up to 1e5 so the O(n^2)
// double loop (05 - Sweep.cpp, countSegInter) is
  out. isSimple (11 - PolygonOps.cpp) is the
// polygon-shaped special case and is O(n^2); replace
  it with this when n is large.
// COMPLEXITY: O(n log n) time, O(n) memory.
// DEGENERATE: touching at a shared endpoint COUNTS
  as an intersection (segCross is inclusive), so
// for "is this polygon simple" you must skip the
  adjacent pairs yourself - easiest is to shrink
// every polygon edge by a hair, or to special-case
  the pair (i, i+1 mod n) in the answer.
// Overlapping collinear segments are reported.
  Zero-length segments are reported against
  anything
// they touch. Vertical segments are fine here
  (unlike point location).
// The ANSWER (which pair) is exact - segCross is
  pure integer orientation. Only the SET ORDER
// uses floating point, and only to decide adjacency,
  so an eps slip can at worst make the sweep
// miss a crossing whose two segments are within 1e-9
  of each other in y; with |coord| <= 1e9 and
// ld that does not happen for integer input.
struct SSeg {
    Pi a, b; int id;
    // normalised so that a.x <= b.x
    ld yAt(ld x) const {
        return a.x == b.x ? (ld)a.y : a.y + (ld)(b.y
- a.y) * (x - a.x) / (b.x - a.x);
    }
    bool operator<(const SSeg& o) const {
    // compare where both are already alive
        ld x = max((ld)a.x, (ld)o.a.x);
        return yAt(x) < o.yAt(x) - eps;
    }
};
pair<int, int> anyIntersect(const vector<pair<Pi,
  Pi>>& in) {
    int n = sz(in);
    vector<SSeg> s(n);
    vector<array<ll, 3>> ev;
    // {x, +1 open / -1 close, id}
    for (int i = 0; i < n; i++) {
        s[i] = {in[i].fi, in[i].se, i};
        if (s[i].b < s[i].a) swap(s[i].a, s[i].b);
        ev.pb({s[i].a.x, 1, i}), ev.pb({s[i].b.x, -1,
i});
    }
    sort(all(ev), [](const array<ll, 3>& p, const
array<ll, 3>& q) {
        return p[0] != q[0] ? p[0] < q[0] : p[1] >
q[1];               // open before close
    });
    set<SSeg> t;
    vector<set<SSeg>::iterator> pos(n);
    typedef set<SSeg>::iterator sit;
    auto hit = [&](sit x, sit y) {
        return x != t.end() && y != t.end() &&
segCross(x->a, x->b, y->a, y->b);
    };
    for (auto& e : ev) {
        int i = (int)e[2];
        if (e[1] > 0) {
            sit nx = t.lower_bound(s[i]), pv = nx ==
t.begin() ? t.end() : prev(nx);
            sit me = t.insert(nx, s[i]);
    // insert first, then test both neighbours
            if (hit(nx, me)) return {nx->id, i};
            if (hit(pv, me)) return {pv->id, i};
            pos[i] = me;
```

```
        } else {
            sit nx = next(pos[i]), pv = pos[i] ==
  ↪ t.begin() ? t.end() : prev(pos[i]);
            if (hit(nx, pv)) return {pv->id, nx->id};
  ↪   // they become adjacent once i leaves
            t.erase(pos[i]);
        }
    }
    return {-1, -1};
}
// COUNT or REPORT ALL k intersections:
  ↪ BENTLEY-OTTMANN, O((n + k) log n). Same sweep,
  ↪ but the
// event queue is a priority queue that also receives
  ↪ the intersection points themselves; when two
// segments swap order at a crossing you re-test each
  ↪ against its new neighbour. It is long,
// fiddly with degeneracies (three segments through
  ↪ one point), and almost never what a problem
// wants - if you need a COUNT of crossing pairs of
  ↪ ARBITRARY segments and n is 1e5, look for
// extra structure (all horizontal/vertical -> BIT
  ↪ sweep; all chords of a circle -> inversions).
// IS A POLYGON SIMPLE, in O(n log n): shrink every
  ↪ edge towards its midpoint by a factor
// (1 - 1e-9) so consecutive edges no longer share
  ↪ their endpoint, then this returns {-1, -1}
// exactly for a simple polygon. Shrinking is the
  ↪ honest fix; filtering "adjacent indices" out of
// the answer is not, because the sweep stops at the
  ↪ FIRST adjacent pair it meets.
```

11.1.18 InscribedCircle

LARGEST INSCRIBED CIRCLE IN A CONVEX POLYGON - the CHEBYSHEV CENTRE. Returns {centre, radius}.

```
// Needs: Core.cpp, Polygon.cpp (area2).
// The trick is that f(p) = min over edges of the
  ↪ signed distance from p to that edge's LINE is a
// minimum of affine functions, hence CONCAVE on the
  ↪ whole plane, and it is exactly the distance
// to the boundary for p inside. So a nested ternary
  ↪ search on (x, y) finds its maximum with no
// case analysis at all: no medial axis, no
  ↪ half-plane shrinking, no binary search on r.
// WHEN: "place a disc of maximum radius inside this
  ↪ region", "the point of a convex region
// farthest from its boundary", robot clearance, and
  ↪ 2D LP with an objective that is a min of
// linear functions (this IS that LP - maximise r
  ↪ subject to n linear constraints).
// COMPLEXITY: O(n * it^2) with it = 100 iterations
  ↪ per axis; about 1e4 * n operations, so keep n
// below a few thousand. If n is large, run the same
  ↪ search on the O(n) half-plane form or binary
// search r and test emptiness with halfPlaneInter
  ↪ (06) at O(n log n log(1/eps)).
// DEGENERATE: the polygon must be CONVEX and given
  ↪ in order; the code reverses a CW input for
// you. A degenerate (zero-area) polygon returns
  ↪ radius 0 at some point of it. Fewer than 3
// vertices returns radius 0. For a NON-CONVEX
  ↪ polygon f is not concave and this silently
  ↪ returns
// a local optimum - the correct answer there lives
  ↪ on the medial axis and is genuinely hard.
// FLOATING POINT, always: the answer is irrational.
  ↪ 100 ternary iterations over a 2e9-wide box
// leave the centre accurate to ~1e-9 * box, which is
  ↪ the best ld can do anyway.
pair<Pd, ld> inscribedCircle(vector<Pd> v) {
    int n = sz(v);
    if (n < 3) return {n ? v[0] : Pd(), 0};
    if (area2(v) < 0) reverse(all(v));
  ↪   // f must be POSITIVE inside
    auto f = [&](ld x, ld y) {
  ↪   // distance to the nearest edge LINE
        ld r = 1e18;
        for (int i = 0; i < n; i++)
            r = min(r, v[i].cross(v[(i + 1) % n],
  ↪ Pd(x, y)) / (v[(i + 1) % n] - v[i]).dist());
        return r;
    };
    ld lo = 1e18, hi = -1e18, bo = 1e18, bi = -1e18;
    for (Pd& p : v) lo = min(lo, p.x), hi = max(hi,
  ↪ p.x), bo = min(bo, p.y), bi = max(bi, p.y);
    auto best = [&](ld x) {
  ↪   // max over y for a fixed x, still concave
        ld a = bo, b = bi;
        for (int it = 0; it < 100; it++) {
            ld m1 = a + (b - a) / 3, m2 = b - (b - a)
  ↪ / 3;
            f(x, m1) < f(x, m2) ? a = m1 : b = m2;
        }
        return (a + b) / 2;
    };
    for (int it = 0; it < 100; it++) {
        ld m1 = lo + (hi - lo) / 3, m2 = hi - (hi -
  ↪ lo) / 3;
        f(m1, best(m1)) < f(m2, best(m2)) ? lo = m1 :
  ↪ hi = m2;
    }
    ld x = (lo + hi) / 2, y = best(x);
    return {Pd(x, y), max((ld)0, f(x, y))};
}
// WHY THE NESTED SEARCH IS LEGAL: g(x) = max over y
  ↪ of f(x, y) is the partial maximisation of a
// concave function, and that is concave in x. The
  ↪ same argument licenses geoMedian (12) and every
// other nested ternary search - check concavity of
  ↪ the PARTIAL max, not just of f.
// LP FORM, if you prefer it or need n large:
  ↪ maximise r subject to
//   cross(b_i - a_i, p - a_i) / |b_i - a_i| >= r
  ↪ for every edge,
// which is n constraints in three variables (x, y,
  ↪ r) - Seidel's randomised LP solves it in O(n)
// expected. Equivalently, binary search r, push
  ↪ every edge line inward by r, and ask whether the
// half-plane intersection is non-empty.
// LARGEST INSCRIBED CIRCLE CENTRED ON A GIVEN LINE:
  ↪ one ternary search along the line.
// LARGEST INSCRIBED AXIS-ALIGNED RECTANGLE is a
  ↪ different and much harder problem - it needs a
// sweep with a stack and is not a two-line variation
  ↪ of this. Do not improvise it under time.
// MINIMUM ENCLOSING CIRCLE is mec (04 -
  ↪ Circles.cpp); minimum enclosing rectangle is
  ↪ minRectArea
// (03 - Polygon.cpp) by rotating calipers.
```

11.1.19 LineClip

TOTAL LENGTH OF A LINE INSIDE A POLYGON, including NON-CONVEX and self-intersecting ones, in

```
// Needs: Core.cpp.
// O(n log n). The line a -> b generally enters and
   ↪ leaves the region many times; this sums all of
// the pieces. Works by collecting the parameters t
   ↪ at which the line crosses each edge, tagging
// each crossing with the direction the boundary
   ↪ sweeps past it, sorting, and integrating the
// stretches where the running WINDING NUMBER is
   ↪ non-zero. cut() (03 - Polygon.cpp) only handles
// the convex case and only gives you one side; this
   ↪ gives you the measure.
// WHEN: "how much of this laser / road / sight line
   ↪ is inside the region", area of a polygon
// swept by a moving line, and any integral over a
   ↪ region done by integrating over parallel lines
// (fix a direction, sweep the offset, and this is
   ↪ the inner integrand).
// COMPLEXITY: O(n log n) for the sort, O(n) memory.
// DEGENERATE: a vertex lying exactly ON the line is
   ↪ handled by treating a zero orientation as
// POSITIVE, which is the same as nudging the line
   ↪ infinitesimally toward its right; that is
// consistent, so a vertex merely touching the line
   ↪ contributes nothing and a vertex the boundary
// genuinely passes through contributes once. An EDGE
   ↪ lying along the line is skipped for the same
// reason - if you need boundary-along-the-line to
   ↪ count, add its length yourself.
// SELF-INTERSECTING input is fine and uses the
   ↪ non-zero winding rule (windingNumber, 11), so a
// doubly-wound region counts once, not twice.
   ↪ Orientation (CW or CCW) does not matter.
// The crossing TEST is exact on P<ll> (|coord| <=
   ↪ 1e9); only the parameter t is floating point,
// so nearly-tangent edges can shift a boundary by
   ↪ ~1e-9 relative.
template <class P> ld lineInPoly(const vector<P>& v,
   ↪ P a, P b) {
    P d = b - a;
    int n = sz(v);
    vector<pair<ld, int>> ev;
    for (int i = 0; i < n; i++) {
        P p = v[i], q = v[(i + 1) % n];
        int s1 = sgn(d.cross(p - a)), s2 =
   ↪ sgn(d.cross(q - a));
        if (!s1) s1 = 1;
   ↪   // a point ON the line counts as LEFT
        if (!s2) s2 = 1;
        if (s1 == s2) continue;
        ev.pb({(ld)(p - a).cross(q - p) / d.cross(q -
   ↪ p), s1 > s2 ? 1 : -1});
    }
    sort(all(ev));
    ld len = 0;
    int c = 0;
    for (int i = 0; i + 1 < sz(ev); i++)
        if ((c += ev[i].second)) len += ev[i + 1].fi
   ↪ - ev[i].fi;        // inside <=> winding != 0
    return len * d.dist();
}
// SEGMENT instead of a line: same events, but clamp
   ↪ the accumulated interval to [0, 1] before
// adding, i.e. len += max((ld)0, min((ld)1,
   ↪ ev[i+1].fi) - max((ld)0, ev[i].fi)).
// RAY: clamp to [0, infinity) the same way.
// THE PIECES THEMSELVES, not just their total
   ↪ length: emit the interval [ev[i].fi, ev[i+1].fi]
// whenever the running count is non-zero; a + d * t
   ↪ gives the endpoints.
// AREA OF THE POLYGON, as a sanity check on this
   ↪ file: integrate lineInPoly over a family of
// parallel lines. That is also how you get "the area
   ↪ of the part of a polygon on one side of an
// arbitrary curve" when the curve is not a line and
   ↪ cut() does not apply.
// LINE x CONVEX POLYGON in O(log n): lineHull (11 -
   ↪ PolygonOps.cpp) gives the two crossed edges
// directly; there is exactly one interval and you do
   ↪ not need this file.
// LINE x CIRCLE is circleLine (04); LINE x DISC
   ↪ intersected with a polygon: clip against both.
```

11.1.20 MaxAreaPolygon

MAXIMUM-AREA POLYGON WITH GIVEN SIDE LENGTHS. Given n side lengths in any order, the polygon

```
// of largest area that has exactly those sides is
   ↪ CYCLIC - all its vertices lie on one circle
// (and the order of the sides does not matter, which
   ↪ is the surprising half of the theorem). So
// the whole problem is "find the circumradius R",
   ↪ and each side a_i subtends a central angle
// 2 * asin(a_i / (2R)). Two cases: if the centre is
   ↪ INSIDE, the angles sum to 2 * pi; if the
// longest side is long enough to push the centre
   ↪ OUTSIDE, its angle is subtracted instead and the
// signed sum is 0. Both are monotone in R, so binary
   ↪ search.
// WHEN: "arrange these n sticks into the polygon of
   ↪ largest area"; the n = 3 case is Heron and
// the n = 4 case is Brahmagupta, and this is the
   ↪ general answer for both. Also the standing proof
// that a regular polygon maximises area among
   ↪ equilateral ones.
// COMPLEXITY: O(n * 100) - one binary search with
   ↪ 100 iterations, each O(n).
// DEGENERATE: no polygon exists at all unless the
   ↪ longest side is strictly shorter than the sum
// of the others (the polygon inequality); the
   ↪ routine returns -1 for that, and for n < 3. Sides
// of length 0 are harmless. All sides equal gives
   ↪ the regular n-gon, area n r^2 sin(2 pi / n) / 2
// with r = a / (2 sin(pi / n)) - use that to check
   ↪ the code.
// FLOATING POINT, necessarily: R is irrational. asin
   ↪ loses precision when a_i / (2R) is near 1,
// which is exactly the degenerate "longest side is a
   ↪ diameter" case, so clamp the argument.
ld maxAreaPolygon(vector<ld> a) {
    int n = sz(a);
    if (n < 3) return -1;
    int m = max_element(all(a)) - a.begin();
    ld s = 0;
    for (int i = 0; i < n; i++) if (i != m) s += a[i];
    if (a[m] >= s) return -1;
   ↪   // the polygon inequality fails
    auto ang = [&](ld R, int sg) {
   ↪   // sum of central angles, a[m] signed
        ld t = 0;
        for (int i = 0; i < n; i++) {
            ld u = 2 * asinl(min((ld)1, a[i] / (2 *
   ↪ R)));
            t += (i == m ? sg * u : u);
        }
```

```
        return t;
    };
    ld lo = a[m] / 2, hi = 1e18;
    // R >= a[m] / 2 always
    int sg = ang(lo, 1) > 2 * PI ? 1 : -1;
    // centre inside (+1) or outside (-1)
    ld tgt = sg > 0 ? 2 * PI : 0;
    // ang(.) is DECREASING in R for sg = +1,
    for (int it = 0; it < 200; it++) {
    // INCREASING for sg = -1
        ld mid = (lo + hi) / 2;
        ((ang(mid, sg) > tgt) == (sg > 0) ? lo : hi)
= mid;
    }
    ld R = (lo + hi) / 2, area = 0;
    for (int i = 0; i < n; i++) {
        ld u = 2 * asinl(min((ld)1, a[i] / (2 * R)));
        area += (i == m ? sg : 1) * R * R * sinl(u) /
2;
    }
    return area;
}
// THE VERTICES, if the problem wants them and not
   just the area: walk the angles. Start at
// (R, 0), and after each side rotate by its central
   angle (negated for the long side when the
// centre is outside). p_0 = R * (cos 0, sin 0),
   theta_{k+1} = theta_k + u_k, p_k = R * (cos,
   sin).
// TRIANGLE (n = 3), closed form, no search: HERON
   with the stable sorted form,
//   sort a >= b >= c, A =
   sqrt((a+(b+c))(c-(a-b))(c+(a-b))(a+(b-c))) / 4.
// CYCLIC QUADRILATERAL (n = 4): BRAHMAGUPTA, A =
   sqrt((s-a)(s-b)(s-c)(s-d)) with s the
// semiperimeter - and by this theorem that IS the
   maximum over all quadrilaterals with those
// sides. General quadrilateral with diagonals p, q
   (BRETSCHNEIDER):
//   A = sqrt(4 p^2 q^2 - (b^2 + d^2 - a^2 - c^2)^2)
   / 4.
// MAXIMUM AREA WITH GIVEN PERIMETER, no side
   constraints: the circle. Among n-gons: the
   regular
// one. This file is the constrained version of the
   same isoperimetric fact.
// A DIFFERENT PROBLEM often confused with this one:
   pick the triangle of maximum area from a set
// of given points - that is rotating calipers on the
   hull, O(n^2), and is not here.
```

11.1.21 DynamicHalfPlane

DYNAMIC HALF-PLANE INTERSECTION - a convex region you can CUT online, in O(log n) amortised per

```
// Needs: Core.cpp (angCmp).
// cut, while asking for its area or its extreme
   point in any direction between cuts. The static
// routine (06 - HalfPlaneIntersection.cpp) needs
   every constraint up front and re-sorts; this one
// does not, so it is what you want when the
   constraints arrive one at a time, or when you
   must
// report an answer after each one.
// REPRESENTATION: the region's boundary, as a
   multimap from EDGE DIRECTION to the vertex where
// that edge begins, in counter-clockwise angular
   order. That single choice is the whole trick -
// the edge a new cut attacks is found by one
   lower_bound on its direction, and every vertex is
// erased at most once, so the amortised cost is a
   lookup plus the erases you paid to insert.
// It starts as a huge box, so the region is always
   bounded; make B larger than any coordinate you
// care about and remember that a surviving box
   corner is an artefact, not an answer.
// WHEN: "after each of q constraints, print the area
   of the feasible region"; online 2D LP;
// incremental visibility / lit-region problems;
   kernel of a polygon built edge by edge.
// COMPLEXITY: O(log n) amortised per cut, O(log n)
   per extreme-point query, O(n) per area query
// (keep a running sum of cross products if you need
   area after every cut - update it in the same
// places the code erases and inserts).
// DEGENERATE: cutting with a line that misses the
   region entirely is a no-op; cutting everything
// away leaves the map EMPTY and every later cut
   returns immediately, so check empty() for
// infeasible. Duplicate directions are allowed (that
   is why it is a multimap). A cut exactly
// along an existing edge removes the now-redundant
   vertex.
// FLOATING POINT: directions and vertices are ld and
   compared with the usual eps through sgn.
// Feed it integer-valued Pd if you can; a cut that
   is within eps of tangent may keep or drop a
// zero-length edge, so filter edges shorter than eps
   before counting them.
struct DirLess { bool operator()(Pd a, Pd b) const {
   return angCmp<ld>(a, b); } };
struct HPSet {
    typedef multimap<Pd, Pd, DirLess> M;
    typedef M::iterator it_t;
    M m;
    // edge direction -> start vertex of edge
    HPSet(ld B = 1e9) {
        m.insert({Pd(1, 0), Pd(-B, -B)}),
    m.insert({Pd(0, 1), Pd(B, -B)});
        m.insert({Pd(-1, 0), Pd(B, B)}),
    m.insert({Pd(0, -1), Pd(-B, B)});
    }
    it_t nx(it_t i) { return next(i) == m.end() ?
    m.begin() : next(i); }
    it_t pv(it_t i) { return i == m.begin() ?
    prev(m.end()) : prev(i); }
    it_t fix(it_t i) { return i == m.end() ?
    m.begin() : i; }
    void cut(Pd a, Pd b) {
    // keep only the part LEFT of a -> b
        if (m.empty()) return;
        int old = sz(m);
        Pd d = b - a;
        auto ev = [&](it_t i) { return sgn(a.cross(b,
    i->second)); };
        auto ix = [&](it_t i) {
    // where i's edge meets the line a -> b
            return i->second + i->first * (d.cross(a
    - i->second) / d.cross(i->first));
        };
        it_t it = fix(m.lower_bound(d));
    // the vertex farthest RIGHT of d
        if (ev(it) >= 0) return;
    // it survives, so everything does
        while (sz(m) && ev(pv(it)) < 0)
    m.erase(pv(it));
        while (sz(m) && ev(nx(it)) < 0) it =
```

```
    fix(m.erase(it));
        if (m.empty()) return;
        if (ev(nx(it)) > 0) it->second = ix(it);
    // pull this vertex onto the cutting line
        else it = fix(m.erase(it));
        if (old <= 2) return;
        it = pv(it);
        m.insert(it, {d, ix(it)});
    // the new edge starts where we re-entered
        if (ev(it) == 0) m.erase(it);
    // the old vertex is now redundant
    }
    Pd extreme(Pd dir) {
    // vertex maximising dot(v, dir), O(log n)
        return fix(m.lower_bound(dir.perp()))->second;
    }
    ld area() {
        if (sz(m) <= 2) return 0;
        ld s = 0;
        for (it_t i = m.begin(); i != m.end(); ++i) s
    += i->second.cross(nx(i)->second);
        return fabsl(s) / 2;
    }
    vector<Pd> poly() {
    // the region as a CCW vertex list
        vector<Pd> v;
        for (auto& e : m) v.pb(e.second);
        return v;
    }
};
// USAGE: HPSet h; then h.cut(p, q) for each
  constraint "left of the directed line p -> q",
  which
// is the same convention as HP in 06. To cut with
  the half-plane {x : n.dot(x) <= c}, take a
// point p on the boundary line and cut(p, p +
  n.perp()).
// AREA AFTER EVERY CUT in O(log n): keep ld s = sum
  of cross(v_i, v_{i+1}). Every erase of a
// vertex b between a and c does s -= cross(a,b) +
  cross(b,c) - cross(a,c), every insert adds it,
// and moving a vertex is an erase plus an insert.
  Same bookkeeping as 16 - DynamicHull.cpp.
// 2D LP ONLINE: after each constraint the optimum of
  a linear objective c is extreme(c) - one
// lower_bound, no scan. That is the reason to prefer
  this over rebuilding with 06.
// DELETIONS are not supported. If constraints are
  added AND removed, go offline: segment tree on
// time, and rebuild with the static routine in each
  node.
```

11.1.22 OnionDecomposition

ONION DECOMPOSITION (convex layers) in O(n log^2 n): strip the convex hull, then the hull of

```
// Needs: Core.cpp.
// what remains, and so on, and report which layer
  each point lands in. The naive loop in
// 05 - Sweep.cpp rebuilds a hull per layer and is
  O(n^2 log n), which dies at n = 1e5 on the
// worst input (points on nested circles give
  Theta(n^(2/3)) layers, and a spiral gives
  Theta(n)).
// THE STRUCTURE: a DECREMENTAL hull - it only
  supports DELETE, which is exactly what peeling
// needs. A segment tree over the points sorted by y
  stores, in every node, the two endpoints of
// the BRIDGE joining the hulls of its two children;
  pull() finds that bridge by the classic
// simultaneous descent in O(log n), so a delete
  costs O(log^2 n) and reading off the current hull
// costs O(hull size). Two copies handle the two
  sides: the LEFT hull of the points, and the left
// hull of the negated-and-reversed points, which is
  the right hull.
// WHEN: "how many peels until the set is empty",
  "which layer is point i on", k-th layer queries,
// and the classic "repeatedly remove the hull"
  simulation. Also the fastest route to the DEPTH
  of
// a point set (the number of layers), which some
  rotating-calipers-free arguments need.
// COMPLEXITY: O(n log^2 n) time, O(n) memory. Beats
  the naive version from about n = 3000.
// DEGENERATE: ALL POINTS MUST BE DISTINCT - dedupe
  first. COLLINEAR points on a hull edge are
// KEPT on that layer (the bridge search uses strict
  turns, so a point in the middle of an edge is
// still reported), which differs from hull() in 03 -
  Polygon.cpp; if you need them dropped,
// filter each layer afterwards. n <= 2 puts
  everything on layer 1.
// EXACT on P<ll>: cross fits in ll for |coord| <=
  1e9, and the one product of a cross with a
// coordinate inside pull() is widened to __int128,
  so nothing here overflows at that budget.
struct LeftHull {
  // points must be sorted by (y, x)
    struct Nd { int bl, br, L, R, lc, rc; };
    vector<Pi> ps;
    vector<Nd> t;
    int root = 0;
    bool leaf(int w) { return t[w].lc < 0 && t[w].rc
  < 0; }
    void pull(int w) {
  // find the bridge between the two children
        int l = t[w].lc, r = t[w].rc;
        ll sy = ps[t[r].L].y;
        while (!leaf(l) || !leaf(r)) {
            int a = t[l].bl, b = t[l].br, c =
  t[r].bl, d = t[r].br;
            if (a != b && ps[a].cross(ps[b], ps[c]) >
  0) l = t[l].lc;
            else if (c != d && ps[b].cross(ps[c],
  ps[d]) > 0) r = t[r].rc;
            else if (a == b) r = t[r].lc;
            else if (c == d) l = t[l].rc;
            else {
                ll s1 = ps[a].cross(ps[b], ps[c]), s2
  = ps[b].cross(ps[a], ps[d]);
                if (s1 + s2 == 0 || (__int128)s1 *
  ps[d].y + (__int128)s2 * ps[c].y <
                                    (__int128)sy *
  (s1 + s2)) l = t[l].rc;
                else r = t[r].lc;
            }
        }
        t[w].bl = t[l].L, t[w].br = t[r].L;
    }
    void build(int w, int L, int R) {
        t[w].L = L, t[w].R = R;
        if (R - L == 1) { t[w].lc = t[w].rc = -1,
  t[w].bl = t[w].br = L; return; }
        int M = (L + R) / 2;
        t[w].lc = w + 1, t[w].rc = w + 2 * (M - L);
        build(t[w].lc, L, M), build(t[w].rc, M, R),
```

```
    pull(w);
    }
    int erase(int w, int L, int R) {
        if (R <= t[w].L || L >= t[w].R) return w;
        if (L <= t[w].L && R >= t[w].R) return -1;
        t[w].lc = erase(t[w].lc, L, R), t[w].rc =
    erase(t[w].rc, L, R);
        if (t[w].lc < 0) return t[w].rc;
        if (t[w].rc < 0) return t[w].lc;
        return pull(w), w;
    }
    void walk(int w, int l, int r, vector<int>& res) {
        if (leaf(w)) res.pb(t[w].L);
        else if (r <= t[w].bl) walk(t[w].lc, l, r,
    res);
        else if (l >= t[w].br) walk(t[w].rc, l, r,
    res);
        else walk(t[w].lc, l, t[w].bl, res),
    walk(t[w].rc, t[w].br, r, res);
    }
    LeftHull(const vector<Pi>& p) : ps(p),
    t(max<size_t>(2, p.size() * 2)) { build(0, 0,
    sz(p)); }
    void erase(int i) { root = erase(root, i, i + 1);
    }
    vector<int> hull() {
        vector<int> res;
        if (root >= 0) walk(root, 0, sz(ps) - 1, res);
        return res;
    }
};
// layer[i] = 1-based convex-layer index of the
    ORIGINAL point i. All points must be distinct.
vector<int> onionLayers(const vector<Pi>& p) {
    int n = sz(p);
    if (!n) return {};
    vector<int> ord(n), lay(n), res(n);
    iota(all(ord), 0);
    sort(all(ord), [&](int i, int j) { return
    tie(p[i].y, p[i].x) < tie(p[j].y, p[j].x); });
    vector<Pi> a(n), b(n);
    for (int i = 0; i < n; i++) a[i] = p[ord[i]];
    for (int i = 0; i < n; i++) b[i] = Pi(-a[n - 1 -
    i].x, -a[n - 1 - i].y);   // the right hull
    LeftHull L(a), R(b);
    for (int id = 1, cnt = 0; cnt < n; id++) {
        set<int> h;
        for (int i : L.hull()) h.insert(i);
        for (int i : R.hull()) h.insert(n - 1 - i);
        for (int i : h) cnt++, lay[i] = id,
    L.erase(i), R.erase(n - 1 - i);
    }
    for (int i = 0; i < n; i++) res[ord[i]] = lay[i];
    return res;
}
// The same structure answers "hull of the set after
    these deletions" at any moment, so it also
// does OFFLINE DELETIONS on a hull: process the
    deletes in order and read hull() in between.
// INSERTIONS instead of deletions are 16 -
    DynamicHull.cpp; you cannot have both here.
// LAYER COUNT bounds worth knowing: n points in
    convex position give 1 layer, a spiral gives n/3,
// and a uniform random square gives Theta(n^(2/3)) -
    so "peel until empty" is NOT O(sqrt n).
```

11.2 Geometry 3D

11.2.1 Geometry3D

3D point / vector, planes, and the few 3D things that actually appear.

```
template <class T>
struct P3 {
    typedef P3 Pt; T x = 0, y = 0, z = 0;
    P3(T x = 0, T y = 0, T z = 0) : x(x), y(y), z(z)
    {}
    Pt operator+(Pt p) const { return {x + p.x, y +
    p.y, z + p.z}; }
    Pt operator-(Pt p) const { return {x - p.x, y -
    p.y, z - p.z}; }
    Pt operator*(T d) const { return {x * d, y * d, z
    * d}; }
    Pt operator/(T d) const { return {x / d, y / d, z
    / d}; }
    T dot(Pt p) const { return x * p.x + y * p.y + z
    * p.z; }
    Pt cross(Pt p) const { return {y * p.z - z * p.y,
    z * p.x - x * p.z, x * p.y - y * p.x}; }
    T dist2() const { return x * x + y * y + z * z; }
    ld dist() const { return sqrtl((ld)dist2()); }
    Pt unit() const { return *this / dist(); }
    ld phi() const { return atan2l(y, x); }
                // longitude, (-pi, pi]
    ld theta() const { return atan2l(sqrtl((ld)(x * x
    + y * y)), z); }  // colatitude, [0, pi]
    Pt rot(ld a, Pt ax) const {
                // Rodrigues, ccw around ax
        Pt u = ax.unit(); ld s = sinl(a), c = cosl(a);
        return u * dot(u) * (1 - c) + (*this) * c +
    u.cross(*this) * s;
    }
    bool operator<(Pt p) const { return tie(x, y, z)
    < tie(p.x, p.y, p.z); }
    bool operator==(Pt p) const { return tie(x, y, z)
    == tie(p.x, p.y, p.z); }
    friend istream& operator>>(istream& i, Pt& p) {
    return i >> p.x >> p.y >> p.z; }
    friend ostream& operator<<(ostream& o, Pt p) {
    return o << p.x << ' ' << p.y << ' ' << p.z; }
};
typedef P3<ll> Pi3;
typedef P3<ld> Pd3;

// PLANE  n.dot(X) = d.   Build from 3 points, or
    from a normal and a point.
template <class P> struct Plane {
    P n; ld d;
    Plane(P n = P(), ld d = 0) : n(n), d(d) {}
    Plane(P a, P b, P c) : n((b - a).cross(c - a)),
    d(n.dot(a)) {}
    ld side(P p) const { return n.dot(p) - d; }
                // sign = which side
    ld dist(P p) const { return fabsl(side(p)) /
    n.dist(); }
    P proj(P p) const { return p - n * (side(p) /
    n.dist2()); }
    P refl(P p) const { return p - n * (2 * side(p) /
    n.dist2()); }
};
// LINE a->b vs plane. {0 parallel-disjoint, 1
    unique, -1 line lies in plane}
template <class P> pair<int, P> planeLine(const
    Plane<P>& pl, P a, P b) {
    ld da = pl.side(a), db = pl.side(b);
    if (fabsl(da - db) < eps) return {fabsl(da) < eps
```

```
    ? -1 : 0, P()};
    return {1, (b * da - a * db) / (da - db)};
}
// PLANE x PLANE -> a line, given as {point on it, direction}. dir==0 means parallel.
template <class P> pair<P, P> planePlane(const Plane<P>& A, const Plane<P>& B) {
    P dir = A.n.cross(B.n);
    if (dir.dist2() < eps) return {P(), P()};
    P p = (B.n * A.d - A.n * B.d).cross(dir) / dir.dist2();
    return {p, dir};
}

// VOLUME of the tetrahedron abcd, signed. The x6 form is exact on P3<ll> only up to
// |coordinate| ~ 5.7e5, since |vol6| <= 48 * C^3; above that make the return type __int128.
template <class P> auto vol6(P a, P b, P c, P d) { return (b - a).cross(c - a).dot(d - a); }
template <class P> ld tetraVolume(P a, P b, P c, P d) { return fabsl((ld)vol6(a, b, c, d)) / 6; }

// SPHERE x LINE: points where line a->b meets the sphere (o, r).
template <class P> vector<P> sphereLine(P o, ld r, P a, P b) {
    P d = b - a, f = a - o;
    ld A = d.dist2(), B = 2 * f.dot(d), C = f.dist2() - r * r, D = B * B - 4 * A * C;
    if (D < -eps) return {};
    D = sqrtl(max((ld)0, D));
    if (D < eps) return {a + d * (-B / (2 * A))};
    return {a + d * ((-B - D) / (2 * A)), a + d * ((-B + D) / (2 * A))};
}

// GREAT-CIRCLE distance on a sphere of radius R, inputs in DEGREES (haversine).
ld greatCircle(ld lat1, ld lon1, ld lat2, ld lon2, ld R) {
    auto rad = [](ld d) { return d * PI / 180; };
    ld dl = rad(lat2 - lat1) / 2, dg = rad(lon2 - lon1) / 2;
    ld h = sinl(dl) * sinl(dl) + cosl(rad(lat1)) * cosl(rad(lat2)) * sinl(dg) * sinl(dg);
    return 2 * R * asinl(sqrtl(min((ld)1, h)));
}
```

11.2.2 Lines3D

LINES, SEGMENTS AND TRIANGLES IN 3D. L3 is the line {o + t*d} through two points, carrying

```
// Needs: Geometry3D.cpp, Core.cpp (sgn).
// distance / projection / reflection of a point. On top of it the two things everyone re-derives
// under pressure: the SMALLEST DISTANCE BETWEEN TWO LINES (project the gap onto the common normal
// d1 x d2) and the PAIR OF CLOSEST POINTS realising it, one on each line. Then point-segment,
// segment-segment and point-triangle distance, and SEGMENT x TRIANGLE intersection (does segment
// d-e cross triangle abc, and where). Everything is O(1) time and memory.
// You reach for this the moment the input stops being planar: two 3D lines generically MISS, so
// lineInter is meaningless and "distance between the lines" replaces it; and mesh problems
// (visibility, is this ray blocked, closest approach of two moving points) are exactly segTri and
// segSegDist3.
// DEGENERATE: parallel lines fall back to point-line, and lineClosest then returns {a.o, its foot
// on b} - one of infinitely many optimal pairs. A zero-length segment degrades to its point. A
// collinear triangle makes triDist3 return the min over its three edges and makes segTri report
// -1. segTri with d == e reports 0 (or -1 if that point is in the plane of abc), never 1.
// EXACTNESS: the predicates are sgn(vol6), so segTri is EXACT on P3<ll> up to |coord| <= 5.7e5
// (the vol6 budget of Geometry3D.cpp; wrap vol6 in __int128 for 1e9). triDist3's inside test and
// L3::dist2 are DEGREE-4 forms (((v-u) x (p-u)).n and |d x (p-o)|^2), so they hold in ll only for
// |coord| <= 1.5e4 - again __int128 lifts that to 9e8. Anything returning a distance or a point
// divides, so it comes back in ld whatever you passed in; and note
// proj / refl / lineClosest silently TRUNCATE on P3<ll>. The eps tests below are absolute, so
// scale your input to O(1)..O(1e6) if you care about near-degenerate cases.
template <class P> struct L3 {
    P o, d;
    // {o + t*d}; d must be nonzero
    L3(P a = P(), P b = P()) : o(a), d(b - a) {}
    // point + DIRECTION: L3(o, o + dir)
    ld dist2(P p) const { return (ld)d.cross(p - o).dist2() / d.dist2(); }
    ld dist(P p) const { return sqrtl(dist2(p)); }
    // ratio of two exact integer forms
    P proj(P p) const { return o + d * ((ld)d.dot(p - o) / d.dist2()); }
    P refl(P p) const { return proj(p) * 2 - p; }
    bool cmpProj(P p, P q) const { return d.dot(p) < d.dot(q); }   // sort points ALONG the line
};
// SMALLEST DISTANCE BETWEEN TWO LINES: the gap vector measured along the common normal.
template <class P> ld lineLineDist(L3<P> a, L3<P> b) {
    P n = a.d.cross(b.d);
    if (n == P()) return a.dist(b.o);
    // parallel: any point of b will do
    return fabsl((ld)(b.o - a.o).dot(n)) / n.dist();
}
// The pair of points realising it, {on a, on b}. The point on a is where a pierces the plane
// spanned by b and the common normal, so the "closest point on a" needs only one division.
template <class P> pair<P, P> lineClosest(L3<P> a, L3<P> b) {
    P n = a.d.cross(b.d);
    if (n == P()) return {a.o, b.proj(a.o)};
    // parallel: infinitely many pairs
    P na = b.d.cross(n), nb = a.d.cross(n);
    return {a.o + a.d * ((ld)(b.o - a.o).dot(na) / a.d.dot(na)),
            b.o + b.d * ((ld)(a.o - b.o).dot(nb) / b.d.dot(nb))};
}
// POINT -> SEGMENT. Clamp the line parameter to [0, 1] and expand the norm, so no point of type
// P is ever constructed: this stays correct when you hand it P3<ll>.
template <class P> ld segDist3(P p, P a, P b) {
```

```
    P u = b - a, w = p - a;
    ld L = (ld)u.dist2(), uw = (ld)u.dot(w);
    if (L < eps) return w.dist();
  ↪       // a == b
    ld t = min((ld)1, max((ld)0, uw / L));
    return sqrtl(max((ld)0, (ld)w.dist2() - t * (2 *
  ↪ uw - t * L)));
}
// SEGMENT -> SEGMENT. The minimum is either at an
  ↪ endpoint or at the unconstrained closest pair
// of the two LINES, and that one only counts when
  ↪ both parameters land strictly inside (0, 1).
// den = |u|^2|v|^2 - (u.v)^2 loses absolute
  ↪ precision at huge coordinates, but a garbage (s,
  ↪ t)
// still names two REAL points of the segments, so it
  ↪ can only fail to improve the answer.
template <class P> ld segSegDist3(P a, P b, P c, P d)
  ↪ {
    P u = b - a, v = d - c, w = a - c;
    ld A = (ld)u.dist2(), B = (ld)u.dot(v), C =
  ↪ (ld)v.dist2();
    ld D = (ld)u.dot(w), E = (ld)v.dot(w), W =
  ↪ (ld)w.dist2(), den = A * C - B * B;
    ld r = min({segDist3(a, c, d), segDist3(b, c, d),
  ↪ segDist3(c, a, b), segDist3(d, a, b)});
    if (den > eps) {
      ld s = (B * E - C * D) / den, t = (A * E - B
  ↪ * D) / den;
      if (s > 0 && s < 1 && t > 0 && t < 1)
  ↪       // |w + s*u - t*v|^2, expanded
        r = min(r, sqrtl(max((ld)0, W + s * s * A
  ↪ + t * t * C
                                 + 2 * s * D - 2
  ↪ * t * E - 2 * s * t * B)));
    }
    return r;
  ↪       // parallel / collinear: endpoints win
}
// POINT -> TRIANGLE (the solid triangle). Project
  ↪ onto the plane; if the projection is inside,
// the plane distance is the answer, else the min
  ↪ over the three edges. The inside test uses p
// itself, NOT the projection: p and its projection
  ↪ differ by a multiple of n, and ((v-u) x n).n
// is 0, so the three orientations are unchanged -
  ↪ which keeps the test exact on integers.
template <class P> ld triDist3(P p, P a, P b, P c) {
    P n = (b - a).cross(c - a);
    ld e = min({segDist3(p, a, b), segDist3(p, b, c),
  ↪ segDist3(p, c, a)});
    if (n.dist2() < eps) return e;
  ↪       // collinear abc: no plane to hit
    auto f = [&](P u, P v) { return (v - u).cross(p -
  ↪ u).dot(n); };
    auto s1 = f(a, b), s2 = f(b, c), s3 = f(c, a);
    bool in = (s1 >= 0 && s2 >= 0 && s3 >= 0) || (s1
  ↪ <= 0 && s2 <= 0 && s3 <= 0);
    return in ? fabsl((ld)n.dot(p - a)) / n.dist() :
  ↪ e;      // no eps: the two branches AGREE on
}
  ↪       // the boundary, so either is right
// SEGMENT d-e x TRIANGLE a-b-c, by four orientation
  ↪ signs and nothing else.
// {1, p} they meet in the single point p (grazing a
  ↪ vertex or an edge counts);  {0, _} disjoint;
// {-1, _} d-e is COPLANAR with abc, where the meet
  ↪ can be empty, a point or a whole segment -
// map both to 2D with 03 - CoplanarFrame.cpp and
  ↪ clip there.
template <class P> pair<int, Pd3> segTri(P a, P b, P
  ↪ c, P d, P e) {
    auto va = vol6(a, b, c, d), vb = vol6(a, b, c, e);
    int sa = sgn(va), sb = sgn(vb);
    if (!sa && !sb) return {-1, Pd3()};
  ↪       // (also fires when abc is collinear)
    if (sa == sb) return {0, Pd3()};
  ↪       // segment strictly on one side
    int t1 = sgn(vol6(d, e, a, b)), t2 = sgn(vol6(d,
  ↪ e, b, c)), t3 = sgn(vol6(d, e, c, a));
    // The LINE d-e pierces the triangle iff it turns
  ↪ the same way around all three edges.
    if (!((t1 >= 0 && t2 >= 0 && t3 >= 0) || (t1 <= 0
  ↪ && t2 <= 0 && t3 <= 0))) return {0, Pd3()};
    Pd3 D((ld)d.x, (ld)d.y, (ld)d.z), E((ld)e.x,
  ↪ (ld)e.y, (ld)e.z);
    return {1, (E * (ld)va - D * (ld)vb) / ((ld)va -
  ↪ (ld)vb)};    // = planeLine, in vol6 terms
}
/* RELATED
 SEGMENT x PLANE          planeLine (01) then check
  ↪ the parameter lies in [0, 1].
 TRIANGLE -> TRIANGLE     min over the 6 (edge,
  ↪ triangle) pairs of segment-to-triangle distance;
                           that one is a ternary
  ↪ search over the segment parameter - the distance
                           from a point to a convex
  ↪ set is convex, so it is safe.
 SEGMENT -> TRIANGLE      same ternary search; 0
  ↪ exactly when segTri says 1.
 RAY x TRIANGLE           same four signs, but
  ↪ replace the sa != sb test with sa != 0 and sa
                           opposite to the side the
  ↪ ray points at (Moller-Trumbore is this test
                           with the divisions folded
  ↪ in - no reason to type it).
 TWO SEGMENTS MEET        in 3D only if coplanar:
  ↪ vol6(a, b, c, d) == 0, then use the 2D
                           segInter inside a coplanar
  ↪ frame. segSegDist3 == 0 is the cheap test.
 CLOSEST APPROACH of two points moving linearly =
  ↪ segSegDist3 of the two trajectories with a
                           shared parameter; if the
  ↪ speeds differ, minimise |w + t(v1 - v2)|^2
                           directly, t* = -w.(v1-v2) /
  ↪ |v1-v2|^2 clamped to the time window. */
```

11.2.3 CoplanarFrame

THE COPLANAR FRAME - the single most valuable file in 3D geometry, because it deletes 3D.

```
// Needs: Geometry3D.cpp, Core.cpp, Polygon.cpp.
// Given points known to lie on one plane, it builds
  ↪ a frame (o, dx, dy, dz) with dx, dy in the
// plane and dz along the normal, and maps each point
  ↪ to a 2D point: pos2d. From there EVERY 2D
// routine in the notebook (hull, area2, inPoly,
  ↪ inConvex, rotating calipers, cut, minkowski,
// Pick, ...) applies UNCHANGED, and to3d maps the
  ↪ answer back. pos3d keeps the third coordinate,
// which is the signed offset off the plane - 0
  ↪ exactly for the coplanar points.
// You reach for it whenever a statement says "a
  ↪ polygon in space", "the points lie on a common
// plane", or you have just cut a polyhedron by a
  ↪ plane and hold the cross-section as a bag of
// unordered 3D points. Building it is O(1); mapping
```

```
  ↪ is O(1) per point.
//
// TWO FRAMES, AND THE CHOICE IS THE WHOLE POINT.
// ORTHONORMAL (ortho = true, the default): dx, dy,
  ↪ dz are unit and mutually perpendicular, so the
//   map is a rigid motion - DISTANCES, ANGLES and
  ↪ AREAS are preserved exactly as they are in 3D.
//   It normalises, so it needs P3<ld> and inherits
  ↪ float error.
// AFFINE (ortho = false): dx = q - p and dy = r - p
  ↪ as they came in, no normalisation, so it
//   stays on P3<ll> and every 2D predicate
  ↪ downstream stays EXACT. It does NOT preserve
  ↪ distances
//   or angles - a square can come out as a
  ↪ parallelogram - but it is an invertible linear
  ↪ map of
//   the plane, so it preserves COLLINEARITY,
  ↪ CONVEXITY, ORIENTATION, ratios along a line and
//   inside/outside. That is exactly enough for hull,
  ↪ inPoly, inConvex, isConvex, orientation
//   tests and point location. Areas are multiplied
  ↪ by the constant |dx x dy| (the 2D area2 comes
//   out |dx x dy|^2 times the area in plane
  ↪ coordinates, and the true 3D area is the 2D area
//   divided by |dx x dy|), so ratios of areas
  ↪ survive too. Never feed the affine map to
  ↪ anything
//   metric: closest pair, minimum enclosing circle,
  ↪ width, perimeter, angle sorting.
// DEGENERATE: p, q, r must not be collinear (both
  ↪ constructors produce dz = 0 otherwise, and the
// orthonormal one divides by 0). pos2d does not
  ↪ check that its argument is on the plane - an
// off-plane point is silently flattened onto it, so
  ↪ test (p - o).dot(dz) == 0 first if you are
// not sure. to3d is the inverse of pos2d for the
  ↪ ORTHONORMAL frame only; for the affine frame
// carry the input INDEX through the 2D routine
  ↪ instead of mapping back (see hullCoplanar), or
// solve the 2x2 Gram system: with A = dx.dx, B =
  ↪ dx.dy, C = dy.dy, det = AC - B^2,
// a = (C*X - B*Y)/det, b = (A*Y - B*X)/det and the
  ↪ point is o + dx*a + dy*b.
// EXACTNESS: affine frame on P3<ll> is exact; pos2d
  ↪ is a dot product, so |coord| <= 1e9 keeps it
// in ll, and a downstream area2 squares that again -
  ↪ drop to |coord| <= 3e4 or use __int128
// there. Orthonormal frame is floating throughout.
template <class Q> struct Frame {
    typedef decltype(Q().dot(Q())) T;
  ↪       // ll for P3<ll>, ld for P3<ld>
    Q o, dx, dy, dz;
  ↪       // dz is the plane NORMAL
    Frame(Q p, Q q, Q r, bool ortho = true) : o(p),
  ↪ dx(q - p), dy(r - p) {
        dz = dx.cross(dy);
  ↪       // exact normal, before any scaling
        if (ortho) { dx = dx.unit(); dz = dz.unit();
  ↪ dy = dz.cross(dx); }
    }
    P<T> pos2d(Q p) const { return {(p - o).dot(dx),
  ↪ (p - o).dot(dy)}; }
    Q pos3d(Q p) const { return {(p - o).dot(dx), (p
  ↪ - o).dot(dy), (p - o).dot(dz)}; }
    Q to3d(P<T> p) const { return o + dx * p.x + dy *
  ↪ p.y; }  // ORTHONORMAL frames only
};
// CONVEX HULL OF COPLANAR 3D POINTS, returned as 3D
  ↪ points, ccw when seen from the dz side.
// Exact on P3<ll>: it uses the affine frame and
  ↪ carries the point itself through a map, so
// nothing is ever mapped back. O(n log n).
template <class Q> vector<Q> hullCoplanar(vector<Q>
  ↪ p) {
    sort(all(p)); p.erase(unique(all(p)), p.end());
    int n = sz(p), j = 2;
    if (n < 3) return p;
    while (j < n && (p[1] - p[0]).cross(p[j] - p[0])
  ↪ == Q()) j++;
    if (j == n) return {p[0], p[n - 1]};
  ↪       // all collinear: the two extremes
    Frame<Q> F(p[0], p[1], p[j], false);
    vector<decltype(F.pos2d(p[0]))> q;
    map<decltype(F.pos2d(p[0])), Q> back;
  ↪       // pos2d is injective on the plane
    for (Q x : p) q.push_back(F.pos2d(x)),
  ↪ back[q.back()] = x;
    vector<Q> res;
    for (auto& y : hull(q)) res.push_back(back[y]);
    return res;
}
// IS A 3D POINT INSIDE A COPLANAR POLYGON? Exact on
  ↪ P3<ll>. The plane test comes FIRST: without
// it a point floating above the polygon projects
  ↪ straight into it and reports true.
template <class Q> bool inCoplanar(const vector<Q>&
  ↪ v, Q p, bool strict = true) {
    int n = sz(v), j = 2;
    while (j < n && (v[1] - v[0]).cross(v[j] - v[0])
  ↪ == Q()) j++;
    if (j == n) return false;
  ↪       // degenerate polygon: no plane
    Frame<Q> F(v[0], v[1], v[j], false);
    if (sgn((p - F.o).dot(F.dz)) != 0) return false;
  ↪       // off the plane
    vector<decltype(F.pos2d(p))> q;
    for (Q x : v) q.push_back(F.pos2d(x));
    return inPoly(q, F.pos2d(p), strict);
}
/* RECIPES
 AREA OF A PLANAR 3D POLYGON   no frame needed: |sum
  ↪ p_i x p_(i+1)| / 2, the vector area
                                (faceArea in 06 -
  ↪ Polyhedron.cpp). Through the frame you get the
                                same number as
  ↪ area(pos2d(.)) for an ORTHONORMAL frame, and
                                area(pos2d(.)) / |dx x
  ↪ dy| for the affine one - use that identity
                                as your sanity check
  ↪ that the frame is built right.
 IS A SET OF 3D POINTS COPLANAR - fix the first three
  ↪ non-collinear ones, then vol6 == 0 for the
                                rest. O(n), exact.
 CUT A CONVEX POLYHEDRON BY A PLANE - clip each face,
  ↪ collect every new point on the cutting
                                plane, and close the
  ↪ hole with hullCoplanar of those points.
 SORT COPLANAR POINTS AROUND THEIR CENTROID - map
  ↪ with the affine frame and use angCmp (01 -
                                Core.cpp) - exact, and
  ↪ gives the ccw face order a mesh wants.
 POLYGON x POLYGON IN SPACE    only meaningful if
  ↪ they are coplanar (planePlane first); then map
                                both with ONE frame
  ↪ and run the 2D clip.
```

```
A 3D ROTATION MATRIX from the frame: the rows dx, dy, dz of an orthonormal frame ARE the matrix that takes the plane to z = 0, and its transpose takes it back. */
```

11.2.4 ConvexHull3D

3D CONVEX HULL, incremental. Input: n points (duplicates allowed, any order). Output: the hull

```
// Needs: Geometry3D.cpp.
// surface as TRIANGULAR faces, each an index triple {i, j, k} into the input vector, ordered so
// that (p[j]-p[i]) x (p[k]-p[i]) points OUTWARD - equivalently vol6(p[i], p[j], p[k], x) <= 0 for
// every input point x. Coplanar facets come back cut into triangles by an arbitrary
// triangulation, and points strictly inside a facet are simply absent from the output.
// Reach for it for "volume/surface of the hull", "is this point inside the hull", "diameter of a
// 3D point set", "smallest enclosing box", or any time a 2D hull idea has to survive one more
// coordinate. O(n^2) faces touched (O(n^2 log n) as written, the log being the sort of the
// horizon edges - swap in an n x n matrix to drop it; it has never mattered).
// Memory O(n) for the face list, O(n) for the horizon.
// DEGENERATE, and this is the part people get wrong: n < 4, ALL POINTS EQUAL, ALL POINTS
// COLLINEAR and ALL POINTS COPLANAR each return an EMPTY face list - a flat hull has no volume
// and no consistently outward orientation, so it cannot be expressed in this format. Detect it
// (the return is empty) and rerun the problem in 2D through 03 - CoplanarFrame.cpp, which is
// where a coplanar input belongs. Duplicate points are harmless: a repeat never strictly sees a
// face, so it is skipped and only one copy can appear in the output.
// EXACTNESS: no divisions anywhere - only vol6 signs - so on P3<ll> the hull is EXACT with
// |coord| <= 3.6e5 (the orientation test sums four vol6, 192*C^3 <= 9.2e18; make vol6 return
// __int128 to reach 1e9). The three seeding tests convert to ld to compare against eps, which is
// what makes the same code safe on P3<ld>; there eps is absolute, so scale your input to O(1).
template <class P> vector<array<int, 3>> hull3d(const vector<P>& p) {
  int n = sz(p), k;
  if (n < 4) return {};
  // no solid hull exists
  vector<array<int, 3>> F;
  vector<int> id(n);
  iota(all(id), 0);
  auto nz = [](P v) { return max({fabsl((ld)v.x), fabsl((ld)v.y), fabsl((ld)v.z)}) > eps; };
  for (k = 1; k < n && !nz(p[id[k]] - p[id[0]]); k++);
  // a second DISTINCT point
  if (k == n) return {};
  swap(id[1], id[k]);
  for (k = 2; k < n && !nz((p[id[1]] - p[id[0]]).cross(p[id[k]] - p[id[0]])); k++);
  if (k == n) return {};
  // every point on one line
  swap(id[2], id[k]);
  for (k = 3; k < n && fabsl((ld)vol6(p[id[0]], p[id[1]], p[id[2]], p[id[k]])) < eps; k++);
  if (k == n) return {};
  // every point on one plane
  swap(id[3], id[k]);
  // Orient against the centroid of the seed tetrahedron, which is strictly inside it and so
  // stays strictly inside every later hull. Summing the four vol6 IS testing that centroid,
  // scaled by 4, so the test never ties and never divides.
  auto face = [&](int a, int b, int c) {
    auto s = vol6(p[a], p[b], p[c], p[id[0]]) + vol6(p[a], p[b], p[c], p[id[1]])
           + vol6(p[a], p[b], p[c], p[id[2]]) + vol6(p[a], p[b], p[c], p[id[3]]);
    F.push_back(s > 0 ? array<int, 3>{a, c, b} : array<int, 3>{a, b, c});
  };
  for (int i = 0; i < 4; i++) face(id[(i + 1) % 4], id[(i + 2) % 4], id[(i + 3) % 4]);
  for (int t = 4; t < n; t++) {
    int u = id[t];
    vector<array<int, 3>> keep;
    vector<pii> E;
    // directed edges of the dying faces
    for (auto& f : F)
      if (vol6(p[f[0]], p[f[1]], p[f[2]], p[u]) > eps) {
        // u strictly SEES f
        for (int j = 0; j < 3; j++)
          E.push_back({f[j], f[(j + 1) % 3]});
      } else keep.push_back(f);
    if (E.empty()) continue;
    // u is inside the hull: nothing to do
    F = keep;
    sort(all(E));
    // The HORIZON is the directed edges whose reverse did not also die: those are the ones
    // whose other face survives. Cone them to u.
    for (auto& [a, b] : E) if (!binary_search(all(E), pii(b, a))) face(a, b, u);
  }
  return F;
}
/* RELATED
 VOLUME / AREA / CENTROID of the result: build the face list and call 06 - Polyhedron.cpp - vector<vector<P>> fs; for (auto& f : hull3d(p)) fs.push_back({p[f[0]], p[f[1]], p[f[2]]}); the faces are already consistently outward, so polyVol6 comes out POSITIVE.
 IS A POINT INSIDE THE HULL  vol6(p[f[0]], p[f[1]], p[f[2]], q) <= 0 for every face, O(F).
 EXTREME POINT in a direction d: max dot over the hull VERTICES (the indices that appear in F).
 DIAMETER of a 3D point set: no rotating calipers in 3D worth typing - take the hull vertices and brute force the pairs, it is O(h^2) with a tiny h.
 O(n log n) HULL exists (divide and conquer, or randomised incremental with a conflict graph). It is 150 lines of merge bookkeeping for a constant factor you do not need: n = 2000 here is 4e6 orientation tests, a few milliseconds.
 DELAUNAY TRIANGULATION of 2D points: lift each (x, y) to (x, y, x^2 + y^2) on the paraboloid, take the 3D hull, and keep the faces whose outward normal has NEGATIVE z (the LOWER hull) - those triangles, projected back, are exactly the
```

```
   ↪ Delaunay triangulation. The upper hull is
     the farthest-point Delaunay. Lifting squares the
   ↪ coordinates, so |coord| <= 1e4 to stay in
     ll here; the Voronoi diagram is the dual of what
   ↪ comes out.
 HALFSPACE INTERSECTION in 3D is the dual problem:
   ↪ map each plane n.x = d (d > 0, so that the
     origin is interior) to the point n/d, take the
   ↪ hull of those, and dualise the faces back. */
```

11.2.5 Spherical

SPHERICAL GEOMETRY: solid angles, spherical triangles and polygons, caps, the lens of two

```
// Needs: Geometry3D.cpp.
// spheres, the circumsphere of four points and the
   ↪ smallest enclosing sphere of a point set.
// Points "on the sphere" are always passed as
   ↪ vectors FROM ITS CENTRE and only their DIRECTION
   ↪ is
// read, so you may hand in unnormalised vectors, and
   ↪ a sphere centred elsewhere is handled by
// subtracting the centre first. Angles are radians,
   ↪ areas are on a sphere of radius R.
// You reach for it for radar/antenna coverage, "what
   ↪ fraction of the sky", area of a country on
// a globe, overlapping-ball volumes, and the two
   ↪ enclosing-object questions. All O(1) except
// sphPolyArea (O(n)) and minSphere (O(n) per
   ↪ iteration).
// DEGENERATE: solidAngleTet returns 0 when a, b, c
   ↪ are coplanar with the origin (a flat corner)
// and is unaffected by their lengths. sphAngle is
   ↪ undefined when a is parallel to b or c (a
// zero-length spherical segment) and returns 0
   ↪ there. sphPolyArea needs a SIMPLE polygon, ccw
// seen from outside, with no two consecutive
   ↪ vertices antipodal (the arc between them is not
// unique); it returns the area of the complement if
   ↪ you feed it cw. circumsphere needs four
// points in general position - vol6 == 0 makes it
   ↪ divide by zero. minSphere needs a non-empty
// input and returns {p[0], 0} for a single point.
// EXACTNESS: this whole file is floating point -
   ↪ every routine either normalises, calls a trig
// function, or divides. The predicates it rests on
   ↪ (vol6, dot, cross) are still exact on P3<ll>,
// so use integer input where you can and only the
   ↪ final value carries error. atan2 is used
// everywhere in place of acos: acos(dot/|a||b|)
   ↪ loses half its digits near 0 and pi, which is
// exactly the regime these formulas live in. eps is
   ↪ absolute here, so normalise your radii to
// O(1)..O(1e6).
// SOLID ANGLE of a cone of half-angle th
   ↪ (steradians): the full sphere is 4*pi.
ld solidAngleCone(ld th) { return 2 * PI * (1 -
   ↪ cosl(th)); }
// SOLID ANGLE of the tetrahedron corner at the
   ↪ ORIGIN spanned by a, b, c (VAN
   ↪ OOSTEROM-STRACKEE):
// tan(Om/2) = |a.(b x c)| / (|a||b||c| + (a.b)|c| +
   ↪ (a.c)|b| + (b.c)|a|).
// atan2 takes the correct branch by itself, so this
   ↪ is right on the whole range [0, 2*pi] with no
// sign patching, and it stays accurate for the
   ↪ needle-thin corners that Girard's formula ruins.
template <class P> ld solidAngleTet(P a, P b, P c) {
    ld A = a.dist(), B = b.dist(), C = c.dist();
    ld num = fabsl((ld)a.dot(b.cross(c)));
    ld den = A * B * C + (ld)a.dot(b) * C +
   ↪ (ld)a.dot(c) * B + (ld)b.dot(c) * A;
    return 2 * atan2l(num, den);
}
// ANGLE at a of the spherical triangle abc = the
   ↪ dihedral angle between the planes Oab and Oac,
// in [0, pi]. The ORIENTED version measures how far
   ↪ ccw you turn from arc a->b to arc a->c, in
// [0, 2*pi), which is what a signed area needs.
template <class P> ld sphAngle(P a, P b, P c) {
    P u = a.cross(b), v = a.cross(c);
    return atan2l(u.cross(v).dist(), (ld)u.dot(v));
}
template <class P> ld sphAngleOr(P a, P b, P c) {
    ld t = sphAngle(a, b, c);
    return a.cross(b).dot(c) >= 0 ? t : 2 * PI - t;
}
// SPHERICAL TRIANGLE AREA on a sphere of radius R.
   ↪ GIRARD: R^2 * (A + B + C - pi), the spherical
// EXCESS - a spherical triangle's angles overshoot
   ↪ pi by exactly its area. Computed through the
// solid angle instead, which is the same number
   ↪ without the catastrophic cancellation of summing
// three angles that are each close to pi/2 and
   ↪ subtracting pi.
template <class P> ld sphTriArea(P a, P b, P c, ld R)
   ↪ { return R * R * solidAngleTet(a, b, c); }
// SPHERICAL POLYGON AREA, vertices in ccw order seen
   ↪ from OUTSIDE: R^2 * (sum of the interior
// angles - (n-2)*pi), the same excess with n
   ↪ corners. Handles non-convex polygons.
template <class P> ld sphPolyArea(const vector<P>& p,
   ↪ ld R) {
    int n = sz(p);
    ld s = -(n - 2) * PI;
    for (int i = 0; i < n; i++) s += sphAngleOr(p[(i
   ↪ + 1) % n], p[(i + 2) % n], p[i]);
    return R * R * s;
}
// SPHERICAL CAP of height h on a sphere of radius R
   ↪ (a plane at distance R - h from the centre;
// from the polar half-angle th instead, h = R * (1 -
   ↪ cos th) and the base radius is R * sin th).
// capArea is the CURVED part only - add pi*a^2 for
   ↪ the flat disc, a^2 = h * (2R - h).
ld capArea(ld R, ld h) { return 2 * PI * R * h; }
ld capVolume(ld R, ld h) { return PI * h * h * (3 * R
   ↪ - h) / 3; }
// SPHERICAL SEGMENT / zone: the slab between two
   ↪ parallel planes, base radii a and b, height h.
ld zoneArea(ld R, ld h) { return 2 * PI * R * h; }
   ↪        // curved part; independent of WHERE
ld zoneVolume(ld a, ld b, ld h) { return PI * h * (3
   ↪ * a * a + 3 * b * b + h * h) / 6; }
// SPHERE x SPHERE, centres d apart: {surface area,
   ↪ volume} of the lens they share. The lens is
// two caps glued on the plane of the intersection
   ↪ circle. CONTAINMENT is checked FIRST - there
// the cap formula silently returns nonsense, and the
   ↪ answer is just the smaller ball.
pair<ld, ld> sphereSphere(ld R, ld r, ld d) {
    if (R < r) swap(R, r);
    if (d >= R + r) return {0, 0};
   ↪        // disjoint (tangent counts)
    if (d + r <= R) return {4 * PI * r * r, 4 * PI *
   ↪ r * r * r / 3};   // small one swallowed
    ld x = (R * R - r * r + d * d) / (2 * d);
```

```
    ↪        // plane of the circle, from centre R
     ld h1 = R - x, h2 = r - (d - x);
    ↪        // the two cap heights
     return {capArea(R, h1) + capArea(r, h2),
    ↪ capVolume(R, h1) + capVolume(r, h2)};
}
// The intersection CIRCLE itself has radius sqrt(R^2
  ↪ - x^2) and sits at x along the centre line.
// CIRCUMSPHERE of four points, {centre, radius}.
  ↪ Needs P3<ld>; the denominator IS 2*vol6, so
// coplanar input divides by zero - test vol6 first.
template <class P> pair<P, ld> circumsphere(P a, P b,
  ↪ P c, P d) {
    P B = b - a, C = c - a, D = d - a;
    ld den = 2 * (ld)B.dot(C.cross(D));
    P o = a + (C.cross(D) * (ld)B.dist2() +
  ↪ D.cross(B) * (ld)C.dist2()
              + B.cross(C) * (ld)D.dist2()) / den;
    return {o, (o - a).dist()};
}
// SMALLEST ENCLOSING SPHERE, {centre, radius}, by
  ↪ shrinking steps toward the current farthest
// point. Start at the centroid and step a fraction q
  ↪ toward the farthest point, with q decaying
// geometrically: the centre converges to the true
  ↪ one and the step budget bounds the error by
// (initial spread) * 0.999^30000 ~ 1e-13 of the
  ↪ diameter - fine for 1e-6 answers, NOT a proof.
// O(n) per iteration. Welzl with a 4-point basis is
  ↪ the exact O(n) expected version and is 40
// lines; this is 8 and has never lost a contest
  ↪ problem.
template <class P> pair<P, ld> minSphere(const
  ↪ vector<P>& p) {
    P c;
    for (P q : p) c = c + q;
    c = c / sz(p);
    for (ld q = 0.1; q > 1e-13; q *= 0.999) {
        int f = 0;
        for (int i = 1; i < sz(p); i++) if ((c -
  ↪ p[i]).dist2() > (c - p[f]).dist2()) f = i;
        c = c + (p[f] - c) * q;
    }
    ld r = 0;
    for (P x : p) r = max(r, (c - x).dist());
    return {c, r};
}
// GREAT CIRCLE, the part 01 - Geometry3D.cpp does
  ↪ not cover: its greatCircle takes lat/lon in
// DEGREES, these take 3D vectors. Distance along the
  ↪ surface between two points of the sphere,
// and the lat/lon -> vector conversion (latitude
  ↪ from the equator, longitude east, degrees).
template <class P> ld sphDist(P a, P b, ld R) {
    return R * atan2l(a.cross(b).dist(),
  ↪ (ld)a.dot(b));
}
Pd3 fromLatLon(ld R, ld lat, ld lon) {
    lat *= PI / 180, lon *= PI / 180;
    return {R * cosl(lat) * cosl(lon), R * cosl(lat)
  ↪ * sinl(lon), R * sinl(lat)};
}
/* RELATED
 TANGENT CONE from an external point at distance d:
  ↪ half-angle acos(R / d), and the cap it cuts
    off the sphere has height R * (1 - R / d). d < R
  ↪ means the point is inside: no tangent.
 SPHERE x PLANE   a circle of radius sqrt(R^2 -
  ↪ dist(centre, plane)^2), centred at the
  ↪ projection.
 SPHERE x LINE / SEGMENT   sphereLine (01 -
  ↪ Geometry3D.cpp), then clamp the parameters.
 VOLUME OF A UNION OF BALLS - no closed form beyond
  ↪ two; for three or more integrate over one axis
    numerically, or use inclusion-exclusion only
  ↪ when the statement promises few overlaps.
 SPHERICAL SEGMENT x SEGMENT - two great-circle arcs
  ↪ ab and cd cross where the plane normals a x b
    and c x d cross: the candidate directions are
  ↪ +-((a x b) x (c x d)); accept the one lying
    inside both arcs, which is the exact 3D test
  ↪ sgn((a x b).c) != sgn((a x b).d) and vice versa.
 SPHERICAL CONVEX HULL of directions = the 3D convex
  ↪ hull of the unit vectors (04), as long as
    they fit in a hemisphere.
 SOLID ANGLE OF A POLYGONAL CONE - triangulate it
  ↪ from one vertex and sum solidAngleTet - that is
    exactly what sphPolyArea does with R = 1. */
```

11.2.6 Polyhedron

POLYHEDRA GIVEN AS A FACE LIST: fs[i] is one face, a vector of its vertices in order around it,

```
// Needs: Geometry3D.cpp, Spherical.cpp (winding3
  ↪ only).
// coplanar, no repeat of the first vertex at the
  ↪ end. Faces may be any polygon, not just
// triangles. Everything here wants the list CLOSED
  ↪ (every edge shared by exactly two faces) and
// CONSISTENTLY oriented (all normals outward, or all
  ↪ inward); reorient produces the consistency
// from a face list that has it in neither direction,
  ↪ and the sign of polyVol6 tells you which of
// the two you ended up with. Computes: signed
  ↪ 6*volume, surface area, volume centroid, the
// orientation fixup, and the 3D winding number that
  ↪ answers "is the origin inside this thing".
// You reach for this file after a 3D hull, after
  ↪ cutting a solid by planes, or when the statement
// hands you a mesh (an .obj-shaped input) and asks
  ↪ for volume, mass centre, or containment.
// COMPLEXITY: volume, area, centroid O(total
  ↪ vertices). reorient O(E log E) for the edge map,
// O(E) memory. winding3 O(total vertices) with a few
  ↪ trig calls each.
// DEGENERATE: an empty face list gives volume 0,
  ↪ area 0 and centroid 0/0 - guard it. A face with
// fewer than 3 vertices contributes nothing (its fan
  ↪ is empty). A polyhedron that is NOT closed
// still returns a number from polyVol6, and that
  ↪ number is meaningless - there is no cheap check,
// so verify that every directed edge appears exactly
  ↪ once (which is what reorient's map does).
// reorient BFSes the face adjacency graph, so a
  ↪ DISCONNECTED surface (two separate shells) comes
// back with only the component of face 0 fixed; run
  ↪ it per component. Non-manifold edges (three
// faces meeting) break the map, which keeps only the
  ↪ first face per directed edge.
// EXACTNESS: polyVol6 and faceArea's vector sum are
  ↪ exact on P3<ll> (|coord| <= 5.7e5 for vol6);
// faceArea then takes one sqrt, and centroid
  ↪ divides, so those return ld / need P3<ld>.
  ↪ reorient
// compares vertices for EQUALITY, so on P3<ld> two
  ↪ faces only share an edge if the coordinates
// are bit-identical - with float input, or any input
```

```
  ↪ where the same vertex was computed twice,
// index your vertices and run reorient on
  ↪ vector<vector<int>> with the map keyed by pii
  ↪ instead.
// SIGNED 6*VOLUME, by the divergence theorem: fan
  ↪ every face from the ORIGIN and sum the
// tetrahedra. The parts outside the solid cancel
  ↪ exactly. Positive iff the faces are OUTWARD.
template <class P> auto polyVol6(const
  ↪ vector<vector<P>>& fs) {
    decltype(fs[0][0].dot(fs[0][0])) s = 0;
    for (auto& f : fs) for (int i = 1; i + 1 < sz(f);
  ↪ i++) s += vol6(P(), f[0], f[i], f[i + 1]);
    return s;
}
template <class P> ld polyVolume(const
  ↪ vector<vector<P>>& fs) {
    return fabsl((ld)polyVol6(fs)) / 6;
}
// AREA of one planar face, from its VECTOR AREA sum
  ↪ p_i x p_(i+1) (which is 2A times the unit
// normal, and is independent of where the origin
  ↪ sits because the face is closed and planar).
template <class P> ld faceArea(const vector<P>& f) {
    P s;
    for (int i = 0, m = sz(f); i < m; i++) s = s +
  ↪ f[i].cross(f[(i + 1) % m]);
    return s.dist() / 2;
}
template <class P> ld surfaceArea(const
  ↪ vector<vector<P>>& fs) {
    ld a = 0;
    for (auto& f : fs) a += faceArea(f);
    return a;
}
// CENTROID of the SOLID (not of the vertices, not of
  ↪ the surface): the volume-weighted average of
// the tetrahedron centroids of the same fan. Needs
  ↪ P3<ld>. Signs cancel exactly as in polyVol6.
template <class P> P polyCentroid(const
  ↪ vector<vector<P>>& fs) {
    P c; ld V = 0;
    for (auto& f : fs) for (int i = 1; i + 1 < sz(f);
  ↪ i++) {
        ld v = (ld)vol6(P(), f[0], f[i], f[i + 1]);
  ↪         // 6 * signed volume of O-f0-fi-fi+1
        c = c + (f[0] + f[i] + f[i + 1]) * (v / 4);
  ↪         // its centroid is (O+a+b+c)/4
        V += v;
    }
    return c / V;
}
// FACE-ORIENTATION FIXUP. Flip a subset of the faces
  ↪ so that all normals agree (either all out or
// all in - which one you get depends on face 0;
  ↪ check sgn(polyVol6) afterwards and reverse every
// face if you wanted the other). THE RULE: two faces
  ↪ sharing an edge agree exactly when they
// traverse that edge in OPPOSITE directions, so
  ↪ seeing the SAME directed edge twice means exactly
// one of the two faces must flip. BFS that parity
  ↪ over the face adjacency graph.
template <class P> vector<vector<P>>
  ↪ reorient(vector<vector<P>> fs) {
    int n = sz(fs);
    if (!n) return fs;
    vector<vector<pii>> g(n);
  ↪         // {neighbour, same direction?}
    map<pair<P, P>, int> es;
    for (int u = 0; u < n; u++)
        for (int i = 0, m = sz(fs[u]); i < m; i++) {
            P a = fs[u][i], b = fs[u][(i + 1) % m];
            if (es.count({b, a})) { int v = es[{b,
  ↪ a}]; g[u].pb({v, 0}); g[v].pb({u, 0}); }
            else if (es.count({a, b})) { int v =
  ↪ es[{a, b}]; g[u].pb({v, 1}); g[v].pb({u, 1}); }
            else es[{a, b}] = u;
        }
    vector<int> vis(n, 0), flip(n, 0), q = {0};
    vis[0] = 1;
  ↪         // face 0 defines the common direction
    for (int i = 0; i < sz(q); i++)
        for (auto& [v, same] : g[q[i]])
            if (!vis[v]) vis[v] = 1, flip[v] =
  ↪ flip[q[i]] ^ same, q.push_back(v);
    for (int u = 0; u < n; u++) if (flip[u])
  ↪ reverse(all(fs[u]));
    return fs;
}
// 3D WINDING NUMBER about the ORIGIN: how many times
  ↪ the surface wraps around it. 0 = the origin
// is OUTSIDE; +1 = inside with the faces oriented
  ↪ outward; -1 = inside with them oriented inward.
// Each face is projected onto the unit sphere, where
  ↪ it covers a signed solid angle; the total is
// 4*pi per wrap. remainder folds each face's
  ↪ contribution into (-2pi, 2pi] so that faces
  ↪ covering
// more than a hemisphere still add up right. To test
  ↪ a point other than the origin, subtract it
// from every vertex first. The origin exactly ON the
  ↪ surface makes a face's solid angle
// undefined - handle that case before calling
  ↪ (segTri / inCoplanar in 02 and 03).
template <class P> int winding3(const
  ↪ vector<vector<P>>& fs) {
    ld s = 0;
    for (auto& f : fs) s += remainderl(sphPolyArea(f,
  ↪ 1), 4 * PI);
    return (int)llroundl(s / (4 * PI));
}
/* RELATED
 IS A POINT INSIDE A CONVEX POLYHEDRON - do not call
  ↪ winding3: test sgn of the side of every face
    plane, O(F), exact on integers. Convex + outward
  ↪ faces => inside iff all sides <= 0.
 EULER    V - E + F = 2 for any closed surface of
  ↪ genus 0; a triangulated one has E = 3F/2, so
    F = 2V - 4 and E = 3V - 6. Use it as a free
  ↪ assertion on the output of a 3D hull.
 SURFACE OF A NON-PLANAR "face" - faceArea silently
  ↪ returns the area of its projection onto the
    best-fit plane. Triangulate first if the input
  ↪ can be skew.
 MOMENTS / INERTIA - the same fan works: sum the
  ↪ tetrahedron inertia tensors weighted by vol6.
 CUT BY A PLANE - clip every face (keep the vertices
  ↪ with side <= 0 plus the crossing points), then
    close the hole with hullCoplanar of the new
  ↪ points (03 - CoplanarFrame.cpp).
 MINKOWSKI SUM of two convex polyhedra: hull of the
  ↪ pairwise vertex sums, O((nm)^2) but n and m
    are tiny in practice.
 VOLUME OF A UNION of polyhedra has no cheap formula
  ↪ - sweep a plane and integrate the
    cross-section area, or inclusion-exclusion on
```

```
  ↪ convex pieces. */
```

12 | Problem Solving

12.1 Reframing

REFRAMING: CHANGE THE PROBLEM, NOT THE EFFORT

```
/* ================= REFRAMING: CHANGE THE PROBLEM,
  ↪ NOT THE EFFORT =================
 This chapter has no algorithms - it has the moves
  ↪ you make BEFORE you know which template to
 paste. Stuck for 10 minutes? Do not re-read the
  ↪ statement a fourth time. Walk the six questions,
 then walk the TRIGGERS below and try each against
  ↪ your problem.

 THE SIX QUESTIONS
   1. What object am I counting / optimising over?
  ↪ Say it in one sentence with no input format.
   2. What is INVARIANT under the allowed operation?
   3. What does the ANSWER look like (a prefix? one
  ↪ of the inputs? a threshold? always <= 2?)
   4. Can I CHECK a candidate faster than I can find
  ↪ one?          -> binary search the answer
   5. What do the constraints forbid, and what does
  ↪ the leftover budget exactly equal?
   6. What would the brute force print for n = 1..8?

 --- REVERSE THE PROCESS ---
   Run time backwards: deletions become insertions,
  ↪ splits become merges. Every structure is
   add-friendly and delete-hostile (DSU above all),
  ↪ so reversing turns impossible into trivial.
   TRIGGER: things are removed one at a time and you
  ↪ must answer after each removal.
   EXAMPLE: CF 722C Destroying Array - process
  ↪ destructions in reverse, union with restored
     neighbours.  Also CSES 1163 Traffic Lights.
   TRAP: emit answers in reverse too, and check the
  ↪ off-by-one at both ends with n = 1.

 --- WORK BACKWARDS FROM THE GOAL ---
   Define the answer at terminal states and propagate
  ↪ backwards. Standard for games and for
   "what must have been true one step earlier".
   TRIGGER: few terminal states, huge forward
  ↪ branching, small backward branching.
   TRAP: on a CYCLIC game graph this is NOT a plain
  ↪ DP - you need retrograde BFS with out-degree
     counters (a position is losing only once ALL
  ↪ successors are known winning). See 10 - Game
     Theory. Writing win[v] = OR over children on a
  ↪ cyclic graph silently mislabels draws.

 --- COMPLEMENTARY COUNTING ---
   Count the bad ones and subtract; count P(X <= x)
  ↪ instead of P(X = x).
   TRIGGER: "at least one", "there exists", "not all
  ↪ equal", "some pair".
   EXAMPLE: E[max] = sum over x of P(max >= x), with
  ↪ P(max <= x) = (x/n)^k.
   TRAP: the complement is easy for AT MOST / AT
  ↪ LEAST, not for EXACTLY. Getting from one to the
     other is binomial inversion: exactly_k =
  ↪ sum_{j>=k} (-1)^(j-k) C(j,k) atleast_j.

 --- THE CONTRIBUTION TECHNIQUE ---
   Instead of "for each subarray, compute f", ask
  ↪ "for each element, in how many objects does it
   contribute, and how much". Turns O(n^2)
  ↪ enumeration into O(n).
   TRIGGER: "sum over all subarrays / pairs / subsets
  ↪ of [min | max | gcd | #distinct | bottleneck]".
   EXAMPLE: sum over subarrays of the minimum ->
  ↪ monotonic stack gives each element's span.
     Sum over pairs of the bottleneck -> Kruskal,
  ↪ each merge contributes w * |A| * |B| pairs.
   TRAP: ties. With equal values the span must be
  ↪ strict on ONE side and non-strict on the other,
     or every duplicated minimum is counted twice.
  ↪ Decide which side BEFORE coding.

 --- SWAP THE ORDER OF SUMMATION ---
   sum_i sum_{d | i} f(d) = sum_d f(d) * floor(n/d).
  ↪ Move the outer loop to the variable whose
   inner count has a closed form.
   TRIGGER: a double sum whose inner range depends on
  ↪ the outer index; divisors, multiples,
     ancestors, intervals containing a point.
   EXAMPLE: sum of sigma(i) for i <= n = sum_d d *
  ↪ floor(n/d), then divisor blocks -> O(sqrt n).

 --- LINEARITY OF EXPECTATION ---
   E[sum X_i] = sum E[X_i], with NO independence
  ↪ needed. Decompose into indicators.
   TRIGGER: "expected number of ...", or an
  ↪ expectation over a process you cannot untangle.
   TRAP: linearity does not survive max, min, or
  ↪ division. E[max] != max E, E[1/X] != 1/E[X].
     For an expected maximum use the
  ↪ complementary-counting tail sum above.

 --- FIX THE EXTREMUM ---
   Add an artificial outer loop "for each choice of
  ↪ the maximum / the leftmost / the last move".
   Everything else becomes bounded relative to the
  ↪ fixed thing.
   TRIGGER: the statement mentions max or min over a
  ↪ range; a Cartesian-tree recursion is available.
   EXAMPLE: CF 1156E - fix the position of the
  ↪ maximum, then enumerate the SMALLER side.
   TRAP: always iterate the smaller side
  ↪ (small-to-large, O(n log n) total). Iterating
  ↪ the left
     side unconditionally is O(n^2) on a sorted
  ↪ permutation - and it passes the samples.

 --- DUALIZE ---
   max-flow = min-cut; max bipartite matching = min
  ↪ vertex cover (Konig); min path cover of a DAG
   = n - matching; longest antichain = min chain
  ↪ cover (Dilworth); "a perfect matching exists" =
   "no Hall violator".
   TRIGGER: the statement asks for a MINIMUM removal
  ↪ / blocking set / number of chains - that is
     almost always the dual of a maximum you can
  ↪ compute.
   TRAP: Konig gives the SIZE for free, but
  ↪ recovering the cover needs the alternating-BFS
     construction (L \ Z) union (R and Z). And Konig
  ↪ is BIPARTITE ONLY.

 --- MODEL IT AS A GRAPH ---
   Objects -> nodes, relations -> edges. States can
```

```
  ↪ be tuples: (cell, fuel), (vertex, parity),
  (position, mask).
  TRIGGER: "can I get from A to B", "minimum number
  ↪ of operations", "are these constraints
    consistent", "each x points at exactly one y"
  ↪ (functional graph).
  TRAP: compute |V| * |E| explicitly before coding.
  ↪ In a layered graph the answer is usually
    "any layer", not layer 0.

--- MODEL IT AS INTERVALS / A TIMELINE ---
  Recast objects as [start, end] and use a sweep,
  ↪ greedy-by-endpoint, or a difference array.
  Tree subtrees become intervals via the Euler tour.
  TRIGGER: "from time a to time b", "covers l..r",
  ↪ "is available during", subtree aggregates.
  TRAP: decide ONCE whether [1,3] and [3,5] overlap,
  ↪ and encode it in exactly one place.

--- MODEL IT AS A POLYNOMIAL ---
  "For every possible sum, count the ways" is a
  ↪ convolution. Multiplying GFs = combining
  independent choices; x^k = "used k of the budget".
  TRIGGER: modulus 998244353 (NTT is intended);
  ↪ independent parts whose sizes add.
  TRAP: convolving an array with itself
  ↪ double-counts i != j and mis-counts i = j.

--- MODEL IT AS LINEAR ALGEBRA OVER GF(2) ---
  Each element is a bit-vector; "can I make X" = "is
  ↪ X in the span"; the number of subsets giving
  a representable Y is 2^(n - rank).
  TRIGGER: XOR anywhere; "product is a perfect
  ↪ square" (exponent parity vector over primes);
    switch-toggling puzzles; "each constraint
  ↪ satisfied an even number of times".
  TRAP: 2^(n-rank) counts solutions only if the
  ↪ system is CONSISTENT - check first. And subtract
    the empty subset when the problem wants a
  ↪ non-empty one.

--- MODEL IT AS THE SUBSET LATTICE ---
  Bitmask DP plus zeta/Mobius (SOS) to aggregate
  ↪ over submasks in O(2^n n) instead of O(3^n).
  TRIGGER: n <= 20 with a "set of chosen items"
  ↪ state, or a <= 22-bit value domain with queries
    "over all y contained in x".
  TRAP: the bit loop goes OUTSIDE the mask loop.
  ↪ Swapping them gives a wrong-but-plausible array.
    Submask enumeration must be  for (s = m; ; s =
  ↪ (s-1) & m) { ...; if (!s) break; }.

--- THINK ABOUT WHAT THE ANSWER LOOKS LIKE ---
  Characterise the optimum's SHAPE first: it is a
  ↪ prefix; it is one of the inputs; it is at most
  2; it has a threshold structure. Then search only
  ↪ that shape.
  TRIGGER: the output is one small number regardless
  ↪ of n; the problem screams "constructive".
  TRAP: "the answer is at most k" needs a matching
  ↪ lower-bound example, or you print k-1.

--- CHANGE OF VARIABLES ---
  Difference array (range add -> two point updates;
  ↪ "make all equal" -> zero the differences),
  prefix sums (subarray -> pair of points),
  ↪ Chebyshev <-> Manhattan via (x+y, x-y), logs
  (products -> sums).
  TRIGGER: operations act on ranges; the objective
  ↪ is max(|dx|,|dy|) or |dx|+|dy|.
  TRAP: C++ % is negative for negative operands -
  ↪ use ((x % n) + n) % n. Seed the empty prefix.

--- PER-BIT / PER-PRIME INDEPENDENCE ---
  If the objective decomposes across bits
  ↪ (XOR/AND/OR sums, popcount weights) or across
  ↪ primes
  (gcd/lcm exponent-wise), solve 30 tiny problems
  ↪ and recombine.
  EXAMPLE: sum over pairs of (a_i XOR a_j) = sum_b
  ↪ 2^b * cnt0_b * cnt1_b.
  TRAP: bit-independence holds for SUMS, not for
  ↪ COMPARISONS. "Count pairs with a_i^a_j < k" is
    not separable - that needs a binary trie walked
  ↪ high bit first.

--- REDUCE TO SOMETHING YOU ALREADY HAVE ---
  "minimum adjacent swaps to sort" = inversions;
  ↪ "maximum pairwise-compatible set" = matching;
  "minimise the maximum edge on a path" = MST
  ↪ bottleneck / Kruskal reconstruction tree;
  ↪ "longest
  chain of pairs" = LIS; "pairwise gap constraints"
  ↪ = difference constraints + Bellman-Ford;
  "each person points at one other" = functional
  ↪ graph.
  TRIGGER: you can state the problem without the
  ↪ story. Do that, then match it.
  TRAP: the disguise is usually ALMOST the classic -
  ↪ one extra constraint breaks the reduction.
    Check the reduction on the samples by hand
  ↪ before trusting it.
============================================================
  ↪ */
```

12.2 SearchAndProof

SEARCH: TURN "FIND" INTO "CHECK"

```
/* ============ SEARCH: TURN "FIND" INTO "CHECK"
  ↪ ============

 BINARY SEARCH ON THE ANSWER. If feasible(x) is
  ↪ monotone, stop optimising and find the boundary.
  TRIGGER: "minimise the maximum", "maximise the
  ↪ minimum", "minimum time such that", "possible
    within k". Also: the answer is an integer in a
  ↪ huge range and no formula presents itself.
  TRAPS: (1) the check itself overflows - break
  ↪ early once a running count passes the target;
    (2) 'hi' must be PROVABLY feasible - seed it
  ↪ with the trivial answer, not 1e9;
    (3) if the predicate is not monotone you get a
  ↪ plausible wrong answer and no crash.

 BINARY SEARCH THE VALUE, COUNT WITH A SECOND SEARCH.
  ↪ For "k-th smallest X" over a set you cannot
  materialise (pairs, products, distances, subarray
  ↪ sums): binary search x, count how many are
  <= x. The counting routine is the real problem.
  TRAP: use firstTrue(count(x) >= k) so the returned
  ↪ x actually occurs.

 PARAMETRIC SEARCH / LAGRANGIAN (Aliens, wqs).
  ↪ Replace "exactly k" by a penalty per item and
  binary search the penalty. Requires the optimum to
  ↪ be CONVEX in k. See 08 - DP/01/07.
```

```
  TRAP: verify convexity by brute-forcing f(k) for
  ↪ small n; and the inner DP must break ties
    toward the largest count or the final
  ↪ subtraction is off by an arbitrary amount.

TERNARY SEARCH. Unimodal f: cut a third each step.
  ↪ Reals: fixed ~200 iterations (never an eps
  loop). Integers: while (hi - lo > 2), then scan
  ↪ the survivors.
  TRAP: PLATEAUS KILL IT. f(m1) == f(m2) on a flat
  ↪ region can discard the true optimum. For any
    discretely convex f use binary search on the
  ↪ difference instead:
      argmin = firstTrue(lo, hi-1, [&](ll x){ return
  ↪ f(x+1) >= f(x); });

TWO POINTERS - and why it is legal. Legal precisely
  ↪ when validity is MONOTONE IN l for fixed r:
  if [l, r] is valid then so is [l', r] for every l'
  ↪ >= l.
  TRAP: "subarray with sum <= S" is two-pointerable
  ↪ with positive values and WRONG with negative
    ones. Every time you reach for two pointers, say
  ↪ out loud "shrinking can only help". If it
    cannot, stop.

SWEEP LINE. Sort events by one coordinate, maintain
  ↪ a structure over the other.
  TRAP: event ordering at TIES is the whole problem
  ↪ - for closed intervals process opens first;
    for touching rectangles process closes first.
  ↪ Getting it wrong is off-by-exactly-the-ties.

COORDINATE COMPRESSION. Only the order matters; map
  ↪ <= 2n values to 0..m-1.
  TRAP: if the problem cares about GAPS (lengths,
  ↪ areas, "how many integers strictly between"),
    keep the original values to weight each
  ↪ compressed cell. Insert both l and r+1.

DIVIDE AND CONQUER ON TIME. Put each object's
  ↪ lifetime on a segment tree over the timeline, DFS
  it with a ROLLBACK structure, answer at the
  ↪ leaves. This is how you delete from a DSU.
  TRAP: rollback DSU must use union by size only -
  ↪ path compression makes undo impossible, so
    find is O(log n), not alpha. Budget accordingly.

MEET IN THE MIDDLE. Split in half, enumerate 2^(n/2)
  ↪ per side, recombine by sorting + binary
  search / two pointers / hash map.
  TRIGGER: n <= 40..44 with a subset-flavoured
  ↪ question.
  TRAP: memory (2^20 ll = 8 MB per array); and in
  ↪ "mod m" variants you must also consider the
    wrap-around pair. Put the extra element of an
  ↪ odd n in the CHEAPER half.

SQRT DECOMPOSITION OF THE PROBLEM (not the data
  ↪ structure). Two complementary bad algorithms,
  O(x) and O(n/x): threshold at sqrt and use each
  ↪ where it wins. Heavy vs light vertices, small
  vs large divisors, big vs small queries, "sum a_i
  ↪ <= 1e5 so at most sqrt distinct values".
  TRAP: tune the threshold to the ACTUAL costs; if
  ↪ updates are 10x cheaper the optimum is far
    from sqrt(n). Measure.

OFFLINE REORDERING. If nothing forces you online,
  ↪ sort the queries: by right endpoint (+ BIT),
  by threshold (+ DSU), by Mo's ordering, by time.
  ↪ The highest-yield contest move for "q queries
  on a static array".
  TRIGGER: no "xor with the previous answer" marker;
  ↪ all queries readable up front.
  TRAP: store answers by the ORIGINAL query id and
  ↪ print in input order.

SMALL TO LARGE. Always insert the smaller container
  ↪ into the larger: each element moves
  O(log n) times. SWAP THE HANDLES, never copy -
  ↪ copying the big one in is O(n^2) and looks
  identical.

AMORTIZATION / POTENTIAL. An occasionally-expensive
  ↪ operation is cheap on average if each
  expensive step DESTROYS something expensive to
  ↪ create. Justifies monotonic stacks, ODT, beats.
  TRAP: ODT's amortization needs assign-heavy input;
  ↪ it is O(n) per op in the worst case.
    Monotonic-stack amortization dies if you ever
  ↪ re-push an index.

ADD ONE ELEMENT AT A TIME. Solve for prefixes and
  ↪ maintain the answer incrementally. Turns "for
  every prefix, output X" from O(n^2) into O(n log
  ↪ n) and usually reveals the structure.
  TRAP: LIS - lower_bound gives strictly increasing,
  ↪ upper_bound gives non-decreasing. The tails
    array is NOT the subsequence; reconstruct with
  ↪ parent pointers.

PARALLEL BINARY SEARCH. q independent binary
  ↪ searches over one shared timeline: bucket queries
  by their current mid, sweep the timeline once,
  ↪ resolve one bit for every query.
  TRAP: reset or rebuild the shared structure
  ↪ between LEVELS. Forgetting produces monotone-
    looking garbage.

============ PROOF TOOLS: THE TEN MINUTES THAT SAVE
  ↪ THE SUBMIT ============

EXCHANGE ARGUMENT (for greedy). Take any optimum;
  ↪ show that swapping two ADJACENT elements
  toward your greedy order never makes it worse. For
  ↪ a sorting greedy this DERIVES the
  comparator: compare cost(a then b) against cost(b
  ↪ then a).
  TRAP: the derived comparator must be a STRICT WEAK
  ↪ ORDERING. Ties returning true, or an
    overflowing cross-multiplication, makes
  ↪ std::sort run off the array - an RE that looks
  ↪ like
    a logic bug. Test with all-equal input, and
  ↪ cross-multiply in __int128.

INVARIANTS. A quantity unchanged by every legal
  ↪ operation. If start and target differ on it the
  answer is "impossible" - and you have a proof
  ↪ instead of a guess.
  TRAP: an invariant proves IMPOSSIBILITY only. You
  ↪ still owe a construction for the other side.

MONOVARIANTS. A quantity that strictly decreases and
  ↪ is bounded: proves termination and bounds
```

```
   the number of operations. (Euclid: a+b strictly
  ↪ decreases.)
   TRAP: on reals a monovariant proves nothing - you
  ↪ need integrality or a floor.

PARITY AND COLOURING. Colour the board so every
  ↪ legal move changes the colour counts in a fixed
   way, then count. Try in order: checkerboard,
  ↪ columns mod k, (i+j) mod 3, a weight function.
   EXAMPLE: the mutilated chessboard has 32/30 of the
  ↪ two colours, so no domino tiling exists.
   TRAP: if no colouring produces an imbalance in
  ↪ five minutes, the obstruction is not parity.

THE EXTREMAL PRINCIPLE. Argue about the largest /
  ↪ smallest / deepest object - extremes have
   neighbours on one side only, which collapses the
  ↪ cases.
   EXAMPLE: two-BFS tree diameter is correct because
  ↪ the farthest vertex from ANY start is an
     endpoint of SOME diameter.
   TRAP: that theorem is TREE-ONLY. It is false on
  ↪ general graphs and on negative weights.

PIGEONHOLE. n+1 objects in n boxes. In practice: n+1
  ↪ prefix sums but only n residues, so some
   subarray sum is divisible by n.
   TRAP: the constructive version needs the FIRST
  ↪ repeat - store the earliest index per residue.

SYMMETRY AND WLOG. If the problem is invariant under
  ↪ swapping two roles, assume an order and
   divide by the symmetry factor at the end.
   TRAP: handle the diagonal i == j separately:
  ↪ (total - diagonal)/2 + diagonal. Under a modulus,
     "divide by 2" means multiply by inv2.

STRATEGY STEALING. In a symmetric game where an
  ↪ extra move never hurts, the first player wins or
   draws: if the second player had a winning strategy
  ↪ the first could steal it.
   TRAP: pure EXISTENCE. It gives you no winning
  ↪ move, and it fails under misere play.

ADVERSARY ARGUMENTS / QUERY LOWER BOUNDS. q binary
  ↪ queries distinguish at most 2^q inputs, so
   q >= log2(#inputs); comparison sorting needs
  ↪ log2(n!) ~ n log n.
   TRAP: assume the interactor is ADAPTIVE unless
  ↪ told otherwise - it fixes nothing in advance,
     only stays consistent. Strategies relying on
  ↪ "the answer was fixed before I started" lose.

============ CONSTRAINTS ARE A MESSAGE FROM THE
  ↪ SETTER ============
   n <= 10         permutations n!, or backtracking
  ↪ with pruning
   n <= 20         2^n bitmask DP, SOS, 3^n submask
  ↪ partition
   n <= 24..28     2^n with a small factor
   n <= 40..44     MEET IN THE MIDDLE, 2^(n/2)
   n <= 100        O(n^3)/O(n^4): Floyd, Gauss,
  ↪ interval DP, min-cost flow, Hungarian
   n <= 500        O(n^3) or O(n^2 log n); flows on
  ↪ n^2 edges
   n <= 5000       O(n^2): LCS, tree knapsack, or
  ↪ O(n^2/64) with a bitset
   n <= 1e5, log^2 slack   segtree of segtrees,
  ↪ merge-sort tree, parallel binary search, HLD
   n <= 2e5        O(n log n). This is the default and
  ↪ it means "find the n log n"
   n <= 1e6        O(n) or O(n log n) with a SMALL
  ↪ constant: no map, no set, fast IO mandatory
   n,q <= 1e5, offline     Mo's (n sqrt q) is in
  ↪ budget, ~3e7 pointer moves
   value <= 1e9, n small   coordinate compression, or
  ↪ binary search the value
   a_i <= 1e18     Miller-Rabin + Pollard rho,
  ↪ __int128, no sieve
   sum n <= 2e5 over tests per-test O(n log n); NEVER
  ↪ memset a MAXN global per test
   t <= 1e4, no sum bound  the intended solution is
  ↪ O(1) or O(log) per test
   mod 998244353   NTT / polynomial methods intended
   mod 1e9+7       counting DP / combinatorics, NOT NTT
   real output, 1e-6       geometry, ternary/binary
  ↪ search on reals, or expectation
   <= 20-30 oracle queries binary search (2^20 > 1e6)
  ↪ or bit-by-bit determination
   TL 3-5 s at n <= 1e5    the intended solution
  ↪ really is n sqrt n or n log^2 n
   ML 32-64 MB     offline/streaming, rolling DP
  ↪ arrays, no persistence

THE /64 ARGUMENT. An "impossible" O(n^2) or O(n^3)
  ↪ over BOOLEANS becomes n^2/64 - fine up to
   ~1e10 raw ops. Reachability, subset sum, LCS,
  ↪ string matching, set intersection.
   TRAP: memory. n bitsets of n bits is n^2/8 bytes:
  ↪ at n = 5e4 that is 312 MB. Process in
     blocks of ~1000 and reuse. bitset needs a
  ↪ compile-time size; a vector<u64> hand-roll is
     ~30% slower and always available.

THE "ANSWER IS SMALL" ARGUMENT. Prove the answer is
  ↪ <= a tiny constant, then brute force over
   it. Each operation halves something / clears a bit
  ↪ / merges two components; "OR of a range
   equals its SUM" means the values are bit-disjoint
  ↪ so at most 30 are nonzero; sum a_i <= 1e5
   means at most ~632 distinct values; a_i <= 1e6
  ↪ means at most 7 distinct primes, 240 divisors;
   gcd over an expanding range takes only O(log V)
  ↪ distinct values.
   TRAP: prove the bound, do not observe it on
  ↪ samples, and handle "impossible" explicitly.
============================================================
  ↪ */
```

12.3 ContestCraft

CONSTRUCTIVE, INTERACTIVE, RANDOMISED, EMPIRICAL

```
/* ============ CONSTRUCTIVE, INTERACTIVE,
  ↪ RANDOMISED, EMPIRICAL ============

BUILD GREEDILY AND PROVE AS YOU GO. Emit the answer
  ↪ one decision at a time, always taking the
   best local choice that KEEPS THE REMAINDER
  ↪ FEASIBLE - that check is the whole problem.
   TRAP: for LEXICOGRAPHICALLY SMALLEST, greedy alone
  ↪ is wrong. You need "pick the smallest
     candidate such that the rest is still
  ↪ completable", and the completability test must be
     exact. If you cannot write that test, you cannot
  ↪ do lexicographic greedy.
```

```
BUILD RECURSIVELY. Solve n from n/2 (or n-1, or four quadrants) and glue with a fixed pattern.
TRIGGER: n is a power of two, or the object is self-similar (Gray code, Hanoi, de Bruijn).
TRAP: base cases are usually n = 1 AND n = 2. Recursive output of 2^n lines needs buffered output - endl will TLE.

EXTEND FROM A KNOWN SMALL CASE. Brute force n = 1..8, find the pattern of CONSTRUCTION (not just the count), then prove the inductive step.
TRAP: the small cases are exactly where the pattern fails. Find the smallest n where it first works and special-case everything below. Nearly every constructive WA is a small-n exception.

CERTIFICATE CONSTRUCTION (prove the NO). Output the witness: the negative cycle, the odd cycle, the Hall violator, the parity mismatch. Producing it also validates your YES logic.
TRAP: after n Bellman-Ford rounds you have a vertex AFFECTED BY a negative cycle, not one ON it. Walk n parent pointers back first, then follow parents until a vertex repeats.

INTERACTIVE: BUDGET FIRST, ALGORITHM SECOND. Compare 2^Q against the number of possibilities. Q ~ log n -> binary search;  Q ~ n -> one query per element;  Q ~ n log n -> sorting/merging.
TRAPS: (1) FLUSH after every query (endl, or fflush) or you get "idleness limit exceeded", which looks nothing like the real bug; (2) count your queries locally and assert the limit; (3) assume the interactor is ADAPTIVE - it only has to stay consistent with past answers.

RANDOM SHUFFLE to destroy adversarial order: Kuhn's matching, quickselect, incremental convex hull, minimum enclosing circle (which is O(n) only after a shuffle), KD-tree splits.
TRAP: seed from the clock (mt19937_64 with steady_clock), never srand(time(0)) + rand(). unordered_map needs the splitmix64 custom hash or it is O(n^2) on a crafted test.

RANDOM SAMPLING. If a target occupies more than a 1/c fraction, k samples miss it with probability (1 - 1/c)^k.
TRIGGER: "majority element of a range", "a value shared by at least half".
TRAP: sampling gives a CANDIDATE - you must verify it exactly. And the failure probability compounds over q queries: with q = 1e5 you need per-query failure below 1e-12, so ~40 samples, not 20.

RANDOM WEIGHTS FOR UNIQUENESS (Zobrist / XOR hashing). Give each value a random 64-bit label; then multiset equality, "every value appears a multiple of k times", and "this subtree shape occurred before" become single-number comparisons.
TRAP: one random assignment has a real false-positive rate - repeat ~30 times with FRESH randoms. A fixed small set of randoms is breakable by Gaussian elimination.

HASHING AS O(1) EQUALITY. Polynomial hashing for substrings, 2D for submatrices.
TRAP: RANDOM base, and either double-mod or one 61-bit Mersenne mod. Fixed base 31 with mod 1e9+7 is anti-hashed by a Thue-Morse construction. Birthday bound: comparing m hashes pairwise collides with probability ~m^2/(2*MOD), so m = 1e6 against a single 1e9+7 mod is a coin flip.

THE REPETITION MATH. k independent trials fail with p^k. Budget PER QUERY, not per test, then multiply by 100 for safety. For primality use the DETERMINISTIC witness set {2,3,5,7,11,13,17,19,23,29,31,37} (valid below 3.3e24) - zero failure probability, same cost.

STRESS TESTING AS A THINKING TOOL. gen + brute + a diff loop; then SHRINK the counterexample until it fits on one line and stare at it.
TRAPS: (1) the generator must produce DEGENERATE cases - all equal, n = 1, all zero, maximum values, duplicate coordinates. A uniform generator over [1, 1e9] never produces a tie and will never find your tie bug. (2) If the problem says "print any", diffing is useless - write a CHECKER. (3) Stress the brute force against the samples first.

BRUTE FORCE SMALL CASES, THEN LOOK IT UP. Compute n = 1..12 exactly, then search OEIS, or feed the sequence to Berlekamp-Massey, or fit a polynomial by Lagrange interpolation.
TRAP: an OEIS match on 4 terms is noise - get 8, then confirm the 9th. BM needs terms computed under the SAME prime you will use, and at least 2x the order.

PRINT THE DP TABLE AND LOOK FOR STRUCTURE. Dump the n x n table (and the argmin table) for n = 20 and look for monotone rows, monotone opt[i][j], convex differences, sparsity. That is how you decide between Knuth, divide & conquer, and Aliens.
TRAP: monotonicity that holds at n = 20 can fail at n = 200. Also verify the quadrangle inequality numerically on random quadruples.

INSTRUMENT, DO NOT GUESS (for TLE). Print a counter of inner-loop iterations on your largest local test and compare against 1e8-1e9. That distinguishes "wrong complexity" from "wrong constant", which need completely different fixes. Compile locally with -O2.

============ CONTEST CRAFT ============

READING THE SET
  Minute 0-15: all three read DIFFERENT problems, front to back. Nobody codes. On the board, per problem: a one-line restatement, n, and a guess at the family.
  Order by (confidence x 1/effort), not by letter. Setters mis-order ~30% of the time.
  Use the scoreboard from minute 30 - it is the best difficulty signal you have.
  ONE coder at the keyboard, always. A problem is not "started" until it has a written plan with its complexity and its main data structure.
```

```
WHEN TO ABANDON - when any TWO hold:
  * 25+ minutes with no NEW idea (the idea count stopped growing, not the code).
  * The complexity is 10x over budget with no plan to close it.
  * Two teammates failed to find the bug and 5 minutes of stress testing found nothing
    => you are testing the wrong thing or misread the statement. Re-read it OUT LOUD.
  * Another unattempted problem has more solves.
  Write the state on paper before switching, and set a hard return time.

DEBUGGING UNDER TIME PRESSURE - in this order:
  1. RE-READ THE STATEMENT: output format, the modulus, 1-based vs 0-based, multi-test.
     About a quarter of "impossible" bugs are misreads.
  2. Re-read your input parsing (n and m swapped; looping to n over an array sized m).
  3. Run ALL the samples, including the one you skipped.
  4. Run n = 1, n = 2, and the all-equal case.
  5. Stress test. This beats reading code whenever you have a brute force.
  6. Only then read the code - out loud, to a teammate, for INTENT. Silent re-reading just
     re-hallucinates the same thing.
  7. Nothing after 15 minutes: REWRITE the suspicious function from scratch.

PRE-SUBMIT CHECKLIST (40 seconds, every submit)
  [ ] Overflow: every product of two inputs; inside comparators; inside prefix sums.
  [ ] Modulus: right constant, +MOD after subtraction, inverse instead of division, non-negative.
  [ ] n = 1, m = 1, k = 0, single-element array.
  [ ] Degenerate input: empty string, zero edges, one-node tree, all equal, all zero, negatives.
  [ ] STATE RESET between test cases: every global, vector, counter, visited array, the answer
      variable, and any function-local static. Clear only what you touched if sum n is bounded.
  [ ] Right bound read: MAXN vs the actual limit; the sum over queries can exceed the per-test n.
  [ ] Uninitialised: dp arrays that need -INF not 0; DSU parent[i] = i.
  [ ] Recursion depth: DFS on a path with n = 2e5 is 2e5 frames - move big locals out.
  [ ] Output format: newline vs space, YES vs Yes, -1 vs IMPOSSIBLE, setprecision.
  [ ] endl removed from any loop running more than ~1e4 times; debug output removed.
  [ ] The 'cin >> tc' line matches whether the problem is multi-test.

VERDICT TRIAGE
  WA  -> which test? Test 1 means a misread or a format error; a late test means an edge case or
         overflow. Then: modulus, overflow, ties/duplicates (strict vs non-strict), stress test,
         unreset state, floating-point comparison.
  TLE -> is the complexity really what you think (count the loops on paper)? endl in a loop /
         missing sync_with_stdio. map/set in a hot loop. Reallocation and pass-by-value. memset
         of a huge global per test. Infinite loop: a binary search with l = mid.
         Only then change algorithm. A /64 win never fixes a wrong complexity class.
  RE  -> array bounds first: rebuild with -fsanitize=address,undefined -D_GLIBCXX_DEBUG and run
         the samples; it finds it in seconds. Then stack overflow, division by zero, a comparator
         that is not a strict weak ordering (crashes INSIDE std::sort), .back()/.top() on an
         empty container, 0- vs 1-indexed access.
  MLE -> n x n arrays (5000^2 ints = 100 MB); vector<vector<int>> for a graph (5-8x the raw
         data - use CSR); persistent structures (count the nodes before allocating); recursion
         frames; go offline and stream instead of storing versions.
  Idleness on an interactive -> a missing flush, or debug output on stdout.

TEAM PROTOCOL
  Never submit without the checklist: a rejection costs 20 penalty minutes plus morale.
  One person owns the printed notebook and fetches pages; the coder does not browse.
  Paste, adapt, then read once - the #1 template bug is an index-convention mismatch.
  Two solutions disagree? Trust the brute force. Always.
  Last 30 minutes: stop starting new problems; finish the nearest one and re-check the submits.

THE TEN MOST EXPENSIVE RECURRING BUGS (by contest-time cost)
  1. Overflow in an intermediate product, invisible because the final answer is small.
  2. State not reset between test cases - passes locally, fails on test 2.
  3. Wrong strictness (< vs <=) in a comparator, a binary-search predicate, or a tie-break.
  4. Two pointers on a non-monotone predicate: plausible output, silently wrong.
  5. Binary search bounds: hi not provably feasible, or l = mid causing an infinite loop.
  6. 0- vs 1-based off-by-one when pasting a template.
  7. endl in an output loop, or a missing sync_with_stdio, at n = 1e6.
  8. Negative modulo in C++ (-3 % 5 == -3).
  9. A greedy that was never proved and passes the samples.
 10. Recursion depth on a path-shaped tree with n = 2e5.
============================================================
*/
```

Index of every routine (alphabetical)

Q

R

S

T

U

V

W